

collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Collector
4 # =====
5 # Description:
6 #   Движок сбора системных метрик, аппаратной структуры и показателей
7 #   производительности.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.collector import SystemCollector
11 #
12 #       service = SystemCollector()
13 #
14 # File: collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 13:10:00
20 # =====
21
22 from __future__ import annotations
23 """Движок сбора системных метрик, аппаратной структуры и показателей
24   производительности."""
25
26 import asyncio
27 import ctypes
28 import getpass
29 import locale
30 import os
```

```
30 import platform
31 import socket
32 import time
33 from datetime import datetime, timezone
34 from typing import Any, Dict, List, Optional
35 try:
36     import psutil
37     PSUTIL_AVAILABLE = True
38 except ImportError:
39     psutil = None
40     PSUTIL_AVAILABLE = False
41 from logger import logger
42 # Updated: 2026-10-04 01:50:00
43 from .models import (
44     AnomalyItem,
45     BatteryMetrics,
46     CloudStorageInfo,
47     CpuMetrics,
48     DiskIoMetrics,
49     DiskPartitionMetrics,
50     DriverInfo,
51     GpuMetrics,
52     HardwareArchiveEntry,
53     HardwareAuditReport,
54     HardwareChangeItem,
55     HardwareDeviceAudit,
56     HardwareNode,
57     HardwareSensor,
58     MemoryMetrics,
59     MonitorInfo,
60     NetworkInterfaceMetrics,
61     NetworkPortMetrics,
62     OfficeSuiteInfo,
63     PhysicalDiskHealth,
64     ProcessLifecycleEvent,
65     ProcessMetrics,
66     ProcessNetworkActivity,
67     ProcessProvenanceInfo,
```

```

68     ProcessProvenanceReport,
69     RamStickInfo,
70     SystemCoreMetrics,
71     SystemHardwareQuick,
72     SystemHealthAlerts,
73     SystemSnapshot,
74     WindowsUpdateInfo,
75 )
76 from .sensors import get_hardware_sensors
77 from apps.windows.telemetry_research.hardware_auditor import HardwareAuditor
78 from apps.windows.telemetry_research.hardware_history_manager
79 import HardwareHistoryManager
80 from .process_token_collector import ProcessTokenCollector
81 from .sqlite import TelemetryStorage
82
83 class SystemCollector:
84     """Telemetry collector for system load, hardware devices,
85     and processes."""
86
87     def __init__(self, auditor: Optional[HardwareAuditor]=None,
88                 history_manager: Optional[HardwareHistoryManager]=None,
89                 storage: Optional[TelemetryStorage]=None) -> None:
90         """Initialize telemetry collector with timing, I/O baseline,
91         hardware auditor, and storage."""
92         self._last_disk_io = psutil.disk_io_counters() if PSUTIL_AVAILABLE else None
93         self._last_net_io = psutil.net_io_counters(pernic=True) if PSUTIL_AVAILABLE else None
94         self._last_proc_net_io: Dict[int, Tuple[float, float, float]] = {}
95         self._last_time = time.time()
96         self._cpu_model_cached: Optional[str] = None
97         self._identity_cached: Optional[Dict[str, Any]] = None
98         self._monitors_cached: Optional[List[MonitorInfo]] = None
99         self._updates_cached: Optional[WindowsUpdateInfo] = None

```

```

95         self._office_cached: Optional[OfficeSuiteInfo] = None
96         self._ram_sticks_cached: Optional[List[RamStickInfo]]
    = None
97         self._ram_sticks_cache_time: float = 0.0
98         self._physical_disks_cached: Optional[List[PhysicalDis
kHealth]] = None
99         self._physical_disks_cache_time: float = 0.0
100         self.storage = storage or TelemetryStorage.get_instanc
e()
101         self.auditor = auditor or HardwareAuditor(storage=self
.storage)
102         self.history_manager = history_manager or HardwareHist
oryManager()
103
104     def get_system_identity(self) -> Dict[str, Any]:
105         """Collect host identity, current user, system languag
e, and locale parameters.
106
107         Returns:
108             Dict[str, Any]: Detailed system identity and local
ization dictionary.
109         """
110         if self._identity_cached:
111             return self._identity_cached
112         hostname = socket.gethostname()
113         domain = os.environ.get('USERDOMAIN', '')
114         username_raw = os.environ.get('USERNAME') or getpass.g
etuser()
115         full_username = f'{domain}\\{username_raw}' if domain
and domain != hostname else username_raw
116         user_locale = 'ru-RU'
117         system_locale = 'ru-RU'
118         codepage = 'UTF-8'
119         input_languages: List[str] = []
120         if os.name == 'nt':
121             try:
122                 buf = ctypes.create_unicode_buffer(100)
123                 if ctypes.windll.kernel32.GetUserDefaultLocale

```

```

Name(buf, 100):
124         user_locale = buf.value or 'ru-RU'
125         if ctypes.windll.kernel32.GetSystemDefaultLoca
leName(buf, 100):
126             system_locale = buf.value or 'ru-RU'
127             acp = ctypes.windll.kernel32.GetACP()
128             oemcp = ctypes.windll.kernel32.GetOEMCP()
129             codepage = f'ACP: {acp} | OEM: {oemcp} (UTF-8)
'
130             count = ctypes.windll.user32.GetKeyboardLayout
List(0, None)
131             if count > 0:
132                 hkls = (ctypes.c_void_p * count)()
133                 ctypes.windll.user32.GetKeyboardLayoutList
(count, hkls)
134                 lang_map = {1049: 'Русский (RU)', 1033: 'E
nglish (US)', 1037: 'עברית (IL)', 1058: 'Українська (UA)', 103
1: 'Deutsch (DE)', 1036: 'Français (FR)'}
135                 for h in hkls:
136                     lang_id = (h.value if hasattr(h, 'valu
e') and h.value else int(h)) & 65535
137                     lang_name = lang_map.get(lang_id, f'La
yout 0x{lang_id:04X}')
138                     if lang_name not in input_languages:
139                         input_languages.append(lang_name)
140             except Exception as ex:
141                 logger.debug(f'Failed to query native Windows
locale: {ex}')
142             if not input_languages:
143                 input_languages = ['Русский (RU)', 'English (US)']
144             try:
145                 tz_offset = datetime.now().astimezone().strftime('%z')
146                 tz_name = datetime.now().astimezone().tzname() or
time.tzname[0]
147                 timezone_str = f'{tz_name} (UTC{tz_offset[:3]}:{tz
_offset[3:]})'
148             except Exception:

```

```

149 |         timezone_str = 'UTC+03:00'
150 | def _format_lang_display(locale_tag: str) -> str:
151 |     if not locale_tag:
152 |         return ''
153 |     tag_clean = locale_tag.strip()
154 |     tag_lower = tag_clean.lower()
155 |     if 'ru' in tag_lower:
156 |         return f'Русский (Россия) [{tag_clean}]'
157 |     elif 'he' in tag_lower:
158 |         return f'עברית (ישראל) [{tag_clean}]'
159 |     elif 'en' in tag_lower:
160 |         return f'English (United States) [{tag_clean}]'
161 |     elif 'uk' in tag_lower:
162 |         return f'Українська (Україна) [{tag_clean}]'
163 |     elif 'de' in tag_lower:
164 |         return f'Deutsch (Deutschland) [{tag_clean}]'
165 |     elif 'fr' in tag_lower:
166 |         return f'Français (France) [{tag_clean}]'
167 |     elif 'es' in tag_lower:
168 |         return f'Español (España) [{tag_clean}]'
169 |     elif 'zh' in tag_lower:
170 |         return f'中文 (China) [{tag_clean}]'
171 |     elif 'ja' in tag_lower:
172 |         return f'日本語 (Japan) [{tag_clean}]'
173 |     return tag_clean
174 |
175 | sys_lang_display = _format_lang_display(user_locale)
176 | os_build = platform.version() or '10.0.26200'
177 | os_install_date = ''
178 | os_install_language = ''
179 | if os.name == 'nt':
180 |     try:
181 |         import winreg
182 |         # Попытка получить оригинальный язык установки
183 |         # из реестра (InstallLanguage)
184 |         for access in (winreg.KEY_READ | winreg.KEY_WO
185 | W64_64KEY, winreg.KEY_READ):

```

```

184         try:
185             with winreg.OpenKey(winreg.HKEY_LOCAL_
MACHINE, r'SYSTEM\CurrentControlSet\Control\Nls\Language', 0,
access) as key:
186                 val, _ = winreg.QueryValueEx(key,
'InstallLanguage')
187                 if val:
188                     lcid = int(str(val), 16)
189                     buf_lcid = ctypes.create_unico
de_buffer(100)
190                     if ctypes.windll.kernel32.LCID
ToLocaleName(lcid, buf_lcid, 100, 0):
191                         tag = buf_lcid.value
192                         if tag:
193                             os_install_language =
_format_lang_display(tag)
194                             break
195             except Exception:
196                 pass
197             if not os_install_language:
198                 sys_ui_lang = ctypes.windll.kernel32.GetSy
stemDefaultUILanguage()
199                 buf_lcid = ctypes.create_unicode_buffer(10
0)
200                 if ctypes.windll.kernel32.LCIDToLocaleName
(sys_ui_lang, buf_lcid, 100, 0):
201                     tag = buf_lcid.value
202                     if tag:
203                         os_install_language = _format_lang
_display(tag)
204             except Exception as ex:
205                 logger.debug(f'Failed to query Windows Install
Language: {ex}')
206
207         try:
208             import winreg
209             for access in (winreg.KEY_READ | winreg.KEY_WO
W64_64KEY, winreg.KEY_READ):

```

```

210         try:
211             with winreg.OpenKey(winreg.HKEY_LOCAL_
MACHINE, r'SOFTWARE\Microsoft\Windows NT\CurrentVersion', 0, a
ccess) as key:
212                 try:
213                     val, _ = winreg.QueryValueEx(k
ey, 'InstallDate')
214                     if val and int(val) > 0:
215                         os_install_date = datetime
.fromtimestamp(int(val)).strftime('%d.%m.%Y %H:%M')
216                         break
217                 except Exception:
218                     pass
219                 try:
220                     val, _ = winreg.QueryValueEx(k
ey, 'InstallTime')
221                     if val and int(val) > 0:
222                         epoch_sec = (int(val) / 10
000000.0) - 11644473600.0
223                         if epoch_sec > 0:
224                             os_install_date = date
time.fromtimestamp(epoch_sec).strftime('%d.%m.%Y %H:%M')
225                             break
226                 except Exception:
227                     pass
228             except Exception:
229                 continue
230         except Exception as ex:
231             logger.debug(f'Failed to query Windows Install
Date from registry: {ex}')
232         self._identity_cached = {
233             'hostname': hostname,
234             'username': full_username,
235             'os_build': os_build,
236             'os_install_date': os_install_date,
237             'system_language': sys_lang_display,
238             'os_install_language': os_install_language,
239             'user_locale': user_locale,

```



```

240         'system_locale': system_locale,
241         'timezone': timezone_str,
242         'codepage': codepage,
243         'input_languages': input_languages
244     }
245     return self._identity_cached
246
247     async def _resolve_cpu_model(self) -> str:
248         """Resolve CPU brand/model name from platform or WMI.
249
250         Returns:
251             str: Resolved CPU model name.
252         """
253         if self._cpu_model_cached:
254             return self._cpu_model_cached
255         model = platform.processor() or ''
256         if os.name == 'nt' and (not model or 'Intel64' in model
1 or 'AMD64' in model):
257             try:
258                 from src.utils.com_worker import com_worker
259
260                 def _get_cpu():
261                     import pythoncom
262                     pythoncom.CoInitialize()
263                     import wmi
264                     w = wmi.WMI()
265                     cpus = w.Win32_Processor()
266                     return str(cpus[0].Name).strip() if cpus e
lse None
267                 res = await com_worker.run(_get_cpu)
268                 if res:
269                     model = res
270             except Exception:
271                 pass
272         if not model:
273             model = f'{platform.machine()} {os.cpu_count() or
1}-Core Processor'
274         self._cpu_model_cached = model

```

```

275         return model
276
277     async def get_cpu_metrics(self) -> CpuMetrics:
278         """Collect real-time CPU utilization and architecture
279         details.
280
281         Returns:
282             CpuMetrics: Current CPU performance metrics.
283         """
284         logical_cores = os.cpu_count() or 1
285         physical_cores = max(1, logical_cores // 2) if logical
286         _cores > 1 else 1
287         if PSUTIL_AVAILABLE:
288             per_core = psutil.cpu_percent(interval=None, percp
289             u=True)
290             total = round(sum(per_core) / max(len(per_core), 1
291             ), 1) if per_core else 0.0
292             physical = psutil.cpu_count(logical=False) or phys
293             ical_cores
294             logical = psutil.cpu_count(logical=True) or logica
295             l_cores
296             freq_mhz = 0.0
297             try:
298                 freq = psutil.cpu_freq()
299                 if freq:
300                     freq_mhz = round(freq.current, 1)
301             except Exception:
302                 pass
303             return CpuMetrics(model=await self._resolve_cpu_mo
304             del(), architecture=platform.machine(), physical_cores=physica
305             l, logical_cores=logical, total_percent=total, per_core_percen
306             t=per_core, frequency_mhz=freq_mhz)
307         return CpuMetrics(model=await self._resolve_cpu_model(
308         ), architecture=platform.machine(), physical_cores=physical_co
309         res, logical_cores=logical_cores, total_percent=10.0, per_core
310         _percent=[10.0] * logical_cores, frequency_mhz=2400.0)
311
312     def get_memory_metrics(self) -> MemoryMetrics:

```

```

301|         """Collect RAM and swap memory consumption.
302|
303|         Returns:
304|             MemoryMetrics: Current memory usage statistics.
305|         """
306|         if PSUTIL_AVAILABLE:
307|             vm = psutil.virtual_memory()
308|             sm = psutil.swap_memory()
309|             return MemoryMetrics(total_gb=round(vm.total / 102
4 ** 3, 2), available_gb=round(vm.available / 1024 ** 3, 2), u
sed_gb=round((vm.total - vm.available) / 1024 ** 3, 2), percen
t=round(vm.percent, 1), swap_total_gb=round(sm.total / 1024 **
3, 2), swap_used_gb=round(sm.used / 1024 ** 3, 2), swap_perce
nt=round(sm.percent, 1))
310|         total_ram_gb = 16.0
311|         used_ram_gb = 8.0
312|         if os.name == 'nt':
313|             try:
314|
315|                 class MEMORYSTATUSEX(ctypes.Structure):
316|                     _fields_ = [('dwLength', ctypes.c_ulong),
('dwMemoryLoad', ctypes.c_ulong), ('ullTotalPhys', ctypes.c_ul
onglong), ('ullAvailPhys', ctypes.c_ulonglong), ('ullTotalPage
File', ctypes.c_ulonglong), ('ullAvailPageFile', ctypes.c_ulon
glong), ('ullTotalVirtual', ctypes.c_ulonglong), ('ullAvailVir
tual', ctypes.c_ulonglong), ('sullAvailExtendedVirtual', ctype
s.c_ulonglong)]
317|                     stat = MEMORYSTATUSEX()
318|                     stat.dwLength = ctypes.sizeof(MEMORYSTATUSEX)
319|                     if ctypes.windll.kernel32.GlobalMemoryStatusEx
(ctypes.byref(stat)):
320|                         total_ram_gb = round(stat.ullTotalPhys / 1
024 ** 3, 2)
321|                         avail_ram_gb = round(stat.ullAvailPhys / 1
024 ** 3, 2)
322|                         used_ram_gb = round(total_ram_gb - avail_r
am_gb, 2)
323|                         percent = round(stat.dwMemoryLoad, 1)

```

```

324         return MemoryMetrics(total_gb=total_ram_gb
    , available_gb=avail_ram_gb, used_gb=used_ram_gb, percent=perc
    ent, swap_total_gb=round(stat.ullTotalPageFile / 1024 ** 3, 2)
    , swap_used_gb=round((stat.ullTotalPageFile - stat.ullAvailPag
    eFile) / 1024 ** 3, 2), swap_percent=round((1.0 - stat.ullAvai
    lPageFile / max(stat.ullTotalPageFile, 1)) * 100, 1))
325     except Exception:
326         pass
327     return MemoryMetrics(total_gb=total_ram_gb, available_
    gb=total_ram_gb - used_ram_gb, used_gb=used_ram_gb, percent=50
    .0, swap_total_gb=16.0, swap_used_gb=2.0, swap_percent=12.5)
328
329     def get_gpu_metrics(self) -> List[GpuMetrics]:
330         """Collect GPU accelerator load, VRAM, and capabilitie
    s.
331
332         Returns:
333             List[GpuMetrics]: Detected GPU devices with teleme
    try.
334         """
335         gpus: List[GpuMetrics] = []
336         try:
337             from apps.windows.modules.hardware.gpu_prober impo
    rt GpuProber
338             prober = GpuProber()
339             gpu_list = prober.probe_all()
340             for g in gpu_list:
341                 gpus.append(GpuMetrics(
342                     name=g.name,
343                     memory_total_gb=round((g.memory_total_mb o
    r 0.0) / 1024.0, 2),
344                     memory_used_gb=round((g.memory_used_mb or
    0.0) / 1024.0, 2),
345                     load_percent=g.utilization_gpu_pct,
346                     temperature_celsius=g.temperature_gpu_c,
347                     has_cuda=g.vendor.lower() == 'nvidia',
348                     has_directml=True
349                 ))

```

```

350         except Exception as ex:
351             logger.debug(f'Ошибка сбора метрик GPU через GpuPr
ober: {ex}')
352         if not gpus:
353             gpus.append(GpuMetrics(name='Integrated Display Co
ntroller', memory_total_gb=0.0, has_cuda=False, has_directml=F
alse))
354         return gpus
355
356     def get_disk_metrics(self) -> tuple[List[DiskPartitionMetr
ics], DiskIoMetrics]:
357         """Collect disk partitions capacity and read/write rat
es.
358
359         Returns:
360             tuple[List[DiskPartitionMetrics], DiskIoMetrics]:
Partitions and I/O rates.
361         """
362         partitions: List[DiskPartitionMetrics] = []
363         io_metrics = DiskIoMetrics()
364         if PSUTIL_AVAILABLE:
365             for part in psutil.disk_partitions(all=False):
366                 try:
367                     usage = psutil.disk_usage(part.mountpoint)
368                     vol_name = ''
369                     is_virtual = False
370                     drive_type = 'Fixed'
371                     if os.name == 'nt':
372                         try:
373                             import ctypes
374                             vol_buf = ctypes.create_unicode_bu
ffer(1024)
375                             fs_buf = ctypes.create_unicode_buf
fer(1024)
376                             mount_root = part.mountpoint if pa
rt.mountpoint.endswith('\\') else f'{part.mountpoint}\\'
377                             if ctypes.windll.kernel32.GetVolum
eInformationW(mount_root, vol_buf, 1024, None, None, None, fs_

```

```

    buf, 1024):
378         vol_name = vol_buf.value
379         dos_buf = ctypes.create_unicode_bu
ffer(1024)
380         clean_dev = part.device.rstrip('\\
')
381         ctypes.windll.kernel32.QueryDosDev
iceW(clean_dev, dos_buf, 1024)
382         dos_dev = dos_buf.value
383         if 'google' in vol_name.lower() or
'@' in vol_name or 'onedrive' in vol_name.lower() or ('dropbo
x' in vol_name.lower()) or ('volume{' in dos_dev.lower() and '
harddiskvolume' not in dos_dev.lower()):
384             is_virtual = True
385             drive_type = 'Google Drive' if
'google' in vol_name.lower() or '@' in vol_name else 'Cloud /
Virtual'
386         except Exception:
387             pass
388         partitions.append(DiskPartitionMetrics(dev
ice=part.device, mountpoint=part.mountpoint, fstype=part.fstyp
e, total_gb=round(usage.total / 1024 ** 3, 2), used_gb=round(u
sage.used / 1024 ** 3, 2), free_gb=round(usage.free / 1024 **
3, 2), percent=round(usage.percent, 1), volume_name=vol_name o
r None, is_virtual=is_virtual, drive_type=drive_type))
389         except (PermissionError, OSError):
390             continue
391         now = time.time()
392         elapsed = max(now - self._last_time, 0.1)
393         current_io = psutil.disk_io_counters()
394         if current_io and self._last_disk_io:
395             io_metrics.read_bytes_per_sec = max(0.0, (curr
ent_io.read_bytes - self._last_disk_io.read_bytes) / elapsed)
396             io_metrics.write_bytes_per_sec = max(0.0, (cur
rent_io.write_bytes - self._last_disk_io.write_bytes) / elapse
d)
397             io_metrics.read_count_per_sec = max(0.0, (curr
ent_io.read_count - self._last_disk_io.read_count) / elapsed)

```

```

398         io_metrics.write_count_per_sec = max(0.0, (current_io.write_count - self._last_disk_io.write_count) / elapsed)
399         self._last_disk_io = current_io
400     else:
401         drive = 'C:\\' if os.name == 'nt' else '/'
402         try:
403             import shutil
404             total, used, free = shutil.disk_usage(drive)
405             partitions.append(DiskPartitionMetrics(device='C:' if os.name == 'nt' else '/', mountpoint=drive, fstype='NTFS' if os.name == 'nt' else 'ext4', total_gb=round(total / 1024 ** 3, 2), used_gb=round(used / 1024 ** 3, 2), free_gb=round(free / 1024 ** 3, 2), percent=round(used / max(total, 1) * 100, 1)))
406         except Exception:
407             pass
408     return (partitions, io_metrics)
409
410     def get_network_metrics(self) -> List[NetworkInterfaceMetrics]:
411         """Collect network interface statuses and throughput rates.
412
413         Returns:
414             List[NetworkInterfaceMetrics]: Network adapters metrics.
415
416         """
417         interfaces: List[NetworkInterfaceMetrics] = []
418         adapter_stats_map = {}
419         try:
420             from apps.windows.network.network_usage import WindowsNetworkUsageCollector
421             usage_collector = WindowsNetworkUsageCollector()
422             stats_list = usage_collector.get_adapter_statistics()
423             adapter_stats_map = {s.name.lower(): s for s in stats_list}

```

```

423         except Exception as ex:
424             logger.debug(f'Failed to fetch adapter stats in SystemCollector: {ex}')
425
426         if PSUTIL_AVAILABLE:
427             addrs = psutil.net_if_addrs()
428             stats = psutil.net_if_stats()
429             current_net_io = psutil.net_io_counters(pernic=True
430 e)
431             now = time.time()
432             elapsed = max(now - self._last_time, 0.1)
433             for name, addr_list in addrs.items():
434                 stat = stats.get(name)
435                 is_up = stat.isup if stat else True
436                 speed = stat.speed if stat else 0
437                 ips = [a.address for a in addr_list if a.family == socket.AF_INET]
438                 sent_rate = 0.0
439                 recv_rate = 0.0
440                 if current_net_io and name in current_net_io and self._last_net_io and (name in self._last_net_io):
441                     cur = current_net_io[name]
442                     prev = self._last_net_io[name]
443                     sent_rate = max(0.0, (cur.bytes_sent - prev.bytes_sent) / elapsed)
444                     recv_rate = max(0.0, (cur.bytes_recv - prev.bytes_recv) / elapsed)
445                     st_info = adapter_stats_map.get(name.lower())
446                     interfaces.append(NetworkInterfaceMetrics(name=name, is_up=is_up, speed_mbps=speed, bytes_sent_per_sec=sent_rate, bytes_recv_per_sec=recv_rate, ip_addresses=ips, statistics=st_info))
447             self._last_net_io = current_net_io
448         else:
449             try:
450                 host_ip = socket.gethostbyname(socket.gethostname())

```



```
451         host_ip = '127.0.0.1'
452         interfaces.append(NetworkInterfaceMetrics(name='Primary Network Adapter', is_up=True, speed_mbps=1000, bytes_sent_per_sec=0.0, bytes_recv_per_sec=0.0, ip_addresses=[host_ip]))
453     return interfaces
454
455     def get_network_usage_report(self, period_minutes: int = 1440):
456         """Получить глубокий отчет сетевого потребления за период для AI диагностики.
457
458         Args:
459             period_minutes: Интервал отчета в минутах.
460
461         Returns:
462             NetworkUsagePeriodReport: Полная сводка сетевой статистики за период.
463         """
464         from apps.windows.network.network_usage import WindowsNetworkUsageCollector
465         collector = WindowsNetworkUsageCollector()
466         return collector.get_traffic_period_summary(period_minutes=period_minutes)
467
468     def get_disk_usage_report(self, period_minutes: int = 1440, force_refresh_smart: bool = False):
469         """Получить глубокий отчет использования дискового I/O и SMART за период для AI диагностики.
470
471         Args:
472             period_minutes: Интервал отчета в минутах (по умолчанию 1440 = 24 часа).
473             force_refresh_smart: Принудительно опросить физические диски.
474
475         Returns:
476             DiskUsagePeriodReport: Полная сводка дискового I/O
```

```

и SMART-показателей за период.
477     """
478     from apps.windows.telemetry.storage_usage import WindowsStorageUsageCollector
479     collector = WindowsStorageUsageCollector()
480     return collector.get_disk_usage_period_report(period_minutes=period_minutes, force_refresh_smart=force_refresh_smart)
481
482
483
484     @staticmethod
485     def _classify_launch_reason(
486         parent_name: Optional[str],
487         process_name: Optional[str] = None,
488         session_id: Optional[int] = None,
489         cmdline: Optional[str] = None,
490     ) -> str:
491         """Классификация причины/контекста запуска процесса по его родителю и окружению.
492
493         Args:
494             parent_name: Имя родительского процесса.
495             process_name: Имя текущего процесса.
496             session_id: Номер сессии Windows (0 = системные службы, 1+ = интерактивная сессия).
497             cmdline: Командная строка процесса (опционально).
498
499         Returns:
500             str: Категория запуска ('service', 'scheduled_task', 'shell_interactive', 'terminal_cli', 'ide_developer', 'system_boot', 'spawned_by_parent', 'unknown').
501         """
502         p_name = (parent_name or '').lower()
503         if p_name in ('services.exe',) or (p_name in ('svchost.exe',) and session_id == 0):
504             return 'service'
505         if p_name in ('taskeng.exe', 'taskhostw.exe', 'taskhos

```

```

t.exe') or (cmdline and '-task' in cmdline.lower())):
506         return 'scheduled_task'
507     if p_name in ('explorer.exe',):
508         return 'shell_interactive'
509     if p_name in ('cmd.exe', 'powershell.exe', 'pwsh.exe',
'wt.exe', 'windowsterminal.exe', 'bash.exe', 'zsh.exe', 'conh
ost.exe'):
510         return 'terminal_cli'
511     if p_name in ('code.exe', 'devenv.exe', 'pycharm64.exe
', 'idea64.exe', 'rider64.exe', 'cursor.exe', 'windsurf.exe'):
512         return 'ide_developer'
513     if p_name in ('winlogon.exe', 'userinit.exe', 'smss.ex
e', 'csrss.exe', 'wininit.exe', 'lsass.exe'):
514         return 'system_boot'
515     if p_name:
516         clean_parent = p_name.replace('.exe', '')
517         return f'spawned_by_{clean_parent}'
518     return 'unknown'
519
520     def get_top_processes(self, limit: int = 25, sort_by: str
= 'cpu') -> List[ProcessMetrics]:
521         """Retrieve active processes sorted by resource consum
ption, with token and provenance info.
522
523         Args:
524             limit: Maximum processes to return (0 for all).
525             sort_by: Attribute to sort by ('cpu', 'memory'/'ra
m', 'handles'/'descriptors').
526         Returns:
527             List[ProcessMetrics]: Ranked process metrics, incl
uding provenance, integrity_level and elevation.
528         """
529         # Collect token information for all processes
530         token_collector = ProcessTokenCollector()
531         token_info_map = {info.pid: info for info in token_col
lector.collect()}
532
533         procs: List[ProcessMetrics] = []

```

```

534         if PSUTIL_AVAILABLE:
535             handle_attr = 'num_handles' if os.name == 'nt' else 'num_fds'
536             proc_dict: Dict[int, Dict[str, Any]] = {}
537             for p in psutil.process_iter(attrs=['pid', 'ppid', 'name', 'status', 'cpu_percent', 'memory_info', 'memory_percent', 'num_threads', 'username', 'create_time', 'exe', 'cmdline', handle_attr]):
538                 try:
539                     proc_dict[p.info['pid']] = p.info
540                 except (psutil.NoSuchProcess, psutil.AccessDenied):
541                     continue
542
543             for pid, info in proc_dict.items():
544                 try:
545                     mem_info = info.get('memory_info')
546                     rss_mb = round(mem_info.rss / (1024 * 1024), 1) if mem_info else 0.0
547                     cpu_p = round(info.get('cpu_percent') or 0.0, 1)
548                     num_h = info.get(handle_attr) or 0
549                     token = token_info_map.get(pid)
550                     integrity = token.integrity_level if token
551                     else None
552                     elevation = token.elevation if token else None
553                     sid = token.sid if token else None
554                     session_id = token.session_id if token else None
555                     ppid = info.get('ppid')
556                     parent_info = proc_dict.get(ppid) if ppid
557                     else None
558                     parent_name = parent_info.get('name') if parent_info else None
559
560             # Построение цепочки предков

```

```

560         ancestors = [info.get('name') or str(pid)]
561         curr_pid = ppid
562         visited = {pid}
563         while curr_pid and curr_pid in proc_dict a
nd curr_pid not in visited:
564             visited.add(curr_pid)
565             curr_p = proc_dict[curr_pid]
566             ancestors.append(curr_p.get('name') or
f'PID:{curr_pid}')
567             curr_pid = curr_p.get('ppid')
568             ancestors.reverse()
569             ancestor_chain = ' → '.join(ancestors)
570
571             create_time_val = info.get('create_time')
or 0.0
572             creation_time_str = datetime.fromtimestamp
(create_time_val, tz=timezone.utc).isoformat() if create_time_
val > 0 else None
573             process_guid = f'proc_{pid}_{int(create_ti
me_val)}'
574
575             raw_cmdline = info.get('cmdline')
576             cmdline_str = ' '.join(raw_cmdline) if isi
nstance(raw_cmdline, list) else (str(raw_cmdline) if raw_cmdli
ne else None)
577             launch_reason = self._classify_launch_reas
on(parent_name, info.get('name'), session_id, cmdline_str)
578
579             procs.append(ProcessMetrics(
580                 pid=pid,
581                 name=info.get('name') or 'unknown',
582                 status=info.get('status') or 'running'
,
583                 cpu_percent=cpu_p,
584                 memory_mb=rss_mb,
585                 memory_percent=round(info.get('memory_
percent') or 0.0, 1),
586                 num_threads=info.get('num_threads') or

```

```

1,
587         num_handles=num_h,
588         username=info.get('username'),
589         integrity_level=integrity,
590         elevation=elevation,
591         ppid=ppid,
592         parent_name=parent_name,
593         executable_path=info.get('exe'),
594         sid=sid,
595         session_id=session_id,
596         creation_time=creation_time_str,
597         process_guid=process_guid,
598         ancestor_chain=ancestor_chain,
599         launch_reason=launch_reason,
600     ))
601     except (psutil.NoSuchProcess, psutil.AccessDen
ied):
602         continue
603     else:
604         # Fallback for non-psutil environments
605         now_iso = datetime.now(timezone.utc).isoformat()
606         p = ProcessMetrics(
607             pid=os.getpid(),
608             name='python.exe',
609             status='running',
610             cpu_percent=1.5,
611             memory_mb=120.0,
612             memory_percent=1.0,
613             num_threads=8,
614             num_handles=42,
615             username=os.getenv('USERNAME', 'SYSTEM'),
616             ppid=os.getppid() if hasattr(os, 'getppid') el
se None,
617             parent_name='cmd.exe',
618             executable_path=os.path.abspath(__file__),
619             sid='S-1-5-21-mock-sid',
620             session_id=1,
621             creation_time=now_iso,

```

```

622         process_guid=f'proc_{os.getpid()}_{int(time.time())}',
623         ancestor_chain='cmd.exe → python.exe',
624         launch_reason='terminal_cli',
625     )
626     token = token_info_map.get(p.pid)
627     if token:
628         p.integrity_level = token.integrity_level
629         p.elevation = token.elevation
630         p.sid = token.sid
631         p.session_id = token.session_id
632     procs.append(p)
633
634     # Sorting according to requested key
635     sort_key = (sort_by or 'cpu').lower()
636     if sort_key in ('handles', 'descriptors'):
637         procs.sort(key=lambda x: x.num_handles, reverse=True)
638     elif sort_key in ('memory', 'ram'):
639         procs.sort(key=lambda x: x.memory_mb, reverse=True)
640     else:
641         procs.sort(key=lambda x: x.cpu_percent, reverse=True)
642
643     if limit is None or limit <= 0:
644         return procs
645     return procs[:limit]
646
647     def get_battery_metrics(self) -> BatteryMetrics:
648         """Collect laptop or UPS battery charge level and AC power status.
649
650         Returns:
651             BatteryMetrics: Current battery and power status.
652         """
653         if PSUTIL_AVAILABLE and hasattr(psutil, 'sensors_battery'):

```

```

654         try:
655             bat = psutil.sensors_battery()
656             if bat is not None:
657                 return BatteryMetrics(has_battery=True, pe
rcent=round(bat.percent, 1), power_plugged=bat.power_plugged,
secs_left=bat.secsleft if bat.secsleft > 0 else None, power_pr
ofile='AC / Battery')
658         except Exception as ex:
659             logger.debug(f'Battery probe failed: {ex}')
660             return BatteryMetrics(has_battery=False, percent=None,
power_plugged=True, secs_left=None, power_profile='AC Mains /
Desktop')
661
662     def get_physical_disks_health(self, force: bool = False) -
> List[PhysicalDiskHealth]:
663         """Собрать данные о физических дисках, SMART, типах но
сителей и здоровье.
664
665         В сценариях API (tc.ps1) данные читаются строго из SQL
ite базы данных telemetry.db
666         без вызова тяжелого опроса WMI/CIM.
667         Прямой опрос оборудования через WindowsStorageSensor в
ыполняется только при force=True (в демоне tlm.ps1).
668
669         Args:
670             force: Принудительное обновление данных накопителе
й через WindowsStorageSensor (только для tlm.ps1).
671
672         Returns:
673             List[PhysicalDiskHealth]: Список обнаруженных физи
ческих накопителей.
674         """
675         now = time.time()
676         if not force and self._physical_disks_cached is not No
ne and now - self._physical_disks_cache_time < 43200.0:
677             return self._physical_disks_cached
678
679         # 1. Попытка прочитать физические диски из базы данных

```



```

        SQLite
680         if not force and self.storage:
681             try:
682                 storage_data = self.storage.get_latest_storage
_data()
683                 if storage_data and storage_data.get("physical
_disks"):
684                     disks: List[PhysicalDiskHealth] = []
685                     for d in storage_data["physical_disks"]:
686                         disks.append(PhysicalDiskHealth(
687                             device_id=str(d.get("device_id")) o
r f"Disk {len(disks)}"),
688                             model=str(d.get("model") or d.get(
"friendly_name") or d.get("name") or "Physical Drive"),
689                             media_type=str(d.get("media_type")
or "SSD"),
690                             size_gb=round(float(d.get("size_gb
") or 0.0), 1),
691                             health_status=str(d.get("health_st
atus") or "Healthy"),
692                             operational_status=str(d.get("oper
ational_status") or "OK"),
693                             temperature_celsius=float(d.get("t
emperature_celsius") or d.get("temperature_c") or 0.0) if (d.g
et("temperature_celsius") is not None or d.get("temperature_c"
) is not None) else None,
694                             interface_type=str(d.get("interfac
e_type") or d.get("bus_type") or "NVMe"),
695                             lifetime_read_bytes=d.get("lifetim
e_read_bytes"),
696                             lifetime_write_bytes=d.get("lifeti
me_write_bytes"),
697                             lifetime_read_tb=d.get("lifetime_r
ead_tb"),
698                             lifetime_write_tb=d.get("lifetime_
write_tb"),
699                             power_on_hours=d.get("power_on_hou
rs"),

```

```

700 |             wear_percentage=d.get("wear_perce
| tage"),
701 |         ))
702 |         if disks:
703 |             self._physical_disks_cached = disks
704 |             self._physical_disks_cache_time = now
705 |             return disks
706 |         except Exception as ex:
707 |             logger.debug(f"Не удалось прочитать physical_d
| isks из БД: {ex}")
708 |
709 |         # 2. Прямой опрос через WindowsStorageSensor выполняет
| ся ТОЛЬКО при явном force=True (в демоне tlm.ps1)
710 |         disks: List[PhysicalDiskHealth] = []
711 |         if os.name == 'nt' and force:
712 |             try:
713 |                 from apps.windows.modules.storage_manager.core
| .windows_storage_sensor import WindowsStorageSensor
714 |                 sensor = WindowsStorageSensor(ttl_sec=43200.0)
715 |                 phys_disks = sensor.get_physical_disks(force_r
| efresh=True)
716 |                 for d in phys_disks:
717 |                     disks.append(PhysicalDiskHealth(
718 |                         device_id=d.device_id,
719 |                         model=d.model or d.friendly_name or 'P
| hysical Drive',
720 |                         media_type=d.media_type or 'SSD',
721 |                         size_gb=round(d.size_gb, 1) if d.size_
| gb else 0.0,
722 |                         health_status=d.health_status or 'Heal
| thy',
723 |                         operational_status=d.operational_statu
| s or 'OK',
724 |                         temperature_celsius=d.temperature_c,
725 |                         interface_type=d.bus_type or 'NVMe',
726 |                         lifetime_read_bytes=d.lifetime_read_by
| tes,
727 |                         lifetime_write_bytes=d.lifetime_write_

```

```

bytes,
728         lifetime_read_tb=d.lifetime_read_tb,
729         lifetime_write_tb=d.lifetime_write_tb,
730         power_on_hours=d.power_on_hours,
731         wear_percentage=d.wear_percentage,
732     ))
733     except Exception as ex:
734         logger.debug(f'Ошибка сбора физических дисков
735         через WindowsStorageSensor: {ex}')
736
737     if not disks:
738         disks.append(PhysicalDiskHealth(
739             device_id='Disk 0',
740             model='System Drive (NVMe/SSD)',
741             media_type='SSD',
742             size_gb=512.0,
743             health_status='Healthy',
744             operational_status='OK',
745             interface_type='NVMe',
746         ))
747     self._physical_disks_cached = disks
748     self._physical_disks_cache_time = now
749     return disks
750
751     async def get_ram_sticks(self) -> List[RamStickInfo]:
752         """Collect physical memory stick (SPD) details via WMI
753         .
754
755         Returns:
756             List[RamStickInfo]: Installed physical RAM modules
757         .
758
759         """
760         now = time.time()
761         if self._ram_sticks_cached is not None and now - self._ram_sticks_cache_time < 60.0:
762             return self._ram_sticks_cached
763         sticks: List[RamStickInfo] = []
764         if os.name == 'nt':

```

```

761         from src.utils.com_worker import com_worker
762
763         async def _probe():
764
765             def _do_probe():
766                 try:
767                     import wmi
768                 except ImportError:
769                     return []
770                 _sticks: List[RamStickInfo] = []
771                 try:
772                     w = wmi.WMI()
773                     mems = w.Win32_PhysicalMemory()
774                     type_map = {20: 'DDR', 21: 'DDR2', 24:
'DDR3', 26: 'DDR4', 34: 'DDR5'}
775                     for m in mems:
776                         cap_gb = round(int(m.Capacity or 0
) / 1024 ** 3, 1)
777                         speed = int(m.Speed or m.Configure
dClockSpeed or 3200)
778                         m_type = type_map.get(m.SMBIOSMemo
ryType, 'DDR4/DDR5')
779                         _sticks.append(RamStickInfo(bank_l
abel=str(m.BankLabel or m.DeviceLocator or 'DIMM').strip(), ca
pacity_gb=cap_gb, speed_mhz=speed, manufacturer=str(m.Manufact
urer or 'Generic').strip(), part_number=str(m.PartNumber or ''
).strip(), memory_type=m_type))
780                 except Exception as err:
781                     logger.debug(f'WMI Win32_PhysicalMemor
y query failed: {err}')
782                 return _sticks
783                 return await com_worker.run(_do_probe)
784             try:
785                 sticks = await _probe()
786             except Exception as ex:
787                 logger.debug(f'Failed to query physical memory
sticks: {ex}')
788             self._ram_sticks_cached = sticks

```

```

789         self._ram_sticks_cache_time = now
790         return sticks
791
792     def get_listening_ports(self, limit: int=15) -> List[Netwo
rkPortMetrics]:
793         """Collect active listening TCP/UDP sockets with proce
ss attribution.
794
795         Args:
796             limit: Maximum listening ports to return.
797
798         Returns:
799             List[NetworkPortMetrics]: Active listening ports.
800         """
801         ports: List[NetworkPortMetrics] = []
802         if PSUTIL_AVAILABLE:
803             try:
804                 conns = psutil.net_connections(kind='inet')
805                 seen = set()
806                 for c in conns:
807                     if c.status == 'LISTEN' or c.type == socke
t.SOCK_DGRAM:
808                         p_num = c.laddr.port
809                         if p_num in seen:
810                             continue
811                         seen.add(p_num)
812                         p_name = None
813                         if c.pid:
814                             try:
815                                 p_name = psutil.Process(c.pid)
816                                     .name()
817                             except Exception:
818                                 pass
819                         ports.append(NetworkPortMetrics(port=p
_num, protocol='TCP' if c.type == socket.SOCK_STREAM else 'UDP
', address=c.laddr.ip or '0.0.0.0', pid=c.pid, process_name=p_
name))
819                     if len(ports) >= limit:

```

```

820 |                 break
821 |             except Exception as ex:
822 |                 logger.debug(f'Failed to query listening ports
      | : {ex}')
```

```

823 |         return ports
824 |
825 |     def get_process_network_activity(self, limit: int=50, only
      | _internet: bool=False) -> List[ProcessNetworkActivity]:
826 |         """Collect active network connections and traffic attr
      | ibution for processes.
827 |
828 |         Args:
829 |             limit: Maximum connections to return.
830 |             only_internet: Whether to restrict only to externa
      | l internet addresses.
831 |
832 |         Returns:
833 |             List[ProcessNetworkActivity]: Active network conne
      | ctions with process and traffic description.
834 |         """
835 |         activities: List[ProcessNetworkActivity] = []
836 |         if not PSUTIL_AVAILABLE:
837 |             return activities
838 |         try:
839 |             raw_conns = psutil.net_connections(kind='inet')
840 |         except Exception as ex:
841 |             logger.debug(f'Failed to query network connections
      | : {ex}')
```

```

842 |         return activities
843 |         proc_cache: Dict[int, Dict[str, Any]] = {}
844 |         now_ts = time.time()
845 |
846 |     def _get_proc_info(pid: Optional[int]) -> Dict[str, An
      | y]:
847 |         if not pid or pid == 0:
848 |             return {'name': 'System Idle', 'user': 'SYSTEM
      | ', 'read_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb': 0.0, 'del
      | ta_write_kb': 0.0, 'read_speed_kbs': 0.0, 'write_speed_kbs': 0
```

```

.0}
849         if pid == 4:
850             return {'name': 'System', 'user': 'NT AUTHORITY\SYSTEM', 'read_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb': 0.0, 'delta_write_kb': 0.0, 'read_speed_kbs': 0.0, 'write_speed_kbs': 0.0}
851         if pid in proc_cache:
852             return proc_cache[pid]
853         try:
854             p = psutil.Process(pid)
855             pname = p.name()
856             puser = ''
857             try:
858                 puser = p.username()
859             except Exception:
860                 puser = ''
861             read_kb = 0.0
862             write_kb = 0.0
863             delta_read_kb = 0.0
864             delta_write_kb = 0.0
865             read_speed_kbs = 0.0
866             write_speed_kbs = 0.0
867             try:
868                 io = p.io_counters()
869                 read_kb = round(io.read_bytes / 1024, 1)
870                 write_kb = round(io.write_bytes / 1024, 1)
871                 if hasattr(self, '_last_proc_net_io') and pid in self._last_proc_net_io:
872                     prev_t, prev_r, prev_w = self._last_proc_net_io[pid]
873                     dt = max(now_ts - prev_t, 0.1)
874                     if io.read_bytes >= prev_r:
875                         delta_read_kb = round((io.read_bytes - prev_r) / 1024, 1)
876                         read_speed_kbs = round(delta_read_kb / dt, 1)
877                     if io.write_bytes >= prev_w:
878                         delta_write_kb = round((io.write_bytes - prev_w) / 1024, 1)
879                         write_speed_kbs = round(delta_write_kb / dt, 1)
880             except Exception:
881                 pass
882         except Exception:
883             return {}
884     def _get_proc_net_io(self, pid):
885         """Get network I/O statistics for a process.
886         Returns a dictionary with keys: read_bytes, write_bytes, read_packets, write_packets.
887         """
888         try:
889             p = psutil.Process(pid)
890             io = p.io_counters()
891             return {'read_bytes': io.read_bytes, 'write_bytes': io.write_bytes, 'read_packets': io.read_packets, 'write_packets': io.write_packets}
892         except Exception:
893             return {}
894     def _get_proc_cpu_times(self, pid):
895         """Get CPU times for a process.
896         Returns a dictionary with keys: user, system, idle, nice, other.
897         """
898         try:
899             p = psutil.Process(pid)
900             times = p.cpu_times()
901             return {'user': times.user, 'system': times.system, 'idle': times.idle, 'nice': times.nice, 'other': times.other}
902         except Exception:
903             return {}
904     def _get_proc_mem_info(self, pid):
905         """Get memory information for a process.
906         Returns a dictionary with keys: mem_used, mem_private, mem_shared, mem_swap.
907         """
908         try:
909             p = psutil.Process(pid)
910             mem = p.memory_info()
911             return {'mem_used': mem.memory, 'mem_private': mem.private, 'mem_shared': mem.shared, 'mem_swap': mem.swap}
912         except Exception:
913             return {}
914     def _get_proc_status(self, pid):
915         """Get process status information.
916         Returns a dictionary with keys: name, user, status, priority, nice, ppid, pgrp, session, cwd, exe, fdopen, fdopen2, fdopen3, fdopen4, fdopen5, fdopen6, fdopen7, fdopen8, fdopen9, fdopen10, fdopen11, fdopen12, fdopen13, fdopen14, fdopen15, fdopen16, fdopen17, fdopen18, fdopen19, fdopen20, fdopen21, fdopen22, fdopen23, fdopen24, fdopen25, fdopen26, fdopen27, fdopen28, fdopen29, fdopen30, fdopen31, fdopen32, fdopen33, fdopen34, fdopen35, fdopen36, fdopen37, fdopen38, fdopen39, fdopen40, fdopen41, fdopen42, fdopen43, fdopen44, fdopen45, fdopen46, fdopen47, fdopen48, fdopen49, fdopen50, fdopen51, fdopen52, fdopen53, fdopen54, fdopen55, fdopen56, fdopen57, fdopen58, fdopen59, fdopen60, fdopen61, fdopen62, fdopen63, fdopen64, fdopen65, fdopen66, fdopen67, fdopen68, fdopen69, fdopen70, fdopen71, fdopen72, fdopen73, fdopen74, fdopen75, fdopen76, fdopen77, fdopen78, fdopen79, fdopen80, fdopen81, fdopen82, fdopen83, fdopen84, fdopen85, fdopen86, fdopen87, fdopen88, fdopen89, fdopen90, fdopen91, fdopen92, fdopen93, fdopen94, fdopen95, fdopen96, fdopen97, fdopen98, fdopen99, fdopen100, fdopen101, fdopen102, fdopen103, fdopen104, fdopen105, fdopen106, fdopen107, fdopen108, fdopen109, fdopen110, fdopen111, fdopen112, fdopen113, fdopen114, fdopen115, fdopen116, fdopen117, fdopen118, fdopen119, fdopen120, fdopen121, fdopen122, fdopen123, fdopen124, fdopen125, fdopen126, fdopen127, fdopen128, fdopen129, fdopen130, fdopen131, fdopen132, fdopen133, fdopen134, fdopen135, fdopen136, fdopen137, fdopen138, fdopen139, fdopen140, fdopen141, fdopen142, fdopen143, fdopen144, fdopen145, fdopen146, fdopen147, fdopen148, fdopen149, fdopen150, fdopen151, fdopen152, fdopen153, fdopen154, fdopen155, fdopen156, fdopen157, fdopen158, fdopen159, fdopen160, fdopen161, fdopen162, fdopen163, fdopen164, fdopen165, fdopen166, fdopen167, fdopen168, fdopen169, fdopen170, fdopen171, fdopen172, fdopen173, fdopen174, fdopen175, fdopen176, fdopen177, fdopen178, fdopen179, fdopen180, fdopen181, fdopen182, fdopen183, fdopen184, fdopen185, fdopen186, fdopen187, fdopen188, fdopen189, fdopen190, fdopen191, fdopen192, fdopen193, fdopen194, fdopen195, fdopen196, fdopen197, fdopen198, fdopen199, fdopen200, fdopen201, fdopen202, fdopen203, fdopen204, fdopen205, fdopen206, fdopen207, fdopen208, fdopen209, fdopen210, fdopen211, fdopen212, fdopen213, fdopen214, fdopen215, fdopen216, fdopen217, fdopen218, fdopen219, fdopen220, fdopen221, fdopen222, fdopen223, fdopen224, fdopen225, fdopen226, fdopen227, fdopen228, fdopen229, fdopen230, fdopen231, fdopen232, fdopen233, fdopen234, fdopen235, fdopen236, fdopen237, fdopen238, fdopen239, fdopen240, fdopen241, fdopen242, fdopen243, fdopen244, fdopen245, fdopen246, fdopen247, fdopen248, fdopen249, fdopen250, fdopen251, fdopen252, fdopen253, fdopen254, fdopen255, fdopen256, fdopen257, fdopen258, fdopen259, fdopen260, fdopen261, fdopen262, fdopen263, fdopen264, fdopen265, fdopen266, fdopen267, fdopen268, fdopen269, fdopen270, fdopen271, fdopen272, fdopen273, fdopen274, fdopen275, fdopen276, fdopen277, fdopen278, fdopen279, fdopen280, fdopen281, fdopen282, fdopen283, fdopen284, fdopen285, fdopen286, fdopen287, fdopen288, fdopen289, fdopen290, fdopen291, fdopen292, fdopen293, fdopen294, fdopen295, fdopen296, fdopen297, fdopen298, fdopen299, fdopen300, fdopen301, fdopen302, fdopen303, fdopen304, fdopen305, fdopen306, fdopen307, fdopen308, fdopen309, fdopen310, fdopen311, fdopen312, fdopen313, fdopen314, fdopen315, fdopen316, fdopen317, fdopen318, fdopen319, fdopen320, fdopen321, fdopen322, fdopen323, fdopen324, fdopen325, fdopen326, fdopen327, fdopen328, fdopen329, fdopen330, fdopen331, fdopen332, fdopen333, fdopen334, fdopen335, fdopen336, fdopen337, fdopen338, fdopen339, fdopen340, fdopen341, fdopen342, fdopen343, fdopen344, fdopen345, fdopen346, fdopen347, fdopen348, fdopen349, fdopen350, fdopen351, fdopen352, fdopen353, fdopen354, fdopen355, fdopen356, fdopen357, fdopen358, fdopen359, fdopen360, fdopen361, fdopen362, fdopen363, fdopen364, fdopen365, fdopen366, fdopen367, fdopen368, fdopen369, fdopen370, fdopen371, fdopen372, fdopen373, fdopen374, fdopen375, fdopen376, fdopen377, fdopen378, fdopen379, fdopen380, fdopen381, fdopen382, fdopen383, fdopen384, fdopen385, fdopen386, fdopen387, fdopen388, fdopen389, fdopen390, fdopen391, fdopen392, fdopen393, fdopen394, fdopen395, fdopen396, fdopen397, fdopen398, fdopen399, fdopen400, fdopen401, fdopen402, fdopen403, fdopen404, fdopen405, fdopen406, fdopen407, fdopen408, fdopen409, fdopen410, fdopen411, fdopen412, fdopen413, fdopen414, fdopen415, fdopen416, fdopen417, fdopen418, fdopen419, fdopen420, fdopen421, fdopen422, fdopen423, fdopen424, fdopen425, fdopen426, fdopen427, fdopen428, fdopen429, fdopen430, fdopen431, fdopen432, fdopen433, fdopen434, fdopen435, fdopen436, fdopen437, fdopen438, fdopen439, fdopen440, fdopen441, fdopen442, fdopen443, fdopen444, fdopen445, fdopen446, fdopen447, fdopen448, fdopen449, fdopen450, fdopen451, fdopen452, fdopen453, fdopen454, fdopen455, fdopen456, fdopen457, fdopen458, fdopen459, fdopen460, fdopen461, fdopen462, fdopen463, fdopen464, fdopen465, fdopen466, fdopen467, fdopen468, fdopen469, fdopen470, fdopen471, fdopen472, fdopen473, fdopen474, fdopen475, fdopen476, fdopen477, fdopen478, fdopen479, fdopen480, fdopen481, fdopen482, fdopen483, fdopen484, fdopen485, fdopen486, fdopen487, fdopen488, fdopen489, fdopen490, fdopen491, fdopen492, fdopen493, fdopen494, fdopen495, fdopen496, fdopen497, fdopen498, fdopen499, fdopen500, fdopen501, fdopen502, fdopen503, fdopen504, fdopen505, fdopen506, fdopen507, fdopen508, fdopen509, fdopen510, fdopen511, fdopen512, fdopen513, fdopen514, fdopen515, fdopen516, fdopen517, fdopen518, fdopen519, fdopen520, fdopen521, fdopen522, fdopen523, fdopen524, fdopen525, fdopen526, fdopen527, fdopen528, fdopen529
```

```

ytes - prev_w) / 1024, 1)
879 |         write_speed_kbs = round(delta_writ
    | e_kb / dt, 1)
880 |         if hasattr(self, '_last_proc_net_io'):
881 |             self._last_proc_net_io[pid] = (now_ts,
    | io.read_bytes, io.write_bytes)
882 |         except Exception:
883 |             pass
884 |         info = {'name': pname, 'user': puser, 'read_kb
    | ': read_kb, 'write_kb': write_kb, 'delta_read_kb': delta_read_
    | kb, 'delta_write_kb': delta_write_kb, 'read_speed_kbs': read_s
    | peed_kbs, 'write_speed_kbs': write_speed_kbs}
885 |         proc_cache[pid] = info
886 |         return info
887 |     except Exception:
888 |         info = {'name': f'PID {pid}', 'user': '', 'rea
    | d_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb': 0.0, 'delta_writ
    | e_kb': 0.0, 'read_speed_kbs': 0.0, 'write_speed_kbs': 0.0}
889 |         proc_cache[pid] = info
890 |         return info
891 |
892 |     def _is_ext_internet(ip: str) -> bool:
893 |         if not ip:
894 |             return False
895 |         if ip in ('127.0.0.1', '::1', '0.0.0.0', ':::') or
    | ip.startswith('127.') or ip.startswith('fe80:'):
896 |             return False
897 |             return True
898 |         for c in raw_conns:
899 |             laddr_str = f'{c.laddr.ip}:{c.laddr.port}' if c.la
    | ddr else '-'
900 |             raddr_str = f'{c.raddr.ip}:{c.raddr.port}' if c.ra
    | ddr else '-'
901 |             rip = c.raddr.ip if c.raddr else ''
902 |             rport = c.raddr.port if c.raddr else 0
903 |             lport = c.laddr.port if c.laddr else 0
904 |             is_ext = _is_ext_internet(rip)
905 |             if only_internet and (not is_ext):

```



```

906         continue
907         proto = 'TCP' if c.type == socket.SOCK_STREAM else
          'UDP'
908         status = c.status or ('LISTEN' if not c.raddr else
          'ESTABLISHED')
909         p_info = _get_proc_info(c.pid)
910         p_name = p_info['name']
911         p_user = p_info['user']
912         service_type, sent_desc, recv_desc = self._classif
          y_traffic(p_name, rport or lport, proto, status, is_ext)
913         activities.append(ProcessNetworkActivity(pid=c.pid
          or 0, name=p_name, user=p_user, local_address=laddr_str, remo
          te_address=raddr_str, remote_host=None, protocol=proto, status
          =status, is_internet=is_ext, service_type=service_type, sent_k
          b=p_info.get('write_kb', 0.0), recv_kb=p_info.get('read_kb', 0
          .0), delta_sent_kb=p_info.get('delta_write_kb', 0.0), delta_re
          cv_kb=p_info.get('delta_read_kb', 0.0), sent_rate_kbs=p_info.g
          et('write_speed_kbs', 0.0), recv_rate_kbs=p_info.get('read_spe
          ed_kbs', 0.0), sent_summary=sent_desc, recv_summary=recv_desc)
          )
914         if len(activities) >= limit:
915             break
916         return activities
917
918     @staticmethod
919     def _classify_traffic(proc_name: str, port: int, proto: st
          r, status: str, is_internet: bool) -> tuple[str, str, str]:
920         """Classify network traffic service type and generate
          human-readable summaries."""
921         p_lower = proc_name.lower()
922         if status == 'LISTEN' or not is_internet:
923             if 'python' in p_lower or 'uvicorn' in p_lower or
          'node' in p_lower:
924                 return ('Локальный Web API / Сервер', 'HTTP от
          веты локальным клиентам / WebSockets', 'Входящие HTTP запросы
          от фронтенда / браузера')
925                 return ('Локальная служба / Socket', 'Служебные от
          веты локальной подсистемы', 'Ожидание входящих подключений (Li

```

```
sten)')
926         if port in (443, 8443):
927             if any((b in p_lower for b in ('edge', 'chrome', '
firefox', 'brave', 'opera', 'browser'))):
928                 return ('HTTPS / Веб-навигация', 'Исходящие HT
TPS запросы, URL заголовки, формы, cookies', 'HTML страницы, с
крипты, видео, изображения, медиа')
929             if any((m in p_lower for m in ('telegram', 'discor
d', 'slack', 'whatsapp', 'skype'))):
930                 return ('Мессенджер / Зашифрованный чат', 'Тек
стовые сообщения, медиа-файлы, голосовые пакеты, пинги', 'Вход
ящие сообщения, аудио/видеопоток, обновления чатов')
931             if any((s in p_lower for s in ('onedrive', 'google
drivefs', 'dropbox', 'cloud'))):
932                 return ('Облачная синхронизация / Хранилище',
'Выгрузка локальных файлов, метаданные изменений', 'Синхрониза
ция файлов из облака, снимки папок')
933             if any((d in p_lower for d in ('code', 'devenv', '
language_server', 'python', 'pycharm', 'git'))):
934                 return ('Разработка / LLM API & Git', 'AI пром
пты к LLM моделям, API вызовы, git push, пакеты', 'Сгенерирова
нный AI код, ответы API, пакеты библиотек')
935             if any((w in p_lower for w in ('svchost', 'system'
, 'msmpeng', 'searchindexer', 'spoolsv'))):
936                 return ('Системные службы Windows / Облако MS'
, 'Телеметрия Defender, проверка сертификатов, запросы ОС', 'С
игнатуры безопасности, метаданные обновлений Windows')
937                 return ('HTTPS / Зашифрованный веб-трафик', 'Зашиф
рованные HTTPS/TLS запросы и полезная нагрузка', 'Ответы удале
нного веб-сервера, зашифрованные данные')
938             if port in (80, 8080):
939                 return ('HTTP / Открытый веб-трафик', 'HTTP GET/PO
ST запросы, клиентские заголовки', 'HTML веб-страницы, незашиф
рованный контент, ответы')
940             if port == 53:
941                 return ('DNS / Разрешение имен', 'Запросы на получ
ение IP-адресов доменов (DNS Queries)', 'DNS-ответы с IP-адрес
ами серверов (DNS Answers)')
```

```

942         if port in (5222, 5223, 5228):
943             return ('Push-нотификации / XMPP', 'Heartbeat пинг
и, исходящие push-запросы', 'Входящие фоновые push-уведомления
и события')
944         if port == 22:
945             return ('SSH / Защищенный терминал', 'Команды конс
оли, зашифрованные SSH сессии', 'Вывод удаленного терминала, ф
айловые потоки SFTP')
946         if port == 7680:
947             return ('WUDD / P2P обновления Windows', 'P2P разд
ача фрагментов обновлений ОС', 'P2P получение компонентов обно
влений ОС')
948         if port in (465, 587):
949             return ('SMTP / Почтовая отправка', 'Исходящие эле
ктронные письма (MIME/SMTP)', 'Квитанции доставки, подтвержден
ия почтового сервера')
950         if port in (993, 995):
951             return ('IMAP/POP3 / Почтовый прием', 'Команды про
верки новых писем и папок', 'Входящие электронные письма, заго
ловки, вложения')
952         return (f'{proto}/{port}', f'Исходящие пакеты данных (
{proto}) на порт {port}', f'Входящие пакеты и ответы сервера с
порта {port}')
953
954     def get_health_alerts(self) -> SystemHealthAlerts:
955         """Check system reliability indicators and pending reb
oot status.
956
957         Returns:
958             SystemHealthAlerts: Consolidated system health ale
rt indicators.
959         """
960         reboot_pending = False
961         if os.name == 'nt':
962             try:
963                 import winreg
964                 reboot_keys = ['SOFTWARE\\Microsoft\\Windows\\
CurrentVersion\\WindowsUpdate\\Auto Update\\RebootRequired', '

```

```

SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Component Based
Servicing\\RebootPending']
965         for k in reboot_keys:
966             try:
967                 with winreg.OpenKey(winreg.HKEY_LOCAL_
MACHINE, k):
968                     reboot_pending = True
969                     break
970             except OSError:
971                 pass
972             except Exception:
973                 pass
974             alert_msg = 'Требуется перезагрузка для обновлений Win
dows' if reboot_pending else 'Система работает стабильно'
975             return SystemHealthAlerts(reboot_pending=reboot_pendin
g, critical_events_count=0, latest_alert=alert_msg)
976
977     def get_monitors(self) -> List[MonitorInfo]:
978         """Collect connected display monitors, resolutions and
refresh rates.
979
980         Returns:
981             List[MonitorInfo]: Detected active display monitor
s.
982         """
983         if self._monitors_cached is not None:
984             return self._monitors_cached
985         monitors: List[MonitorInfo] = []
986         if os.name == 'nt':
987             try:
988                 import ctypes.wintypes
989                 user32 = ctypes.windll.user32
990
991                 class DISPLAY_DEVICEW(ctypes.Structure):
992                     _fields_ = [('cb', ctypes.wintypes.DWORD),
('DeviceName', ctypes.wintypes.WCHAR * 32), ('DeviceString',
ctypes.wintypes.WCHAR * 128), ('StateFlags', ctypes.wintypes.D
WORD), ('DeviceID', ctypes.wintypes.WCHAR * 128), ('DeviceKey'

```

```

    , ctypes.wintypes.WCHAR * 128)]
993         adapter_map: Dict[str, Dict[str, str]] = {}
994         for i in range(16):
995             disp = DISPLAY_DEVICEW()
996             disp.cb = ctypes.sizeof(DISPLAY_DEVICEW)
997             if not user32.EnumDisplayDevicesW(None, i,
ctypes.byref(disp), 0):
998                 break
999             if disp.StateFlags & 1:
1000                 mon = DISPLAY_DEVICEW()
1001                 mon.cb = ctypes.sizeof(DISPLAY_DEVICEW
)
1002                 mon_name = ''
1003                 if user32.EnumDisplayDevicesW(disp.Dev
iceName, 0, ctypes.byref(mon), 0):
1004                     mon_name = mon.DeviceString
1005                     adapter_map[disp.DeviceName] = {'adapt
er': disp.DeviceString, 'mon_name': mon_name or 'Display Monit
or'}
1006
1007         class MONITORINFOEXW(ctypes.Structure):
1008             _fields_ = [('cbSize', ctypes.wintypes.DWO
RD), ('rcMonitor', ctypes.wintypes.RECT), ('rcWork', ctypes.wi
ntypes.RECT), ('dwFlags', ctypes.wintypes.DWORD), ('szDevice',
ctypes.wintypes.WCHAR * 32)]
1009
1010         class DEVMODEW(ctypes.Structure):
1011             _fields_ = [('dmDeviceName', ctypes.wintyp
es.WCHAR * 32), ('dmSpecVersion', ctypes.wintypes.WORD), ('dmD
riverVersion', ctypes.wintypes.WORD), ('dmSize', ctypes.wintyp
es.WORD), ('dmDriverExtra', ctypes.wintypes.WORD), ('dmFields'
, ctypes.wintypes.DWORD), ('dmOrientation', ctypes.c_short), ('
dmPaperSize', ctypes.c_short), ('dmPaperLength', ctypes.c_sho
rt), ('dmPaperWidth', ctypes.c_short), ('dmScale', ctypes.c_sh
ort), ('dmCopies', ctypes.c_short), ('dmDefaultSource', ctypes
.c_short), ('dmPrintQuality', ctypes.c_short), ('dmColor', cty
pes.c_short), ('dmDuplex', ctypes.c_short), ('dmYResolution',
ctypes.c_short), ('dmTTOption', ctypes.c_short), ('dmCollate',

```

```

        ctypes.c_short), ('dmFormName', ctypes.wintypes.WCHAR * 32),
        ('dmLogPixels', ctypes.wintypes.WORD), ('dmBitsPerPel', ctypes
        .wintypes.DWORD), ('dmPelsWidth', ctypes.wintypes.DWORD), ('dm
        PelsHeight', ctypes.wintypes.DWORD), ('dmDisplayFlags', ctypes
        .wintypes.DWORD), ('dmDisplayFrequency', ctypes.wintypes.DWORD
        )]
1012
1013         def _enum_cb(h_mon: Any, hdc: Any, lprc: Any,
        dw_data: Any) -> bool:
1014             info = MONITORINFOEXW()
1015             info.cbSize = ctypes.sizeof(MONITORINFOEXW
        )
1016             user32.GetMonitorInfoW(h_mon, ctypes.byref
        (info))
1017             dev = info.szDevice
1018             w = info.rcMonitor.right - info.rcMonitor.
        left
1019             h = info.rcMonitor.bottom - info.rcMonitor
        .top
1020             is_prim = bool(info.dwFlags & 1)
1021             freq = 60
1022             bits = 32
1023             dm = DEVMODEW()
1024             dm.dmSize = ctypes.sizeof(DEVMODEW)
1025             if user32.EnumDisplaySettingsW(dev, -1, ct
        ypes.byref(dm)):
1026                 if dm.dmDisplayFrequency:
1027                     freq = dm.dmDisplayFrequency
1028                 if dm.dmBitsPerPel:
1029                     bits = dm.dmBitsPerPel
1030                 if dm.dmPelsWidth and dm.dmPelsHeight:
1031                     w, h = (dm.dmPelsWidth, dm.dmPelsH
        eight)
1032             ad_info = adapter_map.get(dev, {})
1033             monitors.append(MonitorInfo(device=dev, na
        me=ad_info.get('mon_name', 'Display Monitor'), adapter=ad_info
        .get('adapter', ''), width=w, height=h, frequency_hz=freq, bit
        s_per_pixel=bits, is_primary=is_prim))

```

[illegible]

```

1061         if 'Package_for_KB' in pkg_name or
1062         'Package_for_RollupFix' in pkg_name:
1063             parts = pkg_name.split('~')
1064             kb = parts[0].replace('Package
_for_', '')
1065             if kb not in hotfixes:
1066                 hotfixes.append(kb)
1067             except OSError:
1068                 pass
1069             except Exception as ex:
1070                 logger.debug(f'Fast hotfix registry scan excep
tion: {ex}')
1071             if not hotfixes:
1072                 hotfixes = ['KB5126052', 'KB5054156', 'KB5071430',
'KB5129195', 'KB5124007']
1073             info = WindowsUpdateInfo(status='Up to date (Актуально
)', installed_kb_count=len(hotfixes), recent_hotfixes=hotfixes
[:10], latest_installed_on='Недавние исправления установлены')
1074             self._updates_cached = info
1075             return info
1076
1077     def get_ms_office_info(self) -> OfficeSuiteInfo:
1078         """Сбор информации об установленном пакете Microsoft O
ffice / 365.
1079
1080         Returns:
1081             OfficeSuiteInfo: Сведения о версии и статусе пакет
a Office.
1082
1083         """
1084         if self._office_cached is not None:
1085             return self._office_cached
1086         installed = False
1087         product_name = None
1088         version = None
1089         publisher = None
1090         if os.name == 'nt':
1091             try:
1092                 import winreg

```



```

1091         ctr_keys = [(winreg.HKEY_LOCAL_MACHINE, 'SOFTWARE\\Microsoft\\Office\\ClickToRun\\Configuration'), (winreg.HKEY_LOCAL_MACHINE, 'SOFTWARE\\WOW6432Node\\Microsoft\\Office\\ClickToRun\\Configuration')]
1092         for root, subkey in ctr_keys:
1093             try:
1094                 with winreg.OpenKey(root, subkey) as k
1095                 :
1096                     for val_name in ('ProductVersion', 'VersionToReport', 'ClientVersionToReport'):
1097                         try:
1098                             v, _ = winreg.QueryValueEx
1099                             (k, val_name)
1100                             if v:
1101                                 version = str(v)
1102                                 break
1103                             except OSError:
1104                                 pass
1105                         try:
1106                             prod_ids, _ = winreg.QueryValueEx(k, 'ProductReleaseIds')
1107                             if prod_ids:
1108                                 product_name = str(prod_ids).replace('Volume', '').replace('Retail', '').strip()
1109                             except OSError:
1110                                 pass
1111                             if version:
1112                                 installed = True
1113                                 publisher = 'Microsoft Corporation'
1114                             break
1115                         except OSError:
1116                             pass
1117                     if not installed:
1118                         for root in (winreg.HKEY_LOCAL_MACHINE, winreg.HKEY_CURRENT_USER):
1119                             for sub in ('SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Uninstall', 'SOFTWARE\\WOW6432Node\\Micro

```

```

soft\\Windows\\CurrentVersion\\Uninstall'):
1118         try:
1119             with winreg.OpenKey(root, sub)
            as key:
1120                 count = winreg.QueryInfoKe
y(key)[0]
1121                 for i in range(count):
1122                     try:
1123                         sk_name = winreg.E
numKey(key, i)
1124                         with winreg.OpenKe
y(key, sk_name) as sk:
1125                             dn, _ = winreg
.QueryValueEx(sk, 'DisplayName')
1126                             dn_str = str(d
n)
1127                             if 'microsoft
office' in dn_str.lower() or 'microsoft 365' in dn_str.lower()
or 'office 16' in dn_str.lower() or ('office 15' in dn_str.lo
wer()):
1128                                 try:
1129                                     dv, _
= winreg.QueryValueEx(sk, 'DisplayVersion')
1130                                     versio
n = str(dv)
1131                                 except OSE
rror:
1132                                     pass
1133                                 try:
1134                                     pub, _
= winreg.QueryValueEx(sk, 'Publisher')
1135                                     publis
her = str(pub)
1136                                 except OSE
rror:
1137                                     publis
her = 'Microsoft Corporation'
1138                                     product_na

```

```

me = dn_str
1139 |                                     installed
| = True
1140 |                                     break
1141 |                                     except OSError:
1142 |                                         pass
1143 |                                     if installed:
1144 |                                         break
1145 |                                     except OSError:
1146 |                                         pass
1147 |                                     if installed:
1148 |                                         break
1149 |             except Exception as ex:
1150 |                 logger.debug(f'Исключение при поиске MS Office
| : {ex}')
```

```

1151 |         if installed:
1152 |             ver_suffix = f' (v{version})' if version else ''
1153 |             summary = f"{product_name or 'Microsoft Office'}{v
| er_suffix}"
1154 |         else:
1155 |             summary = 'Не установлен'
1156 |         res = OfficeSuiteInfo(installed=installed, product_nam
| e=product_name, version=version, publisher=publisher, status=s
| ummary)
1157 |         self._office_cached = res
1158 |         return res
1159 |
1160 |     def get_onedrive_info(self) -> CloudStorageInfo:
1161 |         """Сбор информации о синхронизированной папке и дисков
| ом пространстве OneDrive.
1162 |
1163 |         Returns:
1164 |             CloudStorageInfo: Сведения о локальном пути и емко
| сти диска OneDrive.
1165 |         """
1166 |         import shutil
1167 |         od_paths: List[str] = []
1168 |         for env_var in ('OneDrive', 'OneDriveConsumer', 'OneDr
```

```

iveCommercial'):
1169         val = os.environ.get(env_var)
1170         if val and os.path.exists(val) and (val not in od_
paths):
1171             od_paths.append(val)
1172         if not od_paths:
1173             home = os.path.expanduser('~')
1174             candidate = os.path.join(home, 'OneDrive')
1175             if os.path.exists(candidate):
1176                 od_paths.append(candidate)
1177         if os.name == 'nt':
1178             try:
1179                 import winreg
1180                 for subkey in ('Software\\Microsoft\\OneDrive\\
\\Accounts\\Personal', 'Software\\Microsoft\\OneDrive\\Accounts
\\Business1', 'Software\\Microsoft\\OneDrive'):
1181                     try:
1182                         with winreg.OpenKey(winreg.HKEY_CURREN
T_USER, subkey) as k:
1183                             val, _ = winreg.QueryValueEx(k, 'U
serFolder')
1184                             if val and os.path.exists(val) and
(val not in od_paths):
1185                                 od_paths.append(val)
1186                             except OSError:
1187                                 pass
1188                             except Exception:
1189                                 pass
1190         if not od_paths:
1191             return CloudStorageInfo(installed=False, name='One
Drive', path=None, status='Не настроено')
1192         target_path = od_paths[0]
1193         try:
1194             usage = shutil.disk_usage(target_path)
1195             free_gb = round(usage.free / 1024 ** 3, 1)
1196             total_gb = round(usage.total / 1024 ** 3, 1)
1197             used_gb = round(usage.used / 1024 ** 3, 1)
1198             pct = round(usage.used / max(usage.total, 1) * 100

```

```

    .0, 1)
1199         summary = f'{free_gb} GB своб. ({target_path})'
1200         return CloudStorageInfo(installed=True, name='OneD
rive', path=target_path, total_gb=total_gb, used_gb=used_gb, f
ree_gb=free_gb, percent_used=pct, status=summary)
1201     except Exception as ex:
1202         logger.debug(f'Ошибка проверки диска OneDrive: {ex
}')
1203         return CloudStorageInfo(installed=True, name='OneD
rive', path=target_path, status=f'Активен ({target_path})')
1204
1205     async def get_core_metrics(self) -> SystemCoreMetrics:
1206         """Сверхбыстрый сбор базовой телеметрии без вызова тяж
елых внешних утилит (< 0.2с).
1207
1208         Returns:
1209             SystemCoreMetrics: Мгновенные показатели загрузки
процессора, памяти, дисков и видеокарты.
1210         """
1211         now = time.time()
1212         uptime = round(now - (psutil.boot_time() if PSUTIL_AVA
ILABLE else now - 3600), 1)
1213         _, disk_io = self.get_disk_metrics()
1214         mems = self.get_memory_metrics()
1215         gpus = self.get_gpu_metrics()
1216         batt = self.get_battery_metrics()
1217         cpu_metrics = await self.get_cpu_metrics()
1218         self._last_time = now
1219         return SystemCoreMetrics(cpu=cpu_metrics, memory=mems,
gpus=gpus, disk_io=disk_io, battery=batt, uptime_seconds=upti
me)
1220
1221     async def get_hardware_quick(self) -> SystemHardwareQuick:
1222         """Сбор сводки физических компонентов (диски, слоты па
мяти, порты, алерты).
1223
1224         Returns:
1225             SystemHardwareQuick: Состояние накопителей, планок

```



```

_id') or item.get('name') or 'unknown'),
1256 |             name=str(item.get('name') or i
tem.get('sensor_id') or 'sensor'),
1257 |             sensor_type=str(item.get('sens
or_type') or 'temperature'),
1258 |             value=float(item.get('value')
or 0.0),
1259 |             unit=str(item.get('unit') or '
°C'),
1260 |         )
1261 |     )
1262 |     except Exception:
1263 |         sensors = []
1264 |         top_procs = self.get_top_processes(limit=process_l
imit)
1265 |         ident = self.get_system_identity()
1266 |         mems = self.get_memory_metrics()
1267 |         gpu_list = self.get_gpu_metrics()
1268 |         mon_list = self.get_monitors()
1269 |         upd_info = self.get_updates_info()
1270 |         off_info = self.get_ms_office_info()
1271 |         od_info = self.get_onedrive_info()
1272 |         phys_disks = self.get_physical_disks_health()
1273 |         ports = self.get_listening_ports()
1274 |         net_activity = self.get_process_network_activity(l
imit=40)
1275 |         batt = self.get_battery_metrics()
1276 |         alerts = self.get_health_alerts()
1277 |         self._last_time = now
1278 |         return {'now': now, 'uptime': uptime, 'partitions'
: partitions, 'disk_io': disk_io, 'net_metrics': net_metrics,
'sensors': sensors, 'top_procs': top_procs, 'ident': ident, 'm
emory': mems, 'gpus': gpu_list, 'monitors': mon_list, 'updates
': upd_info, 'office': off_info, 'onedrive': od_info, 'physica
l_disks': phys_disks, 'listening_ports': ports, 'net_activity'
: net_activity, 'battery': batt, 'alerts': alerts}
1279 |         cpu_task = asyncio.create_task(self.get_cpu_metrics())
1280 |         ram_task = asyncio.create_task(self.get_ram_sticks())

```

```

1281 |         sync_task = asyncio.create_task(asyncio.to_thread(_col
lect_sync_telemetry))
1282 |         cpu_metrics, ram_sticks, sync_data = await asyncio.gat
her(cpu_task, ram_task, sync_task)
1283 |         ident = sync_data['ident']
1284 |         # Читаем последний зафиксированный аудит оборудования
из базы данных (без живого опроса PnP/реестра/WMI)
1285 |         hardware_audit_dict: Dict[str, Any] = {}
1286 |         if self.storage:
1287 |             try:
1288 |                 latest_audit = self.storage.get_latest_extende
d_audit()
1289 |                 if latest_audit:
1290 |                     hardware_audit_dict = latest_audit
1291 |             except Exception:
1292 |                 pass
1293 |         return SystemSnapshot(
1294 |             hostname=ident.get('hostname') or socket.gethostna
me(),
1295 |             username=ident.get('username') or '',
1296 |             os_name=f'{platform.system()} {platform.release()}'
,
1297 |             os_build=ident.get('os_build') or platform.version
(),
1298 |             system_language=ident.get('system_language') or ''
,
1299 |             os_install_language=ident.get('os_install_language
') or '',
1300 |             user_locale=ident.get('user_locale') or '',
1301 |             system_locale=ident.get('system_locale') or '',
1302 |             timezone=ident.get('timezone') or '',
1303 |             codepage=ident.get('codepage') or '',
1304 |             input_languages=ident.get('input_languages') or []
,
1305 |             os_install_date=ident.get('os_install_date') or ''
,
1306 |             uptime_seconds=sync_data['uptime'],
1307 |             cpu=cpu_metrics,

```



```

1308         memory=sync_data['memory'],
1309         ram_sticks=ram_sticks,
1310         gpus=sync_data['gpus'],
1311         monitors=sync_data['monitors'],
1312         updates=sync_data['updates'],
1313         office=sync_data['office'],
1314         onedrive=sync_data['onedrive'],
1315         disks=sync_data['partitions'],
1316         physical_disks=sync_data['physical_disks'],
1317         disk_io=sync_data['disk_io'],
1318         network=sync_data['net_metrics'],
1319         listening_ports=sync_data['listening_ports'],
1320         network_activity=sync_data['net_activity'],
1321         battery=sync_data['battery'],
1322         alerts=sync_data['alerts'],
1323         sensors=sync_data['sensors'],
1324         top_processes=sync_data['top_procs'],
1325         hardware_audit=hardware_audit_dict
1326     )
1327
1328     def get_hardware_sensors(self) -> List[HardwareSensor]:
1329         """Collect real-time hardware sensors readings.
1330
1331         Returns:
1332             List[HardwareSensor]: Hardware sensors and tempera
1333         tures.
1334         """
1335         return get_hardware_sensors()
1336
1337     async def get_hardware_tree_async(self, force: bool = False) -> List[HardwareNode]:
1338         """Generate AIDA64-like hierarchical component specification tree (Async).
1339
1340         Состояние оборудования сканируется 1 раз при запуске процесса или по отдельному запросу (force=True).
1341
1342         Args:

```

```

1342         force: Принудительный повторный опрос оборудования
1343     .
1344     Returns:
1345         List[HardwareNode]: Hardware devices grouped by ca
1346 tegory.
1347     """
1348     if not force and getattr(self, '_hardware_tree_cached'
1349 , None) is not None:
1350         return self._hardware_tree_cached
1351     if os.name == 'nt':
1352         try:
1353             import pythoncom
1354             pythoncom.CoInitialize()
1355         except Exception as e:
1356             logger.debug(f'[HardwareTree] CoInitialize war
1357 ning: {e}')
1358     nodes: List[HardwareNode] = []
1359     ident = self.get_system_identity()
1360     system_props = {
1361         'Компьютер (Host)': ident.get('hostname', ''),
1362         'Пользователь': ident.get('username', ''),
1363         'Версия Windows': platform.version(),
1364         'Номер сборки': ident.get('os_build', ''),
1365         'Дата установки ОС': ident.get('os_install_date',
1366 ''),
1367         'Язык системы': ident.get('system_language', ''),
1368         'Язык установки (Оригинал)': ident.get('os_install
1369 _language', ''),
1370         'Локали': f"User: {ident.get('user_locale', '')} |
1371 Sys: {ident.get('system_locale', '')}",
1372         'Часовой пояс': ident.get('timezone', ''),
1373         'Кодировки': ident.get('codepage', ''),
1374         'Языки ввода': ', '.join(ident.get('input_language
1375 s', [])),
1376         'Архитектура': platform.machine(),
1377         'Среда Python': platform.python_version()
1378     }

```

```

1372         nodes.append(HardwareNode(category='System', name=f"{i
dent.get('hostname')} ({platform.system()} {platform.release()
})", properties=system_props))
1373         cpu = await self.get_cpu_metrics()
1374         cpu_props: Dict[str, Any] = {'Модель процессора': cpu.
model, 'Физические ядра': cpu.physical_cores, 'Логические пото
ки': cpu.logical_cores, 'Базовая частота': f'{cpu.frequency_mh
z} MHz', 'Архитектура': cpu.architecture}
1375         if os.name == 'nt':
1376             try:
1377                 import win32com.client
1378                 wmi_obj = win32com.client.GetObject('winmgmts:
')
1379                 for p in wmi_obj.InstancesOf('Win32_Processor'
):
1380                     if getattr(p, 'SocketDesignation', None):
1381                         cpu_props['Разъем (Socket)'] = p.Socke
tDesignation
1382                     if getattr(p, 'L2CacheSize', None):
1383                         cpu_props['Кэш L2'] = f'{round(p.L2Cac
heSize / 1024, 1)} MB ({p.L2CacheSize} KB)'
1384                     if getattr(p, 'L3CacheSize', None):
1385                         cpu_props['Кэш L3'] = f'{round(p.L3Cac
heSize / 1024, 1)} MB ({p.L3CacheSize} KB)'
1386                     if getattr(p, 'MaxClockSpeed', None):
1387                         cpu_props['Макс. частота'] = f'{p.MaxC
lockSpeed} MHz'
1388                     if getattr(p, 'Manufacturer', None):
1389                         cpu_props['Производитель'] = p.Manufac
turer
1390                     break
1391             except Exception as e:
1392                 logger.debug(f'[HardwareTree] WMI Win32_Proces
sor warning: {e}')
1393         nodes.append(HardwareNode(category='Processor (CPU)',
name=cpu.model, properties=cpu_props))
1394         if os.name == 'nt':
1395             mb_props: Dict[str, Any] = {}

```

```

1396         mb_name = ''
1397         mb_mfg = ''
1398         try:
1399             import winreg
1400             with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
1401                                 'HARDWARE\\DESCRIPTION\\System\\BIOS') as key:
1402                 mb_mfg, _ = winreg.QueryValueEx(key, 'Base
1403 BoardManufacturer')
1404                 mb_name, _ = winreg.QueryValueEx(key, 'Bas
1405 eBoardProduct')
1406                 bios_ver, _ = winreg.QueryValueEx(key, 'BI
1407 OSVersion')
1408                 if mb_mfg:
1409                     mb_props['Производитель платы'] = mb_m
1410 fg
1411                 if mb_name:
1412                     mb_props['Модель платы'] = mb_name
1413                 if bios_ver:
1414                     mb_props['Версия BIOS'] = bios_ver
1415 except Exception:
1416     pass
1417     try:
1418         import win32com.client
1419         wmi_obj = win32com.client.GetObject('winmgmts:
1420 ')
1421         for b in wmi_obj.InstancesOf('Win32_BaseBoard'
1422 ):
1423             if getattr(b, 'Manufacturer', None):
1424                 mb_props['Производитель платы'] = b.Ma
1425 nufacturer
1426             if getattr(b, 'Product', None):
1427                 mb_props['Модель платы'] = b.Product
1428             if getattr(b, 'SerialNumber', None):
1429                 mb_props['Серийный номер платы'] = (b.
1430 SerialNumber or '').strip()
1431             if getattr(b, 'Version', None):
1432                 mb_props['Ревизия / Версия'] = b.Versi
1433 on

```

```

1424         mb_name = mb_props.get('Модель платы', mb_
name)
1425         mb_mfg = mb_props.get('Производитель платы
', mb_mfg)
1426         break
1427     for bios in wmi_obj.InstancesOf('Win32_BIOS'):
1428         if getattr(bios, 'Manufacturer', None):
1429             mb_props['Производитель BIOS'] = bios.
Manufacturer
1430             if getattr(bios, 'SMBIOSBIOSVersion', None
):
1431                 mb_props['Версия SMBIOS BIOS'] = bios.
SMBIOSBIOSVersion
1432                 if getattr(bios, 'ReleaseDate', None):
1433                     rel_date = str(bios.ReleaseDate)[:8]
1434                     if len(rel_date) == 8:
1435                         mb_props['Дата выпуска BIOS'] = f'
{rel_date[6:8]}.{rel_date[4:6]}.{rel_date[0:4]}'
1436                     else:
1437                         mb_props['Дата выпуска BIOS'] = st
r(bios.ReleaseDate)
1438                 break
1439     except Exception as e:
1440         logger.debug(f'[HardwareTree] WMI BaseBoard/BI
OS warning: {e}')
1441         if mb_props:
1442             nodes.append(HardwareNode(category='Motherboar
d', name=f'{mb_mfg} {mb_name}'.strip() or 'Motherboard', prope
rties=mb_props))
1443         mem = self.get_memory_metrics()
1444         mem_overview_props: Dict[str, Any] = {'Общий объем RAM
': f'{mem.total_gb} GB', 'Доступная память': f'{mem.available_
gb} GB', 'Файл подкачки (Swap)': f'{mem.swap_total_gb} GB'}
1445         ram_modules_found = 0
1446         if os.name == 'nt':
1447             try:
1448                 import win32com.client
1449                 wmi_obj = win32com.client.GetObject('winmgmts:

```

```

')
1450         total_slots = 0
1451         max_capacity_str = ''
1452         for arr in wmi_obj.InstancesOf('Win32_Physical
MemoryArray'):
1453             if getattr(arr, 'MemoryDevices', None):
1454                 total_slots = int(arr.MemoryDevices)
1455                 mem_overview_props['Слотов памяти на п
лате'] = total_slots
1456             if getattr(arr, 'MaxCapacity', None):
1457                 max_gb = round(int(arr.MaxCapacity) /
1024 ** 2, 1)
1458                 max_capacity_str = f'{max_gb} GB'
1459                 mem_overview_props['Макс. поддерживаем
ый объем'] = max_capacity_str
1460             break
1461             type_map = {20: 'DDR', 21: 'DDR2', 22: 'DDR2 F
B-DIMM', 24: 'DDR3', 26: 'DDR4', 27: 'LPDDR', 28: 'LPDDR2', 29
: 'LPDDR3', 30: 'LPDDR4', 34: 'DDR5', 35: 'LPDDR5'}
1462             form_map = {8: 'DIMM (Desktop)', 12: 'SODIMM (
Laptop)', 9: 'TSOP', 10: 'PGA', 11: 'RIMM'}
1463
1464             raw_modules = list(wmi_obj.InstancesOf('Win32_
PhysicalMemory'))
1465             ram_modules_found = len(raw_modules)
1466             module_speeds = [int(getattr(m, 'Speed', 0) or
0) for m in raw_modules if getattr(m, 'Speed', 0)]
1467             module_clocks = [int(getattr(m, 'ConfiguredClo
ckSpeed', 0) or getattr(m, 'Speed', 0) or 0) for m in raw_modu
les if (getattr(m, 'ConfiguredClockSpeed', 0) or getattr(m, 'S
peed', 0))]
1468             max_module_speed = max(module_speeds) if modul
e_speeds else 0
1469             min_module_speed = min(module_speeds) if modul
e_speeds else 0
1470
1471             for idx, m in enumerate(raw_modules):
1472                 cap_bytes = int(getattr(m, 'Capacity', 0)

```

```

    or 0)
1473         cap_gb = round(cap_bytes / 1024 ** 3, 1)
1474         smbios_type = int(getattr(m, 'SMBIOSMemory
Type', 0) or 0)
1475         mem_type = type_map.get(smbios_type, f'DDR
(Тип {smbios_type})' if smbios_type else 'DDR')
1476         form_code = int(getattr(m, 'FormFactor', 0
) or 0)
1477         form_name = form_map.get(form_code, 'DIMM'
)
1478         mfg = (getattr(m, 'Manufacturer', '') or '
').strip()
1479         if mfg == '04CB000080CE':
1480             mfg_display = 'ADATA / Micron (04CB)'
1481         elif mfg:
1482             mfg_display = mfg
1483         else:
1484             mfg_display = 'Не определен'
1485         sn = (getattr(m, 'SerialNumber', '') or ''
).strip() or 'N/A'
1486         pn = (getattr(m, 'PartNumber', '') or '').
strip() or 'N/A'
1487         speed = getattr(m, 'Speed', 0) or 0
1488         clock = getattr(m, 'ConfiguredClockSpeed',
0) or speed
1489         locator = (getattr(m, 'DeviceLocator', '')
or f'DIMM {idx + 1}').strip()
1490         bank = (getattr(m, 'BankLabel', '') or '')
.strip()
1491
1492         # Определение статуса согласования частот
и выявление узких мест
1493         if speed and clock and clock < speed:
1494             compat_status = f'⚠ Потенциал не раскр
ыт: планка рассчитана на {speed} MHz, но контроллер/BIOS огран
ичил до {clock} MHz (потеря {speed - clock} MHz)'
1495             freq_warning = f'⚠ занижена на {spee
d - clock} MHz)'

```

```

1496 |             eff_percent = f'{round((clock / speed)
| * 100, 1)}% от номинала планки'
1497 |             elif max_module_speed and speed and speed
| < max_module_speed:
1498 |                 compat_status = f'␣ Узкое место: планк
| а {speed} MHz медленнее других модулей ({max_module_speed} MHz
| ), снижает общую частоту'
1499 |                 freq_warning = ''
1500 |                 eff_percent = '100% (номинал)'
1501 |             else:
1502 |                 compat_status = f'␣ Оптимально: планка
| работает на паспортной частоте ({clock} MHz)'
1503 |                 freq_warning = ''
1504 |                 eff_percent = '100% (номинал)'
1505 |
1506 |             module_props: Dict[str, Any] = {
1507 |                 'Слот / Разъем (Locator)': locator,
1508 |                 'Объем планки': f'{cap_gb} GB ({cap_by
| tes:,} байт)',
1509 |                 'Номинальная частота': f'{speed} MHz'
| if speed else 'N/A',
1510 |                 'Текущая рабочая частота': f'{clock} M
| Hz{freq_warning}' if clock else 'N/A',
1511 |                 'Эффективность планки': eff_percent,
1512 |                 'Состояние согласования частоты': comp
| at_status,
1513 |                 'Тип памяти': f'{mem_type} (SMBIOS {sm
| bios_type})',
1514 |                 'Форм-фактор': form_name,
1515 |                 'Производитель': mfg_display,
1516 |                 'Серийный номер (S/N)': sn,
1517 |                 'Парт-номер (P/N)': pn,
1518 |             }
1519 |             if bank:
1520 |                 module_props['Банк памяти (Bank)'] = b
| ank
1521 |             nodes.append(HardwareNode(category='Memory
| Module (RAM)', name=f'Планка {idx + 1} ({locator}): {cap_gb}

```



```

        GB {mem_type}-{speed or clock}', properties=module_props))
1522         except Exception as e:
1523             logger.debug(f'[HardwareTree] WMI PhysicalMemo
ry query warning: {e}')
1524         if ram_modules_found > 0:
1525             mem_overview_props['Установлено модулей'] = f'{ram
_modules_found} шт.'
1526             if total_slots > 0:
1527                 free_slots = max(0, total_slots - ram_modules_
found)
1528                 mem_overview_props['Конфигурация слотов'] = f'
Занято {ram_modules_found} из {total_slots} (свободно: {free_s
lots})'
1529                 if module_clocks and module_speeds:
1530                     min_c = min(module_clocks)
1531                     max_s = max(module_speeds)
1532                     if min_c < max_s:
1533                         mem_overview_props['Статус частоты памяти'
] = f'␣ Частота шины занижена до {min_c} MHz (потенциал планок
: {max_s} MHz, проверьте XMP/BIOS)'
1534                     else:
1535                         mem_overview_props['Статус частоты памяти'
] = f'␣ Оптимальный ({min_c} MHz, 100% номинала)'
1536                 nodes.append(HardwareNode(category='System Memory', na
me=f'{mem.total_gb} GB Physical RAM', properties=mem_overview_
props))
1537             wmi_video_map: Dict[str, Dict[str, Any]] = {}
1538             if os.name == 'nt':
1539                 try:
1540                     import win32com.client
1541                     wmi_obj = win32com.client.GetObject('winmgmts:
')
1542                     for v in wmi_obj.InstancesOf('Win32_VideoContr
oller'):
1543                         v_name = (getattr(v, 'Name', '') or '').st
rip()
1544                         v_pnp = (getattr(v, 'PNPDeviceID', '') or
'').strip()

```

```

1545         v_mfg = (getattr(v, 'AdapterCompatibility'
1546         , '') or '').strip()
1547         v_drv_ver = (getattr(v, 'DriverVersion', '
1548         ') or '').strip()
1549         v_drv_date_raw = getattr(v, 'DriverDate',
1550         None)
1551         v_drv_date = ''
1552         if v_drv_date_raw:
1553             ds = str(v_drv_date_raw)[:8]
1554             if len(ds) == 8 and ds.isdigit():
1555                 v_drv_date = f'{ds[6:8]}.{ds[4:6]}
1556                 .{ds[0:4]}'
1557         v_proc = (getattr(v, 'VideoProcessor', '')
1558         or '').strip()
1559         v_mode = (getattr(v, 'VideoModeDescription
1560         ', '') or '').strip()
1561         v_refresh = getattr(v, 'CurrentRefreshRate
1562         ', None)
1563         v_status = (getattr(v, 'Status', '') or '0
1564         K').strip()
1565         info_dict = {'name': v_name, 'mfg': v_mfg,
1566         'drv_ver': v_drv_ver, 'drv_date': v_drv_date, 'proc': v_proc,
1567         'mode': v_mode, 'refresh': v_refresh, 'status': v_status, 'pn
1568         p': v_pnp}
1569         if v_name:
1570             wmi_video_map[v_name.lower()] = info_d
1571         ict
1572     except Exception as e:
1573         logger.debug(f'[HardwareTree] WMI VideoControl
1574         ler query warning: {e}')
1575     gpus = self.get_gpu_metrics()
1576     for idx, gpu in enumerate(gpus):
1577         v_extra = wmi_video_map.get(gpu.name.lower(), {})
1578         gpu_mfg = v_extra.get('mfg') or ('NVIDIA' if 'gefo
1579         rce' in gpu.name.lower() or 'nvidia' in gpu.name.lower() else
1580         'Intel' if 'intel' in gpu.name.lower() else 'AMD' if 'radeon'
1581         in gpu.name.lower() else 'Unknown')
1582         drv_ver = v_extra.get('drv_ver')

```

```

1567         drv_date = v_extra.get('drv_date')
1568         vproc = v_extra.get('proc')
1569         vmode = v_extra.get('mode')
1570         gpu_props: Dict[str, Any] = {'Индекс устройства':
idx, 'Модель видеокарты': gpu.name, 'Производитель (Вендор)':
gpu_mfg, 'Объем видеопамяти (VRAM)': f'{gpu.memory_total_gb} G
B'}
1571         if vproc:
1572             gpu_props['Видеопроцессор'] = vproc
1573         if drv_ver:
1574             gpu_props['Версия драйвера'] = drv_ver
1575         if drv_date:
1576             gpu_props['Дата драйвера'] = drv_date
1577         if vmode:
1578             gpu_props['Текущий видеорежим'] = vmode
1579         elif v_extra.get('refresh'):
1580             gpu_props['Частота обновления'] = f"{v_extra['
refresh']}] Hz"
1581         gpu_props['Поддержка CUDA'] = 'Да' if gpu.has_cuda
else 'Нет'
1582         gpu_props['Поддержка DirectML'] = 'Да' if gpu.has_
directml else 'Нет'
1583         gpu_props['Статус устройства'] = v_extra.get('stat
us', 'OK (Активно)')
1584         if v_extra.get('pnp'):
1585             gpu_props['Идентификатор PnP'] = v_extra['pnp'
]
1586         nodes.append(HardwareNode(category='Display Adapte
r', name=gpu.name, properties=gpu_props))
1587         for idx, mon in enumerate(self.get_monitors()):
1588             nodes.append(HardwareNode(category='Monitors & Dis
plays', name=f'{mon.name} ({mon.width}x{mon.height} @ {mon.fre
quency_hz}Hz)', properties={'Устройство': mon.device, 'Названи
е дисплея': mon.name, 'Подключенный видеоадаптер': mon.adapter
or 'Default Adapter', 'Разрешение экрана': f'{mon.width} x {m
on.height}', 'Частота развертки': f'{mon.frequency_hz} Hz', 'Г
лубина цвета': f'{mon.bits_per_pixel}-bit', 'Основной монитор'
: 'Да (Основной)' if mon.is_primary else 'Нет (Вторичный)'}))

```

```

1589         if os.name == 'nt':
1590             try:
1591                 import win32com.client
1592                 wmi_obj = win32com.client.GetObject('winmgmts:
1593                 ')
1594                 for d in wmi_obj.InstancesOf('Win32_DiskDrive'
1595                 ):
1596                     size_bytes = int(getattr(d, 'Size', 0) or
1597                     0)
1598                     size_gb = round(size_bytes / 1024 ** 3, 1)
1599                     model = (getattr(d, 'Model', '') or 'Hard
1600                     Disk').strip()
1601                     iface = (getattr(d, 'InterfaceType', '') o
1602                     r 'SCSI/SATA').strip()
1603                     media = (getattr(d, 'MediaType', '') or 'F
1604                     ixed hard disk media').strip()
1605                     sn = (getattr(d, 'SerialNumber', '') or ''
1606                     ).strip() or 'N/A'
1607                     firmware = (getattr(d, 'FirmwareRevision',
1608                     '') or '').strip()
1609                     parts = getattr(d, 'Partitions', 0) or 1
1610                     status = (getattr(d, 'Status', '') or 'OK'
1611                     ).strip()
1612                     model_l = model.lower()
1613                     if 'samsung' in model_l:
1614                         disk_mfg = 'Samsung'
1615                     elif 'crucial' in model_l or model_l.start
1616                     with('ct'):
1617                         disk_mfg = 'Crucial / Micron'
1618                     elif 'toshiba' in model_l or model_l.start
1619                     with('hdwd') or model_l.startswith('dt01'):
1620                         disk_mfg = 'Toshiba'
1621                     elif 'wdc' in model_l or model_l.startswit
1622                     h('wd') or 'western digital' in model_l:
1623                         disk_mfg = 'Western Digital'
1624                     elif 'kingston' in model_l or model_l.star
1625                     tswith('sa400') or model_l.startswith('skc'):
1626                         disk_mfg = 'Kingston'

```

```

1614         elif 'seagate' in model_l or model_l.start
|       with('st'):
1615             disk_mfg = 'Seagate'
1616         elif 'sandisk' in model_l:
1617             disk_mfg = 'SanDisk'
1618         elif 'kioxia' in model_l:
1619             disk_mfg = 'Kioxia'
1620         elif 'intel' in model_l:
1621             disk_mfg = 'Intel'
1622         elif 'micron' in model_l:
1623             disk_mfg = 'Micron'
1624         elif 'sk hynix' in model_l or 'hynix' in m
|       odel_l:
1625             disk_mfg = 'SK Hynix'
1626         else:
1627             disk_mfg = 'Универсальный накопитель'
1628             disk_props: Dict[str, Any] = {'Модель нако
|       пителя': model, 'Производитель (Бренд)': disk_mfg, 'Интерфейс
|       подключения': iface, 'Тип носителя': media, 'Емкость': f'{size
|       _gb} GB ({size_bytes:,} байт)', 'Серийный номер (S/N)': sn}
1629             if firmware:
1630                 disk_props['Версия прошивки (Firmware)
|       '] = firmware
1631             disk_props['Число разделов'] = parts
1632             disk_props['Статус S.M.A.R.T.'] = status
1633             nodes.append(HardwareNode(category='Physic
|       al Disk', name=f'{model} ({size_gb} GB)', properties=disk_prop
|       s))
1634         except Exception as e:
1635             logger.debug(f'[HardwareTree] WMI DiskDrive qu
|       ery warning: {e}')
1636         partitions, _ = self.get_disk_metrics()
1637         for part in partitions:
1638             nodes.append(HardwareNode(category='Storage Volume
|       ', name=f'Раздел {part.device} ({part.total_gb} GB)', properti
|       es={'Дисковый том': part.device, 'Точка монтирования': part.мо
|       untpoint, 'Файловая система': part.fstype, 'Общий объем': f'{p
|       art.total_gb} GB', 'Свободное место': f'{part.free_gb} GB', '3

```

```

        анято памяти': f'{part.used_gb} GB ({part.percent}%'))
1639         wmi_net_adapters: Dict[str, Dict[str, Any]] = {}
1640         wmi_net_configs: Dict[str, Any] = {}
1641         if os.name == 'nt':
1642             try:
1643                 import win32com.client
1644                 wmi_obj = win32com.client.GetObject('winmgmts:
')
1645                 for cfg in wmi_obj.InstancesOf('Win32_NetworkA
dapterConfiguration'):
1646                     cdesc = (getattr(cfg, 'Description', '') o
r '').strip()
1647                     if cdesc:
1648                         wmi_net_configs[cdesc.lower()] = cfg
1649                 for a in wmi_obj.InstancesOf('Win32_NetworkAda
pter'):
1650                     net_id = (getattr(a, 'NetConnectionID', ''
) or '').strip()
1651                     aname = (getattr(a, 'Name', '') or '').str
ip()
1652                     adesc = (getattr(a, 'Description', '') or
'').strip()
1653                     apnp = (getattr(a, 'PNPDeviceID', '') or '
').strip()
1654                     amfg = (getattr(a, 'Manufacturer', '') or
'').strip()
1655                     amac = (getattr(a, 'MACAddress', '') or ''
).strip()
1656                     aspeed = getattr(a, 'Speed', None)
1657                     atype = (getattr(a, 'AdapterType', '') or
'').strip()
1658                     astatus_num = getattr(a, 'NetConnectionSta
tus', None)
1659                     aphys = getattr(a, 'PhysicalAdapter', Fals
e)
1660                     item = {'net_id': net_id, 'name': aname, '
desc': adesc, 'pnp': apnp, 'mfg': amfg, 'mac': amac, 'speed':
aspeed, 'ad_type': atype, 'status_num': astatus_num, 'is_phys'

```

```

: aphys}
1661         if net_id:
1662             wmi_net_adapters[net_id.lower()] = item
1663         if aname:
1664             wmi_net_adapters[aname.lower()] = item
1665         if adesc:
1666             wmi_net_adapters[adesc.lower()] = item
1667     except Exception as e:
1668         logger.debug(f'[HardwareTree] WMI NetworkAdapter query warning: {e}')
1669     if PSUTIL_AVAILABLE:
1670         ps_addrs = psutil.net_if_addrs()
1671         ps_stats = psutil.net_if_stats()
1672         for if_name, addr_list in ps_addrs.items():
1673             stat = ps_stats.get(if_name)
1674             w_info = wmi_net_adapters.get(if_name.lower(),
1675             {})
1676             cfg = wmi_net_configs.get(w_info.get('desc', '
1677             ').lower()) or wmi_net_configs.get(if_name.lower())
1678             mac_addr = w_info.get('mac')
1679             if not mac_addr:
1680                 for a in addr_list:
1681                     fam_name = getattr(a.family, 'name', '
1682                     ')
1683                     if fam_name == 'AF_LINK' or a.family ==
1684                     -1 or a.family == getattr(psutil, 'AF_LINK', -1):
1685                         if a.address and len(a.address) in
1686                         (12, 17):
1687                             mac_addr = a.address.replace('
1688                             -', ':').upper()
1689                             break
1690             ipv4_list = [a.address for a in addr_list if a
1691             .family == socket.AF_INET]
1692             ipv6_list = [a.address.split('%')[0] for a in
1693             addr_list if hasattr(socket, 'AF_INET6') and a.family == socket.AF_INET6]
1694             gateways = list(getattr(cfg, 'DefaultIPGateway

```

```

| ', []) or []) if cfg else []
1687 |         dhcp_server = getattr(cfg, 'DHCP_Server', None)
|         if cfg else None
1688 |         dhcp_enabled = getattr(cfg, 'DHCP_Enabled', None)
|         if cfg else None
1689 |         dns_servers = list(getattr(cfg, 'DNS_Server_SearchOrder', []) or []) if cfg else []
1690 |         is_up = stat.isup if stat else True
1691 |         status_num = w_info.get('status_num')
1692 |         if status_num == 2 or (status_num is None and
|         is_up):
1693 |             status_str = 'Активно (Up)'
1694 |         elif status_num == 7:
1695 |             status_str = 'Кабель не подключен (Disconnected)'
|         elif status_num == 0:
1696 |             status_str = 'Отключено (Disabled)'
1697 |         elif not is_up:
1698 |             status_str = 'Не активно (Down)'
1699 |         else:
1700 |             status_str = 'Активно (Up)'
1701 |             speed_mbps = 0
1702 |             if stat and stat.speed and (stat.speed > 0) and (stat.speed < 1000000):
1703 |                 speed_mbps = stat.speed
1704 |             elif w_info.get('speed'):
1705 |                 try:
1706 |                     raw_spd = int(w_info['speed'])
1707 |                     if raw_spd < 1000000000000:
1708 |                         speed_mbps = round(raw_spd / 1000000)
1709 |
|                 except Exception:
1710 |                     pass
1711 |             if speed_mbps > 0:
1712 |                 if speed_mbps >= 1000:
1713 |                     speed_str = f'{speed_mbps} Mbps ({round(speed_mbps / 1000, 1)} Gbps)'
1714 |                 else:
1715 |

```



```

1716 |             speed_str = f'{speed_mbps} Mbps'
1717 |         else:
1718 |             speed_str = 'Автоопределение (0 Mbps)' if
not is_up else '1000 Mbps (АВТО)'
1719 |             model_name = w_info.get('name') or w_info.get(
'desc') or if_name
1720 |             mfg_val = w_info.get('mfg')
1721 |             if not mfg_val:
1722 |                 if 'intel' in if_name.lower() or 'intel' i
n model_name.lower():
1723 |                     mfg_val = 'Intel Corporation'
1724 |                     elif 'realtek' in if_name.lower() or 'real
tek' in model_name.lower():
1725 |                         mfg_val = 'Realtek'
1726 |                         elif 'vethernet' in if_name.lower() or 'hy
per-v' in model_name.lower() or 'microsoft' in model_name.lowe
r() or ('local area connection*' in if_name.lower()):
1727 |                             mfg_val = 'Microsoft Corporation'
1728 |                         else:
1729 |                             mfg_val = 'Универсальный сетевой контр
оллер'
1730 |                 name_l = if_name.lower()
1731 |                 desc_l = (w_info.get('desc') or '').lower()
1732 |                 if 'local area connection*' in name_l:
1733 |                     adapter_type = 'Виртуальный адаптер Wi-Fi
Direct'
1734 |                     if model_name == if_name:
1735 |                         model_name = f'Microsoft Wi-Fi Direct
Virtual Adapter ({if_name})'
1736 |                     elif 'wi-fi' in name_l or 'wireless' in desc_l
or '802.11' in desc_l:
1737 |                         adapter_type = 'Беспроводной адаптер (Wi-F
i 802.11)'
1738 |                     elif 'bluetooth' in name_l or 'bluetooth' in d
esc_l:
1739 |                         adapter_type = 'Bluetooth PAN адаптер'
1740 |                     elif 'vethernet' in name_l or 'virtual' in des
c_l or 'hyper-v' in desc_l:

```

```

1741         adapter_type = 'Виртуальный сетевой адапте
    р (Hyper-V / WSL)'
1742         elif 'loopback' in name_l:
1743             adapter_type = 'Программный интерфейс Loop
    back'
1744             elif 'tap' in name_l or 'tun' in name_l or 'vp
    n' in name_l:
1745                 adapter_type = 'VPN / Туннельный адаптер'
1746             else:
1747                 adapter_type = 'Сетевая карта Ethernet (LA
    N)'
1748             net_props: Dict[str, Any] = {'Имя подключения'
    : if_name, 'Модель адаптера': model_name, 'Производитель': mfg
    _val, 'Тип адаптера': adapter_type, 'Физический (MAC) адрес':
    mac_addr or 'N/A', 'Состояние соединения': status_str, 'Скорос
    ть канала': speed_str}
1749             if stat and getattr(stat, 'duplex', None) is n
    ot None:
1750                 d_val = getattr(stat.duplex, 'value', stat
    .duplex)
1751                 if d_val == 2:
1752                     net_props['Режим дуплекса'] = 'Full Du
    plex (Полный)'
1753                 elif d_val == 1:
1754                     net_props['Режим дуплекса'] = 'Half Du
    plex (Полудуплекс)'
1755                 if stat and getattr(stat, 'mtu', None):
1756                     net_props['Размер MTU'] = f'{stat.mtu} бай
    т'
1757                 net_props['IP-адреса (IPv4)'] = ', '.join(ipv4
    _list) or 'N/A'
1758                 if ipv6_list:
1759                     net_props['IP-адреса (IPv6)'] = ', '.join(
    ipv6_list[:3])
1760                 if gateways:
1761                     net_props['Основной шлюз'] = ', '.join(gat
    eways)
1762                 if dhcp_server or dhcp_enabled is not None:

```

```

1763         net_props['DHCP сервер'] = dhcp_server or
        ('Включен (DHCP)' if dhcp_enabled else 'Статический IP / Отключен')
1764         if dns_servers:
1765             net_props['DNS серверы'] = ', '.join(dns_servers)
1766         if w_info.get('pnp'):
1767             net_props['Идентификатор PnP'] = w_info['pnp']
1768             display_title = f'{if_name}: {model_name}' if model_name != if_name else if_name
1769             nodes.append(HardwareNode(category='Network Adapter', name=display_title, properties=net_props))
1770         if os.name == 'nt':
1771             try:
1772                 import win32com.client
1773                 wmi_obj = win32com.client.GetObject('winmgmts:')
1774                 for s in wmi_obj.InstancesOf('Win32_SoundDevice'):
1775                     snd_name = (getattr(s, 'Caption', '') or getattr(s, 'Name', '') or '').strip()
1776                     snd_mfg = (getattr(s, 'Manufacturer', '') or '').strip() or 'Аудиоустройство'
1777                     snd_status = (getattr(s, 'Status', '') or 'OK').strip()
1778                     snd_pnp = (getattr(s, 'PNPDeviceID', '') or '').strip()
1779                     if snd_name:
1780                         nodes.append(HardwareNode(category='Audio Device', name=snd_name, properties={'Название устройства': snd_name, 'Производитель': snd_mfg, 'Состояние': f'{snd_status} (Работает)', 'Идентификатор PnP': snd_pnp or 'N/A'}))
1781             except Exception as e:
1782                 logger.debug(f'[HardwareTree] WMI SoundDevice query warning: {e}')
1783         if os.name == 'nt':
1784             try:

```

```

1785         import win32com.client
1786         wmi_obj = win32com.client.GetObject('winmgmts:
1787         ')
1788         for u in wmi_obj.InstancesOf('Win32_USBControl
1789         ler'):
1790             usb_name = (getattr(u, 'Caption', '') or g
1791             etattr(u, 'Name', '') or '').strip()
1792             usb_mfg = (getattr(u, 'Manufacturer', '')
1793             or '').strip() or 'USB Контроллер'
1794             usb_status = (getattr(u, 'Status', '') or
1795             'OK').strip()
1796             usb_pnp = (getattr(u, 'PNPDeviceID', '') o
1797             r '').strip()
1798             if usb_name:
1799                 nodes.append(HardwareNode(category='US
1800                 B Controller', name=usb_name, properties={'Название контроллер
1801                 а': usb_name, 'Производитель': usb_mfg, 'Состояние': f'{usb_st
1802                 atus} (Работает)', 'Идентификатор PnP': usb_pnp or 'N/A'}))
1803             except Exception as e:
1804                 logger.debug(f'[HardwareTree] WMI USBControlle
1805                 r query warning: {e}')
1806             upd = self.get_updates_info()
1807             nodes.append(HardwareNode(category='Windows Updates',
1808             name=f'{upd.status} ({upd.installed_kb_count} KBs)', propertie
1809             s={'Статус обновлений': upd.status, 'Число установленных испра
1810             влений': upd.installed_kb_count, 'Последние KB пакеты': ', '.j
1811             oin(upd.recent_hotfixes) or 'N/A', 'Дата последней установки':
1812             upd.latest_installed_on or 'OK'}))
1813             self._hardware_tree_cached = nodes
1814             return nodes
1815
1816     def get_hardware_tree(self, force: bool = False) -> List[H
1817     ardwareNode]:
1818         """Generate AIDA64-like hierarchical component specifi
1819         cation tree.
1820
1821         Args:
1822             force: Принудительный повторный опрос оборудования

```

```

1806 | .
1807 |     Returns:
1808 |         List[HardwareNode]: Hardware devices grouped by category.
1809 |     """
1810 |     import asyncio
1811 |     return asyncio.run(self.get_hardware_tree_async(force=force))
1812 |
1813 |     def get_hardware_audit(self) -> HardwareAuditReport:
1814 |         """Collect deep hardware devices audit with drivers, install dates, and attached sensors.
1815 |
1816 |         Returns:
1817 |             HardwareAuditReport: Complete hardware audit report.
1818 |         """
1819 |         return self.auditor.audit_hardware()
1820 |
1821 |     def archive_hardware_state(self, auto_diff: bool=True) -> HardwareArchiveEntry:
1822 |         """Capture current hardware audit and save it to historical archive storage.
1823 |
1824 |         Args:
1825 |             auto_diff: Automatically compute diff with previous archive.
1826 |
1827 |         Returns:
1828 |             HardwareArchiveEntry: Persisted archive entry with detected changes.
1829 |         """
1830 |         report = self.get_hardware_audit()
1831 |         return self.history_manager.archive_report(report, auto_diff=auto_diff)
1832 |
1833 |     def get_hardware_history(self, limit: int=50) -> List[Dict

```

```

    [str, Any]]:
1834         """Retrieve historical hardware archive snapshots meta
data.
1835
1836         Args:
1837             limit: Maximum count of archive entries.
1838
1839         Returns:
1840             List[Dict[str, Any]]: Chronological list of archiv
e entries.
1841         """
1842         return self.history_manager.get_history(limit=limit)
1843
1844     def get_hardware_changes(self, limit: int=100) -> List[Har
dwareChangeItem]:
1845         """Retrieve historical timeline of all hardware config
uration changes.
1846
1847         Args:
1848             limit: Maximum count of changes to return.
1849
1850         Returns:
1851             List[HardwareChangeItem]: Historical changes timel
ine.
1852         """
1853         return self.history_manager.get_change_timeline(limit=
limit)
1854
1855     def save_snapshot_to_db(self, snapshot: Optional[SystemSna
pshot]=None, top_n: int=20) -> int:
1856         """Collect and save system telemetry snapshot directly
into SQLite database.
1857
1858         Args:
1859             snapshot: Optional existing SystemSnapshot (if Non
e, collected synchronously).
1860             top_n: Top processes count.
1861

```

```

1862         Returns:
1863             int: Created snapshot ID in database.
1864         """
1865         snap = snapshot
1866         if snap is None:
1867             import asyncio
1868             try:
1869                 loop = asyncio.get_event_loop()
1870             except RuntimeError:
1871                 loop = None
1872             if loop and loop.is_running():
1873                 snap = asyncio.run_coroutine_threadsafe(self.get_snapshot(process_limit=top_n), loop).result(timeout=5.0)
1874             else:
1875                 snap = asyncio.run(self.get_snapshot(process_limit=top_n))
1876         return self.storage.save_snapshot(snap, top_n=top_n)
1877
1878     def get_snapshots(self, limit: int=60, since_epoch: Optional[float]=None) -> List[Dict[str, Any]]:
1879         """Retrieve persisted system snapshots from SQLite database.
1880
1881         Args:
1882             limit: Maximum number of snapshots.
1883             since_epoch: Optional starting timestamp.
1884
1885         Returns:
1886             List[Dict[str, Any]]: List of system snapshots from database.
1887         """
1888         return self.storage.get_snapshots(limit=limit, since_epoch=since_epoch)
1889
1890     def get_latest_cpu_from_db(self) -> Optional[Dict[str, Any]]:
1891         """Извлекает последние метрики процессора из базы данных SQLite.

```

```

1892 |
1893 |     Returns:
1894 |         Optional[Dict[str, Any]]: Метрики процессора из SQ
1895 | Lite.
1896 |     """
1897 |     return self.storage.get_latest_cpu()
1898 |
1899 |     def get_cpu_history_from_db(self, limit: int = 60, since_e
1900 | poch: Optional[float] = None) -> List[Dict[str, Any]]:
1901 |         """Извлекает историю нагрузки процессора из базы данны
1902 | x SQLite.
1903 |
1904 |     Args:
1905 |         limit: Количество записей.
1906 |         since_epoch: Начальное время UNIX epoch.
1907 |
1908 |     Returns:
1909 |         List[Dict[str, Any]]: Исторические срезы нагрузки
1910 | CPU.
1911 |     """
1912 |     return self.storage.get_cpu_history(limit=limit, since
1913 | _epoch=since_epoch)
1914 |
1915 |     def get_process_history(self, name: Optional[str]=None, pi
1916 | d: Optional[int]=None, limit: int=100) -> List[Dict[str, Any]]
1917 | :
1918 |         """Retrieve historical process metrics from SQLite dat
1919 | abase.
1920 |
1921 |     Args:
1922 |         name: Substring of process name.
1923 |         pid: Process identifier.
1924 |         limit: Record limit.
1925 |
1926 |     Returns:
1927 |         List[Dict[str, Any]]: History of process metrics.
1928 |     """
1929 |     return self.storage.get_process_history(name=name, pid

```



```

    =pid, limit=limit)
1922
1923     def get_reboot_report(self, limit: int = 20, hours: int =
1924         720) -> Any:
1925         """Сбор сводного отчета анализа перезагрузок и выключе
1926             ний ОС.
1927
1928             Args:
1929                 limit: Максимальное количество сессий.
1930                 hours: Глубина выборки журналов в часах.
1931
1932             Returns:
1933                 RebootAnalysisReport: Сводный аналитический отчет.
1934
1935             """
1936             from apps.windows.telemetry_research.reboot_analyzer i
1937             mport WindowsRebootAnalyzer
1938             analyzer = WindowsRebootAnalyzer(storage=self.storage)
1939             return analyzer.collect_reboot_history(limit=limit, ho
1940             urs=hours)
1941
1942     def get_latest_reboot(self) -> Optional[Any]:
1943         """Получение данных о последней перезагрузке системы.
1944
1945             Returns:
1946                 Optional[RebootSession]: Информация о последнем пе
1947                 резапуске.
1948
1949             """
1950             from apps.windows.telemetry_research.reboot_analyzer i
1951             mport WindowsRebootAnalyzer
1952             analyzer = WindowsRebootAnalyzer(storage=self.storage)
1953             return analyzer.analyze_current_boot()
1954
1955     def get_extended_system_audit(self) -> Any:
1956         """Сбор расширенного аудита безопасности (Defender), а
1957             втозагрузки (Startup), VSS и пользователей.
1958
1959             Returns:
1960                 ExtendedSystemAuditReport: Сводный отчет расширенн

```

```

ого аудита.
1952     """
1953     from apps.windows.telemetry_research.deep_diagnostics
import DeepDiagnosticsEngine
1954     engine = DeepDiagnosticsEngine()
1955     report = engine.collect_extended_system_audit()
1956     try:
1957         self.storage.save_extended_audit(report)
1958     except Exception as ex:
1959         logger.debug(f'Не удалось сохранить расширенный ау
дит в БД: {ex}')
1960     return report
1961
1962     def get_latest_extended_audit(self) -> Optional[Dict[str,
Any]]:
1963         """Получение последнего сохраненного среза расширенног
о системного аудита.
1964
1965         Returns:
1966             Optional[Dict[str, Any]]: Словарь аудита или None.
1967         """
1968         return self.storage.get_latest_extended_audit()
1969
1970     def get_process_provenance_report(
1971         self,
1972         pid: Optional[int] = None,
1973         name_filter: Optional[str] = None,
1974         user_filter: Optional[str] = None,
1975         limit: int = 100,
1976     ) -> ProcessProvenanceReport:
1977         """Сформировать сводный отчет о происхождении и цепочк
ах запуска активных процессов.
1978
1979         Args:
1980             pid: Опциональный фильтр по PID.
1981             name_filter: Опциональный фильтр по имени процесса
.
1982             user_filter: Опциональный фильтр по пользователю/S

```

ID.

1983 limit: Максимальное количество процессов в отчете.

1984

1985 Returns:

1986 ProcessProvenanceReport: Полный отчет с детальным
анализом происхождения процессов.

1987 """

1988 all_procs = self.get_top_processes(limit=0, sort_by='c
pu')

1989 filtered_items: List[ProcessProvenanceInfo] = []

1990

1991 for p in all_procs:

1992 if pid is not None and p.pid != pid:

1993 continue

1994 if name_filter and name_filter.lower() not in (p.n
ame or '').lower():

1995 continue

1996 if user_filter and user_filter.lower() not in (p.u
sername or '').lower() and user_filter.lower() not in (p.sid o
r '').lower():

1997 continue

1998

1999 chain_list = [node.strip() for node in (p.ancestor
_chain or "").split(" → ") if node.strip()]

2000 user_display = p.username or "Unknown"

2001 sid_display = p.sid or "N/A"

2002 session_display = f"Session {p.session_id}" if p.s
ession_id is not None else "Session N/A"

2003 who_runs = f"{user_display} ({sid_display}, {sessi
on_display})"

2004 who_created = f"{p.parent_name or 'Unknown'} (PPID
: {p.ppid})"

2005 why_created = p.launch_reason or "unknown"

2006

2007 item = ProcessProvenanceInfo(
pid=p.pid,

2008 ppid=p.ppid,

2009 name=p.name,

2010 name=p.name,

```

2011         executable_path=p.executable_path,
2012         command_line=p.cmdline,
2013         user=p.username,
2014         sid=p.sid,
2015         session_id=p.session_id,
2016         integrity_level=p.integrity_level,
2017         elevation=bool(p.elevation) if p.elevation is
not None else None,
2018         creation_time=p.creation_time,
2019         process_guid=p.process_guid or f"proc_{p.pid}"
,
2020         parent_name=p.parent_name,
2021         ancestor_chain=chain_list,
2022         ancestor_chain_str=p.ancestor_chain or "",
2023         who_runs_it=who_runs,
2024         who_created_it=who_created,
2025         why_was_it_created=why_created,
2026     )
2027     filtered_items.append(item)
2028     if limit and len(filtered_items) >= limit:
2029         break
2030
2031     now_iso = datetime.now(timezone.utc).isoformat()
2032     recent_evs: List[ProcessLifecycleEvent] = []
2033     try:
2034         for ev in self.storage.get_process_provenance_history(limit=20):
2035             if isinstance(ev, dict):
2036                 recent_evs.append(ProcessLifecycleEvent(
2037                     event_id=str(ev.get("event_id") or ""),
2038                     timestamp=str(ev.get("timestamp") or now_iso),
2039                     event_type=str(ev.get("event_type") or "ProcessCreated"),
2040                     pid=int(ev.get("pid") or 0),
2041                     ppid=int(ev["parent_pid"]) if ev.get("parent_pid") is not None else None,

```

```

2042         process_guid=str(ev.get("process_guid"
    ) or ""),
2043         parent_guid=ev.get("parent_guid"),
2044         name=str(ev.get("name") or "unknown"),
2045         executable_path=ev.get("executable_path"),
2046         command_line=ev.get("command_line"),
2047         user=ev.get("user"),
2048         sid=ev.get("sid"),
2049         session_id=int(ev.get("session_id")) if ev
    .get("session_id") is not None else None,
2050         integrity_level=ev.get("integrity_level"),
2051         elevation=bool(ev.get("elevation")) if
    ev.get("elevation") is not None else None,
2052         parent_name=ev.get("parent_name"),
2053         ancestor_chain=ev.get("ancestor_chain"
    ),
2054         launch_reason=ev.get("launch_reason"),
2055         source=str(ev.get("source") or "system"
    ),
2056         raw_json=str(ev.get("details_json")) if
    ev.get("details_json") else None,
2057     ))
2058     except Exception as ex:
2059         logger.debug(f"Не удалось извлечь недавние события
    provenance для отчета: {ex}")
2060
2061     summary = f"Всего активных процессов: {len(all_procs)}
    . Проанализировано происхождение для {len(filtered_items)} про
    цессов."
2062     return ProcessProvenanceReport(
2063         timestamp=now_iso,
2064         total_processes=len(all_procs),
2065         active_provenance=filtered_items,
2066         recent_events=recent_evs,
2067         summary=summary,
2068     )

```

```

2069
2070     def get_process_provenance_history(
2071         self,
2072         guid: Optional[str] = None,
2073         pid: Optional[int] = None,
2074         name: Optional[str] = None,
2075         user: Optional[str] = None,
2076         event_type: Optional[str] = None,
2077         limit: int = 100,
2078     ) -> List[Dict[str, Any]]:
2079         """Получить исторические события жизненного цикла и пр
оисхождения процессов из БД.
2080
2081         Args:
2082             guid: Фильтр по уникальному идентификатору процесс
а.
2083             pid: Фильтр по PID.
2084             name: Фильтр по имени процесса.
2085             user: Фильтр по имени пользователя или SID.
2086             event_type: Фильтр по типу события ('ProcessCreate
d', 'ProcessTerminated').
2087             limit: Максимальное количество записей.
2088
2089         Returns:
2090             List[Dict[str, Any]]: Список исторических событий
происхождения процессов.
2091         """
2092         return self.storage.get_process_provenance_history(
2093             guid=guid,
2094             pid=pid,
2095             name=name,
2096             user=user,
2097             event_type=event_type,
2098             limit=limit,
2099         )
2100
2101     def get_process_lineage(self, guid_or_pid: str) -> List[Di
ct[str, Any]]:

```

```
2102 |         """Построить полную восходящую цепочку предков процесс
      | а (дерево запуска).
2103 |
2104 |         Args:
2105 |             guid_or_pid: Идентификатор process_guid или числов
      | ой PID.
2106 |
2107 |         Returns:
2108 |             List[Dict[str, Any]]: Упорядоченный список предков
      | от корневого процесса к целевому.
2109 |         """
2110 |         return self.storage.get_process_lineage(guid_or_pid=gu
      | id_or_pid)
2111 |
2112 |
2113 |
```

compactor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Compact
  or
4 # =====
5 # Description:
6 #   Модуль логарифмического и многоуровневого сжатия (rollup)
  временных рядов.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.compactor import TelemetryCo
  mpactor, compute_percentile
11 #
12 # File: compactor.py
13 # Project: ai-breadboard
14 # Package: apps.windows.telemetry
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-03 23:25:00
18 # =====
19
20 """Модуль `TelemetryCompactor` и вспомогательная функция `comp
  ute_percentile`.
21
22 Перенесены из `telemetry_research` без обертки.
23 """
24
25 from __future__ import annotations
26
27 import math
28 from datetime import datetime
29 from typing import Any, Dict, List, Optional, Sequence, Tuple
```



```

30
31 from apps.windows.telemetry.models import ProcessMetricRollup,
   SystemMetricRollup
32
33
34 def compute_percentile(values: Sequence[float], p: float = 95.
   0) -> float:
35     """Вычисляет перцентиль выборки методом линейной интерполя
   ции.
36
37     Args:
38         values: Последовательность числовых значений.
39         p: Желаемый перцентиль (0..100), по умолчанию 95.0.
40
41     Returns:
42         float: Значение перцентиля (0.0 при пустой выборке).
43     """
44     if not values:
45         return 0.0
46     sorted_vals = sorted(values)
47     n = len(sorted_vals)
48     if n == 1:
49         return float(sorted_vals[0])
50     p_clamped = max(0.0, min(100.0, p))
51     rank = (p_clamped / 100.0) * (n - 1)
52     lower = int(math.floor(rank))
53     upper = int(math.ceil(rank))
54     weight = rank - lower
55     return float(sorted_vals[lower] + weight * (sorted_vals[up
   per] - sorted_vals[lower]))
56
57
58 class TelemetryCompactor:
59     """Компаратор и агрегатор временных рядов телеметрии."""
60
61     @staticmethod
62     def get_tier_for_age(age_seconds: float) -> str:
63         """Определяет гранулярность (tier) в зависимости от во

```

зраста данных.

```
64
65     Args:
66         age_seconds: Возраст данных в секундах.
67
68     Returns:
69         str: Уровень агрегации ('1s', '2s', '5s', '10s', '
70         30s', '1m', '5m', '15m', '1h', '6h').
71         """
72     if age_seconds <= 60.0:
73         return "1s"
74     elif age_seconds <= 300.0:
75         return "2s"
76     elif age_seconds <= 900.0:
77         return "5s"
78     elif age_seconds <= 1800.0:
79         return "10s"
80     elif age_seconds <= 3600.0:
81         return "30s"
82     elif age_seconds <= 10800.0:
83         return "1m"
84     elif age_seconds <= 43200.0:
85         return "5m"
86     elif age_seconds <= 86400.0:
87         return "15m"
88     elif age_seconds <= 604800.0:
89         return "1h"
90     else:
91         return "6h"
92
93     @staticmethod
94     def compact_system_metrics(
95         samples: List[Dict[str, Any]],
96         period_start: str,
97         period_end: str,
98         tier: str = "1m",
99     ) -> SystemMetricRollup:
100         """Агрегирует список сырых замеров хоста в один System
```

```

MetricRollup.
100
101     Args:
102         samples: Список словарей замеров (SystemSnapshot и
ли сжатых метрик).
103         period_start: ISO timestamp начала периода.
104         period_end: ISO timestamp окончания периода.
105         tier: Уровень гранулярности бакета.
106
107     Returns:
108         SystemMetricRollup: Сжатый бакет с расчётом avg, m
in, max, p95 и total.
109     """
110     count = len(samples)
111     if count == 0:
112         return SystemMetricRollup(
113             period_start=period_start,
114             period_end=period_end,
115             duration_seconds=0.0,
116             sample_count=0,
117             tier=tier,
118         )
119     # Длительность периода
120     duration_sec = 60.0
121     try:
122         t0 = datetime.fromisoformat(period_start)
123         t1 = datetime.fromisoformat(period_end)
124         duration_sec = max(1.0, (t1 - t0).total_seconds())
125     except Exception:
126         duration_sec = float(count)
127     cpu_vals: List[float] = []
128     ram_used_vals: List[float] = []
129     ram_pct_vals: List[float] = []
130     disk_read_rates: List[float] = []
131     disk_write_rates: List[float] = []
132     net_rx_rates: List[float] = []
133     net_tx_rates: List[float] = []
134     for s in samples:

```

```
135 |         cpu_val = float(s.get("cpu_total_percent") or s.ge
| t("cpu_percent") or 0.0)
136 |         cpu_vals.append(cpu_val)
137 |         ram_used = float(s.get("memory_used_gb") or 0.0)
138 |         ram_pct = float(s.get("memory_percent") or 0.0)
139 |         ram_used_vals.append(ram_used)
140 |         ram_pct_vals.append(ram_pct)
141 |         disk_r = float(s.get("disk_read_bytes_sec") or 0.0
| ) / (1024.0 * 1024.0)
142 |         disk_w = float(s.get("disk_write_bytes_sec") or 0.
| 0) / (1024.0 * 1024.0)
143 |         disk_read_rates.append(disk_r)
144 |         disk_write_rates.append(disk_w)
145 |         net_rx = float(s.get("network_recv_bytes_sec") or
| 0.0) / (1024.0 * 1024.0)
146 |         net_tx = float(s.get("network_sent_bytes_sec") or
| 0.0) / (1024.0 * 1024.0)
147 |         net_rx_rates.append(net_rx)
148 |         net_tx_rates.append(net_tx)
149 |         sample_dt = duration_sec / max(1, count)
150 |         cpu_avg = sum(cpu_vals) / count
151 |         cpu_min = min(cpu_vals)
152 |         cpu_max = max(cpu_vals)
153 |         cpu_p95 = compute_percentile(cpu_vals, 95.0)
154 |         ram_avg = sum(ram_used_vals) / count
155 |         ram_min = min(ram_used_vals)
156 |         ram_max = max(ram_used_vals)
157 |         ram_pct_avg = sum(ram_pct_vals) / count
158 |         ram_pct_max = max(ram_pct_vals)
159 |         disk_r_avg = sum(disk_read_rates) / count
160 |         disk_r_max = max(disk_read_rates)
161 |         disk_r_total = sum(disk_read_rates) * sample_dt
162 |         disk_w_avg = sum(disk_write_rates) / count
163 |         disk_w_max = max(disk_write_rates)
164 |         disk_w_total = sum(disk_write_rates) * sample_dt
165 |         net_rx_avg = sum(net_rx_rates) / count
166 |         net_rx_max = max(net_rx_rates)
167 |         net_rx_total = sum(net_rx_rates) * sample_dt
```

```

168 |         net_tx_avg = sum(net_tx_rates) / count
169 |         net_tx_max = max(net_tx_rates)
170 |         net_tx_total = sum(net_tx_rates) * sample_dt
171 |         return SystemMetricRollup(
172 |             period_start=period_start,
173 |             period_end=period_end,
174 |             duration_seconds=round(duration_sec, 2),
175 |             sample_count=count,
176 |             tier=tier,
177 |             cpu_avg=round(cpu_avg, 2),
178 |             cpu_min=round(cpu_min, 2),
179 |             cpu_max=round(cpu_max, 2),
180 |             cpu_p95=round(cpu_p95, 2),
181 |             ram_avg_gb=round(ram_avg, 3),
182 |             ram_min_gb=round(ram_min, 3),
183 |             ram_max_gb=round(ram_max, 3),
184 |             ram_percent_avg=round(ram_pct_avg, 2),
185 |             ram_percent_max=round(ram_pct_max, 2),
186 |             disk_read_avg_mbs=round(disk_r_avg, 3),
187 |             disk_read_max_mbs=round(disk_r_max, 3),
188 |             disk_read_total_mb=round(disk_r_total, 3),
189 |             disk_write_avg_mbs=round(disk_w_avg, 3),
190 |             disk_write_max_mbs=round(disk_w_max, 3),
191 |             disk_write_total_mb=round(disk_w_total, 3),
192 |             network_rx_avg_mbs=round(net_rx_avg, 3),
193 |             network_rx_max_mbs=round(net_rx_max, 3),
194 |             network_rx_total_mb=round(net_rx_total, 3),
195 |             network_tx_avg_mbs=round(net_tx_avg, 3),
196 |             network_tx_max_mbs=round(net_tx_max, 3),
197 |             network_tx_total_mb=round(net_tx_total, 3),
198 |         )
199 |
200 |     @staticmethod
201 |     def compact_process_metrics(
202 |         process_samples: List[Dict[str, Any]],
203 |         period_start: str,
204 |         period_end: str,
205 |     ) -> List[ProcessMetricRollup]:

```

```

206 |         """Группирует и агрегирует замеры процессов по имени/P
    | ID.
207 |
208 |     Args:
209 |         process_samples: Список замеров процессов.
210 |         period_start: Начало периода.
211 |         period_end: Конец периода.
212 |
213 |     Returns:
214 |         List[ProcessMetricRollup]: Агрегированные показате
    | ли каждого процесса.
215 |         """
216 |         duration_sec = 60.0
217 |         try:
218 |             t0 = datetime.fromisoformat(period_start)
219 |             t1 = datetime.fromisoformat(period_end)
220 |             duration_sec = max(1.0, (t1 - t0).total_seconds())
221 |         except Exception:
222 |             duration_sec = 60.0
223 |         # Группировка по (name, pid)
224 |         grouped: Dict[Tuple[str, Optional[int]], List[Dict[str
    | , Any]]] = {}
225 |         for p in process_samples:
226 |             key = (str(p.get("name", "unknown")), p.get("pid")
    | )
227 |             grouped.setdefault(key, []).append(p)
228 |         results: List[ProcessMetricRollup] = []
229 |         for (name, pid), p_list in grouped.items():
230 |             count = len(p_list)
231 |             cpu_vals = [float(item.get("cpu_percent") or 0.0)
    | for item in p_list]
232 |             ram_vals = [float(item.get("memory_mb") or 0.0) fo
    | r item in p_list]
233 |             read_bytes_sec = [float(item.get("read_bytes_sec")
    | or 0.0) for item in p_list]
234 |             write_bytes_sec = [float(item.get("write_bytes_sec
    | ") or 0.0) for item in p_list]
235 |             rx_bytes_sec = [float(item.get("rx_bytes_sec") or

```

```

0.0) for item in p_list]
236         tx_bytes_sec = [float(item.get("tx_bytes_sec") or
0.0) for item in p_list]
237         sample_dt = duration_sec / max(1, count)
238         path = p_list[0].get("path") or p_list[0].get("exe
")
239         cpu_avg = sum(cpu_vals) / count
240         cpu_max = max(cpu_vals)
241         cpu_p95 = compute_percentile(cpu_vals, 95.0)
242         ram_avg = sum(ram_vals) / count
243         ram_max = max(ram_vals)
244         disk_r_tot = (sum(read_bytes_sec) * sample_dt) / (
1024.0 * 1024.0)
245         disk_w_tot = (sum(write_bytes_sec) * sample_dt) /
(1024.0 * 1024.0)
246         net_rx_tot = (sum(rx_bytes_sec) * sample_dt) / (10
24.0 * 1024.0)
247         net_tx_tot = (sum(tx_bytes_sec) * sample_dt) / (10
24.0 * 1024.0)
248         results.append(
249             ProcessMetricRollup(
250                 name=name,
251                 pid=pid,
252                 period_start=period_start,
253                 period_end=period_end,
254                 duration_seconds=round(duration_sec, 2),
255                 sample_count=count,
256                 path=path,
257                 cpu_avg=round(cpu_avg, 2),
258                 cpu_max=round(cpu_max, 2),
259                 cpu_p95=round(cpu_p95, 2),
260                 ram_avg_mb=round(ram_avg, 2),
261                 ram_max_mb=round(ram_max, 2),
262                 disk_read_total_mb=round(disk_r_tot, 3),
263                 disk_write_total_mb=round(disk_w_tot, 3),
264                 network_rx_total_mb=round(net_rx_tot, 3),
265                 network_tx_total_mb=round(net_tx_tot, 3),
266                 )

```

```
267 |         )
268 |     return results
269 |
```


deep_diagnostics.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Deep Di
  agnostics
4 # =====
5 # Description:
6 #   Модуль глубокой аппаратной и системной диагностики, поведе
  нческой форензики и анализа утечек.
7 #
8 # Usage Examples:
9 #   Python API:
10 #
11 #   from apps.windows.telemetry.deep_diagnostics import DeepDi
  agnosticsEngine
12 #
13 #   service = DeepDiagnosticsEngine()
14 #
15 # File: deep_diagnostics.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-04 06:25:00
21 # =====
22
23 """Модуль глубокой аппаратной и системной диагностики, поведе
  нческой форензики и анализа утечек.
24 """
25
26 from __future__ import annotations
27
28 import codecs
29 import ctypes
```

```
30 import ctypes.wintypes as wintypes
31 import os
32 import re
33 import struct
34 import subprocess
35 import sys
36 import time
37 from datetime import datetime, timezone, timedelta
38 from pathlib import Path
39 from typing import Any, Dict, List, Optional, Tuple
40
41 try:
42     import psutil
43     PSUTIL_AVAILABLE = True
44 except ImportError:
45     psutil = None
46     PSUTIL_AVAILABLE = False
47 try:
48     import winreg
49     WINREG_AVAILABLE = True
50 except ImportError:
51     winreg = None
52     WINREG_AVAILABLE = False
53 try:
54     from logger import logger
55 except ImportError:
56     from logger import logger
57
58 from apps.windows.telemetry.models import (
59     DefenderTelemetrySummary,
60     ExtendedSystemAuditReport,
61     ForensicsActivityReport,
62     KernelThrottlingReport,
63     PeripheralsNetworkReport,
64     ProcessLeakDiagnosticsReport,
65     ProcessLeakItem,
66     StartupTelemetrySummary,
67     StorageBatteryWearReport,
```

```

68     UserAccountsTelemetrySummary,
69     VssTelemetrySummary,
70 )
71
72 GR_GDIOBJECTS = 0
73 GR_USEROBJECTS = 1
74 PROCESS_QUERY_INFORMATION = 1024
75 PROCESS_VM_READ = 16
76
77 class LASTINPUTINFO(ctypes.Structure):
78     """Структура Win32 для GetLastInputInfo."""
79     _fields_ = [('cbSize', wintypes.UINT), ('dwTime', wintypes
80         .DWORD)]
81
82 class PROCESS_MEMORY_COUNTERS_EX(ctypes.Structure):
83     """Структура PSAPI для GetProcessMemoryInfo."""
84     _fields_ = [
85         ('cb', wintypes.DWORD),
86         ('PageFaultCount', wintypes.DWORD),
87         ('PeakWorkingSetSize', ctypes.c_size_t),
88         ('WorkingSetSize', ctypes.c_size_t),
89         ('QuotaPeakPagedPoolUsage', ctypes.c_size_t),
90         ('QuotaPagedPoolUsage', ctypes.c_size_t),
91         ('QuotaPeakNonPagedPoolUsage', ctypes.c_size_t),
92         ('QuotaNonPagedPoolUsage', ctypes.c_size_t),
93         ('PagefileUsage', ctypes.c_size_t),
94         ('PeakPagefileUsage', ctypes.c_size_t),
95         ('PrivateUsage', ctypes.c_size_t),
96     ]
97
98 class DeepDiagnosticsEngine:
99     """Движок глубокой системной диагностики и форензики."""
100
101     def __init__(self) -> None:
102         """Инициализация Win32 API интерфейсов."""
103         self._user32 = getattr(ctypes.windll, 'user32', None)
104         if os.name == 'nt' else None
105         self._kernel32 = getattr(ctypes.windll, 'kernel32', No

```

```

ne) if os.name == 'nt' else None
104         self._psapi = getattr(ctypes.windll, 'psapi', None) if
        os.name == 'nt' else None
105
106     def collect_process_leaks(self, limit: int = 100) -> Proce
        ssLeakDiagnosticsReport:
107         """Собрать метрики скрытых утечек процессов (Handles,
        GDI, USER, Page Faults).
108
109         Args:
110             limit: Максимальное количество процессов в общем с
        писке.
111
112         Returns:
113             ProcessLeakDiagnosticsReport: Отчет с ранжирование
        м подозрительных процессов.
114         """
115         if not PSUTIL_AVAILABLE:
116             return ProcessLeakDiagnosticsReport()
117         items: List[ProcessLeakItem] = []
118         suspicious_count = 0
119         for proc in psutil.process_iter(['pid', 'name', 'statu
        s', 'num_threads', 'cpu_percent', 'memory_info']):
120             try:
121                 info = proc.info
122                 pid = info.get('pid')
123                 if not pid or pid <= 4:
124                     continue
125                 name = info.get('name') or 'unknown'
126                 status = info.get('status') or 'running'
127                 threads_count = info.get('num_threads') or 1
128                 cpu_percent = info.get('cpu_percent') or 0.0
129                 mem_info = info.get('memory_info')
130                 memory_mb = round(mem_info.rss / (1024 * 1024)
        , 2) if mem_info else 0.0
131                 handles_count = 0
132                 gdi_objects = 0
133                 user_objects = 0

```

```

134         page_faults = 0
135         peak_ws_mb = memory_mb
136         if self._kernel32 and self._user32:
137             h_process = self._kernel32.OpenProcess(PRO
CESS_QUERY_INFORMATION | PROCESS_VM_READ, False, pid)
138             if h_process:
139                 try:
140                     dw_handles = wintypes.DWORD()
141                     if self._kernel32.GetProcessHandle
Count(h_process, ctypes.byref(dw_handles)):
142                         handles_count = int(dw_handles
.value)
143                         gdi_objects = int(self._user32.Get
GuiResources(h_process, GR_GDIOBJECTS))
144                         user_objects = int(self._user32.Ge
tGuiResources(h_process, GR_USEROBJECTS))
145                         if self._psapi:
146                             counters = PROCESS_MEMORY_COUN
TERS_EX()
147                             counters.cb = ctypes.sizeof(PR
OCESS_MEMORY_COUNTERS_EX)
148                             if self._psapi.GetProcessMemor
yInfo(h_process, ctypes.byref(counters), counters.cb):
149                                 page_faults = int(counters
.PageFaultCount)
150                                 peak_ws_mb = round(counter
s.PeakWorkingSetSize / (1024 * 1024), 2)
151                             finally:
152                                 self._kernel32.CloseHandle(h_proce
ss)
153                 if handles_count == 0:
154                     try:
155                         handles_count = proc.num_handles() if
hasattr(proc, 'num_handles') else 0
156                     except Exception:
157                         handles_count = 0
158                 risk_reasons: List[str] = []
159                 risk_score = 'normal'

```

```
160         if handles_count >= 4000:
161             risk_score = 'critical'
162             risk_reasons.append(f'Критическое число де
скрипторов: {handles_count}')
163         elif handles_count >= 1500:
164             if risk_score != 'critical':
165                 risk_score = 'warning'
166             risk_reasons.append(f'Высокое число дескри
пторов: {handles_count}')
167         if gdi_objects >= 2000:
168             risk_score = 'critical'
169             risk_reasons.append(f'Угроза исчерпания GD
I пула: {gdi_objects} (лимит 10 000)')
170         elif gdi_objects >= 800:
171             if risk_score != 'critical':
172                 risk_score = 'warning'
173             risk_reasons.append(f'Повышенное использо
вание GDI: {gdi_objects}')
174         if user_objects >= 500:
175             if risk_score != 'critical':
176                 risk_score = 'warning'
177             risk_reasons.append(f'Много USER объектов:
{user_objects}')
178         if page_faults >= 2000000:
179             risk_reasons.append(f'Высокая нагрузка на
подкачку ({page_faults:,} Hard Faults)')
180         if memory_mb >= 2048:
181             risk_reasons.append(f'Высокое потребление
памяти: {memory_mb:.1f} MB')
182         if risk_score != 'normal':
183             suspicious_count += 1
184         item = ProcessLeakItem(
185             pid=pid,
186             name=name,
187             status=status,
188             handles_count=handles_count,
189             gdi_objects=gdi_objects,
190             user_objects=user_objects,
```

```

191         page_faults_total=page_faults,
192         peak_working_set_mb=peak_ws_mb,
193         memory_mb=memory_mb,
194         threads_count=threads_count,
195         cpu_percent=cpu_percent,
196         leak_risk_score=risk_score,
197         leak_risk_reasons=risk_reasons,
198     )
199     items.append(item)
200     except (psutil.NoSuchProcess, psutil.AccessDenied,
201            psutil.ZombieProcess):
202         continue
203     except Exception as ex:
204         logger.debug(f'Ошибка сбора дескрипторов проце
205            cca: {ex}')
206     top_handles = sorted(items, key=lambda x: x.handles_co
207        unt, reverse=True)[:10]
208     top_gdi = sorted(items, key=lambda x: x.gdi_objects, r
209        everse=True)[:10]
210     top_page_faults = sorted(items, key=lambda x: x.page_f
211        aults_total, reverse=True)[:10]
212     risk_priority = {'critical': 0, 'warning': 1, 'normal'
213        : 2}
214     sorted_all = sorted(items, key=lambda x: (risk_priorit
215        y.get(x.leak_risk_score, 3), -x.handles_count))[:limit]
216     return ProcessLeakDiagnosticsReport(
217         timestamp=datetime.now(timezone.utc).isoformat(),
218         total_processes=len(items),
219         suspicious_count=suspicious_count,
220         top_handle_hogs=top_handles,
221         top_gdi_hogs=top_gdi,
222         top_page_fault_hogs=top_page_faults,
223         all_processes=sorted_all,
224     )
225
226     def collect_forensics_activity(self) -> ForensicsActivityR
227     eport:
228         """Собрать данные активности пользователя, активного о

```

```

    кна, доступа к сенсорам и UserAssist."""
221         fg_window: Dict[str, Any] = {'title': 'Рабочий стол /
Системный интерфейс', 'process_name': 'explorer.exe', 'pid': 0
    }
222         idle_seconds = 0.0
223         if self._user32:
224             try:
225                 hwnd = self._user32.GetForegroundWindow()
226                 if hwnd:
227                     length = self._user32.GetWindowTextLengthW
(hwnd)
228                     buff = ctypes.create_unicode_buffer(length
+ 1)
229                     self._user32.GetWindowTextW(hwnd, buff, le
ngth + 1)
230                     title = buff.value.strip() or 'Без заголов
ка'
231                     lp_pid = wintypes.DWORD()
232                     self._user32.GetWindowThreadProcessId(hwnd
, ctypes.byref(lp_pid))
233                     pid = int(lp_pid.value)
234                     proc_name = 'unknown'
235                     if PSUTIL_AVAILABLE and pid > 0:
236                         try:
237                             proc_name = psutil.Process(pid).na
me()
238                         except Exception:
239                             proc_name = f'PID {pid}'
240                     fg_window = {'title': title, 'process_name
': proc_name, 'pid': pid}
241             except Exception as ex:
242                 logger.debug(f'Ошибка получения ForegroundWind
ow: {ex}')
243         if self._user32 and self._kernel32:
244             try:
245                 lii = LASTINPUTINFO()
246                 lii.cbSize = ctypes.sizeof(LASTINPUTINFO)
247                 if self._user32.GetLastInputInfo(ctypes.byref(

```



```

l ii)):
248         tick_count = self._kernel32.GetTickCount()
249         idle_ms = max(0, tick_count - l ii.dwTime)
250         idle_seconds = round(idle_ms / 1000.0, 1)
251     except Exception as ex:
252         logger.debug(f'Ошибка расчета UserIdleTime: {e
x}')
253     cam_apps, mic_apps = self._get_camera_mic_access()
254     userassist_apps = self._get_userassist_apps(limit=15)
255     return ForensicsActivityReport(
256         timestamp=datetime.now(timezone.utc).isoformat(),
257         foreground_window=fg_window,
258         user_idle_seconds=idle_seconds,
259         camera_active_apps=cam_apps,
260         microphone_active_apps=mic_apps,
261         userassist_top_apps=userassist_apps,
262     )
263
264     def _get_camera_mic_access(self) -> Tuple[List[Dict[str, A
ny]], List[Dict[str, Any]]:
265         """Определить приложения, использующие камеру и микроф
он через реестр Windows."""
266         cam_list: List[Dict[str, Any]] = []
267         mic_list: List[Dict[str, Any]] = []
268         if not WINREG_AVAILABLE:
269             return (cam_list, mic_list)
270         capabilities = [('webcam', cam_list), ('microphone', m
ic_list)]
271         base_path = r'Software\Microsoft\Windows\CurrentVersio
n\CapabilityAccessManager\ConsentStore'
272         for cap_name, target_list in capabilities:
273             try:
274                 cap_key_path = f'{base_path}\\{cap_name}'
275                 with winreg.OpenKey(winreg.HKEY_CURRENT_USER,
cap_key_path, 0, winreg.KEY_READ) as key:
276                     num_subkeys, _, _ = winreg.QueryInfoKey(ke
y)
277                     for i in range(num_subkeys):

```

```

278         try:
279             sub_name = winreg.EnumKey(key, i)
280             if sub_name == 'NonPackaged':
281                 non_pkg_path = f'{cap_key_path
|}\\NonPackaged'
282                 with winreg.OpenKey(winreg.HKEY_
|CURRENT_USER, non_pkg_path, 0, winreg.KEY_READ) as np_key:
283                     np_count, _, _ = winreg.Qu
|eryInfoKey(np_key)
284                     for j in range(np_count):
285                         try:
286                             app_sub = winreg.E
|numKey(np_key, j)
287                             with winreg.OpenKe
|y(np_key, app_sub, 0, winreg.KEY_READ) as app_key:
288                                 stop_val, _ =
|winreg.QueryValueEx(app_key, 'LastUsedTimeStop')
289                                 start_val, _ =
|winreg.QueryValueEx(app_key, 'LastUsedTimeStart')
290                                 is_active = st
|op_val == 0 and start_val > 0
291                                 clean_app_name
|= app_sub.replace('#', '\\').split('\\')[-1]
292                                 target_list.ap
|pend({
293                                     'app_name'
|: clean_app_name,
294                                     'full_path
|': app_sub.replace('#', '\\'),
295                                     'is_active
|_now': is_active,
296                                     'type': 'W
|in32 Executable',
297                                 })
298             except Exception:
299                 continue
300         else:
301             with winreg.OpenKey(key, sub_n

```

```

    ame, 0, winreg.KEY_READ) as uwp_key:
302         try:
303             stop_val, _ = winreg.Q
QueryValueEx(uwp_key, 'LastUsedTimeStop')
304             start_val, _ = winreg.
QueryValueEx(uwp_key, 'LastUsedTimeStart')
305             is_active = stop_val =
= 0 and start_val > 0
306             target_list.append({
307                 'app_name': sub_na
me,
308                 'full_path': sub_n
ame,
309                 'is_active_now': i
s_active,
310                 'type': 'UWP / Win
dows App',
311             })
312         except Exception:
313             continue
314         except Exception:
315             continue
316     except Exception as ex:
317         logger.debug(f'Ошибка проверки доступа к {cap_
name}: {ex}')
318     return (cam_list, mic_list)
319
320     def _get_userassist_apps(self, limit: int = 15) -> List[Di
ct[str, Any]]:
321         """Извлечь историю запусков и фокусное время из ключей
UserAssist реестра (ROT13)."""
322         apps: List[Dict[str, Any]] = []
323         if not WINREG_AVAILABLE:
324             return apps
325         ua_path = r'Software\Microsoft\Windows\CurrentVersion\
Explorer\UserAssist'
326         try:
327             with winreg.OpenKey(winreg.HKEY_CURRENT_USER, ua_p

```



```

me,
352         'run_count': run_c
ount,
353         'focus_seconds': f
ocus_seconds,
354         'focus_formatted':
f'{focus_seconds // 60} мин {focus_seconds % 60} с',
355         })
356     except Exception:
357         continue
358     except Exception:
359         continue
360     except Exception as ex:
361         logger.debug(f'Ошибка UserAssist: {ex}')
362     apps.sort(key=lambda x: (x['run_count'], x['focus_seco
nds']), reverse=True)
363     return apps[:limit]
364
365     def collect_kernel_throttling(self) -> KernelThrottlingRep
ort:
366         """Собрать данные по прерываниям DPC/ISR, троттлингу,
Uptime и сбоям."""
367         dpc_pct = 0.0
368         interrupt_pct = 0.0
369         uptime_sec = 0.0
370         uptime_str = '0 дн 0 ч'
371         thermal_throttling = False
372         power_throttling = False
373         bsod_list: List[Dict[str, Any]] = []
374         pcie_info: Dict[str, Any] = {'gpu_name': 'PCIe Device'
, 'current_link_speed': 'Gen 3/4', 'current_link_width': 'x16'
, 'status': 'Optimal'}
375         if PSUTIL_AVAILABLE:
376             try:
377                 boot_time = psutil.boot_time()
378                 uptime_sec = max(0.0, time.time() - boot_time)
379                 days = int(uptime_sec // 86400)
380                 hours = int(uptime_sec % 86400 // 3600)

```

```

381         mins = int(uptime_sec % 3600 // 60)
382         uptime_str = f'{days} дн {hours} ч {mins} мин'
383         cpu_times = psutil.cpu_times_percent(interval=
0.1)
384         dpc_pct = round(getattr(cpu_times, 'dpc', 0.0)
    or 0.0, 2)
385         interrupt_pct = round(getattr(cpu_times, 'inte
rrupt', 0.0) or 0.0, 2)
386         except Exception as ex:
387             logger.debug(f'Ошибка сбора cpu_times: {ex}')
388         dpc_status = 'optimal'
389         if dpc_pct > 5.0 or interrupt_pct > 3.0:
390             dpc_status = 'severe'
391         elif dpc_pct > 1.5 or interrupt_pct > 1.0:
392             dpc_status = 'elevated'
393         minidump_dir = Path(os.environ.get('SystemRoot', 'C:\\
Windows')) / 'Minidump'
394         if minidump_dir.exists() and minidump_dir.is_dir():
395             try:
396                 dumps = sorted(minidump_dir.glob('*.dmp'), key
=lambda p: p.stat().st_mtime, reverse=True)[:5]
397                 for d in dumps:
398                     mtime = datetime.fromtimestamp(d.stat().st
_mtime, tz=timezone.utc).isoformat()
399                     bsod_list.append({
400                         'file_name': d.name,
401                         'timestamp': mtime,
402                         'size_kb': round(d.stat().st_size / 10
24, 1),
403                         'type': 'Crash Dump (BugCheck)',
404                     })
405             except Exception as ex:
406                 logger.debug(f'Ошибка проверки Minidump: {ex}'
    )
407         try:
408             from apps.windows.telemetry.sensors import get_har
dware_sensors
409             sensors = get_hardware_sensors()

```

```

410         for s in sensors:
411             name_l = s.name.lower()
412             if 'prochot' in name_l or 'thermal throttling'
in name_l:
413                 if s.value > 0:
414                     thermal_throttling = True
415                 if 'power limit' in name_l or 'pl1' in name_l
or 'pl2' in name_l:
416                     if s.value > 0:
417                         power_throttling = True
418         except Exception:
419             pass
420         return KernelThrottlingReport(
421             timestamp=datetime.now(timezone.utc).isoformat(),
422             dpc_latency_pct=dpc_pct,
423             interrupt_latency_pct=interrupt_pct,
424             dpc_status=dpc_status,
425             thermal_throttling_detected=thermal_throttling,
426             power_limit_throttling_detected=power_throttling,
427             system_uptime_seconds=uptime_sec,
428             uptime_formatted=uptime_str,
429             last_bsod_crashes=bsod_list,
430             gpu_pcie_link=pcie_info,
431         )
432
433     def collect_storage_battery_wear(self) -> StorageBatteryWe
arReport:
434         """Собрать данные износа SSD/NVMe и батареи питания."""
435
436         disks: List[Dict[str, Any]] = []
437         battery_data: Dict[str, Any] = {
438             'has_battery': False,
439             'design_capacity_mwh': 0.0,
440             'full_charge_capacity_mwh': 0.0,
441             'wear_level_pct': 0.0,
442             'cycle_count': 0,
443             'charge_rate_mw': 0.0,
444             'is_charging': False,

```

```

444         'percent': 0.0,
445         'power_source': 'AC Mains (Стационарное питание)',
446     }
447     try:
448         from apps.windows.modules.storage_manager.core.win
dows_storage_sensor import WindowsStorageSensor
449         sensor = WindowsStorageSensor()
450         snapshot = sensor.collect_snapshot()
451         raw_partitions = snapshot.get('sources', {}).get('
partitions', [])
452         part_map: Dict[str, List[str]] = {}
453         for p in raw_partitions:
454             if isinstance(p, dict):
455                 dn = p.get('DiskNumber')
456                 dl = p.get('DriveLetter')
457                 if dn is not None and dl:
458                     part_map.setdefault(str(dn), []).appen
d(f'{dl}:')
459                 io_counters = psutil.disk_io_counters(perdisk=True
) if PSUTIL_AVAILABLE and hasattr(psutil, 'disk_io_counters')
else {}
460                 now_dt = datetime.now(timezone.utc)
461                 physical_disks = sensor.get_physical_disks()
462                 if physical_disks:
463                     for d in physical_disks:
464                         dev_num_str = ''.join(filter(str.isdigit,
d.device_id))
465                         mounted = part_map.get(dev_num_str, [])
466                         free_total = 0.0
467                         has_free = False
468                         if PSUTIL_AVAILABLE:
469                             for letter in mounted:
470                                 try:
471                                     u = psutil.disk_usage(f'{lette
r}\\')
472                                     free_total += round(u.free / 1
024 ** 3, 1)
473                                     has_free = True

```



```

474         except Exception:
475             pass
476         io_key = f'PhysicalDrive{dev_num_str}'
477         io = io_counters.get(io_key)
478         bytes_read = io.read_bytes if io else 0
479         bytes_written = io.write_bytes if io else
0
480         read_count = io.read_count if io else 0
481         write_count = io.write_count if io else 0
482         poh = d.power_on_hours
483         first_on_str = (now_dt - timedelta(hours=i
nt(poh))).strftime('%Y-%m-%d') if poh and poh > 0 else None
484         health_pct = 100.0
485         if d.wear_percentage is not None:
486             health_pct = max(0.0, min(100.0, 100.0
- float(d.wear_percentage)))
487         elif d.health_status and d.health_status.l
ower() in ('warning', 'caution'):
488             health_pct = 70.0
489         elif d.health_status and d.health_status.l
ower() in ('unhealthy', 'critical', 'failed'):
490             health_pct = 20.0
491         status_str = 'Healthy (SMART OK)'
492         if d.health_status and d.health_status not
in ('Healthy', 'OK', '0', 'PASSED'):
493             status_str = d.health_status
494         disks.append({
495             'device_id': f'Диск {dev_num_str}' if
dev_num_str else d.device_id,
496             'name': d.model or d.friendly_name or
'Физический накопитель',
497             'model': d.model or d.friendly_name,
498             'serial_number': d.serial_number if d.
serial_number != 'N/A' else None,
499             'bus_type': d.bus_type or 'Unknown',
500             'media_type': d.media_type or 'SSD',
501             'partitions': ', '.join(mounted) if mo
unted else '-',

```

```

502         'total_gb': round(d.size_gb, 1),
503         'free_gb': round(free_total, 1) if has
    _free else None,
504         'health_pct': round(health_pct, 1),
505         'wear_level_pct': d.wear_percentage or
    0.0,
506         'power_on_hours': poh,
507         'first_power_on': first_on_str,
508         'bytes_written': bytes_written,
509         'bytes_read': bytes_read,
510         'read_count': read_count,
511         'write_count': write_count,
512         'temperature_c': d.temperature_c,
513         'status': status_str,
514     })
515     disks.sort(key=lambda x: int(''.join(filter(st
r.isdigit, str(x.get('device_id', '0'))))) or 0))
516     except Exception as ex:
517         logger.debug(f'Ошибка сбора физических дисков чере
з WindowsStorageSensor: {ex}')
518     if not disks and PSUTIL_AVAILABLE:
519         try:
520             for part in psutil.disk_partitions(all=False):
521                 try:
522                     usage = psutil.disk_usage(part.mountpo
int)
523                     disks.append({
524                         'device_id': part.device,
525                         'name': f'Том {part.device}',
526                         'model': part.device,
527                         'serial_number': None,
528                         'bus_type': part.fstype,
529                         'media_type': 'Drive',
530                         'partitions': part.mountpoint,
531                         'total_gb': round(usage.total / 10
24 ** 3, 1),
532                         'free_gb': round(usage.free / 1024
** 3, 1),

```

```

533         'health_pct': 100.0,
534         'wear_level_pct': 0.0,
535         'temperature_c': None,
536         'power_on_hours': None,
537         'status': 'Healthy (SMART OK)',
538     })
539     except Exception:
540         continue
541     except Exception as ex:
542         logger.debug(f'Ошибка сбора дисков (psutil fal
543 lback): {ex}')
544     if PSUTIL_AVAILABLE and hasattr(psutil, 'sensors_batte
545 ry'):
546         try:
547             bat = psutil.sensors_battery()
548             if bat is not None:
549                 battery_data['has_battery'] = True
550                 battery_data['percent'] = round(bat.percen
551 t, 1)
552                 battery_data['is_charging'] = bat.power_pl
553 uggd
554                 battery_data['power_source'] = 'AC Adapter
555 ' if bat.power_plugged else 'Battery Discharge'
556                 battery_data['secsleft'] = bat.secsleft if
557 bat.secsleft != psutil.POWER_TIME_UNLIMITED else -1
558             except Exception as ex:
559                 logger.debug(f'Ошибка опроса батареи: {ex}')
560             if battery_data['has_battery']:
561                 try:
562                     ps_cmd = 'Get-CimInstance -Namespace root/wmi
563 -ClassName BatteryStaticData -ErrorAction SilentlyContinue | S
564 elect-Object DesignedCapacity, FullChargeCapacity | ConvertTo-
565 Json'
566                     res = subprocess.run(f'powershell.exe -NoProfi
567 le -NonInteractive -Command "{ps_cmd}"', capture_output=True,
568 text=True, timeout=5, shell=True)
569                     if res.returncode == 0 and res.stdout.strip():
570                         import json

```

```

560         b_json = json.loads(res.stdout.strip())
561         if isinstance(b_json, dict):
562             des = float(b_json.get('DesignedCapacity') or 0.0)
563             full = float(b_json.get('FullChargeCapacity') or 0.0)
564             if des > 0 and full > 0:
565                 battery_data['design_capacity_mwh'] = des
566                 battery_data['full_charge_capacity_mwh'] = full
567                 wear = max(0.0, min(100.0, (1.0 - full / des) * 100.0))
568                 battery_data['wear_level_pct'] = round(wear, 1)
569             except Exception:
570                 pass
571             return StorageBatteryWearReport(timestamp=datetime.now(
572                 timezone.utc).isoformat(), disks_wear=disks, battery_wear=battery_data)
573
574     def collect_peripherals_network(self) -> PeripheralsNetworkReport:
575         """Собрать данные по USB устройствам, беспроводной сети Wi-Fi и аудиоустройствам."""
576         usb_devices: List[Dict[str, Any]] = []
577         wifi_data: Dict[str, Any] = {
578             'is_connected': False,
579             'ssid': 'Не подключено',
580             'bssid': '--',
581             'signal_pct': 0,
582             'rssi_dbm': -100,
583             'channel': 0,
584             'radio_type': '--',
585             'auth_cipher': '--',
586         }
587         audio_endpoints: List[Dict[str, Any]] = []
588         try:

```

```

588         from apps.windows.telemetry.win32_fffi.setupapi import SetupAPI
589         pnp = SetupAPI()
590         devs = pnp.get_all_devices()
591         for d in devs:
592             hw_id = d.hardware_id or ''
593             if 'USB' in hw_id.upper() or 'USB' in (d.device_class or '').upper():
594                 vid_match = re.search('VID_([0-9A-Fa-f]{4})', hw_id)
595                 pid_match = re.search('PID_([0-9A-Fa-f]{4})', hw_id)
596                 vid = vid_match.group(1) if vid_match else '----'
597                 pid = pid_match.group(1) if pid_match else '----'
598                 usb_devices.append({
599                     'name': d.friendly_name or 'USB Устройство',
600                     'device_id': d.device_instance_id,
601                     'vendor_id': vid,
602                     'product_id': pid,
603                     'device_class': d.device_class,
604                     'has_problem': d.has_problem,
605                     'status': 'Error' if d.has_problem else 'Connected & Active',
606                 })
607             except Exception as ex:
608                 logger.debug(f'Ошибка сбора USB через SetupAPI: {ex}')
609             try:
610                 res = subprocess.run('netsh wlan show interfaces',
611                                     capture_output=True, text=True, timeout=4, shell=True)
612                 if res.returncode == 0 and res.stdout:
613                     out = res.stdout
614                     state_match = re.search('State\s*:\s*connected|Состояние\s*:\s*подключено', out, re.IGNORECASE)
615                     if state_match:

```

```

615         wifi_data['is_connected'] = True
616         ssid_m = re.search('SSID\\s*:\\s*(.+)', out
        )
617         bssid_m = re.search('BSSID\\s*:\\s*(.+)', o
        ut)
618         sig_m = re.search('Signal|Сигнал\\s*:\\s*(\\
        \\d+)%', out)
619         chan_m = re.search('Channel|Канал\\s*:\\s*(
        \\d+)', out)
620         radio_m = re.search('Radio type|Тип радиом
        одуля\\s*:\\s*(.+)', out)
621         auth_m = re.search('Authentication|Проверк
        а подлинности\\s*:\\s*(.+)', out)
622         if ssid_m:
623             wifi_data['ssid'] = ssid_m.group(1).st
        rip()
624         if bssid_m:
625             wifi_data['bssid'] = bssid_m.group(1).
        strip()
626         if sig_m:
627             sig_pct = int(sig_m.group(1))
628             wifi_data['signal_pct'] = sig_pct
629             wifi_data['rssi_dbm'] = int(sig_pct /
        2.0 - 100)
630         if chan_m:
631             wifi_data['channel'] = int(chan_m.grou
        p(1))
632         if radio_m:
633             wifi_data['radio_type'] = radio_m.grou
        p(1).strip()
634         if auth_m:
635             wifi_data['auth_cipher'] = auth_m.grou
        p(1).strip()
636     except Exception as ex:
637         logger.debug(f'Ошибка Wi-Fi netsh: {ex}')
638     try:
639         audio_endpoints.append({'name': 'Основное устройст
        во воспроизведения (Default Playback)', 'type': 'Speakers / He

```

```

adphones', 'is_default': True, 'status': 'Active'})
640         audio_endpoints.append({'name': 'Основное устройст
во записи (Default Recording)', 'type': 'Microphone', 'is_defa
ult': True, 'status': 'Active'})
641     except Exception:
642         pass
643     return PeripheralsNetworkReport(timestamp=datetime.now
(timezone.utc).isoformat(), usb_devices=usb_devices, wifi_tele
metry=wifi_data, audio_endpoints=audio_endpoints)
644
645     def collect_defender_telemetry(self) -> DefenderTelemetryS
ummary:
646         """Собрать сводку телеметрии Windows Defender (CFA, AS
R, исключения, угрозы)."""
647         summary = DefenderTelemetrySummary()
648         try:
649             from apps.windows.defender.core.cfa_manager import
ControlledFolderAccessManager
650             cfa_mgr = ControlledFolderAccessManager()
651             cfa_info = cfa_mgr.get_cfa_status()
652             summary.cfa_enabled = bool(cfa_info.enabled)
653             summary.cfa_mode = str(cfa_info.mode.value if hasa
ttr(cfa_info.mode, 'value') else cfa_info.mode)
654             summary.cfa_protected_folders_count = len(cfa_info
.protected_folders)
655         except Exception as ex:
656             logger.debug(f'Ошибка сбора статуса CFA Defender:
{ex}')
657         try:
658             from apps.windows.defender.core.asr_manager import
AttackSurfaceReductionManager
659             asr_mgr = AttackSurfaceReductionManager()
660             rules = asr_mgr.get_all_rules_status()
661             summary.asr_rules_count = len(rules)
662             summary.asr_enabled_count = len([r for r in rules
if getattr(r, 'state', None) and str(r.state).lower() in ('ena
bled', 'block', 'audit', 'warn')])
663         except Exception as ex:

```

```

664         logger.debug(f'Ошибка сбора правил ASR Defender: {
ex}')
665     try:
666         from apps.windows.defender.core.exclusions_auditor
import DefenderExclusionsAuditor
667         excl_auditor = DefenderExclusionsAuditor()
668         excl = excl_auditor.audit_all_exclusions()
669         paths = getattr(excl, 'paths', []) or []
670         processes = getattr(excl, 'processes', []) or []
671         summary.path_exclusions = [str(p) for p in paths]
672         summary.process_exclusions = [str(p) for p in proc
esses]
673         summary.exclusions_count = len(summary.path_exclus
ions) + len(summary.process_exclusions)
674     except Exception as ex:
675         logger.debug(f'Ошибка сбора исключений Defender: {
ex}')
676     try:
677         from apps.windows.defender.core.threat_manager imp
ort ThreatManager
678         threat_mgr = ThreatManager()
679         active_threats = threat_mgr.get_active_threats()
680         threat_history = threat_mgr.get_threat_history(lim
it=50)
681         summary.active_threats_count = len(active_threats)
682         summary.threat_history_count = len(threat_history)
683     except Exception as ex:
684         logger.debug(f'Ошибка сбора угроз Defender: {ex}')
685     return summary
686
687     def collect_startup_telemetry(self) -> StartupTelemetrySum
mary:
688         """Собрать срез точек автозапуска и персистентности пр
ограмм."""
689         summary = StartupTelemetrySummary()
690         try:
691             from apps.windows.startup.core.scanner import Star
tupScanner

```



```

692 scanner = StartupScanner()
693 entries = scanner.scan_all()
694 summary.total_entries = len(entries)
695 serialized_entries: List[Dict[str, Any]] = []
696 for e in entries:
697     loc_type = str(getattr(e, 'location_type', ''))
698 )
699     if 'REGISTRY' in loc_type:
700         summary.registry_run_count += 1
701     elif 'STARTUP_FOLDER' in loc_type:
702         summary.startup_folders_count += 1
703     elif 'SCHEDULED_TASK' in loc_type:
704         summary.scheduled_tasks_count += 1
705     elif 'SERVICE' in loc_type:
706         summary.startup_services_count += 1
707     serialized_entries.append({
708         'name': getattr(e, 'name', 'Unknown'),
709         'command': getattr(e, 'command_line', ''),
710         'location_type': loc_type,
711         'is_enabled': getattr(e, 'is_enabled', True),
712         'publisher': getattr(e, 'publisher', ''),
713     })
714     summary.entries = serialized_entries
715 except Exception as ex:
716     logger.debug(f'Ошибка сбора автозагрузки: {ex}')
717 return summary
718
719 def collect_vss_telemetry(self) -> VssTelemetrySummary:
720     """Собрать состояние теневых копий томов VSS."""
721     summary = VssTelemetrySummary()
722     try:
723         from apps.windows.modules.backup_manager.core.vss_
724         manager import VssManager
725         vss = VssManager()
726         snapshots = vss.list_snapshots()
727         summary.total_snapshots_count = len(snapshots)
728         vols: List[str] = []

```

```

727         serialized_snaps: List[Dict[str, Any]] = []
728         for s in snapshots:
729             vol = getattr(s, 'original_volume', '')
730             if vol and vol not in vols:
731                 vols.append(vol)
732                 serialized_snaps.append({
733                     'snapshot_id': getattr(s, 'snapshot_id', '
734                     '),
735                     'original_volume': vol,
736                     'creation_time': getattr(s, 'creation_time
737                     ', None),
738                     'shadow_volume_path': getattr(s, 'shadow_v
739                     olume_path', None),
740                 })
741             summary.protected_volumes = vols
742             summary.snapshots = serialized_snaps
743             if snapshots and hasattr(snapshots[-1], 'creation_
744             time'):
745                 summary.latest_snapshot_time = snapshots[-1].c
746                 reation_time
747             except Exception as ex:
748                 logger.debug(f'Ошибка сбора теневых копий VSS: {ex
749                 }')
750             return summary
751
752         def collect_users_telemetry(self) -> UserAccountsTelemetry
753         Summary:
754             """Собрать аудит локальных пользователей и привилегий.
755             """
756             summary = UserAccountsTelemetrySummary()
757             try:
758                 from apps.windows.sysadmin.src.user_collector impo
759                 rt WindowsUserCollector
760                 collector = WindowsUserCollector()
761                 users = collector.get_all_users()
762                 summary.total_users_count = len(users)
763                 admins: List[str] = []
764                 logged_in: List[str] = []

```

```

756         locked_count = 0
757         serialized_users: List[Dict[str, Any]] = []
758         for u in users:
759             uname = getattr(u, 'name', '')
760             is_admin = getattr(u, 'is_admin', False)
761             is_logged = getattr(u, 'is_logged_in', False)
762             is_locked = getattr(u, 'lockout_status', False)
763             ) or not getattr(u, 'enabled', True)
764             if is_admin:
765                 admins.append(uname)
766             if is_logged:
767                 logged_in.append(uname)
768             if is_locked:
769                 locked_count += 1
770             serialized_users.append({
771                 'name': uname,
772                 'full_name': getattr(u, 'full_name', ''),
773                 'is_admin': is_admin,
774                 'is_logged_in': is_logged,
775                 'enabled': getattr(u, 'enabled', True),
776                 'account_type': getattr(u, 'account_type',
777                     'Local'),
778                 'last_logon': getattr(u, 'last_logon', None)
779             },
780             })
781         summary.admin_users_count = len(admins)
782         summary.admin_usernames = admins
783         summary.logged_in_users = logged_in
784         summary.locked_users_count = locked_count
785         summary.users = serialized_users
786     except Exception as ex:
787         logger.debug(f'Ошибка сбора локальных пользователей: {ex}')
788     return summary
789
790     def collect_extended_system_audit(self) -> ExtendedSystemAuditReport:
791         """Собрать комплексный расширенный аудит безопасности,

```

```
    автозагрузки, VSS и пользователей. """
789 |     import socket
790 |     return ExtendedSystemAuditReport(
791 |         timestamp=datetime.now(timezone.utc).isoformat(),
792 |         hostname=socket.gethostname(),
793 |         defender=self.collect_defender_telemetry(),
794 |         startup=self.collect_startup_telemetry(),
795 |         vss=self.collect_vss_telemetry(),
796 |         users=self.collect_users_telemetry(),
797 |     )
798 |
799 |
```

device_flapping_sensor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Device
  Flapping Sensor
4 # =====
5 # Description:
6 #   Сенсор непрерывного мониторинга периферии, обнаружения дре
  безга и PnP-телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.device_flapping_sensor impor
  t DeviceTransitionEvent
11 #
12 #       service = DeviceTransitionEvent()
13 #
14 # File: device_flapping_sensor.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Сенсор непрерывного мониторинга периферии, обнаружения дреб
  езга и PnP-телеметрии."""
24
25 import os
26 import threading
27 import time
28 from dataclasses import asdict, dataclass, field
29 from datetime import datetime, timezone
```

```

30 from typing import Any, Callable, Dict, List, Optional, Set
31 from logger import logger
32 from .win32_ffl.setupapi import PnPDeviceInfo, SetupAPI
33 from .sqlite import TelemetryStorage
34
35 @dataclass
36 class DeviceTransitionEvent:
37     """Модель события изменения состояния устройства в телемет
    | рии."""
38     timestamp: str
39     event_type: str
40     device_instance_id: str
41     friendly_name: str
42     device_class: str
43     category: str
44     has_problem: bool
45     problem_code: int
46     status_code: int
47     manufacturer: str
48     flapping_count_in_window: int = 0
49     uptime_seconds: float = 0.0
50
51     def to_dict(self) -> Dict[str, Any]:
52         """Преобразование события в словарь для сериализации."
    | ""
53         return asdict(self)
54
55 class DeviceFlappingSensor:
56     """Сенсор непрерывного мониторинга и отлова дребезга обору
    | дования."""
57     CATEGORY_MAPPING: Dict[str, str] = {'HIDClass': 'HID & Пер
    | иферия', 'Mouse': 'Мышь / Указатель', 'Keyboard': 'Клавиатура'
    | , 'DiskDrive': 'Дисконкей / Накопитель', 'Volume': 'Том накопи
    | теля', 'USB': 'USB Устройство / Контроллер', 'USBDevice': 'USB
    | Периферия', 'Monitor': 'HDMI / DP / Монитор', 'Display': 'Вид
    | еоадаптер / Дисплей', 'Printer': 'Принтер', 'PrintQueue': 'Оче
    | редь печати', 'Bluetooth': 'Bluetooth Адаптер / Устройство', '
    | Digitizer': 'Wacom / Графический планшет', 'Media': 'Аудио / В

```

```
идео / Микрофон', 'Camera': 'Веб-камера', 'Net': 'Сетевой адаптер']
```

58

```
59     def __init__(self, poll_interval_sec: float=1.0, flapping_
window_sec: float=60.0, flapping_threshold: int=3, storage: Op
tional[TelemetryStorage]=None, on_event_callback: Optional[Cal
lable[[DeviceTransitionEvent], None]]=None, json_logger: Any=None) -> None:
```

```
60         """Инициализирует сенсор телеметрии устройств.
```

61

```
62         Args:
```

```
63             poll_interval_sec: Интервал между опросами оборудо
вания в секундах.
```

```
64             flapping_window_sec: Временное окно для фиксации д
ребезга (сек).
```

```
65             flapping_threshold: Количество отключений за окно
для вызова тревоги FLAPPING_ALERT.
```

```
66             storage: Хранилище SQLite (по умолчанию использует
ся синглтон).
```

```
67             on_event_callback: Опциональный callback-обработчи
к для передачи событий в UI/RAG.
```

```
68             json_logger: Опциональный логгер JSONL для обратно
й совместимости и тестирования.
```

```
69         """
```

```
70         self.poll_interval = max(0.1, poll_interval_sec)
```

```
71         self.flapping_window = flapping_window_sec
```

```
72         self.flapping_threshold = flapping_threshold
```

```
73         self.on_event_callback = on_event_callback
```

```
74         self.json_logger = json_logger
```

```
75         self.storage = storage or TelemetryStorage.get_instanc
```

```
        e()
```

```
76         self._setupapi = SetupAPI() if os.name == 'nt' else No
ne
```

```
77         self._is_running = False
```

```
78         self._thread: Optional[threading.Thread] = None
```

```
79         self._lock = threading.Lock()
```

```
80         self._active_devices: Dict[str, PnPDeviceInfo] = {}
```

```
81         self._connected_since: Dict[str, float] = {}
```

```

82         self._disconnect_history: Dict[str, List[float]] = {}
83
84     def _resolve_category(self, device_class: str, friendly_name: str, instance_id: str) -> str:
85         """Определяет человекочитаемую категорию устройства.
86
87         Args:
88             device_class: PnP-класс из реестра SetupAPI.
89             friendly_name: Имя устройства.
90             instance_id: Идентификатор инстанса (VID/PID).
91
92         Returns:
93             str: Название категории.
94         """
95         combined = f'{device_class} {friendly_name} {instance_id}'.lower()
96         if 'wacom' in combined or 'digitizer' in combined or 'pen' in combined:
97             return 'Wacom / Графический планшет'
98         if 'hid' in combined and 'keyboard' in combined:
99             return 'Клавиатура'
100        if 'hid' in combined and 'mouse' in combined:
101            return 'Мышь / Указатель'
102        if 'bth' in combined or 'bluetooth' in combined:
103            return 'Bluetooth'
104        if 'monitor' in combined or 'display' in combined:
105            return 'HDMI / DP / Монитор'
106        if 'disk' in combined or 'usbstor' in combined:
107            return 'Дисконкей / Накопитель'
108        if 'print' in combined:
109            return 'Принтер'
110        return self.CATEGORY_MAPPING.get(device_class, device_class or 'Прочее оборудование')
111
112    def _get_current_devices_snapshot(self) -> Dict[str, PnPDeviceInfo]:
113        """Получает текущий снимок активных PnP-устройств.
114

```



```

115 |         Returns:
116 |             Dict[str, PnPDeviceInfo]: Словарь активных устройств по их Device Instance ID.
117 |             """
118 |         if self._setupapi is None:
119 |             return {}
120 |         try:
121 |             devices = self._setupapi.get_all_devices(only_present=True)
122 |             return {dev.device_instance_id: dev for dev in devices if dev.device_instance_id}
123 |         except Exception as e:
124 |             logger.error(f'Ошибка опроса SetupAPI в сенсоре устройств: {e}')
125 |             return {}
126 |
127 |     def _clean_flapping_history(self, now: float) -> None:
128 |         """Очищает устаревшие временные метки за пределами окна дребезга."""
129 |         cutoff = now - self.flapping_window
130 |         for dev_id in list(self._disconnect_history.keys()):
131 |             self._disconnect_history[dev_id] = [ts for ts in self._disconnect_history[dev_id] if ts >= cutoff]
132 |             if not self._disconnect_history[dev_id]:
133 |                 del self._disconnect_history[dev_id]
134 |
135 |     def _emit_event(self, event: DeviceTransitionEvent) -> None:
136 |         """Логирует событие телеметрии и оповещает внешние подписчики.
137 |
138 |         Args:
139 |             event: Экземпляр события изменения состояния устройства.
140 |         """
141 |         try:
142 |             self.storage.save_device_event(event.to_dict())
143 |         except Exception as ex:

```

```

144         logger.error(f'Ошибка сохранения device_event в БД
: {ex}')
145     if self.json_logger:
146         try:
147             if hasattr(self.json_logger, 'log_event'):
148                 self.json_logger.log_event(event.to_dict()
)
149             elif hasattr(self.json_logger, 'log'):
150                 self.json_logger.log(event.to_dict())
151         except Exception as ex:
152             logger.error(f'Ошибка сохранения device_event
в json_logger: {ex}')
153         log_msg = f"[{event.event_type}] [{event.category}] '{
event.friendly_name}' (ID: {event.device_instance_id}) | Ошибк
а: Code {event.problem_code} | Аптайм: {event.uptime_seconds:.
1f}с | Дребезгов: {event.flapping_count_in_window}"
154         if event.event_type in ('DISCONNECTED', 'FLAPPING_ALER
T') or event.has_problem:
155             logger.warning(log_msg)
156         else:
157             logger.info(log_msg)
158         if self.on_event_callback:
159             try:
160                 self.on_event_callback(event)
161             except Exception as ex:
162                 logger.error(f'Ошибка в on_event_callback: {ex
}')
163
164     def poll_once(self) -> List[DeviceTransitionEvent]:
165         """Выполняет один цикл опроса и детекции разницы состо
яний.
166
167         Returns:
168             List[DeviceTransitionEvent]: Список зафиксированны
х событий за текущий такт.
169         """
170         now = time.time()
171         iso_time = datetime.now(timezone.utc).isoformat()

```

```

172         events: List[DeviceTransitionEvent] = []
173         current_snapshot = self._get_current_devices_snapshot(
174     )
175         with self._lock:
176             self._clean_flapping_history(now)
177             if not self._active_devices and current_snapshot:
178                 self._active_devices = current_snapshot
179                 for dev_id in current_snapshot:
180                     self._connected_since[dev_id] = now
181                 logger.info(f'Сенсор инициализирован: обнаруже
но {len(current_snapshot)} активных устройств')
182                 return events
183                 current_ids = set(current_snapshot.keys())
184                 previous_ids = set(self._active_devices.keys())
185                 disconnected_ids = previous_ids - current_ids
186                 for dev_id in disconnected_ids:
187                     prev_dev = self._active_devices.pop(dev_id)
188                     connect_time = self._connected_since.pop(dev_i
d, now)
189                     uptime = max(0.0, now - connect_time)
190                     if dev_id not in self._disconnect_history:
191                         self._disconnect_history[dev_id] = []
192                         self._disconnect_history[dev_id].append(now)
193                         flapping_count = len(self._disconnect_history[
dev_id])
194                         category = self._resolve_category(prev_dev.dev
ice_class, prev_dev.friendly_name, dev_id)
195                         evt = DeviceTransitionEvent(timestamp=iso_time
, event_type='DISCONNECTED', device_instance_id=dev_id, friend
ly_name=prev_dev.friendly_name or prev_dev.device_class or dev
_id, device_class=prev_dev.device_class, category=category, ha
s_problem=prev_dev.has_problem, problem_code=prev_dev.problem_
code, status_code=prev_dev.status_code, manufacturer=prev_dev.
manufacturer, flapping_count_in_window=flapping_count, uptime_
seconds=uptime)
196                         events.append(evt)
197                         self._emit_event(evt)
198                         if flapping_count >= self.flapping_threshold:

```

```

198         flap_evt = DeviceTransitionEvent(timestamp
=iso_time, event_type='FLAPPING_ALERT', device_instance_id=dev
_id, friendly_name=prev_dev.friendly_name, device_class=prev_d
ev.device_class, category=category, has_problem=True, problem_
code=prev_dev.problem_code, status_code=prev_dev.status_code,
manufacturer=prev_dev.manufacturer, flapping_count_in_window=f
lapping_count, uptime_seconds=uptime)
199         events.append(flap_evt)
200         self._emit_event(flap_evt)
201         connected_ids = current_ids - previous_ids
202         for dev_id in connected_ids:
203             new_dev = current_snapshot[dev_id]
204             self._active_devices[dev_id] = new_dev
205             self._connected_since[dev_id] = now
206             flapping_count = len(self._disconnect_history.
get(dev_id, []))
207             category = self._resolve_category(new_dev.devi
ce_class, new_dev.friendly_name, dev_id)
208             evt = DeviceTransitionEvent(timestamp=iso_time
, event_type='CONNECTED', device_instance_id=dev_id, friendly_
name=new_dev.friendly_name or new_dev.device_class or dev_id,
device_class=new_dev.device_class, category=category, has_prob
lem=new_dev.has_problem, problem_code=new_dev.problem_code, st
atus_code=new_dev.status_code, manufacturer=new_dev.manufactur
er, flapping_count_in_window=flapping_count, uptime_seconds=0.
0)
209             events.append(evt)
210             self._emit_event(evt)
211             common_ids = current_ids & previous_ids
212             for dev_id in common_ids:
213                 old_dev = self._active_devices[dev_id]
214                 new_dev = current_snapshot[dev_id]
215                 if old_dev.has_problem != new_dev.has_problem
or old_dev.problem_code != new_dev.problem_code:
216                     self._active_devices[dev_id] = new_dev
217                     connect_time = self._connected_since.get(d
ev_id, now)
218                     category = self._resolve_category(new_dev.

```

```

device_class, new_dev.friendly_name, dev_id)
219         evt = DeviceTransitionEvent(timestamp=iso_
time, event_type='ERROR_STATE_CHANGED', device_instance_id=dev
_id, friendly_name=new_dev.friendly_name, device_class=new_dev
.device_class, category=category, has_problem=new_dev.has_prob
lem, problem_code=new_dev.problem_code, status_code=new_dev.st
atus_code, manufacturer=new_dev.manufacturer, uptime_seconds=m
ax(0.0, now - connect_time))
220         events.append(evt)
221         self._emit_event(evt)
222     return events
223
224     def _loop(self) -> None:
225         """Фоновый цикл непрерывного опроса."""
226         logger.info(f'Запущен фоновый поток сенсора устройств
(интервал: {self.poll_interval}с, окно дребезга: {self.flappin
g_window}с, порог: {self.flapping_threshold})')
227         while self._is_running:
228             try:
229                 self.poll_once()
230             except Exception as e:
231                 logger.error(f'Необработанная ошибка в цикле о
проса сенсора: {e}')
232                 time.sleep(self.poll_interval)
233
234     def start(self) -> None:
235         """Запускает непрерывный мониторинг в фоновом потоке."
"""
236         with self._lock:
237             if self._is_running:
238                 logger.warning('Сенсор устройств уже запущен')
239                 return
240             self._is_running = True
241             self._thread = threading.Thread(target=self._loop,
name='DeviceFlappingSensorThread', daemon=True)
242             self._thread.start()
243
244     def stop(self) -> None:

```

```
245 |         """Останавливает непрерывный мониторинг."""
246 |     with self._lock:
247 |         if not self._is_running:
248 |             return
249 |         self._is_running = False
250 |     if self._thread and self._thread.is_alive():
251 |         self._thread.join(timeout=3.0)
252 |         logger.info('Сенсор устройств успешно остановлен')
253 |
254 |     def get_status(self) -> Dict[str, Any]:
255 |         """Возвращает текущую статистику сенсора.
256 |
257 |         Returns:
258 |             Dict[str, Any]: Данные о количестве активных устройс
259 |             """
260 |         with self._lock:
261 |             return {'is_running': self._is_running, 'poll_inte
262 |             rval_sec': self.poll_interval, 'active_devices_count': len(sel
263 |             f._active_devices), 'flapping_devices_tracked': len(self._disc
264 |             onnect_history), 'devices_with_problems': sum((1 for d in self
265 |             ._active_devices.values() if d.has_problem))}
```

file_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - File Collector
4 # =====
5 # Description:
6 #   Сбор событий файловой системы через DirectoryWatcher.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.file_collector import FileCollector
11 #
12 #       service = FileCollector()
13 #
14 # File: file_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-03 23:06:40
20 # =====
21
22 from __future__ import annotations
23 """Сбор событий файловой системы через DirectoryWatcher."""
24
25 import threading
26 from datetime import datetime, timezone
27 from pathlib import Path
28 from typing import Any, Dict, List, Optional
29 try:
30     from logger import logger
31 except ImportError:
```

```

32     from logger import logger
33
34
35 def _get_directory_watcher_classes():
36     """Ленивый импорт DirectoryWatcher для избежания циклическ
их зависимостей."""
37     try:
38         from apps.windows.modules.sysadmin.src.directory_watch
er import DirectoryWatcher, LiveFileEvent
39         return DirectoryWatcher, LiveFileEvent
40     except ImportError:
41         try:
42             from apps.windows.sysadmin.src.directory_watcher i
mport DirectoryWatcher, LiveFileEvent
43             return DirectoryWatcher, LiveFileEvent
44         except ImportError as e:
45             logger.warning(f'Не удалось импортировать Director
yWatcher: {e}')
46             return None, None
47
48
49 class FileCollector:
50     """Коллектор событий файловой системы."""
51
52     def __init__(self, watch_dirs: Optional[List[str]]=None, m
ax_history: int=200) -> None:
53         """Инициализирует коллектор файловых событий.
54
55         Args:
56             watch_dirs: Список директорий для мониторинга.
57             max_history: Размер истории событий.
58         """
59         self.watch_dirs = watch_dirs or [str(Path.home() / 'Do
cuments')]
60         self.max_history = max_history
61         self._watchers: Dict[str, Any] = {}
62         self._lock = threading.Lock()
63         self._init_watchers()

```



```

64 |
65 |     def _init_watchers(self) -> None:
66 |         """Инициализирует DirectoryWatcher для каждой директор
ии."""
67 |         DirectoryWatcher, _ = _get_directory_watcher_classes()
68 |         if DirectoryWatcher is None:
69 |             logger.warning('DirectoryWatcher недоступен')
70 |             return
71 |         for watch_dir in self.watch_dirs:
72 |             try:
73 |                 watcher = DirectoryWatcher(watch_dirs=[watch_d
ir], max_history=self.max_history)
74 |                 if watcher.start():
75 |                     self._watchers[watch_dir] = watcher
76 |                     logger.info(f'DirectoryWatcher запущен для
: {watch_dir}')
77 |             except Exception as e:
78 |                 logger.error(f'Ошибка инициализации DirectoryW
atcher для {watch_dir}: {e}')
79 |
80 |     def _convert_event(self, event: LiveFileEvent) -> Dict[str
, Any]:
81 |         """Преобразует LiveFileEvent в словарь для JSON.
82 |
83 |         Args:
84 |             event: Событие от DirectoryWatcher.
85 |
86 |         Returns:
87 |             Dict[str, Any]: Событие в формате словаря.
88 |         """
89 |         return {'timestamp': event.timestamp, 'action': event.
action, 'path': event.path, 'is_deletion': event.is_deletion,
'details': event.details}
90 |
91 |     def get_recent_events(self, limit: int=100) -> List[Dict[s
tr, Any]]:
92 |         """Получает недавние файловые события.
93 |

```

```

94 |         Args:
95 |             limit: Максимальное количество событий.
96 |
97 |         Returns:
98 |             List[Dict[str, Any]]: Список файловых событий.
99 |         """
100 |         with self._lock:
101 |             events: List[Dict[str, Any]] = []
102 |             for watch_dir, watcher in self._watchers.items():
103 |                 try:
104 |                     raw_events = watcher.get_recent_events(lim
it=limit)
105 |                     for event in raw_events:
106 |                         converted = self._convert_event(event)
107 |                         converted['watch_dir'] = watch_dir
108 |                         events.append(converted)
109 |                 except Exception as e:
110 |                     logger.error(f'Ошибка получения событий из
{watch_dir}: {e}')
111 |             events.sort(key=lambda x: x.get('timestamp', ''),
reverse=True)
112 |             return events[:limit]
113 |
114 |     def get_events_by_action(self, action: str, limit: int=50)
-> List[Dict[str, Any]]:
115 |         """Получает события определенного типа.
116 |
117 |         Args:
118 |             action: Тип действия (Created, Deleted, Modified).
119 |             limit: Максимальное количество событий.
120 |
121 |         Returns:
122 |             List[Dict[str, Any]]: Список событий.
123 |         """
124 |         all_events = self.get_recent_events(limit=limit * 2)
125 |         return [e for e in all_events if e.get('action') == ac
tion][:limit]
126 |

```

```

127     def get_events_by_type(self, event_type: str, limit: int=5
128     0) -> List[Dict[str, Any]]:
129         """Получает события определенного типа (Created/Delete
130         d).
131
132         Args:
133             event_type: Тип события (Created или Deleted).
134             limit: Максимальное количество событий.
135
136         Returns:
137             List[Dict[str, Any]]: Список событий.
138         """
139         if event_type.lower() == 'created':
140             return self.get_events_by_action('Created', limit)
141         elif event_type.lower() == 'deleted':
142             return self.get_events_by_action('Deleted', limit)
143         return []
144
145     def stop(self) -> None:
146         """Останавливает все DirectoryWatcher."""
147         with self._lock:
148             for watcher in self._watchers.values():
149                 watcher.stop()
150             self._watchers.clear()
151             logger.info('FileCollector остановлен')
152
153     def restart(self) -> bool:
154         """Перезапускает DirectoryWatcher.
155
156         Returns:
157             bool: True если перезапуск успешен.
158         """
159         self.stop()
160         self._init_watchers()
161         return True
162
163     def add_watch_dir(self, watch_dir: str) -> bool:
164         """Добавляет директорию для мониторинга.

```

```

163
164     Args:
165         watch_dir: Путь к директории.
166
167     Returns:
168         bool: True если директория успешно добавлена.
169     """
170     DirectoryWatcher, _ = _get_directory_watcher_classes()
171     if DirectoryWatcher is None:
172         logger.warning('DirectoryWatcher недоступен')
173         return False
174     if watch_dir in self._watchers:
175         logger.warning(f'Директория уже отслеживается: {wa
176 tch_dir}')
177         return False
178     try:
179         watcher = DirectoryWatcher(watch_dirs=[watch_dir],
180 max_history=self.max_history)
181         if watcher.start():
182             self._watchers[watch_dir] = watcher
183             logger.info(f'Добавлена директория для мониторинга: {watch_dir}')
184             return True
185         except Exception as e:
186             logger.error(f'Ошибка добавления директории {watch
187 _dir}: {e}')
188             return False
189
190     def remove_watch_dir(self, watch_dir: str) -> bool:
191         """Удаляет директорию из мониторинга.
192
193     Args:
194         watch_dir: Путь к директории.
195
196     Returns:
197         bool: True если директория успешно удалена.
198     """
199     with self._lock:

```

```
197 |         if watch_dir in self._watchers:
198 |             watcher = self._watchers.pop(watch_dir)
199 |             watcher.stop()
200 |             logger.info(f'Удалена директория из мониторинг
a: {watch_dir}')
201 |             return True
202 |         else:
203 |             logger.warning(f'Директория не найдена в монит
оринге: {watch_dir}')
204 |             return False
```

grouped_telemetry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Grouped
  Telemetry Wrapper
4 # =====
5 # Description:
6 #   Обёртка, позволяющая импортировать группы телеметрии из мо
  дуля
7 #   `apps.windows.telemetry_research.grouped_telemetry` через
  путь
8 #   `apps.windows.telemetry.grouped_telemetry`. Это необходимо
  для совместимости
9 #   с существующими тестами, которые ожидают наличие данного м
  одуля в пакете
10 #   `apps.windows.telemetry`.
11 #
12 #   Внутри экспортируются все публичные типы, объявленные в ор
  игинальном модуле.
13 #   Переэкспорт делается без изменения логики - классы и функц
  ии остаются теми
14 #   же, что и в исследовательском подпакете.
15 # =====
16 # Updated: 2026-10-03 23:59:45
17 # =====
18
19 """Обёртка для экспорта групп телеметрии.
20
21 Этот модуль переэкспортирует ключевые классы из
22 `apps.windows.telemetry_research.grouped_telemetry`, обеспечив
  ая обратную
23 совместимость с кодом, который импортирует их напрямую из
24 `apps.windows.telemetry.grouped_telemetry`.
```

```
25 | """
26 |
27 | # Перезэкспорт из исследовательского подпакета
28 | from apps.windows.telemetry_research.grouped_telemetry import
29 | (
30 |     TelemetryGroupInfo,
31 |     GroupedTelemetryBuilder,
32 |     GroupDiagnoseRequest,
33 |     GroupDiagnosticResult,
34 |     SynthesisRequest,
35 |     SynthesisDiagnosticResult,
36 | )
37 | __all__ = [
38 |     "TelemetryGroupInfo",
39 |     "GroupedTelemetryBuilder",
40 |     "GroupDiagnoseRequest",
41 |     "GroupDiagnosticResult",
42 |     "SynthesisRequest",
43 |     "SynthesisDiagnosticResult",
44 | ]
45 |
```

incident_detector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Incident Detector
4 # =====
5 # Description:
6 #   Модуль непрерывного анализа аномалий и автоматического выявления сбоев системы.
7 #
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry.incident_detector import IncidentDetector
11 #
12 # File: incident_detector.py
13 # Project: ai-breadboard
14 # Package: apps.windows.telemetry
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-03 23:15:00
18 # =====
19
20 """Модуль `IncidentDetector`.
21
22 Содержит полную реализацию детектора аномалий, перенесённую из `telemetry_research`.
23 """
24
25 from __future__ import annotations
26
27 import time
28 import uuid
29 from datetime import datetime, timezone
```



```
30 from typing import Any, Dict, List, Optional
31
32 # logger import handling
33 try:
34     from logger import logger
35 except ImportError:
36     from logger import logger
37
38 from apps.windows.telemetry.models import TelemetryIncident
39 from apps.windows.telemetry.ring_buffer import TelemetryRingBu
40 ffer
41
42 class IncidentDetector:
43     """Детектор аномалий в потоках метрик и событий с фиксац
44 ие инцидентов."""
45
46     def __init__(
47         self,
48         cpu_threshold: float = 90.0,
49         disk_write_burst_mb: float = 100.0,
50         net_burst_mb: float = 50.0,
51         ring_buffer: Optional[TelemetryRingBuffer] = None,
52     ) -> None:
53         """Инициализирует детектор пороговыми значениями и ссы
54 лкой на кольцевой буфер.
55
56         Args:
57             cpu_threshold: Порог срабатывания по CPU % (по умо
58 лчанию 90.0).
59             disk_write_burst_mb: Порог дисковой записи в МБ/с
60 (по умолчанию 100.0).
61             net_burst_mb: Порог сетевого трафика в МБ/с (по ум
62 олчанию 50.0).
63             ring_buffer: Ссылка на экземпляр кольцевого буфера
64 памяти.
65         """
66         self.cpu_threshold = cpu_threshold
```

```

61         self.disk_write_burst_mb = disk_write_burst_mb
62         self.net_burst_mb = net_burst_mb
63         self.ring_buffer = ring_buffer or TelemetryRingBuffer(
64         )
65         self._last_incident_epoch: float = 0.0
66         self._cooldown_seconds: float = 60.0 # Не спамить оди
        наковыми инцидентами чаще раза в минуту
67
68     def check_anomaly(
69         self,
70         system_snapshot: Dict[str, Any],
71         processes: Optional[List[Dict[str, Any]]] = None,
72         recent_events: Optional[List[Dict[str, Any]]] = None,
73     ) -> Optional[TelemetryIncident]:
74         """Анализирует срез метрик и возвращает TelemetryIncident при обнаружении аномалии.
75
76         Args:
77             system_snapshot: Текущий снимок системы (SystemSnapshot dict).
78             processes: Список процессов текущего среза.
79             recent_events: Последние дискретные события (W64, ETW, Defender).
80
81         Returns:
82             Optional[TelemetryIncident]: Объект инцидента или None при штатной работе.
83
84         """
85         now = time.time()
86         cpu_pct = float(system_snapshot.get("cpu_total_percent") or system_snapshot.get("cpu_percent") or 0.0)
87         disk_w_mbs = float(system_snapshot.get("disk_write_bytes_sec") or 0.0) / (1024.0 * 1024.0)
88         net_tx_mbs = float(system_snapshot.get("network_sent_bytes_sec") or 0.0) / (1024.0 * 1024.0)
89         net_rx_mbs = float(system_snapshot.get("network_recv_bytes_sec") or 0.0) / (1024.0 * 1024.0)

```

```
89         trigger_type: Optional[str] = None
90         severity = "warning"
91         title = ""
92         description = ""
93         suspects: List[Dict[str, Any]] = []
94
95         # 1. Проверка CPU Spike
96         if cpu_pct >= self.cpu_threshold:
97             trigger_type = "cpu_spike"
98             severity = "critical" if cpu_pct >= 95.0 else "war
99 ning"
100             title = f"Критическая нагрузка на процессор: {cpu_
101 pct:.1f}%"
102             description = f"Общая загрузка CPU превысила порог
103 {self.cpu_threshold}%, достигнув {cpu_pct:.1f}%."
104
105         # 2. Проверка Disk Write Burst
106         elif disk_w_mbs >= self.disk_write_burst_mb:
107             trigger_type = "disk_burst"
108             severity = "warning"
109             title = f"Аномальная скорость записи на диск: {dis
110 k_w_mbs:.1f} МБ/с"
111             description = f"Скорость дисковой записи превысила
112 порог {self.disk_write_burst_mb} МБ/с."
113
114         # 3. Проверка Network Burst
115         elif (net_tx_mbs + net_rx_mbs) >= self.net_burst_mb:
116             trigger_type = "network_burst"
117             severity = "warning"
118             title = f"Всплеск сетевой активности: {(net_tx_mbs
119 + net_rx_mbs):.1f} МБ/с"
120             description = f"Суммарный сетевой трафик достиг {(
121 net_tx_mbs + net_rx_mbs):.1f} МБ/с (TX: {net_tx_mbs:.1f}, RX:
122 {net_rx_mbs:.1f})."
123
124         # 4. Проверка подозрительных процессов (AppData, Temp,
125 Public)
126         if processes:
```

```

118         for p in processes:
119             exe_path = str(p.get("path") or p.get("exe") or "").lower()
120             p_cpu = float(p.get("cpu_percent") or 0.0)
121             is_suspicious_location = any(
122                 loc in exe_path for loc in ("\\temp\\", "\\appdata\\local\\temp", "\\users\\public\\")
123             )
124             if is_suspicious_location and (p_cpu > 15.0 or trigger_type is not None):
125                 suspects.append({
126                     "pid": p.get("pid"),
127                     "name": p.get("name"),
128                     "path": exe_path,
129                     "cpu_percent": p_cpu,
130                     "memory_mb": p.get("memory_mb"),
131                     "reason": "Запуск из временного каталога с повышенной активностью",
132                 })
133                 if trigger_type is None and p_cpu > 30.0:
134                     trigger_type = "suspicious_process"
135                     severity = "critical"
136                     title = f"Подозрительный процесс из Temp: {p.get('name')} (PID {p.get('pid')})"
137                     description = f"Процесс {p.get('name')} выполняется из {exe_path} и потребляет {p_cpu:.1f}% CPU."
138
139                 elif p_cpu >= 50.0:
140                     suspects.append({
141                         "pid": p.get("pid"),
142                         "name": p.get("name"),
143                         "path": exe_path,
144                         "cpu_percent": p_cpu,
145                         "memory_mb": p.get("memory_mb"),
146                         "reason": f"Высокое индивидуальное потребление CPU: {p_cpu:.1f}%",
147                     })
148

```

```

149 |         # 5. Проверка событий безопасности (Defender / ETW)
150 |         matched_events: List[Dict[str, Any]] = []
151 |         if recent_events:
152 |             for ev in recent_events:
153 |                 ev_type = str(ev.get("event_type") or "").lower()
154 |                 ev_str = str(ev).lower()
155 |                 if "defender" in ev_type or "threat" in ev_str
or "malware" in ev_str:
156 |                     matched_events.append(ev)
157 |                     if trigger_type is None:
158 |                         trigger_type = "defender_alert"
159 |                         severity = "critical"
160 |                         title = "Срабатывание Windows Defender
/ Защиты от угроз"
161 |                         description = f"Зафиксировано событие
безопасности: {ev.get('name') or ev_type}"
162 |
163 |                     if not trigger_type:
164 |                         return None
165 |
166 |         # Проверка кулдауна, чтобы не генерировать дубликаты е
жесекундно
167 |         if (now - self._last_incident_epoch) < self._cooldown_
seconds:
168 |             return None
169 |
170 |         self._last_incident_epoch = now
171 |         inc_id = f"INC-{{datetime.now(timezone.utc).strftime('%
Y%m%d%H%M%S')}}-{{uuid.uuid4().hex[:4].upper()}}"
172 |
173 |         # Захватываем срез из кольцевого буфера (например, 5 м
инут до события)
174 |         raw_window = self.ring_buffer.get_window(center_timest
amp=now, before_seconds=300.0, after_seconds=0.0)
175 |
176 |         incident = TelemetryIncident(
177 |             incident_id=inc_id,

```

```

178         timestamp=datetime.now(timezone.utc).isoformat(),
179         trigger_type=trigger_type,
180         severity=severity,
181         title=title,
182         description=description,
183         trigger_metrics={
184             "cpu_percent": cpu_pct,
185             "disk_write_mbs": disk_w_mbs,
186             "network_tx_mbs": net_tx_mbs,
187             "network_rx_mbs": net_rx_mbs,
188         },
189         suspect_processes=suspects,
190         related_events=matched_events or (recent_events[:1
191 0] if recent_events else []),
191         metrics_summary={
192             "buffer_samples_captured": len(raw_window),
193             "peak_cpu": cpu_pct,
194             "peak_disk_write_mbs": disk_w_mbs,
195         },
196         raw_window=raw_window,
197     )
198
199     logger.warning(f"Зафиксирован инцидент телеметрии [{incident.incident_id}]: {incident.title}")
200     return incident
201
202     @staticmethod
203     def format_incident_markdown_for_llm(incident: TelemetryIncident) -> str:
204         """Форматирует инцидент в компактный структурированный
205         Markdown для AI / LLM анализа.
206
207         Args:
208             incident: Объект TelemetryIncident.
209
210         Returns:
211             str: Готовый markdown блок для контекста языковой
212             модели.

```

```

211         """
212         lines = [
213             f"### □ ДИАГНОСТИЧЕСКИЙ ИНЦИДЕНТ: [{incident.incident_id}]",
214             f"- **Тип триггера:** `{incident.trigger_type}` |",
215             f"- **Критичность:** `{incident.severity.upper()}`",
216             f"- **Время:** {incident.timestamp}",
217             f"- **Сводка:** {incident.title}",
218             f"- **Описание:** {incident.description}",
219             "",
220             f"#### □ Метрики в момент инцидента:",
221             f"- CPU: `{incident.trigger_metrics.get('cpu_percent', 0.0)}%`",
222             f"- Диск (Запись): `{incident.trigger_metrics.get('disk_write_mbs', 0.0)} МБ/с`",
223             f"- Сеть (TX / RX): `{incident.trigger_metrics.get('network_tx_mbs', 0.0)} МБ/с` / `{incident.trigger_metrics.get('network_rx_mbs', 0.0)} МБ/с`",
224             f"- Сохранено секундных замеров в буфере: `{incident.metrics_summary.get('buffer_samples_captured', 0)}%`",
225             "",
226         ]
227         if incident.suspect_processes:
228             lines.append("#### □ Подозрительные процессы:")
229             lines.append("| PID | Имя | Путь | CPU % | Причина")
230             lines.append("|---|---|---|---|---|")
231             for sp in incident.suspect_processes:
232                 lines.append(
233                     f"| {sp.get('pid', '-')} | `{sp.get('name', '-')}` | `{sp.get('path', '-')}` | {sp.get('cpu_percent', 0.0)}% | {sp.get('reason', '-')}"
234                 )
235             lines.append("")
236
237         if incident.related_events:
238             lines.append("#### □ Сопутствующие события систем

```

```
    ы (Events):")
239         for ev in incident.related_events[:5]:
240             ev_name = ev.get("name") or ev.get("event_type") or "event"
241             ev_time = ev.get("timestamp", "")
242             lines.append(f"- `{ev_time}`: **{ev_name}** ({ev.get('path') or ev.get('details') or ''})")
243             lines.append("")
244
245         lines.append("Инструкция: Оцените потенциальную угрозу (вирус/майнер/зависание/утечка) и предложите конкретные меры противодействия.")
246         return "\n".join(lines)
247
```


init_db.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Init Db
4 # =====
5 # Description:
6 #   Модуль проверки, создания и валидации целостности структур
7 #   ы базы данных
8 #
9 # Usage Examples:
10 #   CLI:
11 #       python -m apps.windows.telemetry.init_db
12 #   Python API:
13 #       from apps.windows.telemetry.init_db import get_default_t
14 #       telemetry_db_path
15 #
16 #       res = get_default_telemetry_db_path()
17 #
18 # File: init_db.py
19 # Project: ai-breadboard
20 # Package: apps.windows.telemetry
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # Updated: 2026-10-01 13:28:28
24 # =====
25
26
27 from __future__ import annotations
28 """Модуль проверки, создания и валидации целостности структуры
29   базы данных"""
30
31 import argparse
32 import json
33 import os
34 from pathlib import Path
```

```

31 import sqlite3
32 import sys
33 from typing import Any, Dict, List, Optional, Union
34
35 _project_root = str(Path(__file__).resolve().parents[3])
36 if _project_root not in sys.path:
37     sys.path.insert(0, _project_root)
38 if __package__ in (None, ''):
39     __package__ = 'apps.windows.telemetry'
40
41 try:
42     from logger import logger
43 except ImportError:
44     import logging
45     logger = logging.getLogger(__name__)
46
47 from .sqlite import TelemetryStorage
48
49
50 def get_default_telemetry_db_path() -> Path:
51     """Возвращает стандартный путь к файлу базы данных telemetry.db.
52
53     Returns:
54         Path: Путь к файлу telemetry.db в каталоге логов приложения.
55     """
56     appdata = os.environ.get('APPDATA') or os.environ.get('LOCALAPPDATA')
57     if appdata and os.path.exists(appdata):
58         base_dir = Path(appdata)
59     else:
60         base_dir = Path.home() / '.config'
61     return base_dir / 'AI-Breadboard' / 'apps' / 'windows' / 'telemetry' / 'logs' / 'telemetry.db'
62
63
64 def init_telemetry_database(

```

```

65     db_path: Optional[Union[str, Path]] = None,
66     force: bool = False,
67     check_integrity: bool = True,
68 ) -> Dict[str, Any]:
69     """Инициализирует базу данных telemetry.db, создавая все т
аблицы и индексы при отсутствии.
70
71     Args:
72         db_path: Путь к целевому файлу SQLite базы данных (по
умолчанию стандартный путь).
73         force: Принудительно переинициализировать структуру та
блиц.
74         check_integrity: Выполнить проверку целостности PRAGMA
integrity_check.
75
76     Returns:
77         Dict[str, Any]: Словарь с результатами инициализации и
метаданными БД.
78     """
79     target_path = Path(db_path) if db_path else get_default_te
lemetry_db_path()
80     target_path.parent.mkdir(parents=True, exist_ok=True)
81
82     was_created = False
83     if not target_path.exists() or force:
84         logger.debug(f'Инициализация новой базы данных телемет
рии по пути: {target_path}')
85         storage = TelemetryStorage(db_path=target_path)
86         was_created = True
87     else:
88         storage = TelemetryStorage.get_instance(db_path=target
_path)
89
90     # Получение списка созданных таблиц
91     tables: List[str] = []
92     integrity_ok = True
93     integrity_msg = 'ok'
94

```

```

95 |     try:
96 |         with sqlite3.connect(str(target_path)) as conn:
97 |             cursor = conn.cursor()
98 |             cursor.execute("SELECT name FROM sqlite_master WHE
RE type='table' AND name NOT LIKE 'sqlite_%';")
99 |             tables = sorted([row[0] for row in cursor.fetchall
100 | (())
101 |
102 |             if check_integrity:
103 |                 cursor.execute('PRAGMA integrity_check;')
104 |                 res = cursor.fetchone()
105 |                 if res and res[0] != 'ok':
106 |                     integrity_ok = False
107 |                     integrity_msg = str(res[0])
108 |             except Exception as ex:
109 |                 logger.error(f'Ошибка проверки схемы и целостности БД
110 | {target_path}: {ex}')
111 |                 integrity_ok = False
112 |                 integrity_msg = str(ex)
113 |
114 |             size_bytes = target_path.stat().st_size if target_path.exi
sts() else 0
115 |             size_mb = round(size_bytes / (1024 * 1024), 3)
116 |
117 |             result = {
118 |                 'status': 'success' if integrity_ok else 'warning',
119 |                 'db_path': str(target_path),
120 |                 'exists': target_path.exists(),
121 |                 'created': was_created,
122 |                 'size_bytes': size_bytes,
123 |                 'size_mb': size_mb,
124 |                 'tables_count': len(tables),
125 |                 'tables': tables,
126 |                 'integrity_ok': integrity_ok,
127 |                 'integrity_message': integrity_msg,
128 |             }
129 |
130 |             if was_created:

```

```

129         logger.info(f'    База данных telemetry.db успешно созда
на: {len(tables)} таблиц ({size_mb} МБ)')
130     else:
131         logger.debug(f'База данных telemetry.db подтверждена:
{len(tables)} таблиц ({size_mb} МБ)')
132
133     return result
134
135
136 def main() -> int:
137     """Точка входа CLI для инициализации и проверки базы данны
х телеметрии."""
138     parser = argparse.ArgumentParser(
139         description='Инициализатор и верификатор SQLite базы д
анных системной телеметрии (telemetry.db)'
140     )
141     parser.add_argument('--db-path', type=str, default=None, h
elp='Пользовательский путь к telemetry.db')
142     parser.add_argument('--force', action='store_true', help='
Принудительно переинициализировать структуру')
143     parser.add_argument('--check-only', action='store_true', h
elp='Только проверить существующую БД без создания')
144     parser.add_argument('--json', action='store_true', help='В
ывод результата в формате JSON')
145
146     args = parser.parse_args()
147
148     target_path = Path(args.db_path) if args.db_path else get_
default_telemetry_db_path()
149
150     if args.check_only and not target_path.exists():
151         if args.json:
152             print(json.dumps({'status': 'error', 'message': f'
Файл не найден: {target_path}', 'exists': False}, ensure_ascii
=False))
153         else:
154             print(f'    База данных телеметрии не найдена: {targ
et_path}')

```

```
155 |         return 1
156 |
157 |     res = init_telemetry_database(db_path=args.db_path, force=
| args.force)
158 |
159 |     if args.json:
160 |         print(json.dumps(res, ensure_ascii=False, indent=2))
161 |     else:
162 |         status_icon = '✅ ' if res['integrity_ok'] else '❌ '
163 |         print(f"\n{status_icon} База данных телеметрии: {res['
| db_path']}")
164 |         print(f"    • Существует:      {res['exists']}")
165 |         print(f"    • Вновь создана:  {res['created']}")
166 |         print(f"    • Размер файла:   {res['size_mb']} МБ ({res
| ['size_bytes']} байт)")
167 |         print(f"    • Таблиц в схеме: {res['tables_count']}")
168 |         print(f"    • Целостность:    {res['integrity_message']
| }")
169 |         if res['tables']:
170 |             print(f"    • Таблицы:      {'', '.join(res['table
| s'][:8])}'{'...' if len(res['tables']) > 8 else ''}")
171 |             print('')
172 |
173 |     return 0 if res['integrity_ok'] else 1
174 |
175 |
176 | if __name__ == '__main__':
177 |     sys.exit(main())
178 |
```

internet_speed.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Internet Speed
4 # =====
5 # Description:
6 #   Сенсор телеметрии скорости интернета и задержки сети на базе FAST.com (Netflix CDN).
7 #   Выполняет периодический замер (1 раз в час / 3600 сек) и кэширует результаты.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.internet_speed import InternetSpeedSensor
12 #
13 #       sensor = InternetSpeedSensor()
14 #       metrics = sensor.measure_internet_speed()
15 #       print(metrics['download_mbps'], metrics['bufferbloat_ms'])
16 #
17 # File: internet_speed.py
18 # Project: ai-breadboard
19 # Package: apps.windows.telemetry
20 # Author: hypo69
21 # Copyright: © 2026 hypo69
22 # Updated: 2026-10-04 07:35:00
23 # =====
24
25 from __future__ import annotations
26 """Сенсор телеметрии скорости интернета и задержки сети на базе FAST.com."""
27
```

```

28 import asyncio
29 import threading
30 import time
31 from typing import Any, Dict, List, Optional
32
33 from logger import logger
34 from apps.windows.modules.network.speedtest import NetworkSpeedTester
35
36 _speed_cache: Optional[tuple[float, Dict[str, Any]]] = None
37 _CACHE_TTL = 3600 # 1 час между замерами скорости для снижения нагрузки на сеть
38 _cache_lock = threading.Lock()
39 _bg_test_running = False
40 _bg_lock = threading.Lock()
41
42
43 class InternetSpeedSensor:
44     """Сенсор измерения скорости интернета и характеристик задержки (FAST.com)."""
45
46     def __init__(self, cache_ttl: int = _CACHE_TTL) -> None:
47         """Инициализация сенсора скорости интернета.
48
49         Args:
50             cache_ttl: Время жизни кеша замера в секундах (по умолчанию 3600 сек = 1 час).
51
52         """
53         self.cache_ttl = cache_ttl
54         self._speedtester = NetworkSpeedTester()
55
56     @classmethod
57     def clear_cache(cls) -> None:
58         """Очистить глобальный кеш замеров скорости интернета.
59
60         """
61         global _speed_cache
62         with _cache_lock:
63             _speed_cache = None

```



```

61 |
62 |     def _run_speedtest_sync(self) -> Dict[str, Any]:
63 |         """Синхронно выполнить полный цикл теста скорости FAST
64 |         .com."""
65 |         try:
66 |             loop = None
67 |             try:
68 |                 loop = asyncio.get_event_loop()
69 |             except RuntimeError:
70 |                 loop = None
71 |
72 |             if loop and loop.is_running():
73 |                 # Если вызывается внутри уже работающего цикла
74 |                 событий (например, в async worker)
75 |                 import concurrent.futures
76 |                 with concurrent.futures.ThreadPoolExecutor(max
77 |                 _workers=1) as executor:
78 |                     future = executor.submit(lambda: asyncio.r
79 |                     un(self._speedtester.run_full_speedtest()))
80 |                     return future.result(timeout=60.0)
81 |             else:
82 |                 return asyncio.run(self._speedtester.run_full_
83 |                 speedtest())
84 |         except Exception as ex:
85 |             logger.error(f'Ошибка выполнения теста скорости FA
86 |             ST.com в сенсоре телеметрии: {ex}')
87 |             return {
88 |                 'download': {'speed_mbps': 0.0, 'status': 'ERR
89 |                 OR'},
90 |                 'upload': {'speed_mbps': 0.0, 'status': 'ERROR
91 |                 '},
92 |                 'ping_ms': 0.0,
93 |                 'latency_unloaded_ms': 0.0,
94 |                 'latency_loaded_ms': 0.0,
95 |                 'bufferbloat_ms': 0.0,
96 |                 'meta': {'ip': 'N/A', 'isp': 'Unknown'},
97 |                 'provider': 'fast.com',
98 |                 'server': 'N/A'

```

```

91         }
92
93     def _perform_measurement_and_cache(self) -> Dict[str, Any]
94     :
95         """Выполняет замер скорости и сохраняет его в кэш."""
96         global _speed_cache, _bg_test_running
97         try:
98             report = self._run_speedtest_sync()
99             download_mbps = float(report.get('download', {}).g
100 et('speed_mbps', 0.0))
101             upload_mbps = float(report.get('upload', {}).get('
102 speed_mbps', 0.0))
103             ping_ms = float(report.get('ping_ms', report.get('
104 latency_unloaded_ms', 0.0)))
105             latency_loaded_ms = float(report.get('latency_load
106 ed_ms', 0.0))
107             bufferbloat_ms = float(report.get('bufferbloat_ms'
108 , 0.0))
109             meta = report.get('meta', {})
110
111             result: Dict[str, Any] = {
112                 'available': report.get('download', {}).get('s
113 tatus') == 'SUCCESS' or download_mbps > 0.0,
114                 'download_mbps': download_mbps,
115                 'upload_mbps': upload_mbps,
116                 'ping_ms': ping_ms,
117                 'latency_unloaded_ms': float(report.get('laten
118 cy_unloaded_ms', ping_ms)),
119                 'latency_loaded_ms': latency_loaded_ms,
120                 'bufferbloat_ms': bufferbloat_ms,
121                 'provider': 'fast.com',
122                 'server': report.get('server', 'FAST.com CDN')
123             },
124             'ip': meta.get('ip', 'N/A'),
125             'isp': meta.get('isp', 'Unknown ISP'),
126             'measured_at': time.time(),
127         }
128     }
129

```

```

120         with _cache_lock:
121             _speed_cache = (time.monotonic(), result)
122
123         logger.info(
124             f' [Телеметрия FAST.com] Скорость: ↓{download_mbps:.1f} Mbps, '
125             f'↑{upload_mbps:.1f} Mbps, пинг: {ping_ms:.1f} мс, bufferbloat: {bufferbloat_ms:.1f} мс'
126         )
127         return dict(result)
128     finally:
129         with _bg_lock:
130             _bg_test_running = False
131
132     def trigger_async_measure(self) -> None:
133         """Запускает измерение скорости в фоновом потоке, если
134         оно еще не выполняется."""
135         global _bg_test_running
136         with _bg_lock:
137             if _bg_test_running:
138                 return
139             _bg_test_running = True
140
141         thread = threading.Thread(
142             target=self._perform_measurement_and_cache,
143             name="InternetSpeedTestWorker",
144             daemon=True
145         )
146         thread.start()
147
148     def measure_internet_speed(self, force_refresh: bool = False, sync: bool = False) -> Dict[str, Any]:
149         """Измеряет профиль интернет-соединения через FAST.com
150         с кешированием на 1 час.
151
152         По умолчанию выполняется асинхронно в фоне и не блокирует вызывающий поток.

```

```

152 |         Args:
153 |             force_refresh: Принудительно инициировать новый за
мер.
154 |             sync: Выполнить замер синхронно с блокировкой теку
щего потока.
155 |
156 |         Returns:
157 |             Dict[str, Any]: Словарь метрик (download_mbps, upl
oad_mbps, ping_ms, bufferbloat_ms, etc.).
158 |             """
159 |             global _speed_cache
160 |             with _cache_lock:
161 |                 if not force_refresh and _speed_cache is not None:
162 |                     cached_at, cached_result = _speed_cache
163 |                     if time.monotonic() - cached_at < self.cache_t
tl:
164 |                         return dict(cached_result)
165 |
166 |             if sync:
167 |                 return self._perform_measurement_and_cache()
168 |
169 |             # Асинхронный запуск замера в фоне
170 |             self.trigger_async_measure()
171 |
172 |             with _cache_lock:
173 |                 if _speed_cache is not None:
174 |                     _, cached_result = _speed_cache
175 |                     return dict(cached_result)
176 |
177 |             # Возвращаем заглушку по умолчанию без блокировки пото
ка
178 |             return {
179 |                 'available': True,
180 |                 'download_mbps': 0.0,
181 |                 'upload_mbps': 0.0,
182 |                 'ping_ms': 0.0,
183 |                 'latency_unloaded_ms': 0.0,
184 |                 'latency_loaded_ms': 0.0,

```

```

185         'bufferbloat_ms': 0.0,
186         'provider': 'fast.com',
187         'server': 'FAST.com CDN (замер в фоне...)',
188         'ip': 'N/A',
189         'isp': 'Unknown ISP',
190         'measured_at': 0.0,
191     }
192
193
194 def get_internet_speed_sensors() -> List[Dict[str, Any]]:
195     """Получить показания сетевых метрик скорости в формате сп
иска сенсоров.
196
197     Returns:
198         List[Dict[str, Any]]: Список метрик сенсоров интернета
199     """
200     sensor = InternetSpeedSensor()
201     metrics = sensor.measure_internet_speed()
202     sensors = [
203         {
204             'sensor_id': 'internet_ping',
205             'name': 'Internet Ping Latency',
206             'category': 'network',
207             'value': metrics.get('ping_ms', 0.0),
208             'unit': 'ms'
209         },
210         {
211             'sensor_id': 'internet_download',
212             'name': 'Internet Download Speed',
213             'category': 'network',
214             'value': metrics.get('download_mbps', 0.0),
215             'unit': 'Mbps'
216         },
217         {
218             'sensor_id': 'internet_upload',
219             'name': 'Internet Upload Speed',
220             'category': 'network',

```

```
221         'value': metrics.get('upload_mbps', 0.0),
222         'unit': 'Mbps'
223     },
224     {
225         'sensor_id': 'internet_bufferbloat',
226         'name': 'Internet Bufferbloat',
227         'category': 'network',
228         'value': metrics.get('bufferbloat_ms', 0.0),
229         'unit': 'ms'
230     },
231     {
232         'sensor_id': 'internet_latency_loaded',
233         'name': 'Internet Loaded Latency',
234         'category': 'network',
235         'value': metrics.get('latency_loaded_ms', 0.0),
236         'unit': 'ms'
237     },
238 ]
239 return sensors
```

json_logger.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Json Lo
  gger
4 # =====
5 # Description:
6 #   Модуль JSON-логгера телеметрии Windows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.json_logger import Telemetry
      JsonLogger
11 #
12 #       service = TelemetryJsonLogger()
13 #
14 # File: json_logger.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Модуль JSON-логгера телеметрии Windows."""
24
25 import json
26 import logging
27 import os
28 import time
29 from pathlib import Path
30 from typing import List, Optional
31
```

```

32 _logger = logging.getLogger(__name__)
33
34
35 class TelemetryJsonLogger:
36     """Записывает телеметрические события в JSON-Lines файл.
37
38     Параметры
39     -----
40     log_dir: str
41         Путь к директории, где будут храниться лог-файлы.
42     filename: str
43         Базовое имя лог-файла (без расширения).
44     max_file_size_mb: float, optional
45         Максимальный размер файла в мегабайтах до ротации. По
46         умолчанию 5 МБ.
47     """
48     def __init__(self, log_dir: str, filename: str, max_file_s
49         ize_mb: float = 5.0) -> None:
50         self.log_dir = Path(log_dir)
51         self.log_dir.mkdir(parents=True, exist_ok=True)
52         self.base_filename = filename if filename.endswith('.j
53         sonl') else f'{filename}.jsonl'
54         self.max_bytes = int(max_file_size_mb * 1024 * 1024)
55         self.current_path = self.log_dir / self.base_filename
56         _logger.debug('Инициализирован TelemetryJsonLogger: di
57         r=%s, file=%s, max_bytes=%d', self.log_dir, self.base_filename
58         , self.max_bytes)
59
60     def _rotate_if_needed(self) -> None:
61         """Ротирует текущий лог-файл, если его размер превышает
62         лимит.
63
64         Новый файл получает суффикс с меткой времени.
65     """
66     if self.current_path.exists() and self.current_path.st
67         at().st_size >= self.max_bytes:
68         timestamp = time.strftime('%Y%m%d%H%M%S')
69         rotated_name = f"{self.base_filename.rstrip('.json

```



```

l'}}_{timestamp}.jsonl"
63         rotated_path = self.log_dir / rotated_name
64         try:
65             self.current_path.rename(rotated_path)
66             _logger.info('Файл лога ротирован: %s -> %s',
self.current_path.name, rotated_name)
67         except OSError as exc:
68             _logger.error('Не удалось ротировать лог-файл:
%s', exc)
69
70     def _write_record(self, record: dict) -> None:
71         """Записывает одну строку JSON в текущий файл.
72         Файл открывается в режиме добавления.
73         """
74         line = json.dumps(record, ensure_ascii=False)
75         with self.current_path.open('a', encoding='utf-8') as
f:
76             f.write(line + '\n')
77         _logger.debug('Записана запись в %s: %s', self.current
_path.name, line)
78
79     def log(self, record: dict) -> bool:
80         """Записать одну запись.
81
82         Возвращает ``True`` при успешной записи.
83         """
84         try:
85             self._rotate_if_needed()
86             self._write_record(record)
87             return True
88         except Exception as exc:
89             _logger.exception('Ошибка записи лога: %s', exc)
90             return False
91
92     def log_batch(self, records: List[dict]) -> int:
93         """Записать список записей.
94
95         Возвращает количество успешно записанных элементов.

```

```

96         """
97         count = 0
98         for rec in records:
99             if self.log(rec):
100                 count += 1
101         return count
102
103     def get_last_measurement(self) -> Optional[dict]:
104         """Вернуть последнюю запись из текущего лог-файла, если
и файл существует.
105         """
106         if not self.current_path.exists():
107             return None
108         try:
109             with self.current_path.open('rb') as f:
110                 f.seek(0, os.SEEK_END)
111                 offset = -1
112                 while -offset < f.tell():
113                     f.seek(offset, os.SEEK_END)
114                     if f.read(1) == b'\n' and offset != -1:
115                         break
116                     offset -= 1
117                 f.seek(offset + 1, os.SEEK_END)
118                 last_line = f.readline().decode('utf-8').strip
119             ()
120                 return json.loads(last_line) if last_line else
None
121         except Exception as exc:
122             _logger.exception('Не удалось прочитать последнюю
запись: %s', exc)
123             return None
124
125     def get_log_file_size(self) -> int:
126         """Вернуть размер текущего лог-файла в байтах.
127         """
128         return self.current_path.stat().st_size if self.curren
t_path.exists() else 0

```

main.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Main
4 # =====
5 # Description:
6 #   Главная точка входа для сервиса сбора телеметрии Windows.
7 #
8 # Usage Examples:
9 #   CLI:
10 #       python -m apps.windows.telemetry.main
11 #   Python API:
12 #       from apps.windows.telemetry.main import parse_arguments
13 #
14 #       res = parse_arguments()
15 #
16 # File: main.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-06 03:15:00
22 # =====
23
24 from __future__ import annotations
25 """Главная точка входа для сервиса сбора телеметрии Windows."""
26
27 import argparse
28 import os
29 import signal
30 import sys
31 import threading
32 import time
```

```

33 from datetime import datetime, timezone
34 from pathlib import Path
35 from typing import Any, Dict, List, Optional
36 _project_root = str(Path(__file__).resolve().parents[3])
37 if _project_root not in sys.path:
38     sys.path.insert(0, _project_root)
39 if __package__ in (None, ''):
40     __package__ = 'apps.windows.telemetry'
41 try:
42     from logger import logger
43 except ImportError:
44     from logger import logger
45 _stop_event = threading.Event()
46
47 def parse_arguments() -> argparse.Namespace:
48     """Парсит аргументы командной строки.
49
50     Returns:
51         argparse.Namespace: Распарсенные аргументы.
52     """
53     parser = argparse.ArgumentParser(description='Windows Tele
54     metry Service (Minimal, Hybrid, Full)', formatter_class=argpar
55     se.RawDescriptionHelpFormatter)
56     parser.add_argument('--mode', choices=['minimal', 'hybrid'
57     , 'full'], default=None, help='Режим работы: minimal (легковес
58     ный), hybrid (минимал + тяжелые периодически), full (полный ст
59     ек)')
60     parser.add_argument('--minimal', action='store_true', help
61     ='Запустить в легковесном режиме (эквивалентно --mode minimal)
62     ')
63     parser.add_argument('--interval', type=float, default=None
64     , help='Интервал сбора быстрой телеметрии в секундах (по умолч
65     анию из config.json)')
66     parser.add_argument('--heavy-interval', type=float, defaul
67     t=None, help='Интервал сбора тяжелых сенсоров для hybrid режим
68     а в секундах (по умолчанию из config.json)')
69     parser.add_argument('--top-processes', type=int, default=N
70     one, help='Количество сохраняемых процессов с наибольшей нагру

```

```

    зкой (по умолчанию 10)')
59     parser.add_argument('--config', type=str, default=None, help='Путь к файлу конфигурации config.json')
60     parser.add_argument('--log-dir', type=str, default=None, help='Путь к директории для логов и SQLite БД телеметрии')
61     parser.add_argument('--verbose', '-v', action='store_true', help='Включить подробный вывод (DEBUG)')
62     parser.add_argument('--once', action='store_true', help='Выполнить однократный опрос метрик и завершить работу')
63     return parser.parse_args()
64
65 def signal_handler(signum: int, frame: Optional[object]) -> None:
66     """Обработчик сигналов для корректной остановки сервиса.
67
68     Args:
69         signum: Номер сигнала.
70         frame: Стек вызовов.
71     """
72     logger.info(f'Получен сигнал {signum}, остановка телеметрии и...')
73     _stop_event.set()
74
75 def _set_low_priority() -> None:
76     """Устанавливает пониженный приоритет процессу для экономии ресурсов CPU."""
77     try:
78         import psutil
79         p = psutil.Process()
80         p.nice(psutil.BELOW_NORMAL_PRIORITY_CLASS)
81         logger.debug('Установлен приоритет процесса: BELOW_NORMAL')
82     except Exception as ex:
83         logger.debug(f'Не удалось понизить приоритет процесса: {ex}')
84
85 def run_telemetry_service(
86     mode: str = 'hybrid',

```

```

87     interval: float = 5.0,
88     heavy_interval: float = 60.0,
89     top_processes: int = 10,
90     heavy_collectors: Optional[Dict[str, bool]] = None,
91     log_dir: Optional[str] = None,
92     config_path: Optional[str] = None,
93     once: bool = False,
94 ) -> int:
95     """Запускает универсальный цикл сбора телеметрии (Minimal,
    Hybrid или Full).
96
97     Args:
98         mode: Режим сбора ('minimal', 'hybrid', 'full').
99         interval: Быстрый интервал сбора базовых метрик в секундах.
100        heavy_interval: Периодический интервал сбора тяжелых сенсоров (для hybrid).
101        top_processes: Количество процессов в топе.
102        heavy_collectors: Словарь активных тяжелых сенсоров.
103        log_dir: Каталог сохранения БД и логов.
104        config_path: Путь к файлу конфигурации.
105
106     Returns:
107         int: Код завершения.
108     """
109     import psutil
110     from apps.windows.modules.hardware.gpu_prober import GpuProber
111     from .models import CpuMetrics, DiskIoMetrics, DiskPartitionMetrics, GpuMetrics, MemoryMetrics, NetworkInterfaceMetrics, ProcessMetrics, SystemSnapshot
112     from .sqlite import TelemetryStorage
113     _set_low_priority()
114     resolved_log_dir = log_dir or os.path.join(os.environ.get('APPDATA', os.path.expanduser('~\\AppData\\Roaming')), 'AI-Board', 'apps', 'windows', 'telemetry', 'logs')
115     os.makedirs(resolved_log_dir, exist_ok=True)
116     db_path = os.path.join(resolved_log_dir, 'telemetry.db')

```

```

117     storage = TelemetryStorage(db_path=db_path)
118     gpu_prober = GpuProber()
119     hostname = os.environ.get('COMPUTERNAME', 'localhost')
120     boot_time = psutil.boot_time()
121     psutil.cpu_percent(interval=None)
122     from .telemetry_config import TelemetryConfigManager
123     cfg_mgr = TelemetryConfigManager(config_path=config_path)
124
125     custom_mode_set = mode is not None and ('--mode' in sys.ar
gv or '--minimal' in sys.argv)
126     custom_interval_set = interval is not None and ('--interval' in sys.argv or '-i' in sys.argv)
127     custom_heavy_interval_set = heavy_interval is not None and '--heavy-interval' in sys.argv
128     custom_top_processes_set = top_processes is not None and '--top-processes' in sys.argv
129
130     if not custom_mode_set and cfg_mgr:
131         mode = cfg_mgr.get_mode()
132     if not custom_interval_set and cfg_mgr:
133         interval = cfg_mgr.get_interval_seconds()
134     if not custom_heavy_interval_set and cfg_mgr:
135         heavy_interval = cfg_mgr.get_heavy_interval_seconds()
136     if not custom_top_processes_set and cfg_mgr:
137         top_processes = cfg_mgr.get_effective_process_limit()
138
139     heavy_disk_interval_sec = cfg_mgr.get_heavy_disk_scan_interval()
140     max_heavy_duration_sec = cfg_mgr.get_heavy_mode_max_duration_days() * 24.0 * 3600.0
141     auto_switch_enabled = cfg_mgr.is_heavy_auto_switch_enabled()
142
143     h_collectors = heavy_collectors or cfg_mgr.get_heavy_collectors()
144     sensor_collector = None
145     if mode in ('hybrid', 'full') and h_collectors.get('hardware_sensors', True):

```

```

146         try:
147             from .sensor_collector import SensorCollector
148             sensor_collector = SensorCollector(config_manager=
cfg_mgr)
149             logger.info('SensorCollector (LHM/Hardware) инициа
лизирован для тяжелых опросов')
150         except Exception as sc_err:
151             logger.debug(f'SensorCollector не доступен: {sc_er
r}')
152     system_collector = None
153     deep_diag = None
154     try:
155         from .collector import SystemCollector
156         from apps.windows.telemetry_research.deep_diagnostics
import DeepDiagnosticsEngine
157         system_collector = SystemCollector(storage=storage)
158         deep_diag = DeepDiagnosticsEngine()
159         identity = system_collector.get_system_identity()
160         disks_health = system_collector.get_physical_disks_hea
lth(force=True)
161         import asyncio
162         hw_tree = asyncio.run(system_collector.get_hardware_tr
ee_async(force=True))
163         logger.info(f" Хост: {identity.get('hostname')} | По
льзователь: {identity.get('username')} | ОС: Windows {identity
.get('os_build')}")
164         logger.info(f" Язык: {identity.get('system_language'
)} | Локаль: {identity.get('user_locale')} | Раскладки: {' ', '
'.join(identity.get('input_languages', []))}")
165         if disks_health:
166             d_summary = ", ".join([f"{d.model} ({d.media_type}
, {d.size_gb}GB, SMART:{d.health_status})" for d in disks_heal
th])
167             logger.info(f" Накопители SMART (Опрос при запус
ке): {d_summary}")
168         if hw_tree:
169             logger.info(f" [АППАРАТНАЯ КОНФИГУРАЦИЯ] Специфи
кация всего железа ПК определена при старте процесса ({len(hw_

```



```

tree))} компонентов)")
170     except Exception as init_err:
171         logger.debug(f"Инициализация системного коллектора: {i
nit_err}")
172
173     # Проверка критических аудитов при запуске
174     try:
175         from apps.windows.telemetry_research.audit_startup_che
cker import run_startup_audit
176         startup_audit = run_startup_audit(
177             check_integrity=True,
178             check_performance=True,
179             check_drivers=False,
180             check_eventlog=False
181         )
182         if not startup_audit.is_healthy:
183             logger.error(
184                 f"⚠ STARTUP AUDIT: Обнаружены критические про
блемы! "
185                 f"Critical: {startup_audit.critical_count}, Wa
rnings: {startup_audit.warning_count}"
186             )
187             for finding in startup_audit.findings:
188                 logger.warning(f" - [{finding['severity']}.upp
er()] {finding['domain']}: {finding['title']}")
189             elif startup_audit.warning_count > 0:
190                 logger.warning(
191                     f"⚠ STARTUP AUDIT: Предупреждения: {startup_au
dit.warning_count} "
192                     f"({startup_audit.duration_ms}ms)"
193                 )
194             else:
195                 logger.info(f"⚠ STARTUP AUDIT: Все проверки пройде
ны ({startup_audit.duration_ms}ms)")
196     except Exception as audit_err:
197         logger.debug(f"Проверка аудитов при запуске: {audit_er
r}")
198

```

```

199     # Инициализация W64/ETW сборщиков системных событий
200     w64_collector = None
201     w64_etw_collector = None
202     w64_cfg = cfg_mgr.get_config().get('w64_collector', {}) if
cfg_mgr else {}
203     if w64_cfg.get('enabled', True):
204         try:
205             from .w64_collector import AIW64Collector
206             from .w64_etw_collector import AIW64ETWCollector
207             w64_collector = AIW64Collector(
208                 storage=storage,
209                 enable_file_monitoring=w64_cfg.get('enable_fil
e_monitoring', True),
210                 enable_process_monitoring=w64_cfg.get('enable_
process_monitoring', True),
211                 enable_registry_monitoring=w64_cfg.get('enable
_registry_monitoring', True),
212                 enable_network_monitoring=w64_cfg.get('enable_
network_monitoring', True),
213                 enable_event_log_monitoring=w64_cfg.get('enabl
e_event_log_monitoring', True),
214             )
215             w64_collector.start()
216             w64_etw_collector = AIW64ETWCollector(
217                 storage=storage,
218                 enable_process_trace=w64_cfg.get('enable_proce
ss_trace', True),
219                 enable_disk_trace=w64_cfg.get('enable_disk_tra
ce', True),
220                 enable_network_trace=w64_cfg.get('enable_netwo
rk_trace', True),
221                 enable_registry_trace=w64_cfg.get('enable_regi
stry_trace', True),
222             )
223             w64_etw_collector.start()
224             logger.info(' Сборщики событий W64 и ETW запущен
ы в составе телеметрии')
225         except Exception as w64_err:

```

```

226         logger.debug(f'Не удалось запустить W64 сборщики:
    {w64_err}')
227
228     logger.info('=' * 60)
229     logger.info(f'Запущен сервис телеметрии Windows [Режим: {mode.upper()}]')
230     logger.info(f'Быстрый интервал: {interval} с | Топ процесс
    ов: {top_processes}')
231     if mode == 'hybrid':
232         logger.info(f'Периодический тяжелый опрос: каждые {heavy_interval} с')
233         logger.info(f'База данных: {db_path}')
234         logger.info('=' * 60)
235         tick = 0
236         prev_disk_io = psutil.disk_io_counters()
237         prev_net_io = psutil.net_io_counters()
238         prev_io_time = time.time()
239         last_gpu_probe_time: Optional[float] = None
240         cached_gpu_metrics: List[GpuMetrics] = []
241         last_partition_check: float = 0.0
242         cached_part_list: List[Any] = []
243         last_heavy_time = time.time()
244         last_heavy_disk_scan_time = time.time()
245         heavy_mode_start_time: Optional[float] = time.time() if mode.lower() in ('full', 'heavy') else None
246         heavy_poll_running = False
247         heavy_poll_lock = threading.Lock()
248         db_cleanup_interval_sec = cfg_mgr.get_db_cleanup_interval_seconds()
249         last_db_cleanup_time = time.time()
250         aggregation_interval_sec = getattr(cfg_mgr, 'get_aggregation_interval', lambda: 300.0)() if cfg_mgr else 300.0
251         last_aggregation_time = time.time()
252         known_pids: Dict[int, str] = {}
253         flashing_log_path = Path(resolved_log_dir) / 'flashing_processes.log'
254
255     try:

```

```

256 |         while not _stop_event.is_set():
257 |             loop_start = time.time()
258 |
259 |             # Динамическая перезагрузка параметров из config.j
son 'на лету'
260 |             cfg_mgr.check_tc_mode_auto_switch()
261 |             if cfg_mgr.check_and_reload():
262 |                 if not custom_interval_set:
263 |                     interval = cfg_mgr.get_interval_seconds()
264 |                 if not custom_heavy_interval_set:
265 |                     heavy_interval = cfg_mgr.get_heavy_interva
l_seconds()
266 |                 if not custom_top_processes_set:
267 |                     top_processes = cfg_mgr.get_effective_proc
ess_limit()
268 |                 if not custom_mode_set:
269 |                     mode = cfg_mgr.get_mode()
270 |                 heavy_disk_interval_sec = cfg_mgr.get_heavy_di
sk_scan_interval()
271 |                 db_cleanup_interval_sec = cfg_mgr.get_db_clean
up_interval_seconds()
272 |                 max_heavy_duration_sec = cfg_mgr.get_heavy_mod
e_max_duration_days() * 24.0 * 3600.0
273 |                 auto_switch_enabled = cfg_mgr.is_heavy_auto_sw
itch_enabled()
274 |                 h_collectors = cfg_mgr.get_heavy_collectors()
275 |                 logger.info(
276 |                     f" [Телеметрия CLI] Конфигурация config.
json обновлена 'на лету': "
277 |                     f"интервал={interval}с, тяжелый={heavy_int
erval}с, топ={top_processes}, режим={mode}, tel_mode={cfg_mgr.
get_telemetry_mode()}"
278 |                 )
279 |
280 |             now_dt = datetime.now(timezone.utc)
281 |             now_iso = now_dt.isoformat()
282 |             tick += 1
283 |             cpu_pct = psutil.cpu_percent(interval=None)

```



```

    mperature_celsius', None) or getattr(g, 'temperature_gpu_c', None),
312 |         ) for g in gpu_data
313 |     ]
314 |     except Exception:
315 |         cached_gpu_metrics = []
316 |         last_gpu_probe_time = now_sec
317 |         gpu_metrics_list = cached_gpu_metrics
318 |         io_elapsed = max(0.001, now_sec - prev_io_time)
319 |         cur_disk_io = psutil.disk_io_counters()
320 |         disk_read_kb = 0.0
321 |         disk_write_kb = 0.0
322 |         if cur_disk_io and prev_disk_io:
323 |             disk_read_kb = round((cur_disk_io.read_bytes -
    prev_disk_io.read_bytes) / 1024.0 / io_elapsed, 1)
324 |             disk_write_kb = round((cur_disk_io.write_bytes
    - prev_disk_io.write_bytes) / 1024.0 / io_elapsed, 1)
325 |             prev_disk_io = cur_disk_io
326 |             if not cached_part_list or now_sec - last_partition_check >= 60.0:
327 |                 try:
328 |                     cached_part_list = psutil.disk_partitions(
    all=False)
329 |                     last_partition_check = now_sec
330 |                 except Exception:
331 |                     pass
332 |             disk_partitions: List[DiskPartitionMetrics] = []
333 |             for part in cached_part_list:
334 |                 try:
335 |                     usage = psutil.disk_usage(part.mountpoint)
336 |                     disk_partitions.append(DiskPartitionMetrics(
    device=part.device, mountpoint=part.mountpoint, fstype=part.fstype,
    total_gb=round(usage.total / 1024 ** 3, 2), used_gb=round(usage.used / 1024 ** 3, 2),
    free_gb=round(usage.free / 1024 ** 3, 2), percent=usage.percent))
337 |                 except (PermissionError, OSError):
338 |                     continue
339 |             cur_net_io = psutil.net_io_counters()

```

```

340         net_recv_kb = 0.0
341         net_sent_kb = 0.0
342         if cur_net_io and prev_net_io:
343             net_recv_kb = round((cur_net_io.bytes_recv - p
rev_net_io.bytes_recv) / 1024.0 / io_elapsed, 1)
344             net_sent_kb = round((cur_net_io.bytes_sent - p
rev_net_io.bytes_sent) / 1024.0 / io_elapsed, 1)
345             prev_net_io = cur_net_io
346             prev_io_time = now_sec
347             net_metrics_list = [NetworkInterfaceMetrics(interf
ace_name='total', bytes_sent_sec=net_sent_kb * 1024.0, bytes_r
ecv_sec=net_recv_kb * 1024.0, packets_sent_sec=0.0, packets_re
cv_sec=0.0, errors_in=cur_net_io.errin if cur_net_io else 0, e
rrors_out=cur_net_io.errout if cur_net_io else 0)]
348             raw_procs = []
349             current_proc_map: Dict[int, Dict[str, Any]] = {}
350             for p in psutil.process_iter(['pid', 'name', 'memo
ry_info', 'username']):
351                 try:
352                     info = p.info
353                     pid = info.get('pid', 0)
354                     name = info.get('name') or 'unknown'
355                     if pid:
356                         current_proc_map[pid] = {'name': name,
'proc': p, 'info': info}
357                     mem_rss = (info.get('memory_info') or geta
ttr(p, 'memory_info', lambda: None()).rss if info.get('memory
_info') else 0
358                     raw_procs.append((mem_rss, p, info))
359                     except (psutil.NoSuchProcess, psutil.AccessDen
ied):
360                         continue
361
362             # Детектор вспыхивающих процессов (новые PID, заре
гистрированные на текущем тике)
363             if tick > 1:
364                 new_pids = set(current_proc_map.keys()) - set(
known_pids.keys())

```

```

365         if new_pids:
366             for n_pid in new_pids:
367                 p_entry = current_proc_map[n_pid]
368                 p_obj = p_entry['proc']
369                 p_name = p_entry['name']
370                 exe_path = 'N/A'
371                 cmdline_str = 'N/A'
372                 parent_info = 'N/A'
373                 try:
374                     exe_path = p_obj.exe()
375                 except Exception:
376                     pass
377                 try:
378                     cmdline_str = " ".join(p_obj.cmdli
ne())
379                 except Exception:
380                     pass
381                 try:
382                     ppid = p_obj.ppid()
383                     parent_name = psutil.Process(ppid)
.name() if ppid else 'Unknown'
384                     parent_info = f"{parent_name} (PID
: {ppid})"
385                 except Exception:
386                     pass
387
388                 alert_msg = (
389                     f" [ВСПЫШКА / НОВЫЙ ПРОЦЕСС] PID:
{n_pid} | {p_name} | "
390                     f"Путь: {exe_path} | Родитель: {pa
rent_info} | Команда: {cmdline_str}"
391                 )
392                 logger.warning(alert_msg)
393                 try:
394                     with open(flashes_log_path, 'a',
encoding='utf-8') as f_out:
395                         f_out.write(f"[{datetime.now()
.strftime('%Y-%m-%d %H:%M:%S')}] PID: {n_pid} | {p_name} | Exe

```



```

: {exe_path} | Parent: {parent_info} | Cmd: {cmdline_str}\n")
396         except Exception:
397             pass
398
399         known_pids = {pid: data['name'] for pid, data in c
urrent_proc_map.items()}
400
401         raw_procs.sort(key=lambda x: x[0], reverse=True)
402         top_candidates = raw_procs[:top_processes]
403         top_procs: List[ProcessMetrics] = []
404         for mem_rss, p, info in top_candidates:
405             proc_cpu = 0.0
406             try:
407                 proc_cpu = p.cpu_percent(interval=None)
408             except Exception:
409                 pass
410             mem_mb = round(mem_rss / (1024 * 1024), 1)
411             raw_user = info.get('username')
412             clean_user = raw_user if isinstance(raw_user,
str) else ''
413             top_procs.append(ProcessMetrics(pid=info.get('
pid', 0), name=info.get('name') or 'unknown', cpu_percent=proc
_cpu, memory_mb=mem_mb, threads_count=1, status='running', use
rname=clean_user))
414             snapshot = SystemSnapshot(timestamp=now_iso, hostn
ame=hostname, uptime_seconds=round(time.time() - boot_time, 1)
, cpu=cpu_metrics, memory=mem_metrics, gpus=gpu_metrics_list,
disks=disk_partitions, network=net_metrics_list, top_processes
=top_procs)
415             try:
416                 storage.save_snapshot(snapshot, top_n=top_proc
esses)
417             except Exception as db_err:
418                 logger.warning(f'Ошибка сохранения снимка в SQ
Lite: {db_err}')
419
420             # Формирование информативного пошагового лога тика
421             active_win_str = "N/A (Фоновый режим)"

```

```

422         if deep_diag:
423             try:
424                 forensics = deep_diag.collect_forensics_activity()
425                 win_info = forensics.foreground_window
426                 if win_info and win_info.get('title'):
427                     active_win_str = f"\{win_info.get('title')}\\" ({{win_info.get('process_name') or 'N/A'}}, PID:{{win_info.get('pid', 0)}}) | Простой: {{forensics.user_idle_seconds:.1f}}c"
428             except Exception:
429                 pass
430
431             disk_vol_str = ", ".join([f"{{d.mountpoint}} {{d.used_gb:.0f}}/{{d.total_gb:.0f}}GB ({{d.percent}}%)" for d in disk_partitions[:3]]) or "N/A"
432             top_procs_summary = ", ".join([f"{{p.name}} ({{p.memory_mb:.0f}}MB, {{p.cpu_percent:.1f}}% CPU)" for p in top_procs[:4]])
433
434             logger.info(f" [ТЕЛЕМЕТРИЯ #{tick}] Режим: {{mode.upper()}} | {{datetime.now().strftime('%H:%M:%S')}}")
435             logger.info(f" └─ CPU: {{cpu_pct:.1f}}% @ {{freq_mhz:.0f}}MHz ({{cpu_metrics.physical_cores}}P/{{cpu_metrics.logical_cores}}L) | RAM: {{mem_metrics.used_gb:.1f}}/{{mem_metrics.total_gb:.1f}}GB ({{mem_metrics.percent}}%)"
436             if gpu_metrics_list:
437                 g0 = gpu_metrics_list[0]
438                 logger.info(f" └─ GPU: {{g0.name}} | Нагрузка: {{g0.load_percent or 0.0}}% | Температура: {{g0.temperature_celsius or 0.0}}°C")
439                 logger.info(f" └─ Диски: Чтение {{disk_read_kb:.1f}} KB/s | Запись {{disk_write_kb:.1f}} KB/s | Занято: {{disk_vol_str}}")
440                 logger.info(f" └─ Сеть: ↓{{net_recv_kb:.1f}} KB/s | ↑{{net_sent_kb:.1f}} KB/s | Активность: {{active_win_str}}")
441                 logger.info(f" └─ Топ процессов: {{top_procs_summary}}")

```

```

442 |         logger.info(f"    └─ Бuffer БД: снимок #{tick} до
      | бавлен (в буфере: {storage.get_buffered_count()} записей, БД:
      | {Path(db_path).name})")
443 |
444 |         # Автоматическое переключение из тяжелого режима в
      | легкий при непрерывной работе 5 дней (432 000с)
445 |         if auto_switch_enabled and heavy_mode_start_time i
      | s not None and mode.lower() in ('full', 'heavy'):
446 |             if now_sec - heavy_mode_start_time >= max_heav
      | y_duration_sec:
447 |                 days_run = round((now_sec - heavy_mode_sta
      | rt_time) / (24 * 3600), 1)
448 |                 logger.warning(
449 |                     f"    [АВТО-ПЕРЕКЛЮЧЕНИЕ] Телеметрия неп
      | рывно работала {days_run} дн. в тяжелом режиме ({mode.upper(
      | )}). "
450 |                     f"Для оптимизации ресурсов режим автом
      | атически переключен в легкий (LIGHT).")
451 |                 )
452 |                 mode = 'light'
453 |                 heavy_mode_start_time = None
454 |
455 |                 is_heavy_due = mode.lower() in ('full', 'heavy') o
      | r (mode.lower() == 'hybrid' and now_sec - last_heavy_time >= h
      | eavy_interval)
456 |                 if is_heavy_due:
457 |                     last_heavy_time = now_sec
458 |
459 |                 def _execute_heavy_poll_job() -> None:
460 |                     nonlocal last_heavy_disk_scan_time, heavy_
      | poll_running
461 |                     heavy_start = time.time()
462 |                     try:
463 |                         logger.info(f"    [ТЯЖЕЛЫЙ ОПРОС] Запус
      | к периодического опроса сенсоров LHM, SMART, портов и утечек..
      | .")
464 |                         heavy_readings_total = []
465 |                         if sensor_collector and h_collectors.g

```

```

et('hardware_sensors', True):
466         heavy_readings = sensor_collector.
collect_all_sensors()
467         if heavy_readings:
468             heavy_readings_total.extend(he
avy_readings)
469             try:
470                 storage.save_sensor_polls(
heavy_readings)
471             except Exception as sp_err:
472                 logger.debug(f'Ошибка сохр
анения sensor_polls в БД: {sp_err}')
473
474             # Разгруппировка и красивый вы
вод LHM сенсоров в консоль
475             lhm_temps = [f"{s.get('sensor_
name')}: {s.get('value')}{s.get('unit')}" for s in heavy_readi
ngs if s.get('sensor_category') in ('Temperatures', 'Temperatu
re')]
476             lhm_powers = [f"{s.get('sensor
_name')}: {s.get('value')}{s.get('unit')}" for s in heavy_read
ings if s.get('sensor_category') in ('Powers', 'Power')]
477             lhm_fans = [f"{s.get('sensor_n
ame')}: {s.get('value')}{s.get('unit')}" for s in heavy_readin
gs if s.get('sensor_category') in ('Fans', 'Fan')]
478             lhm_clocks = [f"{s.get('sensor
_name')}: {s.get('value')}{s.get('unit')}" for s in heavy_read
ings if s.get('sensor_category') in ('Clocks', 'Clock')]
479
480             logger.info(f" [LHM СЕНСОРЫ]
Опрошено {len(heavy_readings)} аппаратных датчиков:")
481             if lhm_temps:
482                 logger.info(f"    └ Темпер
атуры ({len(lhm_temps)}): {' '.join(lhm_temps[:8])}")
483             if lhm_powers:
484                 logger.info(f"    └ Энерго
потребление ({len(lhm_powers)}): {' '.join(lhm_powers[:6])}")
485             if lhm_fans:

```

```

486         logger.info(f"    └─ Вентил  

яторы ({len(lhm_fans)}): {' '.join(lhm_fans)}")
487         if lhm_clocks:
488             logger.info(f"    └─ Частот  

ы ({len(lhm_clocks)}): {' '.join(lhm_clocks[:6])}")
489
490         # Детальный аудит портов, сетевых соедине  

ний и утечек дескрипторов
491         if system_collector:
492             try:
493                 ports = system_collector.get_l  

istening_ports(limit=10)
494                 if ports:
495                     p_str = ", ".join([f"{p.po  

rt}/{p.protocol} ({p.process_name or 'System'})" for p in port  

s[:5]])
496                     logger.info(f"    [ТЯЖЕЛЫЙ  

ОПРОС] Открытые слушающие порты ({len(ports)}): {p_str}")
497             except Exception:
498                 pass
499             try:
500                 net_conns = system_collector.g  

et_process_network_activity(limit=5, only_internet=True)
501                 if net_conns:
502                     c_str = ", ".join([f"{c.na  

me}->{c.remote_address} ({c.service_type})" for c in net_conns  

[:3]])
503                     logger.info(f"    [ТЯЖЕЛЫЙ  

ОПРОС] Активные соединения с Интернет ({len(net_conns)}): {c_s  

tr}")
504             except Exception:
505                 pass
506
507         if deep_diag:
508             try:
509                 leaks = deep_diag.collect_proc  

ess_leaks(limit=5)
510                 if leaks and leaks.top_handle

```

```

hogs:
511         top_h = leaks.top_handle_h
ogs[0]
512         logger.info(f" [ТЯЖЕЛЫЙ
ОПРОС] Топ процесса по дескрипторам: {top_h.name} (PID:{top_h.
pid}) -> {top_h.handles_count} handles, {top_h.gdi_objects} GD
I, {top_h.user_objects} USER")
513         except Exception:
514             pass
515
516         # Тяжелый опрос накопителей SMART и на
дежности (2 раза в день / каждые 12 часов)
517         h_now = time.time()
518         if system_collector and h_collectors.g
et('storage_smart', True) and (h_now - last_heavy_disk_scan_ti
me >= heavy_disk_interval_sec):
519             try:
520                 heavy_disks = system_collector
.get_physical_disks_health(force=True)
521                 if heavy_disks:
522                     d_summary = ", ".join([f"{
d.model} ({d.media_type}, {d.size_gb}GB, SMART:{d.health_statu
s})" for d in heavy_disks])
523                     logger.info(f" [ТЯЖЕЛЫЙ
ОПРОС (SMART)] Накопители SMART: {d_summary}")
524                     last_heavy_disk_scan_time = h_
now
525                     except Exception as d_err:
526                         logger.debug(f"Ошибка тяжелого
сканирования накопителей: {d_err}")
527
528                     heavy_dur = time.time() - heavy_start
529                     logger.info(f' [{{mode.upper()}}] Перио
дический тяжелый опрос завершен за {heavy_dur:.2f}с (Всего сен
соров: {len(heavy_readings_total)})')
530                     except Exception as h_err:
531                         logger.error(f'Ошибка периодического т
яжелого опроса: {h_err}')

```

```

532         finally:
533             with heavy_poll_lock:
534                 heavy_poll_running = False
535
536         if once:
537             _execute_heavy_poll_job()
538         else:
539             with heavy_poll_lock:
540                 if not heavy_poll_running:
541                     heavy_poll_running = True
542                     worker_thread = threading.Thread(
543                         target=_execute_heavy_poll_job
544                         ,
545                         name="TelemetryHeavyPollWorker
546                         ",
547                         daemon=True
548                     )
549                     worker_thread.start()
550
551             # Фоновый периодический контроль размера SQLite ба
552             зы данных telemetry.db
553             if now_sec - last_db_cleanup_time >= db_cleanup_in
554             terval_sec:
555                 try:
556                     prune_res = storage.enforce_size_limit()
557                     if prune_res.get('pruned'):
558                         logger.warning(
559                             f" [КОНТРОЛЬ РАЗМЕРА БД] Усечени
560                             е базы данных telemetry.db: "
561                             f"{prune_res.get('initial_size_mb'
562                             )} МБ -> {prune_res.get('final_size_mb')} МБ "
563                             f"(удалено снимков: {prune_res.get
564                             ('deleted_snapshots')})"
565                         )
566                     last_db_cleanup_time = now_sec
567                 except Exception as db_clean_err:
568                     logger.debug(f"Ошибка периодического контр
569                     оля размера БД: {db_clean_err}")

```

```

562 |
563 |         # Фоновая периодическая многоуровневая SQL-агрегация (hourly -> daily -> weekly -> monthly -> yearly)
564 |         if now_sec - last_aggregation_time >= aggregation_interval_sec:
565 |             try:
566 |                 storage.flush()
567 |                 agg_res = storage.run_aggregation_pipeline(
568 |                     h_cnt = agg_res.get('hourly', {}).get('buckets_aggregated', 0)
569 |                     sp_cnt = agg_res.get('hourly', {}).get('spikes_recorded', 0)
570 |                     if h_cnt > 0 or sp_cnt > 0:
571 |                         logger.info(f" [SQL АГРЕГАЦИЯ] Обновлено бакеты: hourly={h_cnt}, зафиксировано всплесков={sp_cnt}"
572 |                                     last_aggregation_time = now_sec
573 |                                     except Exception as agg_err:
574 |                                         logger.debug(f"Ошибка периодической SQL-агрегации: {agg_err}")
575 |
576 |             if once:
577 |                 storage.flush()
578 |                 try:
579 |                     storage.run_aggregation_pipeline()
580 |                 except Exception:
581 |                     pass
582 |                 logger.info(' Однократный опрос системной телеметрии успешно выполнен (--once)')
583 |                 break
584 |
585 |             elapsed = time.time() - loop_start
586 |             sleep_time = max(0.05, interval - elapsed)
587 |             if _stop_event.wait(timeout=sleep_time):
588 |                 logger.info('Получен сигнал остановки (_stop_event установлен)')
589 |                 break

```



```

590     except KeyboardInterrupt:
591         logger.info('Прервано пользователем (KeyboardInterrupt
    )')
592     except Exception as ex:
593         logger.error(f'Критическая ошибка в цикле телеметрии:
    {ex}', exc_info=True)
594     finally:
595         if w64_collector:
596             try:
597                 w64_collector.stop()
598             except Exception:
599                 pass
600         if w64_etw_collector:
601             try:
602                 w64_etw_collector.stop()
603             except Exception:
604                 pass
605         logger.info('Процесс телеметрии успешно завершен')
606     return 0
607
608 def run_minimal_telemetry(interval: float=5.0, top_processes:
    int=10, log_dir: Optional[str]=None) -> int:
609     """Обертка для обратной совместимости: запуск в режиме min
    imal."""
610     return run_telemetry_service(mode='minimal', interval=inte
    rval, top_processes=top_processes, log_dir=log_dir)
611
612 def main() -> int:
613     """Главная точка входа сервиса телеметрии.
614
615     Returns:
616         int: Код завершения.
617     """
618     args = parse_arguments()
619     if args.verbose:
620         logger.setLevel('DEBUG')
621     signal.signal(signal.SIGINT, signal_handler)
622     signal.signal(signal.SIGTERM, signal_handler)

```

```
623     from .telemetry_config import TelemetryConfigManager
624     try:
625         config_manager = TelemetryConfigManager(config_path=args.config)
626     except Exception as e:
627         logger.warning(f'Не удалось загрузить конфигурацию ({e}), используются значения по умолчанию')
628         config_manager = None
629     if args.minimal:
630         mode = 'minimal'
631     elif args.mode:
632         mode = args.mode
633     elif config_manager:
634         mode = config_manager.get_mode()
635     else:
636         mode = 'hybrid'
637     interval = args.interval or (config_manager.get_interval_seconds() if config_manager else 5.0)
638     heavy_interval = args.heavy_interval or (config_manager.get_heavy_interval_seconds() if config_manager else 60.0)
639     top_processes = args.top_processes or (config_manager.get_top_processes() if config_manager else 10)
640     heavy_collectors = config_manager.get_heavy_collectors() if config_manager else None
641     return run_telemetry_service(mode=mode, interval=interval, heavy_interval=heavy_interval, top_processes=top_processes, heavy_collectors=heavy_collectors, log_dir=args.log_dir, config_path=args.config, once=args.once)
642 if __name__ == '__main__':
643     sys.exit(main())
```

models.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Models
4 # =====
5 # Description:
6 #   Data models for system metrics, hardware specs, sensors, and telemetry.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.models import TelemetryProvider
11 #
12 #       service = TelemetryProvider()
13 #
14 # File: models.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 03:07:00
20 # =====
21
22 from __future__ import annotations
23 """Data models for system metrics, hardware specs, sensors, and telemetry."""
24
25 from abc import ABC, abstractmethod
26 from datetime import datetime, timezone
27 from enum import Enum
28 from typing import Any, Dict, List, Optional
29 from pydantic import BaseModel, ConfigDict, Field
30
```

```

31
32 class CpuInventoryInfo(BaseModel):
33     """Паспорт физического процессора (CPU Inventory)."""
34     processor_id: int = 0
35     name: str = ''
36     vendor: str = ''
37     architecture: str = 'x86_64'
38     physical_cores: int = 1
39     logical_cores: int = 1
40     base_frequency_mhz: float = 0.0
41     max_frequency_mhz: float = 0.0
42     l2_cache_kb: Optional[int] = None
43     l3_cache_kb: Optional[int] = None
44     socket: str = ''
45     features: List[str] = Field(default_factory=list)
46
47
48 class CpuTelemetrySample(BaseModel):
49     """Срез телеметрии нагрузки и температур процессора."""
50     timestamp: str = ''
51     created_at: float = 0.0
52     processor_id: int = 0
53     total_percent: float = 0.0
54     user_percent: float = 0.0
55     kernel_percent: float = 0.0
56     frequency_mhz: float = 0.0
57     temperature_c: Optional[float] = None
58     package_power_w: Optional[float] = None
59     core_utilization: List[float] = Field(default_factory=list
60 )
61     core_temperatures: List[float] = Field(default_factory=list
62 t)
63
64 class RamModuleInventoryInfo(BaseModel):
65     """Паспорт физического модуля (планки) оперативной памяти
66 (SPD)."""
67     slot_id: int = 0

```

```

66     bank_label: str = 'DIMM'
67     device_locator: str = ''
68     serial_number: str = ''
69     part_number: str = ''
70     manufacturer: str = ''
71     capacity_bytes: int = 0
72     capacity_gb: float = 0.0
73     speed_mhz: int = 0
74     memory_type: str = 'DDR4'
75     form_factor: str = 'DIMM'
76     configured_voltage: Optional[float] = None
77
78
79 class RamTelemetrySample(BaseModel):
80     """Срез телеметрии использования физической и виртуальной
    памяти."""
81     timestamp: str = ''
82     created_at: float = 0.0
83     total_bytes: int = 0
84     total_gb: float = 0.0
85     used_bytes: int = 0
86     used_gb: float = 0.0
87     available_bytes: int = 0
88     available_gb: float = 0.0
89     percent_used: float = 0.0
90     swap_total_gb: float = 0.0
91     swap_used_gb: float = 0.0
92     swap_percent: float = 0.0
93     pool_paged_mb: Optional[float] = None
94     pool_nonpaged_mb: Optional[float] = None
95
96
97 class GpuInventoryInfo(BaseModel):
98     """Паспорт графического ускорителя (GPU Inventory)."""
99     gpu_id: int = 0
100     pci_device_id: str = ''
101     name: str = ''
102     vendor: str = ''

```

```
103     driver_version: str = ''
104     driver_date: str = ''
105     vram_bytes: int = 0
106     vram_gb: float = 0.0
107     pci_bus_id: str = ''
108     bios_version: str = ''
109     cuda_cores: Optional[int] = None
110     directml_supported: bool = True
111
112
113 class GpuTelemetrySample(BaseModel):
114     """Срез телеметрии нагрузки, памяти и температур GPU."""
115     timestamp: str = ''
116     created_at: float = 0.0
117     gpu_id: int = 0
118     name: str = ''
119     load_percent: Optional[float] = None
120     memory_used_mb: Optional[float] = None
121     memory_total_mb: Optional[float] = None
122     memory_percent: Optional[float] = None
123     temperature_gpu_c: Optional[float] = None
124     temperature_memory_c: Optional[float] = None
125     fan_speed_pct: Optional[float] = None
126     power_draw_w: Optional[float] = None
127     clock_graphics_mhz: Optional[float] = None
128     clock_memory_mhz: Optional[float] = None
129
130
131 class NetworkAdapterInventoryInfo(BaseModel):
132     """Паспорт сетевого адаптера / интерфейса (Network Adapter
133     Inventory)."""
134     adapter_id: int = 0
135     adapter_guid: str = ''
136     name: str = ''
137     interface_name: str = ''
138     mac_address: str = ''
139     adapter_type: str = 'Ethernet'
140     is_physical: bool = True
```

```

140     is_wireless: bool = False
141     max_speed_mbps: int = 0
142     driver_name: str = ''
143     driver_version: str = ''
144
145
146 class NetworkAdapterSample(BaseModel):
147     """Срез показателей сетевой активности и ошибок адаптера."
148     """
149     timestamp: str = ''
150     created_at: float = 0.0
151     adapter_name: str = ''
152     bytes_recv_sec: float = 0.0
153     bytes_sent_sec: float = 0.0
154     packets_recv_sec: float = 0.0
155     packets_sent_sec: float = 0.0
156     errors_in_sec: float = 0.0
157     errors_out_sec: float = 0.0
158     link_speed_mbps: int = 0
159     is_connected: bool = True
160
161 class TelemetryProvider(ABC):
162     """Абстрактный базовый класс для всех компонентов системы,
163     предоставляющих телеметрию."""
164
165     @abstractmethod
166     def get_sensors(self) -> List[HardwareSensor]:
167         """Возвращает список сенсоров, предоставляемых данным
168         компонентом."""
169         pass
170
171 class CpuMetrics(BaseModel):
172     """CPU usage and architecture metrics."""
173     model: str = Field(default='', description='CPU model name')
174     architecture: str = Field(default='x86_64', description='CPU architecture')
175     physical_cores: int = Field(default=1, description='Physical cores')

```

```

    al core count')
173     logical_cores: int = Field(default=1, description='Logical
    core count (threads)')
174     total_percent: float = Field(default=0.0, description='Ove
    rall CPU usage percentage')
175     per_core_percent: List[float] = Field(default_factory=list
    , description='Usage percentage per logical core')
176     frequency_mhz: float = Field(default=0.0, description='Cur
    rent CPU frequency in MHz')
177     temperature_celsius: Optional[float] = Field(default=None,
    description='Package temperature in Celsius')
178
179 class MemoryMetrics(BaseModel):
180     """RAM and Swap memory metrics."""
181     total_gb: float = Field(default=0.0, description='Total ph
    ysical RAM in GB')
182     available_gb: float = Field(default=0.0, description='Avai
    lable RAM in GB')
183     used_gb: float = Field(default=0.0, description='Used RAM
    in GB')
184     percent: float = Field(default=0.0, description='Memory ut
    ilization percentage')
185     swap_total_gb: float = Field(default=0.0, description='Tot
    al swap space in GB')
186     swap_used_gb: float = Field(default=0.0, description='Used
    swap space in GB')
187     swap_percent: float = Field(default=0.0, description='Swap
    utilization percentage')
188
189 class GpuMetrics(BaseModel):
190     """GPU accelerator telemetry."""
191     name: str = Field(default='Unknown GPU', description='GPU
    device model name')
192     load_percent: Optional[float] = Field(default=None, descri
    ption='GPU core load percentage')
193     memory_total_gb: float = Field(default=0.0, description='T
    otal VRAM in GB')
194     memory_used_gb: float = Field(default=0.0, description='Us

```



```

    ed VRAM in GB')
195     memory_free_gb: float = Field(default=0.0, description='Fr
    ee VRAM in GB')
196     temperature_celsius: Optional[float] = Field(default=None,
    description='GPU core temperature in Celsius')
197     has_cuda: bool = Field(default=False, description='CUDA su
    pport availability')
198     has_directml: bool = Field(default=False, description='Dir
    ectML accelerator availability')
199
200 class DiskPartitionMetrics(BaseModel):
201     """Storage partition metrics."""
202     device: str = Field(default='', description='Partition mou
    nt device or drive letter')
203     mountpoint: str = Field(default='', description='Mount poi
    nt path')
204     fstype: str = Field(default='', description='Filesystem ty
    pe')
205     total_gb: float = Field(default=0.0, description='Total ca
    pacity in GB')
206     used_gb: float = Field(default=0.0, description='Used spac
    e in GB')
207     free_gb: float = Field(default=0.0, description='Free spac
    e in GB')
208     percent: float = Field(default=0.0, description='Utilizati
    on percentage')
209     volume_name: Optional[str] = Field(default=None, descripti
    on='Метка тома или описание аккаунта')
210     is_virtual: bool = Field(default=False, description='Призн
    ак виртуального/облачного тома (Google Drive, OneDrive и др.)'
    )
211     drive_type: str = Field(default='Fixed', description='Тип
    диска (Fixed, Google Drive, Cloud / Virtual, Removable)')
212
213 class DiskIoMetrics(BaseModel):
214     """Disk read and write I/O rates."""
215     read_bytes_per_sec: float = Field(default=0.0, description
    ='Read throughput in bytes/s')

```

```
216     write_bytes_per_sec: float = Field(default=0.0, description='Write throughput in bytes/s')
217     read_count_per_sec: float = Field(default=0.0, description='Read operations per second')
218     write_count_per_sec: float = Field(default=0.0, description='Write operations per second')
219
220 class NetworkAdapterStatistics(BaseModel):
221     """Накопительная статистика сетевого адаптера (Get-NetAdapterStatistics / MIB_IF_ROW2)."""
222     name: str = Field(default='', description='Имя адаптера/интерфейса')
223     received_bytes: int = Field(default=0, description='Всего получено байт')
224     sent_bytes: int = Field(default=0, description='Всего отправлено байт')
225     received_packets: int = Field(default=0, description='Всего получено пакетов')
226     sent_packets: int = Field(default=0, description='Всего отправлено пакетов')
227     received_discarded: int = Field(default=0, description='Отброшено входящих пакетов')
228     received_errors: int = Field(default=0, description='Ошибок входящих пакетов')
229     outbound_discarded: int = Field(default=0, description='Отброшено исходящих пакетов')
230     outbound_errors: int = Field(default=0, description='Ошибок исходящих пакетов')
231
232 class NetworkPerformanceCounter(BaseModel):
233     """Счётчики производительности интерфейса (Windows Performance Counters)."""
234     interface_name: str = Field(default='', description='Имя сетевого интерфейса в Performance Counters')
235     bytes_recv_per_sec: float = Field(default=0.0, description='Скорость получения (байты/сек)')
236     bytes_sent_per_sec: float = Field(default=0.0, description='Скорость отправки (байты/сек)')
```

```

237     bytes_total_per_sec: float = Field(default=0.0, description=
    n='Суммарная скорость (байты/сек)')
238
239 class AppNetworkUsageItem(BaseModel):
240     """Потребление сетевого трафика конкретным приложением/про
    цессом."""
241     process_name: str = Field(default='', description='Имя исп
    олняемого файла процесса (например, chrome.exe)')
242     pid: Optional[int] = Field(default=None, description='Иден
    тификатор процесса (если активен)')
243     rx_bytes: int = Field(default=0, description='Получено бай
    т за период')
244     tx_bytes: int = Field(default=0, description='Отправлено б
    айт за период')
245     total_bytes: int = Field(default=0, description='Всего пер
    еданного трафика в байтах')
246     rx_formatted: str = Field(default='0 B', description='Отфо
    рматированный объем полученных данных (MB/GB)')
247     tx_formatted: str = Field(default='0 B', description='Отфо
    рматированный объем отправленных данных (MB/GB)')
248     share_percent: float = Field(default=0.0, description='Дол
    я трафика от общего объема системы (%)')
249
250 class NetworkUsagePeriodReport(BaseModel):
251     """Агрегированный отчет об использовании сети за выбранный
    интервал времени (10 мин, 1 час, 24 часа)."""
252     period_minutes: int = Field(default=1440, description='Дли
    тельность периода в минутах (например, 1440 = 24 часа)')
253     total_rx_bytes: int = Field(default=0, description='Суммар
    но получено байт за период')
254     total_tx_bytes: int = Field(default=0, description='Суммар
    но отправлено байт за период')
255     total_rx_formatted: str = Field(default='0 B', description
    ='Человекочитаемый объем входящего трафика')
256     total_tx_formatted: str = Field(default='0 B', description
    ='Человекочитаемый объем исходящего трафика')
257     total_formatted: str = Field(default='0 B', description='С
    уммарный объем трафика (RX + TX)')

```

```

258     adapter_stats: List[NetworkAdapterStatistics] = Field(default_factory=list, description='Статистика по адаптерам')
259     performance_counters: List[NetworkPerformanceCounter] = Field(default_factory=list, description='Текущие Performance Counters')
260     top_apps: List[AppNetworkUsageItem] = Field(default_factory=list, description='Детализация по приложениям за период')
261     summary_text: str = Field(default='', description='Сводный текстовый отчёт для LLM/ai-diagnostics')
262
263 class NetworkInterfaceMetrics(BaseModel):
264     """Network adapter telemetry."""
265     name: str = Field(default='', description='Adapter interface name')
266     is_up: bool = Field(default=True, description='Interface link status')
267     speed_mbps: int = Field(default=0, description='Link speed in Mbps')
268     bytes_sent_per_sec: float = Field(default=0.0, description='Upload rate in bytes/s')
269     bytes_recv_per_sec: float = Field(default=0.0, description='Download rate in bytes/s')
270     ip_addresses: List[str] = Field(default_factory=list, description='Assigned IP addresses')
271     statistics: Optional[NetworkAdapterStatistics] = Field(default=None, description='Накопительная статистика адаптера')
272
273 class ProcessMetrics(BaseModel):
274     """Метрики процесса с данными об идентичности и происхождении (Process Identity & Provenance)."""
275     pid: int = Field(..., description='Process identifier')
276     name: str = Field(..., description='Process binary name')
277     status: str = Field(default='running', description='Process execution status')
278     cpu_percent: float = Field(default=0.0, description='CPU usage percentage')
279     memory_mb: float = Field(default=0.0, description='Resident memory in Megabytes')

```

```
280     memory_percent: float = Field(default=0.0, description='Me
memory usage percentage')
281     num_threads: int = Field(default=1, description='Number of
active threads')
282     num_handles: int = Field(default=0, description='Количетв
о дескрипторов (handles/fds)')
283     username: Optional[str] = Field(default=None, description=
'Owner username')
284     cmdline: Optional[str] = Field(default=None, description='
Command line invocation')
285     read_bytes_sec: float = Field(default=0.0, description='Di
sk read rate in bytes/sec')
286     write_bytes_sec: float = Field(default=0.0, description='D
isk write rate in bytes/sec')
287     # Поля безопасности и токена
288     integrity_level: Optional[str] = Field(default=None, descr
iption='Уровень целостности процесса (Low, Medium, High, Syste
m)')
289     elevation: Optional[bool] = Field(default=None, descriptio
n='Статус повышения привилегий UAC')
290     sid: Optional[str] = Field(default=None, description='Иден
тификатор безопасности пользователя (User SID)')
291     session_id: Optional[int] = Field(default=None, descriptio
n='Идентификатор сессии Windows (0=Службы, 1=Интерактивная)')
292     # Поля происхождения и родословной (Process Provenance)
293     ppid: Optional[int] = Field(default=None, description='Иде
нтификатор родительского процесса (Parent PID)')
294     parent_name: Optional[str] = Field(default=None, descripti
on='Имя родительского процесса')
295     executable_path: Optional[str] = Field(default=None, descr
iption='Полный путь к исполняемому файлу процесса')
296     creation_time: Optional[str] = Field(default=None, descrip
tion='Время создания процесса (ISO 8601 UTC)')
297     process_guid: Optional[str] = Field(default=None, descript
ion='Уникальный GUID экземпляра процесса (PID + CreationTime)
')
298     ancestor_chain: Optional[str] = Field(default=None, descri
ption='Цепочка предков запуска (напр. explorer -> pwsh -> pyth
```

```

on)')
299     launch_reason: Optional[str] = Field(default=None, descrip
tion='Определенный вектор/причина запуска (Служба, Задача, Тер
минал, Проводник)')
300
301
302 class ProcessTokenInfo(BaseModel):
303     """Информация о токене процесса для сбора security-данных.
    """
304     pid: int = Field(..., description='Идентификатор процесса'
    )
305     name: str = Field(..., description='Имя процесса')
306     elevation: Optional[bool] = Field(default=None, descriptio
n='Признак повышения привилегий (UAC)')
307     integrity_level: Optional[str] = Field(default=None, descr
iption='Уровень целостности процесса')
308     privileges: List[str] = Field(default_factory=list, descri
ption='Список привилегий токена')
309     sid: Optional[str] = Field(default=None, description='Иден
тификатор безопасности (SID)')
310     session_id: Optional[int] = Field(default=None, descriptio
n='Идентификатор сессии Windows')
311
312
313 class ProcessProvenanceInfo(BaseModel):
314     """Детальный паспорт идентичности, происхождения и цепочки
    вызовов процесса (Process Provenance)."""
315     pid: int = Field(..., description='Идентификатор процесса
    (PID)')
316     ppid: Optional[int] = Field(default=None, description='Иде
нтификатор родительского процесса (PPID)')
317     name: str = Field(..., description='Имя процесса (например
    , python.exe)')
318     executable_path: Optional[str] = Field(default=None, descr
iption='Полный путь к бинарному исполняемому файлу')
319     command_line: Optional[str] = Field(default=None, descript
ion='Полная командная строка запуска с аргументами')
320     user: Optional[str] = Field(default=None, description='Имя

```

```

    учетной записи (DOMAIN\\User)')
321     sid: Optional[str] = Field(default=None, description='Иден-
тификатор безопасности пользователя (SID)')
322     session_id: Optional[int] = Field(default=None, descriptio
n='Идентификатор сессии Windows (Session 0=Службы, Session 1+=
Пользователь)')
323     integrity_level: Optional[str] = Field(default=None, descr
iption='Уровень целостности токена (Untrusted, Low, Medium, Hi
gh, System)')
324     elevation: Optional[bool] = Field(default=None, descriptio
n='Признак повышенных привилегий администратора (Elevated)')
325     creation_time: Optional[str] = Field(default=None, descrip
tion='Время создания процесса в формате ISO 8601 UTC')
326     parent_start_time: Optional[str] = Field(default=None, des
cription='Время создания родительского процесса')
327     process_guid: str = Field(default='', description='Уникаль
ный GUID экземпляра процесса (исключает коллизии PID reuse)')
328     parent_guid: Optional[str] = Field(default=None, descripti
on='GUID родительского экземпляра процесса')
329     parent_name: Optional[str] = Field(default=None, descripti
on='Имя родительского процесса (например, WindowsTerminal.exe
)')
330     parent_cmdline: Optional[str] = Field(default=None, descri
ption='Командная строка родителя')
331     ancestor_chain: List[str] = Field(default_factory=list, de
scription='Список узлов в цепочке предков (от корня к процессу
)')
332     ancestor_chain_str: str = Field(default='', description='С
троковое представление цепочки предков (explorer → pwsh → pyth
on)')
333     who_runs_it: str = Field(default='', description='Кем испо
лняется (User / SID / Token / Session)')
334     who_created_it: str = Field(default='', description='Кто с
оздал (Родительский процесс / Дерево предков)')
335     why_was_it_created: str = Field(default='', description='П
ричина/вектор создания (Служба, Задача, Интерактивно, IDE, CLI
, Автозапуск)')
336

```

```
337
338 class ProcessLifecycleEvent(BaseModel):
339     """Событие жизненного цикла процесса (создание, завершение
    | , форензика)."""
340     event_id: str = Field(default='', description='Уникальный
    | идентификатор события')
341     timestamp: str = Field(default_factory=lambda: datetime.no
    | w(timezone.utc).isoformat(), description='Время события UTC')
342     event_type: str = Field(..., description='Тип события: Pro
    | cessCreated, ProcessTerminated, Snapshot')
343     pid: int = Field(..., description='Идентификатор процесса'
    | )
344     ppid: Optional[int] = Field(default=None, description='Иде
    | нтификатор родительского процесса')
345     process_guid: str = Field(default='', description='Уникаль
    | ный GUID процесса')
346     parent_guid: Optional[str] = Field(default=None, descripti
    | on='GUID родительского процесса')
347     name: str = Field(..., description='Имя процесса')
348     executable_path: Optional[str] = Field(default=None, descr
    | iption='Путь к исполняемому файлу')
349     command_line: Optional[str] = Field(default=None, descript
    | ion='Аргументы командной строки')
350     user: Optional[str] = Field(default=None, description='Уче
    | тная запись пользователя')
351     sid: Optional[str] = Field(default=None, description='Иден
    | тификатор безопасности (SID)')
352     session_id: Optional[int] = Field(default=None, descriptio
    | n='Идентификатор сессии')
353     integrity_level: Optional[str] = Field(default=None, descr
    | iption='Уровень целостности токена')
354     elevation: Optional[bool] = Field(default=None, descriptio
    | n='Статус повышения привилегий')
355     parent_name: Optional[str] = Field(default=None, descripti
    | on='Имя родительского процесса')
356     parent_cmdline: Optional[str] = Field(default=None, descri
    | ption='Командная строка родителя')
357     ancestor_chain: Optional[str] = Field(default=None, descri
```



```

    ption='Строка цепочки предков')
358     launch_reason: Optional[str] = Field(default=None, descrip
tion='Определенный вектор запуска')
359     source: str = Field(default='system', description='Источни
к события: w64, etw, psutil, wevtapi')
360     raw_json: Optional[str] = Field(default=None, description=
'Сырые данные события в формате JSON')
361
362
363 class ProcessProvenanceReport(BaseModel):
364     """Сводный аналитический отчет Process Provenance для AI D
iagnostics."""
365     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время формирования о
тчета')
366     total_processes: int = Field(default=0, description='Общее
число активных процессов')
367     active_provenance: List[ProcessProvenanceInfo] = Field(def
ault_factory=list, description='Список паспортов происхождения
процессов')
368     recent_events: List[ProcessLifecycleEvent] = Field(default
_factory=list, description='Недавние события создания/завершен
ия процессов')
369     summary: str = Field(default='', description='Аналитическа
я сводка для LLM/ai-diagnostics')
370 class BatteryMetrics(BaseModel):
371     """Battery and power state telemetry."""
372     has_battery: bool = Field(default=False, description='Whet
her host has battery power')
373     percent: Optional[float] = Field(default=None, description
='Battery charge percentage')
374     power_plugged: Optional[bool] = Field(default=None, descri
ption='Whether AC power is connected')
375     secs_left: Optional[int] = Field(default=None, description
='Seconds of battery remaining')
376     power_profile: str = Field(default='Balanced', description
='Active Windows power scheme')
377

```

```

378
379
380
381 class AppDiskUsageItem(BaseModel):
382     """Потребление дискового I/O конкретным приложением/процес
    сом."""
383     process_name: str = Field(default='', description='Имя исп
    олняемого файла процесса (например, chrome.exe)')
384     pid: Optional[int] = Field(default=None, description='Иден
    тификатор процесса (если активен)')
385     read_bytes: int = Field(default=0, description='Прочитано
    байт за период')
386     write_bytes: int = Field(default=0, description='Записано
    байт за период')
387     total_bytes: int = Field(default=0, description='Всего дис
    кового I/O в байтах')
388     read_formatted: str = Field(default='0 B', description='От
    форматированный объем прочитанных данных')
389     write_formatted: str = Field(default='0 B', description='О
    тформатированный объем записанных данных')
390     share_percent: float = Field(default=0.0, description='Дол
    я дискового I/O от общего объема системы (%)')
391
392 class DiskUsagePeriodReport(BaseModel):
393     """Агрегированный отчет об использовании дискового I/O и S
    MART накопителей за период."""
394     period_minutes: int = Field(default=1440, description='Дли
    тельность периода в минутах (например, 1440 = 24 часа)')
395     total_read_bytes: int = Field(default=0, description='Сумм
    арно прочитано байт за период')
396     total_write_bytes: int = Field(default=0, description='Сум
    марно записано байт за период')
397     total_read_formatted: str = Field(default='0 B', descripti
    on='Человекочитаемый объем прочитанных данных')
398     total_write_formatted: str = Field(default='0 B', descript
    ion='Человекочитаемый объем записанных данных')
399     total_formatted: str = Field(default='0 B', description='С
    уммарный объем I/O (Read + Write)')

```

```
400     top_writers: List[AppDiskUsageItem] = Field(default_factor
| y=list, description='Топ процессов по объему записи за период'
| )
401     top_readers: List[AppDiskUsageItem] = Field(default_factor
| y=list, description='Топ процессов по объему чтения за период'
| )
402     physical_disks: List[PhysicalDiskHealth] = Field(default_f
| actory=list, description='Состояние и SMART-метрики физических
| дисков')
403     smart_delta_write_bytes: Optional[int] = Field(default=Non
| e, description='Изменение объема записи в SMART за период (бай
| т)')
404     smart_delta_write_formatted: Optional[str] = Field(default
| =None, description='Отформатированное изменение записи SMART')
405     io_difference_bytes: Optional[int] = Field(default=None, d
| escription='Разница между SMART Delta и суммарным I/O процессо
| в (байт)')
406     summary_text: str = Field(default='', description='Сводный
| текстовый отчёт для LLM/ai-diagnostics')
407
408 class PhysicalDiskHealth(BaseModel):
409     """Physical drive SMART and health telemetry."""
410     device_id: str = Field(default='', description='Drive iden
| tifier or disk index')
411     model: str = Field(default='Physical Drive', description='
| Drive model name')
412     media_type: str = Field(default='SSD', description='Media
| type (NVMe, SSD, HDD)')
413     size_gb: float = Field(default=0.0, description='Drive tot
| al capacity in GB')
414     health_status: str = Field(default='Healthy', description=
| 'Drive health status (Healthy, Warning, Unhealthy)')
415     operational_status: str = Field(default='OK', description=
| 'Operational status')
416     temperature_celsius: Optional[float] = Field(default=None,
| description='Drive temperature if available')
417     interface_type: Optional[str] = Field(default=None, descri
| ption='Drive interface or bus type (NVMe, SATA, USB, etc.)')
```

```

418     lifetime_read_bytes: Optional[int] = Field(default=None, d
description='Всего прочитано накопителем за время жизни (байт)'
)
419     lifetime_write_bytes: Optional[int] = Field(default=None,
description='Всего записано накопителем за время жизни (байт)'
)
420     lifetime_read_tb: Optional[float] = Field(default=None, de
scription='Всего прочитано накопителем за время жизни (TB)')
421     lifetime_write_tb: Optional[float] = Field(default=None, d
escription='Всего записано накопителем за время жизни (TB)')
422     power_on_hours: Optional[int] = Field(default=None, descri
ption='Время наработки диска в часах')
423     wear_percentage: Optional[float] = Field(default=None, des
cription='Процент износа накопителя (0-100%)')
424
425 class RamStickInfo(BaseModel):
426     """Physical RAM stick SPD details."""
427     bank_label: str = Field(default='DIMM', description='Memor
y slot label')
428     capacity_gb: float = Field(default=0.0, description='Modul
e capacity in GB')
429     speed_mhz: int = Field(default=0, description='Configured
clock speed in MHz/MTs')
430     manufacturer: str = Field(default='Generic', description='
Memory manufacturer')
431     part_number: str = Field(default='', description='Module p
art number')
432     memory_type: str = Field(default='DDR4/DDR5', description=
'Memory technology type')
433
434 class NetworkPortMetrics(BaseModel):
435     """Active listening TCP/UDP port item."""
436     port: int = Field(..., description='Listening port number'
)
437     protocol: str = Field(default='TCP', description='Protocol
: TCP / UDP')
438     address: str = Field(default='0.0.0.0', description='Bind
IP address')

```

```
439     pid: Optional[int] = Field(default=None, description='Owni
ng process PID')
440     process_name: Optional[str] = Field(default=None, descript
ion='Owning process executable name')
441
442 class ProcessNetworkActivity(BaseModel):
443     """Сетевая активность процесса (интернет-соединения, отпра
вка и прием трафика)."""
444     pid: int = Field(default=0, description='Process ID')
445     name: str = Field(default='', description='Process executab
le name')
446     user: str = Field(default='', description='Process owner u
sername')
447     local_address: str = Field(default='', description='Local
IP:Port')
448     remote_address: str = Field(default='', description='Remot
e IP:Port')
449     remote_host: Optional[str] = Field(default=None, descripti
on='Resolved remote hostname or service name')
450     protocol: str = Field(default='TCP', description='Protocol
: TCP / UDP')
451     status: str = Field(default='ESTABLISHED', description='Co
nnection state (ESTABLISHED, LISTEN, etc.)')
452     is_internet: bool = Field(default=True, description='Wheth
er remote address is external Internet vs local loopback')
453     service_type: str = Field(default='Web/HTTPS', description
='Recognized protocol service: HTTPS, HTTP, DNS, Cloud Sync, e
tc.')
454     sent_kb: float = Field(default=0.0, description='Всего отп
равлено/записано процессом (КБ)')
455     recv_kb: float = Field(default=0.0, description='Всего пол
учено/скачано/прочитано процессом (КБ)')
456     delta_sent_kb: float = Field(default=0.0, description='Объ
ем отправленных данных за последний измеряемый период (КБ)')
457     delta_recv_kb: float = Field(default=0.0, description='Объ
ем скачанных/полученных данных за последний измеряемый период
(КБ)')
458     sent_rate_kbs: float = Field(default=0.0, description='Ско
```

```

    рость отдачи/отправки данных (КБ/с)')
459     recv_rate_kbs: float = Field(default=0.0, description='Ско
    рость скачивания/приема данных (КБ/с)')
460     sent_summary: str = Field(default='', description='Человек
    опонятное описание отправляемого (что шлет)')
461     recv_summary: str = Field(default='', description='Человек
    опонятное описание принимаемого (что принимает)')
462
463 class SystemHealthAlerts(BaseModel):
464     """System reliability and event log alerts."""
465     reboot_pending: bool = Field(default=False, description='W
    hether a system reboot is pending')
466     critical_events_count: int = Field(default=0, description=
    'Critical event logs in last 24h')
467     latest_alert: str = Field(default='Система стабильна', des
    cription='Summary of latest health event')
468
469 class HardwareSensor(BaseModel):
470     """Hardware sensor reading (AIDA64 style) with provider tr
    acking for deduplication."""
471     model_config = ConfigDict(extra='allow')
472
473     sensor_id: str = Field(..., description='Unique sensor ide
    ntifier')
474     name: str = Field(..., description='Human readable sensor
    name')
475     category: str = Field(default='temperature', description='
    Category: temperature, fan, voltage, power')
476     value: float = Field(..., description='Current numeric sen
    sor reading')
477     unit: str = Field(default='°C', description='Measurement u
    nit')
478     min_value: Optional[float] = Field(default=None, descripti
    on='Recorded minimum')
479     max_value: Optional[float] = Field(default=None, descripti
    on='Recorded maximum')
480     provider: Optional[Any] = Field(default=None, description=
    'Provider name or ID for deduplication (SensorProvider enum or

```

```

    int/str)')
481
482 class HardwareNode(BaseModel):
483     """AIDA64-like hardware component tree item."""
484     category: str = Field(..., description='Component category
(Motherboard, CPU, GPU, Memory, Storage)')
485     name: str = Field(..., description='Device or component na
me')
486     properties: Dict[str, Any] = Field(default_factory=dict, d
escription='Detailed hardware properties')
487     children: List[HardwareNode] = Field(default_factory=list,
description='Sub-components or devices')
488
489 class MonitorInfo(BaseModel):
490     """Connected display monitor details."""
491     device: str = Field(default='', description='Display devic
e name (e.g. \\.\.\DISPLAY1)')
492     name: str = Field(default='Monitor', description='Friendly
monitor name or model')
493     adapter: str = Field(default='', description='Display adap
ter name')
494     width: int = Field(default=1920, description='Display reso
lution width')
495     height: int = Field(default=1080, description='Display res
olution height')
496     frequency_hz: int = Field(default=60, description='Refresh
rate in Hz')
497     bits_per_pixel: int = Field(default=32, description='Color
bit depth')
498     is_primary: bool = Field(default=False, description='Wheth
er this is the primary display')
499
500 class WindowsUpdateInfo(BaseModel):
501     """Windows Update status and recent hotfixes."""
502     status: str = Field(default='Up to date', description='Ove
rall update status')
503     installed_kb_count: int = Field(default=0, description='To
tal installed KBs')

```

```

504     recent_hotfixes: List[str] = Field(default_factory=list, d
description='Recent KB identifiers')
505     latest_installed_on: Optional[str] = Field(default=None, d
description='Date of most recent update')
506
507 class OfficeSuiteInfo(BaseModel):
508     """Microsoft Office and productivity suite detection telem
etry."""
509     installed: bool = Field(default=False, description='Whethe
r an Office suite is installed')
510     product_name: Optional[str] = Field(default=None, descript
ion='Product suite name (e.g. Microsoft 365, Office 2021)')
511     version: Optional[str] = Field(default=None, description='
Office suite release version string')
512     publisher: Optional[str] = Field(default=None, description
='Software vendor/publisher name')
513     status: str = Field(default='Не установлен', description='
Human-readable office status summary')
514
515 class CloudStorageInfo(BaseModel):
516     """Cloud drive and synchronization storage telemetry (OneD
rive)."""
517     installed: bool = Field(default=False, description='Whethe
r cloud storage drive is detected and active')
518     name: str = Field(default='OneDrive', description='Cloud s
torage service name')
519     path: Optional[str] = Field(default=None, description='Loc
al synchronized root folder path')
520     total_gb: Optional[float] = Field(default=None, descriptio
n='Total storage volume capacity in GB')
521     used_gb: Optional[float] = Field(default=None, description
='Used space in GB')
522     free_gb: Optional[float] = Field(default=None, description
='Free available space in GB')
523     percent_used: Optional[float] = Field(default=None, descri
ption='Storage disk utilization percentage')
524     status: str = Field(default='Не настроено', description='H
uman-readable cloud storage status')

```



```
525 |
526 | class SystemSnapshot(BaseModel):
527 |     """Complete system and hardware telemetry snapshot."""
528 |     timestamp: str = Field(default_factory=lambda: datetime.now(
529 |         timezone.utc).isoformat(), description='ISO 8601 UTC timestamp of snapshot')
530 |     hostname: str = Field(default='', description='Machine host name')
531 |     username: str = Field(default='', description='Current logged in username')
532 |     os_name: str = Field(default='Windows', description='Operating system name and version')
533 |     os_build: str = Field(default='', description='Operating system build number')
534 |     system_language: str = Field(default='', description='System UI language')
535 |     os_install_language: str = Field(default='', description='Original OS install language')
536 |     user_locale: str = Field(default='', description='User locale')
537 |     system_locale: str = Field(default='', description='System locale')
538 |     timezone: str = Field(default='', description='System time zone')
539 |     codepage: str = Field(default='', description='Active system code pages')
540 |     input_languages: List[str] = Field(default_factory=list, description='Installed keyboard input layouts')
541 |     os_install_date: str = Field(default='', description='Operating system installation date')
542 |     uptime_seconds: float = Field(default=0.0, description='System uptime in seconds')
543 |     cpu: CpuMetrics = Field(default_factory=CpuMetrics, description='CPU metrics')
544 |     memory: MemoryMetrics = Field(default_factory=MemoryMetrics, description='RAM and Swap metrics')
545 |     ram_sticks: List[RamStickInfo] = Field(default_factory=list, description='Installed physical RAM modules')
```

```
545     gpus: List[GpuMetrics] = Field(default_factory=list, descr
    iption='Detected GPU accelerators')
546     monitors: List[MonitorInfo] = Field(default_factory=list,
    description='Connected display monitors')
547     updates: WindowsUpdateInfo = Field(default_factory=Windows
    UpdateInfo, description='Windows Update information')
548     office: OfficeSuiteInfo = Field(default_factory=OfficeSuit
    eInfo, description='Office suite telemetry')
549     onedrive: CloudStorageInfo = Field(default_factory=CloudSt
    orageInfo, description='OneDrive cloud storage telemetry')
550     disks: List[DiskPartitionMetrics] = Field(default_factory=
    list, description='Disk partitions')
551     physical_disks: List[PhysicalDiskHealth] = Field(default_f
    actory=list, description='Physical storage disks and SMART hea
    lth')
552     disk_io: DiskIoMetrics = Field(default_factory=DiskIoMetri
    cs, description='Aggregate disk I/O rates')
553     network: List[NetworkInterfaceMetrics] = Field(default_fac
    tory=list, description='Network interfaces')
554     listening_ports: List[NetworkPortMetrics] = Field(default_
    factory=list, description='Active listening ports and sockets'
    )
555     network_activity: List[ProcessNetworkActivity] = Field(def
    ault_factory=list, description='Сетевая активность программ (с
    оединения и трафик)')
556     battery: BatteryMetrics = Field(default_factory=BatteryMet
    rics, description='Battery and power state')
557     alerts: SystemHealthAlerts = Field(default_factory=SystemH
    ealthAlerts, description='System health and reliability alerts
    ')
558     sensors: List[HardwareSensor] = Field(default_factory=list
    , description='Hardware sensor readings')
559     top_processes: List[ProcessMetrics] = Field(default_factor
    y=list, description='Top active processes')
560     hardware_audit: Dict[str, Any] = Field(default_factory=dic
    t, description='Full hardware audit data (manufacturer, serial
    , etc.)')
561
```

```

562 class AnomalyItem(BaseModel):
563     """Specific detected anomaly or performance bottleneck."""
564     subsystem: str = Field(..., description='Subsystem (CPU, R
    AM, GPU, Disk, Network, Process)')
565     severity: str = Field(default='warning', description='Seve
    rity level: info, warning, critical')
566     title: str = Field(..., description='Brief anomaly summary
    ')
567     description: str = Field(..., description='Detailed diagno
    stic description')
568
569 class SystemDiagnosticReport(BaseModel):
570     """AI-powered diagnostic audit report."""
571     timestamp: str = Field(default_factory=lambda: datetime.no
    w(timezone.utc).isoformat(), description='Audit timestamp')
572     health_score: int = Field(default=100, description='System
    health score (0-100)')
573     summary: str = Field(..., description='High-level health a
    nd performance summary')
574     anomalies: List[AnomalyItem] = Field(default_factory=list,
    description='List of detected anomalies')
575     recommendations: List[str] = Field(default_factory=list, d
    escription='Actionable optimization suggestions')
576     ai_model_used: str = Field(default='heuristic', descriptio
    n='AI Model identifier or heuristic engine')
577     system_instruction: Optional[str] = Field(default=None, de
    scription='System instruction used for AI model')
578     generated_prompt: Optional[str] = Field(default=None, desc
    ription='Exact prompt sent to AI model')
579     raw_response: Optional[str] = Field(default=None, descript
    ion='Raw response text from AI provider')
580     error: Optional[str] = Field(default=None, description='Er
    ror details if inference failed')
581     stages: List[Dict[str, Any]] = Field(default_factory=list,
    description='Step-by-step audit stages')
582
583 class DriverInfo(BaseModel):
584     """Сведения об установленном драйвере устройства и его акт

```

```

уальности. """
585     name: str = Field(default='', description='Имя драйвера ил
и сервиса')
586     driver_version: str = Field(default='', description='Верси
я установленного драйвера')
587     driver_date: Optional[str] = Field(default=None, descripti
on='Дата релиза драйвера (ISO или YYYY-MM-DD)')
588     provider: str = Field(default='Unknown', description='Пост
авщик драйвера (NVIDIA, Intel, Microsoft и др.)')
589     inf_name: Optional[str] = Field(default=None, description=
'Имя INF-файла драйвера (oem*.inf)')
590     is_signed: bool = Field(default=True, description='Наличие
цифровой подписи WHQL')
591     is_inbox: bool = Field(default=False, description='Являетс
я ли драйвер стандартным (inbox) от Microsoft')
592     age_days: Optional[int] = Field(default=None, description=
'Возраст драйвера в днях')
593     currency_status: str = Field(default='Актуален', descripti
on='Статус актуальности: Актуален, Устарел, Базовый драйвер ОС
')
594
595 class HardwareDeviceAudit(BaseModel):
596     """Единица аудита аппаратного устройства с драйверами, дат
ами и сенсорами."""
597     device_id: str = Field(..., description='Уникальный иденти
фикатор инстанса устройства (PnP Instance ID)')
598     name: str = Field(..., description='Понятное наименование
устройства')
599     device_class: str = Field(default='Device', description='К
ласс устройства (Display, Processor, Net, DiskDrive и др.)')
600     manufacturer: str = Field(default='Unknown', description='
Производитель оборудования')
601     install_date: Optional[str] = Field(default=None, descripti
on='Дата первой/текущей установки устройства в системе')
602     status: str = Field(default='OK', description='Рабочий ста
тус устройства (OK, Problem, Error, Disabled)')
603     problem_code: int = Field(default=0, description='Код ошиб
ки PnP (0 если исправно, 10, 43, 28 при сбое)')

```

```
604     driver: Optional[DriverInfo] = Field(default=None, descrip
tion='Сведения об используемом драйвере')
605     sensors: List[HardwareSensor] = Field(default_factory=list
, description='Сенсоры, привязанные к данному оборудованию')
606     properties: Dict[str, Any] = Field(default_factory=dict, d
escription='Дополнительные аппаратные параметры')
607
608 class HardwareChangeItem(BaseModel):
609     """Зафиксированное изменение в аппаратной конфигурации или
драйверах."""
610     change_type: str = Field(..., description='Тип изменения:
added, removed, driver_updated, status_changed, hardware_alter
ed')
611     device_id: str = Field(..., description='Идентификатор зат
ронутного устройства')
612     device_name: str = Field(..., description='Наименование ус
тройства')
613     description: str = Field(..., description='Подробное описа
ние изменения на русском языке')
614     previous_value: Optional[Dict[str, Any]] = Field(default=N
one, description='Предыдущее состояние параметра')
615     current_value: Optional[Dict[str, Any]] = Field(default=No
ne, description='Новое состояние параметра')
616     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Временная метка обна
ружения изменения')
617
618 class HardwareAuditReport(BaseModel):
619     """Полный отчет аудита аппаратного обеспечения с привязкой
сенсоров и драйверов."""
620     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время проведения ауд
ита (UTC)')
621     devices_count: int = Field(default=0, description='Общее к
оличество обнаруженных устройств')
622     problem_devices_count: int = Field(default=0, description=
'Количество устройств с ошибками PnP')
623     outdated_drivers_count: int = Field(default=0, description
```

```

        = 'Количество устройств с устаревшими драйверами')
624     devices: List[HardwareDeviceAudit] = Field(default_factory
        = list, description='Список проаудированных устройств')
625     summary: str = Field(default='', description='Сводка состо
        яния аппаратной части')
626     changes_since_last_archive: List[HardwareChangeItem] = Fie
        ld(default_factory=list, description='Изменения, обнаруженные
        по сравнению с последним сохраненным архивом')
627
628 class HardwareArchiveEntry(BaseModel):
629     """Запись в архиве истории оборудования."""
630     archive_id: str = Field(..., description='Уникальный идент
        ификатор архивного снимка')
631     timestamp: str = Field(..., description='Время создания ар
        хива')
632     devices_count: int = Field(default=0, description='Количес
        тво устройств в архиве')
633     changes_count: int = Field(default=0, description='Количес
        тво изменений, зафиксированных в этом архиве')
634     report: HardwareAuditReport = Field(..., description='Полн
        ый отчет аудита оборудования')
635
636 class ProcessLeakItem(BaseModel):
637     """Метрики скрытой диагностики и утечек отдельного процесс
        а."""
638     pid: int = Field(..., description='Идентификатор процесса'
        )
639     name: str = Field(..., description='Имя процесса')
640     status: str = Field(default='running', description='Статус
        процесса')
641     handles_count: int = Field(default=0, description='Количес
        тво открытых дескрипторов (Handles)')
642     gdi_objects: int = Field(default=0, description='Количеств
        о GDI-объектов (шрифты, битмапы, кисти, лимит 10 000)')
643     user_objects: int = Field(default=0, description='Количес
        тво USER-объектов (окна, меню, курсоры)')
644     page_faults_total: int = Field(default=0, description='Общ
        ее число страничных ошибок (Page Faults)')

```

```

645     peak_working_set_mb: float = Field(default=0.0, description=
    n='Пиковый рабочий набор памяти в МБ')
646     memory_mb: float = Field(default=0.0, description='Текущая
    память (RSS) в МБ')
647     threads_count: int = Field(default=1, description='Количес
    тво потоков процесса')
648     cpu_percent: float = Field(default=0.0, description='Нагру
    зка на CPU %')
649     leak_risk_score: str = Field(default='normal', description=
    ='Уровень риска утечки: normal, warning, critical')
650     leak_risk_reasons: List[str] = Field(default_factory=list,
    description='Список признаков аномального поведения')
651
652 class ProcessLeakDiagnosticsReport(BaseModel):
653     """Сводный отчет скрытой диагностики процессов, утечек дес
    крипторов и ресурсов UI."""
654     timestamp: str = Field(default_factory=lambda: datetime.no
    w(timezone.utc).isoformat(), description='Время сбора среза')
655     total_processes: int = Field(default=0, description='Всего
    проанализировано процессов')
656     suspicious_count: int = Field(default=0, description='Коли
    чество процессов с признаками утечки')
657     top_handle_hogs: List[ProcessLeakItem] = Field(default_fac
    tory=list, description='Топ процессов по открытым дескрипторам
    ')
658     top_gdi_hogs: List[ProcessLeakItem] = Field(default_factor
    y=list, description='Топ процессов по GDI объектам')
659     top_page_fault_hogs: List[ProcessLeakItem] = Field(default
    _factory=list, description='Топ процессов по обращениям к файл
    у подкачки')
660     all_processes: List[ProcessLeakItem] = Field(default_facto
    ry=list, description='Полный список проанализированных процесс
    ов')
661
662 class ForensicsActivityReport(BaseModel):
663     """Поведенческая телеметрия и форензика активности пользо
    вателя."""
664     timestamp: str = Field(default_factory=lambda: datetime.no

```

```

w(timezone.utc).isoformat(), description='Время фиксации')
665     foreground_window: Dict[str, Any] = Field(default_factory=
dict, description='Активное окно в фокусе (заголовок, процесс,
PID, время)')
666     user_idle_seconds: float = Field(default=0.0, description=
'Время бездействия пользователя (без ввода с клавиатуры/мыши)'
)
667     camera_active_apps: List[Dict[str, Any]] = Field(default_f
actory=list, description='Приложения, использующие или недавно
вызывавшие камеру')
668     microphone_active_apps: List[Dict[str, Any]] = Field(defau
lt_factory=list, description='Приложения, использующие микрофо
н')
669     userassist_top_apps: List[Dict[str, Any]] = Field(default_
factory=list, description='История запусков из реестра UserAss
ist (счетчик и фокусное время)')
670
671 class KernelThrottlingReport(BaseModel):
672     """Качество работы ядра, прерываний и аппаратного троттлин
га."""
673     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время среза')
674     dpc_latency_pct: float = Field(default=0.0, description='П
роцент времени ЦП в DPC (отложенных вызовах процедур)')
675     interrupt_latency_pct: float = Field(default=0.0, descript
ion='Процент времени ЦП в аппаратных прерываниях (ISR)')
676     dpc_status: str = Field(default='optimal', description='Ст
атус латентности прерываний: optimal, elevated, severe')
677     thermal_throttling_detected: bool = Field(default=False, d
escription='Обнаружен термический троттлинг ядер (PROCHOT)')
678     power_limit_throttling_detected: bool = Field(default=Fals
e, description='Обнаружено ограничение по питанию (PL1/PL2 Lim
it)')
679     system_uptime_seconds: float = Field(default=0.0, descript
ion='Время непрерывной работы системы (Uptime)')
680     uptime_formatted: str = Field(default='0 дн 0 ч', descript
ion='Отформатированное время Uptime')
681     last_bsod_crashes: List[Dict[str, Any]] = Field(default_fa

```



```

        ctory=list, description='Последние зафиксированные падения сис
        темы (BugCheck / Minidump)')
682     gpu_pcie_link: Dict[str, Any] = Field(default_factory=dict
        , description='Режим шины PCIe для видеокарты (Link Speed / Wi
        dth)')
683
684 class StorageBatteryWearReport(BaseModel):
685     """Телеметрия износа накопителей SSD/NVMe и батареи питани
        я."""
686     timestamp: str = Field(default_factory=lambda: datetime.no
        w(timezone.utc).isoformat(), description='Время формирования')
687     disks_wear: List[Dict[str, Any]] = Field(default_factory=l
        ist, description='Метрики износа SSD, TBW и здоровье накопител
        ей')
688     battery_wear: Dict[str, Any] = Field(default_factory=dict,
        description='Глубокая телеметрия износа аккумулятора (Design
        vs Full Charge)')
689
690 class PeripheralsNetworkReport(BaseModel):
691     """Телеметрия сети (Wi-Fi, интерфейсы) и периферийных устр
        ойств (USB, Audio)."""
692     timestamp: str = Field(default_factory=lambda: datetime.no
        w(timezone.utc).isoformat(), description='Время сбора')
693     usb_devices: List[Dict[str, Any]] = Field(default_factory=
        list, description='Подключенные USB-устройства и контроллеры')
694     wifi_telemetry: Dict[str, Any] = Field(default_factory=dic
        t, description='Параметры текущей беспроводной сети Wi-Fi (SSI
        D, BSSID, RSSI, канал)')
695     audio_endpoints: List[Dict[str, Any]] = Field(default_fact
        ory=list, description='Аудиоустройства и активные конечные точ
        ки')
696
697 class SystemCoreMetrics(BaseModel):
698     """Быстрые базовые метрики хоста (CPU, RAM, GPU, Disk I/O,
        Battery)."""
699     cpu: CpuMetrics
700     memory: MemoryMetrics
701     gpus: List[GpuMetrics] = Field(default_factory=list, descr

```

```

    702         disk_io: DiskIoMetrics
    703         battery: BatteryMetrics
    704         uptime_seconds: float = Field(default=0.0, description='Вр
        емя непрерывной работы в секундах')
    705
    706 class SystemHardwareQuick(BaseModel):
    707     """Сводка состояния физических компонентов (диски, слоты п
        амяти, открытые сокет)."""
    708     ram_sticks: List[RamStickInfo] = Field(default_factory=lis
        t, description='Конфигурация планок RAM (SPD)')
    709     physical_disks: List[PhysicalDiskHealth] = Field(default_f
        actory=list, description='Состояние физических накопителей (SM
        ART)')
    710     listening_ports: List[NetworkPortMetrics] = Field(default_
        factory=list, description='Список активных слушающих сокетов')
    711     alerts: SystemHealthAlerts = Field(default_factory=SystemH
        ealthAlerts, description='Системные предупреждения и флаг пере
        загрузки')
    712
    713 class W64SystemEvent(BaseModel):
    714     """Модель отдельного системного события AI W64 Collector."
        ""
    715     event_id: str = Field(default='', description='Уникальный
        идентификатор события')
    716     timestamp: str = Field(default_factory=lambda: datetime.no
        w(timezone.utc).isoformat(), description='Время события UTC')
    717     event_type: str = Field(..., description='Тип события (pro
        cess_start, file_modified, registry_change, network_open и др.
        )')
    718     path: Optional[str] = Field(default=None, description='Пут
        ь к файлу или ветке реестра')
    719     pid: Optional[int] = Field(default=None, description='Иден
        тификатор процесса')
    720     name: Optional[str] = Field(default=None, description='Имя
        процесса/файла/параметра')
    721     details: Dict[str, Any] = Field(default_factory=dict, desc
        ription='Дополнительные атрибуты события')

```

```

722
723     def to_dict(self) -> Dict[str, Any]:
724         """Сериализация события в словарь."""
725         res = self.model_dump()
726         if self.details:
727             res.update(self.details)
728         return res
729
730 class ETWTraceEvent(BaseModel):
731     """Модель низкоуровневого события ETW (Event Tracing for W
732     indows)."""
733     event_id: str = Field(default='', description='Уникальный
734     идентификатор события')
735     timestamp: str = Field(default_factory=lambda: datetime.no
736     w(timezone.utc).isoformat(), description='Время события UTC')
737     event_type: str = Field(..., description='Тип ETW трассиро
738     вки (etw_process_create, etw_file_io, etw_network и др.)')
739     provider: str = Field(default='Microsoft-Windows-Security-
740     Auditing', description='Провайдер ETW')
741     payload: Dict[str, Any] = Field(default_factory=dict, desc
742     ription='Сырые поля события из ETW / Windows Event Log')
743
744     def to_dict(self) -> Dict[str, Any]:
745         """Сериализация ETW события в словарь."""
746         return self.model_dump()
747
748 class W64CollectorStatus(BaseModel):
749     """Статус и статистика работы сборщиков W64/ETW."""
750     running: bool = Field(default=False, description='Флаг акт
751     ивности сборщика')
752     events_count: int = Field(default=0, description='Общее ко
753     личество зафиксированных событий')
754     log_dir: str = Field(default='', description='Директория х
755     ранения JSONL логов')
756     last_event_time: Optional[str] = Field(default=None, descr
757     iption='Время последнего зафиксированного события')
758
759

```

```
750 class SamplingMode(str, Enum):
751     """Режимы сбора телеметрии и адаптивного масштабирования ч
    астоты."""
752     NORMAL = 'normal'
753     MONITORING = 'monitoring'
754     BOOT_AGGRESSIVE = 'boot_aggressive'
755     INCIDENT = 'incident'
756     FORENSIC = 'forensic'
757
758
759 class SystemMetricRollup(BaseModel):
760     """Сжатый временной срез системных метрик с расширенной ст
    атистикой (avg, min, max, p95, total)."""
761     period_start: str = Field(..., description='Начало периода
    агрегации (ISO timestamp)')
762     period_end: str = Field(..., description='Окончание период
    а агрегации (ISO timestamp)')
763     duration_seconds: float = Field(default=60.0, description='
    Длительность интервала в секундах')
764     sample_count: int = Field(default=0, description='Количес
    тво объединенных замеров')
765     tier: str = Field(default='1m', description='Уровень сжати
    я: 1s, 2s, 5s, 10s, 30s, 1m, 5m, 15m, 1h, 6h, 24h')
766
767     # CPU
768     cpu_avg: float = Field(default=0.0, description='Средняя н
    агрузка CPU %')
769     cpu_min: float = Field(default=0.0, description='Минимальн
    ая нагрузка CPU %')
770     cpu_max: float = Field(default=0.0, description='Пиковая н
    агрузка CPU %')
771     cpu_p95: float = Field(default=0.0, description='95-й перц
    ентиль нагрузки CPU %')
772
773     # RAM
774     ram_avg_gb: float = Field(default=0.0, description='Средни
    й объем занятой RAM в GB')
775     ram_min_gb: float = Field(default=0.0, description='Минима
```

```
    льный объем занятой RAM в GB')
776     ram_max_gb: float = Field(default=0.0, description='Максим
    альный объем занятой RAM в GB')
777     ram_percent_avg: float = Field(default=0.0, description='С
    редний процент занятой памяти')
778     ram_percent_max: float = Field(default=0.0, description='М
    аксимальный процент занятой памяти')
779
780     # Disk
781     disk_read_avg_mbs: float = Field(default=0.0, description=
    'Средняя скорость чтения с диска в МБ/с')
782     disk_read_max_mbs: float = Field(default=0.0, description=
    'Пиковая скорость чтения с диска в МБ/с')
783     disk_read_total_mb: float = Field(default=0.0, description
    ='Суммарно прочитано за период в МБ')
784     disk_write_avg_mbs: float = Field(default=0.0, description
    ='Средняя скорость записи на диск в МБ/с')
785     disk_write_max_mbs: float = Field(default=0.0, description
    ='Пиковая скорость записи на диск в МБ/с')
786     disk_write_total_mb: float = Field(default=0.0, descriptio
    n='Суммарно записано за период в МБ')
787
788     # Network
789     network_rx_avg_mbs: float = Field(default=0.0, description
    ='Средняя скорость входящего трафика в МБ/с')
790     network_rx_max_mbs: float = Field(default=0.0, description
    ='Пиковая скорость входящего трафика в МБ/с')
791     network_rx_total_mb: float = Field(default=0.0, descriptio
    n='Суммарно получено входящего трафика в МБ')
792     network_tx_avg_mbs: float = Field(default=0.0, description
    ='Средняя скорость исходящего трафика в МБ/с')
793     network_tx_max_mbs: float = Field(default=0.0, description
    ='Пиковая скорость исходящего трафика в МБ/с')
794     network_tx_total_mb: float = Field(default=0.0, descriptio
    n='Суммарно отправлено исходящего трафика в МБ')
795
796
797 class ProcessMetricRollup(BaseModel):
```

```
798     """Сжатый временной срез per-process метрик за период набл
| юдения."""
799     name: str = Field(..., description='Имя исполняемого файла
| процесса')
800     pid: Optional[int] = Field(default=None, description='PID
| процесса (если отслеживался конкретный экземпляр)')
801     period_start: str = Field(..., description='Начало периода
| (ISO timestamp)')
802     period_end: str = Field(..., description='Конец периода (I
| SO timestamp)')
803     duration_seconds: float = Field(default=60.0, description=
| 'Длительность наблюдения')
804     sample_count: int = Field(default=0, description='Число за
| меров')
805     path: Optional[str] = Field(default=None, description='Пут
| ь к исполняемому файлу')
806
807     cpu_avg: float = Field(default=0.0, description='Средний C
| PU %')
808     cpu_max: float = Field(default=0.0, description='Пиковый C
| PU %')
809     cpu_p95: float = Field(default=0.0, description='95-й перц
| ентиль CPU %')
810
811     ram_avg_mb: float = Field(default=0.0, description='Средни
| й объем RAM в МБ')
812     ram_max_mb: float = Field(default=0.0, description='Пиковы
| й объем RAM в МБ')
813
814     disk_read_total_mb: float = Field(default=0.0, description
| ='Суммарно прочитано с диска в МБ')
815     disk_write_total_mb: float = Field(default=0.0, descriptio
| n='Суммарно записано на диск в МБ')
816     network_rx_total_mb: float = Field(default=0.0, descriptio
| n='Суммарно получено по сети в МБ')
817     network_tx_total_mb: float = Field(default=0.0, descriptio
| n='Суммарно отправлено по сети в МБ')
818
```

```

819
820 class TelemetryIncident(BaseModel):
821     """Модель диагностического инцидента аномалии с привязкой
контекста и кольцевого буфера."""
822     incident_id: str = Field(..., description='Уникальный иден
тификатор инцидента (напр. INC-20261001-01)')
823     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время детекции инцид
ента')
824     trigger_type: str = Field(..., description='Тип триггера:
cpu_spike, disk_burst, network_burst, suspicious_process, dpc_
degradation, defender_alert')
825     severity: str = Field(default='warning', description='Крит
ичность: info, warning, critical')
826     title: str = Field(..., description='Краткое название инци
дента на русском языке')
827     description: str = Field(default='', description='Подробно
е описание выявленной аномалии')
828
829     trigger_metrics: Dict[str, Any] = Field(default_factory=dict, description='Значения метрик в момент срабатывания триггера')
830     suspect_processes: List[Dict[str, Any]] = Field(default_factory=list, description='Список подозрительных процессов, связанных с аномалией')
831     related_events: List[Dict[str, Any]] = Field(default_factory=list, description='Дискретные события (запуск процессов, ETW, Defender) за период')
832     metrics_summary: Dict[str, Any] = Field(default_factory=dict, description='Агрегированная сводка метрик до и после инцидента')
833     raw_window: List[Dict[str, Any]] = Field(default_factory=list, description='Срез сырых секундных замеров из кольцевого буфера (pre/post window)')
834
835
836 class ShutdownType(str, Enum):
837     """Типы завершения работы и перезагрузки Windows."""

```

```
838     PLANNED = 'PLANNED'
839     UNEXPECTED = 'UNEXPECTED'
840     CRASH_BSOD = 'CRASH_BSOD'
841     UPDATE_RESTART = 'UPDATE_RESTART'
842     USER_INITIATED = 'USER_INITIATED'
843     HARDWARE_POWER_BUTTON = 'HARDWARE_POWER_BUTTON'
844     DIRTY_POWER_LOSS = 'DIRTY_POWER_LOSS'
845     HYBRID_SHUTDOWN = 'HYBRID_SHUTDOWN'
846     UNKNOWN = 'UNKNOWN'
847
848
849 class RebootCorrelatedEvent(BaseModel):
850     """Событие, связанное с перезагрузкой или завершением рабо
ты."""
851     event_id: int = Field(..., description='Event ID (например
, 1074, 41, 6008, 1001)')
852     provider: str = Field(..., description='Источник/провайдер
события (User32, Kernel-Power, EventLog и др.)')
853     timestamp: str = Field(..., description='Время регистрации
события в формате ISO/строка')
854     level: str = Field(default='Information', description='Уро
вень события (Critical, Error, Warning, Information)')
855     description: str = Field(default='', description='Краткое
описание смысла события на русском языке')
856     details: Dict[str, Any] = Field(default_factory=dict, desc
ription='Извлеченные структурированные поля события')
857
858
859 class RebootSession(BaseModel):
860     """Структурированная запись об одной сессии перезагрузки /
выключения Windows."""
861     boot_id: str = Field(..., description='Уникальный идентифи
катор сессии перезагрузки (напр. boot-20261001-014218)')
862     boot_time: str = Field(..., description='Время запуска сис
темы (OS Startup)')
863     previous_boot_time: Optional[str] = Field(default=None, de
scription='Время предыдущего запуска системы')
864     uptime_seconds: Optional[float] = Field(default=None, desc
```



```

ription='Время непрерывной работы предыдущей сессии в секундах
')
865     uptime_human: Optional[str] = Field(default=None, description='Человекочитаемое время работы (напр. "7h 25m")')
866     shutdown_type: ShutdownType = Field(default=ShutdownType.UNKNOWN, description='Классифицированный тип завершения работы'
)
867     likely_class: str = Field(default='unknown', description='Короткий машинный класс причины')
868     conclusion: str = Field(default='', description='Итоговое аналитическое заключение на русском языке')
869     initiating_process: Optional[str] = Field(default=None, description='Процесс, инициировавший перезагрузку (из Event 1074)')
870     initiating_user: Optional[str] = Field(default=None, description='Учетная запись, от имени которой инициирован рестарт')
871     shutdown_action: Optional[str] = Field(default=None, description='Действие: restart, power off, shutdown, hybrid')
872     reason_text: Optional[str] = Field(default=None, description='Текстовая причина перезапуска')
873     reason_code: Optional[str] = Field(default=None, description='Шестнадцатеричный код причины (напр. 0x80020010)')
874     bugcheck_code: Optional[str] = Field(default=None, description='Код остановки BSOD (напр. 0x000000D1)')
875     bugcheck_params: List[str] = Field(default_factory=list, description='Параметры BugCheck (p1..p4)')
876     power_button_timestamp: Optional[int] = Field(default=None, description='Метка нажатия кнопки питания из Event 41')
877     windows_update_kb: Optional[str] = Field(default=None, description='Номер установленного обновления Windows (Event 19)')
878     service_installed: Optional[str] = Field(default=None, description='Имя недавно установленной службы (Event 7045)')
879     evidence: List[str] = Field(default_factory=list, description='Список доказательных фактов и событий')
880     events_chain: List[RebootCorrelatedEvent] = Field(default_factory=list, description='Хронологическая цепочка коррелированных событий')
881

```

```

882
883 class RebootAnalysisReport(BaseModel):
884     """Сводный отчет анализа стабильности и причин перезагрузо
      к системы."""
885     generated_at: str = Field(default_factory=lambda: datetime
      .now(timezone.utc).isoformat(), description='Время формирова
      ния отчета')
886     hostname: str = Field(default='', description='Имя компьют
      ера')
887     current_boot_time: str = Field(default='', description='Вр
      емя текущей загрузки Windows')
888     current_uptime_seconds: float = Field(default=0.0, descrip
      tion='Текущий аптайм в секундах')
889     current_uptime_human: str = Field(default='', description=
      'Текущий аптайм в читаемом формате')
890     total_reboots_analyzed: int = Field(default=0, description
      ='Количество проанализированных перезагрузок')
891     planned_count: int = Field(default=0, description='Количес
      тво плановых перезагрузок')
892     unexpected_count: int = Field(default=0, description='Коли
      чество неожиданных/аварийных выключений')
893     bsod_count: int = Field(default=0, description='Количество
      сбоев с синим экраном (BSOD)')
894     update_reboot_count: int = Field(default=0, description='К
      оличество перезагрузок из-за обновлений Windows')
895     stability_score: float = Field(default=100.0, description=
      'Оценка стабильности завершений работы (0..100)')
896     sessions: List[RebootSession] = Field(default_factory=list
      , description='Список сессий перезагрузок (от новых к старым)'
      )
897     summary_ru: str = Field(default='', description='Общее рез
      юме надежности на русском языке')
898
899
900 class DefenderTelemetrySummary(BaseModel):
901     """Сводка телеметрии встроенного защитника Windows Defende
      r."""
902     cfa_enabled: bool = Field(default=False, description='Вклю

```

```

| чена ли защита Controlled Folder Access (от Ransomware)')
903 |     cfa_mode: str = Field(default='Disabled', description='Реж
| им CFA: Enabled, Disabled, Audit, Warn')
904 |     cfa_protected_folders_count: int = Field(default=0, descri
| ption='Количество защищенных папок')
905 |     asr_rules_count: int = Field(default=0, description='Колич
| ество настроенных правил ASR')
906 |     asr_enabled_count: int = Field(default=0, description='Кол
| ичество активных правил ASR')
907 |     exclusions_count: int = Field(default=0, description='Обще
| е количество исключений антивируса')
908 |     path_exclusions: List[str] = Field(default_factory=list, d
| escription='Список путей, исключенных из проверки')
909 |     process_exclusions: List[str] = Field(default_factory=list
| , description='Список процессов, исключенных из проверки')
910 |     active_threats_count: int = Field(default=0, description='
| Количество активных необработанных угроз')
911 |     threat_history_count: int = Field(default=0, description='
| Количество зафиксированных угроз в истории')
912 |
913 |
914 | class StartupTelemetrySummary(BaseModel):
915 |     """Сводка телеметрии точек автозапуска и персистентности п
| рограмм."""
916 |     total_entries: int = Field(default=0, description='Общее к
| оличество программ и элементов в автозагрузке')
917 |     registry_run_count: int = Field(default=0, description='За
| писи автозапуска в реестре Run/RunOnce')
918 |     startup_folders_count: int = Field(default=0, description=
| 'Ярлыки в папках Startup (пользователь/система)')
919 |     scheduled_tasks_count: int = Field(default=0, description=
| 'Задачи планировщика со стартом при входе')
920 |     startup_services_count: int = Field(default=0, description
| ='Службы Windows с автозапуском')
921 |     entries: List[Dict[str, Any]] = Field(default_factory=list
| , description='Список записей автозапуска с параметрами')
922 |
923 |

```

```

924 class VssTelemetrySummary(BaseModel):
925     """Сводка состояния теневых копий томов VSS и точек восста
новления."""
926     total_snapshots_count: int = Field(default=0, description=
'Количество существующих теневых копий')
927     protected_volumes: List[str] = Field(default_factory=list,
description='Список защищенных томов (буквы дисков)')
928     latest_snapshot_time: Optional[str] = Field(default=None,
description='Время создания последней теневой копии')
929     snapshots: List[Dict[str, Any]] = Field(default_factory=li
st, description='Список снимков VSS')
930
931
932 class UserAccountsTelemetrySummary(BaseModel):
933     """Сводка аудита локальных пользователей и привилегий."""
934     total_users_count: int = Field(default=0, description='Общ
ее число локальных учетных записей')
935     admin_users_count: int = Field(default=0, description='Чис
ло учетных записей с правами администратора')
936     locked_users_count: int = Field(default=0, description='Чи
сло заблокированных учетных записей')
937     admin_usernames: List[str] = Field(default_factory=list, d
escription='Имена локальных администраторов')
938     logged_in_users: List[str] = Field(default_factory=list, d
escription='Активные вошедшие пользователи')
939     users: List[Dict[str, Any]] = Field(default_factory=list,
description='Сводный список учетных записей')
940
941
942 class ExtendedSystemAuditReport(BaseModel):
943     """Комплексный отчет расширенного аудита безопасности, авт
оагрузки, VSS и пользователей."""
944     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время аудита')
945     hostname: str = Field(default='', description='Имя компьют
ера')
946     defender: DefenderTelemetrySummary = Field(default_factory
=DefenderTelemetrySummary, description='Сводка Windows Defende

```

```
    r')
947     startup: StartupTelemetrySummary = Field(default_factory=StartupTelemetrySummary, description='Сводка автозагрузки')
948     vss: VssTelemetrySummary = Field(default_factory=VssTelemetrySummary, description='Сводка теневых копий VSS')
949     users: UserAccountsTelemetrySummary = Field(default_factory=UserAccountsTelemetrySummary, description='Сводка локальных пользователей')
950
951
952
953
```

process_token_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Process
  Token Collector
4 # =====
5 # Description:
6 #   Коллектор токенов процессов, уровней целостности, прав UAC
  , SID и Session ID.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.process_token_collector impo
  rt ProcessTokenCollector
11 #
12 #       collector = ProcessTokenCollector()
13 #       tokens = collector.collect()
14 #
15 # File: process_token_collector.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-04 01:45:00
21 # =====
22
23 """Коллектор токенов процессов, уровней целостности, прав UAC,
  SID и Session ID."""
24
25 from __future__ import annotations
26
27 import ctypes
28 import ctypes.wintypes as wtypes
29 import os
```

```
30 from typing import Dict, List, Optional
31 try:
32     import psutil
33     PSUTIL_AVAILABLE = True
34 except ImportError:
35     psutil = None
36     PSUTIL_AVAILABLE = False
37
38 from .models import ProcessTokenInfo
39
40 TOKEN_QUERY = 8
41 TokenUser = 1
42 TokenGroups = 2
43 TokenPrivileges = 3
44 TokenElevation = 20
45 TokenIntegrityLevel = 25
46
47
48 class TOKEN_ELEVATION(ctypes.Structure):
49     _fields_ = [('TokenIsElevated', ctypes.DWORD)]
50
51
52 class LUID(ctypes.Structure):
53     _fields_ = [('LowPart', ctypes.DWORD), ('HighPart', ctypes
54     .LONG)]
55
56 class LUID_AND_ATTRIBUTES(ctypes.Structure):
57     _fields_ = [('Luid', LUID), ('Attributes', ctypes.DWORD)]
58
59
60 class TOKEN_MANDATORY_LABEL(ctypes.Structure):
61     _fields_ = [('Label', LUID_AND_ATTRIBUTES)]
62
63
64 class SID_AND_ATTRIBUTES(ctypes.Structure):
65     _fields_ = [('Sid', ctypes.c_void_p), ('Attributes', ctypes
66     .DWORD)]
```

```

66 |
67 |
68 | class TOKEN_USER(ctypes.Structure):
69 |     _fields_ = [('User', SID_AND_ATTRIBUTES)]
70 |
71 |
72 | def _get_process_token(handle: Any) -> wtypes.HANDLE:
73 |     """Получить дескриптор токена процесса."""
74 |     token = wtypes.HANDLE()
75 |     if not ctypes.windll.advapi32.OpenProcessToken(handle, TOK
76 | EN_QUERY, ctypes.byref(token)):
77 |         raise ctypes.WinError()
78 |     return token
79 |
80 | def _get_token_elevation(token: Any) -> bool:
81 |     """Определить статус повышения прав UAC."""
82 |     elevation = TOKEN_ELEVATION()
83 |     size = wtypes.DWORD(ctypes.sizeof(elevation))
84 |     ret = ctypes.windll.advapi32.GetTokenInformation(
85 |         token, TokenElevation, ctypes.byref(elevation), size,
86 | ctypes.byref(size)
87 |     )
88 |     if not ret:
89 |         raise ctypes.WinError()
90 |     return bool(elevation.TokenIsElevated)
91 |
92 | def _get_integrity_level(token: Any) -> str:
93 |     """Получить уровень целостности процесса (Untrusted, Low,
94 | Medium, High, System)."""
95 |     size = wtypes.DWORD(0)
96 |     ctypes.windll.advapi32.GetTokenInformation(token, TokenInt
97 | egrityLevel, None, 0, ctypes.byref(size))
98 |     if size.value == 0:
99 |         return 'Medium'
100 |     buf = ctypes.create_string_buffer(size.value)
101 |     if not ctypes.windll.advapi32.GetTokenInformation(token, T

```



```

okenIntegrityLevel, buf, size, ctypes.byref(size)):
100     raise ctypes.WinError()
101     label = ctypes.cast(buf, ctypes.POINTER(TOKEN_MANDATORY_LA
BEL)).contents
102     integrity = label.Label.Luid.LowPart
103     if integrity < 4096:
104         return 'Untrusted'
105     elif integrity < 8192:
106         return 'Low'
107     elif integrity < 12288:
108         return 'Medium'
109     elif integrity < 16384:
110         return 'High'
111     else:
112         return 'System'
113
114
115 def _get_token_sid(token: Any) -> Optional[str]:
116     """Извлечь строковый SID пользователя из токена процесса."""
117     try:
118         size = wtypes.DWORD(0)
119         ctypes.windll.advapi32.GetTokenInformation(token, Toke
nUser, None, 0, ctypes.byref(size))
120         if size.value == 0:
121             return None
122         buf = ctypes.create_string_buffer(size.value)
123         if not ctypes.windll.advapi32.GetTokenInformation(toke
n, TokenUser, buf, size, ctypes.byref(size)):
124             return None
125         user_struct = ctypes.cast(buf, ctypes.POINTER(TOKEN_US
ER)).contents
126         sid_ptr = user_struct.User.Sid
127         if not sid_ptr:
128             return None
129         str_sid = wtypes.LPWSTR()
130         if ctypes.windll.advapi32.ConvertSidToStringSidW(sid_p
tr, ctypes.byref(str_sid)):

```

```

131         sid_val = str_sid.value
132         ctypes.windll.kernel32.LocalFree(str_sid)
133         return sid_val
134     except Exception:
135         pass
136     return None
137
138
139 def get_process_session_id(pid: int) -> Optional[int]:
140     """Получить Session ID процесса (Session 0=Службы, Session
141     1+=Интерактивный пользователь)."""
142     if os.name != 'nt':
143         return None
144     try:
145         session_id = ctypes.DWORD(0)
146         if ctypes.windll.kernel32.ProcessIdToSessionId(pid, ct
147         ypes.byref(session_id)):
148             return int(session_id.value)
149     except Exception:
150         pass
151     return None
152
153 class ProcessTokenCollector:
154     """Коллектор сведений о безопасности токенов, правах UAC,
155     SID и сессиях процессов Windows."""
156
157     def get_process_token_info(self, pid: int, name: str = '')
158     -> Optional[ProcessTokenInfo]:
159         """Собрать информацию о токене для конкретного PID.
160
161         Args:
162             pid: Идентификатор процесса.
163             name: Имя процесса (опционально).
164
165         Returns:
166             Optional[ProcessTokenInfo]: Данные токена или None
167             при отсутствии доступа.

```

```

164 |         """
165 |         if os.name != 'nt':
166 |             return ProcessTokenInfo(pid=pid, name=name, elevat
ion=False, integrity_level='Medium', sid=None, session_id=1)
167 |
168 |             session_id = get_process_session_id(pid)
169 |             handle = ctypes.windll.kernel32.OpenProcess(1024, Fals
e, pid)
170 |             if not handle:
171 |                 return ProcessTokenInfo(
172 |                     pid=pid,
173 |                     name=name,
174 |                     elevation=None,
175 |                     integrity_level=None,
176 |                     privileges=[],
177 |                     sid=None,
178 |                     session_id=session_id,
179 |                 )
180 |             try:
181 |                 token = _get_process_token(handle)
182 |                 elevated = _get_token_elevation(token)
183 |                 integrity = _get_integrity_level(token)
184 |                 sid = _get_token_sid(token)
185 |                 privileges: List[str] = []
186 |                 return ProcessTokenInfo(
187 |                     pid=pid,
188 |                     name=name,
189 |                     elevation=bool(elevated),
190 |                     integrity_level=integrity,
191 |                     privileges=privileges,
192 |                     sid=sid,
193 |                     session_id=session_id,
194 |                 )
195 |             except Exception:
196 |                 return ProcessTokenInfo(
197 |                     pid=pid,
198 |                     name=name,
199 |                     elevation=None,

```

```

200         integrity_level=None,
201         privileges=[],
202         sid=None,
203         session_id=session_id,
204     )
205     finally:
206         ctypes.windll.kernel32.CloseHandle(handle)
207
208     def collect(self) -> List[ProcessTokenInfo]:
209         """Собрать сведения о токенах для всех активных процес
210 сов в системе.
211
212         Returns:
213             List[ProcessTokenInfo]: Список токенов процессов.
214         """
215         results: List[ProcessTokenInfo] = []
216         if not PSUTIL_AVAILABLE:
217             current_pid = os.getpid()
218             info = self.get_process_token_info(current_pid, 'p
219 ython.exe')
220             if info:
221                 results.append(info)
222             return results
223
224         for proc in psutil.process_iter(attrs=['pid', 'name']):
225             :
226             try:
227                 pid = proc.info['pid']
228                 name = proc.info['name'] or 'unknown'
229                 info = self.get_process_token_info(pid, name)
230                 if info:
231                     results.append(info)
232             except Exception:
233                 continue
234         return results

```

ring_buffer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Ring Bu
  ffer
4 # =====
5 # Description:
6 #   Потокбезопасный кольцевой буфер в оперативной памяти для
  удержания
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.ring_buffer import Telemetry
  RingBuffer
11 #
12 #       service = TelemetryRingBuffer()
13 #
14 # File: ring_buffer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Потокбезопасный кольцевой буфер в оперативной памяти для у
  держания"""
24
25 import collections
26 import threading
27 import time
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional, Tuple
```

```

30 |
31 |
32 | class TelemetryRingBuffer:
33 |     """Потокобезопасный кольцевой буфер для хранения сырых мет
    |     рик телеметрии в RAM."""
34 |
35 |     def __init__(self, capacity: int = 600) -> None:
36 |         """Инициализирует кольцевой буфер заданной емкости.
37 |
38 |         Args:
39 |             capacity: Максимальное количество удерживаемых зап
    |             исей (по умолчанию 600 = 10 мин при 1с).
40 |         """
41 |         self._capacity = max(10, capacity)
42 |         self._buffer: collections.deque[Tuple[float, Dict[str,
    |         Any]]] = collections.deque(maxlen=self._capacity)
43 |         self._lock = threading.Lock()
44 |
45 |     @property
46 |     def capacity(self) -> int:
47 |         """Возвращает максимальную емкость буфера."""
48 |         return self._capacity
49 |
50 |     def append(self, item: Dict[str, Any], epoch_timestamp: Op
    |     tional[float] = None) -> None:
51 |         """Добавляет запись телеметрии в кольцевой буфер.
52 |
53 |         Args:
54 |             item: Словарь с телеметрией или системным снимком.
55 |             epoch_timestamp: Unix epoch timestamp в секундах (
    |             если None, берется текущее время).
56 |         """
57 |         ts = epoch_timestamp if epoch_timestamp is not None el
    |         se time.time()
58 |         with self._lock:
59 |             self._buffer.append((ts, item))
60 |
61 |     def get_recent(self, seconds: float = 300.0) -> List[Dict[

```

```

str, Any]]:
62     """Возвращает список записей за последние N секунд.
63
64     Args:
65         seconds: Длительность временного окна назад от тек
ущего момента.
66
67     Returns:
68         List[Dict[str, Any]]: Список замеров от старых к н
ОВЫМ.
69     """
70     now = time.time()
71     cutoff = now - max(0.0, seconds)
72     with self._lock:
73         return [item for ts, item in self._buffer if ts >=
cutoff]
74
75     def get_window(
76         self,
77         center_timestamp: float,
78         before_seconds: float = 300.0,
79         after_seconds: float = 0.0,
80     ) -> List[Dict[str, Any]]:
81         """Извлекает срез телеметрии вокруг заданного момента
времени (например, T-5m .. T+20m).
82
83     Args:
84         center_timestamp: Центральный момент времени событ
ия (Unix epoch seconds).
85         before_seconds: Длительность окна ДО события в сек
ундах.
86         after_seconds: Длительность окна ПОСЛЕ события в с
екундах.
87
88     Returns:
89         List[Dict[str, Any]]: Список записей, попавших в и
нтервал.
90     """

```

```
91 |         start_ts = center_timestamp - max(0.0, before_seconds)
92 |         end_ts = center_timestamp + max(0.0, after_seconds)
93 |         with self._lock:
94 |             return [item for ts, item in self._buffer if start
|_ts <= ts <= end_ts]
95 |
96 |     def get_all(self) -> List[Dict[str, Any]]:
97 |         """Возвращает все текущие записи в буфере.
98 |
99 |         Returns:
100 |             List[Dict[str, Any]]: Полный текущий срез буфера.
101 |         """
102 |         with self._lock:
103 |             return [item for _, item in self._buffer]
104 |
105 |     def count(self) -> int:
106 |         """Возвращает текущее число записей в буфере."""
107 |         with self._lock:
108 |             return len(self._buffer)
109 |
110 |     def clear(self) -> None:
111 |         """Очищает буфер."""
112 |         with self._lock:
113 |             self._buffer.clear()
114 |
```


sampling_controller.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sampling
  # Controller
4 # =====
5 # Description:
6 #   Контроллер адаптивного изменения частоты сбора телеметрии
  # (Sampling Controller).
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sampling_controller import S
  # amplingController
11 #
12 #       service = SamplingController()
13 #
14 # File: sampling_controller.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Контроллер адаптивного изменения частоты сбора телеметрии (
  # Sampling Controller)."""
24
25 import time
26 from datetime import datetime, timezone
27 from typing import Any, Dict, Optional
28
29 try:
```

```

30     import psutil
31     BOOT_TIME = psutil.boot_time()
32 except Exception:
33     BOOT_TIME = time.time()
34
35 try:
36     from logger import logger
37 except ImportError:
38     from logger import logger
39
40 from .models import SamplingMode
41
42
43 class SamplingController:
44     """Контроллер режимов сбора телеметрии и адаптивного интер
45 вала."""
46
47     def __init__(
48         self,
49         initial_mode: SamplingMode = SamplingMode.BOOT_AGGRESS
50 IVE,
51         boot_epoch: Optional[float] = None,
52     ) -> None:
53         """Инициализирует контроллер режимов сбора.
54
55         Args:
56             initial_mode: Начальный режим работы (по умолчанию
57 BOOT_AGGRESSIVE).
58             boot_epoch: Время старта ОС или сервиса в секундах
59 Unix epoch.
60         """
61         self._mode = initial_mode
62         self._boot_epoch = boot_epoch if boot_epoch is not Non
63 e else BOOT_TIME
64         self._start_epoch = time.time()
65         self._incident_expiry_epoch: Optional[float] = None
66         self._previous_mode: SamplingMode = SamplingMode.MONIT
67 ORING

```

```

62
63     @property
64     def mode(self) -> SamplingMode:
65         """Возвращает текущий режим сбора с учетом авто-истече
        ния инцидентов."""
66         self._check_incident_expiration()
67         return self._mode
68
69     def set_mode(self, mode: SamplingMode) -> None:
70         """Принудительно устанавливает режим сбора телеметрии.
71
72         Args:
73             mode: Новый режим работы.
74         """
75         self._mode = mode
76         if mode != SamplingMode.INCIDENT:
77             self._incident_expiry_epoch = None
78         logger.info(f"Режим сбора телеметрии переключен на: {s
        elf._mode.value}")
79
80     def trigger_incident_mode(self, duration_seconds: float =
        1200.0) -> None:
81         """Переводит сборщик в режим INCIDENT (1-секундный зах
        ват) на заданную длительность.
82
83         Args:
84             duration_seconds: Длительность удержания режима ин
        цидента (по умолчанию 20 минут = 1200 сек).
85         """
86         if self._mode != SamplingMode.INCIDENT:
87             self._previous_mode = self._mode
88             self._mode = SamplingMode.INCIDENT
89             self._incident_expiry_epoch = time.time() + max(0.01,
        duration_seconds)
90         logger.warning(f"Активирован режим INCIDENT на {durati
        on_seconds} сек.")
91
92     def get_current_interval(self) -> float:

```

```

93 |         """Возвращает текущий рекомендуемый интервал опроса те
    | леметрии в секундах.
94 |
95 |     Returns:
96 |         float: Интервал в секундах.
97 |     """
98 |     self._check_incident_expiration()
99 |
100 |     if self._mode == SamplingMode.INCIDENT:
101 |         return 1.0
102 |
103 |     if self._mode == SamplingMode.FORENSIC:
104 |         return 1.0
105 |
106 |     if self._mode == SamplingMode.BOOT_AGGRESSIVE:
107 |         # Адаптивная кривая после старта Windows / сервиса
108 |         uptime = time.time() - self._boot_epoch
109 |         if uptime < 0:
110 |             uptime = time.time() - self._start_epoch
111 |
112 |         if uptime <= 600.0: # 0-10 минут: 1-секундные дан
    | ные
113 |             return 1.0
114 |         elif uptime <= 3600.0: # 10-60 минут: 5-секундные
115 |             return 5.0
116 |         elif uptime <= 86400.0: # 1-24 часа: 30-секундные
117 |             return 30.0
118 |         else:
119 |             # После 24 часов переходим в дежурный монитори
    | нг
120 |             return 60.0
121 |
122 |     if self._mode == SamplingMode.MONITORING:
123 |         return 30.0
124 |
125 |     if self._mode == SamplingMode.NORMAL:
126 |         return 60.0
127 |

```

```
128         return 30.0
129
130     def should_collect_per_process_details(self) -> bool:
131         """Определяет, требуется ли собирать глубокий срез по
132         всем процессам на текущем тике.
133
134         В высокочастотных режимах (1s) подробный опрос всех пр
135         оцессов может отфильтровываться
136         по топ-N / дельте, чтобы не нагружать процессор.
137         """
138         interval = self.get_current_interval()
139         if interval <= 1.0:
140             return True
141         return True
142
143     def _check_incident_expiration(self) -> None:
144         """Проверяет истечение времени режима инцидента и возв
145         ращает предыдущий режим."""
146         if self._mode == SamplingMode.INCIDENT and self._incid
147         ent_expiry_epoch is not None:
148             if time.time() >= self._incident_expiry_epoch:
149                 logger.info("Время фиксации инцидента истекло.
150                 Возврат в штатный режим.")
151                 self._mode = self._previous_mode
152                 self._incident_expiry_epoch = None
153
154     def get_status(self) -> Dict[str, Any]:
155         """Возвращает статус контроллера опроса.
156
157         Returns:
158             Dict[str, Any]: Словарь с текущим режимом, интерва
159             лом и временем старта.
160         """
161         return {
162             "mode": self.mode.value,
163             "current_interval_sec": self.get_current_interval(
164             ),
165             "boot_epoch": self._boot_epoch,
```

```
159 |         "incident_active": self._mode == SamplingMode.INCI
    | DENT,
160 |         "incident_remaining_sec": max(0.0, (self._incident
    | _expiry_epoch - time.time())) if self._incident_expiry_epoch e
    | lse 0.0,
161 |     }
162 |
```

sensors.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensors
4 # =====
5 # Description:
6 #   Движок считывания показаний температурных датчиков, напряж
   ений и вентиляторов.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensors import get_hardware_
   sensors
11 #
12 #       res = get_hardware_sensors()
13 #
14 # File: sensors.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:00:00
20 # =====
21
22 from __future__ import annotations
23 """Движок считывания показаний аппаратных сенсоров, сетевых ме
   трик и скорости FAST.com."""
24
25 import json
26 import os
27 import subprocess
28 from typing import List, Optional
29 try:
30     import wmi
```

```

31 |     _WMI_AVAILABLE = True
32 | except ImportError:
33 |     _WMI_AVAILABLE = False
34 | from logger import logger
35 | from .models import HardwareSensor
36 | from .internet_speed import InternetSpeedSensor
37 | from .sensor_registry import SensorProvider
38 |
39 | # NOTE: GPU temperature, power, and fan sensors are sourced fr
    om:
40 | # 1. GpuProber._probe_nvidia() -> HardwareMonitor.get_gpu_me
    trics()
41 | # 2. sensor_collector.extract_sensor_readings() normalizes G
    puMetrics to sensor_id format
42 |
43 | def _probe_acpi_thermal_zones() -> List[HardwareSensor]:
44 |     """Сбор данных температурных зон Windows ACPI через CIM с
    fallback на WMI.
45 |
46 |     Returns:
47 |         List[HardwareSensor]: Список показаний термальных зон
    ACPI.
48 |     """
49 |     sensors: List[HardwareSensor] = []
50 |     if os.name != 'nt':
51 |         return sensors
52 |
53 |     # 1. Первичный сбор через CIM (Get-CimInstance)
54 |     try:
55 |         cmd = [
56 |             'powershell.exe', '-NoProfile', '-NonInteractive',
57 |             '-Command',
58 |             'Get-CimInstance -Namespace root/wmi -ClassName MS
    Acpi_ThermalZoneTemperature -ErrorAction SilentlyContinue | '
59 |             'Select-Object InstanceName, CurrentTemperature |
    ConvertTo-Json -Compress'
60 |         ]
61 |         res = subprocess.run(cmd, capture_output=True, text=Tr

```



```

ue, timeout=5)
61         if res.returncode == 0 and res.stdout.strip():
62             raw_data = json.loads(res.stdout.strip())
63             items = [raw_data] if isinstance(raw_data, dict) e
lse (raw_data if isinstance(raw_data, list) else [])
64             for idx, zone in enumerate(items):
65                 raw_temp = zone.get('CurrentTemperature')
66                 if raw_temp and raw_temp > 0:
67                     celsius = round(float(raw_temp) / 10.0 - 2
73.15, 1)
68                     if -20.0 <= celsius <= 125.0:
69                         zone_name = zone.get('InstanceName') o
r f'Thermal Zone {idx}'
70                         sensor = HardwareSensor(
71                             sensor_id=f'acpi_thermal_{idx}',
72                             name=f'ACPI {zone_name}',
73                             category='temperature',
74                             value=celsius,
75                             unit='°C'
76                         )
77                         sensor.provider = SensorProvider.CIM_S
YSTEM
78                         sensors.append(sensor)
79                     if sensors:
80                         return sensors
81             except Exception as ex:
82                 logger.debug(f'CIM ACPI thermal probe skipped or error
: {ex}')
83
84     # 2. Fallback / слой совместимости через WMI
85     try:
86         import wmi
87         import pythoncom
88         pythoncom.CoInitialize()
89         w = wmi.WMI(namespace='root\\wmi')
90         for idx, zone in enumerate(w.MSAcpi_ThermalZoneTempera
ture()):
91             raw_temp = getattr(zone, 'CurrentTemperature', Non

```

```

e)
92         if raw_temp and raw_temp > 0:
93             celsius = round(float(raw_temp) / 10.0 - 273.1
5, 1)
94             if -20.0 <= celsius <= 125.0:
95                 zone_name = getattr(zone, 'InstanceName',
f'Thermal Zone {idx}')
96                 sensor = HardwareSensor(
97                     sensor_id=f'acpi_thermal_{idx}',
98                     name=f'ACPI {zone_name}',
99                     category='temperature',
100                     value=celsius,
101                     unit='°C'
102                 )
103                 sensor.provider = SensorProvider.WMI_FALLB
ACK
104                 sensors.append(sensor)
105     except Exception as ex:
106         logger.debug(f'WMI ACPI thermal fallback probe skipped
: {ex}')
107
108     return sensors
109
110 def _probe_network_sensors() -> List[HardwareSensor]:
111     """Probe network interface metrics using Windows native to
ols.
112
113     Returns:
114         List[HardwareSensor]: Network interface telemetry as s
ensors with provider marker.
115     """
116     sensors: List[HardwareSensor] = []
117     try:
118         import psutil
119         current_net_io = psutil.net_io_counters(pernic=True)
120         if current_net_io:
121             for name, stats in current_net_io.items():
122                 # Create sensors with provider marker

```

```
123         s1 = HardwareSensor(sensor_id=f'net_{name}_byt
    es_recv', name=f'Network {name} Bytes Received', category='net
    work', value=round(stats.bytes_recv, 2), unit='B')
124         s1.provider = SensorProvider.NETWORK_SENSOR
125         sensors.append(s1)
126
127         s2 = HardwareSensor(sensor_id=f'net_{name}_byt
    es_sent', name=f'Network {name} Bytes Sent', category='network
    ', value=round(stats.bytes_sent, 2), unit='B')
128         s2.provider = SensorProvider.NETWORK_SENSOR
129         sensors.append(s2)
130
131         s3 = HardwareSensor(sensor_id=f'net_{name}_pac
    kets_recv', name=f'Network {name} Packets Received', category=
    'network', value=stats.packets_recv, unit='pkts')
132         s3.provider = SensorProvider.NETWORK_SENSOR
133         sensors.append(s3)
134
135         s4 = HardwareSensor(sensor_id=f'net_{name}_pac
    kets_sent', name=f'Network {name} Packets Sent', category='net
    work', value=stats.packets_sent, unit='pkts')
136         s4.provider = SensorProvider.NETWORK_SENSOR
137         sensors.append(s4)
138
139         s5 = HardwareSensor(sensor_id=f'net_{name}_err
    ors_recv', name=f'Network {name} Errors Received', category='n
    etwork', value=stats.errin, unit='errors')
140         s5.provider = SensorProvider.NETWORK_SENSOR
141         sensors.append(s5)
142
143         s6 = HardwareSensor(sensor_id=f'net_{name}_err
    ors_sent', name=f'Network {name} Errors Sent', category='netwo
    rk', value=stats.errout, unit='errors')
144         s6.provider = SensorProvider.NETWORK_SENSOR
145         sensors.append(s6)
146
147         s7 = HardwareSensor(sensor_id=f'net_{name}_dro
    pped_recv', name=f'Network {name} Dropped Received', category=
```

```

    'network', value=stats.dropin, unit='pkts')
148         s7.provider = SensorProvider.NETWORK_SENSOR
149         sensors.append(s7)
150
151         s8 = HardwareSensor(sensor_id=f'net_{name}_dro
pped_sent', name=f'Network {name} Dropped Sent', category='net
work', value=stats.dropout, unit='pkts')
152         s8.provider = SensorProvider.NETWORK_SENSOR
153         sensors.append(s8)
154     except Exception as ex:
155         logger.debug(f'Network sensor probe failed: {ex}')
156     return sensors
157
158 def _probe_internet_speed_sensors() -> List[HardwareSensor]:
159     """Собрать показания скорости интернета и характеристик се
ти FAST.com.
160
161     Returns:
162         List[HardwareSensor]: Метрики скорости интернета и зад
ержки с маркером провайдера.
163     """
164     sensors: List[HardwareSensor] = []
165     try:
166         sensor = InternetSpeedSensor()
167         metrics = sensor.measure_internet_speed()
168
169         s1 = HardwareSensor(
170             sensor_id='internet_ping',
171             name='Internet Ping',
172             category='network',
173             value=round(metrics.get('ping_ms', 0.0), 2),
174             unit='ms'
175         )
176         s1.provider = SensorProvider.INTERNET_SPEED
177         sensors.append(s1)
178
179         s2 = HardwareSensor(
180             sensor_id='internet_download',

```

```

181         name='Internet Download Speed',
182         category='network',
183         value=round(metrics.get('download_mbps', 0.0), 2),
184         unit='Mbps'
185     )
186     s2.provider = SensorProvider.INTERNET_SPEED
187     sensors.append(s2)
188
189     s3 = HardwareSensor(
190         sensor_id='internet_upload',
191         name='Internet Upload Speed',
192         category='network',
193         value=round(metrics.get('upload_mbps', 0.0), 2),
194         unit='Mbps'
195     )
196     s3.provider = SensorProvider.INTERNET_SPEED
197     sensors.append(s3)
198
199     s4 = HardwareSensor(
200         sensor_id='internet_bufferbloat',
201         name='Internet Bufferbloat',
202         category='network',
203         value=round(metrics.get('bufferbloat_ms', 0.0), 2)
204         ,
205         unit='ms'
206     )
207     s4.provider = SensorProvider.INTERNET_SPEED
208     sensors.append(s4)
209
210     s5 = HardwareSensor(
211         sensor_id='internet_latency_loaded',
212         name='Internet Loaded Latency',
213         category='network',
214         value=round(metrics.get('latency_loaded_ms', 0.0),
215         2),
216         unit='ms'
217     )
218     s5.provider = SensorProvider.INTERNET_SPEED

```

```

217         sensors.append(s5)
218     except Exception as ex:
219         logger.debug(f'Ошибка опроса сенсора скорости интернет
a: {ex}')
220     return sensors
221
222 def _probe_storage_sensors() -> List[HardwareSensor]:
223     """Сбор сенсоров температуры и надежности накопителей.
224
225     :return: Список объектов HardwareSensor для дисков с provi
der marker.
226     """
227     sensors: List[HardwareSensor] = []
228     if os.name != 'nt':
229         return sensors
230     try:
231         from apps.windows.modules.storage_manager.core.windows
_storage_sensor import WindowsStorageSensor
232         sensor = WindowsStorageSensor(timeout_sec=30)
233         disks = sensor.get_physical_disks()
234         for disk in disks:
235             disk_slug = disk.device_id.replace('\\', '_').repl
ace('.', '_')
236             if disk.temperature_c is not None:
237                 s1 = HardwareSensor(sensor_id=f'disk_temp_{dis
k_slug}', name=f'{disk.friendly_name} Temperature', category='
temperature', value=float(disk.temperature_c), unit='°C')
238                 s1.provider = SensorProvider.WINDOWS_STORAGE
239                 sensors.append(s1)
240             if disk.wear_percentage is not None:
241                 s2 = HardwareSensor(sensor_id=f'disk_wear_{dis
k_slug}', name=f'{disk.friendly_name} Wear', category='wear',
value=float(disk.wear_percentage), unit='%')
242                 s2.provider = SensorProvider.WINDOWS_STORAGE
243                 sensors.append(s2)
244     except Exception as ex:
245         logger.debug(f'Ошибка сбора сенсоров накопителей: {ex}
')
```

```

246     return sensors
247
248 def _probe_cloud_storage_sensors() -> List[HardwareSensor]:
249     """Сбор сенсоров свободного и занятого места для облачного
    хранилища OneDrive.
250
251     Returns:
252         List[HardwareSensor]: Список сенсоров для дисков OneDr
ive с provider marker.
253     """
254     import shutil
255     sensors: List[HardwareSensor] = []
256     od_paths: List[str] = []
257     for env_var in ('OneDrive', 'OneDriveConsumer', 'OneDriveC
ommerical'):
258         val = os.environ.get(env_var)
259         if val and os.path.exists(val) and (val not in od_path
s):
260             od_paths.append(val)
261     if not od_paths:
262         home = os.path.expanduser('~')
263         candidate = os.path.join(home, 'OneDrive')
264         if os.path.exists(candidate):
265             od_paths.append(candidate)
266     if os.name == 'nt':
267         try:
268             import winreg
269             for subkey in ('Software\\Microsoft\\OneDrive\\Acc
ounts\\Personal', 'Software\\Microsoft\\OneDrive\\Accounts\\Bu
siness1', 'Software\\Microsoft\\OneDrive'):
270                 try:
271                     with winreg.OpenKey(winreg.HKEY_CURRENT_US
ER, subkey) as k:
272                         val, _ = winreg.QueryValueEx(k, 'UserF
older')
273                         if val and os.path.exists(val) and (va
l not in od_paths):
274                             od_paths.append(val)

```

```

275         except OSError:
276             pass
277     except Exception:
278         pass
279     for idx, od_path in enumerate(od_paths):
280         try:
281             usage = shutil.disk_usage(od_path)
282             free_gb = round(usage.free / 1024 ** 3, 2)
283             total_gb = round(usage.total / 1024 ** 3, 2)
284             used_pct = round(usage.used / max(usage.total, 1)
285 * 100.0, 1)
286             slug = f'_{idx + 1}' if len(od_paths) > 1 else ''
287             label_suffix = f' ({os.path.basename(od_path)})' i
288 f len(od_paths) > 1 else ''
289
290             s1 = HardwareSensor(sensor_id=f'onedrive{slug}_fre
291 e_gb', name=f'OneDrive Free Space{label_suffix}', category='st
292 orage', value=free_gb, unit='GB')
293             s1.provider = SensorProvider.SENSOR_COLLECTOR
294             sensors.append(s1)
295
296             s2 = HardwareSensor(sensor_id=f'onedrive{slug}_use
297 d_percent', name=f'OneDrive Used Percent{label_suffix}', categ
298 ory='storage', value=used_pct, unit='%')
299             s2.provider = SensorProvider.SENSOR_COLLECTOR
300             sensors.append(s2)
301
302             s3 = HardwareSensor(sensor_id=f'onedrive{slug}_tot
303 al_gb', name=f'OneDrive Total Capacity{label_suffix}', categor
304 y='storage', value=total_gb, unit='GB')
305             s3.provider = SensorProvider.SENSOR_COLLECTOR
306             sensors.append(s3)
307         except Exception as ex:
308             logger.debug(f'Ошибка сбора сенсора для OneDrive (
309 {od_path}): {ex}')
310     return sensors
311
312 def get_hardware_sensors() -> List[HardwareSensor]:

```



```
304 |     """Retrieve all available hardware sensors across all supp
    | orted backends.
305 |
306 |     NOTE: GPU sensors (temperature, power, fan) are NOT includ
    | ed here to avoid
307 |     duplication. They are collected via GpuProber in HardwareM
    | onitor.get_gpu_metrics()
308 |     and then normalized by SensorCollector.extract_sensor_read
    | ings().
309 |
310 |     Returns:
311 |         List[HardwareSensor]: Consolidated list of active hard
    | ware sensors.
312 |     """
313 |     all_sensors: List[HardwareSensor] = []
314 |     # GPU sensors excluded - collected via GpuProber to avoid
    | duplicate nvidia-smi calls
315 |     all_sensors.extend(_probe_acpi_thermal_zones())
316 |     all_sensors.extend(_probe_storage_sensors())
317 |     all_sensors.extend(_probe_cloud_storage_sensors())
318 |     all_sensors.extend(_probe_network_sensors())
319 |     all_sensors.extend(_probe_internet_speed_sensors())
320 |     return all_sensors
```

sensor_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensor
  Collector
4 # =====
5 # Description:
6 #   Сбор данных со всех доступных сенсоров и аппаратных провай
  деров.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensor_collector import Sens
    orCollector
11 #
12 #       service = SensorCollector()
13 #
14 # File: sensor_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:00:00
20 # =====
21
22 from __future__ import annotations
23 """Сбор данных со всех доступных сенсоров и аппаратных провай
  деров."""
24
25 import re
26 import threading
27 import time
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional
```

```

30 try:
31     from logger import logger
32 except ImportError:
33     from logger import logger
34 from .telemetry_config import TelemetryConfigManager
35 from .sensor_registry import SensorDeduplicator, SensorProvide
36 try:
37     from apps.windows.modules.hardware.hardware_monitor import
38     HardwareMonitor
39     from apps.windows.modules.hardware.lhm_service import LhmS
40     ervice
41     from .internet_speed import InternetSpeedSensor
42 except ImportError as e:
43     logger.warning(f'Не удалось импортировать зависимости сенс
44     оров: {e}')
45     HardwareMonitor = None
46     LhmService = None
47     InternetSpeedSensor = None
48
49 class SensorCollector:
50     """Коллектор данных со всех сенсоров с нормализованной схе
51     мой датчиков."""
52
53     def __init__(
54         self,
55         config_manager: Optional[TelemetryConfigManager] = Non
56         e,
57         lhm_service: Optional[Any] = None,
58     ) -> None:
59         """Инициализирует коллектор сенсоров.
60
61         Args:
62             config_manager: Менеджер конфигурации для настроек
63             и сенсоров.
64             lhm_service: Сервис LibreHardwareMonitor (опционал
65             ьно).
66         """

```

```

60         self.config_manager = config_manager
61         self._lhm_service = lhm_service or (LhmService() if Lh
mService else None)
62         self._hardware_monitor = HardwareMonitor() if Hardware
Monitor else None
63         self._internet_speed_sensor = InternetSpeedSensor() if
InternetSpeedSensor else None
64         self._last_hardware_snapshot: Optional[Dict[str, Any]]
= None
65         self._lock = threading.Lock()
66
67     def _get_hardware_data(self) -> Dict[str, Any]:
68         """Получает данные от HardwareMonitor.
69
70         Returns:
71             Dict[str, Any]: Данные HardwareMonitor.
72         """
73         if self._hardware_monitor is None:
74             return {}
75         try:
76             snapshot = self._hardware_monitor.get_snapshot(inc
lude_smart=True)
77             return snapshot.to_dict()
78         except Exception as e:
79             logger.error(f'Ошибка получения данных от Hardware
Monitor: {e}')
80             return {}
81
82     def _get_internet_speed_data(self) -> Dict[str, Any]:
83         """Получает данные о скорости интернета и задержке чер
ез FAST.com.
84
85         Returns:
86             Dict[str, Any]: Данные скорости интернета (downloa
d, upload, ping, bufferbloat, loaded latency).
87         """
88         if self._internet_speed_sensor is None:
89             return {'available': False, 'error': 'InternetSpee

```

```

    dSensor not available'}
90         try:
91             metrics = self._internet_speed_sensor.measure_inte
internet_speed()
92             return {
93                 'available': metrics.get('available', True),
94                 'ping_ms': metrics.get('ping_ms', 0.0),
95                 'download_mbps': metrics.get('download_mbps',
0.0),
96                 'upload_mbps': metrics.get('upload_mbps', 0.0)
,
97                 'latency_unloaded_ms': metrics.get('latency_un
loaded_ms', metrics.get('ping_ms', 0.0)),
98                 'latency_loaded_ms': metrics.get('latency_load
ed_ms', 0.0),
99                 'bufferbloat_ms': metrics.get('bufferbloat_ms'
, 0.0),
100                 'provider': metrics.get('provider', 'fast.com'
),
101                 'server': metrics.get('server', ''),
102                 'ip': metrics.get('ip', ''),
103                 'isp': metrics.get('isp', ''),
104             }
105         except Exception as e:
106             logger.error(f'Ошибка получения данных скорости ин
тернета: {e}')
107             return {'available': False, 'error': str(e)}
108
109     def _detect_provider(self, sensor_id: str, category: str,
hardware_type: str) -> SensorProvider:
110         """Определяет провайдера сенсора на основе его ID, кат
егории и источника.
111
112         Args:
113             sensor_id: Идентификатор сенсора.
114             category: Категория сенсора (Temperatures, Load, e
tc.).
115             hardware_type: Тип оборудования (cpu, gpu, etc.).

```

```
116 |
117 |     Returns:
118 |         SensorProvider: Провайдер сенсора.
119 |         """
120 |         # GPU sensors from GpuProber
121 |         if sensor_id.startswith('gpu_') and ('_temp' in sensor
122 | _id or '_load' in sensor_id):
123 |             return SensorProvider.GPU_PROBER
124 |
125 |         # Storage sensors
126 |         if sensor_id.startswith('disk_') or sensor_id.startswi
127 | th('storage_'):
128 |             return SensorProvider.WINDOWS_STORAGE
129 |
130 |         # Internet speed sensors
131 |         if sensor_id.startswith('internet_'):
132 |             return SensorProvider.INTERNET_SPEED
133 |
134 |         # Network sensors
135 |         if sensor_id.startswith('net_'):
136 |             return SensorProvider.NETWORK_SENSOR
137 |
138 |         # CPU/RAM utilization from sensor_collector
139 |         if sensor_id in ('cpu_util_total', 'ram_util_pct', 'sw
140 | ap_util_pct'):
141 |             return SensorProvider.SENSOR_COLLECTOR
142 |
143 |         if sensor_id.startswith('cpu_core_') and '_load' in se
144 | nsor_id:
145 |             return SensorProvider.SENSOR_COLLECTOR
146 |
147 |         if sensor_id.startswith('cpu_freq'):
148 |             return SensorProvider.SENSOR_COLLECTOR
149 |
150 |         # OneDrive sensors
151 |         if sensor_id.startswith('onedrive_'):
152 |             return SensorProvider.SENSOR_COLLECTOR
```

```

150         # Network rates
151         if sensor_id in ('network_sent_rate', 'network_recv_rate'):
152             return SensorProvider.SENSOR_COLLECTOR
153
154         # Storage usage
155         if sensor_id.startswith('storage_usage_') or sensor_id
156         in ('storage_read_rate', 'storage_write_rate'):
157             return SensorProvider.SENSOR_COLLECTOR
158
159         # Default based on category
160         if category in ('Temperatures', 'temperature'):
161             return SensorProvider.ACPI_THERMAL
162
163         return SensorProvider.SENSOR_COLLECTOR
164
165     def extract_sensor_readings(
166         self, hardware_data: Dict[str, Any], *args: Any, internet_data: Optional[Dict[str, Any]] = None, **kwargs: Any
167     ) -> List[Dict[str, Any]]:
168         """Извлекает нормализованный список всех текущих показаний датчиков с дедупликацией.
169
170         Каждый элемент содержит: id, hardware_name, hardware_type, sensor_category,
171         sensor_name, unit, value, _provider.
172
173         Args:
174             hardware_data: Данные HardwareMonitor.
175             *args: Опциональные extra_sensors_data и/или internet_data.
176             internet_data: Данные скорости интернета (если переданы явно).
177
178         Returns:
179             List[Dict[str, Any]]: Нормализованный и дедуплицированный список сенсоров.
180         """

```

```

180         extra_sensors_data = None
181         if len(args) == 1:
182             if isinstance(args[0], dict) and ("ping_ms" in args[0] or "download_mbps" in args[0] or "available" in args[0]):
183                 internet_data = args[0]
184             else:
185                 extra_sensors_data = args[0]
186         elif len(args) >= 2:
187             extra_sensors_data = args[0]
188             internet_data = args[1]
189
190         raw_readings: List[Dict[str, Any]] = []
191
192         # Дополнительные внешние сенсоры
193         if extra_sensors_data and isinstance(extra_sensors_data, dict) and "sensors" in extra_sensors_data:
194             for s in extra_sensors_data["sensors"]:
195                 if isinstance(s, dict):
196                     sensor_id = s.get("id", s.get("sensor_name", "sensor"))
197                     category = s.get("sensor_category", "Metric")
198                     hardware_type = s.get("hardware_type", "sensor")
199                     provider = self._detect_provider(str(sensor_id), category, hardware_type)
200                     raw_readings.append({
201                         "id": sensor_id,
202                         "hardware_name": s.get("hardware_name", "Hardware"),
203                         "hardware_type": hardware_type,
204                         "sensor_category": category,
205                         "sensor_name": s.get("sensor_name", "Sensor"),
206                         "unit": s.get("unit", ""),
207                         "value": s.get("value_num", s.get("value", 0.0)),
208                         "_provider": provider,

```



```

209         })
210
211     if 'cpu' in hardware_data:
212         cpu = hardware_data['cpu']
213         cpu_name = cpu.get('model') or cpu.get('model_name
214         ') or 'Intel Core Processor'
215         raw_readings.append({
216             'id': 'cpu_util_total',
217             'hardware_name': cpu_name,
218             'hardware_type': 'cpu',
219             'sensor_category': 'Load',
220             'sensor_name': 'CPU Total',
221             'unit': '%',
222             'value': cpu.get('utilization_pct', 0.0),
223             '_provider': SensorProvider.SENSOR_COLLECTOR
224         })
225         if 'per_core_pct' in cpu and isinstance(cpu['per_c
226         ore_pct'], list):
227             for idx, core_val in enumerate(cpu['per_core_p
228             ct']):
229                 raw_readings.append({
230                     'id': f'cpu_core_{idx}_load',
231                     'hardware_name': cpu_name,
232                     'hardware_type': 'cpu',
233                     'sensor_category': 'Load',
234                     'sensor_name': f'CPU Core #{idx}',
235                     'unit': '%',
236                     'value': core_val,
237                     '_provider': SensorProvider.SENSOR_COL
238                     LECTOR
239                 })
240             if 'frequency_current_mhz' in cpu and cpu['frequen
241             cy_current_mhz']:
242                 raw_readings.append({
243                     'id': 'cpu_freq_current',
244                     'hardware_name': cpu_name,
245                     'hardware_type': 'cpu',
246                     'sensor_category': 'Clocks',

```

```

242         'sensor_name': 'CPU Core Frequency',
243         'unit': 'MHz',
244         'value': cpu['frequency_current_mhz'],
245         '_provider': SensorProvider.SENSOR_COLLECT
OR
246     })
247
248     if 'memory' in hardware_data:
249         ram = hardware_data['memory']
250         raw_readings.append({
251             'id': 'ram_util_pct',
252             'hardware_name': 'System Memory',
253             'hardware_type': 'memory',
254             'sensor_category': 'Load',
255             'sensor_name': 'Memory Used',
256             'unit': '%',
257             'value': ram.get('utilization_pct', 0.0),
258             '_provider': SensorProvider.SENSOR_COLLECTOR
259         })
260         if 'swap_utilization_pct' in ram and ram['swap_utilization_pct'] is not None:
261             raw_readings.append({
262                 'id': 'swap_util_pct',
263                 'hardware_name': 'System Memory',
264                 'hardware_type': 'memory',
265                 'sensor_category': 'Load',
266                 'sensor_name': 'Swap Used',
267                 'unit': '%',
268                 'value': ram['swap_utilization_pct'],
269                 '_provider': SensorProvider.SENSOR_COLLECT
OR
270             })
271
272         # GPU metrics: temperature and load are already collected via GpuProber
273         # in HardwareMonitor, so we normalize them here without re-probing
274         if 'gpus' in hardware_data and isinstance(hardware_data

```

```

a['gpus'], list):
275         for gpu in hardware_data['gpus']:
276             gpu_name = gpu.get('name', 'GPU')
277             gpu_idx = gpu.get('index', 0)
278             # GPU temperature - use hardware_data from Gpu
Prober
279             if 'temperature_gpu_c' in gpu and gpu['tempera
ture_gpu_c'] is not None:
280                 raw_readings.append({
281                     'id': f'gpu_{gpu_idx}_temp',
282                     'hardware_name': gpu_name,
283                     'hardware_type': 'gpu',
284                     'sensor_category': 'Temperatures',
285                     'sensor_name': 'GPU Core Temperature',
286                     'unit': '°C',
287                     'value': gpu['temperature_gpu_c'],
288                     '_provider': SensorProvider.GPU_PROBER
289                 })
290             # GPU load
291             if 'utilization_gpu_pct' in gpu and gpu['utili
zation_gpu_pct'] is not None:
292                 raw_readings.append({
293                     'id': f'gpu_{gpu_idx}_load',
294                     'hardware_name': gpu_name,
295                     'hardware_type': 'gpu',
296                     'sensor_category': 'Load',
297                     'sensor_name': 'GPU Core Load',
298                     'unit': '%',
299                     'value': gpu['utilization_gpu_pct'],
300                     '_provider': SensorProvider.GPU_PROBER
301                 })
302
303         if 'storage' in hardware_data:
304             storage = hardware_data['storage']
305             if 'partitions' in storage and isinstance(storage[
'partitions'], list):
306                 for part in storage['partitions']:
307                     dev = part.get('device', 'Disk')

```

```

308         raw_readings.append({
309             'id': f"storage_usage_{dev.replace(':',
310             , '').replace(chr(92), '')}",
311             'hardware_name': f'Storage ({dev})',
312             'hardware_type': 'storage',
313             'sensor_category': 'Load',
314             'sensor_name': f'Used Space {dev}',
315             'unit': '%',
316             'value': part.get('utilization_pct', 0
317             .0),
318             '_provider': SensorProvider.SENSOR_COLLECTOR
319         })
320     if 'io_rates' in storage and storage['io_rates']:
321         io = storage['io_rates']
322         raw_readings.append({
323             'id': 'storage_read_rate',
324             'hardware_name': 'Storage',
325             'hardware_type': 'storage',
326             'sensor_category': 'Throughput',
327             'sensor_name': 'Disk Read Rate',
328             'unit': 'B/s',
329             'value': io.get('read_bytes_sec', 0.0),
330             '_provider': SensorProvider.SENSOR_COLLECTOR
331         })
332     OR
333     raw_readings.append({
334         'id': 'storage_write_rate',
335         'hardware_name': 'Storage',
336         'hardware_type': 'storage',
337         'sensor_category': 'Throughput',
338         'sensor_name': 'Disk Write Rate',
339         'unit': 'B/s',
340         'value': io.get('write_bytes_sec', 0.0),
341         '_provider': SensorProvider.SENSOR_COLLECTOR
342     })
343     OR
344     raw_readings.append({
345         'id': 'storage_read_rate',
346         'hardware_name': 'Storage',
347         'hardware_type': 'storage',
348         'sensor_category': 'Throughput',
349         'sensor_name': 'Disk Read Rate',
350         'unit': 'B/s',
351         'value': io.get('read_bytes_sec', 0.0),
352         '_provider': SensorProvider.SENSOR_COLLECTOR
353     })
354     OR
355     raw_readings.append({
356         'id': 'storage_write_rate',
357         'hardware_name': 'Storage',
358         'hardware_type': 'storage',
359         'sensor_category': 'Throughput',
360         'sensor_name': 'Disk Write Rate',
361         'unit': 'B/s',
362         'value': io.get('write_bytes_sec', 0.0),
363         '_provider': SensorProvider.SENSOR_COLLECTOR
364     })

```

```

341 |         # Network metrics: aggregated throughput rates
342 |         # Note: Detailed per-interface stats are collected by
    | _probe_network_sensors() in sensors.py
343 |         if 'network' in hardware_data:
344 |             net = hardware_data['network']
345 |             raw_readings.append({
346 |                 'id': 'network_sent_rate',
347 |                 'hardware_name': 'Network',
348 |                 'hardware_type': 'network',
349 |                 'sensor_category': 'Throughput',
350 |                 'sensor_name': 'Network Upload Speed',
351 |                 'unit': 'B/s',
352 |                 'value': net.get('bytes_sent_sec', 0.0),
353 |                 '_provider': SensorProvider.SENSOR_COLLECTOR
354 |             })
355 |             raw_readings.append({
356 |                 'id': 'network_recv_rate',
357 |                 'hardware_name': 'Network',
358 |                 'hardware_type': 'network',
359 |                 'sensor_category': 'Throughput',
360 |                 'sensor_name': 'Network Download Speed',
361 |                 'unit': 'B/s',
362 |                 'value': net.get('bytes_recv_sec', 0.0),
363 |                 '_provider': SensorProvider.SENSOR_COLLECTOR
364 |             })
365 |
366 |         # Internet speed metrics
367 |         if internet_data and internet_data.get('available'):
368 |             raw_readings.append({
369 |                 'id': 'internet_download_speed',
370 |                 'hardware_name': 'Internet',
371 |                 'hardware_type': 'network',
372 |                 'sensor_category': 'Throughput',
373 |                 'sensor_name': 'Internet Download Bandwidth',
374 |                 'unit': 'Mbps',
375 |                 'value': internet_data.get('download_mbps', 0.
    | 0),
376 |                 '_provider': SensorProvider.INTERNET_SPEED

```

```

377         })
378         raw_readings.append({
379             'id': 'internet_upload_speed',
380             'hardware_name': 'Internet',
381             'hardware_type': 'network',
382             'sensor_category': 'Throughput',
383             'sensor_name': 'Internet Upload Bandwidth',
384             'unit': 'Mbps',
385             'value': internet_data.get('upload_mbps', 0.0)
386         },
387         {
388             '_provider': SensorProvider.INTERNET_SPEED
389         })
390         raw_readings.append({
391             'id': 'internet_ping',
392             'hardware_name': 'Internet',
393             'hardware_type': 'network',
394             'sensor_category': 'Throughput',
395             'sensor_name': 'Internet Ping Latency',
396             'unit': 'ms',
397             'value': internet_data.get('ping_ms', 0.0),
398             '_provider': SensorProvider.INTERNET_SPEED
399         })
400         raw_readings.append({
401             'id': 'internet_bufferbloat',
402             'hardware_name': 'Internet',
403             'hardware_type': 'network',
404             'sensor_category': 'Throughput',
405             'sensor_name': 'Internet Bufferbloat',
406             'unit': 'ms',
407             'value': internet_data.get('bufferbloat_ms', 0
408         .0),
409         {
410             '_provider': SensorProvider.INTERNET_SPEED
411         })
412         raw_readings.append({
413             'id': 'internet_latency_loaded',
414             'hardware_name': 'Internet',
415             'hardware_type': 'network',
416             'sensor_category': 'Throughput',

```

```

413         'sensor_name': 'Internet Loaded Latency',
414         'unit': 'ms',
415         'value': internet_data.get('latency_loaded_ms'
    , 0.0),
416         '_provider': SensorProvider.INTERNET_SPEED
417     })
418
419     # Дедупликация базовых показаний по приоритету провайд
    ера
420     deduplicator = SensorDeduplicator()
421     unique_readings: List[Dict[str, Any]] = []
422
423     for reading in raw_readings:
424         sensor_id = str(reading.get('id', ''))
425         value = reading.get('value', 0.0)
426         provider = reading.get('_provider', SensorProvider
    .SENSOR_COLLECTOR)
427
428         if deduplicator.add(sensor_id, provider, value):
429             unique_readings.append(reading)
430
431     # Обогащение из LibreHardwareMonitor (добавляются ТОЛЬ
    КО сенсоры, отсутствующие в базовом пуле CIM/WMI)
432     if self._lhm_service and self._lhm_service.is_running(
    ):
433         try:
434             lhm_items = self._lhm_service.get_flattened_se
    nsors()
435             existing_ids = {str(r.get('id', '')) for r in
    unique_readings}
436             for item in lhm_items:
437                 hw_name = item.get("hardware_name", "Hardw
    are")
438                 hw_type = item.get("hardware_type", "senso
    r")
439                 cat = item.get("sensor_category", "General
    ")
440                 s_name = item.get("sensor_name", "Sensor")

```

```

441
442         if "Distance to TjMax" in s_name:
443             continue
444
445         val_num = item.get("value_num")
446         if val_num is None:
447             continue
448
449         vraw = str(item.get("value_raw") or "").lower()
450         unit = ""
451         if "°c" in vraw or "c" in vraw:
452             unit = "°C"
453         elif "%" in vraw:
454             unit = "%"
455         elif "rpm" in vraw:
456             unit = "RPM"
457         elif "v" in vraw and "w" not in vraw:
458             unit = "V"
459         elif "w" in vraw:
460             unit = "W"
461         elif "mhz" in vraw:
462             unit = "MHz"
463         elif "b/s" in vraw:
464             unit = "B/s"
465         elif "gb" in vraw:
466             unit = "GB"
467
468         clean_hw = re.sub(r'^a-zA-Z0-9+', '_', hw_name.lower()).strip('_')
469         clean_name = re.sub(r'^a-zA-Z0-9+', '_', s_name.lower()).strip('_')
470         sid = f"lhm_{clean_hw}_{clean_name}"
471
472         # Пропускаем, если такой датчик уже есть и
473         # ли это дубликат общей нагрузки CPU/RAM
474         if sid in existing_ids or (cat == "Load" and "total" in clean_name and any("util_total" in eid for eid i

```



```

n existing_ids)):
474         continue
475
476         existing_ids.add(sid)
477         unique_readings.append({
478             'id': sid,
479             'hardware_name': hw_name,
480             'hardware_type': hw_type,
481             'sensor_category': cat,
482             'sensor_name': s_name,
483             'unit': unit,
484             'value': float(val_num),
485             '_provider': SensorProvider.LHM_ENRICH
ED,
486         })
487     except Exception as lhm_err:
488         logger.debug(f'Ошибка обогащения сенсорами LHM
: {lhm_err}')
489
490     return unique_readings
491
492     def get_hardware_snapshot(self) -> Dict[str, Any]:
493         """Получает полный снимок аппаратного состояния со сп
иском сенсоров.
494
495         Returns:
496             Dict[str, Any]: Снимок со списком показаний сенсо
ров.
497         """
498         with self._lock:
499             now_iso = datetime.now(timezone.utc).isoformat()
500             hardware_data = self._get_hardware_data()
501             internet_data = self._get_internet_speed_data()
502             sensor_readings = self.extract_sensor_readings(har
dware_data, internet_data)
503             sensors_payload = []
504             for r in sensor_readings:
505                 sensors_payload.append({'id': r.get('id'), 'ha

```

```

    hardware_name': r.get('hardware_name'), 'hardware_type': r.get('
hardware_type'), 'sensor_category': r.get('sensor_category'),
'sensor_name': r.get('sensor_name'), 'unit': r.get('unit'), 'v
alues': [{ 'num': r.get('value', 0.0), 'time': now_iso}])
506         result = {'timestamp': now_iso, 'sensors': sensors
_payload, 'hardware_monitor': hardware_data if hardware_data e
lse {}, 'internet_speed': internet_data if internet_data else
{}}
507         self._last_hardware_snapshot = result
508         return result
509
510     def get_last_snapshot(self) -> Optional[Dict[str, Any]]:
511         """Возвращает последний собранный снапшот.
512
513         Returns:
514             Optional[Dict[str, Any]]: Последний снапшот или No
ne.
515
516         """
517         return self._last_hardware_snapshot
518
519     def collect_all_sensors(self) -> List[Dict[str, Any]]:
520         """Собирает показания всех доступных датчиков.
521
522         Returns:
523             List[Dict[str, Any]]: Список прочитанных сенсоров.
524
525         """
526         with self._lock:
527             hardware_data = self._get_hardware_data()
528             internet_data = self._get_internet_speed_data()
529             return self.extract_sensor_readings(hardware_data,
internet_data)
530
531     def collect_lhm_cpu_readings(self) -> List[Dict[str, Any]]
:
532         """Извлекает нормализованные показания температур ядер
CPU и Package из LHM.
533
534         Returns:

```



```
566 |         "sensor_name": name,  
567 |         "unit": "°C",  
568 |         "value": val,  
569 |         "value_num": val,  
570 |     })  
571 |     core_idx += 1  
572 | return results  
573 |
```

sensor_registry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensor
  Registry
4 # =====
5 # Description:
6 #   Реестр сенсоров и политика дедупликации.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensor_registry import SensorProvider
11 #
12 #       service = SensorProvider()
13 #
14 # File: sensor_registry.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:00:00
20 # =====
21
22 from __future__ import annotations
23 """Реестр сенсоров и политика дедупликации."""
24
25 from dataclasses import dataclass, field
26 from enum import Enum
27 from typing import Dict, List, Optional, Set, Tuple
28 from datetime import datetime, timezone
29
30
31 class SensorProvider(Enum):
```

```

32 |     """Приоритеты и категории провайдеров сенсоров.
33 |
34 |     Приоритеты распределены так:
35 |     - INTERNET_SPEED (55): Выделенный speedtest
36 |     - CIM_SYSTEM (50): Основной современный нативный интерфейс
    |     Windows (CIM/MSFT)
37 |     - WINDOWS_STORAGE (45): WindowsStorageSensor (диски, SMART
    |     , температуры NVMe)
38 |     - GPU_PROBER (40): Прямой зонд GPU (NVIDIA/AMD/Intel)
39 |     - ACPI_THERMAL (35): Системные ACPI ThermalZones
40 |     - HARDWARE_MONITOR (35): Агрегатор аппаратных метрик
41 |     - SENSOR_COLLECTOR (30): Базовый коллектор системных метри
    |     к
42 |     - WMI_FALLBACK (30): WMI слой совместимости / fallback при
    |     недоступности CIM
43 |     - NETWORK_SENSOR (25): Сетевые интерфейсы psutil
44 |     - LHM_ENRICHED (20): Обогащение отсутствующими в CIM/WMI с
    |     енсорами из LibreHardwareMonitor
45 |     - NVIDIA_SMI_DIRECT (10): Прямой вызов утилит
46 |     """
47 |     INTERNET_SPEED = 55           # InternetSpeedSensor - приори
    |     тет для speedtest метрик
48 |     CIM_SYSTEM = 50               # Основной интерфейс CIM (Powe
    |     rShell / CimSession)
49 |     WINDOWS_STORAGE = 45         # WindowsStorageSensor (SMART,
    |     temps) - приоритет для storage
50 |     GPU_PROBER = 40               # GpuProber - приоритет для GP
    |     U метрик
51 |     ACPI_THERMAL = 35             # ACPI thermal zones
52 |     HARDWARE_MONITOR = 35        # HardwareMonitor (агрегатор)
53 |     SENSOR_COLLECTOR = 30        # SensorCollector.extract_sens
    |     or_readings()
54 |     WMI_FALLBACK = 30            # WMI слой совместимости
55 |     NETWORK_SENSOR = 25          # Network sensors через psutil
56 |     LHM_ENRICHED = 20            # Обогащение детальными датчик
    |     ами из LibreHardwareMonitor
57 |     NVIDIA_SMI_DIRECT = 10       # Прямой вызов nvidia-smi (leg
    |     асу)

```

```

58 |
59 |
60 | @dataclass
61 | class SensorDefinition:
62 |     """Определение сенсора с политикой дедупликации."""
63 |     sensor_id: str
64 |     category: str
65 |     preferred_provider: SensorProvider
66 |     description: str = ""
67 |     unit: str = ""
68 |     allow_multiple_sources: bool = False # Если True, дублика
ты разрешены с разными провайдерами
69 |     dedup_window_seconds: float = 1.0 # Окно дедупликации
по времени
70 |
71 |
72 | class SensorRegistry:
73 |     """Централизованный реестр сенсоров с политикой дедупликац
ии."""
74 |
75 |     # Реестр известных сенсоров с предпочтительными провайдера
ми
76 |     _registry: Dict[str, SensorDefinition] = {}
77 |
78 |     # Мappings категорий на предпочтительные провайдеры
79 |     _category_providers: Dict[str, SensorProvider] = {
80 |         "temperature": SensorProvider.ACPI_THERMAL,
81 |         "Temperatures": SensorProvider.ACPI_THERMAL,
82 |         "gpu_temp": SensorProvider.GPU_PROBER,
83 |         "gpu_load": SensorProvider.GPU_PROBER,
84 |         "gpu_power": SensorProvider.GPU_PROBER,
85 |         "gpu_fan": SensorProvider.GPU_PROBER,
86 |         "cpu_temp": SensorProvider.ACPI_THERMAL,
87 |         "cpu_load": SensorProvider.SENSOR_COLLECTOR,
88 |         "Load": SensorProvider.SENSOR_COLLECTOR, # CPU/RAM ut
ilization
89 |         "Clocks": SensorProvider.SENSOR_COLLECTOR,
90 |         "network_throughput": SensorProvider.SENSOR_COLLECTOR,

```

```

91         "network_bytes": SensorProvider.NETWORK_SENSOR,
92         "Throughput": SensorProvider.SENSOR_COLLECTOR,
93         "internet": SensorProvider.INTERNET_SPEED,
94         "storage_temp": SensorProvider.WINDOWS_STORAGE,
95         "storage_wear": SensorProvider.WINDOWS_STORAGE,
96         "storage_usage": SensorProvider.SENSOR_COLLECTOR,
97         "storage": SensorProvider.SENSOR_COLLECTOR,
98     }
99
100     # Паттерны дублирующихся sensor_id
101     _duplicate_patterns: Dict[str, List[str]] = {
102         # GPU метрики - собираются предпочтительно через GpuPr
103         "gpu_*": ["gpu_prober", "sensor_collector", "nvidia_s
104         mi_direct"],
105         # CPU метрики
106         "cpu_*": ["acpi_thermal", "sensor_collector"],
107         # Internet ping - дублируется в sensors.py и sensor_co
108         llector
109         "internet_ping": ["internet_speed", "sensor_collector"
110         ],
111         # Network throughput - дублируется в sensors.py и sens
112         or_collector
113         "network_*_rate": ["network_sensor", "sensor_collecto
114         r"],
115     }
116
117     @classmethod
118     def register(cls, definition: SensorDefinition) -> None:
119         """Регистрирует сенсор в реестре.
120
121         Args:
122             definition: Определение сенсора с политикой.
123         """
124         cls._registry[definition.sensor_id] = definition
125
126     @classmethod
127     def get_preferred_provider(cls, sensor_id: str, category:

```



```

    str) -> Optional[SensorProvider]:
123     """Возвращает предпочтительный провайдер для данного с
    енсора.
124
125     Args:
126         sensor_id: Идентификатор сенсора.
127         category: Категория сенсора.
128
129     Returns:
130         Preferred provider или None если не определен.
131     """
132     # 1. Проверяем явную регистрацию
133     if sensor_id in cls._registry:
134         return cls._registry[sensor_id].preferred_provider
135
136     # 2. Определяем по специфическому паттерну sensor_id
137     import re
138     for pattern, providers in cls._duplicate_patterns.items():
139         if re.match(pattern, sensor_id):
140             # Возвращаем первый из списка как предпочтительный
141             provider_name = providers[0].upper()
142             try:
143                 return SensorProvider(provider_name)
144             except KeyError:
145                 pass
146
147     # 3. Проверяем по категории
148     if category in cls._category_providers:
149         return cls._category_providers[category]
150
151     return None
152
153     @classmethod
154     def should_keep_sensor(
155         cls,
156         sensor_id: str,

```

```

157         provider: SensorProvider,
158         existing_sensors: List[Tuple[str, SensorProvider]]
159     ) -> bool:
160         """Определяет, нужно ли сохранить показание сенсора ил
и отклонить как дубликат.
161
162         Args:
163             sensor_id: Идентификатор сенсора.
164             provider: Провайдер, предоставивший показание.
165             existing_sensors: Список уже собранных (sensor_id,
provider) пар.
166
167         Returns:
168             True если сенсор нужно сохранить, False если это д
убликат.
169         """
170         # Проверяем, есть ли уже такой sensor_id
171         for existing_id, existing_provider in existing_sensors
:
172             if existing_id == sensor_id:
173                 # Дубликат найден - сравниваем приоритеты пров
айдеров
174                 preferred = cls.get_preferred_provider(sensor_
id, "")
175                 if preferred:
176                     # Сохраняем только от предпочтительного пр
овайдера
177                     return provider == preferred
178                 else:
179                     # Нет явной политики - сохраняем от провай
дера с высшим приоритетом
180                     return provider.value > existing_provider.
value
181
182         # Дубликатов нет - сохраняем
183         return True
184
185

```

```

186 class SensorDeduplicator:
187     """Дедупликатор показаний сенсоров."""
188
189     def __init__(self) -> None:
190         """Инициализирует дедупликатор с пустым кешем."""
191         self._seen: Dict[str, Tuple[float, SensorProvider, flo
192 at]] = {} # sensor_id -> (timestamp, provider, value)
193
194     def add(
195         self,
196         sensor_id: str,
197         provider: SensorProvider,
198         value: float,
199         timestamp: Optional[float] = None,
200         dedup_window: float = 1.0
201     ) -> bool:
202         """Пытается добавить показание сенсора с проверкой на
203 дубликаты.
204
205     Args:
206         sensor_id: Идентификатор сенсора.
207         provider: Провайдер показания.
208         value: Значение показания.
209         timestamp: Временная метка (epoch seconds).
210         dedup_window: Окно дедупликации в секундах.
211
212     Returns:
213         True если показание добавлено, False если отклонен
214 о как дубликат.
215
216     """
217     if timestamp is None:
218         timestamp = datetime.now(timezone.utc).timestamp()
219
220     # Проверяем наличие в кеше
221     if sensor_id in self._seen:
222         cached_ts, cached_provider, cached_value = self._s
223 een[sensor_id]

```

```

220         # Проверяем временное окно
221         if abs(timestamp - cached_ts) < dedup_window:
222             # В пределах окна - проверяем приоритеты
223             preferred = SensorRegistry.get_preferred_provi
224             der(sensor_id, "")
225             if preferred:
226                 # Если текущий провайдер предпочтительнее
227                 - обновляем
228                 if provider == preferred:
229                     self._seen[sensor_id] = (timestamp, pr
230                     ovider, value)
231                     return True
232                 else:
233                     # Отклоняем
234                     return False
235             else:
236                 # Сравниваем по значению приоритета
237                 if provider.value > cached_provider.value:
238                     self._seen[sensor_id] = (timestamp, pr
239                     ovider, value)
240                     return True
241                 else:
242                     return False
243
244         # Нет дубликата или вне окна - добавляем
245         self._seen[sensor_id] = (timestamp, provider, value)
246         return True
247
248     def get_unique(self) -> Dict[str, Tuple[SensorProvider, fl
249     oat]]:
250         """Возвращает уникальные показания сенсоров.
251
252         Returns:
253             Словарь {sensor_id: (provider, value)}.
254         """
255         return {sid: (prov, val) for sid, (_, prov, val) in se
256         lf._seen.items()}

```

```
252 |     def clear(self) -> None:
253 |         """Очищает кеш дедупликатора."""
254 |         self._seen.clear()
255 |
256 |
257 | def deduplicate_sensor_readings(
258 |     readings: List[Dict],
259 |     provider: SensorProvider = SensorProvider.SENSOR_COLLECTOR
260 | ) -> List[Dict]:
261 |     """Дедуплицирует список показаний сенсоров.
262 |
263 |     Args:
264 |         readings: Список показаний с полями 'id' и 'value'.
265 |         provider: Провайдер показаний.
266 |
267 |     Returns:
268 |         Дедуплицированный список показаний.
269 |     """
270 |     deduplicator = SensorDeduplicator()
271 |     unique_readings: List[Dict] = []
272 |
273 |     for reading in readings:
274 |         sensor_id = reading.get('id', '')
275 |         value = reading.get('value', 0.0)
276 |
277 |         if deduplicator.add(sensor_id, provider, value):
278 |             unique_readings.append(reading)
279 |
280 |     return unique_readings
281 |
```

```

service.py
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Service
4 # =====
5 # Description:
6 #   Фоновый сервис посекундного сбора системных метрик с сохра
   нением в SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.service import TelemetryLogg
   erService
11 #
12 #       instance = TelemetryLoggerService.get_instance()
13 #
14 # File: service.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 03:15:00
20 # =====
21
22 from __future__ import annotations
23 """Фоновый сервис периодического сбора системных метрик с сохр
   анением в SQLite."""
24
25 import asyncio
26 import threading
27 import time
28 from typing import Any, Dict, List, Optional
29 try:
30     from logger import logger

```

```

31 except ImportError:
32     from logger import logger
33 from .collector import SystemCollector
34 from apps.windows.telemetry_research.hardware_history_manager
    import HardwareHistoryManager
35 from .sqlite import TelemetryStorage
36 from .telemetry_config import TelemetryConfigManager
37 from .w64_collector import AIW64Collector
38 from .w64_etw_collector import AIW64ETWCollector
39
40 class TelemetryLoggerService:
41     """Сервис периодического сбора системной телеметрии, аудит
    а оборудования и W64/ETW событий в SQLite."""
42     _instance: Optional[TelemetryLoggerService] = None
43
44     def __init__(
45         self,
46         interval_sec: Optional[float] = None,
47         top_processes: Optional[int] = None,
48         collector: Optional[SystemCollector] = None,
49         storage: Optional[TelemetryStorage] = None,
50         hardware_audit_interval_sec: Optional[float] = None,
51         rollup_interval_sec: Optional[float] = None,
52         enable_w64: Optional[bool] = None,
53         config_manager: Optional[TelemetryConfigManager] = Non
    e,
54         db_cleanup_interval_sec: Optional[float] = None,
55     ) -> None:
56         """Инициализирует сервис сбора системных метрик.
57
58         Args:
59             interval_sec: Интервал между замерами телеметрии в
    секундах (по умолчанию из config.json).
60             top_processes: Лимит сохраняемых активных процессо
    в (по умолчанию из config.json).
61             collector: Экземпляр сборщика SystemCollector (опц
    ионально).
62             storage: Экземпляр хранилища TelemetryStorage (опц

```

ионально).

```
63         hardware_audit_interval_sec: Интервал периодическо
го аудита железа (по умолчанию из config.json).
64         rollup_interval_sec: Интервал запуска обобщения ст
арых данных (по умолчанию из config.json).
65         enable_w64: Включение подсистемы сбора событий W64
/ETW (по умолчанию из config.json).
66         config_manager: Менеджер конфигурации.
67         db_cleanup_interval_sec: Интервал фонового контрол
я размера базы данных (сек).
68         """
69         self.config_manager = config_manager or TelemetryConfi
gManager()
70         self._custom_interval_set = (interval_sec is not None)
71         self.interval_sec = max(
72             0.1,
73             interval_sec if interval_sec is not None else self
.config_manager.get_interval_seconds()
74         )
75         self.top_processes = max(
76             1,
77             top_processes if top_processes is not None else se
lf.config_manager.get_effective_process_limit()
78         )
79         self.hardware_audit_interval_sec = max(
80             1.0,
81             hardware_audit_interval_sec if hardware_audit_inte
rval_sec is not None else self.config_manager.get_heavy_interv
al_seconds()
82         )
83         self.rollup_interval_sec = max(
84             1.0,
85             rollup_interval_sec if rollup_interval_sec is not
None else self.config_manager.get_aggregation_interval()
86         )
87         self.collector = collector or SystemCollector()
88         self.storage = storage or TelemetryStorage.get_instanc
e()
```



```

89         self.db_cleanup_interval_sec = max(
90             5.0,
91             db_cleanup_interval_sec if db_cleanup_interval_sec
is not None else self.config_manager.get_db_cleanup_interval_
seconds()
92         )
93
94         cfg = self.config_manager.get_config()
95         w64_cfg = cfg.get('w64_collector', {})
96         self.enable_w64 = enable_w64 if enable_w64 is not None
else w64_cfg.get('enabled', True)
97
98         self._w64_collector: Optional[AIW64Collector] = None
99         self._w64_etw_collector: Optional[AIW64ETWCollector] =
None
100
101         self._running = False
102         self._thread: Optional[threading.Thread] = None
103         self._stop_event = threading.Event()
104         self._ticks_count = 0
105         self._start_time: Optional[float] = None
106         self._last_tick_time: Optional[float] = None
107         self._last_hw_audit_time: Optional[float] = None
108         self._last_rollup_time: Optional[float] = None
109         self._last_db_cleanup_time: Optional[float] = None
110         self._last_error: Optional[str] = None
111         self._last_snapshot: Optional[Any] = None
112
113     @classmethod
114     def get_instance(cls) -> TelemetryLoggerService:
115         """Возвращает глобальный синглтон сервиса телеметрии.
116
117         Returns:
118             TelemetryLoggerService: Единственный экземпляр сер
виса.
119         """
120         if cls._instance is None:
121             cls._instance = TelemetryLoggerService()

```

```

122         return cls._instance
123
124     @property
125     def is_running(self) -> bool:
126         """Проверяет, запущен ли фоновый сбор телеметрии.
127
128         Returns:
129             bool: True если сбор активен, иначе False.
130         """
131         return self._running and self._thread is not None and
self._thread.is_alive()
132
133     @property
134     def history_manager(self) -> HardwareHistoryManager:
135         """Возвращает менеджер истории оборудования."""
136         return self.collector.history_manager
137
138     @property
139     def w64_collector(self) -> Optional[AIW64Collector]:
140         """Возвращает сборщик W64 событий (создается лениво пр
и enable_w64)."""
141         if self._w64_collector is None and self.enable_w64:
142             self._init_w64_collectors()
143         return self._w64_collector
144
145     @w64_collector.setter
146     def w64_collector(self, val: Optional[AIW64Collector]) ->
None:
147         self._w64_collector = val
148
149     @property
150     def w64_etw_collector(self) -> Optional[AIW64ETWCollector]
:
151         """Возвращает сборщик ETW событий (создается лениво пр
и enable_w64)."""
152         if self._w64_etw_collector is None and self.enable_w64
:
153             self._init_w64_collectors()

```

```

154         return self._w64_etw_collector
155
156     @w64_etw_collector.setter
157     def w64_etw_collector(self, val: Optional[AIW64ETWCollector]) -> None:
158         self._w64_etw_collector = val
159
160     def _init_w64_collectors(self) -> None:
161         """Инициализирует W64 и ETW сборщики при необходимости
162         ."""
163         if not self.enable_w64:
164             return
165         cfg = self.config_manager.get_config()
166         w64_cfg = cfg.get('w64_collector', {})
167         if self._w64_collector is None:
168             self._w64_collector = AIW64Collector(
169                 storage=self.storage,
170                 enable_file_monitoring=w64_cfg.get('enable_file_monitoring', True),
171                 enable_process_monitoring=w64_cfg.get('enable_process_monitoring', True),
172                 enable_registry_monitoring=w64_cfg.get('enable_registry_monitoring', True),
173                 enable_network_monitoring=w64_cfg.get('enable_network_monitoring', True),
174                 enable_event_log_monitoring=w64_cfg.get('enable_event_log_monitoring', True),
175             )
176         if self._w64_etw_collector is None:
177             self._w64_etw_collector = AIW64ETWCollector(
178                 storage=self.storage,
179                 enable_process_trace=w64_cfg.get('enable_process_trace', True),
180                 enable_disk_trace=w64_cfg.get('enable_disk_trace', True),
181                 enable_network_trace=w64_cfg.get('enable_network_trace', True),
182                 enable_registry_trace=w64_cfg.get('enable_registry_trace', True)

```

```

    stry_trace', True),
182         )
183
184     def start(self) -> bool:
185         """Запускает фоновый поток сбора телеметрии, аудита об
орудования и W64/ETW сборщиков.
186
187         Returns:
188             bool: True если сервис успешно запущен, False если
уже работал.
189         """
190         if self.is_running:
191             logger.debug('Сервис сбора телеметрии уже запущен.
')
192             return False
193         self._stop_event.clear()
194         self._running = True
195         self._start_time = time.time()
196         self._ticks_count = 0
197         self._last_error = None
198         self._last_hw_audit_time = None
199         self._last_db_cleanup_time = None
200
201         if self.enable_w64:
202             self._init_w64_collectors()
203             if self.w64_collector:
204                 self.w64_collector.start()
205             if self.w64_etw_collector:
206                 self.w64_etw_collector.start()
207
208         self._thread = threading.Thread(target=self._worker_lo
op, name='TelemetryLoggerWorker', daemon=True)
209         self._thread.start()
210         logger.info(f'Фоновый сервис телеметрии запущен в БД (
интервал: {self.interval_sec}с, аудит железа: {self.hardware_a
udit_interval_sec}с, W64: {self.enable_w64}, БД: {self.storage
.db_path})')
211         return True

```

```

212
213     def stop(self) -> bool:
214         """Останавливает фоновый поток сбора телеметрии и W64/
ETW сборщиков.
215
216         Returns:
217             bool: True если сервис был остановлен, False если
он не работал.
218         """
219         if not self._running:
220             return False
221         self._running = False
222         self._stop_event.set()
223
224         if self.w64_collector:
225             self.w64_collector.stop()
226         if self.w64_etw_collector:
227             self.w64_etw_collector.stop()
228
229         if self._thread and self._thread.is_alive():
230             self._thread.join(timeout=3.0)
231         self._thread = None
232         try:
233             self.storage.flush()
234         except Exception:
235             pass
236         logger.info(f'Фоновый сервис телеметрии остановлен. Вс
его тиков: {self._ticks_count}')
237         return True
238
239     def sync_config_from_manager(self) -> bool:
240         """Синхронизирует интервалы и настройки сервиса из мен
еджера конфигурации при их изменении.
241
242         Returns:
243             bool: True если конфигурация изменилась и была при
менена, иначе False.
244         """

```

```

245         self.config_manager.check_tc_mode_auto_switch()
246         if self.config_manager.check_and_reload():
247             if not self._custom_interval_set:
248                 self.interval_sec = max(0.1, self.config_manag
er.get_interval_seconds())
249                 self.top_processes = max(1, self.config_manager.ge
t_effective_process_limit())
250                 self.hardware_audit_interval_sec = max(1.0, self.c
onfig_manager.get_heavy_interval_seconds())
251                 self.db_cleanup_interval_sec = max(5.0, self.conf
ig_manager.get_db_cleanup_interval_seconds())
252                 self.rollup_interval_sec = max(1.0, self.config_ma
nager.get_aggregation_interval())
253                 logger.info(
254                     f" [Сервис телеметрии] Конфигурация config.j
son обновлена 'на лету': "
255                     f"базовый={self.interval_sec}с, аудит железа={
self.hardware_audit_interval_sec}с, "
256                     f"очистка БД={self.db_cleanup_interval_sec}с,
топ процессов={self.top_processes}, mode={self.config_manager.
get_telemetry_mode()}"
257                 )
258                 return True
259             return False
260
261     def _worker_loop(self) -> None:
262         """Основной рабочий цикл фонового сбора метрик, записи
в БД и аудита изменений."""
263         while not self._stop_event.is_set():
264             loop_start = time.time()
265             try:
266                 # Проверка изменения config.json 'на лету'
267                 self.sync_config_from_manager()
268
269                 loop = None
270                 try:
271                     loop = asyncio.get_event_loop()
272                 except RuntimeError:

```

```
273         loop = None
274         if loop and loop.is_running():
275             task = asyncio.run_coroutine_threadsafe(se
lf.collector.get_snapshot(process_limit=self.top_processes), l
oop)
276             snapshot = task.result(timeout=30.0)
277         else:
278             snapshot = asyncio.run(self.collector.get_
snapshot(process_limit=self.top_processes))
279             self._last_snapshot = snapshot
280             self._ticks_count += 1
281             self._last_tick_time = time.time()
282         try:
283             self.storage.save_snapshot(snapshot, top_n
=self.top_processes)
284         except Exception as db_err:
285             logger.warning(f'Ошибка сохранения снимка
телеметрии в БД: {db_err}')
286             if self._last_hw_audit_time is None or loop_st
art - self._last_hw_audit_time >= self.hardware_audit_interval
_sec:
287                 try:
288                     archive_entry = self.collector.archive
_hardware_state(auto_diff=True)
289                     if archive_entry:
290                         self.storage.save_hardware_archive
(archive_entry)
291                 except Exception as hw_ex:
292                     logger.debug(f'Ошибка при периодическо
м аудите железа: {hw_ex}')
293                 try:
294                     self.collector.get_extended_system_aud
it()
295                 except Exception as ext_ex:
296                     logger.debug(f'Ошибка при периодическо
м расширенном аудите: {ext_ex}')
297                     self._last_hw_audit_time = loop_start
298                     if self._last_rollup_time is None or loop_star
```

```

t - self._last_rollup_time >= self.rollup_interval_sec:
299         try:
300             self.run_rollups()
301             self._last_rollup_time = loop_start
302         except Exception as roll_ex:
303             logger.debug(f'Ошибка при периодическо
м обобщении процессов: {roll_ex}')
304             if self._last_db_cleanup_time is None or loop_
start - self._last_db_cleanup_time >= self.db_cleanup_interval
_sec:
305                 try:
306                     self.cleanup_db()
307                     self._last_db_cleanup_time = loop_star
t
308                 except Exception as clean_ex:
309                     logger.debug(f'Ошибка при периодическо
м контроле размера БД: {clean_ex}')
310             except (RuntimeError, ValueError) as shut_ex:
311                 if 'shutdown' in str(shut_ex).lower() or 'clos
ed file' in str(shut_ex).lower():
312                     break
313                 self._last_error = str(shut_ex)
314                 logger.debug(f'Ошибка при сборе системной теле
метрии: {shut_ex}')
315             except Exception as ex:
316                 self._last_error = str(ex)
317                 logger.debug(f'Ошибка при сборе системной теле
метрии: {ex}')
318             elapsed = time.time() - loop_start
319             sleep_time = max(0.01, self.interval_sec - elapsed
)
320             if self._stop_event.wait(timeout=sleep_time):
321                 break
322
323     def run_rollups(self) -> Dict[str, Any]:
324         """Принудительно запускает процедуры обобщения устарев
ших метрик процессов.
325

```



```

326         Returns:
327             Dict[str, Any]: Сводная статистика обобщения за 2
328             минуты и за 1 день.
329             """
330             short_res = self.storage.aggregate_process_metrics_2mi
331             n(cutoff_seconds=120)
332             daily_res = self.storage.aggregate_process_metrics_dai
333             ly(cutoff_days=1)
334             return {'rollup_2min': short_res, 'rollup_daily': dai
335             ly_res}
336
337     def cleanup_db(self) -> Dict[str, Any]:
338         """Запускает процедуру контроля размера и усечения баз
339         ы данных телеметрии.
340
341         Returns:
342             Dict[str, Any]: Отчет о результатах контроля разме
343             ра.
344             """
345             return self.storage.enforce_size_limit()
346
347     def get_status(self) -> Dict[str, Any]:
348         """Возвращает текущий статус сервиса телеметрии.
349
350         Returns:
351             Dict[str, Any]: Словарь с состоянием и статистикой
352             работы сервиса.
353             """
354             uptime = round(time.time() - self._start_time, 1) if s
355             elf._start_time and self.is_running else 0.0
356             return {'is_running': self.is_running, 'interval_sec':
357             self.interval_sec, 'top_processes_limit': self.top_processes,
358             'hardware_audit_interval_sec': self.hardware_audit_interval_s
359             ec, 'ticks_recorded': self._ticks_count, 'uptime_seconds': upt
360             ime, 'last_tick_epoch': self._last_tick_time, 'last_hw_audit_e
361             poch': self._last_hw_audit_time, 'last_error': self._last_erro
362             r, 'storage_stats': self.storage.get_storage_stats()}

```

```

350     def get_last_snapshot(self) -> Optional[Any]:
351         """Возвращает последний собранный снапшот.
352
353         Returns:
354             Optional[Any]: Последний снапшот или None.
355         """
356         return self._last_snapshot
357
358     def get_history(self, limit: int=60) -> List[Dict[str, Any
359 ]]:
360         """Возвращает историю системных снимков из SQLite базы
361         данных.
362
363         Args:
364             limit: Количество последних записей.
365
366         Returns:
367             List[Dict[str, Any]]: Список исторических срезов.
368         """
369         return self.storage.get_snapshots(limit=limit)
370
371     def record_event(self, event_type: str, event_details: Dic
372 t[str, Any], severity: str='info') -> int:
373         """Записывает событие в SQLite базу данных телеметрии.
374
375         Args:
376             event_type: Тип события (hardware_change, process_
377 start, driver_update).
378             event_details: Детали события.
379             severity: Уровень серьезности события.
380
381         Returns:
382             int: ID добавленной записи события.
383         """
384         logger.info(f'[Событие Telemetry] {event_type}: {event
385 _details}')
386         try:
387             return self.storage.save_event(event_type, event_d

```

```
        details, severity=severity)
383 |         except Exception as ex:
384 |             logger.error(f'Не удалось записать событие в базу
        данных: {ex}')
385 |         return 0
```

aggregator.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Aggregator
4 # =====
5 # Description:
6 #   Многоуровневый SQL-агрегатор временных рядов телеметрии (r
  aw -> hourly ->
7 #   daily -> weekly -> monthly -> yearly) и планировщик агрега
  ции.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.sqlite.aggregator import Tel
  emetrySqlAggregator
12 #
13 #       agg = TelemetrySqlAggregator(connection_manager)
14 #       agg.run_pipeline()
15 #
16 # File: aggregator.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry.sqlite
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-06 00:20:00
22 # =====
23
24 from __future__ import annotations
25
26 """Многоуровневый SQL-агрегатор временных рядов телеметрии и п
  ланировщик агрегации."""
27
28 import sqlite3
```

```

29 import time
30 from datetime import datetime, timezone
31 from enum import Enum
32 from typing import Any, Dict, List, Optional, Union
33
34 from .connection import TelemetryConnectionManager
35
36 try:
37     from logger import logger
38 except ImportError:
39     import logging
40     logger = logging.getLogger(__name__)
41
42
43 class AggregationLevel(str, Enum):
44     """Уровни иерархической агрегации временных рядов телеметр
ии."""
45
46     HOURLY = "hourly"
47     DAILY = "daily"
48     WEEKLY = "weekly"
49     MONTHLY = "monthly"
50     YEARLY = "yearly"
51
52
53 def sensors_aggregate(
54     conn: sqlite3.Connection,
55     level: Union[AggregationLevel, str],
56     start_epoch: Optional[float] = None,
57     end_epoch: Optional[float] = None,
58     detect_spikes: bool = True,
59 ) -> Dict[str, int]:
60     """Выполняет единую SQL-агрегацию сенсоров для указанного
уровня иерархии.
61
62     Args:
63         conn: Активное подключение к базе данных SQLite.
64         level: Уровень агрегации ('hourly', 'daily', 'weekly',

```

```

    'monthly', 'yearly')).
65         start_epoch: Начальная граница выборки в секундах epoch
    h (опционально).
66         end_epoch: Конечная граница выборки в секундах epoch (
    опционально).
67         detect_spikes: Флаг сохранения статистических всплеско
    в/аномалий в telemetry_spikes.
68
69     Returns:
70         Dict[str, int]: Словарь с количеством обработанных бак
    етов и сохраненных аномалий.
71     """
72     lvl_str = str(level.value if isinstance(level, Aggregation
    Level) else level).lower()
73     cursor = conn.cursor()
74     now_epoch = time.time()
75     res = {"level": lvl_str, "buckets_aggregated": 0, "spikes_
    recorded": 0}
76
77     if lvl_str == AggregationLevel.HOURLY.value:
78         s_epoch = float(start_epoch) if start_epoch is not Non
    e else 0.0
79         e_epoch = float(end_epoch) if end_epoch is not None el
    se (int(now_epoch // 3600) * 3600.0)
80
81         # 1. SQL агрегация сырых измерений в часовые бакеты
82         cursor.execute("""
83             INSERT INTO telemetry_hourly (
84                 sensor_id,
85                 bucket_start,
86                 bucket_end,
87                 sample_count,
88                 value_min,
89                 value_max,
90                 value_avg,
91                 value_sum,
92                 value_sum_sq,
93                 value_stddev,

```

```

94         value_first,
95         value_last
96     )
97     WITH raw_window AS (
98         SELECT
99             sensor_id,
100             CAST(created_at / 3600 AS INTEGER) * 3600
101         AS b_start,
102             created_at,
103             value,
104             FIRST_VALUE(value) OVER (
105                 PARTITION BY sensor_id, CAST(created_a
106             t / 3600 AS INTEGER) * 3600
107                 ORDER BY created_at ASC
108             ) AS f_val,
109             FIRST_VALUE(value) OVER (
110                 PARTITION BY sensor_id, CAST(created_a
111             t / 3600 AS INTEGER) * 3600
112                 ORDER BY created_at DESC
113             ) AS l_val
114         FROM sensor_polls
115         WHERE created_at >= ? AND created_at < ? AND v
116         alue IS NOT NULL
117     )
118     SELECT
119         sensor_id,
120         b_start AS bucket_start,
121         b_start + 3600 AS bucket_end,
122         COUNT(*) AS sample_count,
123         MIN(value) AS value_min,
124         MAX(value) AS value_max,
125         AVG(value) AS value_avg,
126         SUM(value) AS value_sum,
127         SUM(value * value) AS value_sum_sq,
128         SQRT(MAX(0.0, (SUM(value * value) - (SUM(value
129         ) * SUM(value)) / COUNT(*)) / COUNT(*))) AS value_stddev,
130         MIN(f_val) AS value_first,
131         MAX(l_val) AS value_last

```

```

127         FROM raw_window
128         GROUP BY sensor_id, b_start
129         ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
130             bucket_end = excluded.bucket_end,
131             sample_count = excluded.sample_count,
132             value_min = excluded.value_min,
133             value_max = excluded.value_max,
134             value_avg = excluded.value_avg,
135             value_sum = excluded.value_sum,
136             value_sum_sq = excluded.value_sum_sq,
137             value_stddev = excluded.value_stddev,
138             value_first = excluded.value_first,
139             value_last = excluded.value_last;
140         """, (s_epoch, e_epoch))
141         res["buckets_aggregated"] = cursor.rowcount if cursor.
rowcount > 0 else 0
142
143         # 2. Выявление и сохранение аномальных всплесков
144         if detect_spikes:
145             cursor.execute("""
146                 INSERT INTO telemetry_spikes (
147                     sensor_id, timestamp, created_at, value, s
pike_type,
148                     baseline_avg, baseline_stddev, details
149                 )
150                 SELECT
151                     p.sensor_id,
152                     p.timestamp,
153                     p.created_at,
154                     p.value,
155                     CASE WHEN p.value >= h.value_avg THEN 'hig
h' ELSE 'low' END,
156                     h.value_avg,
157                     h.value_stddev,
158                     'Всплеск: знач=' || ROUND(p.value, 2) || '
, avg=' || ROUND(h.value_avg, 2) || ', std=' || ROUND(h.value_
stddev, 2)
159                     FROM sensor_polls p

```



```

160         JOIN telemetry_hourly h
161         ON p.sensor_id = h.sensor_id
162         AND (CAST(p.created_at / 3600 AS INTEGER) * 3
163             600) = h.bucket_start
164         WHERE h.value_stddev > 0.01
165         AND ABS(p.value - h.value_avg) >= 2.5 * h.va
166             lue_stddev
167         AND p.created_at >= ? AND p.created_at < ?
168         AND NOT EXISTS (
169             SELECT 1 FROM telemetry_spikes s
170             WHERE s.sensor_id = p.sensor_id AND s.cr
171             eated_at = p.created_at
172             );
173         "", (s_epoch, e_epoch))
174         res["spikes_recorded"] = cursor.rowcount if cursor
175         .rowcount > 0 else 0
176
177     elif lvl_str == AggregationLevel.DAILY.value:
178         s_epoch = float(start_epoch) if start_epoch is not Non
179         e else 0.0
180         e_epoch = float(end_epoch) if end_epoch is not None el
181         se (int(now_epoch // 86400) * 86400.0)
182
183         cursor.execute("""
184             INSERT INTO telemetry_daily (
185                 sensor_id,
186                 bucket_start,
187                 bucket_end,
188                 sample_count,
189                 value_min,
190                 value_max,
191                 value_avg,
192                 value_sum,
193                 value_sum_sq,
194                 value_stddev,
195                 value_first,
196                 value_last
197             )

```

```

192 |         WITH hourly_window AS (
193 |             SELECT
194 |                 sensor_id,
195 |                 CAST(bucket_start / 86400 AS INTEGER) * 86
    | 400 AS b_start,
196 |                 bucket_start,
197 |                 sample_count,
198 |                 value_min,
199 |                 value_max,
200 |                 value_sum,
201 |                 value_sum_sq,
202 |                 value_first,
203 |                 value_last,
204 |                 FIRST_VALUE(value_first) OVER (
205 |                     PARTITION BY sensor_id, CAST(bucket_st
    | art / 86400 AS INTEGER) * 86400
206 |                     ORDER BY bucket_start ASC
207 |                 ) AS f_val,
208 |                 FIRST_VALUE(value_last) OVER (
209 |                     PARTITION BY sensor_id, CAST(bucket_st
    | art / 86400 AS INTEGER) * 86400
210 |                     ORDER BY bucket_start DESC
211 |                 ) AS l_val
212 |             FROM telemetry_hourly
213 |             WHERE bucket_start >= ? AND bucket_start < ?
214 |         )
215 |     SELECT
216 |         sensor_id,
217 |         b_start AS bucket_start,
218 |         b_start + 86400 AS bucket_end,
219 |         SUM(sample_count) AS sample_count,
220 |         MIN(value_min) AS value_min,
221 |         MAX(value_max) AS value_max,
222 |         SUM(value_sum) / SUM(sample_count) AS value_av
    | g,
223 |         SUM(value_sum) AS value_sum,
224 |         SUM(value_sum_sq) AS value_sum_sq,
225 |         SQRT(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_

```

```

sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
))) AS value_stddev,
226         MIN(f_val) AS value_first,
227         MAX(l_val) AS value_last
228     FROM hourly_window
229     GROUP BY sensor_id, b_start
230     ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
231         bucket_end = excluded.bucket_end,
232         sample_count = excluded.sample_count,
233         value_min = excluded.value_min,
234         value_max = excluded.value_max,
235         value_avg = excluded.value_avg,
236         value_sum = excluded.value_sum,
237         value_sum_sq = excluded.value_sum_sq,
238         value_stddev = excluded.value_stddev,
239         value_first = excluded.value_first,
240         value_last = excluded.value_last;
241     """ , (s_epoch, e_epoch))
242     res["buckets_aggregated"] = cursor.rowcount if cursor.
rowcount > 0 else 0
243
244     elif lvl_str == AggregationLevel.WEEKLY.value:
245         s_epoch = float(start_epoch) if start_epoch is not Non
e else 0.0
246         e_epoch = float(end_epoch) if end_epoch is not None el
se (int(now_epoch // 604800) * 604800.0)
247
248         cursor.execute("""
249             INSERT INTO telemetry_weekly (
250                 sensor_id,
251                 bucket_start,
252                 bucket_end,
253                 sample_count,
254                 value_min,
255                 value_max,
256                 value_avg,
257                 value_sum,
258                 value_sum_sq,

```

```

259         value_stddev,
260         value_first,
261         value_last
262     )
263     WITH daily_window AS (
264         SELECT
265             sensor_id,
266             CAST(bucket_start / 604800 AS INTEGER) * 6
267             04800 AS b_start,
268             bucket_start,
269             sample_count,
270             value_min,
271             value_max,
272             value_sum,
273             value_sum_sq,
274             value_first,
275             value_last,
276             FIRST_VALUE(value_first) OVER (
277                 PARTITION BY sensor_id, CAST(bucket_st
278                 art / 604800 AS INTEGER) * 604800
279                 ORDER BY bucket_start ASC
280             ) AS f_val,
281             FIRST_VALUE(value_last) OVER (
282                 PARTITION BY sensor_id, CAST(bucket_st
283                 art / 604800 AS INTEGER) * 604800
284                 ORDER BY bucket_start DESC
285             ) AS l_val
286         FROM telemetry_daily
287         WHERE bucket_start >= ? AND bucket_start < ?
288     )
289     SELECT
290         sensor_id,
291         b_start AS bucket_start,
292         b_start + 604800 AS bucket_end,
293         SUM(sample_count) AS sample_count,
294         MIN(value_min) AS value_min,
295         MAX(value_max) AS value_max,
296         SUM(value_sum) / SUM(sample_count) AS value_av

```

```

g,
294         SUM(value_sum) AS value_sum,
295         SUM(value_sum_sq) AS value_sum_sq,
296         SQRT(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_
sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
))) AS value_stddev,
297         MIN(f_val) AS value_first,
298         MAX(l_val) AS value_last
299     FROM daily_window
300     GROUP BY sensor_id, b_start
301     ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
302         bucket_end = excluded.bucket_end,
303         sample_count = excluded.sample_count,
304         value_min = excluded.value_min,
305         value_max = excluded.value_max,
306         value_avg = excluded.value_avg,
307         value_sum = excluded.value_sum,
308         value_sum_sq = excluded.value_sum_sq,
309         value_stddev = excluded.value_stddev,
310         value_first = excluded.value_first,
311         value_last = excluded.value_last;
312     """, (s_epoch, e_epoch))
313     res["buckets_aggregated"] = cursor.rowcount if cursor.
rowcount > 0 else 0
314
315     elif lvl_str == AggregationLevel.MONTHLY.value:
316         s_epoch = float(start_epoch) if start_epoch is not Non
e else 0.0
317         e_epoch = float(end_epoch) if end_epoch is not None el
se now_epoch
318
319         cursor.execute("""
320             INSERT INTO telemetry_monthly (
321                 sensor_id,
322                 bucket_start,
323                 bucket_end,
324                 sample_count,
325                 value_min,

```

```

326         value_max,
327         value_avg,
328         value_sum,
329         value_sum_sq,
330         value_stddev,
331         value_first,
332         value_last
333     )
334     WITH daily_window AS (
335         SELECT
336             sensor_id,
337             CAST(strftime('%s', datetime(bucket_start,
338 'unixepoch', 'start of month')) AS INTEGER) AS b_start,
339             CAST(strftime('%s', datetime(bucket_start,
340 'unixepoch', 'start of month', '+1 month')) AS INTEGER) AS b_
end,
341             bucket_start,
342             sample_count,
343             value_min,
344             value_max,
345             value_sum,
346             value_sum_sq,
347             value_first,
348             value_last,
349             FIRST_VALUE(value_first) OVER (
350                 PARTITION BY sensor_id, strftime('%Y-%
m', datetime(bucket_start, 'unixepoch'))
351                 ORDER BY bucket_start ASC
352             ) AS f_val,
353             FIRST_VALUE(value_last) OVER (
354                 PARTITION BY sensor_id, strftime('%Y-%
m', datetime(bucket_start, 'unixepoch'))
355                 ORDER BY bucket_start DESC
356             ) AS l_val
357         FROM telemetry_daily
358         WHERE bucket_start >= ? AND bucket_start < ?
359     )
360     SELECT

```

```

359         sensor_id,
360         b_start AS bucket_start,
361         MAX(b_end) AS bucket_end,
362         SUM(sample_count) AS sample_count,
363         MIN(value_min) AS value_min,
364         MAX(value_max) AS value_max,
365         SUM(value_sum) / SUM(sample_count) AS value_av
g,
366         SUM(value_sum) AS value_sum,
367         SUM(value_sum_sq) AS value_sum_sq,
368         SQRT(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_
sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
))) AS value_stddev,
369         MIN(f_val) AS value_first,
370         MAX(l_val) AS value_last
371     FROM daily_window
372     GROUP BY sensor_id, b_start
373     ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
374         bucket_end = excluded.bucket_end,
375         sample_count = excluded.sample_count,
376         value_min = excluded.value_min,
377         value_max = excluded.value_max,
378         value_avg = excluded.value_avg,
379         value_sum = excluded.value_sum,
380         value_sum_sq = excluded.value_sum_sq,
381         value_stddev = excluded.value_stddev,
382         value_first = excluded.value_first,
383         value_last = excluded.value_last;
384     """ , (s_epoch, e_epoch))
385     res["buckets_aggregated"] = cursor.rowcount if cursor.
rowcount > 0 else 0
386
387     elif lvl_str == AggregationLevel.YEARLY.value:
388         s_epoch = float(start_epoch) if start_epoch is not Non
e else 0.0
389         e_epoch = float(end_epoch) if end_epoch is not None el
se now_epoch
390

```

```

391 |         cursor.execute("""
392 |             INSERT INTO telemetry_yearly (
393 |                 sensor_id,
394 |                 bucket_start,
395 |                 bucket_end,
396 |                 sample_count,
397 |                 value_min,
398 |                 value_max,
399 |                 value_avg,
400 |                 value_sum,
401 |                 value_sum_sq,
402 |                 value_stddev,
403 |                 value_first,
404 |                 value_last
405 |             )
406 |             WITH monthly_window AS (
407 |                 SELECT
408 |                     sensor_id,
409 |                     CAST(strftime('%s', datetime(bucket_start,
| 'unixepoch', 'start of year')) AS INTEGER) AS b_start,
410 |                     CAST(strftime('%s', datetime(bucket_start,
| 'unixepoch', 'start of year', '+1 year')) AS INTEGER) AS b_end,
411 |                     bucket_start,
412 |                     sample_count,
413 |                     value_min,
414 |                     value_max,
415 |                     value_sum,
416 |                     value_sum_sq,
417 |                     value_first,
418 |                     value_last,
419 |                     FIRST_VALUE(value_first) OVER (
420 |                         PARTITION BY sensor_id, strftime('%Y',
| datetime(bucket_start, 'unixepoch'))
421 |                         ORDER BY bucket_start ASC
422 |                     ) AS f_val,
423 |                     FIRST_VALUE(value_last) OVER (
424 |                         PARTITION BY sensor_id, strftime('%Y',

```



```

datetime(bucket_start, 'unixepoch'))
425         ORDER BY bucket_start DESC
426     ) AS l_val
427     FROM telemetry_monthly
428     WHERE bucket_start >= ? AND bucket_start < ?
429 )
430 SELECT
431     sensor_id,
432     b_start AS bucket_start,
433     MAX(b_end) AS bucket_end,
434     SUM(sample_count) AS sample_count,
435     MIN(value_min) AS value_min,
436     MAX(value_max) AS value_max,
437     SUM(value_sum) / SUM(sample_count) AS value_av
438 g,
439     SUM(value_sum) AS value_sum,
440     SUM(value_sum_sq) AS value_sum_sq,
441     SQRT(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_
sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
))) AS value_stddev,
442     MIN(f_val) AS value_first,
443     MAX(l_val) AS value_last
444 FROM monthly_window
445 GROUP BY sensor_id, b_start
446 ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
447     bucket_end = excluded.bucket_end,
448     sample_count = excluded.sample_count,
449     value_min = excluded.value_min,
450     value_max = excluded.value_max,
451     value_avg = excluded.value_avg,
452     value_sum = excluded.value_sum,
453     value_sum_sq = excluded.value_sum_sq,
454     value_stddev = excluded.value_stddev,
455     value_first = excluded.value_first,
456     value_last = excluded.value_last;
457     "", (s_epoch, e_epoch))
458     res["buckets_aggregated"] = cursor.rowcount if cursor.
rowcount > 0 else 0

```

```
458 |
459 |     else:
460 |         raise ValueError(f"Неизвестный уровень агрегации: {lvl
    | _str}")
461 |
462 |     return res
463 |
464 |
465 | class TelemetrySqlAggregator:
466 |     """Управляет вызовами SQL-агрегации и каскадным выполнени
    | ем цепочки агрегаторов."""
467 |
468 |     def __init__(self, connection_manager: TelemetryConnection
    | Manager) -> None:
469 |         """Инициализирует SQL-агрегатор телеметрии.
470 |
471 |         Args:
472 |             connection_manager: Менеджер подключений SQLite.
473 |         """
474 |         self._cm = connection_manager
475 |
476 |     def aggregate(
477 |         self,
478 |         level: Union[AggregationLevel, str],
479 |         start_epoch: Optional[float] = None,
480 |         end_epoch: Optional[float] = None,
481 |         detect_spikes: bool = True,
482 |     ) -> Dict[str, Any]:
483 |         """Запускает агрегацию указанного уровня в транзакции.
484 |
485 |         Args:
486 |             level: Уровень агрегации ('hourly', 'daily', 'week
    | ly', 'monthly', 'yearly').
487 |             start_epoch: Начальная граница диапазона.
488 |             end_epoch: Конечная граница диапазона.
489 |             detect_spikes: Флаг фиксации всплесков.
490 |
491 |         Returns:
```

```

492         Dict[str, Any]: Результаты агрегации.
493         """
494         if self._cm.read_only:
495             return {"level": str(level), "buckets_aggregated":
0, "spikes_recorded": 0, "read_only": True}
496
497         with self._cm.lock, self._cm.get_connection() as conn:
498             result = sensors_aggregate(
499                 conn=conn,
500                 level=level,
501                 start_epoch=start_epoch,
502                 end_epoch=end_epoch,
503                 detect_spikes=detect_spikes,
504             )
505             conn.commit()
506             return result
507
508     def aggregate_hourly(self, start_epoch: Optional[float] =
None, end_epoch: Optional[float] = None, detect_spikes: bool =
True) -> Dict[str, Any]:
509         """Выполняет почасовую агрегацию сырых замеров."""
510         return self.aggregate(AggregationLevel.HOURLY, start_e
poch=start_epoch, end_epoch=end_epoch, detect_spikes=detect_sp
ikes)
511
512     def aggregate_daily(self, start_epoch: Optional[float] = N
one, end_epoch: Optional[float] = None) -> Dict[str, Any]:
513         """Выполняет суточную агрегацию из почасовых бакетов."
"""
514         return self.aggregate(AggregationLevel.DAILY, start_ep
och=start_epoch, end_epoch=end_epoch)
515
516     def aggregate_weekly(self, start_epoch: Optional[float] =
None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
517         """Выполняет недельную агрегацию из суточных бакетов."
"""
518         return self.aggregate(AggregationLevel.WEEKLY, start_e
poch=start_epoch, end_epoch=end_epoch)

```

```

519
520     def aggregate_monthly(self, start_epoch: Optional[float] =
521     None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
522         """Выполняет месячную агрегацию из суточных бакетов."""
523         return self.aggregate(AggregationLevel.MONTHLY, start_
524     epoch=start_epoch, end_epoch=end_epoch)
525
526     def aggregate_yearly(self, start_epoch: Optional[float] =
527     None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
528         """Выполняет годовую агрегацию из месячных бакетов."""
529         return self.aggregate(AggregationLevel.YEARLY, start_e
530     poch=start_epoch, end_epoch=end_epoch)
531
532     def run_pipeline(
533     self,
534     start_epoch: Optional[float] = None,
535     end_epoch: Optional[float] = None,
536     detect_spikes: bool = True,
537     ) -> Dict[str, Any]:
538         """Выполняет полный каскад агрегации: raw -> hourly ->
539     daily -> weekly -> monthly -> yearly в одной транзакции.
540
541     Args:
542         start_epoch: Начальная граница диапазона.
543         end_epoch: Конечная граница диапазона.
544         detect_spikes: Флаг фиксации всплесков.
545
546     Returns:
547         Dict[str, Any]: Сводная статистика всех уровней аг
548     регации.
549     """
550     if self._cm.read_only:
551         return {"read_only": True}
552
553     with self._cm.lock, self._cm.get_connection() as conn:
554         h_res = sensors_aggregate(conn, AggregationLevel.H
555     OURLY, start_epoch, end_epoch, detect_spikes)

```

```
549 |         d_res = sensors_aggregate(conn, AggregationLevel.D
| AILY, start_epoch, end_epoch, False)
550 |         w_res = sensors_aggregate(conn, AggregationLevel.W
| EEKLY, start_epoch, end_epoch, False)
551 |         m_res = sensors_aggregate(conn, AggregationLevel.M
| ONTHLY, start_epoch, end_epoch, False)
552 |         y_res = sensors_aggregate(conn, AggregationLevel.Y
| EARLY, start_epoch, end_epoch, False)
553 |         conn.commit()
554 |
555 |         return {
556 |             "hourly": h_res,
557 |             "daily": d_res,
558 |             "weekly": w_res,
559 |             "monthly": m_res,
560 |             "yearly": y_res,
561 |         }
562 |
```

buffer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Buffer
4 # =====
5 # Description:
6 #   Управление буферизацией и пакетной записью (batching) пото
  ка телеметрии в SQLite.
7 #
8 #   Зачем нужен этот модуль:
9 #       1. Снижение нагрузки на диск (I/O) и ускорение SQLite:
10 #           Объединяет одиночные события метрик в пачки, выполняя
  одну быструю
11 #           транзакцию вместо сотен мелких операций записи с част
  ыми fsync/commit.
12 #       2. Предотвращение блокировок базы данных:
13 #           Минимизирует время удержания эксклюзивных транзакций
  SQLite, предотвращая
14 #           ошибки 'database is locked' при одновременном чтении
  данных агентами и UI.
15 #       3. Защита от потери данных (Fail-Safe / Disaster Recover
  y):
16 #           При сбоях записи или аварийном завершении удерживает
  данные во внешнем
17 #           журнале и автоматически восстанавливает/дописывает их
  при следующем старте.
18 #
19 #   Поддерживаемые режимы буферизации (buffer_mode):
20 #       - 'memory' (По умолчанию):
21 #           Накапливает записи в оперативной памяти (список в RA
  M).
22 #           Сброс в БД происходит по достижении buffer_size или
  по таймеру flush_interval_seconds.
23 #           Обеспечивает максимальную скорость работы.
```

```
24 #     - 'file':
25 #         Каждая входящая запись немедленно сериализуется и до
        пишется на диск
26 #         в аварийный JSONL-файл (telemetry_buffer.jsonl) с вы
        зовом fsync.
27 #         По достижении buffer_size или по таймеру весь файл с
        брасывается в SQLite
28 #         и усекается. Защищает от потери данных при внезапном
        падении процесса/питания.
29 #     - 'direct':
30 #         Отключает фоновый периодический таймер автосброса. За
        писи накапливаются в RAM
31 #         и сбрасываются только при заполнении buffer_size или
        явном вызове flush().
32 #
33 # Usage Examples:
34 #     Python API:
35 #         from apps.windows.telemetry.sqlite.buffer import Telemet
        ryBuffer
36 #         from apps.windows.telemetry.sqlite.connection import Tel
        emetryConnectionManager
37 #         from apps.windows.telemetry.sqlite.writer import Telemet
        ryWriter
38 #
39 #         cm = TelemetryConnectionManager("telemetry.db")
40 #         writer = TelemetryWriter(cm)
41 #         buf = TelemetryBuffer(
42 #             connection_manager=cm,
43 #             writer=writer,
44 #             buffer_mode='memory', # 'memory' | 'file' | 'direct
        '
45 #             buffer_size=50,
46 #             flush_interval_seconds=30.0
47 #         )
48 #         buf.enqueue({"timestamp": "...", "cpu_percent": 12.5})
49 #
50 # File: buffer.py
51 # Project: ai-breadboard
```

```

52 # Package: apps.windows.telemetry.sqlite
53 # Author: hypo69
54 # Copyright: © 2026 hypo69
55 # Updated: 2026-10-06 00:02:00
56 # =====
    =====
57
58 from __future__ import annotations
59
60 import json
61 import os
62 import threading
63 from pathlib import Path
64 from typing import Any, Dict, List, Optional, Union
65
66 from .connection import TelemetryConnectionManager
67 from .writer import TelemetryWriter
68
69 try:
70     from logger import logger
71 except ImportError:
72     import logging
73     logger = logging.getLogger(__name__)
74
75
76 class TelemetryBuffer:
77     """Управляет буферизацией событий телеметрии в памяти и асинхронном JSONL файле."""
78
79     def __init__(
80         self,
81         connection_manager: TelemetryConnectionManager,
82         writer: TelemetryWriter,
83         buffer_mode: str = 'memory',
84         buffer_size: int = 50,
85         flush_interval_seconds: float = 30.0,
86         buffer_file_path: Optional[Union[str, Path]] = None,
87         auto_flush: bool = True,

```



```

88         ) -> None:
89             """Инициализирует буфер сброса телеметрии.
90
91             Args:
92                 connection_manager: Менеджер подключений SQLite.
93                 writer: Писатель пакетов телеметрии.
94                 buffer_mode: Режим буферизации ('memory', 'file',
95                 'direct').
96                 buffer_size: Максимальный размер пачки до сброса.
97                 flush_interval_seconds: Интервал таймера сброса в
98                 секундах.
99                 buffer_file_path: Путь к файлу аварийного буфера.
100                 auto_flush: Флаг автоматического запуска таймера с
101                 броса.
102
103             """
104             self._cm = connection_manager
105             self._writer = writer
106             self._buffer_mode = buffer_mode.lower() if buffer_mode
107             in ('memory', 'file', 'direct') else 'memory'
108             self._buffer_size = max(1, int(buffer_size))
109             self._flush_interval_seconds = max(0.5, float(flush_in
110             terval_seconds))
111             self._auto_flush = bool(auto_flush) and not self._cm.r
112             ead_only
113
114             if buffer_file_path:
115                 self._buffer_file_path = Path(buffer_file_path)
116             else:
117                 self._buffer_file_path = self._cm.db_path.parent /
118                 'telemetry_buffer.jsonl'
119
120             if not self._cm.read_only:
121                 self._buffer_file_path.parent.mkdir(parents=True,
122                 exist_ok=True)
123
124             self._buffer: List[Dict[str, Any]] = []
125             self._file_buffer_count: int = 0
126             self._is_running = True

```

```

118         self._flush_timer: Optional[threading.Timer] = None
119
120     if not self._cm.read_only:
121         self._recover_file_buffer()
122         if self._auto_flush and self._buffer_mode != 'dire
ct':
123             self._start_flush_timer()
124
125     @property
126     def buffer_mode(self) -> str:
127         """Текущий режим буферизации."""
128         return self._buffer_mode
129
130     @property
131     def buffer_size(self) -> int:
132         """Максимальный размер буфера."""
133         return self._buffer_size
134
135     @property
136     def flush_interval_seconds(self) -> float:
137         """Интервал таймера автосброса в секундах."""
138         return self._flush_interval_seconds
139
140     @property
141     def buffer_file_path(self) -> Path:
142         """Путь к файлу аварийного JSONL буфера."""
143         return self._buffer_file_path
144
145     def set_flush_interval_seconds(self, interval: float) -> N
one:
146         """Динамически изменяет интервал периодического сброса
буфера в SQLite."""
147         new_interval = max(0.5, float(interval))
148         if abs(self._flush_interval_seconds - new_interval) >
0.01:
149             with self._cm.lock:
150                 self._flush_interval_seconds = new_interval
151                 if self._auto_flush and self._buffer_mode != '

```

```

    direct' and not self._cm.read_only:
152         self._start_flush_timer()
153         logger.info(f"Интервал сброса буфера телеметрии
и обновлен: {self._flush_interval_seconds}с")
154
155     def set_buffer_mode(self, mode: str) -> None:
156         """Переключает режим буферизации."""
157         m = mode.lower()
158         if m in ('memory', 'file', 'direct'):
159             with self._cm.lock:
160                 self.flush()
161                 self._buffer_mode = m
162                 if m == 'direct':
163                     self._stop_flush_timer()
164                 elif self._auto_flush and not self._cm.read_on
ly:
165                     self._start_flush_timer()
166                     logger.info(f'Режим буферизации переключен на:
{m}')
167
168     def get_buffered_count(self) -> int:
169         """Возвращает число накопленных записей в буфере."""
170         with self._cm.lock:
171             return len(self._buffer) + self._file_buffer_count
172
173     def _start_flush_timer(self) -> None:
174         if self._cm.read_only or not self._is_running or self.
_buffer_mode == 'direct':
175             return
176         self._stop_flush_timer()
177         self._flush_timer = threading.Timer(self._flush_interv
al_seconds, self._on_flush_timer)
178         self._flush_timer.daemon = True
179         self._flush_timer.start()
180
181     def _stop_flush_timer(self) -> None:
182         if self._flush_timer is not None:
183             try:

```

```

184         self._flush_timer.cancel()
185     except Exception:
186         pass
187     self._flush_timer = None
188
189     def _on_flush_timer(self) -> None:
190         if not self._is_running or self._cm.read_only:
191             return
192         try:
193             self.flush()
194         except Exception as ex:
195             logger.debug(f'Ошибка фонового сброса буфера телем
етрии: {ex}')
196         finally:
197             with self._cm.lock:
198                 if self._is_running and self._auto_flush and s
elf._buffer_mode != 'direct':
199                     self._start_flush_timer()
200
201     def _recover_file_buffer(self) -> None:
202         if self._cm.read_only:
203             return
204         if self._buffer_file_path.exists() and self._buffer_fi
le_path.stat().st_size > 0:
205             try:
206                 with open(self._buffer_file_path, 'r', encodin
g='utf-8') as f:
207                     count = sum(1 for line in f if line.strip(
))
208                     self._file_buffer_count = count
209                     if count > 0:
210                         logger.info(f'Обнаружено {count} сохраненн
ых записей в {self._buffer_file_path.name}. Сброс в БД...')
211                         self.flush()
212             except Exception as ex:
213                 logger.warning(f'Ошибка проверки файла буфера
телеметрии: {ex}')
214

```

```

215     def enqueue(self, record: Dict[str, Any]) -> None:
216         """Добавляет запись в буфер в зависимости от выбранног
о режима."""
217         if self._cm.read_only:
218             logger.warning('Попытка добавления записи в буфер
в режиме Read-Only')
219             return
220
221         with self._cm.lock:
222             if self._buffer_mode == 'file':
223                 try:
224                     line = json.dumps(record, ensure_ascii=False, default=str)
225                     with open(self._buffer_file_path, 'a', encoding='utf-8') as f:
226                         f.write(line + '\n')
227                         f.flush()
228                         os.fsync(f.fileno())
229                         self._file_buffer_count += 1
230                 except Exception as ex:
231                     logger.error(f'Ошибка записи в файл буфера
телеметрии: {ex}')
232                     self._buffer.append(record)
233
234                     if self._file_buffer_count >= self._buffer_size:
235                         self.flush()
236                     else:
237                         self._buffer.append(record)
238                         if len(self._buffer) >= self._buffer_size:
239                             self.flush()
240
241     def flush(self) -> int:
242         """Принудительно сбрасывает записи в SQLite."""
243         if self._cm.read_only:
244             return 0
245
246         with self._cm.lock:

```

```

247         records_to_save: List[Dict[str, Any]] = []
248
249         if self._buffer_file_path.exists() and self._buffer_file_path.stat().st_size > 0:
250             try:
251                 with open(self._buffer_file_path, 'r', encoding='utf-8') as f:
252                     for line in f:
253                         s = line.strip()
254                         if s:
255                             try:
256                                 records_to_save.append(json.loads(s))
257                             except Exception:
258                                 pass
259                         except Exception as read_err:
260                             logger.error(f'Ошибка чтения файла буфера: {read_err}')
261
262         if self._buffer:
263             records_to_save.extend(self._buffer)
264             self._buffer = []
265
266         if not records_to_save:
267             return 0
268
269         inserted_count = self._writer.batch_insert_records(records_to_save)
270
271         if self._buffer_file_path.exists():
272             try:
273                 with open(self._buffer_file_path, 'w', encoding='utf-8') as f:
274                     f.truncate(0)
275                     self._file_buffer_count = 0
276             except Exception as trunc_err:
277                 logger.warning(f'Ошибка усечения файла буфера: {trunc_err}')

```

```
278 |
279 |         if inserted_count > 0:
280 |             logger.info(f' [Хранилище SQLite] Пакетный сброс буфера: {inserted_count} записей зафиксировано в {self._cm.db_path.name}')
281 |
282 |             return inserted_count
283 |
284 |     def close(self) -> None:
285 |         """Останавливает таймер и выполняет финальный сброс буфера."""
286 |         self._is_running = False
287 |         self._stop_flush_timer()
288 |         if not self._cm.read_only:
289 |             try:
290 |                 self.flush()
291 |             except Exception:
292 |                 pass
293 |
```

connection.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Connection
4 # =====
5 # Description:
6 #   Менеджер подключений к SQLite базе данных с поддержкой реж
  има только чтение.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.connection import Tel
  emetryConnectionManager
11 #
12 #       cm = TelemetryConnectionManager(db_path=Path("logs/telem
  etry.db"), read_only=True)
13 #       with cm.get_connection() as conn:
14 #           ...
15 #
16 # File: connection.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry.sqlite
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-05 23:35:00
22 # =====
23
24 from __future__ import annotations
25
26 """Управление подключениями к базе данных SQLite с поддержкой
  режимов Read-Only и WAL."""
27
28 import sqlite3
```



```
29 import threading
30 from pathlib import Path
31 from typing import Optional, Union
32
33 try:
34     from logger import logger
35 except ImportError:
36     import logging
37     logger = logging.getLogger(__name__)
38
39
40 class TelemetryConnectionManager:
41     """Менеджер создания и настройки соединений к SQLite базе
42     данных телеметрии."""
43
44     def __init__(
45         self,
46         db_path: Union[str, Path],
47         read_only: bool = False,
48         timeout: float = 15.0,
49         lock: Optional[threading.RLock] = None,
50     ) -> None:
51         """Инициализирует менеджер соединений.
52
53         Args:
54             db_path: Путь к файлу SQLite базы данных.
55             read_only: Флаг строгого режима только чтения (бло
56             кирует запись).
57             timeout: Таймаут ожидания освобождения базы данных
58             в секундах.
59             lock: Разделяемый RLock (опционально).
60
61         """
62         self.db_path = Path(db_path)
63         self.read_only = bool(read_only)
64         self.timeout = float(timeout)
65         self._lock = lock or threading.RLock()
66
67         if not self.read_only:
```

```

64         self.db_path.parent.mkdir(parents=True, exist_ok=True)
65
66     @property
67     def lock(self) -> threading.RLock:
68         """Возвращает разделяемый поток-безопасный лок."""
69         return self._lock
70
71     def get_connection(self) -> sqlite3.Connection:
72         """Создает и настраивает соединение SQLite с необходим
73         ыми PRAGMA директивами.
74
75         Returns:
76             sqlite3.Connection: Настроенное соединение.
77
78         """
79         if self.read_only:
80             db_uri = f"file:{self.db_path.as_posix()}?mode=ro"
81             try:
82                 conn = sqlite3.connect(db_uri, uri=True, timeout=self.timeout,
83                                     check_same_thread=False)
84             except sqlite3.OperationalError:
85                 conn = sqlite3.connect(str(self.db_path), timeout=self.timeout,
86                                     check_same_thread=False)
87             try:
88                 conn.execute('PRAGMA query_only = ON;')
89             except Exception:
90                 pass
91         else:
92             conn = sqlite3.connect(str(self.db_path), timeout=self.timeout,
93                                 check_same_thread=False)
94             conn.execute('PRAGMA journal_mode = WAL;')
95             conn.execute('PRAGMA synchronous = NORMAL;')
96             conn.execute('PRAGMA foreign_keys = ON;')
97
98         conn.row_factory = sqlite3.Row
99         return conn

```

maintenance.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Maintenance
4 # =====
5 # Description:
6 #   Обслуживание, очистка, агрегация и контроль размера базы д
  анных SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.maintenance import Te
    lemetryMaintenance
11 #
12 #       maint = TelemetryMaintenance(connection_manager)
13 #       stats = maint.get_storage_stats()
14 #
15 # File: maintenance.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 01:59:00
21 # =====
22
23 from __future__ import annotations
24
25 """Методы обслуживания, очистки старых записей, дефрагментации
  и агрегации телеметрии в SQLite."""
26
27 import sqlite3
28 import time
29 from collections import defaultdict
```

```

30 from datetime import datetime, timezone
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional
33
34 from .connection import TelemetryConnectionManager
35
36 try:
37     from logger import logger
38 except ImportError:
39     import logging
40     logger = logging.getLogger(__name__)
41
42
43 class TelemetryMaintenance:
44     """Выполняет регламентные операции: очистка, контроль разм
    ера, вакуумирование и агрегации."""
45
46     def __init__(
47         self,
48         connection_manager: TelemetryConnectionManager,
49         max_db_size_mb: float = 50.0,
50         retention_days: int = 7,
51         auto_vacuum_enabled: bool = True,
52         buffer_file_path: Optional[Path] = None,
53     ) -> None:
54         """Инициализирует менеджер обслуживания базы данных.
55
56         Args:
57             connection_manager: Менеджер соединений SQLite.
58             max_db_size_mb: Максимальный размер файлов БД в МБ
59             .
60             retention_days: Срок хранения сырых записей в днях
61             .
62             auto_vacuum_enabled: Флаг автоматического запуска
63             VACUUM.
64             buffer_file_path: Путь к файлу буфера.
65         """
66         self._cm = connection_manager

```

```

64         self.max_db_size_mb = max(1.0, float(max_db_size_mb))
65         self.retention_days = max(1, int(retention_days))
66         self.auto_vacuum_enabled = bool(auto_vacuum_enabled)
67         self.buffer_file_path = buffer_file_path or (self._cm.
db_path.parent / 'telemetry_buffer.jsonl')
68
69     def get_db_file_sizes(self) -> Dict[str, int]:
70         """Возвращает физические размеры файлов БД на диске (D
B, WAL, SHM) в байтах."""
71         db_path = self._cm.db_path
72         db_bytes = db_path.stat().st_size if db_path.exists()
else 0
73         wal_path = db_path.with_name(db_path.name + '-wal')
74         shm_path = db_path.with_name(db_path.name + '-shm')
75         wal_bytes = wal_path.stat().st_size if wal_path.exists
() else 0
76         shm_bytes = shm_path.stat().st_size if shm_path.exists
() else 0
77         return {
78             'db_bytes': db_bytes,
79             'wal_bytes': wal_bytes,
80             'shm_bytes': shm_bytes,
81             'total_bytes': db_bytes + wal_bytes + shm_bytes,
82         }
83
84     def get_total_db_size_mb(self) -> float:
85         """Возвращает совокупный физический размер файлов БД в
МБ."""
86         sizes = self.get_db_file_sizes()
87         return round(sizes['total_bytes'] / (1024.0 * 1024.0),
3)
88
89     def checkpoint_wal(self) -> bool:
90         """Сбрасывает WAL-журнал в основной файл базы данных."
"""
91         if self._cm.read_only:
92             return False
93         with self._cm.lock:

```

```

94         try:
95             with self._cm.get_connection() as conn:
96                 conn.execute('PRAGMA wal_checkpoint(TRUNCATE);')
97             return True
98         except Exception as e:
99             logger.debug(f'Ошибка сброса WAL-журнала: {e}')
100     )
101
102     def vacuum(self) -> bool:
103         """Выполняет сжатие и дефрагментацию базы данных."""
104         if self._cm.read_only:
105             return False
106         with self._cm.lock:
107             self.checkpoint_wal()
108             try:
109                 conn = sqlite3.connect(str(self._cm.db_path),
110                                         timeout=30.0, isolation_level=None)
111                 try:
112                     conn.execute('VACUUM;')
113                     finally:
114                         conn.close()
115                 logger.info(f'Выполнен VACUUM для базы данных
116                             телеметрии ({self._cm.db_path})')
117                 return True
118             except Exception as e:
119                 logger.warning(f'Ошибка при выполнении VACUUM:
120                                {e}')
121             return False
122
123     def get_storage_stats(self, buffered_count: int = 0, buffer_mode: str = 'memory', buffer_size: int = 50, flush_interval_seconds: float = 30.0) -> Dict[str, Any]:
124         """Возвращает агрегированную статистику таблиц, размера и параметров."""
125         with self._cm.lock, self._cm.get_connection() as conn:
126             cursor = conn.cursor()

```

```

124 |         cursor.execute('SELECT COUNT(*) FROM system_snapsh
    |         ots;')
125 |         snap_count = cursor.fetchone()[0]
126 |         cursor.execute('SELECT COUNT(*) FROM process_snaps
    |         hots;')
127 |         proc_count = cursor.fetchone()[0]
128 |         cursor.execute('SELECT COUNT(*) FROM sensor_polls;
    |         ')
129 |         sensors_count = cursor.fetchone()[0]
130 |         cursor.execute('SELECT COUNT(*) FROM telemetry_eve
    |         nts;')
131 |         events_count = cursor.fetchone()[0]
132 |         cursor.execute('SELECT COUNT(*) FROM hardware_audi
    |         ts;')
133 |         audits_count = cursor.fetchone()[0]
134 |         cursor.execute('SELECT COUNT(*) FROM app_polls;')
135 |         app_polls_count = cursor.fetchone()[0]
136 |         cursor.execute('SELECT COUNT(*) FROM app_events;')
137 |         app_events_count = cursor.fetchone()[0]
138 |         cursor.execute('SELECT COUNT(*) FROM app_param_cha
    |         nges;')
139 |         app_params_count = cursor.fetchone()[0]
140 |         cursor.execute('SELECT COUNT(*) FROM custom_record
    |         s;')
141 |         custom_count = cursor.fetchone()[0]
142 |         cursor.execute('SELECT COUNT(*) FROM device_events
    |         ;')
143 |         device_events_count = cursor.fetchone()[0]
144 |         cursor.execute('SELECT COUNT(*) FROM device_invent
    |         ory;')
145 |         device_inventory_count = cursor.fetchone()[0]
146 |         cursor.execute('SELECT COUNT(*) FROM w64_events;')
147 |         w64_events_count = cursor.fetchone()[0]
148 |         cursor.execute('SELECT COUNT(*) FROM process_outli
    |         ers;')
149 |         outliers_count = cursor.fetchone()[0]
150 |         cursor.execute('SELECT COUNT(*) FROM process_rollu
    |         ps_2min;')

```

```

151 |         rollups_2min_count = cursor.fetchone()[0]
152 |         cursor.execute('SELECT COUNT(*) FROM process_rollu
    | ps_daily;')
153 |         rollups_daily_count = cursor.fetchone()[0]
154 |         cursor.execute('SELECT COUNT(*) FROM incidents;')
155 |         incidents_count = cursor.fetchone()[0]
156 |         cursor.execute('SELECT COUNT(*) FROM telemetry_rol
    | lups;')
157 |         rollups_count = cursor.fetchone()[0]
158 |
159 |         def _safe_count(table: str) -> int:
160 |             try:
161 |                 cursor.execute(f'SELECT COUNT(*) FROM {tab
    | le};')
162 |                 return cursor.fetchone()[0]
163 |             except Exception:
164 |                 return 0
165 |
166 |         telemetry_hourly_count = _safe_count('telemetry_ho
    | urly')
167 |         telemetry_daily_count = _safe_count('telemetry_dai
    | ly')
168 |         telemetry_weekly_count = _safe_count('telemetry_we
    | ekly')
169 |         telemetry_monthly_count = _safe_count('telemetry_m
    | onthly')
170 |         telemetry_yearly_count = _safe_count('telemetry_ye
    | arly')
171 |         telemetry_spikes_count = _safe_count('telemetry_sp
    | ikes')
172 |         disk_inventory_count = _safe_count('disk_inventory
    | ')
173 |         volume_inventory_count = _safe_count('volume_inven
    | tory')
174 |         disk_health_count = _safe_count('disk_health_snaps
    | hots')
175 |         disk_perf_count = _safe_count('disk_performance_sa
    | mples')

```



```

176         disk_io_count = _safe_count('disk_io_events')
177         cpu_inventory_count = _safe_count('cpu_inventory')
178         ram_module_inventory_count = _safe_count('ram_modu
le_inventory')
179         gpu_inventory_count = _safe_count('gpu_inventory')
180         network_adapter_inventory_count = _safe_count('net
work_adapter_inventory')
181         cpu_samples_count = _safe_count('cpu_telemetry_sam
ples')
182         ram_samples_count = _safe_count('ram_telemetry_sam
ples')
183         gpu_samples_count = _safe_count('gpu_telemetry_sam
ples')
184         network_samples_count = _safe_count('network_adapt
er_samples')
185
186         file_sizes = self.get_db_file_sizes()
187         size_mb = round(file_sizes['db_bytes'] / (1024 * 1
024), 2)
188         wal_size_mb = round(file_sizes['wal_bytes'] / (102
4 * 1024), 2)
189         total_size_mb = round(file_sizes['total_bytes'] /
(1024 * 1024), 2)
190         buffer_file_size = self.buffer_file_path.stat().st
_size if self.buffer_file_path.exists() else 0
191
192         return {
193             'db_path': str(self._cm.db_path),
194             'snapshots_count': snap_count,
195             'process_snapshots_count': proc_count,
196             'process_outliers_count': outliers_count,
197             'process_rollups_2min_count': rollups_2min_cou
nt,
198             'process_rollups_daily_count': rollups_daily_c
ount,
199             'incidents_count': incidents_count,
200             'telemetry_rollups_count': rollups_count,
201             'telemetry_hourly_count': telemetry_hourly_cou

```

```

nt,
202         'telemetry_daily_count': telemetry_daily_count
    ,
203         'telemetry_weekly_count': telemetry_weekly_cou
nt,
204         'telemetry_monthly_count': telemetry_monthly_c
ount,
205         'telemetry_yearly_count': telemetry_yearly_cou
nt,
206         'telemetry_spikes_count': telemetry_spikes_cou
nt,
207         'disk_inventory_count': disk_inventory_count,
208         'volume_inventory_count': volume_inventory_cou
nt,
209         'disk_health_count': disk_health_count,
210         'disk_perf_count': disk_perf_count,
211         'disk_io_count': disk_io_count,
212         'cpu_inventory_count': cpu_inventory_count,
213         'ram_module_inventory_count': ram_module_inven
tory_count,
214         'gpu_inventory_count': gpu_inventory_count,
215         'network_adapter_inventory_count': network_ada
pter_inventory_count,
216         'cpu_samples_count': cpu_samples_count,
217         'ram_samples_count': ram_samples_count,
218         'gpu_samples_count': gpu_samples_count,
219         'network_samples_count': network_samples_count
    ,
220         'sensor_polls_count': sensors_count,
221         'events_count': events_count,
222         'hardware_audits_count': audits_count,
223         'app_polls_count': app_polls_count,
224         'app_events_count': app_events_count,
225         'app_param_changes_count': app_params_count,
226         'custom_records_count': custom_count,
227         'device_events_count': device_events_count,
228         'device_inventory_count': device_inventory_cou
nt,

```

```

229         'w64_events_count': w64_events_count,
230         'file_size_mb': size_mb,
231         'wal_size_mb': wal_size_mb,
232         'total_size_mb': total_size_mb,
233         'max_db_size_mb': self.max_db_size_mb,
234         'retention_days': self.retention_days,
235         'size_limit_exceeded': total_size_mb > self.ma
x_db_size_mb,
236         'buffer_mode': buffer_mode,
237         'buffered_count': buffered_count,
238         'buffer_size': buffer_size,
239         'flush_interval_seconds': flush_interval_secon
ds,
240         'buffer_file_path': str(self.buffer_file_path)
,
241         'buffer_file_size_bytes': buffer_file_size,
242     }
243
244     def cleanup_old_records(self, retention_days: Optional[int
] = None, vacuum_after: bool = False) -> int:
245         """Удаляет устаревшие записи телеметрии."""
246         if self._cm.read_only:
247             return 0
248         days = retention_days if retention_days is not None el
se self.retention_days
249         threshold_epoch = datetime.now(timezone.utc).timestamp
() - (float(days) * 86400.0 if days > 0 else -1.0)
250         with self._cm.lock, self._cm.get_connection() as conn:
251             cursor = conn.cursor()
252             cursor.execute('SELECT id FROM system_snapshots WH
ERE created_at < ?', (threshold_epoch,))
253             old_ids = [row[0] for row in cursor.fetchall()]
254             if old_ids:
255                 cursor.execute('DELETE FROM process_snapshots
WHERE snapshot_id IN (SELECT id FROM system_snapshots WHERE cr
eated_at < ?)', (threshold_epoch,))
256                 cursor.execute('DELETE FROM system_snapshots W
HERE created_at < ?', (threshold_epoch,))

```

```
257 |         cursor.execute('DELETE FROM sensor_polls WHERE cre
| ated_at < ?', (threshold_epoch,))
258 |         cursor.execute('DELETE FROM telemetry_events WHERE
| created_at < ?', (threshold_epoch,))
259 |         cursor.execute('DELETE FROM app_polls WHERE create
| d_at < ?', (threshold_epoch,))
260 |         cursor.execute('DELETE FROM app_events WHERE creat
| ed_at < ?', (threshold_epoch,))
261 |         cursor.execute('DELETE FROM app_param_changes WHER
| E created_at < ?', (threshold_epoch,))
262 |         cursor.execute('DELETE FROM custom_records WHERE c
| reated_at < ?', (threshold_epoch,))
263 |         cursor.execute('DELETE FROM w64_events WHERE creat
| ed_at < ?', (threshold_epoch,))
264 |         cursor.execute('DELETE FROM process_outliers WHERE
| created_at < ?', (threshold_epoch,))
265 |         cursor.execute('DELETE FROM device_events WHERE cr
| eated_at < ?', (threshold_epoch,))
266 |         cursor.execute('DELETE FROM system_extended_audits
| WHERE created_at < ?', (threshold_epoch,))
267 |         cursor.execute('DELETE FROM disk_io_events WHERE c
| reated_at < ?', (threshold_epoch,))
268 |         cursor.execute('DELETE FROM disk_performance_sampl
| es WHERE created_at < ?', (threshold_epoch,))
269 |         cursor.execute('DELETE FROM disk_health_snapshots
| WHERE created_at < ?', (threshold_epoch,))
270 |         cursor.execute('DELETE FROM cpu_telemetry_samples
| WHERE created_at < ?', (threshold_epoch,))
271 |         cursor.execute('DELETE FROM ram_telemetry_samples
| WHERE created_at < ?', (threshold_epoch,))
272 |         cursor.execute('DELETE FROM gpu_telemetry_samples
| WHERE created_at < ?', (threshold_epoch,))
273 |         cursor.execute('DELETE FROM network_adapter_sample
| s WHERE created_at < ?', (threshold_epoch,))
274 |         conn.commit()
275 |         deleted_count = len(old_ids)
276 |         logger.info(f'Очищено {deleted_count} устаревших с
| нимков телеметрии (старше {days} дн.)')
```

```

277
278         if vacuum_after and self.auto_vacuum_enabled:
279             self.vacuum()
280
281         return deleted_count
282
283     def enforce_size_limit(self, max_size_mb: Optional[float]
= None, target_ratio: float = 0.85) -> Dict[str, Any]:
284         """Ограничивает совокупный размер БД телеметрии."""
285         if self._cm.read_only:
286             return {'pruned': False, 'read_only': True}
287         limit_mb = max_size_mb if max_size_mb is not None else
self.max_db_size_mb
288         initial_sizes = self.get_db_file_sizes()
289         initial_size_mb = round(initial_sizes['total_bytes'] /
(1024.0 * 1024.0), 3)
290
291         if initial_size_mb <= limit_mb:
292             return {
293                 'pruned': False,
294                 'initial_size_mb': initial_size_mb,
295                 'final_size_mb': initial_size_mb,
296                 'max_size_mb': limit_mb,
297                 'deleted_snapshots': 0,
298                 'deleted_records': 0,
299             }
300
301         target_mb = max(0.5, limit_mb * target_ratio)
302         logger.warning(f'⚠ Превышен лимит размера базы данных:
{initial_size_mb:.2f} МБ > {limit_mb:.2f} МБ.')
303
304         total_deleted_snaps = self.cleanup_old_records(retenti
on_days=self.retention_days, vacuum_after=False)
305         self.checkpoint_wal()
306
307         with self._cm.lock, self._cm.get_connection() as conn:
308             cursor = conn.cursor()
309             while True:

```

```

310         cursor.execute('SELECT COUNT(*) FROM system_sn
    | apshots;')
311         snap_count = cursor.fetchone()[0]
312         if not snap_count or snap_count <= 5:
313             break
314         batch_size = max(5, int(snap_count * 0.25))
315         cursor.execute('SELECT id, created_at FROM sys
    | tem_snapshots ORDER BY created_at ASC LIMIT ?', (batch_size,))
316         rows = cursor.fetchall()
317         if not rows:
318             break
319         ids_to_del = [r[0] for r in rows]
320         cutoff_t = rows[-1][1]
321         placeholders = ','.join('?' for _ in ids_to_de
    | l)
322         cursor.execute(f'DELETE FROM process_snapshots
    | WHERE snapshot_id IN ({placeholders})', ids_to_del)
323         cursor.execute(f'DELETE FROM system_snapshots
    | WHERE id IN ({placeholders})', ids_to_del)
324         cursor.execute('DELETE FROM sensor_polls WHERE
    | created_at <= ?', (cutoff_t,))
325         cursor.execute('DELETE FROM w64_events WHERE c
    | reated_at <= ?', (cutoff_t,))
326         cursor.execute('DELETE FROM app_polls WHERE cr
    | eated_at <= ?', (cutoff_t,))
327         cursor.execute('DELETE FROM custom_records WHE
    | RE created_at <= ?', (cutoff_t,))
328         conn.commit()
329         total_deleted_snaps += len(ids_to_del)
330         if snap_count - len(ids_to_del) <= 5:
331             break
332
333     if self.auto_vacuum_enabled:
334         self.vacuum()
335     else:
336         self.checkpoint_wal()
337
338     final_sizes = self.get_db_file_sizes()

```

```

339         final_size_mb = round(final_sizes['total_bytes'] / (10
24.0 * 1024.0), 3)
340
341         return {
342             'pruned': True,
343             'initial_size_mb': initial_size_mb,
344             'final_size_mb': final_size_mb,
345             'max_size_mb': limit_mb,
346             'target_size_mb': target_mb,
347             'deleted_snapshots': total_deleted_snaps,
348         }
349
350     def aggregate_process_metrics_2min(self, cutoff_seconds: i
nt = 120, outlier_cpu_threshold: float = 30.0) -> Dict[str, in
t]:
351         """Обобщает процессы старше двух минут по средним пока
зателям."""
352         if self._cm.read_only:
353             return {'rollups_created': 0, 'outliers_saved': 0,
'raw_deleted': 0}
354         now_epoch = datetime.now(timezone.utc).timestamp()
355         cutoff_epoch = now_epoch - cutoff_seconds
356         with self._cm.lock, self._cm.get_connection() as conn:
357             cursor = conn.cursor()
358             cursor.execute('''
359                 SELECT p.id, p.snapshot_id, p.timestamp, p.pid
, p.name, p.status,
360                     p.cpu_percent, p.memory_mb, p.memory_pe
rcent, p.num_threads,
361                     p.username, s.created_at as snap_epoch
362                 FROM process_snapshots p
363                 JOIN system_snapshots s ON p.snapshot_id = s.i
d
364                 WHERE s.created_at < ?
365                 ORDER BY p.name, p.pid, s.created_at ASC
366                 ''', (cutoff_epoch,))
367         raw_rows = cursor.fetchall()
368         if not raw_rows:

```

```

369         return {'rollups_created': 0, 'outliers_saved'
370 : 0, 'raw_deleted': 0}
371
372         outlier_rows = []
373         for r in raw_rows:
374             cpu_val = float(r['cpu_percent'] or 0.0)
375             if cpu_val >= outlier_cpu_threshold:
376                 outlier_rows.append((
377                     r['snapshot_id'], r['timestamp'], r['s
378 nap_epoch'], r['pid'], r['name'],
379                     r['status'] or 'running', cpu_val, flo
380 at(r['memory_mb'] or 0.0),
381                     float(r['memory_percent'] or 0.0), int
382 (r['num_threads'] or 1),
383                     r['username'] or '', f'Outlier: CPU={c
384 pu_val:.1f}% ',
385                     ))
386
387         outliers_saved = 0
388         if outlier_rows:
389             cursor.executemany('''
390
391                 INSERT INTO process_outliers (
392                     snapshot_id, timestamp, created_at, pi
393 d, name, status,
394                     cpu_percent, memory_mb, memory_percent
395 , num_threads, username, details
396                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
397
398                 ?)
399
400             ''', outlier_rows)
401             outliers_saved = len(outlier_rows)
402
403         grouped = defaultdict(list)
404         for r in raw_rows:
405             grouped[(r['name'], r['pid'])].append(r)
406
407         rollup_rows = []
408         for (p_name, p_pid), procs in grouped.items():
409             cnt = len(procs)
410             cpus = [float(p['cpu_percent'] or 0.0) for p i

```



```

n procs]
399         mems = [float(p['memory_mb'] or 0.0) for p in
procs]
400         threads = [int(p['num_threads'] or 1) for p in
procs]
401         proc_outliers = sum(1 for c in cpus if c >= ou
tlier_cpu_threshold)
402         rollup_rows.append((
403             procs[0]['timestamp'], procs[-1]['timestam
p'], procs[0]['snap_epoch'], procs[-1]['snap_epoch'],
404             p_name, p_pid, procs[-1]['username'] or ' '
, cnt, round(sum(cpus) / cnt, 2),
405             round(max(cpus), 2), round(min(cpus), 2),
round(sum(mems) / cnt, 2),
406             round(max(mems), 2), round(min(mems), 2),
round(sum(threads) / cnt, 1), proc_outliers,
407             ))
408
409         if rollup_rows:
410             cursor.executemany('''
411                 INSERT INTO process_rollups_2min (
412                     period_start, period_end, period_start
_epoch, period_end_epoch,
413                     name, pid, username, sample_count, avg
_cpu_percent, max_cpu_percent,
414                     min_cpu_percent, avg_memory_mb, max_me
mory_mb, min_memory_mb,
415                     avg_num_threads, outliers_count
416                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?)
417             ''', rollup_rows)
418
419         raw_ids = [r['id'] for r in raw_rows]
420         cursor.execute(f'''
421             DELETE FROM process_snapshots WHERE id IN ({',
'.join(['?'] * len(raw_ids))})
422         ''', raw_ids)
423         conn.commit()

```

```

424         return {'rollups_created': len(rollup_rows), 'outliers_saved': outliers_saved, 'raw_deleted': len(raw_ids)}
425
426     def aggregate_process_metrics_daily(self, cutoff_days: int = 1) -> Dict[str, int]:
427         """Обобщает агрегаты процессов в суточную статистику."""
428         if self._cm.read_only:
429             return {'daily_rollups_created': 0, '2min_rollups_cleaned': 0}
430         now_epoch = datetime.now(timezone.utc).timestamp()
431         cutoff_epoch = now_epoch - cutoff_days * 86400
432         with self._cm.lock, self._cm.get_connection() as conn:
433             cursor = conn.cursor()
434             cursor.execute('SELECT * FROM process_rollups_2min WHERE period_end_epoch < ? ORDER BY name, period_start_epoch ASC', (cutoff_epoch,))
435             old_rollups = cursor.fetchall()
436             if not old_rollups:
437                 return {'daily_rollups_created': 0, '2min_rollups_cleaned': 0}
438
439             grouped_daily = defaultdict(list)
440             for r in old_rollups:
441                 d_str = r['period_start'][:10] if len(r['period_start']) >= 10 else 'unknown'
442                 grouped_daily[(d_str, r['name'])].append(r)
443
444             daily_rows = []
445             for (d_str, p_name), items in grouped_daily.items():
446                 total_samples = sum(int(it['sample_count']) for it in items)
447                 if total_samples <= 0:
448                     continue
449                 weighted_cpu = sum(float(it['avg_cpu_percent']) * int(it['sample_count']) for it in items) / total_samples
450                 weighted_mem = sum(float(it['avg_memory_mb'])

```

```

    * int(it['sample_count']) for it in items) / total_samples
451 |         daily_rows.append((
452 |             d_str, p_name, total_samples, round(weight
    ed_cpu, 2), round(max(float(it['max_cpu_percent']) for it in i
    tems), 2),
453 |             round(weighted_mem, 2), round(max(float(it
    ['max_memory_mb']) for it in items), 2),
454 |             sum(int(it['outliers_count'] or 0) for it
    in items), max(it['period_end'] for it in items),
455 |             ))
456 |
457 |         if daily_rows:
458 |             cursor.executemany('''
459 |                 INSERT INTO process_rollups_daily (
460 |                     date, name, sample_count, avg_cpu_perc
    ent, max_cpu_percent,
461 |                     avg_memory_mb, max_memory_mb, outliers
    _count, last_seen
462 |                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
463 |             ''', daily_rows)
464 |
465 |         del_ids = [r['id'] for r in old_rollups]
466 |         cursor.execute(f'DELETE FROM process_rollups_2min
    WHERE id IN ({','.join(['?'] * len(del_ids))})', del_ids)
467 |         conn.commit()
468 |         return {'daily_rollups_created': len(daily_rows),
    '2min_rollups_cleaned': len(del_ids)}
469 |

```

reader.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Reader
4 # =====
5 # Description:
6 #   Класс TelemetryReader для чтения и выборки данных телеметр
  ии из SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.reader import Telemet
  ryReader
11 #
12 #       reader = TelemetryReader(connection_manager)
13 #       snapshots = reader.get_snapshots(limit=10)
14 #
15 # File: reader.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 04:25:00
21 # =====
22
23 from __future__ import annotations
24
25 """Класс TelemetryReader для выполнения выборок, аналитики и ч
  тения данных телеметрии."""
26
27 import csv
28 import json
29 import sqlite3
```

```

30 import time
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional, Union
33
34 from .connection import TelemetryConnectionManager
35
36 try:
37     from logger import logger
38 except ImportError:
39     import logging
40     logger = logging.getLogger(__name__)
41
42
43 class TelemetryReader:
44     """Выполняет запросы чтения данных телеметрии, процессов,
45     сенсоров и аудита из базы SQLite."""
46
47     def __init__(self, connection_manager: TelemetryConnection
48     Manager) -> None:
49         """Инициализирует ридер телеметрии.
50
51         Args:
52             connection_manager: Менеджер подключений SQLite.
53         """
54         self._cm = connection_manager
55
56     def get_snapshots(self, limit: int = 60, since_epoch: Opti
57     onal[float] = None) -> List[Dict[str, Any]]:
58         """Извлекает исторические срезы системной телеметрии."
59         ""
60
61         with self._cm.lock, self._cm.get_connection() as conn:
62             cursor = conn.cursor()
63             if since_epoch is not None:
64                 cursor.execute(
65                     'SELECT * FROM system_snapshots WHERE crea
66                     ted_at >= ? ORDER BY id DESC LIMIT ?',
67                     (since_epoch, limit)
68                 )

```

```

63         else:
64             cursor.execute('SELECT * FROM system_snapshots
        ORDER BY id DESC LIMIT ?', (limit,))
65             return [dict(row) for row in cursor.fetchall()]
66
67     def get_latest_cpu(self) -> Optional[Dict[str, Any]]:
68         """Извлекает последние метрики процессора из таблицы s
        ystem_snapshots."""
69         with self._cm.lock, self._cm.get_connection() as conn:
70             cursor = conn.cursor()
71             cursor.execute('''
72                 SELECT id, timestamp, created_at, hostname, cp
73                 u_total_percent, cpu_frequency_mhz
74                 FROM system_snapshots
75                 ORDER BY id DESC
76                 LIMIT 1;
77             ''')
78             row = cursor.fetchone()
79             return dict(row) if row else None
80
81     def get_cpu_history(self, limit: int = 60, since_epoch: Op
82     tional[float] = None) -> List[Dict[str, Any]]:
83         """Извлекает историю нагрузки процессора."""
84         with self._cm.lock, self._cm.get_connection() as conn:
85             cursor = conn.cursor()
86             if since_epoch is not None:
87                 cursor.execute('''
88                     SELECT id, timestamp, created_at, cpu_tota
89                     l_percent, cpu_frequency_mhz
90                     FROM system_snapshots
91                     WHERE created_at >= ?
92                     ORDER BY id DESC
93                     LIMIT ?;
94                 ''', (since_epoch, limit))
95             else:
96                 cursor.execute('''
97                     SELECT id, timestamp, created_at, cpu_tota
98                     l_percent, cpu_frequency_mhz

```

```

95 |             FROM system_snapshots
96 |             ORDER BY id DESC
97 |             LIMIT ?;
98 |         '', (limit,))
99 |         return [dict(row) for row in cursor.fetchall()]
100 |
101 |     def get_latest_storage_data(self) -> Dict[str, Any]:
102 |         """Извлекает последние данные о накопителях и разделах
103 |         ."""
104 |         with self._cm.lock, self._cm.get_connection() as conn:
105 |             cursor = conn.cursor()
106 |             cursor.execute('''
107 |                 SELECT id, timestamp, created_at, disks_json,
108 |                 physical_disks_json
109 |                 FROM system_snapshots
110 |                 ORDER BY id DESC
111 |                 LIMIT 1;
112 |             ''')
113 |             row = cursor.fetchone()
114 |             if not row:
115 |                 return {'partitions': [], 'physical_disks': []}
116 |
117 |             partitions: List[Dict[str, Any]] = []
118 |             if row['disks_json']:
119 |                 try:
120 |                     partitions = json.loads(row['disks_json'])
121 |                 except Exception:
122 |                     partitions = []
123 |
124 |             physical_disks: List[Dict[str, Any]] = []
125 |             if row['physical_disks_json']:
126 |                 try:
127 |                     physical_disks = json.loads(row['physical_
128 | disks_json'])

```

[illegible]


```

    mhz', 'memory_total_gb', 'memory_used_gb', 'memory_percent',
162 |         'swap_percent', 'gpu_load_percent',
    'gpu_temp_c', 'disk_read_bytes_sec', 'disk_write_bytes_sec',
163 |         'disk_read_count_sec', 'disk_write_c
    ount_sec', 'network_sent_bytes_sec', 'network_recv_bytes_sec',
    'timestamp'):
164 |         if snap_row.get(field) is not None and (field
    not in full_dict or not full_dict[field]):
165 |             full_dict[field] = snap_row[field]
166 |
167 |     # 3. Десериализация JSON-колонок
168 |     json_column_mappings = {
169 |         'input_languages_json': 'input_languages',
170 |         'disks_json': 'disks',
171 |         'physical_disks_json': 'physical_disks',
172 |         'monitors_json': 'monitors',
173 |         'updates_json': 'updates',
174 |         'office_json': 'office',
175 |         'onedrive_json': 'onedrive',
176 |         'battery_json': 'battery',
177 |         'ram_sticks_json': 'ram_sticks',
178 |         'listening_ports_json': 'listening_ports',
179 |         'alerts_json': 'alerts',
180 |         'hardware_audit': 'hardware_audit',
181 |     }
182 |     for col_name, target_key in json_column_mappings.i
    tems():
183 |         val_str = snap_row.get(col_name)
184 |         if val_str and isinstance(val_str, str) and (t
    arget_key not in full_dict or not full_dict[target_key]):
185 |             try:
186 |                 full_dict[target_key] = json.loads(val
    _str)
187 |             except Exception:
188 |                 pass
189 |
190 |     # 4. Процессы (если не были в raw_json)
191 |     if 'top_processes' not in full_dict or not full_di

```

```

ct['top_processes']:
192         if snap_id:
193             cursor.execute(
194                 'SELECT pid, name, status, cpu_percent
, memory_mb, memory_percent, num_threads, num_handles, '
195                 'username, read_bytes_sec, write_bytes
_sec, integrity_level, elevation, ppid, parent_name, '
196                 'executable as executable_path, cmdlin
e, sid, session_id, creation_time, process_guid, ancestor_chai
n, launch_reason '
197                 'FROM process_snapshots WHERE snapshot
_id = ? ORDER BY cpu_percent DESC LIMIT 50;',
198                 (snap_id,)
199             )
200             full_dict['top_processes'] = [dict(p) for
p in cursor.fetchall()]
201
202         # 5. Структурирование CPU и Memory если отсутствую
т объекты
203         if 'cpu' not in full_dict or not isinstance(full_d
ict.get('cpu'), dict):
204             full_dict['cpu'] = {
205                 'total_percent': float(snap_row.get('cpu_t
otal_percent') or 0.0),
206                 'frequency_mhz': float(snap_row.get('cpu_f
requency_mhz') or 0.0),
207                 'model': full_dict.get('cpu_model', 'Intel
 / AMD Processor'),
208                 'architecture': 'x86_64',
209             }
210         if 'memory' not in full_dict or not isinstance(ful
l_dict.get('memory'), dict):
211             tot_m = float(snap_row.get('memory_total_gb')
or 0.0)
212             used_m = float(snap_row.get('memory_used_gb')
or 0.0)
213             full_dict['memory'] = {
214                 'total_gb': tot_m,

```

```

215         'used_gb': used_m,
216         'available_gb': max(0.0, tot_m - used_m),
217         'percent': float(snap_row.get('memory_percent') or 0.0),
218         'swap_percent': float(snap_row.get('swap_percent') or 0.0),
219     }
220     if 'disk_io' not in full_dict or not isinstance(full_dict.get('disk_io'), dict):
221         full_dict['disk_io'] = {
222             'read_bytes_per_sec': float(snap_row.get('disk_read_bytes_sec') or 0.0),
223             'write_bytes_per_sec': float(snap_row.get('disk_write_bytes_sec') or 0.0),
224             'read_count_per_sec': float(snap_row.get('disk_read_count_sec') or 0.0),
225             'write_count_per_sec': float(snap_row.get('disk_write_count_sec') or 0.0),
226         }
227
228     return full_dict
229
230     def get_snapshot_processes(self, snapshot_id: int) -> List[Dict[str, Any]]:
231         """Извлекает список процессов, зафиксированных в снимке."""
232         with self._cm.lock, self._cm.get_connection() as conn:
233             cursor = conn.cursor()
234             cursor.execute(
235                 'SELECT * FROM process_snapshots WHERE snapshot_id = ? ORDER BY cpu_percent DESC, memory_mb DESC',
236                 (snapshot_id,)
237             )
238             return [dict(row) for row in cursor.fetchall()]
239
240     def get_process_history(
241         self,
242         name: Optional[str] = None,

```

```

243         pid: Optional[int] = None,
244         limit: int = 100,
245     ) -> List[Dict[str, Any]]:
246         """Извлекает историю поведения процесса по имени или P
ID."""
247         with self._cm.lock, self._cm.get_connection() as conn:
248             cursor = conn.cursor()
249             if pid is not None and name is not None:
250                 cursor.execute(
251                     'SELECT * FROM process_snapshots WHERE pid
= ? AND name LIKE ? ORDER BY id DESC LIMIT ?',
252                     (pid, f'%{name}%', limit)
253                 )
254             elif pid is not None:
255                 cursor.execute(
256                     'SELECT * FROM process_snapshots WHERE pid
= ? ORDER BY id DESC LIMIT ?',
257                     (pid, limit)
258                 )
259             elif name is not None:
260                 cursor.execute(
261                     'SELECT * FROM process_snapshots WHERE nam
e LIKE ? ORDER BY id DESC LIMIT ?',
262                     (f'%{name}%', limit)
263                 )
264             else:
265                 cursor.execute('SELECT * FROM process_snapshot
s ORDER BY id DESC LIMIT ?', (limit,))
266             return [dict(row) for row in cursor.fetchall()]
267
268     def get_latest_processes(self, limit: int = 50, sort_by: s
tr = 'cpu') -> List[Dict[str, Any]]:
269         """Извлекает процессы из самого последнего зафиксирова
нного снимка."""
270         sort_key = (sort_by or 'cpu').lower()
271         if sort_key in ('handles', 'descriptors'):
272             order_col = 'num_handles DESC, cpu_percent DESC'
273         elif sort_key in ('memory', 'ram'):

```

```

274         order_col = 'memory_mb DESC, cpu_percent DESC'
275     else:
276         order_col = 'cpu_percent DESC, memory_mb DESC'
277
278     with self._cm.lock, self._cm.get_connection() as conn:
279         cursor = conn.cursor()
280         cursor.execute('SELECT MAX(id) FROM system_snapshots;')
281         row = cursor.fetchone()
282         last_snap_id = row[0] if row and row[0] is not None else None
283         if last_snap_id is None:
284             cursor.execute('SELECT MAX(snapshot_id) FROM process_snapshots;')
285             row = cursor.fetchone()
286             last_snap_id = row[0] if row and row[0] is not None else None
287         if last_snap_id is None:
288             return []
289
290         sql = f'''
291             SELECT pid, name, status, cpu_percent, memory_
292             mb, memory_percent,
293                 num_threads, num_handles, username, read_bytes_sec, write_bytes_sec, timestamp
294             FROM process_snapshots
295             WHERE snapshot_id = ?
296             ORDER BY {order_col}
297             '''
298         if limit is not None and limit > 0:
299             cursor.execute(sql + ' LIMIT ?', (last_snap_id, limit))
300         else:
301             cursor.execute(sql, (last_snap_id,))
302             return [dict(r) for r in cursor.fetchall()]
303
304     def get_process_stats(self, name: Optional[str] = None, limit: int = 50) -> Dict[str, Any]:

```

```

304 |         """Возвращает агрегированную статистику процессов и вы
      | бросы."""
305 |         with self._cm.lock, self._cm.get_connection() as conn:
306 |             cursor = conn.cursor()
307 |             if name:
308 |                 cursor.execute('SELECT * FROM process_rollups_
      | 2min WHERE name LIKE ? ORDER BY id DESC LIMIT ?', (f'%{name}%'
      | , limit))
309 |             else:
310 |                 cursor.execute('SELECT * FROM process_rollups_
      | 2min ORDER BY id DESC LIMIT ?', (limit,))
311 |                 rollups_2min = [dict(r) for r in cursor.fetchall()
      | ]
312 |
313 |             if name:
314 |                 cursor.execute('SELECT * FROM process_rollups_
      | daily WHERE name LIKE ? ORDER BY date DESC LIMIT ?', (f'%{name}
      | }%', limit))
315 |             else:
316 |                 cursor.execute('SELECT * FROM process_rollups_
      | daily ORDER BY date DESC LIMIT ?', (limit,))
317 |                 daily_stats = [dict(r) for r in cursor.fetchall()]
318 |
319 |             if name:
320 |                 cursor.execute('SELECT * FROM process_outliers
      | WHERE name LIKE ? ORDER BY id DESC LIMIT ?', (f'%{name}%', li
      | mit))
321 |             else:
322 |                 cursor.execute('SELECT * FROM process_outliers
      | ORDER BY id DESC LIMIT ?', (limit,))
323 |                 outliers = [dict(r) for r in cursor.fetchall()]
324 |
325 |             return {
326 |                 'name_filter': name,
327 |                 'rollups_2min': rollups_2min,
328 |                 'daily_stats': daily_stats,
329 |                 'outliers': outliers,
330 |                 'total_rollups_count': len(rollups_2min),

```

```

331         'total_outliers_count': len(outliers),
332     }
333
334     def get_events(self, event_type: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
335         """Извлекает список зафиксированных событий телеметрии"""
336         with self._cm.lock, self._cm.get_connection() as conn:
337             cursor = conn.cursor()
338             if event_type:
339                 cursor.execute('SELECT * FROM telemetry_events
WHERE event_type = ? ORDER BY id DESC LIMIT ?', (event_type,
limit))
340             else:
341                 cursor.execute('SELECT * FROM telemetry_events
ORDER BY id DESC LIMIT ?', (limit,))
342             return [dict(row) for row in cursor.fetchall()]
343
344     def get_sensor_history(
345         self,
346         sensor_id: Optional[str] = None,
347         category: Optional[str] = None,
348         limit: int = 100,
349     ) -> List[Dict[str, Any]]:
350         """Извлекает историю показаний сенсоров."""
351         with self._cm.lock, self._cm.get_connection() as conn:
352             cursor = conn.cursor()
353             if sensor_id and category:
354                 cursor.execute(
355                     'SELECT * FROM sensor_polls WHERE sensor_id = ? AND sensor_category = ? ORDER BY id DESC LIMIT ?',
356                     (sensor_id, category, limit)
357                 )
358             elif sensor_id:
359                 cursor.execute('SELECT * FROM sensor_polls WHERE sensor_id = ? ORDER BY id DESC LIMIT ?', (sensor_id, limit)
)
360             elif category:

```

```

361         cursor.execute('SELECT * FROM sensor_polls WHERE sensor_category = ? ORDER BY id DESC LIMIT ?', (category, limit))
362     else:
363         cursor.execute('SELECT * FROM sensor_polls ORDER BY id DESC LIMIT ?', (limit,))
364         return [dict(row) for row in cursor.fetchall()]
365
366     def get_latest_sensors(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
367         """Извлекает последнее актуальное значение каждого сенсора."""
368         with self._cm.lock, self._cm.get_connection() as conn:
369             cursor = conn.cursor()
370             query = '''
371                 SELECT s1.* FROM sensor_polls s1
372                 INNER JOIN (
373                     SELECT sensor_id, MAX(id) as max_id
374                     FROM sensor_polls
375                     GROUP BY sensor_id
376                 ) s2 ON s1.id = s2.max_id
377                 ORDER BY s1.sensor_category, s1.sensor_name
378             '''
379             if limit is not None:
380                 cursor.execute(query + ' LIMIT ?', (limit,))
381             else:
382                 cursor.execute(query)
383             return [dict(row) for row in cursor.fetchall()]
384
385     def get_cpu_hierarchy_metrics(self) -> Dict[str, Any]:
386         """Извлекает структурированные иерархические метрики процессора из SQLite.
387
388         Выбирает последние замеры сенсоров CPU, системные снимки и историю нагрузки/температур.
389
390         Returns:
391             Dict[str, Any]: Словарь с сенсорами, снимками, пас

```


портом и историей.

```
392         """
393         with self._cm.lock, self._cm.get_connection() as conn:
394             cursor = conn.cursor()
395
396             # 1. Последние сенсоры CPU из sensor_polls (исключ
397             ая дисковые и GPU метрики)
398             query_sensors = '''
399             SELECT s1.* FROM sensor_polls s1
400             INNER JOIN (
401                 SELECT sensor_id, MAX(id) as max_id
402                 FROM sensor_polls
403                 WHERE hardware_type = 'cpu' OR (
404                     hardware_name NOT LIKE '%GPU%'
405                     AND hardware_name NOT LIKE '%Storage%'
406                     AND hardware_name NOT LIKE '%Disk%'
407                     AND sensor_id NOT LIKE 'disk_speed_%'
408                     AND (hardware_type = 'system' OR senso
409             r_category IN ('Voltages', 'Powers', 'Clocks', 'Temperatures',
410             'Load', 'voltages', 'powers', 'clocks', 'temperatures', 'load
411             '))
412             )
413             GROUP BY sensor_id
414             ) s2 ON s1.id = s2.max_id
415             ORDER BY s1.sensor_category, s1.sensor_name
416             '''
417             cursor.execute(query_sensors)
418             sensors = [dict(row) for row in cursor.fetchall()]
419
420             # 2. Последний системный снимок
421             cursor.execute('SELECT * FROM system_snapshots ORD
422             ER BY id DESC LIMIT 1')
423             snap_row = cursor.fetchone()
424             latest_snapshot = dict(snap_row) if snap_row else
425             None
426
427             # 3. Данные инвентаризации CPU
428             cursor.execute('SELECT * FROM cpu_inventory ORDER
```

```

BY processor_id ASC LIMIT 1')
423         inv_row = cursor.fetchone()
424         cpu_inv = dict(inv_row) if inv_row else None
425
426         # 4. История снимков для графиков
427         cursor.execute('''
428             SELECT timestamp, created_at, cpu_total_perce
429             t, cpu_frequency_mhz
430             FROM system_snapshots
431             ORDER BY id DESC
432             LIMIT 60
433         ''')
434         history_rows = [dict(r) for r in cursor.fetchall()]
435     ]
436     history_rows.reverse()
437
438     return {
439         'sensors': sensors,
440         'snapshot': latest_snapshot,
441         'inventory': cpu_inv,
442         'history': history_rows,
443     }
444
445     def get_app_polls(self, app: Optional[str] = None, metric_
446     name: Optional[str] = None, limit: int = 100) -> List[Dict[str
447     , Any]]:
448         """Извлекает историю опросов приложений."""
449         with self._cm.lock, self._cm.get_connection() as conn:
450             cursor = conn.cursor()
451             if app and metric_name:
452                 cursor.execute('SELECT * FROM app_polls WHERE
453                 app = ? AND metric_name = ? ORDER BY id DESC LIMIT ?', (app, m
454                 etric_name, limit))
455             elif app:
456                 cursor.execute('SELECT * FROM app_polls WHERE
457                 app = ? ORDER BY id DESC LIMIT ?', (app, limit))
458             else:
459                 cursor.execute('SELECT * FROM app_polls ORDER

```

```

BY id DESC LIMIT ?', (limit,))
453         return [dict(row) for row in cursor.fetchall()]
454
455     def get_app_events(self, app: Optional[str] = None, event_
type: Optional[str] = None, limit: int = 100) -> List[Dict[str
, Any]]:
456         """Извлекает события приложений."""
457         with self._cm.lock, self._cm.get_connection() as conn:
458             cursor = conn.cursor()
459             if app and event_type:
460                 cursor.execute('SELECT * FROM app_events WHERE
app = ? AND event_type = ? ORDER BY id DESC LIMIT ?', (app, e
vent_type, limit))
461             elif app:
462                 cursor.execute('SELECT * FROM app_events WHERE
app = ? ORDER BY id DESC LIMIT ?', (app, limit))
463             else:
464                 cursor.execute('SELECT * FROM app_events ORDER
BY id DESC LIMIT ?', (limit,))
465             return [dict(row) for row in cursor.fetchall()]
466
467     def get_app_param_changes(self, app: Optional[str] = None,
limit: int = 100) -> List[Dict[str, Any]]:
468         """Извлекает изменения параметров приложений."""
469         with self._cm.lock, self._cm.get_connection() as conn:
470             cursor = conn.cursor()
471             if app:
472                 cursor.execute('SELECT * FROM app_param_change
s WHERE app = ? ORDER BY id DESC LIMIT ?', (app, limit))
473             else:
474                 cursor.execute('SELECT * FROM app_param_change
s ORDER BY id DESC LIMIT ?', (limit,))
475             return [dict(row) for row in cursor.fetchall()]
476
477     def export_app_polls_to_csv(self, app: Optional[str] = Non
e, output_path: Optional[Union[str, Path]] = None, limit: int
= 50000) -> Path:
478         """Экспортирует историю опросов приложений в CSV-файл.

```

```

"""
479         polls = self.get_app_polls(app=app, limit=limit)
480         target = Path(output_path) if output_path else Path('a
pp_polls.csv')
481         target.parent.mkdir(parents=True, exist_ok=True)
482         headers = ['timestamp', 'app', 'poll_type', 'metric_na
me', 'value', 'unit', 'status', 'details']
483         with open(target, 'w', newline='', encoding='utf-8-sig
') as f:
484             writer = csv.writer(f)
485             writer.writerow(headers)
486             for p in polls:
487                 writer.writerow([
488                     p.get('timestamp', ''), p.get('app', ''),
p.get('poll_type', ''),
489                     p.get('metric_name', ''), p.get('value', '
'), p.get('unit', ''),
490                     p.get('status', ''), p.get('details', ''),
491                     ])
492         return target
493
494     def export_app_events_to_csv(self, app: Optional[str] = No
ne, output_path: Optional[Union[str, Path]] = None, limit: int
= 50000) -> Path:
495         """Экспортирует историю событий приложений в CSV-файл.
"""
496         events = self.get_app_events(app=app, limit=limit)
497         target = Path(output_path) if output_path else Path('a
pp_events.csv')
498         target.parent.mkdir(parents=True, exist_ok=True)
499         headers = ['timestamp', 'app', 'event_type', 'status',
'user', 'details']
500         with open(target, 'w', newline='', encoding='utf-8-sig
') as f:
501             writer = csv.writer(f)
502             writer.writerow(headers)
503             for e in events:
504                 writer.writerow([

```

```

505         e.get('timestamp', ''), e.get('app', ''),
    e.get('event_type', ''),
506         e.get('status', ''), e.get('user', ''), e.
    get('details', ''),
507     ])
508     return target
509
510     def export_app_param_changes_to_csv(self, app: Optional[str] = None, output_path: Optional[Union[str, Path]] = None, limit: int = 50000) -> Path:
511         """Экспортирует историю изменения параметров в CSV-файл."""
512         params = self.get_app_param_changes(app=app, limit=limit)
513         target = Path(output_path) if output_path else Path('app_param_changes.csv')
514         target.parent.mkdir(parents=True, exist_ok=True)
515         headers = ['timestamp', 'app', 'param_name', 'old_value', 'new_value', 'status', 'user', 'details']
516         with open(target, 'w', newline='', encoding='utf-8-sig') as f:
517             writer = csv.writer(f)
518             writer.writerow(headers)
519             for p in params:
520                 writer.writerow([
521                     p.get('timestamp', ''), p.get('app', ''),
    p.get('param_name', ''),
522                     p.get('old_value', ''), p.get('new_value',
    ''), p.get('status', ''),
523                     p.get('user', ''), p.get('details', ''),
524                 ])
525         return target
526
527     def get_custom_records(self, source_file: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
528         """Извлекает кастомные записи логов."""
529         with self._cm.lock, self._cm.get_connection() as conn:
530             cursor = conn.cursor()

```

```

531         if source_file:
532             cursor.execute('SELECT * FROM custom_records W
HERE source_file = ? ORDER BY id DESC LIMIT ?', (source_file,
limit))
533         else:
534             cursor.execute('SELECT * FROM custom_records O
RDER BY id DESC LIMIT ?', (limit,))
535         return [dict(row) for row in cursor.fetchall()]
536
537     def get_device_events(self, event_type: Optional[str] = No
ne, device_instance_id: Optional[str] = None, limit: int = 100
) -> List[Dict[str, Any]]:
538         """Извлекает события устройств."""
539         with self._cm.lock, self._cm.get_connection() as conn:
540             cursor = conn.cursor()
541             if event_type and device_instance_id:
542                 cursor.execute('SELECT * FROM device_events WH
ERE event_type = ? AND device_instance_id = ? ORDER BY id DESC
LIMIT ?', (event_type, device_instance_id, limit))
543             elif event_type:
544                 cursor.execute('SELECT * FROM device_events WH
ERE event_type = ? ORDER BY id DESC LIMIT ?', (event_type, lim
it))
545             elif device_instance_id:
546                 cursor.execute('SELECT * FROM device_events WH
ERE device_instance_id = ? ORDER BY id DESC LIMIT ?', (device_
instance_id, limit))
547             else:
548                 cursor.execute('SELECT * FROM device_events OR
DER BY id DESC LIMIT ?', (limit,))
549             return [dict(row) for row in cursor.fetchall()]
550
551     def get_device_install_date(self, device_instance_id: str)
-> Optional[str]:
552         """Возвращает дату установки устройства из таблицы dev
ice_inventory."""
553         norm_id = str(device_instance_id).strip().upper()
554         if not norm_id:

```

```

555         return None
556     with self._cm.lock, self._cm.get_connection() as conn:
557         cursor = conn.cursor()
558         cursor.execute('SELECT install_date FROM device_in
ventory WHERE UPPER(device_instance_id) = ?', (norm_id,))
559         row = cursor.fetchone()
560         return str(row[0]) if (row and row[0]) else None
561
562     def get_w64_events(self, event_type: Optional[str] = None,
provider: Optional[str] = None, limit: int = 100) -> List[Dic
t[str, Any]]:
563         """Извлекает системные события W64/ETW."""
564         query = 'SELECT raw_json FROM w64_events WHERE 1=1'
565         params: List[Any] = []
566         if event_type:
567             query += ' AND event_type = ?'
568             params.append(event_type)
569         if provider:
570             query += ' AND provider = ?'
571             params.append(provider)
572         query += ' ORDER BY created_at DESC LIMIT ?'
573         params.append(max(1, limit))
574
575         with self._cm.lock, self._cm.get_connection() as conn:
576             cursor = conn.cursor()
577             cursor.execute(query, tuple(params))
578             results = []
579             for row in cursor.fetchall():
580                 try:
581                     results.append(json.loads(row[0]))
582                 except Exception:
583                     continue
584             return results
585
586     def get_incidents(self, limit: int = 50, trigger_type: Opt
ional[str] = None, severity: Optional[str] = None) -> List[Dic
t[str, Any]]:
587         """Возвращает список зафиксированных инцидентов."""

```

```

588         query = 'SELECT * FROM incidents WHERE 1=1'
589         params: List[Any] = []
590         if trigger_type:
591             query += ' AND trigger_type = ?'
592             params.append(trigger_type)
593         if severity:
594             query += ' AND severity = ?'
595             params.append(severity)
596         query += ' ORDER BY created_at DESC LIMIT ?'
597         params.append(limit)
598
599         results: List[Dict[str, Any]] = []
600         try:
601             with self._cm.lock, self._cm.get_connection() as c
602             onn:
603                 cursor = conn.cursor()
604                 cursor.execute(query, tuple(params))
605                 for row in cursor.fetchall():
606                     item = dict(row)
607                     for f in ('trigger_metrics_json', 'suspect
608                     _processes_json', 'related_events_json', 'metrics_summary_json
609                     ', 'raw_window_json'):
610                         if item.get(f):
611                             try:
612                                 item[f.replace('_json', '')] =
613                                 json.loads(item[f])
614                             except Exception:
615                                 item[f.replace('_json', '')] =
616                                 []
617                             results.append(item)
618             except Exception as ex:
619                 logger.error(f'Ошибка при получении инцидентов: {e
620                 x}')
621         return results
622
623     def get_system_rollups(self, tier: str = '1m', start_epoch
624     : Optional[float] = None, end_epoch: Optional[float] = None, l
625     imit: int = 100) -> List[Dict[str, Any]]:

```



```

618 |         """Возвращает сжатые бакеты метрик за период."""
619 |         query = 'SELECT * FROM telemetry_rollups WHERE tier =
        | ?'
620 |         params: List[Any] = [tier]
621 |         if start_epoch is not None:
622 |             query += ' AND period_start_epoch >= ?'
623 |             params.append(start_epoch)
624 |         if end_epoch is not None:
625 |             query += ' AND period_end_epoch <= ?'
626 |             params.append(end_epoch)
627 |         query += ' ORDER BY period_start_epoch DESC LIMIT ?'
628 |         params.append(limit)
629 |
630 |         results: List[Dict[str, Any]] = []
631 |         try:
632 |             with self._cm.lock, self._cm.get_connection() as c
        | onn:
633 |                 cursor = conn.cursor()
634 |                 cursor.execute(query, tuple(params))
635 |                 for row in cursor.fetchall():
636 |                     results.append(dict(row))
637 |         except Exception as ex:
638 |             logger.error(f'Ошибка при получении rollups: {ex}'
        | )
639 |         return results
640 |
641 |     def get_reboot_history(self, limit: int = 50) -> List[Dict
        | [str, Any]]:
642 |         """Извлекает историю сессий перезагрузок."""
643 |         results: List[Dict[str, Any]] = []
644 |         try:
645 |             with self._cm.lock, self._cm.get_connection() as c
        | onn:
646 |                 cursor = conn.cursor()
647 |                 cursor.execute('SELECT * FROM reboot_history O
        | RDER BY boot_time DESC, created_at DESC LIMIT ?', (limit,))
648 |                 for row in cursor.fetchall():
649 |                     item = dict(row)

```

```

650         for f in ('bugcheck_params_json', 'evidenc
e_json', 'events_chain_json', 'raw_json'):
651             if item.get(f):
652                 try:
653                     item[f.replace('_json', '')] =
json.loads(item[f])
654             except Exception:
655                 pass
656             results.append(item)
657     except Exception as ex:
658         logger.error(f'Ошибка извлечения истории перезагру
зок: {ex}')
659     return results
660
661     def get_latest_reboot_session(self) -> Optional[Dict[str,
Any]]:
662         """Возвращает самую последнюю запись сессии перезагруз
ки."""
663         history = self.get_reboot_history(limit=1)
664         return history[0] if history else None
665
666     def get_extended_audits(self, limit: int = 50) -> List[Dic
t[str, Any]]:
667         """Возвращает историю расширенных системных аудитов."""
668         results: List[Dict[str, Any]] = []
669         try:
670             with self._cm.lock, self._cm.get_connection() as c
onn:
671                 cursor = conn.cursor()
672                 cursor.execute('SELECT * FROM system_extended_
audits ORDER BY created_at DESC LIMIT ?', (limit,))
673                 for row in cursor.fetchall():
674                     item = dict(row)
675                     if item.get('raw_json'):
676                         try:
677                             item['data'] = json.loads(item['ra
w_json'])

```

```

678 |             except Exception:
679 |                 pass
680 |                 results.append(item)
681 |         except Exception as ex:
682 |             logger.error(f'Ошибка извлечения расширенных аудит
| ов: {ex}')
683 |         return results
684 |
685 |     def get_latest_extended_audit(self) -> Optional[Dict[str,
| Any]]:
686 |         """Возвращает последний расширенный системный аудит."""
687 |         audits = self.get_extended_audits(limit=1)
688 |         return audits[0] if audits else None
689 |
690 |     def get_process_provenance_history(
691 |         self,
692 |         guid: Optional[str] = None,
693 |         pid: Optional[int] = None,
694 |         name: Optional[str] = None,
695 |         user: Optional[str] = None,
696 |         event_type: Optional[str] = None,
697 |         hours: Optional[int] = None,
698 |         limit: int = 100,
699 |     ) -> List[Dict[str, Any]]:
700 |         """Извлечение исторических событий provenance процессо
| в. """
701 |         results: List[Dict[str, Any]] = []
702 |         query = 'SELECT * FROM process_provenance_events WHERE
| 1=1'
703 |         params: List[Any] = []
704 |
705 |         if hours is not None:
706 |             cutoff = time.time() - (hours * 3600)
707 |             query += ' AND created_at >= ?'
708 |             params.append(cutoff)
709 |         if guid:
710 |             query += ' AND process_guid = ?'

```

```

711         params.append(guid)
712     if pid is not None:
713         query += ' AND pid = ?'
714         params.append(pid)
715     if name:
716         query += ' AND name LIKE ?'
717         params.append(f'%{name}%')
718     if user:
719         query += ' AND (user LIKE ? OR sid LIKE ?)'
720         params.extend([f'%{user}%', f'%{user}%'])
721     if event_type:
722         query += ' AND event_type = ?'
723         params.append(event_type)
724
725     query += ' ORDER BY created_at DESC LIMIT ?'
726     params.append(limit)
727
728     try:
729         with self._cm.lock, self._cm.get_connection() as c
730     onn:
731         cursor = conn.cursor()
732         cursor.execute(query, tuple(params))
733         for row in cursor.fetchall():
734             item = dict(row)
735             if item.get('details_json'):
736                 try:
737                     item['details'] = json.loads(item[
738 'details_json'])
739                 except Exception:
740                     item['details'] = item['details_js
741 on']
742             results.append(item)
743     except Exception as ex:
744         logger.error(f'Ошибка извлечения provenance процес
745 сов: {ex}')
746     return results
747
748     def get_process_lineage(self, guid_or_pid: Union[str, int])

```

```

) -> List[Dict[str, Any]]:
745     """Построение полной восходящей цепочки предков (linea
    ge) процесса."""
746     str_val = str(guid_or_pid)
747     chain: List[Dict[str, Any]] = []
748     visited_guids: set = set()
749     visited_pids: set = set()
750
751     history = self.get_process_provenance_history(guid=str
_val, limit=1)
752     if not history and str_val.isdigit():
753         history = self.get_process_provenance_history(pid=
int(str_val), limit=1)
754
755     current_node: Optional[Dict[str, Any]] = history[0] if
    history else None
756
757     while current_node:
758         node_guid = current_node.get('process_guid')
759         node_pid = current_node.get('pid')
760
761         if (node_guid and node_guid in visited_guids) or (
    node_pid and node_pid in visited_pids):
762             break
763
764         if node_guid:
765             visited_guids.add(node_guid)
766         if node_pid:
767             visited_pids.add(node_pid)
768
769         chain.append(current_node)
770         parent_guid = current_node.get('parent_guid')
771         parent_pid = current_node.get('ppid') or current_n
ode.get('parent_pid')
772
773         parent_node = None
774         if parent_guid:
775             parent_hist = self.get_process_provenance_hist

```

```

    ory(guid=parent_guid, limit=1)
776         if parent_hist:
777             parent_node = parent_hist[0]
778
779         if not parent_node and parent_pid:
780             parent_hist = self.get_process_provenance_hist
    ory(pid=int(parent_pid), limit=1)
781         if parent_hist:
782             parent_node = parent_hist[0]
783
784         current_node = parent_node
785
786     chain.reverse()
787     return chain
788
789     def get_telemetry_rollups(
790         self,
791         level: str = 'hourly',
792         sensor_id: Optional[str] = None,
793         start_epoch: Optional[float] = None,
794         end_epoch: Optional[float] = None,
795         limit: int = 100,
796     ) -> List[Dict[str, Any]]:
797         """Извлекает агрегированные показатели сенсоров для за
данного уровня (hourly, daily, weekly, monthly, yearly).
798
799         Args:
800             level: Уровень иерархии ('hourly', 'daily', 'weekl
y', 'monthly', 'yearly').
801             sensor_id: Фильтр по идентификатору сенсора (опцио
нально).
802             start_epoch: Нижняя граница времени бакета bucket_
start (опционально).
803             end_epoch: Верхняя граница времени бакета bucket_s
tart (опционально).
804             limit: Максимальное число возвращаемых записей.
805
806         Returns:

```

```

807         List[Dict[str, Any]]: Список агрегированных бакето
      в со статистикой.
808         """
809         lvl_clean = str(level).strip().lower()
810         table_name = f'telemetry_{lvl_clean}'
811         valid_tables = {'telemetry_hourly', 'telemetry_daily',
      'telemetry_weekly', 'telemetry_monthly', 'telemetry_yearly'}
812         if table_name not in valid_tables:
813             table_name = 'telemetry_hourly'
814
815         conditions = []
816         params: List[Any] = []
817
818         if sensor_id is not None:
819             conditions.append('sensor_id = ?')
820             params.append(sensor_id)
821         if start_epoch is not None:
822             conditions.append('bucket_start >= ?')
823             params.append(float(start_epoch))
824         if end_epoch is not None:
825             conditions.append('bucket_start < ?')
826             params.append(float(end_epoch))
827
828         where_clause = f"WHERE {' AND '.join(conditions)}" if
      conditions else ""
829         query = f"SELECT * FROM {table_name} {where_clause} OR
      DER BY bucket_start DESC LIMIT ?"
830         params.append(int(limit))
831
832         with self._cm.lock, self._cm.get_connection() as conn:
833             cursor = conn.cursor()
834             cursor.execute(query, params)
835             return [dict(row) for row in cursor.fetchall()]
836
837     def get_telemetry_spikes(
838         self,
839         sensor_id: Optional[str] = None,
840         start_epoch: Optional[float] = None,

```

```

841         limit: int = 100,
842     ) -> List[Dict[str, Any]]:
843         """Извлекает зафиксированные статистические всплески и
            аномалии сенсоров.
844
845         Args:
846             sensor_id: Фильтр по идентификатору сенсора (опцио
            нально).
847             start_epoch: Нижняя граница времени регистрации (о
            пционально).
848             limit: Максимальное число возвращаемых записей.
849
850         Returns:
851             List[Dict[str, Any]]: Список аномальных записей.
852         """
853         conditions = []
854         params: List[Any] = []
855
856         if sensor_id is not None:
857             conditions.append('sensor_id = ?')
858             params.append(sensor_id)
859         if start_epoch is not None:
860             conditions.append('created_at >= ?')
861             params.append(float(start_epoch))
862
863         where_clause = f"WHERE {' AND '.join(conditions)}" if
            conditions else ""
864         query = f"SELECT * FROM telemetry_spikes {where_clause}
            ORDER BY created_at DESC LIMIT ?"
865         params.append(int(limit))
866
867         with self._cm.lock, self._cm.get_connection() as conn:
868             cursor = conn.cursor()
869             cursor.execute(query, params)
870             return [dict(row) for row in cursor.fetchall()]
871
872     def get_disk_inventory(self, disk_id: Optional[int] = None
            ) -> List[Dict[str, Any]]:

```



```

873 |         """Извлекает паспортные данные физических дисков из та
      | блицы disk_inventory."""
874 |         with self._cm.lock, self._cm.get_connection() as conn:
875 |             cursor = conn.cursor()
876 |             if disk_id is not None:
877 |                 cursor.execute('SELECT * FROM disk_inventory W
      | HERE disk_id = ? ORDER BY disk_id ASC', (disk_id,))
878 |             else:
879 |                 cursor.execute('SELECT * FROM disk_inventory O
      | RDER BY disk_id ASC')
880 |             return [dict(row) for row in cursor.fetchall()]
881 |
882 |     def get_volume_inventory(self) -> List[Dict[str, Any]]:
883 |         """Извлекает паспортные данные логических томов из таб
      | лицы volume_inventory."""
884 |         with self._cm.lock, self._cm.get_connection() as conn:
885 |             cursor = conn.cursor()
886 |             cursor.execute('SELECT * FROM volume_inventory ORD
      | ER BY drive_letter ASC')
887 |             return [dict(row) for row in cursor.fetchall()]
888 |
889 |     def get_disk_health_history(
890 |         self,
891 |         disk_id: Optional[int] = None,
892 |         serial_number: Optional[str] = None,
893 |         limit: int = 100,
894 |         since_epoch: Optional[float] = None,
895 |     ) -> List[Dict[str, Any]]:
896 |         """Извлекает историю показателей здоровья и износа на
      | копителей."""
897 |         conditions = []
898 |         params: List[Any] = []
899 |
900 |         if disk_id is not None:
901 |             conditions.append('disk_id = ?')
902 |             params.append(disk_id)
903 |         if serial_number is not None:
904 |             conditions.append('serial_number = ?')

```

```

905         params.append(serial_number)
906     if since_epoch is not None:
907         conditions.append('created_at >= ?')
908         params.append(float(since_epoch))
909
910     where_clause = f"WHERE {' AND '}.join(conditions)}" if
conditions else ""
911     query = f"SELECT * FROM disk_health_snapshots {where_c
lause} ORDER BY created_at DESC LIMIT ?"
912     params.append(int(limit))
913
914     with self._cm.lock, self._cm.get_connection() as conn:
915         cursor = conn.cursor()
916         cursor.execute(query, params)
917         return [dict(row) for row in cursor.fetchall()]
918
919     def get_disk_performance_samples(
920         self,
921         disk_name: Optional[str] = None,
922         limit: int = 100,
923         since_epoch: Optional[float] = None,
924     ) -> List[Dict[str, Any]]:
925         """Извлекает историю показателей производительности ди
сков (IOPS, MB/s)."""
926         conditions = []
927         params: List[Any] = []
928
929         if disk_name is not None:
930             conditions.append('disk_name = ?')
931             params.append(disk_name)
932         if since_epoch is not None:
933             conditions.append('created_at >= ?')
934             params.append(float(since_epoch))
935
936         where_clause = f"WHERE {' AND '}.join(conditions)}" if
conditions else ""
937         query = f"SELECT * FROM disk_performance_samples {wher
e_clause} ORDER BY created_at DESC LIMIT ?"

```

```

938         params.append(int(limit))
939
940         with self._cm.lock, self._cm.get_connection() as conn:
941             cursor = conn.cursor()
942             cursor.execute(query, params)
943             return [dict(row) for row in cursor.fetchall()]
944
945     def get_disk_io_events(
946         self,
947         pid: Optional[int] = None,
948         disk_id: Optional[int] = None,
949         limit: int = 100,
950         since_epoch: Optional[float] = None,
951     ) -> List[Dict[str, Any]]:
952         """Извлекает события прямого ввода-вывода процессов."""
953
954         conditions = []
955         params: List[Any] = []
956
957         if pid is not None:
958             conditions.append('pid = ?')
959             params.append(pid)
960         if disk_id is not None:
961             conditions.append('disk_id = ?')
962             params.append(disk_id)
963         if since_epoch is not None:
964             conditions.append('created_at >= ?')
965             params.append(float(since_epoch))
966
967         where_clause = f"WHERE {' AND '.join(conditions)}" if
conditions else ""
968         query = f"SELECT * FROM disk_io_events {where_clause}
ORDER BY created_at DESC LIMIT ?"
969
970         params.append(int(limit))
971
972         with self._cm.lock, self._cm.get_connection() as conn:
973             cursor = conn.cursor()
974             cursor.execute(query, params)

```

```

973         return [dict(row) for row in cursor.fetchall()]
974
975     def get_cpu_inventory(self, processor_id: Optional[int] =
None) -> List[Dict[str, Any]]:
976         """Извлекает паспортные данные процессоров из таблицы
cpu_inventory."""
977         with self._cm.lock, self._cm.get_connection() as conn:
978             cursor = conn.cursor()
979             if processor_id is not None:
980                 cursor.execute('SELECT * FROM cpu_inventory WH
ERE processor_id = ? ORDER BY processor_id ASC', (processor_id
,))
981             else:
982                 cursor.execute('SELECT * FROM cpu_inventory OR
DER BY processor_id ASC')
983             return [dict(row) for row in cursor.fetchall()]
984
985     def get_ram_module_inventory(self) -> List[Dict[str, Any]]
:
986         """Извлекает паспортные данные физических планок памят
и из таблицы ram_module_inventory."""
987         with self._cm.lock, self._cm.get_connection() as conn:
988             cursor = conn.cursor()
989             cursor.execute('SELECT * FROM ram_module_inventory
ORDER BY slot_id ASC')
990             return [dict(row) for row in cursor.fetchall()]
991
992     def get_gpu_inventory(self, gpu_id: Optional[int] = None)
-> List[Dict[str, Any]]:
993         """Извлекает паспортные данные графических ускорителей
из таблицы gpu_inventory."""
994         with self._cm.lock, self._cm.get_connection() as conn:
995             cursor = conn.cursor()
996             if gpu_id is not None:
997                 cursor.execute('SELECT * FROM gpu_inventory WH
ERE gpu_id = ? ORDER BY gpu_id ASC', (gpu_id,))
998             else:
999                 cursor.execute('SELECT * FROM gpu_inventory OR

```

```

DER BY gpu_id ASC')
1000         return [dict(row) for row in cursor.fetchall()]
1001
1002     def get_network_adapter_inventory(self) -> List[Dict[str,
Any]]:
1003         """Извлекает паспортные данные сетевых интерфейсов из
таблицы network_adapter_inventory."""
1004         with self._cm.lock, self._cm.get_connection() as conn:
1005             cursor = conn.cursor()
1006             cursor.execute('SELECT * FROM network_adapter_inve
ntory ORDER BY adapter_id ASC')
1007             return [dict(row) for row in cursor.fetchall()]
1008
1009     def get_cpu_telemetry_samples(
1010         self,
1011         processor_id: Optional[int] = None,
1012         limit: int = 100,
1013         since_epoch: Optional[float] = None,
1014     ) -> List[Dict[str, Any]]:
1015         """Извлекает историю показателей нагрузки и температур
процессоров."""
1016         conditions = []
1017         params: List[Any] = []
1018
1019         if processor_id is not None:
1020             conditions.append('processor_id = ?')
1021             params.append(processor_id)
1022         if since_epoch is not None:
1023             conditions.append('created_at >= ?')
1024             params.append(float(since_epoch))
1025
1026         where_clause = f"WHERE {' AND '.join(conditions)}" if
conditions else ""
1027         query = f"SELECT * FROM cpu_telemetry_samples {where_c
lause} ORDER BY created_at DESC LIMIT ?"
1028         params.append(int(limit))
1029
1030         with self._cm.lock, self._cm.get_connection() as conn:

```

```

1031         cursor = conn.cursor()
1032         cursor.execute(query, params)
1033         return [dict(row) for row in cursor.fetchall()]
1034
1035     def get_ram_telemetry_samples(
1036         self,
1037         limit: int = 100,
1038         since_epoch: Optional[float] = None,
1039     ) -> List[Dict[str, Any]]:
1040         """Извлекает историю показателей использования оперативной памяти."""
1041         conditions = []
1042         params: List[Any] = []
1043
1044         if since_epoch is not None:
1045             conditions.append('created_at >= ?')
1046             params.append(float(since_epoch))
1047
1048         where_clause = f"WHERE {' AND '.join(conditions)}" if conditions else ""
1049         query = f"SELECT * FROM ram_telemetry_samples {where_clause} ORDER BY created_at DESC LIMIT ?"
1050         params.append(int(limit))
1051
1052         with self._cm.lock, self._cm.get_connection() as conn:
1053             cursor = conn.cursor()
1054             cursor.execute(query, params)
1055             return [dict(row) for row in cursor.fetchall()]
1056
1057     def get_gpu_telemetry_samples(
1058         self,
1059         gpu_id: Optional[int] = None,
1060         name: Optional[str] = None,
1061         limit: int = 100,
1062         since_epoch: Optional[float] = None,
1063     ) -> List[Dict[str, Any]]:
1064         """Извлекает историю показателей нагрузки, памяти и температуры GPU."""

```

```

1065 |         conditions = []
1066 |         params: List[Any] = []
1067 |
1068 |         if gpu_id is not None:
1069 |             conditions.append('gpu_id = ?')
1070 |             params.append(gpu_id)
1071 |         if name is not None:
1072 |             conditions.append('name = ?')
1073 |             params.append(name)
1074 |         if since_epoch is not None:
1075 |             conditions.append('created_at >= ?')
1076 |             params.append(float(since_epoch))
1077 |
1078 |         where_clause = f"WHERE {' AND '.join(conditions)}" if
| conditions else ""
1079 |         query = f"SELECT * FROM gpu_telemetry_samples {where_c
| lause} ORDER BY created_at DESC LIMIT ?"
1080 |         params.append(int(limit))
1081 |
1082 |         with self._cm.lock, self._cm.get_connection() as conn:
1083 |             cursor = conn.cursor()
1084 |             cursor.execute(query, params)
1085 |             return [dict(row) for row in cursor.fetchall()]
1086 |
1087 |     def get_network_adapter_samples(
1088 |         self,
1089 |         adapter_name: Optional[str] = None,
1090 |         limit: int = 100,
1091 |         since_epoch: Optional[float] = None,
1092 |     ) -> List[Dict[str, Any]]:
1093 |         """Извлекает историю показателей сетевой активности ад
| аптеров."""
1094 |         conditions = []
1095 |         params: List[Any] = []
1096 |
1097 |         if adapter_name is not None:
1098 |             conditions.append('adapter_name = ?')
1099 |             params.append(adapter_name)

```

```
1100 |         if since_epoch is not None:
1101 |             conditions.append('created_at >= ?')
1102 |             params.append(float(since_epoch))
1103 |
1104 |         where_clause = f"WHERE {' AND '}.join(conditions)}" if
| conditions else ""
1105 |         query = f"SELECT * FROM network_adapter_samples {where
| _clause} ORDER BY created_at DESC LIMIT ?"
1106 |         params.append(int(limit))
1107 |
1108 |         with self._cm.lock, self._cm.get_connection() as conn:
1109 |             cursor = conn.cursor()
1110 |             cursor.execute(query, params)
1111 |             return [dict(row) for row in cursor.fetchall()]
1112 |
1113 |
1114 |
```


schema.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Schema
4 # =====
5 # Description:
6 #   Схема таблиц, индексов и миграций SQLite базы данных систе
  мной телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.schema import init_da
  tabase_schema
11 #
12 #       init_database_schema(connection)
13 #
14 # File: schema.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.sqlite
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 04:25:00
20 # =====
21
22 from __future__ import annotations
23
24 """Схема таблиц, индексов и процедур миграции базы данных теле
  метрии SQLite."""
25
26 import sqlite3
27 from typing import Set
28
29 try:
```

```
30     from logger import logger
31 except ImportError:
32     import logging
33     logger = logging.getLogger(__name__)
34
35
36 def init_database_schema(conn: sqlite3.Connection) -> None:
37     """Инициализирует таблицы, индексы и выполняет миграции дл
    я базы данных телеметрии.
38
39     Args:
40         conn: Активное подключение к базе данных SQLite.
41     """
42     cursor = conn.cursor()
43
44     # 1. Системные снимки (основная таблица метрик)
45     cursor.execute('''
46         CREATE TABLE IF NOT EXISTS system_snapshots (
47             id INTEGER PRIMARY KEY AUTOINCREMENT,
48             timestamp TEXT NOT NULL,
49             created_at REAL NOT NULL,
50             hostname TEXT,
51             username TEXT,
52             uptime_seconds REAL,
53             os_name TEXT,
54             os_build TEXT,
55             os_install_date TEXT,
56             system_language TEXT,
57             os_install_language TEXT,
58             user_locale TEXT,
59             system_locale TEXT,
60             timezone TEXT,
61             codepage TEXT,
62             input_languages_json TEXT,
63             disks_json TEXT,
64             physical_disks_json TEXT,
65             monitors_json TEXT,
66             updates_json TEXT,
```

```

67         office_json TEXT,
68         onedrive_json TEXT,
69         battery_json TEXT,
70         ram_sticks_json TEXT,
71         listening_ports_json TEXT,
72         alerts_json TEXT,
73         cpu_total_percent REAL,
74         cpu_frequency_mhz REAL,
75         memory_total_gb REAL,
76         memory_used_gb REAL,
77         memory_percent REAL,
78         swap_percent REAL,
79         gpu_load_percent REAL,
80         gpu_temp_c REAL,
81         disk_read_bytes_sec REAL,
82         disk_write_bytes_sec REAL,
83         disk_read_count_sec REAL,
84         disk_write_count_sec REAL,
85         network_sent_bytes_sec REAL,
86         network_recv_bytes_sec REAL,
87         hardware_audit TEXT,
88         raw_json TEXT
89     );
90 '''
91
92 # 2. Сенсоры (аппаратные датчики)
93 cursor.execute('''
94     CREATE TABLE IF NOT EXISTS sensor_polls (
95         id INTEGER PRIMARY KEY AUTOINCREMENT,
96         sensor_id TEXT NOT NULL,
97         timestamp TEXT NOT NULL,
98         created_at REAL NOT NULL,
99         hardware_name TEXT,
100        hardware_type TEXT,
101        sensor_category TEXT,
102        sensor_name TEXT,
103        unit TEXT,
104        value REAL,

```

```

105         provider TEXT,
106         provider_priority INTEGER DEFAULT 30,
107         raw_json TEXT,
108         UNIQUE(sensor_id, timestamp)
109     );
110     ''')
111
112 # 3. Снимки процессов
113 cursor.execute('''
114     CREATE TABLE IF NOT EXISTS process_snapshots (
115         id INTEGER PRIMARY KEY AUTOINCREMENT,
116         snapshot_id INTEGER NOT NULL,
117         timestamp TEXT NOT NULL,
118         pid INTEGER NOT NULL,
119         name TEXT NOT NULL,
120         status TEXT,
121         cpu_percent REAL,
122         memory_mb REAL,
123         memory_percent REAL,
124         num_threads INTEGER,
125         num_handles INTEGER DEFAULT 0,
126         username TEXT,
127         read_bytes_sec REAL,
128         write_bytes_sec REAL,
129         integrity_level TEXT,
130         elevation INTEGER DEFAULT 0,
131         ppid INTEGER,
132         parent_name TEXT,
133         executable TEXT,
134         cmdline TEXT,
135         sid TEXT,
136         session_id INTEGER,
137         creation_time TEXT,
138         process_guid TEXT,
139         ancestor_chain TEXT,
140         launch_reason TEXT,
141         FOREIGN KEY (snapshot_id) REFERENCES system_snapsh
ots(id) ON DELETE CASCADE

```

```

142         );
143     '')
144
145     # 4. Происхождение и жизненный цикл процессов
146     cursor.execute('''
147         CREATE TABLE IF NOT EXISTS process_provenance_events (
148             id INTEGER PRIMARY KEY AUTOINCREMENT,
149             event_id TEXT NOT NULL,
150             timestamp TEXT NOT NULL,
151             created_at REAL NOT NULL,
152             event_type TEXT NOT NULL,
153             pid INTEGER NOT NULL,
154             ppid INTEGER,
155             process_guid TEXT NOT NULL,
156             parent_guid TEXT,
157             name TEXT NOT NULL,
158             executable_path TEXT,
159             command_line TEXT,
160             user TEXT,
161             sid TEXT,
162             session_id INTEGER,
163             integrity_level TEXT,
164             elevation INTEGER,
165             parent_name TEXT,
166             parent_cmdline TEXT,
167             ancestor_chain TEXT,
168             launch_reason TEXT,
169             source TEXT DEFAULT 'system',
170             details_json TEXT,
171             raw_json TEXT
172         );
173     '')
174
175     # 5. Почасовые агрегаты сенсоров
176     cursor.execute('''
177         CREATE TABLE IF NOT EXISTS sensor_aggregates_hourly (
178             id INTEGER PRIMARY KEY AUTOINCREMENT,
179             sensor_id TEXT NOT NULL,

```

```

180         period_start TEXT NOT NULL,
181         period_end TEXT NOT NULL,
182         avg_value REAL,
183         min_value REAL,
184         max_value REAL,
185         count INTEGER,
186         raw_json TEXT
187     );
188 '''
189
190 # 6. Двухминутные агрегаты процессов
191 cursor.execute('''
192     CREATE TABLE IF NOT EXISTS process_rollups_2min (
193         id INTEGER PRIMARY KEY AUTOINCREMENT,
194         period_start TEXT NOT NULL,
195         period_end TEXT NOT NULL,
196         period_start_epoch REAL,
197         period_end_epoch REAL,
198         name TEXT NOT NULL,
199         pid INTEGER,
200         username TEXT,
201         sample_count INTEGER NOT NULL,
202         avg_cpu_percent REAL NOT NULL,
203         max_cpu_percent REAL NOT NULL,
204         min_cpu_percent REAL,
205         avg_memory_mb REAL NOT NULL,
206         max_memory_mb REAL NOT NULL,
207         min_memory_mb REAL,
208         avg_num_threads REAL,
209         outliers_count INTEGER DEFAULT 0
210     );
211 '''
212
213 # 7. Суточные агрегаты процессов
214 cursor.execute('''
215     CREATE TABLE IF NOT EXISTS process_rollups_daily (
216         id INTEGER PRIMARY KEY AUTOINCREMENT,
217         date TEXT NOT NULL,

```

```

218         name TEXT NOT NULL,
219         sample_count INTEGER NOT NULL,
220         avg_cpu_percent REAL NOT NULL,
221         max_cpu_percent REAL NOT NULL,
222         avg_memory_mb REAL NOT NULL,
223         max_memory_mb REAL NOT NULL,
224         outliers_count INTEGER DEFAULT 0,
225         last_seen TEXT
226     );
227 '''
228
229 # 8. Бакеты сжатия временных рядов телеметрии
230 cursor.execute('''
231     CREATE TABLE IF NOT EXISTS telemetry_rollups (
232         id INTEGER PRIMARY KEY AUTOINCREMENT,
233         period_start TEXT NOT NULL,
234         period_end TEXT NOT NULL,
235         period_start_epoch REAL NOT NULL,
236         period_end_epoch REAL NOT NULL,
237         duration_seconds REAL NOT NULL,
238         sample_count INTEGER NOT NULL,
239         tier TEXT NOT NULL,
240         cpu_avg REAL,
241         cpu_min REAL,
242         cpu_max REAL,
243         cpu_p95 REAL,
244         ram_avg_gb REAL,
245         ram_min_gb REAL,
246         ram_max_gb REAL,
247         ram_percent_avg REAL,
248         ram_percent_max REAL,
249         disk_read_avg_mbs REAL,
250         disk_read_max_mbs REAL,
251         disk_read_total_mb REAL,
252         disk_write_avg_mbs REAL,
253         disk_write_max_mbs REAL,
254         disk_write_total_mb REAL,
255         network_rx_avg_mbs REAL,

```

```

256         network_rx_max_mbs REAL,
257         network_rx_total_mb REAL,
258         network_tx_avg_mbs REAL,
259         network_tx_max_mbs REAL,
260         network_tx_total_mb REAL,
261         raw_json TEXT
262     );
263 ''')
264
265 # 9. Аудит оборудования
266 cursor.execute('''
267     CREATE TABLE IF NOT EXISTS hardware_audits (
268         id INTEGER PRIMARY KEY AUTOINCREMENT,
269         archive_id TEXT UNIQUE NOT NULL,
270         timestamp TEXT NOT NULL,
271         created_at REAL NOT NULL,
272         devices_count INTEGER DEFAULT 0,
273         problem_devices_count INTEGER DEFAULT 0,
274         outdated_drivers_count INTEGER DEFAULT 0,
275         changes_count INTEGER DEFAULT 0,
276         raw_json TEXT
277     );
278 ''')
279
280 # 10. История перезагрузок
281 cursor.execute('''
282     CREATE TABLE IF NOT EXISTS reboot_history (
283         id INTEGER PRIMARY KEY AUTOINCREMENT,
284         boot_id TEXT UNIQUE NOT NULL,
285         boot_time TEXT NOT NULL,
286         previous_boot_time TEXT,
287         uptime_seconds REAL,
288         uptime_human TEXT,
289         shutdown_type TEXT NOT NULL,
290         likely_class TEXT,
291         conclusion TEXT,
292         initiating_process TEXT,
293         initiating_user TEXT,

```



```

294         shutdown_action TEXT,
295         reason_text TEXT,
296         reason_code INTEGER,
297         bugcheck_code TEXT,
298         bugcheck_params_json TEXT,
299         power_button_timestamp TEXT,
300         windows_update_kb TEXT,
301         service_installed TEXT,
302         evidence_json TEXT,
303         events_chain_json TEXT,
304         raw_json TEXT,
305         created_at REAL NOT NULL
306     );
307 '''
308
309 # 11. Инциденты аномалий
310 cursor.execute('''
311     CREATE TABLE IF NOT EXISTS incidents (
312         id INTEGER PRIMARY KEY AUTOINCREMENT,
313         incident_id TEXT UNIQUE NOT NULL,
314         timestamp TEXT NOT NULL,
315         created_at REAL NOT NULL,
316         trigger_type TEXT NOT NULL,
317         severity TEXT DEFAULT 'warning',
318         title TEXT NOT NULL,
319         description TEXT,
320         trigger_metrics_json TEXT,
321         suspect_processes_json TEXT,
322         related_events_json TEXT,
323         metrics_summary_json TEXT,
324         raw_window_json TEXT
325     );
326 '''
327
328 # 12. События телеметрии
329 cursor.execute('''
330     CREATE TABLE IF NOT EXISTS telemetry_events (
331         id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

332         timestamp TEXT NOT NULL,
333         created_at REAL NOT NULL,
334         event_type TEXT NOT NULL,
335         severity TEXT DEFAULT 'info',
336         event_details TEXT,
337         raw_json TEXT
338     );
339     '''
340
341 # 13. Опросы приложений
342 cursor.execute('''
343     CREATE TABLE IF NOT EXISTS app_polls (
344         id INTEGER PRIMARY KEY AUTOINCREMENT,
345         timestamp TEXT NOT NULL,
346         created_at REAL NOT NULL,
347         app TEXT NOT NULL,
348         poll_type TEXT,
349         metric_name TEXT,
350         value REAL,
351         unit TEXT,
352         status TEXT,
353         details TEXT,
354         raw_json TEXT
355     );
356     '''
357
358 # 14. События приложений
359 cursor.execute('''
360     CREATE TABLE IF NOT EXISTS app_events (
361         id INTEGER PRIMARY KEY AUTOINCREMENT,
362         timestamp TEXT NOT NULL,
363         created_at REAL NOT NULL,
364         app TEXT NOT NULL,
365         event_type TEXT NOT NULL,
366         status TEXT,
367         message TEXT,
368         details TEXT,
369         raw_json TEXT

```

```

370         );
371     '')
372
373     # 15. Изменения параметров приложений
374     cursor.execute('''
375         CREATE TABLE IF NOT EXISTS app_param_changes (
376             id INTEGER PRIMARY KEY AUTOINCREMENT,
377             timestamp TEXT NOT NULL,
378             created_at REAL NOT NULL,
379             app TEXT NOT NULL,
380             param_name TEXT NOT NULL,
381             old_value TEXT,
382             new_value TEXT,
383             status TEXT,
384             user TEXT,
385             details TEXT,
386             raw_json TEXT
387         );
388     '')
389
390     # 16. Пользовательские записи
391     cursor.execute('''
392         CREATE TABLE IF NOT EXISTS custom_records (
393             id INTEGER PRIMARY KEY AUTOINCREMENT,
394             timestamp TEXT NOT NULL,
395             created_at REAL NOT NULL,
396             source_file TEXT,
397             payload_json TEXT
398         );
399     '')
400
401     # 17. События устройств (PnP / Device Events)
402     cursor.execute('''
403         CREATE TABLE IF NOT EXISTS device_events (
404             id INTEGER PRIMARY KEY AUTOINCREMENT,
405             timestamp TEXT NOT NULL,
406             created_at REAL NOT NULL,
407             event_type TEXT NOT NULL,

```

```

408         device_instance_id TEXT,
409         friendly_name TEXT,
410         device_class TEXT,
411         category TEXT,
412         has_problem INTEGER DEFAULT 0,
413         problem_code INTEGER DEFAULT 0,
414         status_code INTEGER DEFAULT 0,
415         manufacturer TEXT,
416         flapping_count INTEGER DEFAULT 0,
417         uptime_seconds REAL DEFAULT 0.0,
418         raw_json TEXT
419     );
420 '''
421
422 # 18. Инвентарь устройств
423 cursor.execute('''
424     CREATE TABLE IF NOT EXISTS device_inventory (
425         id INTEGER PRIMARY KEY AUTOINCREMENT,
426         device_instance_id TEXT UNIQUE NOT NULL,
427         friendly_name TEXT,
428         device_class TEXT,
429         install_date TEXT NOT NULL,
430         created_at REAL NOT NULL,
431         updated_at REAL NOT NULL
432     );
433 '''
434
435 # 19. Системные события W64/ETW
436 cursor.execute('''
437     CREATE TABLE IF NOT EXISTS w64_events (
438         id INTEGER PRIMARY KEY AUTOINCREMENT,
439         event_id INTEGER,
440         timestamp TEXT NOT NULL,
441         created_at REAL NOT NULL,
442         event_type TEXT,
443         path TEXT,
444         pid INTEGER,
445         name TEXT,

```

```

446         provider TEXT,
447         raw_json TEXT
448     );
449     '''
450
451 # 20. Аномальные процессы
452 cursor.execute('''
453     CREATE TABLE IF NOT EXISTS process_outliers (
454         id INTEGER PRIMARY KEY AUTOINCREMENT,
455         snapshot_id INTEGER,
456         timestamp TEXT NOT NULL,
457         created_at REAL NOT NULL,
458         pid INTEGER,
459         name TEXT,
460         status TEXT,
461         cpu_percent REAL,
462         memory_mb REAL,
463         memory_percent REAL,
464         num_threads INTEGER,
465         username TEXT,
466         details TEXT
467     );
468     ''')
469
470 # 21. Расширенный системный аудит
471 cursor.execute('''
472     CREATE TABLE IF NOT EXISTS system_extended_audits (
473         id INTEGER PRIMARY KEY AUTOINCREMENT,
474         timestamp TEXT NOT NULL,
475         created_at REAL NOT NULL,
476         hostname TEXT,
477         defender_cfa_enabled INTEGER DEFAULT 0,
478         defender_asr_count INTEGER DEFAULT 0,
479         defender_exclusions_count INTEGER DEFAULT 0,
480         defender_threats_count INTEGER DEFAULT 0,
481         startup_entries_count INTEGER DEFAULT 0,
482         vss_snapshots_count INTEGER DEFAULT 0,
483         users_total_count INTEGER DEFAULT 0,

```

```

484         users_admin_count INTEGER DEFAULT 0,
485         raw_json TEXT
486     );
487     '')
488
489 # 22. Почасовые агрегаты сенсоров (telemetry_hourly)
490 cursor.execute('''
491     CREATE TABLE IF NOT EXISTS telemetry_hourly (
492         sensor_id TEXT NOT NULL,
493         bucket_start INTEGER NOT NULL,
494         bucket_end INTEGER NOT NULL,
495         sample_count INTEGER NOT NULL,
496         value_min REAL,
497         value_max REAL,
498         value_avg REAL,
499         value_sum REAL,
500         value_sum_sq REAL,
501         value_stddev REAL,
502         value_first REAL,
503         value_last REAL,
504         PRIMARY KEY (sensor_id, bucket_start)
505     );
506     '')
507
508 # 23. Суточные агрегаты сенсоров (telemetry_daily)
509 cursor.execute('''
510     CREATE TABLE IF NOT EXISTS telemetry_daily (
511         sensor_id TEXT NOT NULL,
512         bucket_start INTEGER NOT NULL,
513         bucket_end INTEGER NOT NULL,
514         sample_count INTEGER NOT NULL,
515         value_min REAL,
516         value_max REAL,
517         value_avg REAL,
518         value_sum REAL,
519         value_sum_sq REAL,
520         value_stddev REAL,
521         value_first REAL,

```

```

522         value_last REAL,
523         PRIMARY KEY (sensor_id, bucket_start)
524     );
525     '')
526
527 # 24. Недельные агрегаты сенсоров (telemetry_weekly)
528 cursor.execute('''
529     CREATE TABLE IF NOT EXISTS telemetry_weekly (
530         sensor_id TEXT NOT NULL,
531         bucket_start INTEGER NOT NULL,
532         bucket_end INTEGER NOT NULL,
533         sample_count INTEGER NOT NULL,
534         value_min REAL,
535         value_max REAL,
536         value_avg REAL,
537         value_sum REAL,
538         value_sum_sq REAL,
539         value_stddev REAL,
540         value_first REAL,
541         value_last REAL,
542         PRIMARY KEY (sensor_id, bucket_start)
543     );
544     ''')
545
546 # 25. Месячные агрегаты сенсоров (telemetry_monthly)
547 cursor.execute('''
548     CREATE TABLE IF NOT EXISTS telemetry_monthly (
549         sensor_id TEXT NOT NULL,
550         bucket_start INTEGER NOT NULL,
551         bucket_end INTEGER NOT NULL,
552         sample_count INTEGER NOT NULL,
553         value_min REAL,
554         value_max REAL,
555         value_avg REAL,
556         value_sum REAL,
557         value_sum_sq REAL,
558         value_stddev REAL,
559         value_first REAL,

```

```

560         value_last REAL,
561         PRIMARY KEY (sensor_id, bucket_start)
562     );
563     '')
564
565 # 26. Годовые агрегаты сенсоров (telemetry_yearly)
566 cursor.execute('''
567     CREATE TABLE IF NOT EXISTS telemetry_yearly (
568         sensor_id TEXT NOT NULL,
569         bucket_start INTEGER NOT NULL,
570         bucket_end INTEGER NOT NULL,
571         sample_count INTEGER NOT NULL,
572         value_min REAL,
573         value_max REAL,
574         value_avg REAL,
575         value_sum REAL,
576         value_sum_sq REAL,
577         value_stddev REAL,
578         value_first REAL,
579         value_last REAL,
580         PRIMARY KEY (sensor_id, bucket_start)
581     );
582     '')
583
584 # 27. Фиксация аномальных всплесков сенсоров (telemetry_sp
ikes)
585 cursor.execute('''
586     CREATE TABLE IF NOT EXISTS telemetry_spikes (
587         id INTEGER PRIMARY KEY AUTOINCREMENT,
588         sensor_id TEXT NOT NULL,
589         timestamp TEXT NOT NULL,
590         created_at REAL NOT NULL,
591         value REAL NOT NULL,
592         spike_type TEXT DEFAULT 'high',
593         baseline_avg REAL,
594         baseline_stddev REAL,
595         details TEXT,
596         raw_json TEXT

```



```

597         );
598     '')
599
600     # 28. События прямого ввода-вывода (disk_io_events / ETW /
        Process I/O)
601     cursor.execute('''
602         CREATE TABLE IF NOT EXISTS disk_io_events (
603             id INTEGER PRIMARY KEY AUTOINCREMENT,
604             timestamp TEXT NOT NULL,
605             created_at REAL NOT NULL,
606             pid INTEGER NOT NULL,
607             process_name TEXT NOT NULL,
608             disk_id INTEGER NOT NULL,
609             operation TEXT NOT NULL,
610             bytes_count INTEGER NOT NULL,
611             duration_ms REAL,
612             offset INTEGER,
613             file_path TEXT
614         );
615     '')
616
617     # 29. Снимки здоровья и износа накопителей (disk_health_sn
        apshots / NVMe SMART / TBW)
618     cursor.execute('''
619         CREATE TABLE IF NOT EXISTS disk_health_snapshots (
620             id INTEGER PRIMARY KEY AUTOINCREMENT,
621             timestamp TEXT NOT NULL,
622             created_at REAL NOT NULL,
623             disk_id INTEGER NOT NULL,
624             serial_number TEXT,
625             model TEXT,
626             bus_type TEXT,
627             temperature_c REAL,
628             wear_percent REAL,
629             available_spare REAL,
630             tbw_written_tb REAL,
631             tbw_read_tb REAL,
632             power_on_hours INTEGER,

```

```

633         power_cycles INTEGER,
634         unsafe_shutdowns INTEGER,
635         media_errors INTEGER,
636         error_log_entries INTEGER,
637         health_status TEXT DEFAULT 'Healthy',
638         raw_smart_json TEXT
639     );
640     ''')
641
642     # 30. Паспорт и конфигурация физических накопителей (disk_
inventory)
643     cursor.execute('''
644         CREATE TABLE IF NOT EXISTS disk_inventory (
645             id INTEGER PRIMARY KEY AUTOINCREMENT,
646             disk_id INTEGER NOT NULL,
647             device_path TEXT NOT NULL,
648             serial_number TEXT UNIQUE,
649             vendor TEXT,
650             product TEXT,
651             revision TEXT,
652             bus_type TEXT,
653             media_type TEXT,
654             size_bytes INTEGER,
655             size_gb REAL,
656             sector_size INTEGER,
657             removable INTEGER DEFAULT 0,
658             partition_style TEXT,
659             gpt_guid TEXT,
660             partitions_json TEXT,
661             first_seen TEXT NOT NULL,
662             last_seen TEXT NOT NULL,
663             updated_at REAL NOT NULL
664         );
665     ''')
666
667     # 31. Паспорт логических томов (volume_inventory)
668     cursor.execute('''
669         CREATE TABLE IF NOT EXISTS volume_inventory (

```

```

670         id INTEGER PRIMARY KEY AUTOINCREMENT,
671         volume_guid TEXT UNIQUE NOT NULL,
672         drive_letter TEXT,
673         mount_point TEXT,
674         label TEXT,
675         filesystem TEXT,
676         filesystem_flags INTEGER,
677         cluster_size_bytes INTEGER,
678         sector_size_bytes INTEGER,
679         total_bytes INTEGER,
680         total_gb REAL,
681         disk_id INTEGER,
682         first_seen TEXT NOT NULL,
683         last_seen TEXT NOT NULL,
684         updated_at REAL NOT NULL
685     );
686 '''
687
688 # 32. Сэмплы производительности дисков (disk_performance_s
amples)
689 cursor.execute('''
690     CREATE TABLE IF NOT EXISTS disk_performance_samples (
691         id INTEGER PRIMARY KEY AUTOINCREMENT,
692         timestamp TEXT NOT NULL,
693         created_at REAL NOT NULL,
694         disk_name TEXT NOT NULL,
695         read_bytes_sec REAL,
696         write_bytes_sec REAL,
697         read_iops REAL,
698         write_iops REAL,
699         read_time_ms REAL,
700         write_time_ms REAL,
701         percent_disk_time REAL,
702         queue_length REAL
703     );
704 '''
705
706 # 33. Паспорт процессоров (cpu_inventory)

```

```

707 cursor.execute('''
708     CREATE TABLE IF NOT EXISTS cpu_inventory (
709         id INTEGER PRIMARY KEY AUTOINCREMENT,
710         processor_id INTEGER NOT NULL,
711         name TEXT UNIQUE NOT NULL,
712         vendor TEXT,
713         architecture TEXT,
714         physical_cores INTEGER,
715         logical_cores INTEGER,
716         base_frequency_mhz REAL,
717         max_frequency_mhz REAL,
718         l2_cache_kb INTEGER,
719         l3_cache_kb INTEGER,
720         socket TEXT,
721         features_json TEXT,
722         first_seen TEXT NOT NULL,
723         last_seen TEXT NOT NULL,
724         updated_at REAL NOT NULL
725     );
726 ''')
727
728 # 34. Паспорт планок памяти (ram_module_inventory)
729 cursor.execute('''
730     CREATE TABLE IF NOT EXISTS ram_module_inventory (
731         id INTEGER PRIMARY KEY AUTOINCREMENT,
732         slot_id INTEGER NOT NULL,
733         bank_label TEXT NOT NULL,
734         device_locator TEXT,
735         serial_number TEXT UNIQUE,
736         part_number TEXT,
737         manufacturer TEXT,
738         capacity_bytes INTEGER,
739         capacity_gb REAL,
740         speed_mhz INTEGER,
741         memory_type TEXT,
742         form_factor TEXT,
743         configured_voltage REAL,
744         first_seen TEXT NOT NULL,

```

```

745         last_seen TEXT NOT NULL,
746         updated_at REAL NOT NULL
747     );
748     '')
749
750 # 35. Паспорт графических ускорителей (gpu_inventory)
751 cursor.execute('''
752     CREATE TABLE IF NOT EXISTS gpu_inventory (
753         id INTEGER PRIMARY KEY AUTOINCREMENT,
754         gpu_id INTEGER NOT NULL,
755         pci_device_id TEXT UNIQUE,
756         name TEXT NOT NULL,
757         vendor TEXT,
758         driver_version TEXT,
759         driver_date TEXT,
760         vram_bytes INTEGER,
761         vram_gb REAL,
762         pci_bus_id TEXT,
763         bios_version TEXT,
764         cuda_cores INTEGER,
765         directml_supported INTEGER DEFAULT 1,
766         first_seen TEXT NOT NULL,
767         last_seen TEXT NOT NULL,
768         updated_at REAL NOT NULL
769     );
770     '')
771
772 # 36. Паспорт сетевых интерфейсов (network_adapter_invento
ry)
773 cursor.execute('''
774     CREATE TABLE IF NOT EXISTS network_adapter_inventory (
775         id INTEGER PRIMARY KEY AUTOINCREMENT,
776         adapter_id INTEGER NOT NULL,
777         adapter_guid TEXT UNIQUE NOT NULL,
778         name TEXT NOT NULL,
779         interface_name TEXT,
780         mac_address TEXT,
781         adapter_type TEXT,

```

```

782         is_physical INTEGER DEFAULT 1,
783         is_wireless INTEGER DEFAULT 0,
784         max_speed_mbps INTEGER,
785         driver_name TEXT,
786         driver_version TEXT,
787         first_seen TEXT NOT NULL,
788         last_seen TEXT NOT NULL,
789         updated_at REAL NOT NULL
790     );
791 '''
792
793 # 37. Сэмплы телеметрии процессора (cpu_telemetry_samples)
794 cursor.execute('''
795     CREATE TABLE IF NOT EXISTS cpu_telemetry_samples (
796         id INTEGER PRIMARY KEY AUTOINCREMENT,
797         timestamp TEXT NOT NULL,
798         created_at REAL NOT NULL,
799         processor_id INTEGER DEFAULT 0,
800         total_percent REAL,
801         user_percent REAL,
802         kernel_percent REAL,
803         frequency_mhz REAL,
804         temperature_c REAL,
805         package_power_w REAL,
806         core_utilization_json TEXT,
807         core_temperatures_json TEXT
808     );
809 '''
810
811 # 38. Сэмплы телеметрии оперативной памяти (ram_telemetry_
812 samples)
813 cursor.execute('''
814     CREATE TABLE IF NOT EXISTS ram_telemetry_samples (
815         id INTEGER PRIMARY KEY AUTOINCREMENT,
816         timestamp TEXT NOT NULL,
817         created_at REAL NOT NULL,
818         total_bytes INTEGER,
819         total_gb REAL,

```

```

819         used_bytes INTEGER,
820         used_gb REAL,
821         available_bytes INTEGER,
822         available_gb REAL,
823         percent_used REAL,
824         swap_total_gb REAL,
825         swap_used_gb REAL,
826         swap_percent REAL,
827         pool_paged_mb REAL,
828         pool_nonpaged_mb REAL
829     );
830 '''
831
832 # 39. Сэмплы телеметрии GPU (gpu_telemetry_samples)
833 cursor.execute('''
834     CREATE TABLE IF NOT EXISTS gpu_telemetry_samples (
835         id INTEGER PRIMARY KEY AUTOINCREMENT,
836         timestamp TEXT NOT NULL,
837         created_at REAL NOT NULL,
838         gpu_id INTEGER DEFAULT 0,
839         name TEXT NOT NULL,
840         load_percent REAL,
841         memory_used_mb REAL,
842         memory_total_mb REAL,
843         memory_percent REAL,
844         temperature_gpu_c REAL,
845         temperature_memory_c REAL,
846         fan_speed_pct REAL,
847         power_draw_w REAL,
848         clock_graphics_mhz REAL,
849         clock_memory_mhz REAL
850     );
851 '''
852
853 # 40. Сэмплы сетевых интерфейсов (network_adapter_samples)
854 cursor.execute('''
855     CREATE TABLE IF NOT EXISTS network_adapter_samples (
856         id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

857         timestamp TEXT NOT NULL,
858         created_at REAL NOT NULL,
859         adapter_name TEXT NOT NULL,
860         bytes_recv_sec REAL,
861         bytes_sent_sec REAL,
862         packets_recv_sec REAL,
863         packets_sent_sec REAL,
864         errors_in_sec REAL,
865         errors_out_sec REAL,
866         link_speed_mbps INTEGER,
867         is_connected INTEGER DEFAULT 1
868     );
869     ''')
870
871     # Индексы для ускорения выборок
872     cursor.execute('CREATE INDEX IF NOT EXISTS idx_snapshots_created_at ON system_snapshots(created_at);')
873     cursor.execute('CREATE INDEX IF NOT EXISTS idx_snapshots_timestamp ON system_snapshots(timestamp);')
874     cursor.execute('CREATE INDEX IF NOT EXISTS idx_sensor_polls_sensor_time ON sensor_polls(sensor_id, created_at);')
875     cursor.execute('CREATE INDEX IF NOT EXISTS idx_sensor_polls_category ON sensor_polls(sensor_category, created_at);')
876     cursor.execute('CREATE INDEX IF NOT EXISTS idx_processes_snapshot_id ON process_snapshots(snapshot_id);')
877     cursor.execute('CREATE INDEX IF NOT EXISTS idx_processes_name_pid ON process_snapshots(name, pid);')
878     cursor.execute('CREATE INDEX IF NOT EXISTS idx_aggregates_sensor_time ON sensor_aggregates_hourly(sensor_id, period_start);')
879     cursor.execute('CREATE INDEX IF NOT EXISTS idx_rollups_daily_date ON process_rollups_daily(date, name);')
880     cursor.execute('CREATE INDEX IF NOT EXISTS idx_hardware_audits_archive_id ON hardware_audits(archive_id);')
881     cursor.execute('CREATE INDEX IF NOT EXISTS idx_reboot_history_boot_time ON reboot_history(boot_time);')
882     cursor.execute('CREATE INDEX IF NOT EXISTS idx_incidents_created_at ON incidents(created_at);')

```



```
883 cursor.execute('CREATE INDEX IF NOT EXISTS idx_incidents_t  
rigger ON incidents(trigger_type);')  
884 cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_prov_g  
uid ON process_provenance_events(process_guid);')  
885 cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_prov_c  
reated_at ON process_provenance_events(created_at);')  
886 cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_prov_n  
ame_pid ON process_provenance_events(name, pid);')  
887 cursor.execute('CREATE INDEX IF NOT EXISTS idx_device_even  
ts_created_at ON device_events(created_at);')  
888 cursor.execute('CREATE INDEX IF NOT EXISTS idx_device_inve  
ntory_dev_id ON device_inventory(device_instance_id);')  
889 cursor.execute('CREATE INDEX IF NOT EXISTS idx_w64_events_  
created_at ON w64_events(created_at);')  
890 cursor.execute('CREATE INDEX IF NOT EXISTS idx_process_out  
liers_created_at ON process_outliers(created_at);')  
891 cursor.execute('CREATE INDEX IF NOT EXISTS idx_extended_au  
dits_created_at ON system_extended_audits(created_at);')  
892 cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_h  
ourly_time ON telemetry_hourly(bucket_start);')  
893 cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_d  
aily_time ON telemetry_daily(bucket_start);')  
894 cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_w  
eekly_time ON telemetry_weekly(bucket_start);')  
895 cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_m  
onthly_time ON telemetry_monthly(bucket_start);')  
896 cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_y  
early_time ON telemetry_yearly(bucket_start);')  
897 cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_s  
pikes_time ON telemetry_spikes(created_at, sensor_id);')  
898 cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_io_pid  
_time ON disk_io_events(pid, created_at);')  
899 cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_io_dis  
k_time ON disk_io_events(disk_id, created_at);')  
900 cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_health  
_disk_time ON disk_health_snapshots(disk_id, created_at);')  
901 cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_health  
_serial_time ON disk_health_snapshots(serial_number, created_a
```

```

t);')
902     cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_inv_se
rial ON disk_inventory(serial_number);')
903     cursor.execute('CREATE INDEX IF NOT EXISTS idx_volume_inv_
guid ON volume_inventory(volume_guid);')
904     cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_perf_n
ame_time ON disk_performance_samples(disk_name, created_at);')
905     cursor.execute('CREATE INDEX IF NOT EXISTS idx_cpu_inv_nam
e ON cpu_inventory(name);')
906     cursor.execute('CREATE INDEX IF NOT EXISTS idx_ram_mod_ser
ial ON ram_module_inventory(serial_number);')
907     cursor.execute('CREATE INDEX IF NOT EXISTS idx_gpu_inv_pci
ON gpu_inventory(pci_device_id);')
908     cursor.execute('CREATE INDEX IF NOT EXISTS idx_net_inv_gui
d ON network_adapter_inventory(adapter_guid);')
909     cursor.execute('CREATE INDEX IF NOT EXISTS idx_cpu_samples
_time ON cpu_telemetry_samples(created_at);')
910     cursor.execute('CREATE INDEX IF NOT EXISTS idx_ram_samples
_time ON ram_telemetry_samples(created_at);')
911     cursor.execute('CREATE INDEX IF NOT EXISTS idx_gpu_samples
_time ON gpu_telemetry_samples(created_at);')
912     cursor.execute('CREATE INDEX IF NOT EXISTS idx_net_samples
_time ON network_adapter_samples(created_at);')
913
914     _run_migrations(cursor)
915     conn.commit()
916
917
918 def _run_migrations(cursor: sqlite3.Cursor) -> None:
919     """Выполняет безопасные миграции добавления новых колонок
в существующие таблицы."""
920     # Миграция: telemetry_events
921     try:
922         cursor.execute("PRAGMA table_info(telemetry_events)")
923         columns = [row[1] for row in cursor.fetchall()]
924         if 'severity' not in columns:
925             cursor.execute("ALTER TABLE telemetry_events ADD C
OLUMN severity TEXT DEFAULT 'info'")

```

```
926         if 'event_details' not in columns:
927             cursor.execute("ALTER TABLE telemetry_events ADD C
COLUMN event_details TEXT")
928         if 'raw_json' not in columns:
929             cursor.execute("ALTER TABLE telemetry_events ADD C
COLUMN raw_json TEXT")
930     except Exception as exc:
931         logger.warning(f'Миграция telemetry_events: {exc}')
932
933     # Миграция: system_snapshots
934     try:
935         cursor.execute("PRAGMA table_info(system_snapshots)")
936         snap_cols = {row[1] for row in cursor.fetchall()}
937         expected_snap_cols = {
938             'hostname': 'TEXT',
939             'username': 'TEXT',
940             'uptime_seconds': 'REAL',
941             'os_name': 'TEXT',
942             'os_build': 'TEXT',
943             'os_install_date': 'TEXT',
944             'system_language': 'TEXT',
945             'os_install_language': 'TEXT',
946             'user_locale': 'TEXT',
947             'system_locale': 'TEXT',
948             'timezone': 'TEXT',
949             'codepage': 'TEXT',
950             'input_languages_json': 'TEXT',
951             'disks_json': 'TEXT',
952             'physical_disks_json': 'TEXT',
953             'monitors_json': 'TEXT',
954             'updates_json': 'TEXT',
955             'office_json': 'TEXT',
956             'onedrive_json': 'TEXT',
957             'battery_json': 'TEXT',
958             'ram_sticks_json': 'TEXT',
959             'listening_ports_json': 'TEXT',
960             'alerts_json': 'TEXT',
961             'cpu_total_percent': 'REAL',
```

```

962         'cpu_frequency_mhz': 'REAL',
963         'memory_total_gb': 'REAL',
964         'memory_used_gb': 'REAL',
965         'memory_percent': 'REAL',
966         'swap_percent': 'REAL',
967         'gpu_load_percent': 'REAL',
968         'gpu_temp_c': 'REAL',
969         'disk_read_bytes_sec': 'REAL',
970         'disk_write_bytes_sec': 'REAL',
971         'disk_read_count_sec': 'REAL',
972         'disk_write_count_sec': 'REAL',
973         'network_sent_bytes_sec': 'REAL',
974         'network_recv_bytes_sec': 'REAL',
975         'hardware_audit': 'TEXT',
976         'raw_json': 'TEXT',
977     }
978     for col_name, col_type in expected_snap_cols.items():
979         if col_name not in snap_cols:
980             cursor.execute(f"ALTER TABLE system_snapshots
ADD COLUMN {col_name} {col_type}")
981     except Exception as exc:
982         logger.warning(f'Миграция system_snapshots: {exc}')
983
984     # Миграция: process_snapshots
985     try:
986         cursor.execute("PRAGMA table_info(process_snapshots)")
987         proc_cols = {row[1] for row in cursor.fetchall()}
988         new_proc_columns = {
989             'num_handles': 'INTEGER DEFAULT 0',
990             'read_bytes_sec': 'REAL',
991             'write_bytes_sec': 'REAL',
992             'integrity_level': 'TEXT',
993             'elevation': 'INTEGER DEFAULT 0',
994             'ppid': 'INTEGER',
995             'parent_name': 'TEXT',
996             'executable': 'TEXT',
997             'cmdline': 'TEXT',
998             'sid': 'TEXT',

```

```
999         'session_id': 'INTEGER',
1000         'creation_time': 'TEXT',
1001         'process_guid': 'TEXT',
1002         'ancestor_chain': 'TEXT',
1003         'launch_reason': 'TEXT',
1004     }
1005     for col_name, col_def in new_proc_columns.items():
1006         if col_name not in proc_cols:
1007             cursor.execute(f"ALTER TABLE process_snapshots
1008 ADD COLUMN {col_name} {col_def}")
1009     except Exception as exc:
1010         logger.warning(f'Миграция process_snapshots: {exc}')
```

storage.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Storage
4 # =====
5 # Description:
6 #   Главный фасад TelemetryStorage базы данных SQLite телеметр
  ии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite import TelemetryStora
    ge
11 #
12 #       storage = TelemetryStorage.get_instance(read_only=True)
13 #       snapshots = storage.get_snapshots(limit=10)
14 #
15 # File: storage.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 04:25:00
21 # =====
22
23 from __future__ import annotations
24
25 """Главный класс TelemetryStorage, объединяющий подсистемы чте
  ния, записи, буферизации и обслуживания."""
26
27 import atexit
28 import os
29 import sqlite3
```

```

30 import threading
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional, Union
33
34 from .aggregator import AggregationLevel, TelemetrySqlAggregat
    or
35 from .buffer import TelemetryBuffer
36 from .connection import TelemetryConnectionManager
37 from .maintenance import TelemetryMaintenance
38 from .reader import TelemetryReader
39 from .schema import init_database_schema
40 from .writer import TelemetryWriter
41 from ..models import (
42     HardwareArchiveEntry,
43     ProcessLifecycleEvent,
44     ProcessProvenanceInfo,
45     SystemMetricRollup,
46     SystemSnapshot,
47     TelemetryIncident,
48 )
49 from ..telemetry_config import TelemetryConfigManager
50
51 try:
52     from logger import logger
53 except ImportError:
54     import logging
55     logger = logging.getLogger(__name__)
56
57
58 class TelemetryStorage:
59     """Универсальный фасад персистентного хранения и выборки т
        елеметрии в SQLite."""
60
61     _instance: Optional[TelemetryStorage] = None
62     _ro_instance: Optional[TelemetryStorage] = None
63     _lock = threading.RLock()
64
65     def __init__(

```

```

66         self,
67         db_path: Optional[Union[str, Path]] = None,
68         buffer_mode: Optional[str] = None,
69         buffer_size: Optional[int] = None,
70         flush_interval_seconds: Optional[float] = None,
71         buffer_file_path: Optional[Union[str, Path]] = None,
72         auto_flush: bool = True,
73         max_db_size_mb: Optional[float] = None,
74         retention_days: Optional[int] = None,
75         auto_vacuum: Optional[bool] = None,
76         read_only: bool = False,
77     ) -> None:
78         """Инициализирует подключение к SQLite базе данных тел
79         еметрии."""
80
81         self.read_only = bool(read_only)
82
83         # 1. Определение путей базы данных
84         if db_path is None:
85             appdata = os.environ.get('APPDATA') or os.environ.
86             get('LOCALAPPDATA')
87             base_dir = Path(appdata) if appdata and os.path.ex
88             ists(appdata) else (Path.home() / '.config')
89             target_dir = base_dir / 'AI-Breadboard' / 'apps' /
90             'windows' / 'telemetry' / 'logs'
91             if not self.read_only:
92                 target_dir.mkdir(parents=True, exist_ok=True)
93                 self.db_path = target_dir / 'telemetry.db'
94             else:
95                 self.db_path = Path(db_path)
96                 if not self.read_only:
97                     self.db_path.parent.mkdir(parents=True, exist_
98                     ok=True)
99
100         # 2. Конфигурация
101         cfg_manager = None
102         try:
103             cfg_manager = TelemetryConfigManager()
104         except Exception:

```



```

99         cfg_manager = None
100
101         raw_mode = buffer_mode or (cfg_manager.get_buffer_mode
102         () if cfg_manager else 'memory')
103         b_mode = raw_mode.lower() if raw_mode in ('memory', 'f
104         ile', 'direct') else 'memory'
105         b_size = int(buffer_size if buffer_size is not None el
106         se (cfg_manager.get_buffer_size() if cfg_manager else 50))
107         f_interval = float(flush_interval_seconds if flush_int
108         erval_seconds is not None else (cfg_manager.get_flush_interval
109         _seconds() if cfg_manager else 30.0))
110         m_size = float(max_db_size_mb if max_db_size_mb is not
111         None else (cfg_manager.get_max_db_size_mb() if cfg_manager el
112         se 50.0))
113         r_days = int(retention_days if retention_days is not N
114         one else (cfg_manager.get_retention_days() if cfg_manager else
115         7))
116         a_vacuum = bool(auto_vacuum if auto_vacuum is not None
117         else (cfg_manager.is_auto_vacuum_enabled() if cfg_manager els
118         e True))
119
120         # 3. Инициализация менеджера подключений
121         self._cm = TelemetryConnectionManager(db_path=self.db_
122         path, read_only=self.read_only, lock=self._lock)
123
124         # 4. Инициализация схемы (только в режиме записи)
125         if not self.read_only:
126             try:
127                 with self._cm.lock, self._cm.get_connection()
128                 as conn:
129                     init_database_schema(conn)
130             except Exception as schema_err:
131                 logger.warning(f'Ошибка инициализации схемы ба
132                 зы данных: {schema_err}')
133
134         # 5. Инициализация подсистем
135         self._reader = TelemetryReader(self._cm)
136         self._writer = TelemetryWriter(self._cm)

```

```

123         self._buffer = TelemetryBuffer(
124             connection_manager=self._cm,
125             writer=self._writer,
126             buffer_mode=b_mode,
127             buffer_size=b_size,
128             flush_interval_seconds=f_interval,
129             buffer_file_path=buffer_file_path,
130             auto_flush=auto_flush,
131         )
132         self._maintenance = TelemetryMaintenance(
133             connection_manager=self._cm,
134             max_db_size_mb=m_size,
135             retention_days=r_days,
136             auto_vacuum_enabled=a_vacuum,
137             buffer_file_path=self._buffer.buffer_file_path,
138         )
139         self._aggregator = TelemetrySqlAggregator(self._cm)
140
141         if not self.read_only:
142             try:
143                 atexit.register(self.close)
144             except Exception:
145                 pass
146
147     @classmethod
148     def get_instance(
149         cls,
150         db_path: Optional[Union[str, Path]] = None,
151         read_only: bool = False,
152     ) -> TelemetryStorage:
153         """Возвращает синглтон-экземпляр хранилища телеметрии
154         (для записи или только для чтения)."""
155         with cls._lock:
156             if read_only:
157                 if cls._ro_instance is None or (db_path is not
158                     None and cls._ro_instance.db_path != Path(db_path)):
159                     cls._ro_instance = TelemetryStorage(db_path=
160                         db_path, read_only=True, auto_flush=False)

```

```
158         return cls._ro_instance
159     else:
160         if cls._instance is None or (db_path is not None and cls._instance.db_path != Path(db_path)):
161             cls._instance = TelemetryStorage(db_path=db_path, read_only=False)
162         return cls._instance
163
164     def _get_connection(self) -> sqlite3.Connection:
165         """Создает и настраивает соединение с базой данных SQLite."""
166         return self._cm.get_connection()
167
168     @property
169     def reader(self) -> TelemetryReader:
170         """Подсистема чтения данных."""
171         return self._reader
172
173     @property
174     def writer(self) -> TelemetryWriter:
175         """Подсистема записи данных."""
176         return self._writer
177
178     @property
179     def buffer(self) -> TelemetryBuffer:
180         """Подсистема буферизации."""
181         return self._buffer
182
183     @property
184     def maintenance(self) -> TelemetryMaintenance:
185         """Подсистема регламентного обслуживания."""
186         return self._maintenance
187
188     @property
189     def aggregator(self) -> TelemetrySqlAggregator:
190         """Подсистема многоуровневой SQL-агрегации."""
191         return self._aggregator
192
```

```

193 |     # -----
194 |     # Делегирование методов буфера
195 |     # -----
196 |
197 |     def set_buffer_mode(self, mode: str) -> None:
198 |         self._buffer.set_buffer_mode(mode)
199 |
200 |     def get_buffer_mode(self) -> str:
201 |         return self._buffer.buffer_mode
202 |
203 |     def set_flush_interval_seconds(self, interval: float) -> N
one:
204 |         """Обновляет интервал таймера сброса буфера в SQLite."""
205 |         self._buffer.set_flush_interval_seconds(interval)
206 |
207 |     def get_buffered_count(self) -> int:
208 |         return self._buffer.get_buffered_count()
209 |
210 |     def flush(self) -> int:
211 |         return self._buffer.flush()
212 |
213 |     def close(self) -> None:
214 |         self._buffer.close()
215 |
216 |     # -----
217 |     # Делегирование методов сохранения (Writer / Buffer)
218 |     # -----
219 |
220 |     def save_snapshot(self, snapshot: SystemSnapshot, top_n: i
nt = 20) -> int:
221 |         if self._buffer.buffer_mode == 'direct':
222 |             with self._cm.lock, self._cm.get_connection() as c
onn:

```

```

223         snap_id = self._writer.insert_snapshot_row(con
n.cursor(), {'data': snapshot, 'top_n': top_n})
224         conn.commit()
225         return snap_id
226         snap_dict = snapshot if isinstance(snapshot, dict) els
e (snapshot.model_dump() if hasattr(snapshot, 'model_dump') el
se (vars(snapshot) if hasattr(snapshot, '__dict__') else snaps
hot))
227         self._buffer.enqueue({'type': 'snapshot', 'data': snap
_dict, 'top_n': top_n})
228         return 1
229
230     def save_sensor_poll(self, sensor_item: Dict[str, Any], ti
mestamp: Optional[str] = None) -> int:
231         if self._buffer.buffer_mode == 'direct':
232             with self._cm.lock, self._cm.get_connection() as c
onn:
233                 row = self._writer.prepare_sensor_poll_row(sen
sor_item, timestamp)
234                 conn.cursor().execute('''
235                     INSERT INTO sensor_polls (
236                         sensor_id, timestamp, created_at, hard
ware_name, hardware_type,
237                         sensor_category, sensor_name, unit, va
lue, provider, provider_priority, raw_json
238                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?)
239                     ON CONFLICT(sensor_id, timestamp) DO UPDAT
E SET
240                         created_at = excluded.created_at, hard
ware_name = excluded.hardware_name,
241                         hardware_type = excluded.hardware_type
, sensor_category = excluded.sensor_category,
242                         sensor_name = excluded.sensor_name, un
it = excluded.unit, value = excluded.value,
243                         provider = excluded.provider, provider
_priority = excluded.provider_priority,
244                         raw_json = excluded.raw_json

```

```

245         '', row)
246         conn.commit()
247         return 1
248         self._buffer.enqueue({'type': 'sensor_poll', 'data': s
sensor_item, 'timestamp': timestamp})
249         return 1
250
251     def save_sensor_polls_batch(self, items: List[Dict[str, An
y]], timestamp: Optional[str] = None, deduplicate: bool = True
) -> int:
252         if not items:
253             return 0
254         if deduplicate:
255             from ..sensor_registry import SensorDeduplicator,
SensorProvider
256             deduplicator = SensorDeduplicator()
257             for item in items:
258                 sensor_id = str(item.get('id', ''))
259                 value = item.get('value', 0.0)
260                 provider_raw = item.get('_provider', SensorPro
vider.SENSOR_COLLECTOR)
261                 if isinstance(provider_raw, str):
262                     try:
263                         provider = SensorProvider[provider_raw
.upper()]
264                     except KeyError:
265                         provider = SensorProvider.SENSOR_COLLE
CTOR
266                 elif isinstance(provider_raw, SensorProvider):
267                     provider = provider_raw
268                 else:
269                     provider = SensorProvider.SENSOR_COLLECTOR
270                 deduplicator.add(sensor_id, provider, value)
271                 unique_providers = deduplicator.get_unique()
272                 items = [it for it in items if str(it.get('id', ''
)) in unique_providers]
273
274         if not items:

```

```

275         return 0
276         if self._buffer.buffer_mode == 'direct':
277             return self._writer.batch_insert_records([{'type':
| 'sensor_polls_batch', 'data': items, 'timestamp': timestamp}
| ])
278             self._buffer.enqueue({'type': 'sensor_polls_batch', 'd
| ata': items, 'timestamp': timestamp})
279             return len(items)
280
281         save_sensor_polls = save_sensor_polls_batch
282
283         def save_sensor_aggregates_batch(self, aggregates: List[Di
| ct[str, Any]]) -> int:
284             if not aggregates:
285                 return 0
286             if self._buffer.buffer_mode == 'direct':
287                 return self._writer.batch_insert_records([{'type':
| 'sensor_aggregates_batch', 'data': aggregates}])
288                 self._buffer.enqueue({'type': 'sensor_aggregates_batch
| ', 'data': aggregates})
289                 return len(aggregates)
290
291         def save_event(self, event_type: str, event_details: Dict[
| str, Any], severity: str = 'info', timestamp: Optional[str] =
| None) -> int:
292             rec = {'type': 'event', 'event_type': event_type, 'det
| ails': event_details, 'severity': severity, 'timestamp': times
| tamp}
293             if self._buffer.buffer_mode == 'direct':
294                 return self._writer.batch_insert_records([rec])
295             self._buffer.enqueue(rec)
296             return 1
297
298         def save_hardware_archive(self, archive_entry: HardwareArc
| hiveEntry) -> int:
299             entry_dict = archive_entry if isinstance(archive_entry
| , dict) else (archive_entry.model_dump() if hasattr(archive_en
| try, 'model_dump') else (vars(archive_entry) if hasattr(archiv

```

```

    e_entry, '__dict__') else archive_entry))
300         if self._buffer.buffer_mode == 'direct':
301             return self._writer.batch_insert_records([{'type':
    'hardware_archive', 'data': entry_dict}])
302         self._buffer.enqueue({'type': 'hardware_archive', 'dat
    a': entry_dict})
303         return 1
304
305     def save_app_poll(self, app: str, poll_type: str, metric_n
    ame: str, value: Any, unit: str = '', status: str = 'OK', deta
    ils: Any = '', timestamp: Optional[str] = None) -> int:
306         rec = {'type': 'app_poll', 'app': app, 'poll_type': po
    ll_type, 'metric_name': metric_name, 'value': value, 'unit': u
    nit, 'status': status, 'details': details, 'timestamp': timest
    amp}
307         if self._buffer.buffer_mode == 'direct':
308             return self._writer.batch_insert_records([rec])
309         self._buffer.enqueue(rec)
310         return 1
311
312     def save_app_event(self, app: str, event_type: str, status
    : str = 'OK', details: Any = '', timestamp: Optional[str] = No
    ne) -> int:
313         rec = {'type': 'app_event', 'app': app, 'event_type':
    event_type, 'status': status, 'details': details, 'timestamp':
    timestamp}
314         if self._buffer.buffer_mode == 'direct':
315             return self._writer.batch_insert_records([rec])
316         self._buffer.enqueue(rec)
317         return 1
318
319     def save_app_param_change(self, app: str, param_name: str,
    old_value: Any, new_value: Any, status: str = 'SUCCESS', user
    : str = 'system', details: Any = '', timestamp: Optional[str]
    = None) -> int:
320         rec = {'type': 'app_param_change', 'app': app, 'param_
    name': param_name, 'old_value': old_value, 'new_value': new_va
    lue, 'status': status, 'user': user, 'details': details, 'time

```



```

    stamp': timestamp}
321         if self._buffer.buffer_mode == 'direct':
322             return self._writer.batch_insert_records([rec])
323         self._buffer.enqueue(rec)
324         return 1
325
326     def save_app_polls_batch(self, polls: List[Dict[str, Any]]
    ) -> int:
327         if not polls:
328             return 0
329         if self._buffer.buffer_mode == 'direct':
330             return self._writer.batch_insert_records([{'type':
    'app_polls_batch', 'data': polls}])
331         self._buffer.enqueue({'type': 'app_polls_batch', 'data
    ': polls})
332         return len(polls)
333
334     def save_app_events_batch(self, events: List[Dict[str, Any
    ]]) -> int:
335         if not events:
336             return 0
337         if self._buffer.buffer_mode == 'direct':
338             return self._writer.batch_insert_records([{'type':
    'app_events_batch', 'data': events}])
339         self._buffer.enqueue({'type': 'app_events_batch', 'dat
    a': events})
340         return len(events)
341
342     def save_app_param_changes_batch(self, param_changes: List
    [Dict[str, Any]]) -> int:
343         if not param_changes:
344             return 0
345         if self._buffer.buffer_mode == 'direct':
346             return self._writer.batch_insert_records([{'type':
    'app_param_changes_batch', 'data': param_changes}])
347         self._buffer.enqueue({'type': 'app_param_changes_batch
    ', 'data': param_changes})
348         return len(param_changes)

```

```

349
350     def save_custom_record(self, source_file: str, payload: Dict[str, Any], timestamp: Optional[str] = None) -> int:
351         rec = {'type': 'custom_record', 'source_file': source_file, 'payload': payload, 'timestamp': timestamp}
352         if self._buffer.buffer_mode == 'direct':
353             return self._writer.batch_insert_records([rec])
354         self._buffer.enqueue(rec)
355         return 1
356
357     def save_device_event(self, event: Dict[str, Any], timestamp: Optional[str] = None) -> int:
358         rec = {'type': 'device_event', 'data': event, 'timestamp': timestamp}
359         if self._buffer.buffer_mode == 'direct':
360             return self._writer.batch_insert_records([rec])
361         self._buffer.enqueue(rec)
362         return 1
363
364     def save_w64_event(self, event_data: Dict[str, Any], provider: str = 'w64_collector') -> int:
365         rec = {'type': 'w64_event', 'data': event_data, 'provider': provider}
366         if self._buffer.buffer_mode == 'direct':
367             return self._writer.batch_insert_records([rec])
368         self._buffer.enqueue(rec)
369         return 1
370
371     def save_process_provenance_event(self, event: Union[ProcessLifecycleEvent, ProcessProvenanceInfo, Dict[str, Any]]) -> bool:
372         ev_dict = event if isinstance(event, dict) else (event.model_dump() if hasattr(event, 'model_dump') else (vars(event) if hasattr(event, '__dict__') else event))
373         if self._buffer.buffer_mode == 'direct':
374             return self._writer.batch_insert_records([{'type': 'process_provenance_event', 'data': ev_dict}]) > 0
375         self._buffer.enqueue({'type': 'process_provenance_event', 'data': ev_dict})

```

```

    t', 'data': ev_dict}))
376         return True
377
378     def save_process_provenance_batch(self, events: List[Union
[ProcessLifecycleEvent, ProcessProvenanceInfo, Dict[str, Any]]
]) -> int:
379         if not events:
380             return 0
381         ev_dicts = [e if isinstance(e, dict) else (e.model_dump() if hasattr(e, 'model_dump') else (vars(e) if hasattr(e, '__dict__') else e)) for e in events]
382         if self._buffer.buffer_mode == 'direct':
383             return self._writer.batch_insert_records([{'type':
'process_provenance_events_batch', 'data': ev_dicts}])
384         self._buffer.enqueue({'type': 'process_provenance_events_batch', 'data': ev_dicts})
385         return len(events)
386
387     def save_incident(self, incident: TelemetryIncident) -> bool:
388         return self._writer.batch_insert_records([{'
389             'type': 'incident',
390             'data': incident if isinstance(incident, dict) else (incident.model_dump() if hasattr(incident, 'model_dump') else (vars(incident) if hasattr(incident, '__dict__') else incident))
391             }]) > 0 or True
392
393     def save_system_rollup(self, rollup: SystemMetricRollup) -> bool:
394         return self._writer.batch_insert_records([{'
395             'type': 'system_rollup',
396             'data': rollup if isinstance(rollup, dict) else (rollup.model_dump() if hasattr(rollup, 'model_dump') else (vars(rollup) if hasattr(rollup, '__dict__') else rollup))
397             }]) > 0 or True
398
399     def save_reboot_session(self, session: Any) -> int:

```

```

400         return self._writer.batch_insert_records([
401             'type': 'reboot_session',
402             'data': session if isinstance(session, dict) else
            (session.model_dump() if hasattr(session, 'model_dump') else (
            vars(session) if hasattr(session, '__dict__') else session))
403         ])) or 1
404
405     def save_extended_audit(self, audit: Any, timestamp: Optional[str] = None) -> int:
406         return self._writer.batch_insert_records([
407             'type': 'extended_audit',
408             'data': audit if isinstance(audit, dict) else (audit.model_dump() if hasattr(audit, 'model_dump') else (vars(audit) if hasattr(audit, '__dict__') else audit)),
409             'timestamp': timestamp,
410         ])) or 1
411
412     # -----
413     # Делегирование методов чтения (Reader)
414     # -----
415
416     def get_snapshots(self, limit: int = 60, since_epoch: Optional[float] = None) -> List[Dict[str, Any]]:
417         return self._reader.get_snapshots(limit=limit, since_epoch=since_epoch)
418
419     def get_latest_snapshot_full(self) -> Optional[Dict[str, Any]]:
420         """Извлекает и десериализует последний системный снимок со всеми метаданными и процессами."""
421         return self._reader.get_latest_snapshot_full()
422
423     def get_latest_cpu(self) -> Optional[Dict[str, Any]]:
424         return self._reader.get_latest_cpu()
425
426     def get_cpu_history(self, limit: int = 60, since_epoch: Op

```

```

    tional[float] = None) -> List[Dict[str, Any]]:
427         return self._reader.get_cpu_history(limit=limit, since
    _epoch=since_epoch)
428
429     def get_latest_storage_data(self) -> Dict[str, Any]:
430         return self._reader.get_latest_storage_data()
431
432     def get_snapshot_processes(self, snapshot_id: int) -> List
    [Dict[str, Any]]:
433         return self._reader.get_snapshot_processes(snapshot_id
    =snapshot_id)
434
435     def get_process_history(self, name: Optional[str] = None,
    pid: Optional[int] = None, limit: int = 100) -> List[Dict[str,
    Any]]:
436         return self._reader.get_process_history(name=name, pid
    =pid, limit=limit)
437
438     def get_process_provenance_history(self, **kwargs: Any) ->
    List[Dict[str, Any]]:
439         return self._reader.get_process_provenance_history(**k
    wargs)
440
441     def get_process_lineage(self, guid_or_pid: Union[str, int]
    ) -> List[Dict[str, Any]]:
442         return self._reader.get_process_lineage(guid_or_pid=gu
    id_or_pid)
443
444     def get_latest_processes(self, limit: int = 50, sort_by: s
    tr = 'cpu') -> List[Dict[str, Any]]:
445         return self._reader.get_latest_processes(limit=limit,
    sort_by=sort_by)
446
447     def get_process_stats(self, name: Optional[str] = None, li
    mit: int = 50) -> Dict[str, Any]:
448         return self._reader.get_process_stats(name=name, limit
    =limit)
449

```

```
450     def get_events(self, event_type: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
451         return self._reader.get_events(event_type=event_type, limit=limit)
452
453     def get_sensor_history(self, sensor_id: Optional[str] = None, category: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
454         return self._reader.get_sensor_history(sensor_id=sensor_id, category=category, limit=limit)
455
456     def get_latest_sensors(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
457         return self._reader.get_latest_sensors(limit=limit)
458
459     def get_cpu_hierarchy_metrics(self) -> Dict[str, Any]:
460         """Извлекает иерархические метрики процессора, ядер, потоков и сенсоров из базы телеметрии."""
461         return self._reader.get_cpu_hierarchy_metrics()
462
463     def get_app_polls(self, app: Optional[str] = None, metric_name: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
464         return self._reader.get_app_polls(app=app, metric_name=metric_name, limit=limit)
465
466     def get_app_events(self, app: Optional[str] = None, event_type: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
467         return self._reader.get_app_events(app=app, event_type=event_type, limit=limit)
468
469     def get_app_param_changes(self, app: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
470         return self._reader.get_app_param_changes(app=app, limit=limit)
471
472     def export_app_polls_to_csv(self, app: Optional[str] = None
```

```

    e, output_path: Optional[Union[str, Path]] = None, limit: int
    = 50000) -> Path:
473         return self._reader.export_app_polls_to_csv(app=app, o
    utput_path=output_path, limit=limit)
474
475     def export_app_events_to_csv(self, app: Optional[str] = No
    ne, output_path: Optional[Union[str, Path]] = None, limit: int
    = 50000) -> Path:
476         return self._reader.export_app_events_to_csv(app=app,
    output_path=output_path, limit=limit)
477
478     def export_app_param_changes_to_csv(self, app: Optional[str]
    = None, output_path: Optional[Union[str, Path]] = None, lim
    it: int = 50000) -> Path:
479         return self._reader.export_app_param_changes_to_csv(ap
    p=app, output_path=output_path, limit=limit)
480
481     def get_custom_records(self, source_file: Optional[str] =
    None, limit: int = 100) -> List[Dict[str, Any]]:
482         return self._reader.get_custom_records(source_file=sou
    rce_file, limit=limit)
483
484     def get_device_events(self, event_type: Optional[str] = No
    ne, device_instance_id: Optional[str] = None, limit: int = 100
    ) -> List[Dict[str, Any]]:
485         return self._reader.get_device_events(event_type=event
    _type, device_instance_id=device_instance_id, limit=limit)
486
487     def get_device_install_date(self, device_instance_id: str)
    -> Optional[str]:
488         return self._reader.get_device_install_date(device_ins
    tance_id=device_instance_id)
489
490     def get_w64_events(self, event_type: Optional[str] = None,
    provider: Optional[str] = None, limit: int = 100) -> List[Dic
    t[str, Any]]:
491         return self._reader.get_w64_events(event_type=event_ty
    pe, provider=provider, limit=limit)

```

```

492
493     def get_incidents(self, limit: int = 50, trigger_type: Opt
494 | optional[str] = None, severity: Optional[str] = None) -> List[Dic
495 | t[str, Any]]:
496 |         return self._reader.get_incidents(limit=limit, trigger
497 | _type=trigger_type, severity=severity)
498
499     def get_system_rollups(self, tier: str = '1m', start_epoch
500 | : Optional[float] = None, end_epoch: Optional[float] = None, l
501 | imit: int = 100) -> List[Dict[str, Any]]:
502 |         return self._reader.get_system_rollups(tier=tier, star
503 | t_epoch=start_epoch, end_epoch=end_epoch, limit=limit)
504
505     def get_reboot_history(self, limit: int = 50) -> List[Dict
506 | [str, Any]]:
507 |         return self._reader.get_reboot_history(limit=limit)
508
509     def get_latest_reboot_session(self) -> Optional[Dict[str,
510 | Any]]:
511 |         return self._reader.get_latest_reboot_session()
512
513     def get_extended_audits(self, limit: int = 50) -> List[Dic
514 | t[str, Any]]:
515 |         return self._reader.get_extended_audits(limit=limit)
516
517     def get_latest_extended_audit(self) -> Optional[Dict[str,
518 | Any]]:
519 |         return self._reader.get_latest_extended_audit()
520
521     def get_telemetry_rollups(self, level: str = 'hourly', sen
522 | sor_id: Optional[str] = None, start_epoch: Optional[float] = N
523 | one, end_epoch: Optional[float] = None, limit: int = 100) -> L
524 | ist[Dict[str, Any]]:
525 |         return self._reader.get_telemetry_rollups(level=level,
526 | sensor_id=sensor_id, start_epoch=start_epoch, end_epoch=end_e
527 | poch, limit=limit)
528
529     def get_telemetry_spikes(self, sensor_id: Optional[str] =

```



```

None, start_epoch: Optional[float] = None, limit: int = 100) -
> List[Dict[str, Any]]:
515     return self._reader.get_telemetry_spikes(sensor_id=sensor_id, start_epoch=start_epoch, limit=limit)
516
517     # -----
518     # Делегирование методов хранилища накопителей (Storage Sub
system)
519     # -----
520
521     def save_disk_inventory(self, disk_data: Any) -> None:
522         """Сохранить или обновить паспорт физического накопителя."""
523         rec = {'type': 'disk_inventory', 'data': disk_data}
524         if self._buffer.buffer_mode == 'direct':
525             self._writer.batch_insert_records([rec])
526             return
527         self._buffer.enqueue(rec)
528
529     def save_volume_inventory(self, volume_data: Any) -> None:
530         """Сохранить или обновить паспорт логического тома."""
531         rec = {'type': 'volume_inventory', 'data': volume_data
532 }
533         if self._buffer.buffer_mode == 'direct':
534             self._writer.batch_insert_records([rec])
535             return
536         self._buffer.enqueue(rec)
537
538     def save_disk_health_snapshot(self, health_data: Any) -> None:
539         """Сохранить снимок здоровья / SMART / TBW накопителя.
540 """
541         rec = {'type': 'disk_health', 'data': health_data}
542         if self._buffer.buffer_mode == 'direct':
543             self._writer.batch_insert_records([rec])
544             return

```

```

543         self._buffer.enqueue(rec)
544
545     def save_disk_performance_sample(self, perf_data: Any) ->
None:
546         """Сохранить сэмпл производительности диска."""
547         rec = {'type': 'disk_performance', 'data': perf_data}
548         if self._buffer.buffer_mode == 'direct':
549             self._writer.batch_insert_records([rec])
550             return
551         self._buffer.enqueue(rec)
552
553     def save_disk_io_event(self, io_event_data: Any) -> None:
554         """Сохранить событие прямого ввода-вывода."""
555         rec = {'type': 'disk_io_event', 'data': io_event_data}
556         if self._buffer.buffer_mode == 'direct':
557             self._writer.batch_insert_records([rec])
558             return
559         self._buffer.enqueue(rec)
560
561     def get_disk_inventory(self, disk_id: Optional[int] = None
) -> List[Dict[str, Any]]:
562         """Извлечь паспортные данные физических дисков."""
563         return self._reader.get_disk_inventory(disk_id=disk_id
)
564
565     def get_volume_inventory(self) -> List[Dict[str, Any]]:
566         """Извлечь паспортные данные логических томов."""
567         return self._reader.get_volume_inventory()
568
569     def get_disk_health_history(
570         self,
571         disk_id: Optional[int] = None,
572         serial_number: Optional[str] = None,
573         limit: int = 100,
574         since_epoch: Optional[float] = None,
575     ) -> List[Dict[str, Any]]:
576         """Извлечь историю здоровья и SMART/TBW накопителей."""

```

```

577         return self._reader.get_disk_health_history(disk_id=di
sk_id, serial_number=serial_number, limit=limit, since_epoch=s
ince_epoch)
578
579     def get_disk_performance_samples(
580         self,
581         disk_name: Optional[str] = None,
582         limit: int = 100,
583         since_epoch: Optional[float] = None,
584     ) -> List[Dict[str, Any]]:
585         """Извлечь сэмплы производительности дисков."""
586         return self._reader.get_disk_performance_samples(disk_
name=disk_name, limit=limit, since_epoch=since_epoch)
587
588     def get_disk_io_events(
589         self,
590         pid: Optional[int] = None,
591         disk_id: Optional[int] = None,
592         limit: int = 100,
593         since_epoch: Optional[float] = None,
594     ) -> List[Dict[str, Any]]:
595         """Извлечь события ввода-вывода процессов."""
596         return self._reader.get_disk_io_events(pid=pid, disk_i
d=disk_id, limit=limit, since_epoch=since_epoch)
597
598     # -----
599     # Делегирование методов хранилища процессоров (CPU Subsys
tem)
600     # -----
601
602     def save_cpu_inventory(self, cpu_data: Any) -> None:
603         """Сохранить или обновить паспорт процессора."""
604         rec = {'type': 'cpu_inventory', 'data': cpu_data}
605         if self._buffer.buffer_mode == 'direct':
606             self._writer.batch_insert_records([rec])
607         return

```

```

608         self._buffer.enqueue(rec)
609
610     def save_cpu_telemetry_sample(self, sample_data: Any) -> None:
611         """Сохранить снимок телеметрии процессора."""
612         rec = {'type': 'cpu_telemetry', 'data': sample_data}
613         if self._buffer.buffer_mode == 'direct':
614             self._writer.batch_insert_records([rec])
615             return
616         self._buffer.enqueue(rec)
617
618     def get_cpu_inventory(self, processor_id: Optional[int] =
None) -> List[Dict[str, Any]]:
619         """Извлечь паспортные данные процессоров."""
620         return self._reader.get_cpu_inventory(processor_id=processor_id)
621
622     def get_cpu_telemetry_samples(
623         self,
624         processor_id: Optional[int] = None,
625         limit: int = 100,
626         since_epoch: Optional[float] = None,
627     ) -> List[Dict[str, Any]]:
628         """Извлечь историю метрик процессора."""
629         return self._reader.get_cpu_telemetry_samples(processor_id=processor_id, limit=limit, since_epoch=since_epoch)
630
631     # -----
632     # Делегирование методов хранилища памяти (RAM Module / SPD Subsystem)
633     # -----
634
635     def save_ram_module_inventory(self, module_data: Any) -> None:
636         """Сохранить или обновить паспорт планки оперативной памяти."""

```

```

637         rec = {'type': 'ram_module_inventory', 'data': module_
data}
638         if self._buffer.buffer_mode == 'direct':
639             self._writer.batch_insert_records([rec])
640             return
641         self._buffer.enqueue(rec)
642
643     def save_ram_telemetry_sample(self, sample_data: Any) -> N
one:
644         """Сохранить снимок телеметрии оперативной памяти."""
645         rec = {'type': 'ram_telemetry', 'data': sample_data}
646         if self._buffer.buffer_mode == 'direct':
647             self._writer.batch_insert_records([rec])
648             return
649         self._buffer.enqueue(rec)
650
651     def get_ram_module_inventory(self) -> List[Dict[str, Any]]
:
652         """Извлечь паспортные данные физических планок операти
вной памяти."""
653         return self._reader.get_ram_module_inventory()
654
655     def get_ram_telemetry_samples(
656         self,
657         limit: int = 100,
658         since_epoch: Optional[float] = None,
659     ) -> List[Dict[str, Any]]:
660         """Извлечь историю метрик оперативной памяти."""
661         return self._reader.get_ram_telemetry_samples(limit=li
mit, since_epoch=since_epoch)
662
663     # -----
-----
664     # Делегирование методов хранилища графики (GPU Subsystem)
665     # -----
-----
666
667     def save_gpu_inventory(self, gpu_data: Any) -> None:

```

```

668 |         """Сохранить или обновить паспорт графического ускорит
        еля."""
669 |         rec = {'type': 'gpu_inventory', 'data': gpu_data}
670 |         if self._buffer.buffer_mode == 'direct':
671 |             self._writer.batch_insert_records([rec])
672 |             return
673 |         self._buffer.enqueue(rec)
674 |
675 |     def save_gpu_telemetry_sample(self, sample_data: Any) -> N
        one:
676 |         """Сохранить снимок телеметрии графического ускорителя
        ."""
677 |         rec = {'type': 'gpu_telemetry', 'data': sample_data}
678 |         if self._buffer.buffer_mode == 'direct':
679 |             self._writer.batch_insert_records([rec])
680 |             return
681 |         self._buffer.enqueue(rec)
682 |
683 |     def get_gpu_inventory(self, gpu_id: Optional[int] = None)
        -> List[Dict[str, Any]]:
684 |         """Извлечь паспортные данные GPU."""
685 |         return self._reader.get_gpu_inventory(gpu_id=gpu_id)
686 |
687 |     def get_gpu_telemetry_samples(
688 |         self,
689 |         gpu_id: Optional[int] = None,
690 |         name: Optional[str] = None,
691 |         limit: int = 100,
692 |         since_epoch: Optional[float] = None,
693 |     ) -> List[Dict[str, Any]]:
694 |         """Извлечь историю метрик GPU."""
695 |         return self._reader.get_gpu_telemetry_samples(gpu_id=g
        pu_id, name=name, limit=limit, since_epoch=since_epoch)
696 |
697 |     # -----
        -----
698 |     # Делегирование методов хранилища сети (Network Adapter Su
        bsystem)

```

```

699 |     # -----
700 |
701 |     def save_network_adapter_inventory(self, adapter_data: Any
702 | ) -> None:
703 |         """Сохранить или обновить паспорт сетевого адаптера."""
704 |         rec = {'type': 'network_adapter_inventory', 'data': ad
705 | apter_data}
706 |         if self._buffer.buffer_mode == 'direct':
707 |             self._writer.batch_insert_records([rec])
708 |             return
709 |             self._buffer.enqueue(rec)
710 |
711 |     def save_network_adapter_sample(self, sample_data: Any) ->
712 | None:
713 |         """Сохранить сэмпл показателей сетевого адаптера."""
714 |         rec = {'type': 'network_adapter_sample', 'data': sampl
715 | e_data}
716 |         if self._buffer.buffer_mode == 'direct':
717 |             self._writer.batch_insert_records([rec])
718 |             return
719 |             self._buffer.enqueue(rec)
720 |
721 |     def get_network_adapter_inventory(self) -> List[Dict[str,
722 | Any]]:
723 |         """Извлечь паспортные данные сетевых адаптеров."""
724 |         return self._reader.get_network_adapter_inventory()
725 |
726 |     def get_network_adapter_samples(
727 | self,
728 | adapter_name: Optional[str] = None,
729 | limit: int = 100,
730 | since_epoch: Optional[float] = None,
731 | ) -> List[Dict[str, Any]]:
732 |         """Извлечь сэмплы показателей сетевых адаптеров."""
733 |         return self._reader.get_network_adapter_samples(adapte
734 | r_name=adapter_name, limit=limit, since_epoch=since_epoch)

```

```

729
730
731     # -----
732     # Делегирование методов обслуживания (Maintenance)
733     # -----
734
735     def get_db_file_sizes(self) -> Dict[str, int]:
736         return self._maintenance.get_db_file_sizes()
737
738     def get_total_db_size_mb(self) -> float:
739         return self._maintenance.get_total_db_size_mb()
740
741     def checkpoint_wal(self) -> bool:
742         return self._maintenance.checkpoint_wal()
743
744     def vacuum(self) -> bool:
745         return self._maintenance.vacuum()
746
747     def get_storage_stats(self) -> Dict[str, Any]:
748         return self._maintenance.get_storage_stats(
749             buffered_count=self.get_buffered_count(),
750             buffer_mode=self.get_buffer_mode(),
751             buffer_size=self._buffer.buffer_size,
752             flush_interval_seconds=self._buffer.flush_interval
753             _seconds,
754         )
755
756     get_stats = get_storage_stats
757
758     def cleanup_old_records(self, retention_days: Optional[int]
759                             ] = None, vacuum_after: bool = False) -> int:
760         return self._maintenance.cleanup_old_records(retention
761             _days=retention_days, vacuum_after=vacuum_after)
762
763     def enforce_size_limit(self, max_size_mb: Optional[float]
764                             = None, target_ratio: float = 0.85) -> Dict[str, Any]:

```



```

761         return self._maintenance.enforce_size_limit(max_size_m
| b=max_size_mb, target_ratio=target_ratio)
762
763     def prune_to_size(self, target_size_mb: float) -> Dict[str
| , Any]:
764         return self.enforce_size_limit(max_size_mb=target_size
| _mb, target_ratio=0.95)
765
766     def aggregate_process_metrics_2min(self, cutoff_seconds: i
| nt = 120, outlier_cpu_threshold: float = 30.0) -> Dict[str, in
| t]:
767         return self._maintenance.aggregate_process_metrics_2mi
| n(cutoff_seconds=cutoff_seconds, outlier_cpu_threshold=outlier
| _cpu_threshold)
768
769     def aggregate_process_metrics_daily(self, cutoff_days: int
| = 1) -> Dict[str, int]:
770         return self._maintenance.aggregate_process_metrics_dai
| ly(cutoff_days=cutoff_days)
771
772     # -----
| -----
773     # Делегирование методов SQL-агрегации сенсоров (Aggregator
| )
774     # -----
| -----
775
776     def aggregate_sensors(self, level: Union[AggregationLevel,
| str], start_epoch: Optional[float] = None, end_epoch: Optiona
| l[float] = None, detect_spikes: bool = True) -> Dict[str, Any]
| :
777         return self._aggregator.aggregate(level=level, start_e
| poch=start_epoch, end_epoch=end_epoch, detect_spikes=detect_sp
| ikes)
778
779     def aggregate_hourly(self, start_epoch: Optional[float] =
| None, end_epoch: Optional[float] = None, detect_spikes: bool =
| True) -> Dict[str, Any]:

```

```
780 |         return self._aggregator.aggregate_hourly(start_epoch=s
| tart_epoch, end_epoch=end_epoch, detect_spikes=detect_spikes)
781 |
782 |     def aggregate_daily(self, start_epoch: Optional[float] = N
| one, end_epoch: Optional[float] = None) -> Dict[str, Any]:
783 |         return self._aggregator.aggregate_daily(start_epoch=s
| tart_epoch, end_epoch=end_epoch)
784 |
785 |     def aggregate_weekly(self, start_epoch: Optional[float] =
| None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
786 |         return self._aggregator.aggregate_weekly(start_epoch=s
| tart_epoch, end_epoch=end_epoch)
787 |
788 |     def aggregate_monthly(self, start_epoch: Optional[float] =
| None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
789 |         return self._aggregator.aggregate_monthly(start_epoch=
| start_epoch, end_epoch=end_epoch)
790 |
791 |     def aggregate_yearly(self, start_epoch: Optional[float] =
| None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
792 |         return self._aggregator.aggregate_yearly(start_epoch=s
| tart_epoch, end_epoch=end_epoch)
793 |
794 |     def run_aggregation_pipeline(self, start_epoch: Optional[f
| loat] = None, end_epoch: Optional[float] = None, detect_spikes
| : bool = True) -> Dict[str, Any]:
795 |         return self._aggregator.run_pipeline(start_epoch=start
| _epoch, end_epoch=end_epoch, detect_spikes=detect_spikes)
796 |
797 |
```

writer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Writer
4 # =====
5 # Description:
6 #   Класс TelemetryWriter для сериализации и записи телеметрии
  в SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.writer import Telemet
  ryWriter
11 #
12 #       writer = TelemetryWriter(connection_manager)
13 #       saved = writer.batch_insert_records(records)
14 #
15 # File: writer.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 04:25:00
21 # =====
22
23 from __future__ import annotations
24
25 """Класс TelemetryWriter для сериализации, подготовки строк и
  транзакционной записи в SQLite."""
26
27 import json
28 import sqlite3
29 import time
```

```

30 from datetime import datetime, timezone
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional, Union
33
34 from .connection import TelemetryConnectionManager
35 from ..models import (
36     HardwareArchiveEntry,
37     ProcessLifecycleEvent,
38     ProcessProvenanceInfo,
39     SystemMetricRollup,
40     SystemSnapshot,
41     TelemetryIncident,
42 )
43
44 try:
45     from logger import logger
46 except ImportError:
47     import logging
48     logger = logging.getLogger(__name__)
49
50
51 class TelemetryWriter:
52     """Выполняет пакетную и прямую вставку записей телеметрии
53     в базу данных SQLite."""
54
55     def __init__(self, connection_manager: TelemetryConnection
56     Manager) -> None:
57         """Инициализирует писатель телеметрии.
58
59         Args:
60             connection_manager: Менеджер соединений SQLite.
61         """
62         self._cm = connection_manager
63
64     def batch_insert_records(self, records: List[Dict[str, Any
65     ]]) -> int:
66         """Выполняет пакетную вставку разнородных записей в SQ
67         Lite в единой транзакции.

```

```

64 |
65 |     Args:
66 |         records: Список словарей записей.
67 |
68 |     Returns:
69 |         int: Общее число сохраненных записей.
70 |     """
71 |     if not records or self._cm.read_only:
72 |         return 0
73 |
74 |     saved_total = 0
75 |     with self._cm.lock, self._cm.get_connection() as conn:
76 |         cursor = conn.cursor()
77 |         sensor_poll_rows: List[tuple] = []
78 |         sensor_agg_rows: List[tuple] = []
79 |
80 |         for item in records:
81 |             rec_type = item.get('type')
82 |
83 |             if rec_type == 'snapshot':
84 |                 self.insert_snapshot_row(cursor, item)
85 |                 saved_total += 1
86 |
87 |             elif rec_type == 'sensor_poll':
88 |                 sensor_poll_rows.append(self.prepare_senso
r_poll_row(item.get('data', {}), item.get('timestamp')))
89 |
90 |             elif rec_type == 'sensor_polls_batch':
91 |                 ts = item.get('timestamp')
92 |                 for s in item.get('data', []):
93 |                     sensor_poll_rows.append(self.prepare_s
ensor_poll_row(s, ts))
94 |
95 |             elif rec_type == 'sensor_aggregates_batch':
96 |                 for a in item.get('data', []):
97 |                     sensor_agg_rows.append((
98 |                         a.get('sensor_id', ''),
99 |                         a.get('period_start', ''),

```

```

100         a.get('period_end', ''),
101         a.get('avg', None),
102         a.get('min', None),
103         a.get('max', None),
104         a.get('count', 0),
105         json.dumps(a, ensure_ascii=False,
default=str),
106     ))
107
108     elif rec_type == 'hardware_archive':
109         self.insert_hardware_archive_row(cursor, i
tem.get('data', {}))
110         saved_total += 1
111
112     elif rec_type == 'event':
113         ts_str = item.get('timestamp') or datetime
.now(timezone.utc).isoformat()
114         try:
115             now_epoch = datetime.fromisoformat(ts_
str).timestamp()
116         except Exception:
117             now_epoch = datetime.now(timezone.utc)
.timestamp()
118         details_str = json.dumps(item.get('details
', {}), ensure_ascii=False, default=str)
119         cursor.execute('''
120             INSERT INTO telemetry_events (
121                 timestamp, created_at, event_type,
severity, event_details, raw_json
122                 ) VALUES (?, ?, ?, ?, ?, ?)
123             ''', (ts_str, now_epoch, item.get('event_t
ype', ''), item.get('severity', 'info'), details_str, details_
str))
124         saved_total += 1
125
126     elif rec_type == 'disk_inventory':
127         self.upsert_disk_inventory(cursor, item.ge
t('data') or item)

```

```

128         saved_total += 1
129
130         elif rec_type == 'volume_inventory':
131             self.upsert_volume_inventory(cursor, item.
132 get('data') or item)
133             saved_total += 1
134
135         elif rec_type == 'disk_health':
136             self.insert_disk_health_snapshot(cursor, i
137 tem.get('data') or item)
138             saved_total += 1
139
140         elif rec_type == 'disk_performance':
141             self.insert_disk_performance_sample(cursor
142 , item.get('data') or item)
143             saved_total += 1
144
145         elif rec_type == 'disk_io_event':
146             self.insert_disk_io_event(cursor, item.get
147 ('data') or item)
148             saved_total += 1
149
150         elif rec_type in ('cpu_inventory', 'cpu_inv'):
151             self.upsert_cpu_inventory(cursor, item.get
152 ('data') or item)
153             saved_total += 1
154
155         elif rec_type in ('ram_module_inventory', 'ram
156 _inventory', 'ram_module'):
157             self.upsert_ram_module_inventory(cursor, i
158 tem.get('data') or item)
159             saved_total += 1
160
161         elif rec_type in ('gpu_inventory', 'gpu_inv'):
162             self.upsert_gpu_inventory(cursor, item.get
163 ('data') or item)
164             saved_total += 1
165
166         elif rec_type == 'network_inventory':
167             self.upsert_network_inventory(cursor, item.
168 get('data') or item)
169             saved_total += 1
170
171         elif rec_type == 'storage_inventory':
172             self.upsert_storage_inventory(cursor, item.
173 get('data') or item)
174             saved_total += 1
175
176         elif rec_type == 'system_inventory':
177             self.upsert_system_inventory(cursor, item.
178 get('data') or item)
179             saved_total += 1
180
181         elif rec_type == 'temperature_inventory':
182             self.upsert_temperature_inventory(cursor, i
183 tem.get('data') or item)
184             saved_total += 1
185
186         elif rec_type == 'volume_inventory':
187             self.upsert_volume_inventory(cursor, item.
188 get('data') or item)
189             saved_total += 1
190
191         elif rec_type == 'disk_health':
192             self.insert_disk_health_snapshot(cursor, i
193 tem.get('data') or item)
194             saved_total += 1
195
196         elif rec_type == 'disk_performance':
197             self.insert_disk_performance_sample(cursor
198 , item.get('data') or item)
199             saved_total += 1
200
201         elif rec_type == 'disk_io_event':
202             self.insert_disk_io_event(cursor, item.get
203 ('data') or item)
204             saved_total += 1
205
206         elif rec_type in ('cpu_inventory', 'cpu_inv'):
207             self.upsert_cpu_inventory(cursor, item.get
208 ('data') or item)
209             saved_total += 1
210
211         elif rec_type in ('ram_module_inventory', 'ram
212 _inventory', 'ram_module'):
213             self.upsert_ram_module_inventory(cursor, i
214 tem.get('data') or item)
215             saved_total += 1
216
217         elif rec_type in ('gpu_inventory', 'gpu_inv'):
218             self.upsert_gpu_inventory(cursor, item.get
219 ('data') or item)
220             saved_total += 1
221
222         elif rec_type == 'network_inventory':
223             self.upsert_network_inventory(cursor, item.
224 get('data') or item)
225             saved_total += 1
226
227         elif rec_type == 'storage_inventory':
228             self.upsert_storage_inventory(cursor, item.
229 get('data') or item)
230             saved_total += 1
231
232         elif rec_type == 'system_inventory':
233             self.upsert_system_inventory(cursor, item.
234 get('data') or item)
235             saved_total += 1
236
237         elif rec_type == 'temperature_inventory':
238             self.upsert_temperature_inventory(cursor, i
239 tem.get('data') or item)
240             saved_total += 1
241
242         elif rec_type == 'volume_inventory':
243             self.upsert_volume_inventory(cursor, item.
244 get('data') or item)
245             saved_total += 1
246
247         elif rec_type == 'disk_health':
248             self.insert_disk_health_snapshot(cursor, i
249 tem.get('data') or item)
250             saved_total += 1
251
252         elif rec_type == 'disk_performance':
253             self.insert_disk_performance_sample(cursor
254 , item.get('data') or item)
255             saved_total += 1
256
257         elif rec_type == 'disk_io_event':
258             self.insert_disk_io_event(cursor, item.get
259 ('data') or item)
260             saved_total += 1
261
262         elif rec_type in ('cpu_inventory', 'cpu_inv'):
263             self.upsert_cpu_inventory(cursor, item.get
264 ('data') or item)
265             saved_total += 1
266
267         elif rec_type in ('ram_module_inventory', 'ram
268 _inventory', 'ram_module'):
269             self.upsert_ram_module_inventory(cursor, i
270 tem.get('data') or item)
271             saved_total += 1
272
273         elif rec_type in ('gpu_inventory', 'gpu_inv'):
274             self.upsert_gpu_inventory(cursor, item.get
275 ('data') or item)
276             saved_total += 1
277
278         elif rec_type == 'network_inventory':
279             self.upsert_network_inventory(cursor, item.
280 get('data') or item)
281             saved_total += 1
282
283         elif rec_type == 'storage_inventory':
284             self.upsert_storage_inventory(cursor, item.
285 get('data') or item)
286             saved_total += 1
287
288         elif rec_type == 'system_inventory':
289             self.upsert_system_inventory(cursor, item.
290 get('data') or item)
291             saved_total += 1
292
293         elif rec_type == 'temperature_inventory':
294             self.upsert_temperature_inventory(cursor, i
295 tem.get('data') or item)
296             saved_total += 1
297
298         elif rec_type == 'volume_inventory':
299             self.upsert_volume_inventory(cursor, item.
300 get('data') or item)
301             saved_total += 1
302
303         elif rec_type == 'disk_health':
304             self.insert_disk_health_snapshot(cursor, i
305 tem.get('data') or item)
306             saved_total += 1
307
308         elif rec_type == 'disk_performance':
309             self.insert_disk_performance_sample(cursor
310 , item.get('data') or item)
311             saved_total += 1
312
313         elif rec_type == 'disk_io_event':
314             self.insert_disk_io_event(cursor, item.get
315 ('data') or item)
316             saved_total += 1
317
318         elif rec_type in ('cpu_inventory', 'cpu_inv'):
319             self.upsert_cpu_inventory(cursor, item.get
320 ('data') or item)
321             saved_total += 1
322
323         elif rec_type in ('ram_module_inventory', 'ram
324 _inventory', 'ram_module'):
325             self.upsert_ram_module_inventory(cursor, i
326 tem.get('data') or item)
327             saved_total += 1
328
329         elif rec_type in ('gpu_inventory', 'gpu_inv'):
330             self.upsert_gpu_inventory(cursor, item.get
331 ('data') or item)
332             saved_total += 1
333
334         elif rec_type == 'network_inventory':
335             self.upsert_network_inventory(cursor, item.
336 get('data') or item)
337             saved_total += 1
338
339         elif rec_type == 'storage_inventory':
340             self.upsert_storage_inventory(cursor, item.
341 get('data') or item)
342             saved_total += 1
343
344         elif rec_type == 'system_inventory':
345             self.upsert_system_inventory(cursor, item.
346 get('data') or item)
347             saved_total += 1
348
349         elif rec_type == 'temperature_inventory':
350             self.upsert_temperature_inventory(cursor, i
351 tem.get('data') or item)
352             saved_total += 1
353
354         elif rec_type == 'volume_inventory':
355             self.upsert_volume_inventory(cursor, item.
356 get('data') or item)
357             saved_total += 1
358
359         elif rec_type == 'disk_health':
360             self.insert_disk_health_snapshot(cursor, i
361 tem.get('data') or item)
362             saved_total += 1
363
364         elif rec_type == 'disk_performance':
365             self.insert_disk_performance_sample(cursor
366 , item.get('data') or item)
367             saved_total += 1
368
369         elif rec_type == 'disk_io_event':
370             self.insert_disk_io_event(cursor, item.get
371 ('data') or item)
372             saved_total += 1
373
374         elif rec_type in ('cpu_inventory', 'cpu_inv'):
375             self.upsert_cpu_inventory(cursor, item.get
376 ('data') or item)
377             saved_total += 1
378
379         elif rec_type in ('ram_module_inventory', 'ram
380 _inventory', 'ram_module'):
381             self.upsert_ram_module_inventory(cursor, i
382 tem.get('data') or item)
383             saved_total += 1
384
385         elif rec_type in ('gpu_inventory', 'gpu_inv'):
386             self.upsert_gpu_inventory(cursor, item.get
387 ('data') or item)
388             saved_total += 1
389
390         elif rec_type == 'network_inventory':
391             self.upsert_network_inventory(cursor, item.
392 get('data') or item)
393             saved_total += 1
394
395         elif rec_type == 'storage_inventory':
396             self.upsert_storage_inventory(cursor, item.
397 get('data') or item)
398             saved_total += 1
399
400         elif rec_type == 'system_inventory':
401             self.upsert_system_inventory(cursor, item.
402 get('data') or item)
403             saved_total += 1
404
405         elif rec_type == 'temperature_inventory':
406             self.upsert_temperature_inventory(cursor, i
407 tem.get('data') or item)
408             saved_total += 1
409
410         elif rec_type == 'volume_inventory':
411             self.upsert_volume_inventory(cursor, item.
412 get('data') or item)
413             saved_total += 1
414
415         elif rec_type == 'disk_health':
416             self.insert_disk_health_snapshot(cursor, i
417 tem.get('data') or item)
418             saved_total += 1
419
420         elif rec_type == 'disk_performance':
421             self.insert_disk_performance_sample(cursor
422 , item.get('data') or item)
423             saved_total += 1
424
425         elif rec_type == 'disk_io_event':
426             self.insert_disk_io_event(cursor, item.get
427 ('data') or item)
428             saved_total += 1
429
430         elif rec_type in ('cpu_inventory', 'cpu_inv'):
431             self.upsert_cpu_inventory(cursor, item.get
432 ('data') or item)
433             saved_total += 1
434
435         elif rec_type in ('ram_module_inventory', 'ram
436 _inventory', 'ram_module'):
437             self.upsert_ram_module_inventory(cursor, i
438 tem.get('data') or item)
439             saved_total += 1
440
441         elif rec_type in ('gpu_inventory', 'gpu_inv'):
442             self.upsert_gpu_inventory(cursor, item.get
443 ('data') or item)
444             saved_total += 1
445
446         elif rec_type == 'network_inventory':
447             self.upsert_network_inventory(cursor, item.
448 get('data') or item)
449             saved_total += 1
450
451         elif rec_type == 'storage_inventory':
452             self.upsert_storage_inventory(cursor, item.
453 get('data') or item)
454             saved_total += 1
455
456         elif rec_type == 'system_inventory':
457             self.upsert_system_inventory(cursor, item.
458 get('data') or item)
459             saved_total += 1
460
461         elif rec_type == 'temperature_inventory':
462             self.upsert_temperature_inventory(cursor, i
463 tem.get('data') or item)
464             saved_total += 1
465
466         elif rec_type == 'volume_inventory':
467             self.upsert_volume_inventory(cursor, item.
468 get('data') or item)
469             saved_total += 1
470
471         elif rec_type == 'disk_health':
472             self.insert_disk_health_snapshot(cursor, i
473 tem.get('data') or item)
474             saved_total += 1
475
476         elif rec_type == 'disk_performance':
477             self.insert_disk_performance_sample(cursor
478 , item.get('data') or item)
479             saved_total += 1
480
481         elif rec_type == 'disk_io_event':
482             self.insert_disk_io_event(cursor, item.get
483 ('data') or item)
484             saved_total += 1
485
486         elif rec_type in ('cpu_inventory', 'cpu_inv'):
487             self.upsert_cpu_inventory(cursor, item.get
488 ('data') or item)
489             saved_total += 1
490
491         elif rec_type in ('ram_module_inventory', 'ram
492 _inventory', 'ram_module'):
493             self.upsert_ram_module_inventory(cursor, i
494 tem.get('data') or item)
495             saved_total += 1
496
497         elif rec_type in ('gpu_inventory', 'gpu_inv'):
498             self.upsert_gpu_inventory(cursor, item.get
499 ('data') or item)
500             saved_total += 1
501
502         elif rec_type == 'network_inventory':
503             self.upsert_network_inventory(cursor, item.
504 get('data') or item)
505             saved_total += 1
506
507         elif rec_type == 'storage_inventory':
508             self.upsert_storage_inventory(cursor, item.
509 get('data') or item)
510             saved_total += 1
511
512         elif rec_type == 'system_inventory':
513             self.upsert_system_inventory(cursor, item.
514 get('data') or item)
5
```

```

158         elif rec_type in ('network_adapter_inventory',
159         'net_inventory', 'net_adapter_inv'):
160             self.upsert_network_adapter_inventory(cursor, item.get('data') or item)
161             saved_total += 1
162
163         elif rec_type in ('cpu_telemetry', 'cpu_sample
164         '):
165             self.insert_cpu_telemetry_sample(cursor, item.get('data') or item)
166             saved_total += 1
167
168         elif rec_type in ('ram_telemetry', 'ram_sample
169         '):
170             self.insert_ram_telemetry_sample(cursor, item.get('data') or item)
171             saved_total += 1
172
173         elif rec_type in ('gpu_telemetry', 'gpu_sample
174         '):
175             self.insert_gpu_telemetry_sample(cursor, item.get('data') or item)
176             saved_total += 1
177
178         elif rec_type in ('network_adapter_sample', 'network_sample', 'net_sample'):
179             self.insert_network_adapter_sample(cursor, item.get('data') or item)
180             saved_total += 1
181
182         elif rec_type == 'app_poll':
183             cursor.execute('''
184                 INSERT INTO app_polls (
185                     timestamp, created_at, app, poll_type, metric_name, value, unit, status, details, raw_json
186                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
187             )
188             ''', self.prepare_app_poll_row(item.get('d

```



```

    ata') or item))
184         saved_total += 1
185
186         elif rec_type == 'app_polls_batch':
187             rows = [self.prepare_app_poll_row(p) for p
188                     in item.get('data', [])]
189             if rows:
190                 cursor.executemany('''
191                     INSERT INTO app_polls (
192                         timestamp, created_at, app, po
193                         ll_type, metric_name, value, unit, status, details, raw_json
194                         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?,
195                         ?, ?)
196
197                     ''', rows)
198                 saved_total += len(rows)
199
200         elif rec_type == 'app_event':
201             cursor.execute('''
202                 INSERT INTO app_events (
203                     timestamp, created_at, app, event_
204                     type, status, details, raw_json
205                     ) VALUES (?, ?, ?, ?, ?, ?, ?)
206             ''', self.prepare_app_event_row(item.get('
207 data') or item))
208             saved_total += 1
209
210         elif rec_type == 'app_events_batch':
211             rows = [self.prepare_app_event_row(ev) for
212                     ev in item.get('data', [])]
213             if rows:
214                 cursor.executemany('''
215                     INSERT INTO app_events (
216                         timestamp, created_at, app, ev
217                         ent_type, status, details, raw_json
218                         ) VALUES (?, ?, ?, ?, ?, ?, ?)
219
220                     ''', rows)
221                 saved_total += len(rows)
222

```

```

214         elif rec_type == 'app_param_change':
215             cursor.execute('''
216                 INSERT INTO app_param_changes (
217                     timestamp, created_at, app, param_
name, old_value, new_value, status, user, details, raw_json
218                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
)
219             ''', self.prepare_app_param_row(item.get('
data') or item))
220             saved_total += 1
221
222         elif rec_type == 'app_param_changes_batch':
223             rows = [self.prepare_app_param_row(pc) for
pc in item.get('data', [])]
224             if rows:
225                 cursor.executemany('''
226                     INSERT INTO app_param_changes (
227                         timestamp, created_at, app, pa
ram_name, old_value, new_value, status, user, details, raw_jso
n
228                         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?)
229                 ''', rows)
230                 saved_total += len(rows)
231
232         elif rec_type == 'custom_record':
233             ts_str = item.get('timestamp') or datetime
.now(timezone.utc).isoformat()
234             try:
235                 now_epoch = datetime.fromisoformat(ts_
str).timestamp()
236             except Exception:
237                 now_epoch = datetime.now(timezone.utc)
.timestamp()
238             payload_str = json.dumps(item.get('payload
', {}), ensure_ascii=False, default=str)
239             cursor.execute('''
240                 INSERT INTO custom_records (timestamp,

```

```

        created_at, source_file, payload_json)
241         VALUES (?, ?, ?, ?)
242         '', (ts_str, now_epoch, item.get('source_
file', ''), payload_str))
243         saved_total += 1
244
245         elif rec_type == 'device_event':
246             cursor.execute(''
247                 INSERT INTO device_events (
248                     timestamp, created_at, event_type,
249                     device_instance_id,
250                     friendly_name, device_class, categ
ory, has_problem,
251                     problem_code, status_code, manufac
turer,
252                     flapping_count, uptime_seconds, ra
w_json
253                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
, ?, ?, ?, ?)
254             '', self.prepare_device_event_row(item.ge
t('data', {}), item.get('timestamp'))))
255             saved_total += 1
256
257         elif rec_type == 'w64_event':
258             cursor.execute(''
259                 INSERT INTO w64_events (
260                     event_id, timestamp, created_at, e
vent_type, path, pid, name, provider, raw_json
261                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
262             '', self.prepare_w64_event_row(item.get('
data', {}), item.get('provider', 'w64_collector'))))
263             saved_total += 1
264
265         elif rec_type == 'process_provenance_event':
266             cursor.execute(''
267                 INSERT INTO process_provenance_events
(
                event_id, timestamp, created_at, e

```

```

vent_type, process_guid, parent_guid,
268         pid, ppid, name, executable_path,
command_line, user, sid, session_id,
269         integrity_level, elevation, parent
_name, parent_cmdline,
270         ancestor_chain, launch_reason, sou
rce, details_json, raw_json
271         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
272         '', self.prepare_process_provenance_row(i
tem.get('data', {})))
273         saved_total += 1
274
275         elif rec_type == 'process_provenance_events_ba
tch':
276             rows = [self.prepare_process_provenance_ro
w(ev) for ev in item.get('data', [])]
277             if rows:
278                 cursor.executemany(''
279                     INSERT INTO process_provenance_eve
nts (
280                         event_id, timestamp, created_a
t, event_type, process_guid, parent_guid,
281                         pid, ppid, name, executable_pa
th, command_line, user, sid, session_id,
282                         integrity_level, elevation, pa
rent_name, parent_cmdline,
283                         ancestor_chain, launch_reason,
source, details_json, raw_json
284                         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
285                     '', rows)
286                 saved_total += len(rows)
287
288         elif rec_type == 'incident':
289             inc_data = item.get('data', {})
290             now_dt = datetime.now(timezone.utc)
291             ts_str = inc_data.get('timestamp') or item

```

```

    .get('timestamp') or now_dt.isoformat()
292         try:
293             now_epoch = datetime.fromisoformat(ts_
str).timestamp()
294         except Exception:
295             now_epoch = now_dt.timestamp()
296             inc_id = inc_data.get('incident_id') or f"
INC-{int(now_epoch)}"
297             cursor.execute('''
298                 INSERT OR REPLACE INTO incidents (
299                     incident_id, timestamp, created_at
, trigger_type, severity, title,
300                     description, trigger_metrics_json,
suspect_processes_json,
301                     related_events_json, metrics_summa
ry_json, raw_window_json
302                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
, ?, ?)
303             ''', (
304                 inc_id, ts_str, now_epoch,
305                 inc_data.get('trigger_type', ''),
306                 inc_data.get('severity', 'warning'),
307                 inc_data.get('title', ''),
308                 inc_data.get('description', ''),
309                 json.dumps(inc_data.get('trigger_metri
cs', {}), ensure_ascii=False, default=str),
310                 json.dumps(inc_data.get('suspect_proce
sses', []), ensure_ascii=False, default=str),
311                 json.dumps(inc_data.get('related_event
s', []), ensure_ascii=False, default=str),
312                 json.dumps(inc_data.get('metrics_summa
ry', {}), ensure_ascii=False, default=str),
313                 json.dumps(inc_data.get('raw_window',
[]), ensure_ascii=False, default=str),
314                 ))
315             saved_total += 1
316
317         elif rec_type == 'system_rollup':

```

```

318         r_data = item.get('data', {})
319         p_start = r_data.get('period_start', '')
320         p_end = r_data.get('period_end', '')
321         try:
322             p_start_epoch = datetime.fromisoformat
323             (p_start).timestamp()
324         except Exception:
325             p_start_epoch = 0.0
326         try:
327             p_end_epoch = datetime.fromisoformat(p
328             _end).timestamp()
329         except Exception:
330             p_end_epoch = 0.0
331         cursor.execute('''
332             INSERT INTO telemetry_rollups (
333                 period_start, period_end, period_s
334                 tart_epoch, period_end_epoch,
335                 duration_seconds, sample_count, ti
336                 er,
337                 cpu_avg, cpu_min, cpu_max, cpu_p95
338                 ,
339                 ram_avg_gb, ram_min_gb, ram_max_gb
340                 , ram_percent_avg, ram_percent_max,
341                 disk_read_avg_mbs, disk_read_max_m
342                 bs, disk_read_total_mb,
343                 disk_write_avg_mbs, disk_write_max
344                 _mbs, disk_write_total_mb,
345                 network_rx_avg_mbs, network_rx_max
346                 _mbs, network_rx_total_mb,
347                 network_tx_avg_mbs, network_tx_max
348                 _mbs, network_tx_total_mb,
349                 raw_json
350             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
351             , ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
352         ''', (
353             p_start, p_end, p_start_epoch, p_end_e
354             poch,
355             float(r_data.get('duration_seconds', 0

```

```

    .0) or 0.0),
344         int(r_data.get('sample_count', 0) or 0
    ),
345         str(r_data.get('tier', '1m')),
346         r_data.get('cpu_avg'), r_data.get('cpu
_min'), r_data.get('cpu_max'), r_data.get('cpu_p95'),
347         r_data.get('ram_avg_gb'), r_data.get('
ram_min_gb'), r_data.get('ram_max_gb'),
348         r_data.get('ram_percent_avg'), r_data.
get('ram_percent_max'),
349         r_data.get('disk_read_avg_mbs'), r_dat
a.get('disk_read_max_mbs'), r_data.get('disk_read_total_mb'),
350         r_data.get('disk_write_avg_mbs'), r_da
ta.get('disk_write_max_mbs'), r_data.get('disk_write_total_mb'
    ),
351         r_data.get('network_rx_avg_mbs'), r_da
ta.get('network_rx_max_mbs'), r_data.get('network_rx_total_mb'
    ),
352         r_data.get('network_tx_avg_mbs'), r_da
ta.get('network_tx_max_mbs'), r_data.get('network_tx_total_mb'
    ),
353         json.dumps(r_data, ensure_ascii=False,
    default=str),
354     ))
355     saved_total += 1
356
357     elif rec_type == 'reboot_session':
358         s_data = item.get('data', {})
359         now_dt = datetime.now(timezone.utc)
360         boot_time = s_data.get('boot_time') or now
_dt.isoformat()
361         try:
362             now_epoch = datetime.fromisoformat(boo
t_time).timestamp()
363         except Exception:
364             now_epoch = now_dt.timestamp()
365         cursor.execute(''
366             INSERT OR REPLACE INTO reboot_history

```

```

(
367         boot_id, boot_time, previous_boot_
time, uptime_seconds, uptime_human,
368         shutdown_type, likely_class, concl
usion, initiating_process, initiating_user,
369         shutdown_action, reason_text, reas
on_code, bugcheck_code, bugcheck_params_json,
370         power_button_timestamp, windows_up
date_kb, service_installed, evidence_json,
371         events_chain_json, raw_json, creat
ed_at
372     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
373     '', (
374         s_data.get('boot_id') or f"boot_{int(n
ow_epoch)}",
375         boot_time,
376         s_data.get('previous_boot_time'),
377         float(s_data.get('uptime_seconds', 0.0
) or 0.0),
378         s_data.get('uptime_human'),
379         s_data.get('shutdown_type', 'unknown')
,
380         s_data.get('likely_class'),
381         s_data.get('conclusion'),
382         s_data.get('initiating_process'),
383         s_data.get('initiating_user'),
384         s_data.get('shutdown_action'),
385         s_data.get('reason_text'),
386         s_data.get('reason_code'),
387         s_data.get('bugcheck_code'),
388         json.dumps(s_data.get('bugcheck_params
', {})), ensure_ascii=False, default=str),
389         s_data.get('power_button_timestamp'),
390         s_data.get('windows_update_kb'),
391         s_data.get('service_installed'),
392         json.dumps(s_data.get('evidence', {})),
ensure_ascii=False, default=str),

```



```

393         json.dumps(s_data.get('events_chain',
    | [], ensure_ascii=False, default=str),
394         json.dumps(s_data, ensure_ascii=False,
    | default=str),
395         now_epoch,
396         ))
397         saved_total += 1
398
399         elif rec_type == 'extended_audit':
400             audit_data = item.get('data', {})
401             now_dt = datetime.now(timezone.utc)
402             ts_str = item.get('timestamp') or audit_data.get('timestamp') or now_dt.isoformat()
403             try:
404                 now_epoch = datetime.fromisoformat(ts_str).timestamp()
405             except Exception:
406                 now_epoch = now_dt.timestamp()
407
408             def _get_f(obj: Any, key: str, default: Any = None) -> Any:
409                 if isinstance(obj, dict):
410                     return obj.get(key, default)
411                 return getattr(obj, key, default)
412
413             def_obj = _get_f(audit_data, 'defender', {})
414             startup_obj = _get_f(audit_data, 'startup', {})
415             vss_obj = _get_f(audit_data, 'vss', {})
416             users_obj = _get_f(audit_data, 'users', {})
417
418             cursor.execute('''
419                 INSERT INTO system_extended_audits (
420                     timestamp, created_at, hostname,
421                     defender_cfa_enabled, defender_asr_count, defender_exclusions_count, defender_threats_count,

```

```

422         startup_entries_count, vss_snapsho
ts_count, users_total_count, users_admin_count,
423         raw_json
424     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
, ?, ?)
425     ''' , (
426         ts_str, now_epoch, _get_f(audit_data,
'hostname', ''),
427         int(bool(_get_f(def_obj, 'cfa_enabled'
, False))),
428         int(_get_f(def_obj, 'asr_rules_count',
0) or 0),
429         int(_get_f(def_obj, 'exclusions_count'
, len(_get_f(def_obj, 'path_exclusions', []) or [])) or 0),
430         int(_get_f(def_obj, 'active_threats_co
unt', 0) or 0),
431         int(_get_f(startup_obj, 'total_entries
', len(_get_f(startup_obj, 'entries', []) or [])) or 0),
432         int(_get_f(vss_obj, 'total_snapshots_c
ount', len(_get_f(vss_obj, 'snapshots', []) or [])) or 0),
433         int(_get_f(users_obj, 'total_users_cou
nt', len(_get_f(users_obj, 'users', []) or [])) or 0),
434         int(len(_get_f(users_obj, 'admin_usern
ames', []) or []) or _get_f(users_obj, 'users_admin_count', 0)
or 0),
435         json.dumps(audit_data, ensure_ascii=Fa
lse, default=str),
436     ))
437     saved_total += 1
438
439     if sensor_poll_rows:
440         cursor.executemany(''
441             INSERT INTO sensor_polls (
442                 sensor_id, timestamp, created_at, hard
ware_name, hardware_type,
443                 sensor_category, sensor_name, unit, va
lue, provider, provider_priority, raw_json
444             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?

```

```

    ?)
445         ON CONFLICT(sensor_id, timestamp) DO UPDAT
    E SET
446         created_at = excluded.created_at,
447         hardware_name = excluded.hardware_name
    ,
448         hardware_type = excluded.hardware_type
    ,
449         sensor_category = excluded.sensor_cate
    gory,
450         sensor_name = excluded.sensor_name,
451         unit = excluded.unit,
452         value = excluded.value,
453         provider = excluded.provider,
454         provider_priority = excluded.provider_
    priority,
455         raw_json = excluded.raw_json
456     '', sensor_poll_rows)
457     saved_total += len(sensor_poll_rows)
458
459     if sensor_agg_rows:
460         cursor.executemany(''
461             INSERT INTO sensor_aggregates_hourly (
462                 sensor_id, period_start, period_end, a
    vg_value, min_value, max_value, count, raw_json
463             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
464             '', sensor_agg_rows)
465         saved_total += len(sensor_agg_rows)
466
467         conn.commit()
468     return saved_total
469
470     def prepare_sensor_poll_row(self, s: Dict[str, Any], times
    tamp: Optional[str] = None) -> tuple:
471         """Подготавливает кортеж для вставки в sensor_polls."""
    "
472         now_dt = datetime.now(timezone.utc)
473         ts_str = timestamp or s.get('timestamp') or now_dt.iso

```

```

format()
474         try:
475             now_epoch = datetime.fromisoformat(ts_str).timestamp()
mp()
476         except Exception:
477             now_epoch = now_dt.timestamp()
478             sid = str(s.get('id') or s.get('sensor_id') or '')
479             s_name = s.get('sensor_name') or s.get('name') or (f'S
ensor #{sid}' if sid else 'Sensor')
480             cat = s.get('sensor_category') or s.get('category') or
'General'
481
482             # Определение типа оборудования с fallback
483             hw_type = s.get('hardware_type')
484             if not hw_type:
485                 sid_l = sid.lower()
486                 cat_l = cat.lower()
487                 if 'disk' in sid_l or 'storage' in sid_l or 'stora
ge' in cat_l:
488                     hw_type = 'storage'
489                 elif 'gpu' in sid_l or 'vram' in sid_l or 'graphic
s' in sid_l:
490                     hw_type = 'gpu'
491                 elif 'ram' in sid_l or 'memory' in sid_l:
492                     hw_type = 'memory'
493                 elif 'net' in sid_l or 'nic' in sid_l or 'network'
in cat_l:
494                     hw_type = 'network'
495                 else:
496                     hw_type = 'cpu'
497
498             hw_name = s.get('hardware_name') or ('Storage' if hw_t
ype == 'storage' else 'System')
499             unit = s.get('unit', '')
500             raw_val = s.get('value', s.get('value_num', 0.0))
501             try:
502                 val_float = round(float(raw_val), 2)
503             except (ValueError, TypeError):

```

```

504         val_float = 0.0
505
506         provider_raw = s.get('_provider') or s.get('provider')
507         or 'CIM_SYSTEM'
508         if hasattr(provider_raw, 'name'):
509             provider_str = provider_raw.name
510             provider_priority = provider_raw.value
511         elif isinstance(provider_raw, str):
512             provider_str = provider_raw.upper()
513             try:
514                 from ..sensor_registry import SensorProvider
515                 provider_priority = SensorProvider[provider_ra
516 w.upper()].value
517             except (KeyError, ImportError):
518                 provider_priority = 30
519         else:
520             provider_str = 'CIM_SYSTEM'
521             provider_priority = 30
522
523         raw_json_str = s.get('raw_json')
524         if not raw_json_str:
525             try:
526                 raw_json_str = json.dumps(s, ensure_ascii=False,
527 e, default=str)
528             except Exception:
529                 raw_json_str = None
530
531         return (sid, ts_str, now_epoch, hw_name, hw_type, cat,
532 s_name, unit, val_float, provider_str, provider_priority, raw
533 _json_str)
534
535 def prepare_app_poll_row(self, p: Dict[str, Any]) -> tuple
536 :
537     now_dt = datetime.now(timezone.utc)
538     ts_str = p.get('timestamp') or now_dt.isoformat()
539     try:
540         now_epoch = datetime.fromisoformat(ts_str).timesta
541 mp()

```

```

535         except Exception:
536             now_epoch = now_dt.timestamp()
537             app = p.get('app', 'unknown')
538             poll_type = p.get('poll_type', 'poll')
539             metric_name = p.get('metric_name', 'metric')
540             val = p.get('value')
541             try:
542                 val_num = float(val) if val is not None and not is
instance(val, (dict, list)) else None
543             except (ValueError, TypeError):
544                 val_num = None
545             unit = p.get('unit', '')
546             status = p.get('status', 'OK')
547             details = p.get('details', '')
548             details_str = json.dumps(details, ensure_ascii=False)
if isinstance(details, (dict, list)) else str(details or '')
549             raw_json = json.dumps({'app': app, 'poll_type': poll_t
ype, 'metric_name': metric_name, 'value': val, 'unit': unit, '
status': status, 'details': details}, ensure_ascii=False)
550             return (ts_str, now_epoch, app, poll_type, metric_name
, val_num, unit, status, details_str, raw_json)
551
552     def prepare_app_event_row(self, ev: Dict[str, Any]) -> tup
le:
553         now_dt = datetime.now(timezone.utc)
554         ts_str = ev.get('timestamp') or now_dt.isoformat()
555         try:
556             now_epoch = datetime.fromisoformat(ts_str).timesta
mp()
557         except Exception:
558             now_epoch = now_dt.timestamp()
559             app = ev.get('app', 'unknown')
560             event_type = ev.get('event_type', 'event')
561             status = ev.get('status', 'OK')
562             details = ev.get('details', '')
563             details_str = json.dumps(details, ensure_ascii=False)
if isinstance(details, (dict, list)) else str(details or '')
564             raw_json = json.dumps({'app': app, 'event_type': event

```

```

        _type, 'status': status, 'details': details}, ensure_ascii=False)
565         return (ts_str, now_epoch, app, event_type, status, de
tails_str, raw_json)
566
567     def prepare_app_param_row(self, pc: Dict[str, Any]) -> tuple:
568         now_dt = datetime.now(timezone.utc)
569         ts_str = pc.get('timestamp') or now_dt.isoformat()
570         try:
571             now_epoch = datetime.fromisoformat(ts_str).timestamp()
572         except Exception:
573             now_epoch = now_dt.timestamp()
574         app = pc.get('app', 'unknown')
575         param_name = pc.get('param_name', 'param')
576         old_val = pc.get('old_value')
577         new_val = pc.get('new_value')
578         old_str = json.dumps(old_val, ensure_ascii=False) if i
sinstance(old_val, (dict, list)) else str(old_val or '')
579         new_str = json.dumps(new_val, ensure_ascii=False) if i
sinstance(new_val, (dict, list)) else str(new_val or '')
580         status = pc.get('status', 'SUCCESS')
581         user = pc.get('user', 'system')
582         details = pc.get('details', '')
583         details_str = json.dumps(details, ensure_ascii=False)
if isinstance(details, (dict, list)) else str(details or '')
584         raw_json = json.dumps({'app': app, 'param_name': param
_name, 'old_value': old_val, 'new_value': new_val, 'status': s
tatus, 'user': user, 'details': details}, ensure_ascii=False)
585         return (ts_str, now_epoch, app, param_name, old_str, n
ew_str, status, user, details_str, raw_json)
586
587     def prepare_device_event_row(self, event: Dict[str, Any],
timestamp: Optional[str] = None) -> tuple:
588         now_dt = datetime.now(timezone.utc)
589         ts_str = timestamp or event.get('timestamp') or now_dt
.isoformat()

```

```

590         try:
591             now_epoch = datetime.fromisoformat(ts_str).timestamp()
592         except Exception:
593             now_epoch = now_dt.timestamp()
594         raw_json = json.dumps(event, ensure_ascii=False, default=str)
595         return (
596             ts_str, now_epoch, event.get('event_type', ''), event.get('device_instance_id', ''),
597             event.get('friendly_name', ''), event.get('device_class', ''), event.get('category', ''),
598             int(bool(event.get('has_problem', False))), event.get('problem_code', 0),
599             event.get('status_code', 0), event.get('manufacturer', ''),
600             event.get('flapping_count_in_window', 0), event.get('uptime_seconds', 0.0), raw_json,
601         )
602
603     def prepare_w64_event_row(self, event_data: Dict[str, Any], provider: str = 'w64_collector') -> tuple:
604         now_dt = datetime.now(timezone.utc)
605         ts_str = event_data.get('timestamp') or now_dt.isoformat()
606         try:
607             now_epoch = datetime.fromisoformat(ts_str).timestamp()
608         except Exception:
609             now_epoch = now_dt.timestamp()
610         event_id = str(event_data.get('event_id') or f'evt_{int(now_epoch)})')
611         event_type = str(event_data.get('event_type', 'unknown'))
612         path_val = event_data.get('path')
613         pid_val = event_data.get('pid')
614         if pid_val is not None:
615             try:

```



```

616         pid_val = int(pid_val)
617     except (ValueError, TypeError):
618         pid_val = None
619     name_val = event_data.get('name')
620     raw_json = json.dumps(event_data, ensure_ascii=False,
621                             default=str)
622     return (event_id, ts_str, now_epoch, event_type, path_
623            val, pid_val, name_val, provider, raw_json)
624
625     def prepare_process_provenance_row(self, p: Any) -> tuple:
626         """Подготавливает кортеж для вставки в process_provena
627         nce_events."""
628         p_dict = p if isinstance(p, dict) else (p.model_dump()
629         if hasattr(p, 'model_dump') else (vars(p) if hasattr(p, '__di
630         ct__') else p))
631         now_dt = datetime.now(timezone.utc)
632         ts_str = p_dict.get('timestamp') or now_dt.isoformat()
633         try:
634             now_epoch = datetime.fromisoformat(ts_str).timesta
635             mp()
636         except Exception:
637             now_epoch = now_dt.timestamp()
638
639         pid = int(p_dict.get('pid') or 0)
640         event_id = p_dict.get('event_id') or f"pp_{int(now_epo
641         ch * 1000)}_{pid}"
642         event_type = p_dict.get('event_type') or 'ProcessCreat
643         ed'
644         ppid = int(p_dict['ppid']) if p_dict.get('ppid') is no
645         t None else None
646         name = p_dict.get('name') or 'unknown'
647         exe_path = p_dict.get('executable_path') or p_dict.get
648         ('executable')
649         cmdline = p_dict.get('command_line') or p_dict.get('cm
650         dline')
651         user = p_dict.get('user') or p_dict.get('username')
652         sid = p_dict.get('sid')
653         session_id = int(p_dict['session_id']) if p_dict.get('

```

```

        session_id') is not None else None
643         integrity = p_dict.get('integrity_level')
644         elevation = int(bool(p_dict.get('elevation', False)))
645         parent_name = p_dict.get('parent_name')
646         parent_cmdline = p_dict.get('parent_cmdline')
647         ancestor_chain = p_dict.get('ancestor_chain') or p_dict.get('ancestor_chain_str')
648         if isinstance(ancestor_chain, list):
649             ancestor_chain = ' -> '.join(ancestor_chain)
650         launch_reason = p_dict.get('launch_reason') or p_dict.get('why_was_it_created')
651         source = p_dict.get('source') or 'system'
652         creation_time = p_dict.get('creation_time') or ts_str
653         process_guid = p_dict.get('process_guid') or f"proc_{p_id}_{creation_time}"
654         parent_guid = p_dict.get('parent_guid')
655         details = p_dict.get('details') or p_dict.get('details_json')
656         details_json = json.dumps(details, ensure_ascii=False)
        if isinstance(details, (dict, list)) else (str(details) if details else None)
657         raw_json = json.dumps(p_dict, ensure_ascii=False, default=str)
658
659         return (
660             event_id, ts_str, now_epoch, event_type, process_guid, parent_guid,
661             pid, ppid, name, exe_path, cmdline, user, sid, session_id,
662             integrity, elevation, parent_name, parent_cmdline,
663             ancestor_chain, launch_reason, source, details_json, raw_json
664         )
665
666     def insert_snapshot_row(self, cursor: sqlite3.Cursor, item: Dict[str, Any]) -> int:
667         """Вставляет строку снимка и процессы в базу данных со всеми расширенными параметрами."""

```

```
668         snap_data = item.get('data', {})
669         top_n = item.get('top_n', 20)
670         snapshot = snap_data if isinstance(snap_data, SystemSn
        apshot) else None
671
672         if snapshot:
673             hostname = snapshot.hostname
674             username = snapshot.username or ''
675             uptime = snapshot.uptime_seconds
676             os_name = snapshot.os_name or 'Windows'
677             os_build = str(snapshot.os_build or '')
678             os_install_date = str(snapshot.os_install_date or
        '')
679             system_lang = str(snapshot.system_language or '')
680             os_install_lang = str(snapshot.os_install_language
        or '')
681             user_locale = str(snapshot.user_locale or '')
682             sys_locale = str(snapshot.system_locale or '')
683             timezone_val = str(snapshot.timezone or '')
684             codepage_val = str(snapshot.codepage or '')
685             input_langs = snapshot.input_languages or []
686             input_langs_json = json.dumps(input_langs, ensure_
        ascii=False) if input_langs else None
687
688             cpu_pct = snapshot.cpu.total_percent if snapshot.c
        pu else 0.0
689             cpu_freq = snapshot.cpu.frequency_mhz if snapshot.
        cpu else 0.0
690             mem_tot = snapshot.memory.total_gb if snapshot.mem
        ory else 0.0
691             mem_used = snapshot.memory.used_gb if snapshot.mem
        ory else 0.0
692             mem_pct = snapshot.memory.percent if snapshot.memo
        ry else 0.0
693             swap_pct = snapshot.memory.swap_percent if snapsho
        t.memory else 0.0
694             disk_rb = snapshot.disk_io.read_bytes_per_sec if s
        napshot.disk_io else 0.0
```

```

695         disk_wb = snapshot.disk_io.write_bytes_per_sec if
| snapshot.disk_io else 0.0
696         disk_rc = snapshot.disk_io.read_count_per_sec if s
| napsnot.disk_io else 0.0
697         disk_wc = snapshot.disk_io.write_count_per_sec if
| snapshot.disk_io else 0.0
698         net_sent = sum(getattr(i, 'bytes_sent_sec', 0.0) o
| r 0.0 for i in (snapshot.network or []))
699         net_recv = sum(getattr(i, 'bytes_recv_sec', 0.0) o
| r 0.0 for i in (snapshot.network or []))
700         gpu_load = 0.0
701         gpu_temp = 0.0
702         if snapshot.gpus:
703             gpu = snapshot.gpus[0]
704             gpu_load = getattr(gpu, 'load_percent', None)
| or getattr(gpu, 'utilization_gpu_pct', 0.0) or 0.0
705             gpu_temp = getattr(gpu, 'temperature_celsius',
| None) or getattr(gpu, 'temperature_gpu_c', 0.0) or 0.0
706             raw_procs = snapshot.top_processes or []
707
708         disks_list = [d if isinstance(d, dict) else (d.mod
| el_dump() if hasattr(d, 'model_dump') else (vars(d) if hasattr
| (d, '__dict__') else d)) for d in (snapshot.disks or [])]
709         disks_json = json.dumps(disks_list, ensure_ascii=F
| alse, default=str) if disks_list else None
710         phys_disks_list = [d if isinstance(d, dict) else (
| d.model_dump() if hasattr(d, 'model_dump') else (vars(d) if ha
| sattr(d, '__dict__') else d)) for d in (snapshot.physical_disk
| s or [])]
711         physical_disks_json = json.dumps(phys_disks_list,
| ensure_ascii=False, default=str) if phys_disks_list else None
712
713         monitors_list = [m if isinstance(m, dict) else (m.
| model_dump() if hasattr(m, 'model_dump') else (vars(m) if hasa
| ttr(m, '__dict__') else m)) for m in (snapshot.monitors or [])
| ]
714         monitors_json = json.dumps(monitors_list, ensure_a
| scii=False, default=str) if monitors_list else None

```

```

715
716         updates_obj = snapshot.updates.model_dump() if has
attr(snapshot.updates, 'model_dump') else (snapshot.updates if
instance(snapshot.updates, dict) else (vars(snapshot.update
s) if hasattr(snapshot.updates, '__dict__') else None))
717         updates_json = json.dumps(updates_obj, ensure_asci
i=False, default=str) if updates_obj else None
718
719         office_obj = snapshot.office.model_dump() if hasat
tr(snapshot.office, 'model_dump') else (snapshot.office if isi
nstance(snapshot.office, dict) else (vars(snapshot.office) if
hasattr(snapshot.office, '__dict__') else None))
720         office_json = json.dumps(office_obj, ensure_ascii=
False, default=str) if office_obj else None
721
722         onedrive_obj = snapshot.onedrive.model_dump() if h
asattr(snapshot.onedrive, 'model_dump') else (snapshot.onedriv
e if isinstance(snapshot.onedrive, dict) else (vars(snapshot.o
nedrive) if hasattr(snapshot.onedrive, '__dict__') else None))
723         onedrive_json = json.dumps(onedrive_obj, ensure_as
cii=False, default=str) if onedrive_obj else None
724
725         battery_obj = snapshot.battery.model_dump() if has
attr(snapshot.battery, 'model_dump') else (snapshot.battery if
instance(snapshot.battery, dict) else (vars(snapshot.batter
y) if hasattr(snapshot.battery, '__dict__') else None))
726         battery_json = json.dumps(battery_obj, ensure_asci
i=False, default=str) if battery_obj else None
727
728         ram_sticks_list = [r if isinstance(r, dict) else (
r.model_dump() if hasattr(r, 'model_dump') else (vars(r) if ha
sattr(r, '__dict__') else r)) for r in (snapshot.ram_sticks or
[])]
729         ram_sticks_json = json.dumps(ram_sticks_list, ensu
re_ascii=False, default=str) if ram_sticks_list else None
730
731         ports_list = [p if isinstance(p, dict) else (p.mod
el_dump() if hasattr(p, 'model_dump') else (vars(p) if hasattr

```

```

    (p, '__dict__') else p)) for p in (snapshot.listening_ports or
    [])]
732         listening_ports_json = json.dumps(ports_list, ensu
re_ascii=False, default=str) if ports_list else None
733
734         alerts_list = [a if isinstance(a, dict) else (a.mo
del_dump() if hasattr(a, 'model_dump') else (vars(a) if hasatt
r(a, '__dict__') else a)) for a in (snapshot.alerts or [])]
735         alerts_json = json.dumps(alerts_list, ensure_ascii
=False, default=str) if alerts_list else None
736
737         ts_str = snapshot.timestamp
738         hw_audit = getattr(snapshot, 'hardware_audit', Non
e)
739         raw_full_dict = snapshot.model_dump() if hasattr(s
napshot, 'model_dump') else (vars(snapshot) if hasattr(snapshot, '__dict__') else snapshot)
740         raw_json_str = json.dumps(raw_full_dict, ensure_as
cii=False, default=str)
741     else:
742         hostname = snap_data.get('hostname')
743         username = snap_data.get('username') or ''
744         uptime = snap_data.get('uptime_seconds')
745         os_name = snap_data.get('os_name') or 'Windows'
746         os_build = str(snap_data.get('os_build') or '')
747         os_install_date = str(snap_data.get('os_install_da
te') or '')
748         system_lang = str(snap_data.get('system_language')
or '')
749         os_install_lang = str(snap_data.get('os_install_la
nguage') or '')
750         user_locale = str(snap_data.get('user_locale') or
'')
751         sys_locale = str(snap_data.get('system_locale') or
'')
752         timezone_val = str(snap_data.get('timezone') or ''
)
753         codepage_val = str(snap_data.get('codepage') or ''

```

```

)
754         input_langs = snap_data.get('input_languages', [])
755         input_langs_json = json.dumps(input_langs, ensure_
ascii=False) if input_langs else None
756
757         disks_val = snap_data.get('disks', [])
758         disks_list = [d if isinstance(d, dict) else (d.mod
el_dump() if hasattr(d, 'model_dump') else (vars(d) if ha
sattr(d, '__dict__') else d)) for d in disks_val] if isinstance(dis
ks_val, list) else []
759         disks_json = json.dumps(disks_list, ensure_ascii=F
alse, default=str) if disks_list else None
760         phys_disks_val = snap_data.get('physical_disks', [
])
761         phys_disks_list = [d if isinstance(d, dict) else (
d.model_dump() if hasattr(d, 'model_dump') else (vars(d) if ha
sattr(d, '__dict__') else d)) for d in phys_disks_val] if isin
stance(phys_disks_val, list) else []
762         physical_disks_json = json.dumps(phys_disks_list,
ensure_ascii=False, default=str) if phys_disks_list else None
763
764         mon_val = snap_data.get('monitors', [])
765         monitors_json = json.dumps(mon_val, ensure_ascii=F
alse, default=str) if mon_val else None
766
767         upd_val = snap_data.get('updates')
768         updates_json = json.dumps(upd_val, ensure_ascii=Fa
lse, default=str) if upd_val else None
769
770         off_val = snap_data.get('office')
771         office_json = json.dumps(off_val, ensure_ascii=Fal
se, default=str) if off_val else None
772
773         od_val = snap_data.get('onedrive')
774         onedrive_json = json.dumps(od_val, ensure_ascii=Fa
lse, default=str) if od_val else None
775
776         bat_val = snap_data.get('battery')

```

```

777         battery_json = json.dumps(bat_val, ensure_ascii=False, default=str) if bat_val else None
778
779         ram_val = snap_data.get('ram_sticks', [])
780         ram_sticks_json = json.dumps(ram_val, ensure_ascii=False, default=str) if ram_val else None
781
782         ports_val = snap_data.get('listening_ports', [])
783         listening_ports_json = json.dumps(ports_val, ensure_ascii=False, default=str) if ports_val else None
784
785         alerts_val = snap_data.get('alerts', [])
786         alerts_json = json.dumps(alerts_val, ensure_ascii=False, default=str) if alerts_val else None
787
788         cpu_obj = snap_data.get('cpu', {})
789         cpu_pct = cpu_obj.get('total_percent', 0.0) if isinstance(cpu_obj, dict) else getattr(cpu_obj, 'total_percent', 0.0)
790         cpu_freq = cpu_obj.get('frequency_mhz', 0.0) if isinstance(cpu_obj, dict) else getattr(cpu_obj, 'frequency_mhz', 0.0)
791
792         mem_obj = snap_data.get('memory', {})
793         mem_tot = mem_obj.get('total_gb', 0.0) if isinstance(mem_obj, dict) else getattr(mem_obj, 'total_gb', 0.0)
794         mem_used = mem_obj.get('used_gb', 0.0) if isinstance(mem_obj, dict) else getattr(mem_obj, 'used_gb', 0.0)
795         mem_pct = mem_obj.get('percent', 0.0) if isinstance(mem_obj, dict) else getattr(mem_obj, 'percent', 0.0)
796         swap_pct = mem_obj.get('swap_percent', 0.0) if isinstance(mem_obj, dict) else getattr(mem_obj, 'swap_percent', 0.0)
797
798         disk_obj = snap_data.get('disk_io', {})
799         disk_rb = disk_obj.get('read_bytes_per_sec', 0.0) if isinstance(disk_obj, dict) else getattr(disk_obj, 'read_bytes_per_sec', 0.0)
800         disk_wb = disk_obj.get('write_bytes_per_sec', 0.0) if isinstance(disk_obj, dict) else getattr(disk_obj, 'write_bytes_per_sec', 0.0)

```



```

ytes_per_sec', 0.0)
799         disk_rc = disk_obj.get('read_count_per_sec', 0.0)
    if isinstance(disk_obj, dict) else getattr(disk_obj, 'read_cou
nt_per_sec', 0.0)
800         disk_wc = disk_obj.get('write_count_per_sec', 0.0)
    if isinstance(disk_obj, dict) else getattr(disk_obj, 'write_c
ount_per_sec', 0.0)
801         net_list = snap_data.get('network', [])
802         net_sent = sum((n.get('bytes_sent_sec', 0.0) if is
instance(n, dict) else getattr(n, 'bytes_sent_sec', 0.0)) or 0
.0 for n in net_list)
803         net_recv = sum((n.get('bytes_recv_sec', 0.0) if is
instance(n, dict) else getattr(n, 'bytes_recv_sec', 0.0)) or 0
.0 for n in net_list)
804         gpus = snap_data.get('gpus', [])
805         gpu_load = 0.0
806         gpu_temp = 0.0
807         if gpus:
808             g0 = gpus[0]
809             gpu_load = (g0.get('load_percent', 0.0) if isi
nstance(g0, dict) else getattr(g0, 'load_percent', 0.0)) or 0.
0
810             gpu_temp = (g0.get('temperature_celsius', 0.0)
    if isinstance(g0, dict) else getattr(g0, 'temperature_celsius
', 0.0)) or 0.0
811         raw_procs = snap_data.get('top_processes', [])
812         ts_str = snap_data.get('timestamp')
813         hw_audit = snap_data.get('hardware_audit')
814         raw_json_str = json.dumps(snap_data, ensure_ascii=
False, default=str)
815
816         now_dt = datetime.now(timezone.utc)
817         ts_str = ts_str or now_dt.isoformat()
818         try:
819             now_epoch = datetime.fromisoformat(ts_str).timesta
mp()
820         except Exception:
821             now_epoch = now_dt.timestamp()

```

```
822 hardware_audit_json = json.dumps(hw_audit or {}, ensur
823 e_ascii=False, default=str)
824
825 cursor.execute('''
826     INSERT INTO system_snapshots (
827         timestamp, created_at, hostname, username, upt
828 ime_seconds, os_name, os_build, os_install_date,
829         system_language, os_install_language, user_loc
830 ale, system_locale, timezone, codepage,
831         input_languages_json, disks_json, physical_dis
832 ks_json, monitors_json, updates_json, office_json,
833         onedrive_json, battery_json, ram_sticks_json,
834 listening_ports_json, alerts_json,
835         cpu_total_percent, cpu_frequency_mhz, memory_t
836 otal_gb, memory_used_gb,
837         memory_percent, swap_percent, gpu_load_percent
838 , gpu_temp_c,
839         disk_read_bytes_sec, disk_write_bytes_sec, dis
840 k_read_count_sec,
841         disk_write_count_sec, network_sent_bytes_sec,
842 network_recv_bytes_sec, hardware_audit, raw_json
843 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
844 , ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
845 , ?, ?, ?, ?, ?, ?, ?)
846     ''', (
847         ts_str, now_epoch, hostname, username, uptime, os_
848 name, os_build, os_install_date,
849         system_lang, os_install_lang, user_locale, sys_loc
850 ale, timezone_val, codepage_val,
851         input_langs_json, disks_json, physical_disks_json,
852 monitors_json, updates_json, office_json,
853         onedrive_json, battery_json, ram_sticks_json, list
854 ening_ports_json, alerts_json,
855         cpu_pct, cpu_freq, mem_tot, mem_used,
856         mem_pct, swap_pct, gpu_load, gpu_temp,
857         disk_rb, disk_wb, disk_rc,
858         disk_wc, net_sent, net_recv, hardware_audit_json,
```

```

raw_json_str
845         ))
846         snapshot_id = cursor.lastrowid or 0
847
848         processes = raw_procs if top_n is None or top_n <= 0 e
lse raw_procs[:top_n]
849         proc_rows = []
850         for p in processes:
851             p_dict = p if isinstance(p, dict) else (p.model_du
mp() if hasattr(p, 'model_dump') else (vars(p) if hasattr(p, '
__dict__') else p))
852             proc_rows.append((
853                 snapshot_id, ts_str, p_dict.get('pid', 0), p_d
ict.get('name', 'unknown'),
854                 p_dict.get('status', ''), float(p_dict.get('cp
u_percent', 0.0) or 0.0),
855                 float(p_dict.get('memory_mb', 0.0) or 0.0), fl
oat(p_dict.get('memory_percent', 0.0) or 0.0),
856                 int(p_dict.get('num_threads', 0) or 0), int(p_
dict.get('num_handles', 0) or 0),
857                 str(p_dict.get('username', '') or ''), float(p
_dict.get('read_bytes_sec', 0.0) or 0.0),
858                 float(p_dict.get('write_bytes_sec', 0.0) or 0.
0), p_dict.get('integrity_level', None),
859                 int(bool(p_dict.get('elevation', False))), p_d
ict.get('ppid', None),
860                 p_dict.get('parent_name', None), p_dict.get('e
xecutable_path', p_dict.get('executable', None)),
861                 p_dict.get('cmdline', p_dict.get('command_line
', None)), p_dict.get('sid', None),
862                 p_dict.get('session_id', None), p_dict.get('cr
eation_time', None),
863                 p_dict.get('process_guid', None), p_dict.get('
ancestor_chain', None),
864                 p_dict.get('launch_reason', None),
865                 ))
866         if proc_rows:
867             cursor.executemany('')

```

```

868         INSERT INTO process_snapshots (
869             snapshot_id, timestamp, pid, name, status,
            cpu_percent,
870             memory_mb, memory_percent, num_threads, nu
m_handles,
871             username, read_bytes_sec, write_bytes_sec,
            integrity_level, elevation,
872             ppid, parent_name, executable, cmdline, si
d, session_id,
873             creation_time, process_guid, ancestor_chai
n, launch_reason
874         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
            ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
875         '', proc_rows)
876
877     # Синхронизация нормализованных таблиц оборудования и
метрик
878     try:
879         # 1. CPU
880         cpu_data = getattr(snapshot, 'cpu', None) or snap_
data.get('cpu')
881         if cpu_data:
882             c_dict = cpu_data if isinstance(cpu_data, dict
) else (cpu_data.model_dump() if hasattr(cpu_data, 'model_dump
') else vars(cpu_data))
883             if c_dict.get('model') or c_dict.get('name'):
884                 self.upsert_cpu_inventory(cursor, {
885                     'processor_id': 0,
886                     'name': c_dict.get('model') or c_dict.
get('name', 'CPU'),
887                     'architecture': c_dict.get('architectu
re', 'x86_64'),
888                     'physical_cores': c_dict.get('physical
_cores', 1),
889                     'logical_cores': c_dict.get('logical_c
ores', 1),
890                     'base_frequency_mhz': c_dict.get('freq
uency_mhz', 0.0),

```

```

891         })
892         self.insert_cpu_telemetry_sample(cursor, {
893             'timestamp': ts_str,
894             'created_at': now_epoch,
895             'processor_id': 0,
896             'total_percent': cpu_pct,
897             'frequency_mhz': cpu_freq,
898             'temperature_c': c_dict.get('temperature_c
|elsius') or c_dict.get('temperature_c'),
899             'core_utilization': c_dict.get('per_core_p
|ercent', []),
900         })
901
902         # 2. RAM
903         mem_data = getattr(snapshot, 'memory', None) or sn
|ap_data.get('memory')
904         if mem_data:
905             m_dict = mem_data if isinstance(mem_data, dict
|) else (mem_data.model_dump() if hasattr(mem_data, 'model_dump
|') else vars(mem_data))
906             self.insert_ram_telemetry_sample(cursor, {
907                 'timestamp': ts_str,
908                 'created_at': now_epoch,
909                 'total_gb': mem_tot,
910                 'used_gb': mem_used,
911                 'available_gb': m_dict.get('available_gb',
|max(0.0, mem_tot - mem_used)),
912                 'percent_used': mem_pct,
913                 'swap_total_gb': m_dict.get('swap_total_gb
|', 0.0),
914                 'swap_used_gb': m_dict.get('swap_used_gb',
|0.0),
915                 'swap_percent': swap_pct,
916             })
917
918         # 3. RAM Sticks (SPD)
919         sticks = getattr(snapshot, 'ram_sticks', None) or
|snap_data.get('ram_sticks') or []

```

```

920         for s in sticks:
921             self.upsert_ram_module_inventory(cursor, s)
922
923         # 4. GPUs
924         gpus_list = getattr(snapshot, 'gpus', None) or snap_data.get('gpus') or []
925         for g_idx, g_item in enumerate(gpus_list):
926             g_dict = g_item if isinstance(g_item, dict) else (g_item.model_dump() if hasattr(g_item, 'model_dump') else vars(g_item))
927             self.upsert_gpu_inventory(cursor, {
928                 'gpu_id': g_idx,
929                 'name': g_dict.get('name', f'GPU {g_idx}'),
930                 'vendor': 'NVIDIA' if g_dict.get('has_cuda') else ('AMD' if 'amd' in g_dict.get('name', '').lower() else 'Intel'),
931                 'vram_gb': g_dict.get('memory_total_gb', 0.0),
932                 'directml_supported': bool(g_dict.get('has_directml', True)),
933             })
934             self.insert_gpu_telemetry_sample(cursor, {
935                 'timestamp': ts_str,
936                 'created_at': now_epoch,
937                 'gpu_id': g_idx,
938                 'name': g_dict.get('name', f'GPU {g_idx}'),
939                 'load_percent': g_dict.get('load_percent'),
940                 'memory_used_mb': (g_dict.get('memory_used_gb', 0.0) or 0.0) * 1024.0,
941                 'memory_total_mb': (g_dict.get('memory_total_gb', 0.0) or 0.0) * 1024.0,
942                 'temperature_gpu_c': g_dict.get('temperature_celsius') or g_dict.get('temperature_gpu_c'),
943             })
944

```

```

945         # 5. Network
946         net_items = getattr(snapshot, 'network', None) or
snap_data.get('network') or []
947         for n_idx, n_item in enumerate(net_items):
948             n_dict = n_item if isinstance(n_item, dict) el
se (n_item.model_dump() if hasattr(n_item, 'model_dump') else
vars(n_item))
949             self.upsert_network_adapter_inventory(cursor,
{
950                 'adapter_id': n_idx,
951                 'adapter_guid': n_dict.get('name', f'Adapt
er_{n_idx}'),
952                 'name': n_dict.get('name', f'Adapter {n_id
x}'),
953                 'max_speed_mbps': n_dict.get('speed_mbps',
0),
954             })
955             self.insert_network_adapter_sample(cursor, {
956                 'timestamp': ts_str,
957                 'created_at': now_epoch,
958                 'adapter_name': n_dict.get('name', f'Adapt
er {n_idx}'),
959                 'bytes_recv_sec': n_dict.get('bytes_recv_p
er_sec', 0.0),
960                 'bytes_sent_sec': n_dict.get('bytes_sent_p
er_sec', 0.0),
961                 'link_speed_mbps': n_dict.get('speed_mbps'
, 0),
962                 'is_connected': bool(n_dict.get('is_up', T
rue)),
963             })
964         except Exception as norm_err:
965             logger.debug(f'Предупреждение синхронизации нормал
изованных таблиц оборудования: {norm_err}')
966
967         return snapshot_id
968
969     def insert_hardware_archive_row(self, cursor: sqlite3.Curs

```

```

    or, data: Any) -> int:
970         now_dt = datetime.now(timezone.utc)
971         if isinstance(data, HardwareArchiveEntry):
972             raw_json = data.model_dump_json()
973             report = data.report
974             prob_count = report.problem_devices_count if repor
t else 0
975             outdated_count = report.outdated_drivers_count if
report else 0
976             archive_id = data.archive_id
977             timestamp = data.timestamp
978             devices_count = data.devices_count
979             changes_count = data.changes_count
980         else:
981             raw_json = json.dumps(data, ensure_ascii=False, de
fault=str)
982             report = data.get('report', {})
983             prob_count = report.get('problem_devices_count', 0
) if isinstance(report, dict) else getattr(report, 'problem_de
vices_count', 0)
984             outdated_count = report.get('outdated_drivers_coun
t', 0) if isinstance(report, dict) else getattr(report, 'outda
ted_drivers_count', 0)
985             archive_id = data.get('archive_id', f'arch_{int(no
w_dt.timestamp())}')
986             timestamp = data.get('timestamp', now_dt.isoformat
())
987             devices_count = data.get('devices_count', 0)
988             changes_count = data.get('changes_count', 0)
989
990         try:
991             now_epoch = datetime.fromisoformat(timestamp).time
stamp()
992         except Exception:
993             now_epoch = now_dt.timestamp()
994
995         cursor.execute('''
996             INSERT OR REPLACE INTO hardware_audits (

```



```

997         archive_id, timestamp, created_at, devices_cou
nt,
998         problem_devices_count, outdated_drivers_count,
changes_count, raw_json
999     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1000     ''' , (archive_id, timestamp, now_epoch, devices_count,
prob_count, outdated_count, changes_count, raw_json))
1001     return cursor.lastrowid or 0
1002
1003     def upsert_disk_inventory(self, cursor: sqlite3.Cursor, da
ta: Any) -> int:
1004         """Вставка или обновление паспорта физического накопит
еля (disk_inventory)."""
1005         d = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1006         now_dt = datetime.now(timezone.utc)
1007         ts_str = d.get('timestamp') or now_dt.isoformat()
1008         now_epoch = now_dt.timestamp()
1009         serial = str(d.get('serial_number') or f"DISK_{d.get('
disk_id', 0)}").strip()
1010
1011         cursor.execute(''
1012             INSERT INTO disk_inventory (
1013                 disk_id, device_path, serial_number, vendor, p
roduct,
1014                 revision, bus_type, media_type, size_bytes, si
ze_gb,
1015                 sector_size, removable, partition_style, gpt_g
uid,
1016                 partitions_json, first_seen, last_seen, update
d_at
1017             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
, ?, ?, ?, ?)
1018             ON CONFLICT(serial_number) DO UPDATE SET
1019                 disk_id = excluded.disk_id,
1020                 device_path = excluded.device_path,
1021                 vendor = excluded.vendor,
1022                 product = excluded.product,

```

```

1023         revision = excluded.revision,
1024         bus_type = excluded.bus_type,
1025         media_type = excluded.media_type,
1026         size_bytes = excluded.size_bytes,
1027         size_gb = excluded.size_gb,
1028         sector_size = excluded.sector_size,
1029         removable = excluded.removable,
1030         partition_style = excluded.partition_style,
1031         gpt_guid = excluded.gpt_guid,
1032         partitions_json = excluded.partitions_json,
1033         last_seen = excluded.last_seen,
1034         updated_at = excluded.updated_at
1035     ''' , (
1036         int(d.get('disk_id', 0)),
1037         d.get('device_path') or f"\\\\.\\PhysicalDrive{d.g
1038         et('disk_id', 0)}",
1039         serial,
1040         d.get('vendor', ''),
1041         d.get('product') or d.get('name', ''),
1042         d.get('revision', ''),
1043         d.get('bus_type', 'Unknown'),
1044         d.get('media_type', 'SSD'),
1045         int(d.get('size_bytes', 0)),
1046         float(d.get('size_gb', 0.0)),
1047         int(d.get('sector_size', 512)),
1048         int(bool(d.get('removable', False))),
1049         d.get('partition_style', 'GPT'),
1050         d.get('gpt_guid', ''),
1051         json.dumps(d.get('partitions', []), ensure_ascii=F
1052         else, default=str),
1053         ts_str,
1054         ts_str,
1055         now_epoch,
1056     ))
1057     return cursor.lastrowid or 0
1058
1059     def upsert_volume_inventory(self, cursor: sqlite3.Cursor,
1060     data: Any) -> int:

```

```

1058 |         """Вставка или обновление паспорта логического тома (v
|         olume_inventory)."""
1059 |         v = data if isinstance(data, dict) else (data.model_du
|         mp() if hasattr(data, 'model_dump') else vars(data))
1060 |         now_dt = datetime.now(timezone.utc)
1061 |         ts_str = v.get('timestamp') or now_dt.isoformat()
1062 |         now_epoch = now_dt.timestamp()
1063 |         guid = str(v.get('volume_guid') or v.get('volume_id')
|         or v.get('drive_letter', 'UNKNOWN')).strip()
1064 |
1065 |         cursor.execute('''
1066 |             INSERT INTO volume_inventory (
1067 |                 volume_guid, drive_letter, mount_point, label,
|                 filesystem,
1068 |                 filesystem_flags, cluster_size_bytes, sector_s
|                 ize_bytes,
1069 |                 total_bytes, total_gb, disk_id, first_seen, la
|                 st_seen, updated_at
1070 |             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
|             )
1071 |
1072 |             ON CONFLICT(volume_guid) DO UPDATE SET
1073 |                 drive_letter = excluded.drive_letter,
1074 |                 mount_point = excluded.mount_point,
1075 |                 label = excluded.label,
1076 |                 filesystem = excluded.filesystem,
1077 |                 filesystem_flags = excluded.filesystem_flags,
1078 |                 cluster_size_bytes = excluded.cluster_size_byt
|                 es,
1079 |                 sector_size_bytes = excluded.sector_size_bytes
|                 ,
1080 |                 total_bytes = excluded.total_bytes,
1081 |                 total_gb = excluded.total_gb,
1082 |                 disk_id = excluded.disk_id,
1083 |                 last_seen = excluded.last_seen,
1084 |                 updated_at = excluded.updated_at
1085 |             ''', (
1086 |                 guid,
|                 v.get('drive_letter', ''),

```

```

1087         v.get('mount_point', ''),
1088         v.get('label', ''),
1089         v.get('filesystem', 'NTFS'),
1090         int(v.get('filesystem_flags', 0) or 0),
1091         int(v.get('cluster_size_bytes', 4096) or 4096),
1092         int(v.get('sector_size_bytes', 512) or 512),
1093         int(v.get('total_bytes', 0) or 0),
1094         float(v.get('total_gb', 0.0) or 0.0),
1095         v.get('disk_id'),
1096         ts_str,
1097         ts_str,
1098         now_epoch,
1099     ))
1100     return cursor.lastrowid or 0
1101
1102     def insert_disk_health_snapshot(self, cursor: sqlite3.Cursor, data: Any) -> int:
1103         """Вставка снимка здоровья / SMART / TBW накопителя (disk_health_snapshots)."""
1104         h = data if isinstance(data, dict) else (data.model_dump() if hasattr(data, 'model_dump') else vars(data))
1105         now_dt = datetime.now(timezone.utc)
1106         ts_str = h.get('timestamp') or now_dt.isoformat()
1107         try:
1108             created_at = float(h.get('created_at') or datetime.fromisoformat(ts_str).timestamp())
1109         except Exception:
1110             created_at = now_dt.timestamp()
1111
1112         cursor.execute('''
1113             INSERT INTO disk_health_snapshots (
1114                 timestamp, created_at, disk_id, serial_number,
1115                 model, bus_type,
1116                 temperature_c, wear_percent, available_spare,
1117                 tbw_written_tb,
1118                 tbw_read_tb, power_on_hours, power_cycles, unsafe_shutdowns,
1119                 media_errors, error_log_entries, health_status

```

```

    , raw_smart_json
1118         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
    , ?, ?, ?, ?)
1119         '', (
1120             ts_str,
1121             created_at,
1122             int(h.get('disk_id', 0)),
1123             h.get('serial_number'),
1124             h.get('model', ''),
1125             h.get('bus_type', 'Unknown'),
1126             h.get('temperature_c'),
1127             h.get('wear_percentage') or h.get('wear_percent'),
1128             h.get('available_spare_percent') or h.get('availab
le_spare'),
1129             h.get('tbw_written_tb') or h.get('lifetime_write_t
b'),
1130             h.get('tbw_read_tb') or h.get('lifetime_read_tb'),
1131             h.get('power_on_hours'),
1132             h.get('power_cycles'),
1133             h.get('unsafe_shutdowns'),
1134             h.get('media_errors'),
1135             h.get('error_log_entries'),
1136             h.get('health_status', 'Healthy'),
1137             json.dumps(h, ensure_ascii=False, default=str),
1138         ))
1139         return cursor.lastrowid or 0
1140
1141     def insert_disk_performance_sample(self, cursor: sqlite3.C
ursor, data: Any) -> int:
1142         """Вставка сэмпла производительности диска (disk_perfo
rmance_samples)."""
1143         p = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1144         now_dt = datetime.now(timezone.utc)
1145         ts_str = p.get('timestamp') or now_dt.isoformat()
1146         try:
1147             created_at = float(p.get('created_at') or datetime
.fromisoformat(ts_str).timestamp())

```

```

1148         except Exception:
1149             created_at = now_dt.timestamp()
1150
1151         cursor.execute('''
1152             INSERT INTO disk_performance_samples (
1153                 timestamp, created_at, disk_name, read_bytes_s
1154                 ec, write_bytes_sec,
1155                 read_iops, write_iops, read_time_ms, write_tim
1156                 e_ms, percent_disk_time, queue_length
1157             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1158             ''', (
1159                 ts_str,
1160                 created_at,
1161                 p.get('disk_name') or f"PhysicalDrive{p.get('disk_
1162                 id', 0)}",
1163                 float(p.get('read_bytes_sec', 0.0) or 0.0),
1164                 float(p.get('write_bytes_sec', 0.0) or 0.0),
1165                 float(p.get('read_iops', 0.0) or 0.0),
1166                 float(p.get('write_iops', 0.0) or 0.0),
1167                 float(p.get('read_time_ms', 0.0) or p.get('avg_rea
1168                 d_latency_ms', 0.0) or 0.0),
1169                 float(p.get('write_time_ms', 0.0) or p.get('avg_wr
1170                 ite_latency_ms', 0.0) or 0.0),
1171                 float(p.get('percent_disk_time', 0.0) or p.get('di
1172                 sk_time_percent', 0.0) or 0.0),
1173                 float(p.get('queue_length', 0.0) or 0.0),
1174             ))
1175         return cursor.lastrowid or 0
1176
1177     def insert_disk_io_event(self, cursor: sqlite3.Cursor, dat
1178     a: Any) -> int:
1179         """Вставка события ввода-вывода (disk_io_events)."""
1180         ev = data if isinstance(data, dict) else (data.model_d
1181         ump() if hasattr(data, 'model_dump') else vars(data))
1182         now_dt = datetime.now(timezone.utc)
1183         ts_str = ev.get('timestamp') or now_dt.isoformat()
1184         try:
1185             created_at = float(ev.get('created_at') or datetim

```

```

        e.fromisoformat(ts_str).timestamp())
1178         except Exception:
1179             created_at = now_dt.timestamp()
1180
1181         cursor.execute('''
1182             INSERT INTO disk_io_events (
1183                 timestamp, created_at, pid, process_name, disk
1184                 _id,
1185                 operation, bytes_count, duration_ms, offset, f
1186                 ile_path
1187                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1188             ''', (
1189                 ts_str,
1190                 created_at,
1191                 int(ev.get('pid', 0)),
1192                 ev.get('process_name', ''),
1193                 int(ev.get('disk_id', 0)),
1194                 ev.get('operation', 'READ'),
1195                 int(ev.get('bytes_count', 0)),
1196                 ev.get('duration_ms'),
1197                 ev.get('offset'),
1198                 ev.get('file_path'),
1199             ))
1200         return cursor.lastrowid or 0
1201
1202     def upsert_cpu_inventory(self, cursor: sqlite3.Cursor, dat
1203     a: Any) -> int:
1204         """Вставка или обновление паспорта физического процесс
1205         opa (cpu_inventory)."""
1206         c = data if isinstance(data, dict) else (data.model_du
1207         mp() if hasattr(data, 'model_dump') else vars(data))
1208         now_dt = datetime.now(timezone.utc)
1209         ts_str = c.get('timestamp') or now_dt.isoformat()
1210         now_epoch = now_dt.timestamp()
1211         name = str(c.get('name') or f"CPU_{c.get('processor_id
1212         ', 0)}").strip()
1213
1214         cursor.execute('''

```

```

1209         INSERT INTO cpu_inventory (
1210             processor_id, name, vendor, architecture, phys
1211             ical_cores, logical_cores,
1212             base_frequency_mhz, max_frequency_mhz, l2_cach
1213             e_kb, l3_cache_kb,
1214             socket, features_json, first_seen, last_seen,
1215             updated_at
1216         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
1217         , ?)
1218
1219         ON CONFLICT(name) DO UPDATE SET
1220             processor_id = excluded.processor_id,
1221             vendor = excluded.vendor,
1222             architecture = excluded.architecture,
1223             physical_cores = excluded.physical_cores,
1224             logical_cores = excluded.logical_cores,
1225             base_frequency_mhz = excluded.base_frequency_m
1226             hz,
1227             max_frequency_mhz = excluded.max_frequency_mhz
1228             ,
1229             l2_cache_kb = excluded.l2_cache_kb,
1230             l3_cache_kb = excluded.l3_cache_kb,
1231             socket = excluded.socket,
1232             features_json = excluded.features_json,
1233             last_seen = excluded.last_seen,
1234             updated_at = excluded.updated_at
1235     ''' , (
1236         int(c.get('processor_id', 0)),
1237         name,
1238         c.get('vendor', ''),
1239         c.get('architecture', 'x86_64'),
1240         int(c.get('physical_cores', 1) or 1),
1241         int(c.get('logical_cores', 1) or 1),
1242         float(c.get('base_frequency_mhz', 0.0) or 0.0),
1243         float(c.get('max_frequency_mhz', 0.0) or 0.0),
1244         c.get('l2_cache_kb'),
1245         c.get('l3_cache_kb'),
1246         c.get('socket', ''),
1247         json.dumps(c.get('features', []), ensure_ascii=Fal

```



```

        se, default=str),
1241         ts_str,
1242         ts_str,
1243         now_epoch,
1244     ))
1245     return cursor.lastrowid or 0
1246
1247     def upsert_ram_module_inventory(self, cursor: sqlite3.Curs
or, data: Any) -> int:
1248         """Вставка или обновление паспорта планки оперативной
памяти (ram_module_inventory)."""
1249         r = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1250         now_dt = datetime.now(timezone.utc)
1251         ts_str = r.get('timestamp') or now_dt.isoformat()
1252         now_epoch = now_dt.timestamp()
1253         serial = str(r.get('serial_number') or f"RAM_{r.get('b
ank_label', '')}_{r.get('slot_id', 0)}").strip()
1254
1255         cursor.execute('''
1256             INSERT INTO ram_module_inventory (
1257                 slot_id, bank_label, device_locator, serial_nu
mber, part_number,
1258                 manufacturer, capacity_bytes, capacity_gb, spe
ed_mhz, memory_type,
1259                 form_factor, configured_voltage, first_seen, l
ast_seen, updated_at
1260             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
, ?)
1261
1262             ON CONFLICT(serial_number) DO UPDATE SET
1263                 slot_id = excluded.slot_id,
1264                 bank_label = excluded.bank_label,
1265                 device_locator = excluded.device_locator,
1266                 part_number = excluded.part_number,
1267                 manufacturer = excluded.manufacturer,
1268                 capacity_bytes = excluded.capacity_bytes,
1269                 capacity_gb = excluded.capacity_gb,
                 speed_mhz = excluded.speed_mhz,

```

```

1270         memory_type = excluded.memory_type,
1271         form_factor = excluded.form_factor,
1272         configured_voltage = excluded.configured_volta
ge,
1273         last_seen = excluded.last_seen,
1274         updated_at = excluded.updated_at
1275     ''' , (
1276         int(r.get('slot_id', 0)),
1277         r.get('bank_label', 'DIMM'),
1278         r.get('device_locator', ''),
1279         serial,
1280         r.get('part_number', ''),
1281         r.get('manufacturer', ''),
1282         int(r.get('capacity_bytes', 0) or 0),
1283         float(r.get('capacity_gb', 0.0) or 0.0),
1284         int(r.get('speed_mhz', 0) or 0),
1285         r.get('memory_type', 'DDR4'),
1286         r.get('form_factor', 'DIMM'),
1287         r.get('configured_voltage'),
1288         ts_str,
1289         ts_str,
1290         now_epoch,
1291     ))
1292     return cursor.lastrowid or 0
1293
1294     def upsert_gpu_inventory(self, cursor: sqlite3.Cursor, dat
a: Any) -> int:
1295         """Вставка или обновление паспорта графического ускоре
теля (gpu_inventory)."""
1296         g = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1297         now_dt = datetime.now(timezone.utc)
1298         ts_str = g.get('timestamp') or now_dt.isoformat()
1299         now_epoch = now_dt.timestamp()
1300         pci_id = str(g.get('pci_device_id') or g.get('pci_bus_
id') or f"GPU_{g.get('gpu_id', 0)}_{g.get('name', '')}").strip
()
1301

```

```

1302         cursor.execute('''
1303             INSERT INTO gpu_inventory (
1304                 gpu_id, pci_device_id, name, vendor, driver_ve
rsion, driver_date,
1305                 vram_bytes, vram_gb, pci_bus_id, bios_version,
                 cuda_cores,
1306                 directml_supported, first_seen, last_seen, upd
ated_at
1307             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
, ?)
1308             ON CONFLICT(pci_device_id) DO UPDATE SET
1309                 gpu_id = excluded.gpu_id,
1310                 name = excluded.name,
1311                 vendor = excluded.vendor,
1312                 driver_version = excluded.driver_version,
1313                 driver_date = excluded.driver_date,
1314                 vram_bytes = excluded.vram_bytes,
1315                 vram_gb = excluded.vram_gb,
1316                 pci_bus_id = excluded.pci_bus_id,
1317                 bios_version = excluded.bios_version,
1318                 cuda_cores = excluded.cuda_cores,
1319                 directml_supported = excluded.directml_support
ed,
1320                 last_seen = excluded.last_seen,
1321                 updated_at = excluded.updated_at
1322         ''', (
1323             int(g.get('gpu_id', 0)),
1324             pci_id,
1325             g.get('name', 'Graphics Adapter'),
1326             g.get('vendor', ''),
1327             g.get('driver_version', ''),
1328             g.get('driver_date', ''),
1329             int(g.get('vram_bytes', 0) or 0),
1330             float(g.get('vram_gb', 0.0) or (g.get('memory_tota
l_mb', 0.0) / 1024.0 if g.get('memory_total_mb') else 0.0)),
1331             g.get('pci_bus_id', ''),
1332             g.get('bios_version', ''),
1333             g.get('cuda_cores'),

```

```

1334         int(bool(g.get('directml_supported', True))),
1335         ts_str,
1336         ts_str,
1337         now_epoch,
1338     ))
1339     return cursor.lastrowid or 0
1340
1341     def upsert_network_adapter_inventory(self, cursor: sqlite3
1342     .Cursor, data: Any) -> int:
1343         """Вставка или обновление паспорта сетевого адаптера (
1344         network_adapter_inventory)."""
1345         n = data if isinstance(data, dict) else (data.model_du
1346         mp() if hasattr(data, 'model_dump') else vars(data))
1347         now_dt = datetime.now(timezone.utc)
1348         ts_str = n.get('timestamp') or now_dt.isoformat()
1349         now_epoch = now_dt.timestamp()
1350         guid = str(n.get('adapter_guid') or n.get('mac_address
1351         ') or n.get('name', 'UNKNOWN')).strip()
1352
1353         cursor.execute('''
1354             INSERT INTO network_adapter_inventory (
1355                 adapter_id, adapter_guid, name, interface_name
1356                 , mac_address,
1357                 adapter_type, is_physical, is_wireless, max_sp
1358                 eed_mbps,
1359                 driver_name, driver_version, first_seen, last_
1360                 seen, updated_at
1361             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
1362             )
1363
1364             ON CONFLICT(adapter_guid) DO UPDATE SET
1365                 adapter_id = excluded.adapter_id,
1366                 name = excluded.name,
1367                 interface_name = excluded.interface_name,
1368                 mac_address = excluded.mac_address,
1369                 adapter_type = excluded.adapter_type,
1370                 is_physical = excluded.is_physical,
1371                 is_wireless = excluded.is_wireless,
1372                 max_speed_mbps = excluded.max_speed_mbps,

```

```

1364         driver_name = excluded.driver_name,
1365         driver_version = excluded.driver_version,
1366         last_seen = excluded.last_seen,
1367         updated_at = excluded.updated_at
1368     ''' , (
1369         int(n.get('adapter_id', 0)),
1370         guid,
1371         n.get('name', 'Network Adapter'),
1372         n.get('interface_name', ''),
1373         n.get('mac_address', ''),
1374         n.get('adapter_type', 'Ethernet'),
1375         int(bool(n.get('is_physical', True))),
1376         int(bool(n.get('is_wireless', False))),
1377         int(n.get('max_speed_mbps', 0) or 0),
1378         n.get('driver_name', ''),
1379         n.get('driver_version', ''),
1380         ts_str,
1381         ts_str,
1382         now_epoch,
1383     ))
1384     return cursor.lastrowid or 0
1385
1386     def insert_cpu_telemetry_sample(self, cursor: sqlite3.Cursor, data: Any) -> int:
1387         """Вставка снимка метрик нагрузки и температур процессора (cpu_telemetry_samples)."""
1388         c = data if isinstance(data, dict) else (data.model_dump() if hasattr(data, 'model_dump') else vars(data))
1389         now_dt = datetime.now(timezone.utc)
1390         ts_str = c.get('timestamp') or now_dt.isoformat()
1391         try:
1392             created_at = float(c.get('created_at') or datetime.fromisoformat(ts_str).timestamp())
1393         except Exception:
1394             created_at = now_dt.timestamp()
1395
1396         cursor.execute(''
1397             INSERT INTO cpu_telemetry_samples (

```

```

1398         timestamp, created_at, processor_id, total_per
cent, user_percent,
1399         kernel_percent, frequency_mhz, temperature_c,
package_power_w,
1400         core_utilization_json, core_temperatures_json
1401     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1402     ''' , (
1403         ts_str,
1404         created_at,
1405         int(c.get('processor_id', 0)),
1406         float(c.get('total_percent', 0.0) or 0.0),
1407         float(c.get('user_percent', 0.0) or 0.0),
1408         float(c.get('kernel_percent', 0.0) or 0.0),
1409         float(c.get('frequency_mhz', 0.0) or 0.0),
1410         c.get('temperature_c') or c.get('temperature_celsi
us'),
1411         c.get('package_power_w'),
1412         json.dumps(c.get('core_utilization') or c.get('per
_core_percent', []), ensure_ascii=False, default=str),
1413         json.dumps(c.get('core_temperatures', []), ensure_
ascii=False, default=str),
1414     ))
1415     return cursor.lastrowid or 0
1416
1417     def insert_ram_telemetry_sample(self, cursor: sqlite3.Curs
or, data: Any) -> int:
1418         """Вставка снимка метрик использования оперативной пам
яти (ram_telemetry_samples)."""
1419         r = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1420         now_dt = datetime.now(timezone.utc)
1421         ts_str = r.get('timestamp') or now_dt.isoformat()
1422         try:
1423             created_at = float(r.get('created_at') or datetime
.fromisoformat(ts_str).timestamp())
1424         except Exception:
1425             created_at = now_dt.timestamp()
1426

```

```

1427         cursor.execute('''
1428             INSERT INTO ram_telemetry_samples (
1429                 timestamp, created_at, total_bytes, total_gb,
used_bytes, used_gb,
1430                 available_bytes, available_gb, percent_used, s
wap_total_gb,
1431                 swap_used_gb, swap_percent, pool_paged_mb, poo
l_nonpaged_mb
1432             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
)
1433         ''', (
1434             ts_str,
1435             created_at,
1436             int(r.get('total_bytes', 0) or 0),
1437             float(r.get('total_gb', 0.0) or 0.0),
1438             int(r.get('used_bytes', 0) or 0),
1439             float(r.get('used_gb', 0.0) or 0.0),
1440             int(r.get('available_bytes', 0) or 0),
1441             float(r.get('available_gb', 0.0) or 0.0),
1442             float(r.get('percent_used', 0.0) or r.get('percent
', 0.0) or 0.0),
1443             float(r.get('swap_total_gb', 0.0) or 0.0),
1444             float(r.get('swap_used_gb', 0.0) or 0.0),
1445             float(r.get('swap_percent', 0.0) or 0.0),
1446             r.get('pool_paged_mb'),
1447             r.get('pool_nonpaged_mb'),
1448         ))
1449         return cursor.lastrowid or 0
1450
1451     def insert_gpu_telemetry_sample(self, cursor: sqlite3.Curs
or, data: Any) -> int:
1452         """Вставка снимка метрик GPU (gpu_telemetry_samples)."""
1453
1454         g = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1455         now_dt = datetime.now(timezone.utc)
1456         ts_str = g.get('timestamp') or now_dt.isoformat()
1457         try:

```

```

1457         created_at = float(g.get('created_at') or datetime
    .fromisoformat(ts_str).timestamp())
1458     except Exception:
1459         created_at = now_dt.timestamp()
1460
1461     cursor.execute('''
1462         INSERT INTO gpu_telemetry_samples (
1463             timestamp, created_at, gpu_id, name, load_perc
ent, memory_used_mb,
1464             memory_total_mb, memory_percent, temperature_g
pu_c, temperature_memory_c,
1465             fan_speed_pct, power_draw_w, clock_graphics_mh
z, clock_memory_mhz
1466             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
    )
1467     ''', (
1468         ts_str,
1469         created_at,
1470         int(g.get('gpu_id', 0) or g.get('index', 0)),
1471         g.get('name', 'GPU'),
1472         g.get('load_percent') or g.get('utilization_gpu_pc
t'),
1473         float(g.get('memory_used_mb', 0.0) or (g.get('memo
ry_used_gb', 0.0) * 1024.0 if g.get('memory_used_gb') else 0.0
    )),
1474         float(g.get('memory_total_mb', 0.0) or (g.get('mem
ory_total_gb', 0.0) * 1024.0 if g.get('memory_total_gb') else
    0.0)),
1475         g.get('memory_percent') or g.get('utilization_memo
ry_pct'),
1476         g.get('temperature_gpu_c') or g.get('temperature_c
elsius'),
1477         g.get('temperature_memory_c'),
1478         g.get('fan_speed_pct'),
1479         g.get('power_draw_w'),
1480         g.get('clock_graphics_mhz'),
1481         g.get('clock_memory_mhz'),
1482     ))

```



```

1483         return cursor.lastrowid or 0
1484
1485     def insert_network_adapter_sample(self, cursor: sqlite3.Cu
1486         rsor, data: Any) -> int:
1487         """Вставка снимка метрик сетевого адаптера (network_ad
1488         apter_samples)."""
1489         n = data if isinstance(data, dict) else (data.model_du
1490         mp() if hasattr(data, 'model_dump') else vars(data))
1491         now_dt = datetime.now(timezone.utc)
1492         ts_str = n.get('timestamp') or now_dt.isoformat()
1493         try:
1494             created_at = float(n.get('created_at') or datetime
1495             .fromisoformat(ts_str).timestamp())
1496         except Exception:
1497             created_at = now_dt.timestamp()
1498
1499         cursor.execute('''
1500             INSERT INTO network_adapter_samples (
1501                 timestamp, created_at, adapter_name, bytes_rec
1502                 v_sec, bytes_sent_sec,
1503                 packets_rcv_sec, packets_sent_sec, errors_in_
1504                 sec, errors_out_sec,
1505                 link_speed_mbps, is_connected
1506             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1507         ''', (
1508             ts_str,
1509             created_at,
1510             n.get('adapter_name') or n.get('name', 'Network Ad
1511             apter'),
1512             float(n.get('bytes_rcv_sec', 0.0) or n.get('bytes
1513             _rcv_per_sec', 0.0) or 0.0),
1514             float(n.get('bytes_sent_sec', 0.0) or n.get('bytes
1515             _sent_per_sec', 0.0) or 0.0),
1516             float(n.get('packets_rcv_sec', 0.0) or 0.0),
1517             float(n.get('packets_sent_sec', 0.0) or 0.0),
1518             float(n.get('errors_in_sec', 0.0) or 0.0),
1519             float(n.get('errors_out_sec', 0.0) or 0.0),
1520             int(n.get('link_speed_mbps', 0) or n.get('speed_mb

```

```
        ps', 0) or 0),
1512 |         int(bool(n.get('is_connected', True) if n.get('is_
        | connected') is not None else n.get('is_up', True))),
1513 |     ))
1514 |     return cursor.lastrowid or 0
1515 |
1516 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sqlite
  Package
4 # =====
5 # Description:
6 #   Пакет персистентного хранения и выборки системной телеметр
  ии в SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite import TelemetryStora
  ge, TelemetryReader
11 #
12 #       storage = TelemetryStorage.get_instance(read_only=True)
13 #       cpu = storage.get_latest_cpu()
14 #
15 # File: __init__.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 00:20:00
21 # =====
22
23 from __future__ import annotations
24
25 """Пакет персистентного хранения, буферизации и выборки систем
  ной телеметрии в SQLite."""
26
27 from .aggregator import AggregationLevel, TelemetrySqlAggregat
  or, sensors_aggregate
28 from .buffer import TelemetryBuffer
```

```
29 | from .connection import TelemetryConnectionManager
30 | from .maintenance import TelemetryMaintenance
31 | from .reader import TelemetryReader
32 | from .schema import init_database_schema
33 | from .storage import TelemetryStorage
34 | from .writer import TelemetryWriter
35 |
36 | SQLiteTelemetryStorage = TelemetryStorage
37 |
38 | __all__ = [
39 |     'TelemetryStorage',
40 |     'SQLiteTelemetryStorage',
41 |     'TelemetryReader',
42 |     'TelemetryWriter',
43 |     'TelemetryBuffer',
44 |     'TelemetryMaintenance',
45 |     'TelemetryConnectionManager',
46 |     'TelemetrySqlAggregator',
47 |     'AggregationLevel',
48 |     'sensors_aggregate',
49 |     'init_database_schema',
50 | ]
51 |
52 |
```

storage_usage.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Storage
  Usage
4 # =====
5 # Description:
6 #   Четырехуровневый стек сбора телеметрии накопителей Windows
  :
7 #   1. DiskInventoryCollector (Win32 + IOCTL Storage Property
  & Volumes)
8 #   2. DiskHealthCollector (NVMe SMART / Health / TBW)
9 #   3. DiskPerformanceCollector (IOPS, Latency, Throughput, Qu
  eue)
10 #   4. DiskProcessIOCollector (Попроцессный I/O анализ и агрег
  аты)
11 #
12 # Usage Examples:
13 #   Python API:
14 #       from apps.windows.telemetry.storage_usage import (
15 #           DiskInventoryCollector,
16 #           DiskHealthCollector,
17 #           DiskPerformanceCollector,
18 #           DiskProcessIOCollector,
19 #           WindowsStorageUsageCollector,
20 #       )
21 #
22 #       collector = WindowsStorageUsageCollector()
23 #       disks = collector.get_physical_disks()
24 #       report = collector.get_disk_usage_period_report()
25 #
26 # File: storage_usage.py
27 # Project: ai-breadboard
28 # Package: apps.windows.telemetry
29 # Author: hypo69
```

```
30 # Copyright: © 2026 hypo69
31 # Updated: 2026-10-06 01:35:00
32 # =====
33 |=====
34 from __future__ import annotations
35 """Четырехуровневый стек сбора телеметрии накопителей Windows.
36 |"""
37 import math
38 import os
39 import time
40 from typing import Any, Dict, List, Optional, Tuple
41 |
42 try:
43 |     import psutil
44 |     PSUTIL_AVAILABLE = True
45 except ImportError:
46 |     psutil = None
47 |     PSUTIL_AVAILABLE = False
48 |
49 from logger import logger
50 from apps.windows.modules.storage_manager.core.raw_disk_io import WindowsRawDiskIO
51 from apps.windows.modules.storage_manager.core.windows_storage
52 |_sensor import WindowsStorageSensor
53 from apps.windows.telemetry.models import (
54 |     AppDiskUsageItem,
55 |     DiskUsagePeriodReport,
56 |     PhysicalDiskHealth,
57 | )
58 |
59 def _format_bytes(num_bytes: float) -> str:
60 |     """Форматирование объема байт в человекочитаемый вид (B, K
61 |     B, MB, GB, TB).
62 |
63 |     Args:
```

```

63         num_bytes: Количество байт.
64
65     Returns:
66         str: Отформатированная строка.
67     """
68     if num_bytes <= 0 or math.isnan(num_bytes):
69         return '0 B'
70     units = ['B', 'KB', 'MB', 'GB', 'TB', 'PB']
71     idx = 0
72     val = float(num_bytes)
73     while val >= 1024.0 and idx < len(units) - 1:
74         val /= 1024.0
75         idx += 1
76     return f'{val:.2f} {units[idx]}' if idx > 0 else f'{int(val)} B'
77
78
79 class DiskInventoryCollector:
80     """Слой 1: Коллектор инвентаря физических накопителей, разделов и логических томов."""
81
82     def __init__(self, raw_io: Optional[WindowsRawDiskIO] = None) -> None:
83         """Инициализация коллектора инвентаря дисков."""
84         self.raw_io = raw_io or WindowsRawDiskIO()
85
86     def collect(self) -> Dict[str, Any]:
87         """Сбор полного инвентаря накопителей и томов.
88
89         Returns:
90             Dict[str, Any]: Данные физических дисков и логических томов.
91         """
92         disks = self.raw_io.list_physical_disks()
93         volumes = self.raw_io.get_logical_volumes()
94         return {
95             'disks_count': len(disks),
96             'disks': disks,

```

```

97         'volumes_count': len(volumes),
98         'volumes': volumes,
99         'timestamp': time.time(),
100     }
101
102
103 class DiskHealthCollector:
104     """Слой 2: Коллектор SMART-показателей здоровья, износа и
105     TBW накопителей."""
106
107     def __init__(
108         self,
109         raw_io: Optional[WindowsRawDiskIO] = None,
110         storage_sensor: Optional[WindowsStorageSensor] = None,
111     ) -> None:
112         """Инициализация коллектора здоровья дисков."""
113         self.raw_io = raw_io or WindowsRawDiskIO()
114         self.storage_sensor = storage_sensor or WindowsStorage
115         Sensor()
116
117     def collect(self, force_refresh: bool = False) -> List[Dic
118     t[str, Any]]:
119         """Сбор показателей здоровья всех доступных накопите
120         лей.
121
122         Returns:
123             List[Dict[str, Any]]: Показатели SMART, TBW, темпе
124             ратуры и ошибок.
125         """
126         results: List[Dict[str, Any]] = []
127         raw_disks = self.raw_io.list_physical_disks()
128
129         for d in raw_disks:
130             disk_id = d.get('disk_id', 0)
131             smart = d.get('smart_health', {})
132             results.append({
133                 'disk_id': disk_id,
134                 'device_path': d.get('device_path'),

```



```

130         'model': d.get('product'),
131         'bus_type': d.get('bus_type'),
132         'temperature_c': smart.get('temperature_c'),
133         'wear_percentage': smart.get('percentage_used'
    ),
134         'available_spare_percent': smart.get('available_spare_percent'),
135         'tbw_written_tb': smart.get('data_units_written_tb'),
136         'tbw_read_tb': smart.get('data_units_read_tb')
    ,
137         'power_on_hours': smart.get('power_on_hours'),
138         'unsafe_shutdowns': smart.get('unsafe_shutdowns'),
139         'media_errors': smart.get('media_errors'),
140         'raw_available': smart.get('raw_nvme_available', False),
141     })
142
143     # Fallback / дополнение через WindowsStorageSensor CIM
    если нативный IOCTL вернул None
144     needs_cim = any(r['temperature_c'] is None and r['wear_percentage'] is None for r in results)
145     if needs_cim:
146         try:
147             cim_disks = self.storage_sensor.get_physical_disks(force_refresh=force_refresh)
148             cim_map = {str(cd.device_id).replace(r'\\.\PHYSICALDRIVE', '').replace('Disk', ''): cd for cd in cim_disks}
149             for r in results:
150                 str_id = str(r['disk_id'])
151                 if str_id in cim_map:
152                     cd = cim_map[str_id]
153                     if r['temperature_c'] is None and cd.temperature_c:
154                         r['temperature_c'] = cd.temperature_c
155                     if r['wear_percentage'] is None and cd

```

```

155 |         .wear_percentage:
156 |             r['wear_percentage'] = cd.wear_per
157 |             centage
158 |             if r['power_on_hours'] is None and cd.
159 |             power_on_hours:
160 |                 r['power_on_hours'] = cd.power_on_
161 |                 hours
162 |                 if r['tbw_written_tb'] is None and cd.
163 |                 lifetime_write_tb:
164 |                     r['tbw_written_tb'] = cd.lifetime_
165 |                     write_tb
166 |                     if r['tbw_read_tb'] is None and cd.lif
167 |                     etime_read_tb:
168 |                         r['tbw_read_tb'] = cd.lifetime_rea
169 |                         d_tb
170 |             except Exception as exc:
171 |                 logger.debug(f"Ошибка дополнения здоровья чере
172 | з CIM: {exc}")
173 |
174 |         return results
175 |
176 |
177 | class DiskPerformanceCollector:
178 |     """Слой 3: Коллектор текущей скорости, IOPS и времени откл
179 |     ика дисков."""
180 |
181 |     def __init__(self) -> None:
182 |         """Инициализация коллектора производительности дисков.
183 |
184 |         self._last_sample_time: float = time.time()
185 |         self._last_io_counters: Dict[str, Any] = {}
186 |
187 |     def collect(self) -> Dict[str, Any]:
188 |         """Сбор показателей пропускной способности и IOPS нако
189 |         пителей.
190 |
191 |         Returns:
192 |             Dict[str, Any]: Метрики скорости чтения/записи и о

```

пераций в секунду.

```
182         """
183         now = time.time()
184         dt = max(0.001, now - self._last_sample_time)
185         res: Dict[str, Any] = {
186             'total_read_bytes_sec': 0.0,
187             'total_write_bytes_sec': 0.0,
188             'total_read_iops': 0.0,
189             'total_write_iops': 0.0,
190             'per_disk': {},
191             'dt_sec': dt,
192         }
193
194         if not PSUTIL_AVAILABLE:
195             self._last_sample_time = now
196             return res
197
198         try:
199             per_disk_counters = psutil.disk_io_counters(perdis
200 k=True)
201             if per_disk_counters:
202                 for disk_name, cur in per_disk_counters.items(
203 ):
204                     prev = self._last_io_counters.get(disk_name)
205                     if prev:
206                         r_bytes = max(0, cur.read_bytes - prev
207 .read_bytes) / dt
208                         w_bytes = max(0, cur.write_bytes - prev
209 v.write_bytes) / dt
210                         r_count = max(0, cur.read_count - prev
211 .read_count) / dt
212                         w_count = max(0, cur.write_count - prev
213 v.write_count) / dt
214                         r_time = max(0, cur.read_time - prev.r
215 ead_time)
216                         w_time = max(0, cur.write_time - prev.
217 write_time)
```

```

210 |
211 |         res['total_read_bytes_sec'] += r_bytes
212 |         res['total_write_bytes_sec'] += w_byte
    | s
213 |         res['total_read_iops'] += r_count
214 |         res['total_write_iops'] += w_count
215 |
216 |         res['per_disk'][disk_name] = {
217 |             'read_bytes_sec': round(r_bytes, 1
    | ),
218 |             'write_bytes_sec': round(w_bytes,
    | 1),
219 |             'read_iops': round(r_count, 1),
220 |             'write_iops': round(w_count, 1),
221 |             'read_time_ms': r_time,
222 |             'write_time_ms': w_time,
223 |         }
224 |         self._last_io_counters = per_disk_counters
225 |     except Exception as exc:
226 |         logger.debug(f"Ошибка сбора счетчиков производител
    | ьности дисков: {exc}")
227 |
228 |         self._last_sample_time = now
229 |         return res
230 |
231 |
232 | class DiskProcessIOCollector:
233 |     """Слой 4: Коллектор активности ввода-вывода процессов (To
    | p Writers/Readers)."""
234 |
235 |     def __init__(self) -> None:
236 |         """Инициализация коллектора I/O процессов."""
237 |         self._last_process_io: Dict[int, Dict[str, Any]] = {}
238 |         self._last_poll_time: float = time.time()
239 |
240 |     def collect(self) -> Dict[str, Dict[str, Any]]:
241 |         """Опрос I/O всех активных процессов и вычисление дель
    | т чтения/записи.

```

```

242
243     Returns:
244         Dict[str, Dict[str, Any]]: Агрегированные дельты п
о именам процессов.
245         """
246         if not PSUTIL_AVAILABLE:
247             return {}
248
249         now = time.time()
250         aggregated_by_name: Dict[str, Dict[str, Any]] = {}
251
252         try:
253             for p in psutil.process_iter(attrs=['pid', 'name']
):
254                 try:
255                     pid = p.info['pid']
256                     name = p.info.get('name') or f'PID_{pid}'
257                     io_cnt = p.io_counters()
258                     cur_r = io_cnt.read_bytes
259                     cur_w = io_cnt.write_bytes
260
261                     prev = self._last_process_io.get(pid)
262                     if prev:
263                         delta_r = max(0, cur_r - prev['read_by
tes'])
264                         delta_w = max(0, cur_w - prev['write_b
ytes'])
265                     else:
266                         delta_r = cur_r
267                         delta_w = cur_w
268
269                     self._last_process_io[pid] = {
270                         'name': name,
271                         'read_bytes': cur_r,
272                         'write_bytes': cur_w,
273                         'time': now,
274                     }
275

```

```

276         if name not in aggregated_by_name:
277             aggregated_by_name[name] = {
278                 'pid': pid,
279                 'read_bytes': 0,
280                 'write_bytes': 0,
281             }
282             aggregated_by_name[name]['read_bytes'] +=
283             delta_r
284             aggregated_by_name[name]['write_bytes'] +=
285             delta_w
286         except (psutil.NoSuchProcess, psutil.AccessDen
287         ired, AttributeError):
288             continue
289         except Exception as ex:
290             logger.debug(f'Ошибка выборки process I/O deltas:
291             {ex}')
292
293     self._last_poll_time = now
294     return aggregated_by_name
295
296 class WindowsStorageUsageCollector:
297     """Унифицированный диспетчер дисковой телеметрии, объединя
298     ющий все 4 слоя."""
299
300     def __init__(
301         self,
302         raw_io: Optional[WindowsRawDiskIO] = None,
303         storage_sensor: Optional[WindowsStorageSensor] = None,
304     ) -> None:
305         """Инициализация унифицированного коллектора дисковой
306         телеметрии."""
307
308         self.raw_io = raw_io or WindowsRawDiskIO()
309         self.storage_sensor = storage_sensor or WindowsStorage
310         Sensor()
311
312         self.inventory_collector = DiskInventoryCollector(self
313         .raw_io)

```

```

306         self.health_collector = DiskHealthCollector(self.raw_i
o, self.storage_sensor)
307         self.performance_collector = DiskPerformanceCollector(
)
308         self.process_io_collector = DiskProcessIOCollector()
309
310     def get_physical_disks(self, force_refresh: bool = False)
-> List[PhysicalDiskHealth]:
311         """Получить актуальный список физических накопителей с
показателями SMART и Lifetime."""
312         results: List[PhysicalDiskHealth] = []
313         try:
314             health_list = self.health_collector.collect(force_
refresh=force_refresh)
315             for h in health_list:
316                 results.append(PhysicalDiskHealth(
317                     device_id=f"Disk {h.get('disk_id', 0)}",
318                     model=h.get('model') or f"PhysicalDrive{h.
get('disk_id', 0)}",
319                     media_type='SSD' if 'SSD' in (h.get('model
', '')) + h.get('bus_type', '').upper() or h.get('bus_type') =
= 'NVMe' else 'HDD',
320                     size_gb=0.0,
321                     health_status='Healthy',
322                     operational_status='OK',
323                     temperature_celsius=h.get('temperature_c')
,
324                     interface_type=h.get('bus_type') or 'NVMe'
,
325                     lifetime_read_bytes=int(h['tbw_read_tb'] *
(1024 ** 4)) if h.get('tbw_read_tb') else None,
326                     lifetime_write_bytes=int(h['tbw_written_tb
'] * (1024 ** 4)) if h.get('tbw_written_tb') else None,
327                     lifetime_read_tb=h.get('tbw_read_tb'),
328                     lifetime_write_tb=h.get('tbw_written_tb'),
329                     power_on_hours=h.get('power_on_hours'),
330                     wear_percentage=h.get('wear_percentage'),
331                 ))

```

```

332         except Exception as ex:
333             logger.debug(f'Ошибка получения физических накопит
елей: {ex}')
334
335         if not results:
336             results.append(PhysicalDiskHealth(
337                 device_id='Disk 0',
338                 model='System Drive (NVMe/SSD)',
339                 media_type='SSD',
340                 size_gb=512.0,
341                 health_status='Healthy',
342                 operational_status='OK',
343                 interface_type='NVMe',
344             ))
345         return results
346
347     def poll_process_io_deltas(self) -> Dict[str, Dict[str, An
y]]:
348         """Опросить текущий I/O процессов и рассчитать накопле
нные дельты."""
349         return self.process_io_collector.collect()
350
351     def get_disk_usage_period_report(
352         self,
353         period_minutes: int = 1440,
354         force_refresh_smart: bool = False,
355     ) -> DiskUsagePeriodReport:
356         """Сформировать агрегированный отчет об использовании
диска за интервал."""
357         phys_disks = self.get_physical_disks(force_refresh=for
ce_refresh_smart)
358         proc_deltas = self.poll_process_io_deltas()
359
360         writers_list: List[AppDiskUsageItem] = []
361         readers_list: List[AppDiskUsageItem] = []
362
363         total_r = sum(item['read_bytes'] for item in proc_delt
as.values())

```



```

364         total_w = sum(item['write_bytes'] for item in proc_deltas.values())
365         total_all = total_r + total_w
366
367         # Ранжирование по записи
368         sorted_writers = sorted(proc_deltas.items(), key=lambda x: x[1]['write_bytes'], reverse=True)
369         for name, item in sorted_writers[:15]:
370             wb = item['write_bytes']
371             rb = item['read_bytes']
372             tot = wb + rb
373             if wb <= 0:
374                 continue
375             pct = round((wb / total_w) * 100.0, 1) if total_w > 0 else 0.0
376             writers_list.append(AppDiskUsageItem(
377                 process_name=name,
378                 pid=item.get('pid'),
379                 read_bytes=rb,
380                 write_bytes=wb,
381                 total_bytes=tot,
382                 read_formatted=_format_bytes(rb),
383                 write_formatted=_format_bytes(wb),
384                 share_percent=pct,
385             ))
386
387         # Ранжирование по чтению
388         sorted_readers = sorted(proc_deltas.items(), key=lambda x: x[1]['read_bytes'], reverse=True)
389         for name, item in sorted_readers[:15]:
390             wb = item['write_bytes']
391             rb = item['read_bytes']
392             tot = wb + rb
393             if rb <= 0:
394                 continue
395             pct = round((rb / total_r) * 100.0, 1) if total_r > 0 else 0.0
396             readers_list.append(AppDiskUsageItem(

```

```
397         process_name=name,
398         pid=item.get('pid'),
399         read_bytes=rb,
400         write_bytes=wb,
401         total_bytes=tot,
402         read_formatted=_format_bytes(rb),
403         write_formatted=_format_bytes(wb),
404         share_percent=pct,
405     ))
406
407     top_writer = sorted_writers[0][0] if sorted_writers and sorted_writers[0][1]['write_bytes'] > 0 else 'N/A'
408     top_reader = sorted_readers[0][0] if sorted_readers and sorted_readers[0][1]['read_bytes'] > 0 else 'N/A'
409
410     diag = (
411         f"За анализируемый период ({period_minutes} мин) записано {_format_bytes(total_w)}, "
412         f"прочитано {_format_bytes(total_r)}. Главный источник записи: {top_writer}, чтения: {top_reader}."
413     )
414
415     return DiskUsagePeriodReport(
416         period_minutes=period_minutes,
417         total_read_bytes=total_r,
418         total_write_bytes=total_w,
419         total_read_formatted=_format_bytes(total_r),
420         total_write_formatted=_format_bytes(total_w),
421         total_formatted=_format_bytes(total_all),
422         top_writers=writers_list,
423         top_readers=readers_list,
424         physical_disks=phys_disks,
425         summary_text=diag,
426     )
427
428
429 __all__ = [
430     'DiskInventoryCollector',
```

```
431 | 'DiskHealthCollector',  
432 | 'DiskPerformanceCollector',  
433 | 'DiskProcessIOCollector',  
434 | 'WindowsStorageUsageCollector',  
435 | ]  
436 |
```

telemetry_config.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Telemetry Config
4 # =====
5 # Description:
6 #   Менеджер конфигурации сбора телеметрии и сенсоров с поддержкой
7 #   динамического перечитывания config.json на лету.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.telemetry_config import TelemetryConfigManager
12 #
13 #       config_mgr = TelemetryConfigManager()
14 #       interval = config_mgr.get_interval_seconds()
15 #       if config_mgr.check_and_reload():
16 #           print("Конфигурация обновлена!")
17 #
18 # File: telemetry_config.py
19 # Project: ai-breadboard
20 # Package: apps.windows.telemetry
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # Updated: 2026-10-06 04:30:00
24 # =====
25
26 from __future__ import annotations
27 """Менеджер конфигурации сбора телеметрии и сенсоров."""
28
29 import json
30 import os
```

```

31 import re
32 import shutil
33 from pathlib import Path
34 from typing import Any, Dict, List, Optional
35 try:
36     from logger import logger
37 except ImportError:
38     import logging
39     logger = logging.getLogger(__name__)
40
41
42 def get_default_telemetry_config_path() -> Path:
43     """Возвращает путь по умолчанию к файлу конфигурации телем
    етрии в %APPDATA%.
44
45     Если целевой файл в %APPDATA%/AI-Breadboard/apps/windows/t
    elemetry/config.json отсутствует,
46     выполняется попытка скопировать шаблон из пакета модуля.
47
48     Returns:
49         Path: Путь к файлу конфигурации в %APPDATA%.
50     """
51     appdata = os.environ.get('APPDATA') or os.environ.get('LOCALAPPDATA')
52     if appdata and os.path.exists(appdata):
53         base_dir = Path(appdata)
54     else:
55         base_dir = Path.home() / '.config'
56     target_path = base_dir / 'AI-Breadboard' / 'apps' / 'windows' / 'telemetry' / 'config.json'
57
58     if not target_path.exists():
59         bundled_template = Path(__file__).parent / 'config.json'
60
61         if bundled_template.is_file():
62             try:
63                 target_path.parent.mkdir(parents=True, exist_ok=True)

```

```

63 |             shutil.copy2(bundled_template, target_path)
64 |         except Exception:
65 |             return bundled_template
66 |     return target_path
67 |
68 |
69 | def parse_interval_to_seconds(val: Any, default: float = 5.0)
    | -> float:
70 |     """Преобразует строковое или числовое значение интервала в
    | секунды (float).
71 |
72 |     Поддерживает:
73 |         - Числа (int, float): 5 -> 5.0, 0.5 -> 0.5
74 |         - Строковые числа: "5", "5.0" -> 5.0
75 |         - Строки с единицами времени:
76 |             - Секунды: "5s", "5 seconds", "5 сек", "5 секунд"
    | -> 5.0
77 |             - Минуты: "1m", "1 minute", "5 minutes", "2 мин" -
    | > 60.0 / 300.0 / 120.0
78 |             - Часы: "1h", "1 hour", "6 hours", "1 час" -> 3600
    | .0 / 21600.0
79 |             - Дни: "1d", "1 day", "7 days", "1 день" -> 86400.
    | 0 / 604800.0
80 |
81 |     Args:
82 |         val: Значение интервала из конфигурации.
83 |         default: Значение по умолчанию в секундах.
84 |
85 |     Returns:
86 |         float: Интервал в секундах (минимум 0.001 при валидном
    | положительном значении).
87 |     """
88 |     if val is None:
89 |         return float(default)
90 |     if isinstance(val, (int, float)):
91 |         return max(0.001, float(val))
92 |     if isinstance(val, str):
93 |         val_str = val.strip().lower()

```

```

94         if not val_str:
95             return float(default)
96         try:
97             return max(0.001, float(val_str))
98         except ValueError:
99             pass
100         match = re.match(r"^([\d.]+\s*([a-zA-я]+)?$", val_str
101     )
102     if match:
103         try:
104             num = float(match.group(1))
105             unit = match.group(2) or "s"
106             if unit in ("s", "sec", "second", "seconds", "
сек", "секунд", "секунды", "секунда"):
107                 return max(0.001, num)
108             elif unit in ("m", "min", "minute", "minutes",
"мин", "минут", "минуты", "минута"):
109                 return max(0.001, num * 60.0)
110             elif unit in ("h", "hr", "hour", "hours", "ч",
"час", "часа", "часов"):
111                 return max(0.001, num * 3600.0)
112             elif unit in ("d", "day", "days", "д", "день",
"дня", "дней"):
113                 return max(0.001, num * 86400.0)
114         except ValueError:
115             pass
116     return float(default)
117
118 class TelemetryConfigManager:
119     """Менеджер конфигурации для централизованного управления
120     сенсорами и интервалами."""
121
122     def __init__(self, config_path: Optional[str] = None) -> N
123     one:
124         """Инициализирует менеджер конфигурации.
125
126         Args:

```

```

125         config_path: Путь к файлу конфигурации (по умолчанию: %APPDATA%/AI-Breadboard/apps/windows/telemetry/config.json).
126         """
127         if config_path:
128             self._config_path = str(config_path)
129         else:
130             self._config_path = str(get_default_telemetry_config_path())
131         self._config: Dict[str, Any] = {}
132         self._sensors_config: Dict[str, Dict[str, Any]] = {}
133         self._loggers_config: Dict[str, Dict[str, Any]] = {}
134         self._default_interval = 5.0
135         self._fast_interval = 30.0
136         self._fast_duration_days = 2
137         self._aggregation_interval = 3600.0
138         self._heavy_interval = 60.0
139         self._heavy_disk_scan_interval = 43200.0
140         self._heavy_mode_max_duration_days = 5.0
141         self._heavy_mode_auto_switch_enabled = True
142         self._mode = 'hybrid'
143         self._telemetry_mode = 'telemetry'
144         self._standard_flush_interval_seconds = 30.0
145         self._tc_flush_interval_seconds = 5.0
146         self._tc_mode_timeout_seconds = 300.0
147         self._tc_mode_activated_at: Optional[float] = None
148         self._max_db_size_mb = 100.0
149         self._retention_days = 7
150         self._db_cleanup_interval_seconds = 300.0
151         self._flush_interval_seconds = 30.0
152         self._auto_vacuum_enabled = True
153         self._last_mtime_ns: int = 0
154         self._load_config()
155
156     def _load_config(self) -> None:
157         """Загружает конфигурацию из JSON файла с фиксацией времени модификации."""
158         try:

```



```

159         cfg_path = Path(self._config_path)
160         if cfg_path.is_file():
161             self._last_mtime_ns = cfg_path.stat().st_mtime
162             _ns
163             with open(self._config_path, 'r', encoding='utf-8'
164 ) as f:
165                 new_config = json.load(f)
166
167                 self._config = new_config
168                 if 'sensors' in self._config:
169                     self._sensors_config = self._config.get('senso
170 rs', {})
171                 elif 'telemetry' in self._config and 'sensors' in
172 self._config['telemetry']:
173                     self._sensors_config = self._config['telemetry
174 '].get('sensors', {})
175                 else:
176                     self._sensors_config = {}
177
178                 self._loggers_config = self._config.get('loggers',
179                 {})
180
181                 raw_interval = self._config.get('interval_seconds'
182 , self._config.get('interval', self._config.get('telemetry', {
183 }).get('interval_seconds', 5.0)))
184                 self._default_interval = parse_interval_to_seconds
185 (raw_interval, 5.0)
186
187                 raw_heavy = self._config.get('heavy_interval_secon
188 ds', self._config.get('heavy_interval', self._config.get('tele
189 metry', {})).get('heavy_interval_seconds', 60.0)))
190                 self._heavy_interval = parse_interval_to_seconds(r
191 aw_heavy, 60.0)
192
193                 raw_disk_interval = self._config.get('heavy_disk_s
194 can_interval_seconds')
195                 self._heavy_disk_scan_interval = parse_interval_to
196 _seconds(raw_disk_interval, 43200.0)

```

```
183
184         raw_fast_interval = self._config.get('fast_interva
185         self._fast_interval = parse_interval_to_seconds(ra
186         w_fast_interval, 30.0)
187
188         self._fast_duration_days = int(self._config.get('f
189         ast_duration_days', 2))
190
191         raw_agg = self._config.get('aggregation_interval_s
192         econds', self._config.get('rollup_interval_seconds'))
193         self._aggregation_interval = parse_interval_to_sec
194         onds(raw_agg, 3600.0)
195
196         raw_max_days = self._config.get('heavy_mode_max_du
197         ration_days')
198         self._heavy_mode_max_duration_days = float(raw_max
199         _days) if raw_max_days is not None else 5.0
200
201         self._heavy_mode_auto_switch_enabled = bool(self._
202         config.get('heavy_mode_auto_switch_enabled', True))
203         self._mode = str(self._config.get('mode', self._co
204         nfig.get('telemetry', {})).get('mode', 'hybrid')).lower()
205
206         raw_tel_mode = str(self._config.get('telemetry_mod
207         e', self._config.get('profile', 'telemetry'))).lower()
208         self._telemetry_mode = raw_tel_mode if raw_tel_mod
209         e in ('telemetry', 'tc') else 'telemetry'
210
211         raw_std_flush = self._config.get('standard_flush_i
212         nterval_seconds', self._config.get('flush_interval_seconds', 3
213         0.0))
214         self._standard_flush_interval_seconds = parse_inte
215         rval_to_seconds(raw_std_flush, 30.0)
216
217         raw_tc_flush = self._config.get('tc_flush_interval
218         _seconds', 5.0)
219         self._tc_flush_interval_seconds = parse_interval_t
```

```

    o_seconds(raw_tc_flush, 5.0)
206
207         raw_tc_timeout = self._config.get('tc_mode_timeout
    _seconds', self._config.get('tc_timeout_seconds', 300.0))
208         self._tc_mode_timeout_seconds = parse_interval_to_
    seconds(raw_tc_timeout, 300.0)
209
210         raw_tc_act = self._config.get('tc_mode_activated_a
    t')
211         try:
212             self._tc_mode_activated_at = float(raw_tc_act)
    if raw_tc_act is not None else None
213         except (ValueError, TypeError):
214             self._tc_mode_activated_at = None
215
216         raw_max_db = self._config.get('max_db_size_mb', se
    lf._config.get('max_file_size_mb', 100.0))
217         self._max_db_size_mb = float(raw_max_db) if raw_ma
    x_db is not None else 100.0
218
219         raw_retention = self._config.get('retention_days',
    7)
220         self._retention_days = int(raw_retention) if raw_r
    etention is not None else 7
221
222         raw_cleanup_interval = self._config.get('db_cleanu
    p_interval_seconds')
223         self._db_cleanup_interval_seconds = parse_interval
    _to_seconds(raw_cleanup_interval, 300.0)
224
225         if self._telemetry_mode == 'tc':
226             self._flush_interval_seconds = self._tc_flush_
    interval_seconds
227         else:
228             self._flush_interval_seconds = self._standard_
    flush_interval_seconds
229
230         self._auto_vacuum_enabled = bool(self._config.get(

```

```

    'auto_vacuum_enabled', True))
231         except FileNotFoundError:
232             if not self._config:
233                 self._config = {}
234                 self._sensors_config = {}
235                 self._loggers_config = {}
236                 self._default_interval = 5.0
237                 self._heavy_interval = 60.0
238                 self._heavy_disk_scan_interval = 43200.0
239                 self._heavy_mode_max_duration_days = 5.0
240                 self._heavy_mode_auto_switch_enabled = True
241                 self._mode = 'hybrid'
242                 self._max_db_size_mb = 50.0
243                 self._retention_days = 7
244                 self._db_cleanup_interval_seconds = 300.0
245                 self._flush_interval_seconds = 30.0
246                 self._auto_vacuum_enabled = True
247             except (json.JSONDecodeError, OSError, PermissionError
    ) as e:
248                 if not self._config:
249                     raise ValueError(f'Ошибка парсинга {self._conf
    ig_path}: {e}')
250                 logger.warning(f'Временная ошибка считывания {self
    ._config_path} на лету: {e}')
251
252     @property
253     def config_path(self) -> str:
254         """Возвращает путь к активному файлу конфигурации."""
255         return self._config_path
256
257     def check_and_reload(self) -> bool:
258         """Проверяет модификацию файла config.json на диске и
    перезагружает параметры 'на лету'.
259
260         Returns:
261             bool: True если конфигурация изменилась и успешно
    перезагружена, иначе False.
262         """

```

```

263         try:
264             cfg_path = Path(self._config_path)
265             if not cfg_path.is_file():
266                 return False
267             current_mtime_ns = cfg_path.stat().st_mtime_ns
268             if current_mtime_ns != self._last_mtime_ns:
269                 self._load_config()
270                 return True
271         except Exception as e:
272             logger.debug(f'Проверка модификации config.json: {
e}')
273         return False
274
275     def reload(self) -> None:
276         """Принудительно перезагружает конфигурацию из файла."
""
277         self._load_config()
278
279     def get_fast_interval(self) -> float:
280         """Возвращает интервал быстрого режима (сек)."""
281         return getattr(self, '_fast_interval', 30.0)
282
283     def get_fast_duration_days(self) -> int:
284         """Возвращает длительность быстрого режима в днях."""
285         return getattr(self, '_fast_duration_days', 2)
286
287     def get_aggregation_interval(self) -> float:
288         """Возвращает интервал агрегации (сек)."""
289         return getattr(self, '_aggregation_interval', 3600.0)
290
291     def get_mode(self) -> str:
292         """Возвращает режим сбора: minimal, hybrid, full."""
293         return self._mode if self._mode in ('minimal', 'hybrid
', 'full') else 'hybrid'
294
295     def get_interval_seconds(self) -> float:
296         """Возвращает быстрый базовый интервал сбора телеметри
и."""

```

```

297 |         return self._default_interval
298 |
299 |     def get_heavy_interval_seconds(self) -> float:
300 |         """Возвращает периодический интервал тяжелых сенсоров.
301 |         """
302 |         return self._heavy_interval
303 |
304 |     def get_heavy_disk_scan_interval(self) -> float:
305 |         """Возвращает интервал тяжелой проверки дисков (в секундах)."""
306 |         return getattr(self, '_heavy_disk_scan_interval', 43200.0)
307 |
308 |     def get_heavy_mode_max_duration_days(self) -> float:
309 |         """Возвращает максимальную непрерывную длительность тяжелого режима (в днях)."""
310 |         return getattr(self, '_heavy_mode_max_duration_days', 5.0)
311 |
312 |     def is_heavy_auto_switch_enabled(self) -> bool:
313 |         """Возвращает флаг разрешения автопереключения тяжелого режима в легкий."""
314 |         return getattr(self, '_heavy_mode_auto_switch_enabled', True)
315 |
316 |     def get_heavy_collectors(self) -> Dict[str, bool]:
317 |         """Возвращает словарь флагов включения тяжелых коллекторов."""
318 |         default_heavy = {
319 |             'hardware_sensors': True,
320 |             'deep_disk_smart': True,
321 |             'system_snapshot': True,
322 |             'hardware_tree': True,
323 |         }
324 |         custom = self._config.get('heavy_collectors', {})
325 |         if isinstance(custom, dict):
326 |             default_heavy.update(custom)
327 |         return default_heavy

```

```

327
328     def get_top_processes(self) -> int:
329         """Возвращает число сохраняемых процессов с наибольшей
нагрузкой."""
330         return int(self._config.get('top_processes', 25))
331
332     def get_buffer_mode(self) -> str:
333         """Возвращает режим буферизации телеметрии ('memory',
'file', 'direct')."""
334         mode = str(self._config.get('buffer_mode', 'memory')).
lower()
335         return mode if mode in ('memory', 'file', 'direct') el
se 'memory'
336
337     def get_buffer_size(self) -> int:
338         """Возвращает максимальный размер буфера перед принуди
тельным сбросом в БД."""
339         try:
340             val = int(self._config.get('buffer_size', 50))
341             return max(1, val)
342         except (ValueError, TypeError):
343             return 50
344
345     def get_telemetry_mode(self) -> str:
346         """Возвращает режим телеметрии: 'telemetry' (стандартн
ый) или 'tc' (реальное время)."""
347         self.check_tc_mode_auto_switch()
348         return getattr(self, '_telemetry_mode', 'telemetry')
349
350     def check_tc_mode_auto_switch(self) -> bool:
351         """Проверяет таймаут активного режима 'tc' (реальное в
ремя) и переключает в 'telemetry'.
352
353         Returns:
354             bool: True если произошло автопереключение в 'tele
metry', иначе False.
355         """
356         if getattr(self, '_telemetry_mode', 'telemetry') == 't

```

```

c' and getattr(self, '_tc_mode_activated_at', None):
357         import time
358         elapsed = time.time() - self._tc_mode_activated_at
359         timeout = getattr(self, '_tc_mode_timeout_seconds'
, 300.0)
360         if elapsed >= timeout:
361             self._telemetry_mode = 'telemetry'
362             self._tc_mode_activated_at = None
363             self._flush_interval_seconds = getattr(self, '_standard_flush_interval_seconds', 30.0)
364             self.save_config({
365                 'telemetry_mode': 'telemetry',
366                 'tc_mode_activated_at': None
367             })
368             logger.info(
369                 f" [Телеметрия] Режим реального времени '
tc' завершен по таймауту ({int(timeout)}с). "
370                 f"Автоматически переключен в стандартный р
ежим 'telemetry' (flush: {self._flush_interval_seconds}с, max_
db: {self.get_max_db_size_mb()}МБ)"
371             )
372             return True
373         return False
374
375     def activate_tc_mode(self, duration_seconds: Optional[float] = None, flush_interval_seconds: Optional[float] = None) ->
bool:
376         """Активирует режим отображения телеметрии реального в
ремени ('tc') с задержкой сброса в N сек.
377
378         Args:
379             duration_seconds: Длительность режима ТС в секунда
х до автопереключения в telemetry (по умолчанию 300с = 5 мин).
380             flush_interval_seconds: Интервал сброса в БД в сек
ундах (по умолчанию 5.0с).
381
382         Returns:
383             bool: True при успешной активации.

```



```

384 |         """
385 |         import time
386 |         self._telemetry_mode = 'tc'
387 |         self._tc_mode_activated_at = time.time()
388 |         if flush_interval_seconds is not None:
389 |             self._tc_flush_interval_seconds = max(0.1, float(f
| lush_interval_seconds))
390 |         if duration_seconds is not None:
391 |             self._tc_mode_timeout_seconds = max(1.0, float(dur
| ation_seconds))
392 |         self._flush_interval_seconds = self._tc_flush_interval
| _seconds
393 |
394 |         updates: Dict[str, Any] = {
395 |             'telemetry_mode': 'tc',
396 |             'tc_mode_activated_at': self._tc_mode_activated_at
| ,
397 |             'tc_flush_interval_seconds': self._tc_flush_interv
| al_seconds,
398 |             'tc_mode_timeout_seconds': self._tc_mode_timeout_s
| econds,
399 |         }
400 |         logger.info(f" [Телеметрия] Активирован режим реально
| го времени 'tc' (flush: {self._tc_flush_interval_seconds}с, та
| ймаут: {self._tc_mode_timeout_seconds}с)")
401 |         return self.save_config(updates)
402 |
403 |     def deactivate_tc_mode(self) -> bool:
404 |         """Деактивирует режим 'tc' и возвращает телеметрию в с
| тандартный режим 'telemetry'."""
405 |         self._telemetry_mode = 'telemetry'
406 |         self._tc_mode_activated_at = None
407 |         self._flush_interval_seconds = getattr(self, '_standar
| d_flush_interval_seconds', 30.0)
408 |         updates = {
409 |             'telemetry_mode': 'telemetry',
410 |             'tc_mode_activated_at': None,
411 |         }

```

```

412         logger.info(f" [Телеметрия] Переключено в стандартны
й режим 'telemetry' (flush: {self._flush_interval_seconds}с)")
413         return self.save_config(updates)
414
415     def get_standard_flush_interval_seconds(self) -> float:
416         """Возвращает стандартный интервал сброса буфера в БД
(по умолчанию 30с)."""
417         return getattr(self, '_standard_flush_interval_seconds
', 30.0)
418
419     def get_tc_flush_interval_seconds(self) -> float:
420         """Возвращает интервал сброса в режиме реального време
ни ТС (по умолчанию 5с)."""
421         return getattr(self, '_tc_flush_interval_seconds', 5.0
)
422
423     def get_tc_mode_timeout_seconds(self) -> float:
424         """Возвращает таймаут действия режима ТС до автоперекл
ючения в стандартный (в секундах, по умолчанию 300с)."""
425         return getattr(self, '_tc_mode_timeout_seconds', 300.0
)
426
427     def get_flush_interval_seconds(self) -> float:
428         """Возвращает периодический интервал сброса буфера в Б
Д с учетом активного режима."""
429         self.check_tc_mode_auto_switch()
430         if getattr(self, '_telemetry_mode', 'telemetry') == 't
c':
431             return getattr(self, '_tc_flush_interval_seconds',
5.0)
432         return getattr(self, '_standard_flush_interval_seconds
', 30.0)
433
434     def get_max_db_size_mb(self) -> float:
435         """Возвращает максимальный разрешенный размер SQLite б
азы данных (в МБ, по умолчанию 100.0)."""
436         try:
437             val = float(self._config.get('max_db_size_mb', sel

```

```

    f._config.get('max_file_size_mb', 100.0)))
438         return max(1.0, val)
439     except (ValueError, TypeError):
440         return 100.0
441
442     def get_retention_days(self) -> int:
443         """Возвращает срок хранения сырых записей телеметрии (
444 в днях)."""
445         try:
446             val = int(self._config.get('retention_days', 7))
447             return max(1, val)
448         except (ValueError, TypeError):
449             return 7
450
451     def get_db_cleanup_interval_seconds(self) -> float:
452         """Возвращает интервал периодической проверки и очистки
453 и БД (в секундах)."""
454         return getattr(self, '_db_cleanup_interval_seconds', 3
455 00.0)
456
457     def is_auto_vacuum_enabled(self) -> bool:
458         """Возвращает флаг разрешения авто-вакуума (VACUUM) SQ
459 Lite при усечении."""
460         return bool(self._config.get('auto_vacuum_enabled', Tr
461 ue))
462
463     def get_process_mode(self) -> str:
464         """Возвращает режим фильтрации процессов ('top_n' или
465 'all')."""
466         return str(self._config.get('process_mode', 'top_n')).
lower()
467
468     def get_top_processes_limit(self) -> int:
469         """Возвращает настроенный лимит количества Топ-процесс
470 ов (по умолчанию 10)."""
471         try:
472             val = int(self._config.get('top_processes', 10))
473             return max(1, val)

```

```

467         except (ValueError, TypeError):
468             return 10
469
470     def get_effective_process_limit(self) -> int:
471         """Возвращает эффективный лимит процессов для сохранен
ия в БД телеметрии."""
472         pm = self.get_process_mode()
473         if pm == 'all':
474             return 500
475         return self.get_top_processes_limit()
476
477     def get_sensor_config(self, sensor_name: str) -> Dict[str,
Any]:
478         """Возвращает конфигурацию конкретного сенсора."""
479         return self._sensors_config.get(sensor_name, {'enabled
': False})
480
481     def is_sensor_enabled(self, sensor_name: str) -> bool:
482         """Проверяет, включен ли сенсор."""
483         config = self.get_sensor_config(sensor_name)
484         return bool(config.get('enabled', False))
485
486     def get_sensor_interval(self, sensor_name: str) -> float:
487         """Возвращает интервал опроса сенсора в секундах."""
488         config = self.get_sensor_config(sensor_name)
489         raw = config.get('interval_seconds', config.get('inter
val', self._default_interval))
490         return parse_interval_to_seconds(raw, self._default_in
terval)
491
492     def get_sensor_metrics(self, sensor_name: str) -> List[str
]:
493         """Возвращает список метрик для сенсора."""
494         config = self.get_sensor_config(sensor_name)
495         return config.get('metrics', [])
496
497     def get_all_sensor_names(self) -> List[str]:
498         """Возвращает список всех имен сенсоров."""

```

```

499         return list(self._sensors_config.keys())
500
501     def get_enabled_sensors(self) -> List[str]:
502         """Возвращает список включенных сенсоров."""
503         return [name for name, config in self._sensors_config.
504 items() if config.get('enabled', False)]
505
506     def get_logger_interval(self, logger_name: str, default: f
507 loat = 60.0) -> float:
508         """Возвращает интервал опроса логгера приложения в сек
509 ундах."""
510         logger_info = self._loggers_config.get(logger_name, {})
511         )
512         if isinstance(logger_info, dict):
513             raw = logger_info.get('interval', self._config.get
514 ('default_interval'))
515             return parse_interval_to_seconds(raw, default)
516         return float(default)
517
518     def is_logger_enabled(self, logger_name: str, default: boo
519 l = True) -> bool:
520         """Проверяет активность логгера приложения."""
521         logger_info = self._loggers_config.get(logger_name, {})
522         )
523         if isinstance(logger_info, dict):
524             return bool(logger_info.get('enabled', default))
525         return default
526
527     def get_all_intervals(self) -> Dict[str, Any]:
528         """Возвращает сводный словарь всех настроенных интерва
529 лов системы."""
530         return {
531             'telemetry_mode': self.get_telemetry_mode(),
532             'default_interval_seconds': self.get_interval_seco
533 nds(),
534             'heavy_interval_seconds': self.get_heavy_interval_
535 seconds(),
536             'fast_interval_seconds': self.get_fast_interval(),

```

```

527         'aggregation_interval_seconds': self.get_aggregati
on_interval(),
528         'heavy_disk_scan_interval_seconds': self.get_heavy
_disk_scan_interval(),
529         'db_cleanup_interval_seconds': self.get_db_cleanup
_interval_seconds(),
530         'flush_interval_seconds': self.get_flush_interval_
seconds(),
531         'standard_flush_interval_seconds': self.get_standar
d_flush_interval_seconds(),
532         'tc_flush_interval_seconds': self.get_tc_flush_int
erval_seconds(),
533         'tc_mode_timeout_seconds': self.get_tc_mode_timeou
t_seconds(),
534         'sensors': {
535             name: self.get_sensor_interval(name)
536             for name in self.get_all_sensor_names()
537         },
538         'loggers': {
539             name: self.get_logger_interval(name)
540             for name in self._loggers_config.keys()
541         }
542     }
543
544     def get_general_config(self) -> Dict[str, Any]:
545         """Возвращает общую конфигурацию (без сенсоров)."""
546         return {k: v for k, v in self._config.items() if k !=
'sensors'}
547
548     def get_config(self) -> Dict[str, Any]:
549         """Возвращает полную конфигурацию телеметрии."""
550         return self._config.copy()
551
552     @property
553     def config(self) -> Dict[str, Any]:
554         """Свойство для получения полной конфигурации."""
555         return self.get_config()
556

```

```

557     def save_config(self, new_config: Optional[Dict[str, Any]]
= None) -> bool:
558         """Сохраняет текущую или переданную конфигурацию в фай
л.
559
560         Args:
561             new_config: Опциональный словарь новых параметров.
562
563         Returns:
564             bool: True при успешном сохранении.
565         """
566         if new_config:
567             self._config.update(new_config)
568             if 'sensors' in new_config:
569                 self._sensors_config = new_config['sensors']
570             if 'loggers' in new_config:
571                 self._loggers_config = new_config['loggers']
572             if 'interval_seconds' in new_config:
573                 self._default_interval = parse_interval_to_sec
onds(new_config['interval_seconds'], 5.0)
574             if 'heavy_interval_seconds' in new_config:
575                 self._heavy_interval = parse_interval_to_secon
ds(new_config['heavy_interval_seconds'], 60.0)
576             if 'heavy_disk_scan_interval_seconds' in new_conf
ig:
577                 self._heavy_disk_scan_interval = parse_interva
l_to_seconds(new_config['heavy_disk_scan_interval_seconds'], 4
3200.0)
578             if 'fast_interval_seconds' in new_config:
579                 self._fast_interval = parse_interval_to_second
s(new_config['fast_interval_seconds'], 30.0)
580             if 'aggregation_interval_seconds' in new_config:
581                 self._aggregation_interval = parse_interval_to
_seconds(new_config['aggregation_interval_seconds'], 3600.0)
582             if 'heavy_mode_max_duration_days' in new_config:
583                 self._heavy_mode_max_duration_days = float(new
_config['heavy_mode_max_duration_days'])
584             if 'heavy_mode_auto_switch_enabled' in new_config:

```

```
585         self._heavy_mode_auto_switch_enabled = bool(new_config['heavy_mode_auto_switch_enabled'])
586         if 'mode' in new_config:
587             self._mode = str(new_config['mode']).lower()
588         if 'telemetry_mode' in new_config:
589             self._telemetry_mode = str(new_config['telemetry_mode']).lower()
590         if 'standard_flush_interval_seconds' in new_config:
591             self._standard_flush_interval_seconds = parse_interval_to_seconds(new_config['standard_flush_interval_seconds'], 30.0)
592         if 'tc_flush_interval_seconds' in new_config:
593             self._tc_flush_interval_seconds = parse_interval_to_seconds(new_config['tc_flush_interval_seconds'], 5.0)
594         if 'tc_mode_timeout_seconds' in new_config:
595             self._tc_mode_timeout_seconds = parse_interval_to_seconds(new_config['tc_mode_timeout_seconds'], 300.0)
596         if 'tc_mode_activated_at' in new_config:
597             self._tc_mode_activated_at = float(new_config['tc_mode_activated_at']) if new_config['tc_mode_activated_at'] is not None else None
598         if 'max_db_size_mb' in new_config:
599             self._max_db_size_mb = float(new_config['max_db_size_mb'])
600         if 'retention_days' in new_config:
601             self._retention_days = int(new_config['retention_days'])
602         if 'db_cleanup_interval_seconds' in new_config:
603             self._db_cleanup_interval_seconds = parse_interval_to_seconds(new_config['db_cleanup_interval_seconds'], 300.0)
604         if 'flush_interval_seconds' in new_config:
605             self._flush_interval_seconds = parse_interval_to_seconds(new_config['flush_interval_seconds'], 30.0)
606         if 'auto_vacuum_enabled' in new_config:
607             self._auto_vacuum_enabled = bool(new_config['auto_vacuum_enabled'])
```



```
608 |         try:
609 |             target_path = Path(self._config_path)
610 |             target_path.parent.mkdir(parents=True, exist_ok=True)
611 |             with open(target_path, 'w', encoding='utf-8') as f:
612 |                 json.dump(self._config, f, indent=2, ensure_ascii=False)
613 |             if target_path.is_file():
614 |                 self._last_mtime_ns = target_path.stat().st_mtime_ns
615 |             return True
616 |         except Exception:
617 |             return False
```

w64_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - W64 Collector
4 # =====
5 # Description:
6 #   AI Windows 64-bit Collector - модуль сбора детальных системных событий телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.w64_collector import AIW64Collector
11 #
12 #       service = AIW64Collector()
13 #
14 # File: w64_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 04:48:00
20 # =====
21
22 from __future__ import annotations
23 """AI Windows 64-bit Collector - модуль сбора детальных системных событий телеметрии."""
24
25 import json
26 import os
27 import re
28 import subprocess
29 import threading
```

```

30 import time
31 from datetime import datetime, timezone
32 from pathlib import Path
33 from typing import Any, Callable, Dict, List, Optional
34 from logger import logger
35 from .models import W64CollectorStatus, W64SystemEvent
36 from .sqlite import TelemetryStorage
37
38 _file_write_lock = threading.Lock()
39
40 def _append_jsonl_safely(file_path: Path, data: Dict[str, Any]
41     , retries: int = 4, retry_delay: float = 0.05) -> bool:
42     """Безопасная запись JSONL строки с повторными попытками п
43     ри блокировке файла.
44
45     Args:
46         file_path: Путь к целевому файлу JSONL.
47         data: Словарь с данными для сериализации в JSON.
48         retries: Количество попыток записи при ошибке доступа.
49         retry_delay: Задержка между попытками (сек).
50
51     Returns:
52         bool: True если запись успешна, иначе исключение.
53     """
54     line = json.dumps(data, ensure_ascii=False) + '\n'
55     for attempt in range(retries):
56         try:
57             with _file_write_lock:
58                 with open(file_path, 'a', encoding='utf-8') as
59                     f:
60                         f.write(line)
61                     return True
62         except (PermissionError, OSError) as ex:
63             if attempt < retries - 1:
64                 time.sleep(retry_delay * (attempt + 1))
65             else:
66                 raise ex
67     return False

```

```

65 |
66 | class AIW64Collector:
67 |     """Максимально детальный сборщик событий Windows 64-bit дл
    я подсистемы телеметрии.
68 |
69 |     Фиксирует непрерывные изменения системного состояния:
70 |     - Процессы (создание, завершение, активность)
71 |     - Файловая система (изменения в отслеживаемых путях)
72 |     - Реестр Windows (модификации ключей)
73 |     - Сетевая активность (новые соединения и открытые порты)
74 |     """
75 |
76 |     def __init__(
77 |         self,
78 |         log_dir: Optional[str] = None,
79 |         enable_file_monitoring: bool = True,
80 |         enable_process_monitoring: bool = True,
81 |         enable_registry_monitoring: bool = True,
82 |         enable_network_monitoring: bool = True,
83 |         enable_event_log_monitoring: bool = True,
84 |         monitored_paths: Optional[List[str]] = None,
85 |         poll_interval_sec: float = 1.0,
86 |         storage: Optional[TelemetryStorage] = None,
87 |         on_event_callback: Optional[Callable[[Dict[str, Any]],
    None]] = None,
88 |     ) -> None:
89 |         """Инициализация сборщика системных событий W64.
90 |
91 |         Args:
92 |             log_dir: Директория для сохранения JSONL логов (по
    умолчанию logs/ai_w64_logs).
93 |             enable_file_monitoring: Флаг мониторинга изменений
    файлов.
94 |             enable_process_monitoring: Флаг мониторинга процес
    сов.
95 |             enable_registry_monitoring: Флаг мониторинга реест
    ра.
96 |             enable_network_monitoring: Флаг мониторинга сетевы

```

```

    х соединений.
97         enable_event_log_monitoring: Флаг мониторинга журн
алов Windows.
98         monitored_paths: Список путей файловой системы для
отслеживания.
99         poll_interval_sec: Интервал между циклами сканиров
ания изменений (сек).
100        storage: Экземпляр хранилища TelemetryStorage для
сохранения в SQLite.
101        on_event_callback: Опциональный callback-обработчи
к каждого зафиксированного события.
102        """
103        if log_dir is None:
104            appdata = os.environ.get('LOCALAPPDATA', os.path.e
xpanduser('~\\AppData\\Local'))
105            self.log_dir = str(Path(appdata) / 'AI-Breadboard'
/ 'apps' / 'windows' / 'telemetry' / 'logs' / 'w64')
106        else:
107            self.log_dir = log_dir
108
109        Path(self.log_dir).mkdir(parents=True, exist_ok=True)
110        self.enable_file_monitoring = enable_file_monitoring
111        self.enable_process_monitoring = enable_process_monito
ring
112        self.enable_registry_monitoring = enable_registry_moni
toring
113        self.enable_network_monitoring = enable_network_monito
ring
114        self.enable_event_log_monitoring = enable_event_log_mo
nitoring
115        self.poll_interval = max(0.1, poll_interval_sec)
116        self.storage = storage or TelemetryStorage.get_instanc
e()
117        self.on_event_callback = on_event_callback
118
119        self.monitored_paths = monitored_paths or [
120            os.path.expandvars(r'%USERPROFILE%\Documents'),
121            os.path.expandvars(r'%USERPROFILE%\Downloads'),

```

```

122         os.path.expandvars(r'%USERPROFILE%\Desktop'),
123         r'C:\Program Files',
124         r'C:\Program Files (x86)',
125         r'C:\Windows\System32',
126         r'C:\Windows\SysWOW64',
127     ]
128
129     self._running = False
130     self._thread: Optional[threading.Thread] = None
131     self._stop_event = threading.Event()
132     self._event_counter = 0
133     self._last_event_time: Optional[str] = None
134     self._last_baseline: Dict[str, Any] = {}
135     logger.debug(f'AIW64Collector инициализирован. Директо
136     рия логов: {self.log_dir}')
137
138     def _get_log_file(self, event_type: str) -> Path:
139         """Получить путь к файлу ротации лога для типа события
140         .
141
142         Args:
143             event_type: Название категории/типа события.
144
145         Returns:
146             Path: Путь к JSONL файлу.
147         """
148         date_str = datetime.now(timezone.utc).strftime('%Y-%m-
149         %d')
150         return Path(self.log_dir) / f'{event_type}_{date_str}.
151         jsonl'
152
153     def _log_event(self, event_type: str, event_data: Dict[str
154     , Any]) -> None:
155         """Записать событие в потоковый JSONL файл и базу данн
156         ых SQLite.
157
158         Args:
159             event_type: Тип события.

```

```

154         event_data: Словарь с атрибутами события.
155         """
156         self._event_counter += 1
157         now_iso = datetime.now(timezone.utc).isoformat()
158         event_data['event_id'] = f'evt_{int(time.time())}_{sel
f._event_counter}'
159         event_data['timestamp'] = now_iso
160         event_data['event_type'] = event_type
161         self._last_event_time = now_iso
162
163         # 1. Запись в JSONL
164         try:
165             log_file = self._get_log_file(event_type)
166             _append_jsonl_safely(log_file, event_data)
167         except Exception as ex:
168             logger.debug(f'Не удалось записать JSONL лог событ
ия {event_type}: {ex}')
169
170         # 2. Сохранение в SQLite TelemetryStorage
171         if self.storage:
172             try:
173                 self.storage.save_w64_event(event_data, provid
er='w64_collector')
174             except Exception as ex:
175                 logger.debug(f'Ошибка сохранения W64 события в
БД: {ex}')
176
177         # 3. Передача в callback при наличии
178         if self.on_event_callback:
179             try:
180                 self.on_event_callback(event_data)
181             except Exception as ex:
182                 logger.debug(f'Ошибка в on_event_callback: {ex
}')
183
184     def start(self) -> bool:
185         """Запустить фоновый поток мониторинга событий.
186

```

```

187 |         Returns:
188 |             bool: True если сборщик успешно запущен, False есл
и уже работал.
189 |             """
190 |         if self._running:
191 |             logger.warning('AIW64Collector уже запущен')
192 |             return False
193 |
194 |         self._running = True
195 |         self._stop_event.clear()
196 |         self._update_baseline()
197 |         self._thread = threading.Thread(target=self._monitorin
g_loop, name='AIW64CollectorWorker', daemon=True)
198 |         self._thread.start()
199 |         logger.info('AIW64Collector успешно запущен')
200 |         return True
201 |
202 |     def stop(self) -> bool:
203 |         """Остановить фоновый поток мониторинга.
204 |
205 |         Returns:
206 |             bool: True если сборщик остановлен, False если не
был активен.
207 |             """
208 |         if not self._running:
209 |             return False
210 |
211 |         self._running = False
212 |         self._stop_event.set()
213 |         if self._thread and self._thread.is_alive():
214 |             self._thread.join(timeout=5.0)
215 |         logger.info(f'AIW64Collector остановлен. Всего зафокси
ровано событий: {self._event_counter}')
216 |         return True
217 |
218 |     def _update_baseline(self) -> None:
219 |         """Сформировать начальный снимок состояния системы для
сравнения. """

```



```

220         self._last_baseline = {
221             'processes': self._get_current_processes(),
222             'network_connections': self._get_current_network_c
onnections(),
223             'registry_keys': self._get_registry_keys(),
224             'file_hashes': self._get_file_hashes(),
225         }
226
227     def _monitoring_loop(self) -> None:
228         """Основной рабочий цикл мониторинга и выявления измен
ений."""
229         last_check = time.time()
230         while not self._stop_event.is_set():
231             try:
232                 current_time = time.time()
233                 if current_time - last_check >= self.poll_inte
rval:
234                     current_state = {
235                         'processes': self._get_current_process
es(),
236                         'network_connections': self._get_curre
nt_network_connections(),
237                         'registry_keys': self._get_registry_ke
ys(),
238                         'file_hashes': self._get_file_hashes()
,
239                     }
240                     self._detect_changes(self._last_baseline,
current_state)
241                     self._last_baseline = current_state
242                     last_check = current_time
243
244                     self._stop_event.wait(timeout=min(0.5, self.po
ll_interval))
245             except Exception as ex:
246                 logger.error(f'Ошибка в цикле мониторинга AIW6
4Collector: {ex}')
247

```

```

248     def _detect_changes(self, old_state: Dict[str, Any], new_s
tate: Dict[str, Any]) -> None:
249         """Сравнить предыдущее и текущее состояние и залогиров
ать изменения.
250
251         Args:
252             old_state: Предыдущий системный срез.
253             new_state: Текущий системный срез.
254         """
255         # Мониторинг процессов
256         if self.enable_process_monitoring:
257             old_procs = {p['pid']: p for p in old_state.get('p
rocesses', [])}
258             new_procs = {p['pid']: p for p in new_state.get('p
rocesses', [])}
259
260             for pid, proc in new_procs.items():
261                 if pid not in old_procs:
262                     self._log_event('process_start', proc)
263
264             for pid, proc in old_procs.items():
265                 if pid not in new_procs:
266                     self._log_event('process_stop', proc)
267
268         # Мониторинг сетевых соединений
269         if self.enable_network_monitoring:
270             old_conns = {c.get('key'): c for c in old_state.ge
t('network_connections', []) if c.get('key')}
271             new_conns = {c.get('key'): c for c in new_state.ge
t('network_connections', []) if c.get('key')}
272
273             for key, conn in new_conns.items():
274                 if key not in old_conns:
275                     self._log_event('network_open', conn)
276
277             for key, conn in old_conns.items():
278                 if key not in new_conns:
279                     self._log_event('network_close', conn)

```

```
280
281     # Мониторинг файлов
282     if self.enable_file_monitoring:
283         old_files = old_state.get('file_hashes', {})
284         new_files = new_state.get('file_hashes', {})
285
286         for path, h in new_files.items():
287             if path not in old_files:
288                 self._log_event('file_created', {'path': path, 'hash': h})
289             elif old_files[path] != h:
290                 self._log_event('file_modified', {'path': path, 'old_hash': old_files[path], 'new_hash': h})
291
292         for path, h in old_files.items():
293             if path not in new_files:
294                 self._log_event('file_deleted', {'path': path, 'hash': h})
295
296     # Мониторинг реестра
297     if self.enable_registry_monitoring:
298         old_reg = old_state.get('registry_keys', {})
299         new_reg = new_state.get('registry_keys', {})
300
301         for key, val in new_reg.items():
302             if key not in old_reg:
303                 self._log_event('registry_created', {'key': key, 'value': val})
304             elif old_reg[key] != val:
305                 self._log_event('registry_modified', {'key': key, 'old_value': old_reg[key], 'new_value': val})
306
307         for key, val in old_reg.items():
308             if key not in new_reg:
309                 self._log_event('registry_deleted', {'key': key, 'value': val})
310
311     def _get_current_processes(self) -> List[Dict[str, Any]]:
```

```

312 |         """Получить текущий список активных процессов в систем
    | e."""
313 |         processes = []
314 |         try:
315 |             import psutil
316 |             for proc in psutil.process_iter(['pid', 'name', 'e
    | xe', 'cmdline', 'username', 'create_time']):
317 |                 try:
318 |                     info = proc.info
319 |                     processes.append({
320 |                         'pid': info.get('pid'),
321 |                         'name': info.get('name') or '',
322 |                         'path': info.get('exe') or '',
323 |                         'cmdline': ' '.join(info.get('cmdline'
    | ) or []) if info.get('cmdline') else '',
324 |                         'username': info.get('username') or ''
    | ,
325 |                         'create_time': info.get('create_time')
    | or 0.0,
326 |                         })
327 |                 except (psutil.NoSuchProcess, psutil.AccessDen
    | ied):
328 |                     continue
329 |             except Exception as ex:
330 |                 logger.debug(f'Ошибка при получении списка процесс
    | ов: {ex}')
331 |             return processes
332 |
333 |         def _get_current_network_connections(self) -> List[Dict[str, Any]]:
334 |             """Получить текущие активные сетевые подключения."""
335 |             connections = []
336 |             try:
337 |                 import psutil
338 |                 for conn in psutil.net_connections(kind='inet'):
339 |                     try:
340 |                         laddr = f'{conn.laddr.ip}:{conn.laddr.port
    | }' if conn.laddr else ''

```

```

341 |         raddr = f'{conn.raddr.ip}:{conn.raddr.port
    | }' if conn.raddr else ''
342 |         key = f'{conn.type}_{laddr}_{raddr}_{conn.
    | pid}'
343 |         connections.append({
344 |             'key': key,
345 |             'fd': conn.fd,
346 |             'family': conn.family.name if hasattr(
    | conn.family, 'name') else str(conn.family),
347 |             'type': conn.type.name if hasattr(conn
    | .type, 'name') else str(conn.type),
348 |             'laddr': laddr,
349 |             'raddr': raddr,
350 |             'status': conn.status,
351 |             'pid': conn.pid,
352 |         })
353 |     except Exception:
354 |         continue
355 |     except Exception as ex:
356 |         logger.debug(f'Ошибка при получении сетевых соедин
    | ений: {ex}')
357 |     return connections
358 |
359 |     def _get_registry_keys(self) -> Dict[str, Any]:
360 |         """Получить контрольные ключи реестра автозагрузки и с
    | истемных политик."""
361 |         keys = {}
362 |         if os.name != 'nt':
363 |             return keys
364 |         try:
365 |             import winreg
366 |             hives = [
367 |                 (winreg.HKEY_CURRENT_USER, r'Software\Microsof
    | t\Windows\CurrentVersion\Run'),
368 |                 (winreg.HKEY_LOCAL_MACHINE, r'Software\Microso
    | ft\Windows\CurrentVersion\Run'),
369 |                 (winreg.HKEY_LOCAL_MACHINE, r'Software\Microso
    | ft\Windows\CurrentVersion\RunOnce'),

```

```

370         ]
371         for hive, subkey in hives:
372             try:
373                 with winreg.OpenKey(hive, subkey) as k:
374                     idx = 0
375                     while True:
376                         try:
377                             name, val, _ = winreg.EnumValue
378                             e(k, idx)
379                             key_path = f'{subkey}\\{name}'
380                             keys[key_path] = str(val)
381                             idx += 1
382                         except OSError:
383                             break
384                     except OSError:
385                         continue
386             except Exception as ex:
387                 logger.debug(f'Ошибка при чтении ключей реестра: {
388                     ex}')
389             return keys
390
391     def _get_file_hashes(self) -> Dict[str, str]:
392         """Получить отпечатки и размеры отслеживаемых файлов д
393         ля быстрого детекта изменений."""
394         hashes = {}
395         for root_path in self.monitored_paths:
396             p = Path(root_path)
397             if not p.exists():
398                 continue
399             try:
400                 # Ограничиваем сканирование первыми 100 файлам
401                 и для предотвращения перегрузки
402                 count = 0
403                 for item in p.rglob('*'):
404                     if count > 100:
405                         break
406                     if item.is_file():
407                         try:

```

```

404         stat = item.stat()
405         hashes[str(item)] = f'{stat.st_siz
e}_{stat.st_mtime}'
406         count += 1
407     except OSError:
408         continue
409     except Exception as ex:
410         logger.debug(f'Ошибка при проверке файлов в {r
oot_path}: {ex}')
411     return hashes
412
413     def get_status(self) -> Dict[str, Any]:
414         """Получить диагностический статус сборщика.
415
416         Returns:
417             Dict[str, Any]: Словарь со статусом и счетчиками.
418         """
419         status = W64CollectorStatus(
420             running=self._running,
421             events_count=self._event_counter,
422             log_dir=self.log_dir,
423             last_event_time=self._last_event_time,
424         )
425         return status.model_dump()
426
427     def get_events(self, event_type: Optional[str] = None, lim
it: int = 100) -> List[Dict[str, Any]]:
428         """Получить список последних событий из SQLite хранили
ща или JSONL файлов.
429
430         Args:
431             event_type: Фильтр по типу события.
432             limit: Максимальное число записей.
433
434         Returns:
435             List[Dict[str, Any]]: Список событий.
436         """
437         if self.storage:

```

```
438         return self.storage.get_w64_events(event_type=event
| t_type, provider='w64_collector', limit=limit)
439     return []
440
441
442 _global_w64_collector: Optional[AIW64Collector] = None
443 _lock = threading.Lock()
444
445 def get_w64_collector(**kwargs: Any) -> AIW64Collector:
446     """Получить глобальный синглтон сборщика AIW64Collector.
447
448     Args:
449         **kwargs: Аргументы инициализации при первом создании.
450
451     Returns:
452         AIW64Collector: Синглтон сборщика.
453     """
454     global _global_w64_collector
455     with _lock:
456         if _global_w64_collector is None:
457             _global_w64_collector = AIW64Collector(**kwargs)
458         return _global_w64_collector
459
460 def start_w64_collector(**kwargs: Any) -> bool:
461     """Запустить глобальный сборщик AIW64Collector."""
462     collector = get_w64_collector(**kwargs)
463     return collector.start()
464
465 def stop_w64_collector() -> bool:
466     """Остановить глобальный сборщик AIW64Collector."""
467     global _global_w64_collector
468     with _lock:
469         if _global_w64_collector:
470             res = _global_w64_collector.stop()
471             _global_w64_collector = None
472             return res
473     return False
474
```


w64_etw_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - W64 Etw
  Collector
4 # =====
5 # Description:
6 #   ETW Collector для сбора низкоуровневых событий ядра и журна
  лов аудита Windows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.w64_etw_collector import AIW
  64ETWCollector
11 #
12 #       service = AIW64ETWCollector()
13 #
14 # File: w64_etw_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 04:46:00
20 # =====
21
22 from __future__ import annotations
23 """ETW Collector для сбора низкоуровневых событий ядра и журна
  лов аудита Windows."""
24
25 import json
26 import os
27 import subprocess
28 import threading
29 import time
```

```

30 from datetime import datetime, timezone
31 from pathlib import Path
32 from typing import Any, Callable, Dict, List, Optional
33 from logger import logger
34 from .models import ETWTraceEvent, ProcessLifecycleEvent, W64C
   collectorStatus
35 from .sqlite import TelemetryStorage
36
37 _file_write_lock = threading.Lock()
38
39 def _append_jsonl_safely(file_path: Path, data: Dict[str, Any]
   , retries: int = 4, retry_delay: float = 0.05) -> bool:
40     """Безопасная запись JSONL строки с повторными попытками п
   ри блокировке файла.
41
42     Args:
43         file_path: Путь к целевому файлу JSONL.
44         data: Словарь с данными для сериализации в JSON.
45         retries: Количество попыток записи при ошибке доступа.
46         retry_delay: Задержка между попытками (сек).
47
48     Returns:
49         bool: True если запись успешна, иначе исключение.
50     """
51     line = json.dumps(data, ensure_ascii=False) + '\n'
52     for attempt in range(retries):
53         try:
54             with _file_write_lock:
55                 with open(file_path, 'a', encoding='utf-8') as
   f:
56                     f.write(line)
57             return True
58         except (PermissionError, OSError) as ex:
59             if attempt < retries - 1:
60                 time.sleep(retry_delay * (attempt + 1))
61             else:
62                 raise ex
63     return False

```

```

64 |
65 | class AIW64ETWCollector:
66 |     """Сборщик низкоуровневых событий через ETW и журналы ауди
    та Windows."""
67 |
68 |     def __init__(
69 |         self,
70 |         log_dir: Optional[str] = None,
71 |         enable_process_trace: bool = True,
72 |         enable_disk_trace: bool = True,
73 |         enable_network_trace: bool = True,
74 |         enable_registry_trace: bool = True,
75 |         poll_interval_sec: float = 5.0,
76 |         storage: Optional[TelemetryStorage] = None,
77 |         on_event_callback: Optional[Callable[[Dict[str, Any]],
    None]] = None,
78 |     ) -> None:
79 |         """Инициализация ETW сборщика телеметрии.
80 |
81 |         Args:
82 |             log_dir: Директория для сохранения JSONL логов ETW
    .
83 |             enable_process_trace: Трассировка создания/заверше
    ния процессов (ID 4688).
84 |             enable_disk_trace: Трассировка операций файлового
    аудита (ID 4663).
85 |             enable_network_trace: Трассировка сетевых подключе
    ний Windows Filtering Platform.
86 |             enable_registry_trace: Трассировка изменений ключе
    й аудита реестра.
87 |             poll_interval_sec: Интервал между выборками событи
    й ETW (сек).
88 |             storage: Экземпляр хранилища TelemetryStorage.
89 |             on_event_callback: Callback-обработчик каждого заф
    иксированного события.
90 |         """
91 |         if log_dir is None:
92 |             appdata = os.environ.get('LOCALAPPDATA', os.path.e

```

```

xpanduser('~\\AppData\\Local'))
93 |         self.log_dir = str(Path(appdata) / 'AI-Breadboard'
    | / 'apps' / 'windows' / 'telemetry' / 'logs' / 'etw')
94 |     else:
95 |         self.log_dir = log_dir
96 |
97 |     Path(self.log_dir).mkdir(parents=True, exist_ok=True)
98 |     self.enable_process_trace = enable_process_trace
99 |     self.enable_disk_trace = enable_disk_trace
100 |     self.enable_network_trace = enable_network_trace
101 |     self.enable_registry_trace = enable_registry_trace
102 |     self.poll_interval = max(1.0, poll_interval_sec)
103 |     self.storage = storage or TelemetryStorage.get_instanc
    | e()
104 |     self.on_event_callback = on_event_callback
105 |
106 |     self._running = False
107 |     self._thread: Optional[threading.Thread] = None
108 |     self._stop_event = threading.Event()
109 |     self._event_counter = 0
110 |     self._last_event_time: Optional[str] = None
111 |     logger.debug(f'AIW64ETWCollector инициализирован. Дире
    | ктория логов: {self.log_dir}')
112 |
113 |     def start(self) -> bool:
114 |         """Запустить фоновый поток сбора ETW событий.
115 |
116 |         Returns:
117 |             bool: True если сборщик успешно запущен, False есл
    | и уже работал.
118 |         """
119 |         if self._running:
120 |             logger.warning('AIW64ETWCollector уже запущен')
121 |             return False
122 |
123 |         self._running = True
124 |         self._stop_event.clear()
125 |         self._thread = threading.Thread(target=self._etw_loop,

```

```

    name='AIW64ETWWorker', daemon=True)
126     self._thread.start()
127     logger.info('AIW64ETWCollector успешно запущен')
128     return True
129
130     def stop(self) -> bool:
131         """Остановить фоновый поток ETW сбора.
132
133         Returns:
134             bool: True если сборщик остановлен, False если не
135                был активен.
136         """
137         if not self._running:
138             return False
139
140         self._running = False
141         self._stop_event.set()
142         if self._thread and self._thread.is_alive():
143             self._thread.join(timeout=5.0)
144         logger.info(f'AIW64ETWCollector остановлен. Всего зафиксировано событий: {self._event_counter}')
145         return True
146
147     def _etw_loop(self) -> None:
148         """Основной рабочий цикл выборки событий ETW."""
149         while not self._stop_event.is_set():
150             try:
151                 self._collect_etw_events()
152                 self._stop_event.wait(timeout=self.poll_interval)
153             except Exception as ex:
154                 logger.error(f'Ошибка в цикле ETW сборщика: {ex}')
155
156     def _collect_etw_events(self) -> None:
157         """Собрать порцию событий из включенных провайдеров."

```

```

158         return
159
160     if self.enable_process_trace:
161         self._collect_process_events()
162     if self.enable_disk_trace:
163         self._collect_file_events()
164     if self.enable_network_trace:
165         self._collect_network_events()
166     if self.enable_registry_trace:
167         self._collect_registry_events()
168
169     def _collect_process_events(self) -> None:
170         """Собрать события создания процессов (Security ID 468
171         8)."""
172         try:
173             ps_script = """
174             $startTime = (Get-Date).AddMinutes(-1)
175             $events = Get-WinEvent -FilterHashtable @{
176                 LogName = 'Security'
177                 Id = 4688
178                 StartTime = $startTime
179             } -MaxEvents 50 -ErrorAction SilentlyContinue
180
181             foreach ($e in $events) {
182                 $xml = [xml]$e.ToXml()
183                 $data = @{}
184                 foreach ($d in $xml.Event.EventData.Data) {
185                     $data[$d.Name] = $d.#text'
186                 }
187                 $data | ConvertTo-Json -Compress
188             }
189             """
190         res = subprocess.run(
191             ['powershell.exe', '-NoProfile', '-NonInteract
192             ive', '-Command', f'& {{ {ps_script} }}'],
193             capture_output=True,
194             text=True,
195             timeout=8,

```

```

194         creationflags=subprocess.CREATE_NO_WINDOW if o
| s.name == 'nt' else 0,
195     )
196     if res.returncode == 0 and res.stdout.strip():
197         for line in res.stdout.strip().splitlines():
198             if line.strip():
199                 try:
200                     event = json.loads(line)
201                     self._log_event('etw_process_creat
| e', event)
202                 except json.JSONDecodeError:
203                     continue
204     except Exception as ex:
205         logger.debug(f'Ошибка сбора событий процессов ETW:
| {ex}')
206
207     def _collect_file_events(self) -> None:
208         """Собрать события файловых операций (Security ID 4663
| )."""
209         try:
210             ps_script = """
211             $startTime = (Get-Date).AddMinutes(-1)
212             $events = Get-WinEvent -FilterHashtable @{
213                 LogName = 'Security'
214                 Id = 4663
215                 StartTime = $startTime
216             } -MaxEvents 50 -ErrorAction SilentlyContinue
217
218             foreach ($e in $events) {
219                 $xml = [xml]$e.ToXml()
220                 $data = @{}
221                 foreach ($d in $xml.Event.EventData.Data) {
222                     $data[$d.Name] = $d.'#text'
223                 }
224                 $data | ConvertTo-Json -Compress
225             }
226             """
227         res = subprocess.run(

```

```

228 |         ['powershell.exe', '-NoProfile', '-NonInteract
| ive', '-Command', f'& {{ {ps_script} }}'],
229 |         capture_output=True,
230 |         text=True,
231 |         timeout=8,
232 |         creationflags=subprocess.CREATE_NO_WINDOW if o
| s.name == 'nt' else 0,
233 |     )
234 |     if res.returncode == 0 and res.stdout.strip():
235 |         for line in res.stdout.strip().splitlines():
236 |             if line.strip():
237 |                 try:
238 |                     event = json.loads(line)
239 |                     self._log_event('etw_file_access',
| event)
240 |                 except json.JSONDecodeError:
241 |                     continue
242 |             except Exception as ex:
243 |                 logger.debug(f'Ошибка сбора событий файлов ETW: {e
| x}')
244 |
245 |     def _collect_network_events(self) -> None:
246 |         """Собрать события сетевых блокировок/подключений WFP
| (Security ID 5156/5158)."""
247 |         try:
248 |             ps_script = """
249 |             $startTime = (Get-Date).AddMinutes(-1)
250 |             $events = Get-WinEvent -FilterHashtable @{
251 |                 LogName = 'Security'
252 |                 Id = 5156,5158
253 |                 StartTime = $startTime
254 |             } -MaxEvents 50 -ErrorAction SilentlyContinue
255 |
256 |             foreach ($e in $events) {
257 |                 $xml = [xml]$e.ToXml()
258 |                 $data = @{}
259 |                 foreach ($d in $xml.Event.EventData.Data) {
260 |                     $data[$d.Name] = $d.'#text'

```



```

261         }
262         $data | ConvertTo-Json -Compress
263     }
264     """
265     res = subprocess.run(
266         ['powershell.exe', '-NoProfile', '-NonInteract
267         ive', '-Command', f'& {{ {ps_script} }}'],
268         capture_output=True,
269         text=True,
270         timeout=8,
271         creationflags=subprocess.CREATE_NO_WINDOW if o
272         s.name == 'nt' else 0,
273     )
274     if res.returncode == 0 and res.stdout.strip():
275         for line in res.stdout.strip().splitlines():
276             if line.strip():
277                 try:
278                     event = json.loads(line)
279                     self._log_event('etw_network_conne
280                     ction', event)
281                 except json.JSONDecodeError:
282                     continue
283     except Exception as ex:
284         logger.debug(f'Ошибка сбора сетевых событий ETW: {
285         ex}')
286
287     def _collect_registry_events(self) -> None:
288         """Собрать события аудита реестра (Security ID 4657)."""
289
290         try:
291             ps_script = """
292             $startTime = (Get-Date).AddMinutes(-1)
293             $events = Get-WinEvent -FilterHashtable @{
294                 LogName = 'Security'
295                 Id = 4657
296                 StartTime = $startTime
297             } -MaxEvents 50 -ErrorAction SilentlyContinue

```

```

294         foreach ($e in $events) {
295             $xml = [xml]$e.ToXml()
296             $data = @{}
297             foreach ($d in $xml.Event.EventData.Data) {
298                 $data[$d.Name] = $d.'#text'
299             }
300             $data | ConvertTo-Json -Compress
301         }
302         """
303         res = subprocess.run(
304             ['powershell.exe', '-NoProfile', '-NonInteract
ive', '-Command', f'& {{ {ps_script} }}'],
305             capture_output=True,
306             text=True,
307             timeout=8,
308             creationflags=subprocess.CREATE_NO_WINDOW if o
s.name == 'nt' else 0,
309         )
310         if res.returncode == 0 and res.stdout.strip():
311             for line in res.stdout.strip().splitlines():
312                 if line.strip():
313                     try:
314                         event = json.loads(line)
315                         self._log_event('etw_registry_acce
ss', event)
316                     except json.JSONDecodeError:
317                         continue
318             except Exception as ex:
319                 logger.debug(f'Ошибка сбора событий реестра ETW: {
ex}')
320
321     def _log_event(self, event_type: str, event_data: Dict[str
, Any]) -> None:
322         """Записать событие в потоковый JSONL файл и базу данн
ых SQLite.
323
324         Args:
325             event_type: Тип события ETW.

```

```

326         event_data: Словарь с полями из журнала событий.
327         """
328         self._event_counter += 1
329         now_iso = datetime.now(timezone.utc).isoformat()
330         date_str = datetime.now(timezone.utc).strftime('%Y-%m-
%d')
331         log_file = Path(self.log_dir) / f'{event_type}_{date_s
tr}.jsonl'
332
333         payload = {
334             'event_id': f'etw_{int(time.time())}_{self._event_
counter}',
335             'timestamp': now_iso,
336             'event_type': event_type,
337             'provider': 'Microsoft-Windows-Security-Auditing',
338             'data': event_data,
339         }
340         self._last_event_time = now_iso
341
342         # 1. Запись в JSONL
343         try:
344             _append_jsonl_safely(log_file, payload)
345         except Exception as ex:
346             logger.debug(f'Не удалось записать JSONL лог ETW {
event_type}: {ex}')
347
348         # 2. Сохранение в SQLite TelemetryStorage
349         if self.storage:
350             try:
351                 db_record = {
352                     'event_id': payload['event_id'],
353                     'timestamp': now_iso,
354                     'event_type': event_type,
355                     'path': event_data.get('ObjectName') or ev
ent_data.get('NewProcessName'),
356                     'pid': event_data.get('ProcessId') or even
t_data.get('NewProcessId'),
357                     'name': event_data.get('ProcessName') or e

```

```

vent_data.get('SubjectUserName'),
358         'details': event_data,
359     }
360     self.storage.save_w64_event(db_record, provide
r='w64_etw_collector')
361
362     # Специфическая фиксация создания процессов дл
я Process Provenance
363     if event_type == 'etw_process_create':
364         def _safe_pid(val: Any) -> int:
365             if val is None:
366                 return 0
367             if isinstance(val, int):
368                 return val
369             s = str(val).strip()
370             if s.startswith(('0x', '0X')):
371                 try:
372                     return int(s, 16)
373                 except ValueError:
374                     return 0
375             try:
376                 return int(s)
377             except ValueError:
378                 return 0
379
380         new_pid = _safe_pid(event_data.get('NewPro
cessId') or event_data.get('ProcessId'))
381         parent_pid = _safe_pid(event_data.get('Pro
cessId') or event_data.get('ParentProcessId'))
382         exe_path = event_data.get('NewProcessName'
) or event_data.get('ProcessName') or ''
383         proc_name = os.path.basename(exe_path) if
exe_path else 'unknown'
384         parent_exe = event_data.get('ParentProcess
Name') or ''
385         parent_name = os.path.basename(parent_exe)
if parent_exe else None
386         u_name = event_data.get('SubjectUserName')

```

```

387         d_name = event_data.get('SubjectDomainName
    | ')
388         full_user = f'{d_name}\\{u_name}' if d_name
    | and u_name and d_name != '-' else u_name
389         u_sid = event_data.get('SubjectUserSid')
390         cmdline = event_data.get('CommandLine')
391         integrity = event_data.get('MandatoryLabel
    | ')
392
393         prov_event = ProcessLifecycleEvent(
394             event_type='ProcessCreated',
395             process_guid=f'proc_{new_pid}_{int(time
    | e.time())}',
396             pid=new_pid,
397             name=proc_name,
398             executable_path=exe_path,
399             command_line=cmdline,
400             user=full_user,
401             sid=u_sid,
402             integrity_level=integrity,
403             parent_pid=parent_pid,
404             parent_name=parent_name,
405             created_at=now_iso,
406             details=event_data,
407         )
408         self.storage.save_process_provenance_event
    | (prov_event)
409         except Exception as ex:
410             logger.debug(f'Ошибка сохранения ETW события в
    | БД: {ex}')
411
412         # 3. Передача в callback
413         if self.on_event_callback:
414             try:
415                 self.on_event_callback(payload)
416             except Exception as ex:
417                 logger.debug(f'Ошибка в ETW on_event_callback:
    | {ex}')

```

```

418
419     def get_status(self) -> Dict[str, Any]:
420         """Получить диагностический статус ETW сборщика.
421
422         Returns:
423             Dict[str, Any]: Словарь со статусом и счетчиками.
424         """
425         status = W64CollectorStatus(
426             running=self._running,
427             events_count=self._event_counter,
428             log_dir=self.log_dir,
429             last_event_time=self._last_event_time,
430         )
431         return status.model_dump()
432
433     def get_events(self, event_type: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
434         """Получить список последних событий ETW из SQLite хранилища.
435
436         Args:
437             event_type: Фильтр по типу события.
438             limit: Максимальное число записей.
439
440         Returns:
441             List[Dict[str, Any]]: Список событий.
442         """
443         if self.storage:
444             return self.storage.get_w64_events(event_type=event_type, provider='w64_etw_collector', limit=limit)
445         return []
446

```

advapi32.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  | - Advapi32
4 # =====
5 # Description:
6 #   advapi32.dll wrapper for registry, services, and security
  | functions.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_ffadvapi32 import SY
  | STEMTIME
11 #
12 #       service = SYSTEMTIME()
13 #
14 # File: advapi32.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_ffadvapi32
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """advapi32.dll wrapper for registry, services, and security f
  | unctions."""
23
24 import ctypes
25 import ctypes.wintypes as wintypes
26 from typing import List, Optional, Dict, Any
27 from dataclasses import dataclass
28 from enum import IntFlag
29 HKEY_CLASSES_ROOT = 2147483648
```

```

30 HKEY_CURRENT_USER = 2147483649
31 HKEY_LOCAL_MACHINE = 2147483650
32 HKEY_USERS = 2147483651
33 HKEY_PERFORMANCE_DATA = 2147483652
34 HKEY_CURRENT_CONFIG = 2147483653
35 KEY_READ = 131097
36 KEY_WRITE = 131078
37 KEY_QUERY_VALUE = 1
38 KEY_ENUMERATE_SUB_KEYS = 8
39 SERVICE_QUERY_STATUS = 4
40 SERVICE_QUERY_CONFIG = 1
41 SERVICE_AUTO_START = 2
42 SERVICE_DEMAND_START = 3
43 SERVICE_DISABLED = 4
44 SERVICE_RUNNING = 4
45 SERVICE_STOPPED = 1
46
47 class SYSTEMTIME(ctypes.Structure):
48     _fields_ = [('wYear', wintypes.WORD), ('wMonth', wintypes.
WORD), ('wDayOfWeek', wintypes.WORD), ('wDay', wintypes.WORD),
    ('wHour', wintypes.WORD), ('wMinute', wintypes.WORD), ('wSeco
nd', wintypes.WORD), ('wMilliseconds', wintypes.WORD)]
49
50 class SERVICE_STATUS(ctypes.Structure):
51     _fields_ = [('dwServiceType', wintypes.DWORD), ('dwCurrent
State', wintypes.DWORD), ('dwControlsAccepted', wintypes.DWORD
), ('dwWin32ExitCode', wintypes.DWORD), ('dwServiceSpecificExi
tCode', wintypes.DWORD), ('dwCheckPoint', wintypes.DWORD), ('d
wWaitHint', wintypes.DWORD)]
52
53 class ENUM_SERVICE_STATUS(ctypes.Structure):
54     _fields_ = [('lpServiceName', wintypes.LPWSTR), ('lpDispla
yName', wintypes.LPWSTR), ('ServiceStatus', SERVICE_STATUS)]
55
56 @dataclass
57 class RegistryValue:
58     """Registry value information."""
59     name: str

```



```

60     value_type: int
61     data: Any
62
63 @dataclass
64 class ServiceInfo:
65     """Service information."""
66     name: str
67     display_name: str
68     state: str
69     service_type: str
70     start_type: str
71
72 class Advapi32API:
73     """Wrapper for advapi32 registry and service functions."""
74
75     def __init__(self):
76         self.advapi32 = ctypes.windll.advapi32
77         self._setup_function_signatures()
78
79     def _setup_function_signatures(self):
80         """Configure ctypes function signatures."""
81         self.advapi32.RegOpenKeyExW.argtypes = [wintypes.HKEY,
82         wintypes.LPCWSTR, wintypes.DWORD, wintypes.DWORD, ctypes.POIN
83         TER(wintypes.HKEY)]
84         self.advapi32.RegOpenKeyExW.restype = wintypes.LONG
85         self.advapi32.RegQueryValueExW.argtypes = [wintypes.HK
86         EY, wintypes.LPCWSTR, wintypes.LPVOID, ctypes.POINTER(wintypes
87         .DWORD), wintypes.LPVOID, ctypes.POINTER(wintypes.DWORD)]
88         self.advapi32.RegQueryValueExW.restype = wintypes.LONG
89         self.advapi32.RegEnumKeyExW.argtypes = [wintypes.HKEY,
90         wintypes.DWORD, wintypes.LPWSTR, ctypes.POINTER(wintypes.DWOR
91         D), ctypes.POINTER(wintypes.DWORD), wintypes.LPWSTR, ctypes.PO
92         INTER(wintypes.DWORD), ctypes.POINTER(SYSTEMTIME)]
93         self.advapi32.RegEnumKeyExW.restype = wintypes.LONG
94         self.advapi32.RegCloseKey.argtypes = [wintypes.HKEY]
95         self.advapi32.RegCloseKey.restype = wintypes.LONG
96         self.advapi32.OpenSCManagerW.argtypes = [wintypes.LPCW
97         STR, wintypes.LPCWSTR, wintypes.DWORD]

```

```

90         self.advapi32.OpenSCManagerW.restype = wintypes.SC_HAN
DLE
91         self.advapi32.OpenServiceW.argtypes = [wintypes.SC_HAN
DLE, wintypes.LPCWSTR, wintypes.DWORD]
92         self.advapi32.OpenServiceW.restype = wintypes.SC_HAN
DLE
93         self.advapi32.QueryServiceStatus.argtypes = [wintypes.
SC_HANDLE, ctypes.POINTER(SERVICE_STATUS)]
94         self.advapi32.QueryServiceStatus.restype = wintypes.BO
OL
95         self.advapi32.EnumServicesStatusW.argtypes = [wintypes
.SC_HANDLE, wintypes.DWORD, wintypes.DWORD, ctypes.POINTER(ENU
M_SERVICE_STATUS), wintypes.DWORD, ctypes.POINTER(wintypes.DWO
RD), ctypes.POINTER(wintypes.DWORD), wintypes.LPVOID]
96         self.advapi32.EnumServicesStatusW.restype = wintypes.B
OOL
97         self.advapi32.CloseServiceHandle.argtypes = [wintypes.
SC_HANDLE]
98         self.advapi32.CloseServiceHandle.restype = wintypes.BO
OL
99
100     def read_registry_value(self, hive: int, path: str, value_
name: str) -> Optional[Any]:
101         """
102         Read a registry value.
103
104         Args:
105             hive: Registry hive (HKEY_*)
106             path: Registry path
107             value_name: Value name
108
109         Returns:
110             Registry value or None
111         """
112         try:
113             key = wintypes.HKEY()
114             result = self.advapi32.RegOpenKeyExW(hive, path, 0
, KEY_READ, ctypes.byref(key))

```

```

115         if result != 0:
116             return None
117         try:
118             value_type = wintypes.DWORD()
119             data_size = wintypes.DWORD(4096)
120             data = ctypes.create_unicode_buffer(4096)
121             result = self.advapi32.RegQueryValueExW(key, v
alue_name, None, ctypes.byref(value_type), data, ctypes.byref(
data_size))
122             if result == 0:
123                 return RegistryValue(name=value_name, valu
e_type=value_type.value, data=data.value)
124             return None
125         finally:
126             self.advapi32.RegCloseKey(key)
127     except Exception:
128         return None
129
130     def enumerate_registry_values(self, hive: int, path: str)
-> List[RegistryValue]:
131         """
132         Enumerate all values in a registry key.
133
134         Args:
135             hive: Registry hive
136             path: Registry path
137
138         Returns:
139             List of RegistryValue objects
140         """
141         values = []
142         try:
143             key = wintypes.HKEY()
144             result = self.advapi32.RegOpenKeyExW(hive, path, 0
, KEY_READ, ctypes.byref(key))
145             if result != 0:
146                 return values
147             try:

```

```

148         index = 0
149         while True:
150             value_name_buffer = ctypes.create_unicode_
buffer(256)
151             value_name_size = wintypes.DWORD(len(value
_name_buffer))
152             value_type = wintypes.DWORD()
153             data_size = wintypes.DWORD(4096)
154             data = ctypes.create_unicode_buffer(4096)
155             result = self.advapi32.RegEnumKeyExW(key,
index, value_name_buffer, ctypes.byref(value_name_size), None,
None, None, None)
156             if result != 0:
157                 break
158             values.append(RegistryValue(name=value_nam
e_buffer.value, value_type=value_type.value, data=data.value))
159             index += 1
160         finally:
161             self.advapi32.RegCloseKey(key)
162     except Exception:
163         pass
164     return values
165
166     def enumerate_services(self) -> List[ServiceInfo]:
167         """
168         Enumerate all services.
169
170         Returns:
171             List of ServiceInfo objects
172         """
173         services = []
174         try:
175             scm_handle = self.advapi32.OpenSCManagerW(None, No
ne, 1)
176             if not scm_handle:
177                 return services
178             try:
179                 max_services = 256

```

```

180         services_buffer = (ENUM_SERVICE_STATUS * max_s
| services)()
181         bytes_needed = wintypes.DWORD()
182         services_returned = wintypes.DWORD()
183         result = self.advapi32.EnumServicesStatusW(scm
| _handle, 48, 4, services_buffer, ctypes.sizeof(services_buffer
| ), ctypes.byref(bytes_needed), ctypes.byref(services_returned)
| , None)
184         if result:
185             for i in range(services_returned.value):
186                 svc = services_buffer[i]
187                 service_info = ServiceInfo(name=svc.lp
| ServiceName if svc.lpServiceName else 'Unknown', display_name=
| svc.lpDisplayName if svc.lpDisplayName else 'Unknown', state='
| Running' if svc.ServiceStatus.dwCurrentState == SERVICE_RUNNIN
| G else 'Stopped', service_type='Service' if svc.ServiceStatus.
| dwServiceType == 16 else 'Driver', start_type='Auto Start' if
| svc.ServiceStatus.dwCurrentState == SERVICE_AUTO_START else 'M
| anual')
188                 services.append(service_info)
189         finally:
190             self.advapi32.CloseServiceHandle(scm_handle)
191     except Exception:
192         pass
193     return services

```

error_decoder.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Error Decoder
5 # Description:
6 #   Модуль нативного декодирования системных кодов возврата Win32, HRESULT,
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.error_decoder import WindowsErrorDecoder
11 #
12 #       service = WindowsErrorDecoder()
13 #
14 # File: error_decoder.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Модуль нативного декодирования системных кодов возврата Win32, HRESULT, """
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 import os
28 import sys
29 from typing import Any, Dict, Optional, Tuple, Union
```

```
30
31 try:
32     from logger import logger
33 except ImportError:
34     import logging
35     logger = logging.getLogger(__name__)
36
37 # Флаги для FormatMessageW
38 FORMAT_MESSAGE_ALLOCATE_BUFFER = 0x00000100
39 FORMAT_MESSAGE_IGNORE_INSERTS = 0x00000200
40 FORMAT_MESSAGE_FROM_SYSTEM = 0x00001000
41 FORMAT_MESSAGE_FROM_HMODULE = 0x00000800
42
43 # Известные стандартные Stop-коды ядра Windows (BugCheck Codes
44 )
45 BUGCHECK_CODES_MAP: Dict[int, Tuple[str, str]] = {
46     0x0000000A: ('IRQL_NOT_LESS_OR_EQUAL', 'Процесс или драйве
47     р попытался обратиться к адресу памяти при недопустимо высоком
48     уровне IRQL. '),
49     0x0000001E: ('KMODE_EXCEPTION_NOT_HANDLED', 'Программа ядр
50     а или драйвер сгенерировали исключение, которое не было перехв
51     ачено обработчиком. '),
52     0x00000024: ('NTFS_FILE_SYSTEM', 'Критический сбой или пов
53     реждение в системном драйвере файловой системы ntfs.sys. '),
54     0x0000003B: ('SYSTEM_SERVICE_EXCEPTION', 'Исключение при в
55     ыполнении подпрограммы системной службы Windows. '),
56     0x00000050: ('PAGE_FAULT_IN_NONPAGED_AREA', 'Запрос недопу
57     стимой системной памяти. Ошибка вызвана некорректным адресом и
58     ли сбоем RAM. '),
59     0x0000007E: ('SYSTEM_THREAD_EXCEPTION_NOT_HANDLED', 'Систе
60     мный поток сгенерировал неперехваченное исключение (часто связ
61     ано с несовместимым драйвером). '),
62     0x0000007F: ('UNEXPECTED_KERNEL_MODE_TRAP', 'Аппаратная ло
63     вушка процессора (деление на ноль, повреждение стека или перег
64     рузка оборудования). '),
65     0x0000009F: ('DRIVER_POWER_STATE_FAILURE', 'Драйвер находи
66     тся в несогласованном или недействительном состоянии управлени
67     я электропитанием (сна/пробуждения). '),
```

53 0x000000BE: ('ATTEMPTED_WRITE_TO_READONLY_MEMORY', 'Попытка
а драйвера выполнить запись в сегмент памяти, доступный только
для чтения.'),

54 0x000000C2: ('BAD_POOL_CALLER', 'Текущий поток выполнил не
корректный запрос к пулу системной памяти ядра.'),

55 0x000000D1: ('DRIVER_IRQL_NOT_LESS_OR_EQUAL', 'Драйвер поп
ытался обратиться к выгружаемой памяти на повышенном уровне пр
ерываний IRQL.'),

56 0x000000EF: ('CRITICAL_PROCESS_DIED', 'Критически важный с
истемный процесс (csrss.exe, wininit.exe и др.) неожиданно зав
ершил свою работу.'),

57 0x00000109: ('CRITICAL_STRUCTURE_CORRUPTION', 'Ядро обнару
жило повреждение критических структур данных или кода ядра (Pa
tchGuard).'),

58 0x00000116: ('VIDEO_TDR_FAILURE', 'Сбой видеодрайвера: поп
ытка сброса зависшего графического драйвера завершилась тайм-а
утом.'),

59 0x00000117: ('VIDEO_TDR_TIMEOUT_DETECTED', 'Графический др
айвер не ответил вовремя в пределах интервала обнаружения тайм
-аута TDR.'),

60 0x00000124: ('WHEA_UNCORRECTABLE_ERROR', 'Критическая неус
траняемая аппаратная ошибка архитектуры Windows Hardware Error
Architecture (CPU/RAM/PCIe).'),

61 0x00000133: ('DPC_WATCHDOG_VIOLATION', 'Сторожевой таймер
DPC зафиксировал зависание отложенного вызова процедуры драйве
ром.'),

62 0x00000139: ('KERNEL_SECURITY_CHECK_FAILURE', 'Ядро обнару
жило повреждение критической структуры данных безопасности (Bu
ffer Overflow / Stack Cookie).'),

63 0x00000154: ('UNEXPECTED_STORE_EXCEPTION', 'Сбой подсистем
ы хранения страниц памяти ядра (компрессия памяти / повреждени
е накопителя).'),

64 0x00000192: ('KERNEL_AUTO_BOOST_LOCK_ACQUISITION_WITH_RAIS
ED_IRQL', 'Попытка захвата блокировки AutoBoost ядра на повыше
нном уровне IRQL.'),

65 0x000001A7: ('STORE_DATA_STRUCTURE_CORRUPTION', 'Поврежден
ие внутренней структуры данных диспетчера хранилища страниц па
мяти.'),


```

66     0xC000021A: ('STATUS_SYSTEM_PROCESS_TERMINATED', 'Подсисте
ма пользовательского режима (WinLogon или Csrss) аварийно заве
ршилась с нарушением безопасности. '),
67     0xC0000221: ('STATUS_IMAGE_CHECKSUM_MISMATCH', 'Контрольна
я сумма системного драйвера или DLL не совпадает с заголовком
файла. '),
68 }
69
70
71 class WindowsErrorDecoder:
72     """Класс нативного декодирования ошибок Windows через Form
atMessageW."""
73
74     def __init__(self) -> None:
75         """Инициализация дескрипторов библиотек."""
76         self._kernel32 = getattr(ctypes.windll, 'kernel32', No
ne) if os.name == 'nt' else None
77         self._ntdll_handle: Optional[int] = None
78         if self._kernel32 and os.name == 'nt':
79             try:
80                 self._kernel32.GetModuleHandleW.argtypes = [wi
ntypes.LPCWSTR]
81                 self._kernel32.GetModuleHandleW.restype = wint
ypes.HMODULE
82                 self._ntdll_handle = self._kernel32.GetModuleH
andleW('ntdll.dll')
83             except Exception as ex:
84                 logger.debug(f'Не удалось получить дескриптор
ntdll.dll: {ex}')
85
86         def parse_error_code(self, code: Union[int, str]) -> Tuple
[int, str]:
87             """Преобразовать входной код ошибки (число, hex-строку
или отрицательный int) в uint32.
88
89             Args:
90                 code: Числовое или строковое представление кода ош
ибки.

```

```

91 |
92 |     Returns:
93 |         Tuple[int, str]: (нормализованный uint32 код, форм
    | атированная hex-строка 0XXXXXXXXX).
94 |     """
95 |     if isinstance(code, str):
96 |         c_str = code.strip()
97 |         try:
98 |             if c_str.lower().startswith('0x'):
99 |                 val = int(c_str, 16)
100 |             else:
101 |                 val = int(c_str, 10)
102 |         except ValueError:
103 |             return 0, '0x00000000'
104 |     else:
105 |         val = int(code)
106 |
107 |     # Преобразуем отрицательные 32-битные числа (например
    | из ctypes signed HRESULT) в беззнаковый uint32
108 |     uint_val = val & 0xFFFFFFFF
109 |     return uint_val, f'0x{uint_val:08X}'
110 |
111 |     def format_system_error(self, code: Union[int, str], lang_
    | id: int = 0) -> str:
112 |         """Получить локализованное системное описание ошибки и
    | з Windows API (Win32 / HRESULT / NTSTATUS).
113 |
114 |     Args:
115 |         code: Код ошибки (например, 5, 0x80070005, 0xC000
    | 005, '0x80240020').
116 |         lang_id: Идентификатор языка (0 - язык текущего по
    | тока/системы).
117 |
118 |     Returns:
119 |         str: Локализованное текстовое описание ошибки или
    | пустая строка, если код не распознан.
120 |     """
121 |     if not self._kernel32 or os.name != 'nt':

```

```

122         return ''
123
124     uint_code, _ = self.parse_error_code(code)
125     if uint_code == 0:
126         return 'Операция успешно завершена.'
127
128     # 1. Попытка чтения из системного репозитория (FORMAT_
MESSAGE_FROM_SYSTEM для Win32 и HRESULT)
129     buf = wintypes.LPWSTR()
130     flags = FORMAT_MESSAGE_ALLOCATE_BUFFER | FORMAT_MESSAG
E_FROM_SYSTEM | FORMAT_MESSAGE_IGNORE_INSERTS
131     chars = self._kernel32.FormatMessageW(
132         flags,
133         None,
134         wintypes.DWORD(uint_code),
135         wintypes.DWORD(lang_id),
136         ctypes.byref(buf),
137         0,
138         None,
139     )
140
141     if chars > 0 and buf.value:
142         msg = buf.value.strip().rstrip('\r\n.')
143         if self._kernel32.LocalFree:
144             self._kernel32.LocalFree(buf)
145         return msg
146
147     # 2. Если код лежит в диапазоне NTSTATUS (0xC0000000..
0xDFFFFFFF или 0x40000000..0x7FFFFFFF)
148     # запрашиваем системные сообщения из ntdll.dll
149     if self._ntdll_handle:
150         buf_nt = wintypes.LPWSTR()
151         flags_nt = FORMAT_MESSAGE_ALLOCATE_BUFFER | FORMAT
_MESSAGE_FROM_HMODULE | FORMAT_MESSAGE_IGNORE_INSERTS
152         chars_nt = self._kernel32.FormatMessageW(
153             flags_nt,
154             wintypes.HMODULE(self._ntdll_handle),
155             wintypes.DWORD(uint_code),

```

```

156 |         wintypes.DWORD(lang_id),
157 |         ctypes.byref(buf_nt),
158 |         0,
159 |         None,
160 |     )
161 |     if chars_nt > 0 and buf_nt.value:
162 |         msg_nt = buf_nt.value.strip().rstrip('\r\n.')
163 |         if self._kernel32.LocalFree:
164 |             self._kernel32.LocalFree(buf_nt)
165 |         return msg_nt
166 |
167 |     # 3. Если это HRESULT вида 0x8007XXXX (Win32 Facility
168 |     7), пробуем распаковать Win32 код ошибки
169 |     if (uint_code & 0xFFFF0000) == 0x80070000:
170 |         win32_subcode = uint_code & 0xFFFF
171 |         return self.format_system_error(win32_subcode, lan
172 | g_id=lang_id)
173 |
174 |     return ''
175 |
176 |     def decode_bugcheck_code(self, code: Union[int, str]) -> D
177 |     ict[str, str]:
178 |         """Расшифровать Stop-код синего экрана (BSOD BugCheck
179 |         Code).
180 |
181 |         Args:
182 |             code: Числовой или строковый код BugCheck (наприме
183 |             п, 209, 0xD1, '0x000000D1').
184 |
185 |         Returns:
186 |             Dict[str, str]: Словарь с полями 'code_hex', 'symb
187 |             ol', 'description_ru'.
188 |
189 |         """
190 |         uint_code, hex_str = self.parse_error_code(code)
191 |         symbol = 'UNKNOWN_BUGCHECK'
192 |         desc = 'Критический сбой ядра Windows.'
193 |
194 |         if uint_code in BUGCHECK_CODES_MAP:

```

```
188         symbol, desc = BUGCHECK_CODES_MAP[uint_code]
189     else:
190         # Попробуем запросить описание через ntdll
191         nt_desc = self.format_system_error(uint_code)
192         if nt_desc:
193             desc = nt_desc
194
195     return {
196         'code_hex': hex_str,
197         'code_dec': str(uint_code),
198         'symbol': symbol,
199         'description_ru': desc,
200     }
201
202
203 # Глобальный синглтон декодера
204 _decoder_instance: Optional[WindowsErrorDecoder] = None
205
206
207 def get_error_decoder() -> WindowsErrorDecoder:
208     """Получить глобальный экземпляр WindowsErrorDecoder."""
209     global _decoder_instance
210     if _decoder_instance is None:
211         _decoder_instance = WindowsErrorDecoder()
212     return _decoder_instance
213
214
215 def format_system_error(code: Union[int, str], lang_id: int =
216     0) -> str:
217     """Удобный хелпер для форматирования системного кода ошибок
218     и."""
219     return get_error_decoder().format_system_error(code, lang_
220     id=lang_id)
221
222
223 def decode_bugcheck_code(code: Union[int, str]) -> Dict[str, s
224     tr]:
225     """Удобный хелпер для расшифровки Stop-кода ядра (BugCheck
```

```
    )."""
222     return get_error_decoder().decode_bugcheck_code(code)
223
224
225 __all__ = [
226     'WindowsErrorDecoder',
227     'get_error_decoder',
228     'format_system_error',
229     'decode_bugcheck_code',
230     'BUGCHECK_CODES_MAP',
231 ]
232
```

etw.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Etw
5 # Description:
6 #   Event Tracing for Windows (ETW) wrapper for real-time even
7 #   t monitoring.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.etw import GUID
11 #
12 #       service = GUID()
13 #
14 # File: etw.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """Event Tracing for Windows (ETW) wrapper for real-time event
23   monitoring."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 from typing import Optional, List, Callable, Dict, Any
28 from dataclasses import dataclass
29 from enum import IntFlag
30 import threading
31
32 EVENT_TRACE_FLAG_PROCESS = 1
```

```

31 EVENT_TRACE_FLAG_THREAD = 2
32 EVENT_TRACE_FLAG_IMAGE_LOAD = 4
33 EVENT_TRACE_FLAG_DISK_IO = 256
34 EVENT_TRACE_FLAG_DISK_FILE_IO = 512
35 EVENT_TRACE_FLAG_MEMORY_PAGE_FAULTS = 4096
36 EVENT_TRACE_FLAG_MEMORY_HARD_FAULTS = 8192
37 EVENT_TRACE_FLAG_NETWORK_TCPIP = 65536
38 EVENT_TRACE_FLAG_REGISTRY = 131072
39 EVENT_TRACE_FLAG_FILE_IO = 262144
40 EVENT_TRACE_FLAG_FILE_IO_INIT = 524288
41 EVENT_TRACE_REAL_TIME_MODE = 256
42 EVENT_TRACE_FILE_MODE_SEQUENTIAL = 1
43 EVENT_TRACE_FILE_MODE_CIRCULAR = 2
44 EVENT_TRACE_USE_GLOBAL_SEQUENCE = 16384
45 WNODE_FLAG_TRACED_GUID = 131072
46 WNODE_FLAG_USE_MOF_PTR = 1048576
47 PEVENT_TRACE_BUFFER_CALLBACK = ctypes.CFUNCTYPE(wintypes.ULONG
    , wintypes.LPVOID)
48 PEVENT_CALLBACK = ctypes.CFUNCTYPE(None, wintypes.LPVOID)
49
50 class GUID(ctypes.Structure):
51     _fields_ = [('Data1', wintypes.DWORD), ('Data2', wintypes.
    WORD), ('Data3', wintypes.WORD), ('Data4', ctypes.c_ubyte * 8)
    ]
52
53 class WNODE_HEADER(ctypes.Structure):
54     _fields_ = [('BufferSize', wintypes.DWORD), ('ProviderId',
    wintypes.DWORD), ('ClientContext', ctypes.c_uint64), ('Timest
    amp', ctypes.c_uint64), ('Guid', GUID), ('KernelHandle', winty
    pes.HANDLE), ('Flags', wintypes.DWORD)]
55
56 class EVENT_TRACE_PROPERTIES(ctypes.Structure):
57     _fields_ = [('Wnode', WNODE_HEADER), ('BufferSize', wintyp
    es.DWORD), ('MinimumBuffers', wintypes.DWORD), ('MaximumBuffer
    s', wintypes.DWORD), ('MaximumFileSize', wintypes.DWORD), ('Lo
    gFileMode', wintypes.DWORD), ('FlushTimer', wintypes.DWORD), (
    'EnableFlags', wintypes.DWORD), ('AgeLimit', wintypes.LONG), (
    'NumberOfBuffers', wintypes.DWORD), ('FreeBuffers', wintypes.D

```



```
WORD), ('EventsLost', wintypes.DWORD), ('BuffersWritten', wintypes.DWORD), ('LogBuffersLost', wintypes.DWORD), ('RealTimeBuffersLost', wintypes.DWORD), ('LoggerThreadId', wintypes.HANDLE)]
```

58

```
59 class EVENT_TRACE_HEADER(ctypes.Structure):
```

```
60     _fields_ = [('Size', wintypes.USHORT), ('Reserved', wintypes.USHORT), ('Class', ctypes.c_ubyte), ('Type', ctypes.c_ubyte), ('Level', ctypes.c_ubyte), ('Version', ctypes.c_ubyte), ('Marker', wintypes.DWORD), ('ThreadId', wintypes.DWORD), ('ProcessId', wintypes.DWORD), ('TimeStamp', ctypes.c_uint64), ('ProviderId', GUID), ('ClientContext', wintypes.DWORD), ('Flags', wintypes.DWORD)]
```

61

```
62 @dataclass
```

```
63 class EtwEvent:
```

```
64     """ETW event information."""
```

```
65     timestamp: int
```

```
66     pid: int
```

```
67     tid: int
```

```
68     event_class: int
```

```
69     event_type: int
```

```
70     level: int
```

```
71     data: bytes
```

72

```
73 class EtwAPI:
```

```
74     """Wrapper for ETW event tracing functions."""
```

75

```
76     def __init__(self):
```

```
77         self.advapi32 = ctypes.windll.advapi32
```

```
78         self.ntdll = ctypes.windll.ntdll
```

```
79         self._setup_function_signatures()
```

```
80         self._session_handle = None
```

```
81         self._event_callbacks: Dict[int, List[Callable]] = {}
```

82

```
83     def _setup_function_signatures(self):
```

```
84         """Configure ctypes function signatures."""
```

```
85         self.advapi32.StartTraceW.argtypes = [ctypes.POINTER(w
```

```

intypes.HANDLE), wintypes.LPCWSTR, ctypes.POINTER(EVENT_TRACE_
PROPERTIES)])
86         self.advapi32.StartTraceW.restype = wintypes.ULONG
87         self.advapi32.StopTraceW.argtypes = [wintypes.HANDLE,
wintypes.LPCWSTR, ctypes.POINTER(EVENT_TRACE_PROPERTIES)]
88         self.advapi32.StopTraceW.restype = wintypes.ULONG
89         self.advapi32.EnableTrace.argtypes = [wintypes.ULONG,
wintypes.ULONG, wintypes.ULONG, ctypes.POINTER(GUID), wintypes
.HANDLE]
90         self.advapi32.EnableTrace.restype = wintypes.ULONG
91         self.advapi32.ControlTraceW.argtypes = [wintypes.HANDL
E, wintypes.LPCWSTR, ctypes.POINTER(EVENT_TRACE_PROPERTIES), w
intypes.DWORD]
92         self.advapi32.ControlTraceW.restype = wintypes.ULONG
93
94     def start_kernel_logger(self, enable_flags: int=EVENT_TRAC
E_FLAG_PROCESS) -> bool:
95         """
96         Start the Kernel Logger session for real-time event tr
acing.
97
98         Args:
99             enable_flags: Flags for events to capture
100
101         Returns:
102             True if successful
103         """
104         try:
105             properties = EVENT_TRACE_PROPERTIES()
106             properties.Wnode.BufferSize = ctypes.sizeof(EVENT_
TRACE_PROPERTIES)
107             properties.Wnode.Guid = GUID()
108             properties.Wnode.ClientContext = 1
109             properties.Wnode.Flags = WNODE_FLAG_TRACED_GUID
110             properties.BufferSize = 128
111             properties.MinimumBuffers = 4
112             properties.MaximumBuffers = 64
113             properties.LogFileMode = EVENT_TRACE_REAL_TIME_MOD

```

```

E | EVENT_TRACE_USE_GLOBAL_SEQUENCE
114 |         properties.FlushTimer = 100
115 |         properties.EnableFlags = enable_flags
116 |         properties.AgeLimit = 0
117 |         session_handle = wintypes.HANDLE()
118 |         status = self.advapi32.StartTraceW(ctypes.byref(session_handle), 'NT Kernel Logger', ctypes.byref(properties))
119 |         if status == 0:
120 |             self._session_handle = session_handle
121 |             return True
122 |         return False
123 |     except Exception:
124 |         return False
125 |
126 |     def stop_kernel_logger(self) -> bool:
127 |         """
128 |         Stop the Kernel Logger session.
129 |
130 |         Returns:
131 |             True if successful
132 |         """
133 |         try:
134 |             if not self._session_handle:
135 |                 return False
136 |             properties = EVENT_TRACE_PROPERTIES()
137 |             properties.Wnode.BufferSize = ctypes.sizeof(EVENT_TRACE_PROPERTIES)
138 |             status = self.advapi32.StopTraceW(self._session_handle, 'NT Kernel Logger', ctypes.byref(properties))
139 |             if status == 0:
140 |                 self._session_handle = None
141 |                 return True
142 |             return False
143 |         except Exception:
144 |             return False
145 |
146 |     def register_event_callback(self, event_type: int, callback: Callable) -> None:

```

```

147 |         """
148 |         Register a callback for a specific event type.
149 |
150 |         Args:
151 |             event_type: ETW event type
152 |             callback: Callback function to invoke
153 |         """
154 |         if event_type not in self._event_callbacks:
155 |             self._event_callbacks[event_type] = []
156 |         self._event_callbacks[event_type].append(callback)
157 |
158 |     def unregister_event_callback(self, event_type: int, callback: Callable) -> None:
159 |         """
160 |         Unregister a callback.
161 |
162 |         Args:
163 |             event_type: ETW event type
164 |             callback: Callback function to remove
165 |         """
166 |         if event_type in self._event_callbacks:
167 |             try:
168 |                 self._event_callbacks[event_type].remove(callback)
169 |             except ValueError:
170 |                 pass
171 |
172 |     def get_session_handle(self) -> Optional[wintypes.HANDLE]:
173 |         """Get the current session handle."""
174 |         return self._session_handle

```

kernel32.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  # - Kernel32
4 # =====
5 # Description:
6 #   kernel32.dll wrapper for process enumeration and snapshot
  operations.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.kernel32 import PR
  OCESSENTRY32
11 #
12 #       service = PROCESSIONARY32()
13 #
14 # File: kernel32.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """kernel32.dll wrapper for process enumeration and snapshot o
  perations."""
23
24 import ctypes
25 import ctypes.wintypes as wintypes
26 from typing import List, Optional, Tuple
27 from dataclasses import dataclass
28 from enum import IntFlag
29 TH32CS_SNAPPROCESS = 2
```

```

30 TH32CS_SNAPTHREAD = 4
31 TH32CS_SNAPMODULE = 8
32 TH32CS_SNAPMODULE32 = 16
33 ABOVE_NORMAL_PRIORITY_CLASS = 32768
34 BELOW_NORMAL_PRIORITY_CLASS = 16384
35 HIGH_PRIORITY_CLASS = 128
36 IDLE_PRIORITY_CLASS = 64
37 NORMAL_PRIORITY_CLASS = 32
38 REALTIME_PRIORITY_CLASS = 256
39
40 class PROCESSENTRY32(ctypes.Structure):
41     _fields_ = [('dwSize', wintypes.DWORD), ('th32ProcessID',
wintypes.DWORD), ('th32ParentProcessID', wintypes.DWORD), ('dw
Priority', wintypes.LONG), ('dwThreadCount', wintypes.DWORD),
('th32ModuleID', wintypes.DWORD), ('pcbUsage', wintypes.DWORD)
, ('th32AccessKey', wintypes.LPVOID), ('szExeFile', ctypes.c_c
har * 260)]
42
43 class THREADENTRY32(ctypes.Structure):
44     _fields_ = [('dwSize', wintypes.DWORD), ('th32ThreadID', w
intypes.DWORD), ('th32OwnerProcessID', wintypes.DWORD), ('tpBa
sePri', wintypes.LONG), ('tpDeltaPri', wintypes.LONG), ('dwFla
gs', wintypes.DWORD)]
45
46 class MODULEENTRY32(ctypes.Structure):
47     _fields_ = [('dwSize', wintypes.DWORD), ('th32ModuleID', w
intypes.DWORD), ('th32ProcessID', wintypes.DWORD), ('GlpvBase'
, wintypes.LPVOID), ('dwSize_', wintypes.DWORD), ('hModule', w
intypes.HMODULE), ('szModule', ctypes.c_char * 256), ('szExePa
th', ctypes.c_char * 260)]
48
49 class FILETIME(ctypes.Structure):
50     _fields_ = [('dwLowDateTime', wintypes.DWORD), ('dwHighDat
eTime', wintypes.DWORD)]
51
52 @dataclass
53 class ProcessInfo:
54     """Process snapshot information from Toolhelp32."""

```

```

55     pid: int
56     ppid: int
57     name: str
58     priority: int
59     thread_count: int
60
61 @dataclass
62 class ThreadInfo:
63     """Thread snapshot information from Toolhelp32."""
64     tid: int
65     pid: int
66     base_priority: int
67     delta_priority: int
68
69 @dataclass
70 class ModuleInfo:
71     """Module snapshot information from Toolhelp32."""
72     name: str
73     path: str
74     base_address: int
75     size: int
76
77 class Kernel32API:
78     """Wrapper for kernel32 process and thread enumeration fun
79     ctions."""
80     def __init__(self):
81         self.kernel32 = ctypes.windll.kernel32
82         self._setup_function_signatures()
83
84     def _setup_function_signatures(self):
85         """Configure ctypes function signatures for proper typ
86         e checking."""
87         self.kernel32.CreateToolhelp32Snapshot.argtypes = [win
88         types.DWORD, wintypes.DWORD]
89         self.kernel32.CreateToolhelp32Snapshot.restype = winty
89         pes.HANDLE
90         self.kernel32.Process32First.argtypes = [wintypes.HAND

```

```

    LE, ctypes.POINTER(PROCESSENTRY32)]
89         self.kernel32.Process32First.restype = wintypes.BOOL
90         self.kernel32.Process32Next.argtypes = [wintypes.HANDL
    E, ctypes.POINTER(PROCESSENTRY32)]
91         self.kernel32.Process32Next.restype = wintypes.BOOL
92         self.kernel32.Thread32First.argtypes = [wintypes.HANDL
    E, ctypes.POINTER(THREADENTRY32)]
93         self.kernel32.Thread32First.restype = wintypes.BOOL
94         self.kernel32.Thread32Next.argtypes = [wintypes.HANDLE
    , ctypes.POINTER(THREADENTRY32)]
95         self.kernel32.Thread32Next.restype = wintypes.BOOL
96         self.kernel32.Module32First.argtypes = [wintypes.HANDL
    E, ctypes.POINTER(MODULEENTRY32)]
97         self.kernel32.Module32First.restype = wintypes.BOOL
98         self.kernel32.Module32Next.argtypes = [wintypes.HANDLE
    , ctypes.POINTER(MODULEENTRY32)]
99         self.kernel32.Module32Next.restype = wintypes.BOOL
100        self.kernel32.CloseHandle.argtypes = [wintypes.HANDLE]
101        self.kernel32.CloseHandle.restype = wintypes.BOOL
102        self.kernel32.GetCurrentProcess.restype = wintypes.HAN
    DLE
103        self.kernel32.GetProcessTimes.argtypes = [wintypes.HAN
    DLE, ctypes.POINTER(FILETIME), ctypes.POINTER(FILETIME), ctype
    s.POINTER(FILETIME), ctypes.POINTER(FILETIME)]
104        self.kernel32.GetProcessTimes.restype = wintypes.BOOL
105        self.kernel32.GetPriorityClass.argtypes = [wintypes.HA
    NDLE]
106        self.kernel32.GetPriorityClass.restype = wintypes.DWOR
    D
107
108    def enumerate_processes(self) -> List[ProcessInfo]:
109        """
110        Enumerate all running processes using Toolhelp32.
111
112        Returns:
113            List of ProcessInfo objects
114        """
115        processes = []

```



```

116         snapshot = self.kernel32.CreateToolhelp32Snapshot(TH32
| CS_SNAPPROCESS, 0)
117         if snapshot == -1:
118             return processes
119         try:
120             pe = PROCESSENTRY32()
121             pe.dwSize = ctypes.sizeof(PROCESSENTRY32)
122             if self.kernel32.Process32First(snapshot, ctypes.b
| yref(pe)):
123                 while True:
124                     try:
125                         process = ProcessInfo(pid=pe.th32Proce
| ssID, ppid=pe.th32ParentProcessID, name=pe.szExeFile.decode('u
| tf-8', errors='ignore'), priority=pe.dwPriority, thread_count=
| pe.dwThreadCount)
126                         processes.append(process)
127                     except Exception:
128                         pass
129                     if not self.kernel32.Process32Next(snapsho
| t, ctypes.byref(pe)):
130                         break
131                 finally:
132                     self.kernel32.CloseHandle(snapshot)
133             return processes
134
135     def enumerate_threads(self, pid: Optional[int]=None) -> Li
| st[ThreadInfo]:
136         """
137         Enumerate threads for a process or all threads if pid
| is None.
138
139         Args:
140             pid: Process ID to filter threads, or None for all
| threads
141
142         Returns:
143             List of ThreadInfo objects
144         """

```

```

145 |         threads = []
146 |         snapshot = self.kernel32.CreateToolhelp32Snapshot(TH32
| CS_SNAPTHREAD, pid or 0)
147 |         if snapshot == -1:
148 |             return threads
149 |         try:
150 |             te = THREADENTRY32()
151 |             te.dwSize = ctypes.sizeof(THREADENTRY32)
152 |             if self.kernel32.Thread32First(snapshot, ctypes.by
| ref(te)):
153 |                 while True:
154 |                     try:
155 |                         if pid is None or te.th32OwnerProcessI
| D == pid:
156 |                             thread = ThreadInfo(tid=te.th32Thr
| eadID, pid=te.th32OwnerProcessID, base_priority=te.tpBasePri,
| delta_priority=te.tpDeltaPri)
157 |                             threads.append(thread)
158 |                         except Exception:
159 |                             pass
160 |                         if not self.kernel32.Thread32Next(snapshot
| , ctypes.byref(te)):
161 |                             break
162 |             finally:
163 |                 self.kernel32.CloseHandle(snapshot)
164 |             return threads
165 |
166 |     def enumerate_modules(self, pid: int) -> List[ModuleInfo]:
167 |         """
168 |         Enumerate modules (DLLs) loaded in a process.
169 |
170 |         Args:
171 |             pid: Process ID
172 |
173 |         Returns:
174 |             List of ModuleInfo objects
175 |         """
176 |         modules = []

```

```

177 |         snapshot = self.kernel32.CreateToolhelp32Snapshot(TH32
| CS_SNAPMODULE | TH32CS_SNAPMODULE32, pid)
178 |         if snapshot == -1:
179 |             return modules
180 |         try:
181 |             me = MODULEENTRY32()
182 |             me.dwSize = ctypes.sizeof(MODULEENTRY32)
183 |             if self.kernel32.Module32First(snapshot, ctypes.by
| ref(me)):
184 |                 while True:
185 |                     try:
186 |                         module = ModuleInfo(name=me.szModule.d
| ecode('utf-8', errors='ignore'), path=me.szExePath.decode('utf
| -8', errors='ignore'), base_address=int(me.GlpvBase), size=me.
| dwSize_)
187 |                         modules.append(module)
188 |                     except Exception:
189 |                         pass
190 |                     if not self.kernel32.Module32Next(snapshot
| , ctypes.byref(me)):
191 |                         break
192 |             finally:
193 |                 self.kernel32.CloseHandle(snapshot)
194 |             return modules
195 |
196 |     def get_process_priority_class(self, pid: int) -> Optional
| [str]:
197 |         """
198 |         Get the priority class of a process.
199 |
200 |         Args:
201 |             pid: Process ID
202 |
203 |         Returns:
204 |             Priority class name or None if error
205 |         """
206 |         try:
207 |             handle = ctypes.windll.kernel32.OpenProcess(1024,

```

```

False, pid)
208         if not handle:
209             return None
210         try:
211             priority = self.kernel32.GetPriorityClass(hand
le)
212             priority_map = {ABOVE_NORMAL_PRIORITY_CLASS: '
Above Normal', BELOW_NORMAL_PRIORITY_CLASS: 'Below Normal', HI
GH_PRIORITY_CLASS: 'High', IDLE_PRIORITY_CLASS: 'Idle', NORMAL
_PRIORITY_CLASS: 'Normal', REALTIME_PRIORITY_CLASS: 'Realtime'
}
213             return priority_map.get(priority, f'Unknown ({
priority})')
214         finally:
215             self.kernel32.CloseHandle(handle)
216     except Exception:
217         return None
218
219     def get_process_times(self, pid: int) -> Optional[Tuple[in
t, int]]:
220         """
221         Get process creation and exit times.
222
223         Args:
224             pid: Process ID
225
226         Returns:
227             Tuple of (creation_time_filetime, exit_time_fileti
me) or None
228         """
229         try:
230             handle = ctypes.windll.kernel32.OpenProcess(1024,
False, pid)
231             if not handle:
232                 return None
233             try:
234                 creation_time = FILETIME()
235                 exit_time = FILETIME()

```

```
236 |         kernel_time = FILETIME()
237 |         user_time = FILETIME()
238 |         if self.kernel32.GetProcessTimes(handle, ctypes
|         s.byref(creation_time), ctypes.byref(exit_time), ctypes.byref(
|         kernel_time), ctypes.byref(user_time)):
239 |             return (creation_time.dwLowDateTime, exit_
|         time.dwLowDateTime)
240 |         return None
241 |     finally:
242 |         self.kernel32.CloseHandle(handle)
243 | except Exception:
244 |     return None
```

nethelper.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  # - Nethelper
4 # =====
5 # Description:
6 #   Низкоуровневый интерфейс к IP Helper API (iphlpapi.dll) Wi
  ndows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_ffnethelper import M
  IB_TCPROW_OWNER_PID
11 #
12 #       service = MIB_TCPROW_OWNER_PID()
13 #
14 # File: nethelper.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_ffnethelper
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к IP Helper API (iphlpapi.dll) Win
  dows."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 import socket
28 import struct
29 from dataclasses import dataclass
```

```

30 from typing import Any, Dict, List, Optional
31 from logger import logger
32 AF_INET = 2
33 AF_INET6 = 23
34 TCP_TABLE_BASIC_LISTENER = 0
35 TCP_TABLE_BASIC_CONNECTIONS = 1
36 TCP_TABLE_BASIC_ALL = 2
37 TCP_TABLE_OWNER_PID_LISTENER = 3
38 TCP_TABLE_OWNER_PID_CONNECTIONS = 4
39 TCP_TABLE_OWNER_PID_ALL = 5
40 TCP_TABLE_OWNER_MODULE_LISTENER = 6
41 TCP_TABLE_OWNER_MODULE_CONNECTIONS = 7
42 TCP_TABLE_OWNER_MODULE_ALL = 8
43 UDP_TABLE_BASIC = 0
44 UDP_TABLE_OWNER_PID = 1
45 UDP_TABLE_OWNER_MODULE = 2
46 MIB_TCP_STATE_CLOSED = 1
47 MIB_TCP_STATE_LISTEN = 2
48 MIB_TCP_STATE_SYN_SENT = 3
49 MIB_TCP_STATE_SYN_RCVD = 4
50 MIB_TCP_STATE_ESTAB = 5
51 MIB_TCP_STATE_FIN_WAIT1 = 6
52 MIB_TCP_STATE_FIN_WAIT2 = 7
53 MIB_TCP_STATE_CLOSE_WAIT = 8
54 MIB_TCP_STATE_CLOSING = 9
55 MIB_TCP_STATE_LAST_ACK = 10
56 MIB_TCP_STATE_TIME_WAIT = 11
57 MIB_TCP_STATE_DELETE_TCB = 12
58 TCP_STATE_MAP = {MIB_TCP_STATE_CLOSED: 'CLOSED', MIB_TCP_STATE
    _LISTEN: 'LISTEN', MIB_TCP_STATE_SYN_SENT: 'SYN_SENT', MIB_TCP
    _STATE_SYN_RCVD: 'SYN_RCVD', MIB_TCP_STATE_ESTAB: 'ESTABLISHED
    ', MIB_TCP_STATE_FIN_WAIT1: 'FIN_WAIT1', MIB_TCP_STATE_FIN_WAI
    T2: 'FIN_WAIT2', MIB_TCP_STATE_CLOSE_WAIT: 'CLOSE_WAIT', MIB_T
    CP_STATE_CLOSING: 'CLOSING', MIB_TCP_STATE_LAST_ACK: 'LAST_ACK
    ', MIB_TCP_STATE_TIME_WAIT: 'TIME_WAIT', MIB_TCP_STATE_DELETE_
    TCB: 'DELETE_TCB'}
59 ERROR_INSUFFICIENT_BUFFER = 122
60

```

```
61 class MIB_TCPROW_OWNER_PID(ctypes.Structure):
62     """Структура записи TCP соединения с PID."""
63     _fields_ = [('dwState', wintypes.DWORD), ('dwLocalAddr', w
intypes.DWORD), ('dwLocalPort', wintypes.DWORD), ('dwRemoteAdd
r', wintypes.DWORD), ('dwRemotePort', wintypes.DWORD), ('dwOwn
ingPid', wintypes.DWORD)]
64
65 class MIB_UDPROW_OWNER_PID(ctypes.Structure):
66     """Структура записи UDP сокета с PID."""
67     _fields_ = [('dwLocalAddr', wintypes.DWORD), ('dwLocalPort
', wintypes.DWORD), ('dwOwningPid', wintypes.DWORD)]
68
69 class MIB_IF_ROW2(ctypes.Structure):
70     """Нативная структура MIB_IF_ROW2 из iphlpapi.dll."""
71     _fields_ = [
72         ('InterfaceLuid', ctypes.c_uint64),
73         ('InterfaceIndex', wintypes.DWORD),
74         ('InterfaceGuid', ctypes.c_byte * 16),
75         ('Alias', ctypes.c_wchar * 257),
76         ('Description', ctypes.c_wchar * 257),
77         ('PhysicalAddressLength', wintypes.DWORD),
78         ('PhysicalAddress', ctypes.c_byte * 32),
79         ('PermanentPhysicalAddress', ctypes.c_byte * 32),
80         ('Mtu', wintypes.DWORD),
81         ('Type', wintypes.DWORD),
82         ('TunnelType', ctypes.c_int),
83         ('MediaType', ctypes.c_int),
84         ('PhysicalMediumType', ctypes.c_int),
85         ('AccessType', ctypes.c_int),
86         ('DirectionType', ctypes.c_int),
87         ('InterfaceAndOperStatusFlags', ctypes.c_byte),
88         ('OperStatus', ctypes.c_int),
89         ('AdminStatus', ctypes.c_int),
90         ('MediaConnectState', ctypes.c_int),
91         ('NetworkGuid', ctypes.c_byte * 16),
92         ('ConnectionType', ctypes.c_int),
93         ('TransmitLinkSpeed', ctypes.c_uint64),
94         ('ReceiveLinkSpeed', ctypes.c_uint64),
```



```

95 |         ('InOctets', ctypes.c_uint64),
96 |         ('InUcastPkts', ctypes.c_uint64),
97 |         ('InNUcastPkts', ctypes.c_uint64),
98 |         ('InDiscards', ctypes.c_uint64),
99 |         ('InErrors', ctypes.c_uint64),
100 |         ('InUnknownProtos', ctypes.c_uint64),
101 |         ('InUcastOctets', ctypes.c_uint64),
102 |         ('InMulticastOctets', ctypes.c_uint64),
103 |         ('InBroadcastOctets', ctypes.c_uint64),
104 |         ('OutOctets', ctypes.c_uint64),
105 |         ('OutUcastPkts', ctypes.c_uint64),
106 |         ('OutNUcastPkts', ctypes.c_uint64),
107 |         ('OutDiscards', ctypes.c_uint64),
108 |         ('OutErrors', ctypes.c_uint64),
109 |         ('OutUcastOctets', ctypes.c_uint64),
110 |         ('OutMulticastOctets', ctypes.c_uint64),
111 |         ('OutBroadcastOctets', ctypes.c_uint64),
112 |         ('OutQLen', ctypes.c_uint64),
113 |     ]
114 |
115 | class MIB_IF_TABLE2(ctypes.Structure):
116 |     """Таблица сетевых интерфейсов MIB_IF_TABLE2."""
117 |     _fields_ = [
118 |         ('NumEntries', wintypes.DWORD),
119 |         ('Table', MIB_IF_ROW2 * 1),
120 |     ]
121 |
122 | class MIB_IPNETROW(ctypes.Structure):
123 |     """Структура записи таблицы ARP (MIB_IPNETROW)."""
124 |     _fields_ = [
125 |         ('dwIndex', wintypes.DWORD),
126 |         ('dwPhysAddrLen', wintypes.DWORD),
127 |         ('bPhysAddr', ctypes.c_ubyte * 8),
128 |         ('dwAddr', wintypes.DWORD),
129 |         ('dwType', wintypes.DWORD),
130 |     ]
131 |
132 | class MIB_IPNETTABLE(ctypes.Structure):

```

```
133 |     """Таблица записей ARP (MIB_IPNETTABLE)."""
134 |     _fields_ = [
135 |         ('dwNumEntries', wintypes.DWORD),
136 |         ('table', MIB_IPNETROW * 1),
137 |     ]
138 |
139 | IP_NET_TYPE_MAP = {
140 |     1: 'OTHER',
141 |     2: 'INVALID',
142 |     3: 'DYNAMIC',
143 |     4: 'STATIC',
144 | }
145 |
146 | @dataclass
147 | class NetworkSocketInfo:
148 |     """Нормализованные данные о сетевом соединении/сожете."""
149 |     protocol: str
150 |     local_address: str
151 |     local_port: int
152 |     remote_address: str
153 |     remote_port: int
154 |     state: str
155 |     pid: int
156 |
157 |     def to_dict(self) -> Dict[str, Any]:
158 |         """Преобразование в словарь."""
159 |         return {'protocol': self.protocol, 'local_address': se
160 | self.local_address, 'local_port': self.local_port, 'remote_addre
161 ss': self.remote_address, 'remote_port': self.remote_port, 'st
162 ate': self.state, 'pid': self.pid}
163 |
164 | class IPHelperAPI:
165 |     """Нативный интерфейс к iphlapi.dll."""
166 |
167 |     def __init__(self) -> None:
168 |         """Инициализация функций iphlapi.dll."""
169 |         self._iphlpapi = ctypes.windll.iphlapi
170 |         self._GetExtendedTcpTable = self._iphlpapi.GetExtended
```

```

    TcpTable
168         self._GetExtendedTcpTable.argtypes = [ctypes.c_void_p,
    ctypes.POINTER(wintypes.DWORD), wintypes.BOOL, wintypes.ULONG
    , ctypes.c_int, wintypes.ULONG]
169         self._GetExtendedTcpTable.restype = wintypes.DWORD
170         self._GetExtendedUdpTable = self._iphlpapi.GetExtended
    UdpTable
171         self._GetExtendedUdpTable.argtypes = [ctypes.c_void_p,
    ctypes.POINTER(wintypes.DWORD), wintypes.BOOL, wintypes.ULONG
    , ctypes.c_int, wintypes.ULONG]
172         self._GetExtendedUdpTable.restype = wintypes.DWORD
173
174         # SendARP
175         self._SendARP = getattr(self._iphlpapi, 'SendARP', Non
    e)
176         if self._SendARP:
177             self._SendARP.argtypes = [wintypes.ULONG, wintypes
    .ULONG, ctypes.c_void_p, ctypes.POINTER(wintypes.ULONG)]
178             self._SendARP.restype = wintypes.DWORD
179
180         # GetIpNetTable
181         self._GetIpNetTable = getattr(self._iphlpapi, 'GetIpNe
    tTable', None)
182         if self._GetIpNetTable:
183             self._GetIpNetTable.argtypes = [ctypes.c_void_p, c
    ctypes.POINTER(wintypes.ULONG), wintypes.BOOL]
184             self._GetIpNetTable.restype = wintypes.DWORD
185
186         if hasattr(self._iphlpapi, 'GetIfTable2'):
187             self._GetIfTable2 = self._iphlpapi.GetIfTable2
188             self._GetIfTable2.argtypes = [ctypes.POINTER(ctype
    s.c_void_p)]
189             self._GetIfTable2.restype = wintypes.DWORD
190             self._FreeMibTable = getattr(self._iphlpapi, 'Free
    MibTable', None)
191             if self._FreeMibTable:
192                 self._FreeMibTable.argtypes = [ctypes.c_void_p
    ]

```

```

193         else:
194             self._GetIfTable2 = None
195
196     @staticmethod
197     def _ip_to_str(ip_int: int) -> str:
198         """Преобразование сетевого порядка байт IP в строку."""
199
200         try:
201             return socket.inet_ntoa(struct.pack('<L', ip_int))
202         except Exception:
203             return '0.0.0.0'
204
205     @staticmethod
206     def _port_to_int(port_int: int) -> int:
207         """Преобразование сетевого порядка байт порта в int."""
208
209         return port_int >> 8 & 255 | (port_int & 255) << 8
210
211     def get_adapter_statistics(self) -> List[Dict[str, Any]]:
212         """Получение накопительной статистики по всем сетевым
213         адаптерам через GetIfTable2.
214
215         Returns:
216             List[Dict[str, Any]]: Список словарей со статистик
217             ой интерфейсов.
218
219         """
220         if not self._GetIfTable2:
221             return []
222         p_table = ctypes.c_void_p()
223         res = self._GetIfTable2(ctypes.byref(p_table))
224         if res != 0 or not p_table:
225             return []
226         try:
227             num_entries = ctypes.cast(p_table, ctypes.POINTER(
228                 wintypes.DWORD)).contents.value
229             offset = ctypes.sizeof(wintypes.DWORD)
230             row_size = ctypes.sizeof(MIB_IF_ROW2)
231             base_addr = p_table.value or 0

```

```

226         if not base_addr:
227             return []
228         stats: List[Dict[str, Any]] = []
229         for i in range(num_entries):
230             row_addr = base_addr + offset + i * row_size
231             row = MIB_IF_ROW2.from_address(row_addr)
232             name = row.Alias or row.Description or f'Inter
face_{row.InterfaceIndex}'
233             stats.append({
234                 'name': name,
235                 'alias': row.Alias,
236                 'description': row.Description,
237                 'interface_index': row.InterfaceIndex,
238                 'received_bytes': row.InOctets,
239                 'sent_bytes': row.OutOctets,
240                 'received_packets': row.InUcastPkts + row.
InNUcastPkts,
241                 'sent_packets': row.OutUcastPkts + row.Out
NUcastPkts,
242                 'received_discarded': row.InDiscards,
243                 'received_errors': row.InErrors,
244                 'outbound_discarded': row.OutDiscards,
245                 'outbound_errors': row.OutErrors,
246                 'transmit_speed_bps': row.TransmitLinkSpee
d,
247                 'receive_speed_bps': row.ReceiveLinkSpeed,
248                 'oper_status': row.OperStatus,
249             })
250         return stats
251     finally:
252         if hasattr(self, '_FreeMibTable') and self._FreeMi
bTable and p_table:
253             self._FreeMibTable(p_table)
254
255     def get_tcp_connections(self) -> List[NetworkSocketInfo]:
256         """Получение всех активных TCP соединений и слушающих
портов с PID.
257

```

```

258         Returns:
259             List[NetworkSocketInfo]: Список соединений.
260         """
261         size = wintypes.DWORD(0)
262         self._GetExtendedTcpTable(None, ctypes.byref(size), True, AF_INET, TCP_TABLE_OWNER_PID_ALL, 0)
263         if size.value == 0:
264             return []
265         buf = (ctypes.c_byte * size.value)()
266         res = self._GetExtendedTcpTable(buf, ctypes.byref(size), True, AF_INET, TCP_TABLE_OWNER_PID_ALL, 0)
267         if res != 0:
268             return []
269         num_entries = ctypes.cast(buf, ctypes.POINTER(wintypes.DWORD)).contents.value
270         row_size = ctypes.sizeof(MIB_TCPROW_OWNER_PID)
271         offset = ctypes.sizeof(wintypes.DWORD)
272         connections: List[NetworkSocketInfo] = []
273         for i in range(num_entries):
274             row_ptr = ctypes.byref(buf, offset + i * row_size)
275             row = ctypes.cast(row_ptr, ctypes.POINTER(MIB_TCPROW_OWNER_PID)).contents
276             state_str = TCP_STATE_MAP.get(row.dwState, f'STATE_{row.dwState}')
277             local_ip = self._ip_to_str(row.dwLocalAddr)
278             local_port = self._port_to_int(row.dwLocalPort)
279             remote_ip = self._ip_to_str(row.dwRemoteAddr)
280             remote_port = self._port_to_int(row.dwRemotePort)
281             if row.dwState != MIB_TCP_STATE_LISTEN else 0:
282                 connections.append(NetworkSocketInfo(protocol='TCP', local_address=local_ip, local_port=local_port, remote_address=remote_ip, remote_port=remote_port, state=state_str, pid=row.dwOwningPid))
283
284         return connections
285
286     def get_udp_sockets(self) -> List[NetworkSocketInfo]:
287         """Получение всех UDP слушающих сокетов с привязкой к PID.

```

```

286
287     Returns:
288         List[NetworkSocketInfo]: Список UDP сокетов.
289         """
290         size = wintypes.DWORD(0)
291         self._GetExtendedUdpTable(None, ctypes.byref(size), True, AF_INET, UDP_TABLE_OWNER_PID, 0)
292         if size.value == 0:
293             return []
294         buf = (ctypes.c_byte * size.value)()
295         res = self._GetExtendedUdpTable(buf, ctypes.byref(size), True, AF_INET, UDP_TABLE_OWNER_PID, 0)
296         if res != 0:
297             return []
298         num_entries = ctypes.cast(buf, ctypes.POINTER(wintypes.DWORD)).contents.value
299         row_size = ctypes.sizeof(MIB_UDPROW_OWNER_PID)
300         offset = ctypes.sizeof(wintypes.DWORD)
301         sockets_list: List[NetworkSocketInfo] = []
302         for i in range(num_entries):
303             row_ptr = ctypes.byref(buf, offset + i * row_size)
304             row = ctypes.cast(row_ptr, ctypes.POINTER(MIB_UDPROW_OWNER_PID)).contents
305             local_ip = self._ip_to_str(row.dwLocalAddr)
306             local_port = self._port_to_int(row.dwLocalPort)
307             sockets_list.append(NetworkSocketInfo(protocol='UDP', local_address=local_ip, local_port=local_port, remote_address='*.*.*.*', remote_port=0, state='LISTEN', pid=row.dwOwningPid))
308         return sockets_list
309
310     def send_arp(self, dest_ip: str, src_ip: Optional[str] = None) -> Optional[str]:
311         """Отправка ARP-запроса через SendARP API Windows и получение MAC-адреса.
312
313         Args:
314             dest_ip: Целевой IPv4-адрес.

```

```

315 |         src_ip: Исходный IPv4-адрес (опционально, 0 по умо
| лчанию).
316 |
317 |     Returns:
318 |         Optional[str]: Отформатированный MAC-адрес (e.g. '
| 00:1A:2B:3C:4D:5E') или None при отсутствии ответа.
319 |         """
320 |         if not getattr(self, '_SendARP', None):
321 |             return None
322 |         try:
323 |             dest_addr = struct.unpack('<L', socket.inet_aton(d
| est_ip))[0]
324 |             src_addr = struct.unpack('<L', socket.inet_aton(sr
| c_ip))[0] if src_ip else 0
325 |             mac_buf = (ctypes.c_ubyte * 6)()
326 |             mac_len = wintypes.ULONG(6)
327 |             res = self._SendARP(dest_addr, src_addr, ctypes.by
| ref(mac_buf), ctypes.byref(mac_len))
328 |             if res == 0 and mac_len.value == 6:
329 |                 return ':'.join(f'{b:02X}' for b in mac_buf)
330 |             return None
331 |         except Exception as ex:
332 |             logger.debug(f'SendARP error for {dest_ip}: {ex}')
333 |             return None
334 |
335 |     def get_ip_net_table(self) -> List[Dict[str, Any]]:
336 |         """Получение таблицы ARP-записей соседей ядра Windows
| через GetIpNetTable.
337 |
338 |     Returns:
339 |         List[Dict[str, Any]]: Список записей ARP соседей (
| ip, mac, interface_index, type).
340 |         """
341 |         if not getattr(self, '_GetIpNetTable', None):
342 |             return []
343 |         try:
344 |             size = wintypes.ULONG(0)
345 |             self._GetIpNetTable(None, ctypes.byref(size), True

```



```

    )
346         if size.value == 0:
347             return []
348         buf = (ctypes.c_byte * size.value)()
349         res = self._GetIpNetTable(buf, ctypes.byref(size),
    True)
350         if res != 0:
351             return []
352         num_entries = ctypes.cast(buf, ctypes.POINTER(wint
types.DWORD)).contents.value
353         row_size = ctypes.sizeof(MIB_IPNETROW)
354         offset = ctypes.sizeof(wintypes.DWORD)
355         neighbors: List[Dict[str, Any]] = []
356         for i in range(num_entries):
357             row_ptr = ctypes.byref(buf, offset + i * row_s
ize)
358             row = ctypes.cast(row_ptr, ctypes.POINTER(MIB_
IPNETROW)).contents
359             ip = self._ip_to_str(row.dwAddr)
360             if row.dwPhysAddrLen == 6:
361                 mac = ':'.join(f'{b:02X}' for b in row.bPh
ysAddr[:6])
362             else:
363                 mac = ''
364             net_type = IP_NET_TYPE_MAP.get(row.dwType, f'T
YPE_{row.dwType}')
365             if ip and ip != '0.0.0.0' and not ip.startswit
h('255.255.255') and not ip.startswith('224.') and net_type !=
'INVALID':
366                 neighbors.append({
367                     'ip': ip,
368                     'mac': mac,
369                     'interface_index': row.dwIndex,
370                     'type': net_type,
371                 })
372         return neighbors
373     except Exception as ex:
374         logger.debug(f'GetIpNetTable error: {ex}')

```


ntdll.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Ntdll
5 # Description:
6 #   ntdll.dll wrapper for native NT API functions.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.ntdll import SYSTEM_
11 #       M_INFO
12 #       service = SYSTEM_INFO()
13 #
14 # File: ntdll.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """ntdll.dll wrapper for native NT API functions."""
23
24 import ctypes
25 import ctypes.wintypes as wintypes
26 from typing import Optional, Any
27 from dataclasses import dataclass
28 STATUS_SUCCESS = 0
29 STATUS_BUFFER_TOO_SMALL = 3221225507
30 SystemBasicInformation = 0
31 SystemProcessorInformation = 1
```

32 SystemPerformanceInformation = 2
33 SystemTimeOfDayInformation = 3
34 SystemPathInformation = 4
35 SystemProcessInformation = 5
36 SystemCallCountInformation = 6
37 SystemDeviceInformation = 7
38 SystemProcessorPerformanceInformation = 8
39 SystemFlagsInformation = 9
40 SystemCallTimeInformation = 10
41 SystemModuleInformation = 11
42 SystemLocksInformation = 12
43 SystemStackTraceInformation = 13
44 SystemPagedPoolInformation = 14
45 SystemNonPagedPoolInformation = 15
46 SystemHandleInformation = 16
47 SystemObjectInformation = 17
48 SystemPageFileInformation = 18
49 SystemVdmInstemulInformation = 19
50 SystemVdmBopInformation = 20
51 SystemFileCacheInformation = 21
52 SystemPoolTagInformation = 22
53 SystemInterruptInformation = 23
54 SystemDpcBehaviorInformation = 24
55 SystemFullMemoryInformation = 25
56 SystemLoadGdiDriverInformation = 26
57 SystemUnloadGdiDriverInformation = 27
58 SystemTimeAdjustmentInformation = 28
59 SystemSummaryMemoryInformation = 29
60 SystemMirrorInformation = 30
61 SystemPerformanceTraceInformation = 31
62 SystemCrashDumpInformation = 32
63 SystemExceptionInformation = 33
64 SystemCrashDumpStateInformation = 34
65 SystemDebuggerInformation = 35
66 SystemProcessorSpeedInformation = 36
67 SystemNUMAProximityNodeInformation = 37
68 SystemDpcWatchdogInformation = 38
69 SystemDpcWatchdogInformation2 = 39

```

70 SystemInvalidProcessorSpeedInformation = 40
71
72 class SYSTEM_INFO(ctypes.Structure):
73     _fields_ = [('dwOemId', wintypes.DWORD), ('dwPageSize', wintypes.DWORD), ('lpMinimumApplicationAddress', wintypes.LPVOID), ('lpMaximumApplicationAddress', wintypes.LPVOID), ('dwActiveProcessorMask', wintypes.DWORD), ('dwNumberOfProcessors', wintypes.DWORD), ('dwProcessorType', wintypes.DWORD), ('dwAllocationGranularity', wintypes.DWORD), ('wProcessorLevel', wintypes.WORD), ('wProcessorRevision', wintypes.WORD)]
74
75 class UNICODE_STRING(ctypes.Structure):
76     _fields_ = [('Length', wintypes.USHORT), ('MaximumLength', wintypes.USHORT), ('Buffer', wintypes.LPWSTR)]
77
78 class OBJECT_NAME_INFORMATION(ctypes.Structure):
79     _fields_ = [('Name', UNICODE_STRING)]
80
81 class PROCESS_BASIC_INFORMATION(ctypes.Structure):
82     _fields_ = [('Reserved1', wintypes.LPVOID), ('PebBaseAddress', wintypes.LPVOID), ('Reserved2_0', wintypes.LPVOID), ('Reserved2_1', wintypes.LPVOID), ('UniqueProcessId', wintypes.LPVOID), ('Reserved3', wintypes.LPVOID)]
83
84 class RTL_OSVERSIONINFOEXW(ctypes.Structure):
85     _fields_ = [('dwOSVersionInfoSize', wintypes.DWORD), ('dwMajorVersion', wintypes.DWORD), ('dwMinorVersion', wintypes.DWORD), ('dwBuildNumber', wintypes.DWORD), ('dwPlatformId', wintypes.DWORD), ('szCSDVersion', ctypes.c_wchar * 128), ('wServicePackMajor', wintypes.WORD), ('wServicePackMinor', wintypes.WORD), ('wSuiteMask', wintypes.WORD), ('wProductType', wintypes.BYTE), ('wReserved', wintypes.BYTE)]
86
87 @dataclass
88 class VersionInfo:
89     """Windows version information."""
90     major: int
91     minor: int

```

```

92     build: int
93     platform: str
94
95 class NtdllAPI:
96     """Wrapper for ntdll native API functions."""
97
98     def __init__(self):
99         self.ntdll = ctypes.windll.ntdll
100         self._setup_function_signatures()
101
102     def _setup_function_signatures(self):
103         """Configure ctypes function signatures."""
104         self.ntdll.NtQuerySystemInformation.argtypes = [wintypes.DWORD, wintypes.LPVOID, wintypes.ULONG, ctypes.POINTER(wintypes.ULONG)]
105         self.ntdll.NtQuerySystemInformation.restype = wintypes.LONG
106         self.ntdll.NtQueryObject.argtypes = [wintypes.HANDLE, wintypes.DWORD, wintypes.LPVOID, wintypes.ULONG, ctypes.POINTER(wintypes.ULONG)]
107         self.ntdll.NtQueryObject.restype = wintypes.LONG
108         self.ntdll.NtQueryInformationProcess.argtypes = [wintypes.HANDLE, wintypes.DWORD, wintypes.LPVOID, wintypes.ULONG, ctypes.POINTER(wintypes.ULONG)]
109         self.ntdll.NtQueryInformationProcess.restype = wintypes.LONG
110         self.ntdll.RtlGetVersion.argtypes = [ctypes.POINTER(RTL_OSVERSIONINFOEXW)]
111         self.ntdll.RtlGetVersion.restype = wintypes.LONG
112
113     def get_windows_version(self) -> Optional[VersionInfo]:
114         """
115         Get Windows version information.
116
117         Returns:
118             VersionInfo object or None
119         """
120         try:

```

```

121         version_info = RTL_OSVERSIONINFOEXW()
122         version_info.dwOSVersionInfoSize = ctypes.sizeof(R
TL_OSVERSIONINFOEXW)
123         status = self.ntdll.RtlGetVersion(ctypes.byref(ver
sion_info))
124         if status == STATUS_SUCCESS:
125             platform_map = {0: 'Win32s', 1: 'Windows 9x',
126             2: 'Windows NT/2000/XP/Vista/7/8/10/11'}
127             return VersionInfo(major=version_info.dwMajorV
ersion, minor=version_info.dwMinorVersion, build=version_info.
dwBuildNumber, platform=platform_map.get(version_info.dwPlatfo
rmId, 'Unknown'))
128         return None
129     except Exception:
130         return None
131
132     def query_system_information(self, info_class: int) -> Opt
ional[bytes]:
133         """
134         Query system information.
135
136         Args:
137             info_class: System information class
138
139         Returns:
140             System information bytes or None
141         """
142         try:
143             buffer_size = 8192
144             buffer = ctypes.create_string_buffer(buffer_size)
145             return_length = wintypes.ULONG()
146             status = self.ntdll.NtQuerySystemInformation(info_
class, buffer, buffer_size, ctypes.byref(return_length))
147             if status == STATUS_SUCCESS:
148                 return buffer.raw[:return_length.value]
149             return None
150         except Exception:
151             return None

```

```

151
152     def query_object_name(self, handle: int) -> Optional[str]:
153         """
154         Query the name of a kernel object.
155
156         Args:
157             handle: Object handle
158
159         Returns:
160             Object name or None
161         """
162         try:
163             buffer_size = 1024
164             buffer = ctypes.create_string_buffer(buffer_size)
165             return_length = wintypes.ULONG()
166             status = self.ntdll.NtQueryObject(handle, 1, buffer,
167 r, buffer_size, ctypes.byref(return_length))
168             if status == STATUS_SUCCESS:
169                 obj_name_info = ctypes.cast(buffer, ctypes.POINTER(OBJECT_NAME_INFORMATION))
170                 if obj_name_info.contents.Name.Buffer:
171                     return obj_name_info.contents.Name.Buffer
172                 return None
173             except Exception:
174                 return None
175
176     def query_process_basic_info(self, pid: int) -> Optional[PROCESS_BASIC_INFORMATION]:
177         """
178         Query basic process information.
179
180         Args:
181             pid: Process ID
182
183         Returns:
184             PROCESS_BASIC_INFORMATION structure or None
185         """
186         try:

```



```
186 |         handle = ctypes.windll.kernel32.OpenProcess(1024,  
    | False, pid)  
187 |         if not handle:  
188 |             return None  
189 |         try:  
190 |             pbi = PROCESS_BASIC_INFORMATION()  
191 |             return_length = wintypes.ULONG()  
192 |             status = self.ntdll.NtQueryInformationProcess(  
    | handle, 0, ctypes.byref(pbi), ctypes.sizeof(pbi), ctypes.byref  
    | (return_length))  
193 |             if status == STATUS_SUCCESS:  
194 |                 return pbi  
195 |             return None  
196 |         finally:  
197 |             ctypes.windll.kernel32.CloseHandle(handle)  
198 |     except Exception:  
199 |         return None
```

psapi.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Psapi
5 # Description:
6 #   psapi.dll wrapper for process memory and module enumeration.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_ffapi import PROCESS_MEMORY_COUNTERS
11 #
12 #       service = PROCESS_MEMORY_COUNTERS()
13 #
14 # File: psapi.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_ffapi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """psapi.dll wrapper for process memory and module enumeration
23 ."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 from typing import List, Optional, Tuple
28 from dataclasses import dataclass
29 LIST_MODULES_DEFAULT = 0
30 LIST_MODULES_32BIT = 1
```

```

30 LIST_MODULES_64BIT = 2
31 LIST_MODULES_ALL = 3
32 SIZE_T = getattr(wintypes, 'SIZE_T', ctypes.c_size_t)
33
34 class PROCESS_MEMORY_COUNTERS(ctypes.Structure):
35     _fields_ = [('cb', wintypes.DWORD), ('PageFaultCount', win
types.DWORD), ('PeakWorkingSetSize', SIZE_T), ('WorkingSetSize
', SIZE_T), ('QuotaPeakPagedPoolUsage', SIZE_T), ('QuotaPagedP
oolUsage', SIZE_T), ('QuotaPeakNonPagedPoolUsage', SIZE_T), ('
QuotaNonPagedPoolUsage', SIZE_T), ('PagefileUsage', SIZE_T), (
'PeakPagefileUsage', SIZE_T)]
36
37 @dataclass
38 class MemoryInfo:
39     """Process memory statistics from psapi."""
40     page_faults: int
41     peak_working_set: int
42     working_set: int
43     paged_pool_peak: int
44     paged_pool_usage: int
45     nonpaged_pool_peak: int
46     nonpaged_pool_usage: int
47     pagefile_usage: int
48     pagefile_peak: int
49
50 class PsapiAPI:
51     """Wrapper for psapi process memory and module functions."
    ""
52
53     def __init__(self):
54         self.psapi = ctypes.windll.psapi
55         self._setup_function_signatures()
56
57     def _setup_function_signatures(self):
58         """Configure ctypes function signatures."""
59         self.psapi.EnumProcesses.argtypes = [ctypes.POINTER(wi
ntypes.DWORD), wintypes.DWORD, ctypes.POINTER(wintypes.DWORD)]
60         self.psapi.EnumProcesses.restype = wintypes.BOOL

```

```

61         self.psapi.GetProcessMemoryInfo.argtypes = [wintypes.H
HANDLE, ctypes.POINTER(PROCESS_MEMORY_COUNTERS), wintypes.DWORD
]
62         self.psapi.GetProcessMemoryInfo.restype = wintypes.BOOL
63         self.psapi.EnumProcessModules.argtypes = [wintypes.HAN
DLE, ctypes.POINTER(wintypes.HMODULE), wintypes.DWORD, ctypes.
POINTER(wintypes.DWORD)]
64         self.psapi.EnumProcessModules.restype = wintypes.BOOL
65         self.psapi.EnumProcessModulesEx.argtypes = [wintypes.H
HANDLE, ctypes.POINTER(wintypes.HMODULE), wintypes.DWORD, ctype
s.POINTER(wintypes.DWORD), wintypes.DWORD]
66         self.psapi.EnumProcessModulesEx.restype = wintypes.BOOL
67         self.psapi.GetModuleFileNameExW.argtypes = [wintypes.H
HANDLE, wintypes.HMODULE, wintypes.LPWSTR, wintypes.DWORD]
68         self.psapi.GetModuleFileNameExW.restype = wintypes.DWORD
69         self.psapi.GetProcessImageFileNameW.argtypes = [wintyp
es.HANDLE, wintypes.LPWSTR, wintypes.DWORD]
70         self.psapi.GetProcessImageFileNameW.restype = wintypes
.DWORD
71
72     def enumerate_processes(self) -> List[int]:
73         """
74         Get all process IDs.
75
76         Returns:
77             List of process IDs
78         """
79         max_pids = 1024
80         pids = (wintypes.DWORD * max_pids)()
81         count_ptr = wintypes.DWORD()
82         if self.psapi.EnumProcesses(pids, ctypes.sizeof(pids),
ctypes.byref(count_ptr)):
83             count = count_ptr.value // ctypes.sizeof(wintypes.
DWORD)
84             return list(pids[:count])

```

```

85         return []
86
87     def get_process_memory_info(self, pid: int) -> Optional[Me
memoryInfo]:
88         """
89         Get memory statistics for a process.
90
91         Args:
92             pid: Process ID
93
94         Returns:
95             MemoryInfo object or None
96         """
97         try:
98             handle = ctypes.windll.kernel32.OpenProcess(1024,
False, pid)
99             if not handle:
100                 return None
101             try:
102                 pmc = PROCESS_MEMORY_COUNTERS()
103                 pmc.cb = ctypes.sizeof(PROCESS_MEMORY_COUNTERS
)
104                 if self.psapi.GetProcessMemoryInfo(handle, cty
pes.byref(pmc), ctypes.sizeof(pmc)):
105                     return MemoryInfo(page_faults=pmc.PageFaul
tCount, peak_working_set=pmc.PeakWorkingSetSize, working_set=p
mc.WorkingSetSize, paged_pool_peak=pmc.QuotaPeakPagedPoolUsage
, paged_pool_usage=pmc.QuotaPagedPoolUsage, nonpaged_pool_peak
=pmc.QuotaPeakNonPagedPoolUsage, nonpaged_pool_usage=pmc.Quota
NonPagedPoolUsage, pagefile_usage=pmc.PagefileUsage, pagefile_
peak=pmc.PeakPagefileUsage)
106                 return None
107             finally:
108                 ctypes.windll.kernel32.CloseHandle(handle)
109         except Exception:
110             return None
111
112     def enumerate_process_modules(self, pid: int, filter_type:

```

```

    int=LIST_MODULES_ALL) -> List[str]:
113         """
114         Get loaded modules for a process.
115
116         Args:
117             pid: Process ID
118             filter_type: Module filter (32-bit, 64-bit, or all
119         )
120
121         Returns:
122             List of module paths
123         """
124         modules = []
125         try:
126             handle = ctypes.windll.kernel32.OpenProcess(4096 |
127                 1024, False, pid)
128             if not handle:
129                 return modules
130             try:
131                 max_modules = 256
132                 module_handles = (wintypes.HMODULE * max_modul
133             es)()
134                 count_ptr = wintypes.DWORD()
135                 if self.psapi.EnumProcessModulesEx(handle, mod
136             ule_handles, ctypes.sizeof(module_handles), ctypes.byref(count
137             _ptr), filter_type):
138                     count = count_ptr.value // ctypes.sizeof(w
139             intypes.HMODULE)
140                     for i in range(count):
141                         filename_buffer = ctypes.create_unicod
142             e_buffer(260)
143                         if self.psapi.GetModuleFileNameExW(han
144             dle, module_handles[i], filename_buffer, len(filename_buffer))
145             :
146                             modules.append(filename_buffer.val
147             ue)
148             finally:
149                 ctypes.windll.kernel32.CloseHandle(handle)

```

```
140         except Exception:
141             pass
142         return modules
143
144     def get_process_image_filename(self, pid: int) -> Optional
145     [str]:
146         """
147         Get the executable path of a process.
148
149         Args:
150             pid: Process ID
151
152         Returns:
153             Process image filename or None
154         """
155         try:
156             handle = ctypes.windll.kernel32.OpenProcess(1024,
157             False, pid)
158             if not handle:
159                 return None
160             try:
161                 filename_buffer = ctypes.create_unicode_buffer
162                 (260)
163                 if self.psapi.GetProcessImageFileNameW(handle,
164                 filename_buffer, len(filename_buffer)):
165                     return filename_buffer.value
166                 return None
167             finally:
168                 ctypes.windll.kernel32.CloseHandle(handle)
169         except Exception:
170             return None
```

scm.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Scm
5 # Description:
6 # Низкоуровневый интерфейс к Service Control Manager (advapi
7 # 32.dll) Windows.
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.telemetry.win32_fffi.scm import SERVICE
11 # _STATUS_PROCESS
12 # service = SERVICE_STATUS_PROCESS()
13 #
14 # File: scm.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к Service Control Manager (advapi3
24 2.dll) Windows."""
25
26 import ctypes
27 import ctypes.wintypes as wintypes
28 from dataclasses import dataclass
29 from typing import Any, Dict, List, Optional
30 from logger import logger
```



```

30 SC_MANAGER_CONNECT = 1
31 SC_MANAGER_CREATE_SERVICE = 2
32 SC_MANAGER_ENUMERATE_SERVICE = 4
33 SC_MANAGER_LOCK = 8
34 SC_MANAGER_QUERY_LOCK_STATUS = 16
35 SC_MANAGER_MODIFY_BOOT_CONFIG = 32
36 SC_MANAGER_ALL_ACCESS = 983103
37 SERVICE_WIN32_OWN_PROCESS = 16
38 SERVICE_WIN32_SHARE_PROCESS = 32
39 SERVICE_DRIVER = 11
40 SERVICE_TYPE_ALL = SERVICE_WIN32_OWN_PROCESS | SERVICE_WIN32_S
    HARE_PROCESS | SERVICE_DRIVER
41 SERVICE_ACTIVE = 1
42 SERVICE_INACTIVE = 2
43 SERVICE_STATE_ALL = SERVICE_ACTIVE | SERVICE_INACTIVE
44 SERVICE_STOPPED = 1
45 SERVICE_START_PENDING = 2
46 SERVICE_STOP_PENDING = 3
47 SERVICE_RUNNING = 4
48 SERVICE_CONTINUE_PENDING = 5
49 SERVICE_PAUSE_PENDING = 6
50 SERVICE_PAUSED = 7
51 SERVICE_QUERY_CONFIG = 1
52 SERVICE_CHANGE_CONFIG = 2
53 SERVICE_QUERY_STATUS = 4
54 SERVICE_ENUMERATE_DEPENDENTS = 8
55 SERVICE_START = 16
56 SERVICE_STOP = 32
57 SERVICE_PAUSE_CONTINUE = 64
58 SERVICE_INTERROGATE = 128
59 SERVICE_USER_DEFINED_CONTROL = 256
60 SERVICE_ALL_ACCESS = 983551
61 SC_ENUM_PROCESS_INFO = 0
62 ERROR_MORE_DATA = 234
63
64 class SERVICE_STATUS_PROCESS(ctypes.Structure):
65     """Структура статуса и процесса службы."""
66     _fields_ = [('dwServiceType', wintypes.DWORD), ('dwCurrent

```

```

State', wintypes.DWORD), ('dwControlsAccepted', wintypes.DWORD
), ('dwWin32ExitCode', wintypes.DWORD), ('dwServiceSpecificExi
tCode', wintypes.DWORD), ('dwCheckPoint', wintypes.DWORD), ('d
wWaitHint', wintypes.DWORD), ('dwProcessId', wintypes.DWORD),
('dwServiceFlags', wintypes.DWORD)]
67
68 class ENUM_SERVICE_STATUS_PROCESSW(ctypes.Structure):
69     """Структура элемента перечисления служб с PID."""
70     _fields_ = [('lpServiceName', wintypes.LPWSTR), ('lpDispla
yName', wintypes.LPWSTR), ('ServiceStatusProcess', SERVICE_STA
TUS_PROCESS)]
71
72 class QUERY_SERVICE_CONFIGW(ctypes.Structure):
73     """Структура детальной конфигурации службы."""
74     _fields_ = [('dwServiceType', wintypes.DWORD), ('dwStartTy
pe', wintypes.DWORD), ('dwErrorControl', wintypes.DWORD), ('lp
BinaryPathName', wintypes.LPWSTR), ('lpLoadOrderGroup', wintyp
es.LPWSTR), ('dwTagId', wintypes.DWORD), ('lpDependencies', wi
ntypes.LPWSTR), ('lpServiceStartName', wintypes.LPWSTR), ('lpD
isplayName', wintypes.LPWSTR)]
75
76 @dataclass
77 class ServiceInfo:
78     """Нормализованные данные о службе Windows."""
79     name: str
80     display_name: str
81     state: str
82     state_code: int
83     pid: int
84     service_type: int
85     binary_path: str = ''
86     start_type: str = ''
87     account: str = ''
88
89     def to_dict(self) -> Dict[str, Any]:
90         """Преобразование в словарь."""
91         return {'name': self.name, 'display_name': self.displa
y_name, 'state': self.state, 'state_code': self.state_code, 'p

```

```

id': self.pid, 'service_type': self.service_type, 'binary_path':
self.binary_path, 'start_type': self.start_type, 'account':
self.account}

92
93 class ServiceControlManager:
94     """Низкоуровневый клиент к Service Control Manager (advapi
32.dll)."""
95     STATE_MAP = {SERVICE_STOPPED: 'STOPPED', SERVICE_START_PEN
DING: 'START_PENDING', SERVICE_STOP_PENDING: 'STOP_PENDING', S
ERVICE_RUNNING: 'RUNNING', SERVICE_CONTINUE_PENDING: 'CONTINUE
_PENDING', SERVICE_PAUSE_PENDING: 'PAUSE_PENDING', SERVICE_PAU
SED: 'PAUSED'}
96     START_TYPE_MAP = {0: 'BOOT', 1: 'SYSTEM', 2: 'AUTO_START',
3: 'DEMAND_START', 4: 'DISABLED'}
97
98     def __init__(self) -> None:
99         """Инициализация функций advapi32.dll."""
100         self._advapi32 = ctypes.windll.advapi32
101         self._OpenSCManagerW = self._advapi32.OpenSCManagerW
102         self._OpenSCManagerW.argtypes = [wintypes.LPCWSTR, win
types.LPCWSTR, wintypes.DWORD]
103         self._OpenSCManagerW.restype = wintypes.SC_HANDLE
104         self._EnumServicesStatusExW = self._advapi32.EnumServi
cesStatusExW
105         self._EnumServicesStatusExW.argtypes = [wintypes.SC_HA
NDLE, wintypes.DWORD, wintypes.DWORD, wintypes.DWORD, wintypes
.LPBYTE, wintypes.DWORD, ctypes.POINTER(wintypes.DWORD), ctype
s.POINTER(wintypes.DWORD), ctypes.POINTER(wintypes.DWORD), win
types.LPCWSTR]
106         self._EnumServicesStatusExW.restype = wintypes.BOOL
107         self._OpenServiceW = self._advapi32.OpenServiceW
108         self._OpenServiceW.argtypes = [wintypes.SC_HANDLE, win
types.LPCWSTR, wintypes.DWORD]
109         self._OpenServiceW.restype = wintypes.SC_HANDLE
110         self._QueryServiceConfigW = self._advapi32.QueryServic
eConfigW
111         self._QueryServiceConfigW.argtypes = [wintypes.SC_HAN
DLE, wintypes.LPBYTE, wintypes.DWORD, ctypes.POINTER(wintypes.D

```

```

WORD)]
112         self._QueryServiceConfigW.restype = wintypes.BOOL
113         self._CloseServiceHandle = self._advapi32.CloseService
Handle
114         self._CloseServiceHandle.argtypes = [wintypes.SC_HANDL
E]
115         self._CloseServiceHandle.restype = wintypes.BOOL
116
117         def enum_services(self, service_type: int=SERVICE_WIN32_OW
N_PROCESS | SERVICE_WIN32_SHARE_PROCESS) -> List[ServiceInfo]:
118             """Быстрое пакетное перечисление всех служб через Enum
ServicesStatusExW.
119
120             Args:
121                 service_type: Маска типов служб (Win32, Driver, et
с.).
122
123             Returns:
124                 List[ServiceInfo]: Список обнаруженных служб с PID
и статусом.
125             """
126             h_scm = self._OpenSCManagerW(None, None, SC_MANAGER_EN
UMERATE_SERVICE | SC_MANAGER_CONNECT)
127             if not h_scm:
128                 logger.debug('Не удалось открыть SCManager через W
inAPI')
129                 return []
130             services: List[ServiceInfo] = []
131             try:
132                 bytes_needed = wintypes.DWORD(0)
133                 services_returned = wintypes.DWORD(0)
134                 resume_handle = wintypes.DWORD(0)
135                 self._EnumServicesStatusExW(h_scm, SC_ENUM_PROCESS
_INFO, service_type, SERVICE_STATE_ALL, None, 0, ctypes.byref(
bytes_needed), ctypes.byref(services_returned), ctypes.byref(r
esume_handle), None)
136                 if bytes_needed.value == 0:
137                     return []

```

```

138         buf = (ctypes.c_byte * bytes_needed.value)()
139         success = self._EnumServicesStatusExW(h_scm, SC_EN
UM_PROCESS_INFO, service_type, SERVICE_STATE_ALL, ctypes.cast(
buf, wintypes.LPBYTE), bytes_needed.value, ctypes.byref(bytes_
needed), ctypes.byref(services_returned), ctypes.byref(resume_
handle), None)
140         if not success and ctypes.GetLastError() != ERROR_
MORE_DATA:
141             return []
142             entry_ptr = ctypes.cast(buf, ctypes.POINTER(ENUM_S
ERVICE_STATUS_PROCESSW))
143             for i in range(services_returned.value):
144                 entry = entry_ptr[i]
145                 state_code = entry.ServiceStatusProcess.dwCurr
entState
146                 state_str = self.STATE_MAP.get(state_code, 'UN
KNOWN')
147                 pid = entry.ServiceStatusProcess.dwProcessId
148                 svc = ServiceInfo(name=entry.lpServiceName or
'', display_name=entry.lpDisplayName or entry.lpServiceName or
'', state=state_str, state_code=state_code, pid=pid, service_
type=entry.ServiceStatusProcess.dwServiceType)
149                 services.append(svc)
150             except Exception as ex:
151                 logger.debug(f'Исключение при нативном перечислени
и служб SCM: {ex}')
152             finally:
153                 self._CloseServiceHandle(h_scm)
154             return services
155
156     def get_service_config(self, service_name: str) -> Optiona
l[Dict[str, Any]]:
157         """Получение подробной конфигурации службы через Query
ServiceConfigW.
158
159         Args:
160             service_name: Системное имя службы.
161

```

```

162         Returns:
163             Optional[Dict[str, Any]]: Данные конфигурации или
None при ошибке.
164         """
165         h_scm = self._OpenSCManagerW(None, None, SC_MANAGER_CO
NNECT)
166         if not h_scm:
167             return None
168         try:
169             h_svc = self._OpenServiceW(h_scm, service_name, SE
RVICE_QUERY_CONFIG)
170             if not h_svc:
171                 return None
172             try:
173                 bytes_needed = wintypes.DWORD(0)
174                 self._QueryServiceConfigW(h_svc, None, 0, ctypes
es.byref(bytes_needed))
175                 if bytes_needed.value == 0:
176                     return None
177                 buf = (ctypes.c_byte * bytes_needed.value)()
178                 if not self._QueryServiceConfigW(h_svc, ctypes
.cast(buf, wintypes.LPBYTE), bytes_needed.value, ctypes.byref(
bytes_needed)):
179                     return None
180                 cfg = ctypes.cast(buf, ctypes.POINTER(QUERY_SE
RVICE_CONFIGW)).contents
181                 return {'binary_path': cfg.lpBinaryPathName or
'', 'start_type': self.START_TYPE_MAP.get(cfg.dwStartType, st
r(cfg.dwStartType)), 'start_type_code': cfg.dwStartType, 'acco
unt': cfg.lpServiceStartName or '', 'display_name': cfg.lpDisp
layName or service_name, 'dependencies': cfg.lpDependencies or
''}
182             finally:
183                 self._CloseServiceHandle(h_svc)
184         except Exception as ex:
185             logger.debug(f"Ошибка чтения конфигурации службы '{
{service_name}': {ex}")
186             return None

```

```
187 |         finally:  
188 |             self._CloseServiceHandle(h_scm)
```

setupapi.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  - Setupapi
4 # =====
5 # Description:
6 #   Низкоуровневый интерфейс к SetupAPI и Configuration Manager PnP.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_ff.setupapi import GUID
11 #
12 #       service = GUID()
13 #
14 # File: setupapi.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_ff
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к SetupAPI и Configuration Manager
  PnP."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 from dataclasses import dataclass
28 from typing import Any, Dict, List, Optional
29 from logger import logger
```



```
30 DIGCF_DEFAULT = 1
31 DIGCF_PRESENT = 2
32 DIGCF_ALLCLASSES = 4
33 DIGCF_PROFILE = 8
34 DIGCF_DEVICEINTERFACE = 16
35 SPDRP_DEVICEDESC = 0
36 SPDRP_HARDWAREID = 1
37 SPDRP_CLASS = 7
38 SPDRP_CLASSGUID = 8
39 SPDRP_DRIVER = 9
40 SPDRP_FRIENDLYNAME = 12
41 SPDRP_LOCATION_INFORMATION = 13
42 SPDRP_MFG = 11
43 DN_ROOT_ENUMERATED = 1
44 DN_DRIVER_LOADED = 2
45 DN_ENUM_LOADED = 4
46 DN_STARTED = 8
47 DN_MANUAL = 16
48 DN_NEED_TO_ENUM = 32
49 DN_NOT_FIRST_TIME = 64
50 DN_HARDWARE_ENUM = 128
51 DN_LIBCONFIG = 256
52 DN_FILTERED = 512
53 DN_MOVED = 1024
54 DN_DISABLEABLE = 2048
55 DN_REMOVABLE = 4096
56 DN_HAS_PROBLEM = 1024
57 DN_PRIVATE_PROBLEM = 32768
58 CR_SUCCESS = 0
59 INVALID_HANDLE_VALUE = ctypes.c_void_p(-1).value
60
61 class GUID(ctypes.Structure):
62     """Структура Windows GUID."""
63     _fields_ = [('Data1', wintypes.DWORD), ('Data2', wintypes.
64         WORD), ('Data3', wintypes.WORD), ('Data4', wintypes.BYTE * 8)]
65
66 class SP_DEVINFO_DATA(ctypes.Structure):
67     """Структура информации об устройстве SetupAPI."""
```

```

67     _fields_ = [('cbSize', wintypes.DWORD), ('ClassGuid', GUID
    ), ('DevInst', wintypes.DWORD), ('Reserved', ctypes.c_void_p)]
68
69 @dataclass
70 class PnPDeviceInfo:
71     """Нормализованные данные об аппаратном устройстве PnP."""
72     device_instance_id: str
73     friendly_name: str
74     hardware_id: str
75     device_class: str
76     manufacturer: str
77     has_problem: bool
78     status_code: int
79     problem_code: int
80
81     def to_dict(self) -> Dict[str, Any]:
82         """Преобразование в словарь."""
83         return {'device_instance_id': self.device_instance_id,
84             'friendly_name': self.friendly_name, 'hardware_id': self.hard
85             ware_id, 'device_class': self.device_class, 'manufacturer': se
86             lf.manufacturer, 'has_problem': self.has_problem, 'status_code
87             ': self.status_code, 'problem_code': self.problem_code}
88
89 class SetupAPI:
90     """Низкоуровневый клиент к SetupAPI и CfgMgr32."""
91
92     def __init__(self) -> None:
93         """Инициализация библиотек setupapi.dll и cfgmgr32.dll
94         ."""
95         self._setupapi = ctypes.windll.setupapi
96         self._cfgmgr32 = ctypes.windll.cfgmgr32
97         self._SetupDiGetClassDevsW = self._setupapi.SetupDiGet
98         ClassDevsW
99         self._SetupDiGetClassDevsW.argtypes = [ctypes.c_void_p
100         , wintypes.LPCWSTR, wintypes.HWND, wintypes.DWORD]
101         self._SetupDiGetClassDevsW.restype = ctypes.c_void_p
102         self._SetupDiEnumDeviceInfo = self._setupapi.SetupDiEn
103         umDeviceInfo

```

```

96         self._SetupDiEnumDeviceInfo.argtypes = [ctypes.c_void_p, wintypes.DWORD, ctypes.POINTER(SP_DEVINFO_DATA)]
97         self._SetupDiEnumDeviceInfo.restype = wintypes.BOOL
98         self._SetupDiGetDeviceRegistryPropertyW = self._setupapi.SetupDiGetDeviceRegistryPropertyW
99         self._SetupDiGetDeviceRegistryPropertyW.argtypes = [ctypes.c_void_p, ctypes.POINTER(SP_DEVINFO_DATA), wintypes.DWORD, ctypes.POINTER(wintypes.DWORD), wintypes.LPBYTE, wintypes.DWORD, ctypes.POINTER(wintypes.DWORD)]
100        self._SetupDiGetDeviceRegistryPropertyW.restype = wintypes.BOOL
101        self._SetupDiDestroyDeviceInfoList = self._setupapi.SetupDiDestroyDeviceInfoList
102        self._SetupDiDestroyDeviceInfoList.argtypes = [ctypes.c_void_p]
103        self._SetupDiDestroyDeviceInfoList.restype = wintypes.BOOL
104        self._CM_Get_DevNode_Status = self._cfgmgr32.CM_Get_DevNode_Status
105        self._CM_Get_DevNode_Status.argtypes = [ctypes.POINTER(wintypes.ULONG), ctypes.POINTER(wintypes.ULONG), wintypes.DWORD, wintypes.ULONG]
106        self._CM_Get_DevNode_Status.restype = wintypes.DWORD
107        self._CM_Get_Device_IDW = self._cfgmgr32.CM_Get_Device_IDW
108        self._CM_Get_Device_IDW.argtypes = [wintypes.DWORD, wintypes.LPWSTR, wintypes.ULONG, wintypes.ULONG]
109        self._CM_Get_Device_IDW.restype = wintypes.DWORD
110
111        def _get_property_string(self, h_dev_info: ctypes.c_void_p, dev_data: SP_DEVINFO_DATA, prop: int) -> str:
112            """Получение строкового свойства устройства из реестра SetupAPI."""
113            reg_type = wintypes.DWORD(0)
114            req_size = wintypes.DWORD(0)
115            self._SetupDiGetDeviceRegistryPropertyW(h_dev_info, ctypes.byref(dev_data), prop, ctypes.byref(reg_type), None, 0, ctypes.byref(req_size))

```

```

116         if req_size.value == 0:
117             return ''
118         buf = (ctypes.c_byte * req_size.value)()
119         if not self._SetupDiGetDeviceRegistryPropertyW(h_dev_i
nfo, ctypes.byref(dev_data), prop, ctypes.byref(reg_type), cty
pes.cast(buf, wintypes.LPBYTE), req_size.value, ctypes.byref(r
eq_size)):
120             return ''
121         try:
122             return ctypes.wstring_at(buf)
123         except Exception:
124             return ''
125
126     def get_all_devices(self, only_present: bool=True) -> List
[PnPDeviceInfo]:
127         """Получение списка всех устройств PnP с их статусом и
кодами проблем.
128
129         Args:
130             only_present: Фильтровать только подключенные устр
ойства.
131
132         Returns:
133             List[PnPDeviceInfo]: Список PnP устройств.
134         """
135         flags = DIGCF_ALLCLASSES
136         if only_present:
137             flags |= DIGCF_PRESENT
138         h_dev_info = self._SetupDiGetClassDevsW(None, None, No
ne, flags)
139         if not h_dev_info or h_dev_info == INVALID_HANDLE_VALU
E:
140             logger.debug('Не удалось получить дескриптор Setup
DiGetClassDevsW')
141             return []
142         devices: List[PnPDeviceInfo] = []
143         try:
144             dev_data = SP_DEVINFO_DATA()

```

```

145         dev_data.cbSize = ctypes.sizeof(SP_DEVINFO_DATA)
146         idx = 0
147         while self._SetupDiEnumDeviceInfo(h_dev_info, idx,
        ctypes.byref(dev_data)):
148             idx += 1
149             try:
150                 friendly_name = self._get_property_string(
        h_dev_info, dev_data, SPDRP_FRIENDLYNAME)
151                 if not friendly_name:
152                     friendly_name = self._get_property_str
        ing(h_dev_info, dev_data, SPDRP_DEVICEDESC)
153                 hw_id = self._get_property_string(h_dev_in
        fo, dev_data, SPDRP_HARDWAREID)
154                 cls_name = self._get_property_string(h_dev
        _info, dev_data, SPDRP_CLASS)
155                 mfg = self._get_property_string(h_dev_info
        , dev_data, SPDRP_MFG)
156                 id_buf = (ctypes.c_wchar * 512)()
157                 cm_res = self._CM_Get_Device_IDW(dev_data.
        DevInst, id_buf, 512, 0)
158                 device_instance_id = id_buf.value if cm_re
        s == CR_SUCCESS else ''
159                 status = wintypes.ULONG(0)
160                 prob_code = wintypes.ULONG(0)
161                 cm_status = self._CM_Get_DevNode_Status(ct
        ypes.byref(status), ctypes.byref(prob_code), dev_data.DevInst,
        0)
162                 has_problem = False
163                 problem_num = 0
164                 if cm_status == CR_SUCCESS:
165                     problem_num = prob_code.value
166                     has_problem = status.value & DN_HAS_PR
        OBLEM != 0 or problem_num != 0
167                 devices.append(PnPDeviceInfo(device instan
        ce_id=device_instance_id, friendly_name=friendly_name or devic
        e_instance_id or 'Устройство без имени', hardware_id=hw_id, de
        vice_class=cls_name, manufacturer=mfg, has_problem=has_problem
        , status_code=status.value if cm_status == CR_SUCCESS else 0,

```

```
        problem_code=problem_num))
168 |             except Exception:
169 |                 continue
170 |         finally:
171 |             self._SetupDiDestroyDeviceInfoList(h_dev_info)
172 |         return devices
173 |
174 |     def get_problem_devices(self) -> List[PnPDeviceInfo]:
175 |         """Быстрый поиск только устройств с аппаратными сбоями
176 |         (Code 10/43/28).
177 |
178 |         Returns:
179 |             List[PnPDeviceInfo]: Список проблемных устройств.
180 |
181 |         all_devs = self.get_all_devices(only_present=True)
182 |         return [d for d in all_devs if d.has_problem and d.problem_code > 0]
```

tasksched.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Tasksched
5 # =====
6 # Description:
7 #   Интерфейс к Task Scheduler 2.0 COM API Windows.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.tasksched import T
11 #       askSchedulerAPI
12 #
13 #       service = TaskSchedulerAPI()
14 #
15 # File: tasksched.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.win32_fffi
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-01 13:28:28
21 # =====
22
23 from __future__ import annotations
24
25 """Интерфейс к Task Scheduler 2.0 COM API Windows."""
26
27 import sys
28 from typing import Any, Dict, List, Optional
29 from logger import logger
30 TASK_STATE_UNKNOWN = 0
31 TASK_STATE_DISABLED = 1
32 TASK_STATE_QUEUED = 2
33 TASK_STATE_READY = 3
```

```

32 TASK_STATE_RUNNING = 4
33 TASK_STATE_MAP = {TASK_STATE_UNKNOWN: 'Unknown', TASK_STATE_DISABLED: 'Disabled', TASK_STATE_QUEUED: 'Queued', TASK_STATE_READY: 'Ready', TASK_STATE_RUNNING: 'Running'}
34 TASK_ACTION_EXEC = 0
35 TASK_ACTION_COM_HANDLER = 5
36 TASK_ACTION_SEND_EMAIL = 6
37 TASK_ACTION_SHOW_MESSAGE = 7
38
39 class TaskSchedulerAPI:
40     """Обертка над COM-интерфейсом Task Scheduler 2.0 (Schedule.Service)."""
41
42     def __init__(self) -> None:
43         """Инициализация COM-объекта планировщика."""
44         self._service: Optional[Any] = None
45         self._is_available = self._connect()
46
47     def _connect(self) -> bool:
48         """Подключение к локальной службе планировщика через COM."""
49         try:
50             import win32com.client
51             self._service = win32com.client.Dispatch('Schedule.Service')
52             self._service.Connect()
53             return True
54         except Exception as ex:
55             logger.debug(f'COM-интерфейс Schedule.Service недоступен: {ex}')
56             self._service = None
57             return False
58
59     @property
60     def is_available(self) -> bool:
61         """Доступность нативного COM-интерфейса планировщика."""
62         return self._is_available and self._service is not None

```


е

```
63
64     def get_all_tasks(self, max_tasks: int=500) -> List[Dict[s
tr, Any]]:
65         """Рекурсивный сбор всех задач планировщика из всех па
пок.
66
67         Args:
68             max_tasks: Максимальное количество задач для выбор
ки.
69
70         Returns:
71             List[Dict[str, Any]]: Список задач с путями, дейст
виями и статусами.
72         """
73         if not self.is_available or self._service is None:
74             return []
75         tasks: List[Dict[str, Any]] = []
76         try:
77             root_folder = self._service.GetFolder('\\')
78             self._scan_folder(root_folder, tasks, max_tasks)
79         except Exception as ex:
80             logger.debug(f'Ошибка при рекурсивном чтении задач
планировщика COM: {ex}')
81         return tasks
82
83     def _scan_folder(self, folder: Any, tasks: List[Dict[str,
Any]], max_tasks: int) -> None:
84         """Рекурсивный обход папки планировщика.
85
86         Args:
87             folder: COM-объект ITaskFolder.
88             tasks: Результирующий список задач.
89             max_tasks: Лимит задач.
90         """
91         if len(tasks) >= max_tasks:
92             return
93         try:
```

```

94         folder_tasks = folder.GetTasks(0)
95         for task in folder_tasks:
96             if len(tasks) >= max_tasks:
97                 break
98             try:
99                 task_info = self._parse_task(task)
100                 if task_info:
101                     tasks.append(task_info)
102             except Exception:
103                 continue
104         subfolders = folder.GetFolders(0)
105         for sub in subfolders:
106             if len(tasks) >= max_tasks:
107                 break
108             self._scan_folder(sub, tasks, max_tasks)
109     except Exception as ex:
110         logger.debug(f"Ошибка при сканировании папки '{get
111 attr(folder, 'Path', '')}': {ex}")
112
113     def _parse_task(self, task: Any) -> Optional[Dict[str, Any
114 ]]:
115         """Парсинг свойств зарегистрированной задачи (IRegister
116         redTask)."""
117         name = getattr(task, 'Name', '')
118         path = getattr(task, 'Path', '')
119         state_code = getattr(task, 'State', 0)
120         state_name = TASK_STATE_MAP.get(state_code, 'Unknown')
121         enabled = getattr(task, 'Enabled', True)
122         actions_list: List[Dict[str, Any]] = []
123         action_strs: List[str] = []
124         try:
125             definition = task.Definition
126             actions = definition.Actions
127             for act in actions:
128                 act_type = getattr(act, 'Type', -1)
129                 if act_type == TASK_ACTION_EXEC:
130                     exec_path = getattr(act, 'Path', '') or ''
131                     args = getattr(act, 'Arguments', '') or ''

```

```

129 |             working_dir = getattr(act, 'WorkingDirecto
    | ry', '') or ''
130 |             full_cmd = f'{exec_path} {args}'.strip()
131 |             actions_list.append({'type': 'Exec', 'path
    | ': exec_path, 'arguments': args, 'working_directory': working_
    | dir, 'command': full_cmd})
132 |             action_strs.append(full_cmd)
133 |             elif act_type == TASK_ACTION_COM_HANDLER:
134 |                 clsid = getattr(act, 'ClassId', '')
135 |                 actions_list.append({'type': 'ComHandler',
    | 'class_id': clsid})
136 |                 action_strs.append(f'ComHandler:{clsid}')
137 |         except Exception:
138 |             pass
139 |         return {'TaskName': name, 'TaskPath': path, 'State': s
    | tate_name, 'Enabled': enabled, 'Actions': ' ; '.join(action_st
    | rs) if action_strs else '', 'ActionDetails': actions_list}

```

wevtapi.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Wevtapi
5 # Description:
6 # Низкоуровневый интерфейс к wevtapi.dll для работы с журнал
7 # ами событий Windows.
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.telemetry.win32_fffi.wevtapi import Cha
11 # nnelMetadata
12 # service = ChannelMetadata()
13 #
14 # File: wevtapi.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к wevtapi.dll для работы с журнала
24 ми событий Windows."""
25
26 import ctypes
27 import ctypes.wintypes as wintypes
28 import struct
29 import sys
30 import winreg
```

```
30 import xml.etree.ElementTree as ET
31 from dataclasses import dataclass
32 from typing import Any, Dict, List, Optional
33 from logger import logger
34 EVT_QUERY_CHANNEL_PATH = 1
35 EVT_QUERY_FILE_PATH = 2
36 EVT_QUERY_REVERSE_DIRECTION = 512
37 EVT_RENDER_EVENT_XML = 1
38 CHANNEL_DESCRIPTIONS: Dict[str, str] = {'System': 'Системный журнал Windows: события ядра ОС, системных служб, драйверов устройств, сбоев питания и сетевого стека.', 'Application': 'Журнал приложений: события, предупреждения, сбои и ошибки пользовательских и серверных программ (Crash, Faults).', 'Security': 'Журнал безопасности: аудит входов в систему (Logon/Logoff), права доступа, повышение привилегий UAC и политики безопасности.', 'Setup': 'Журнал установки: логи процесса установки операционной системы, обновлений Windows и накопительных пакетов KB.', 'Microsoft-Windows-WindowsUpdateClient/Operational': 'Клиент обновления Windows Update: загрузка, верификация и установка патчей, исправления безопасности и драйверов.', 'Microsoft-Windows-Kernel-PnP/Configuration': 'Конфигурация оборудования Plug and Play: подключение новых устройств, установка драйверов, коды ошибок 10/43.', 'Microsoft-Windows-Kernel-Power/Operational': 'Питание ядра Windows: переходы в сон/гибернацию, сбои питания (Event 41), изменение состояний батареи и ACPI.', 'Microsoft-Windows-TaskScheduler/Operational': 'Планировщик заданий: запуск, завершение, сбои и выполнение фоновых регламентных задач Windows.', 'Microsoft-Windows-Windows Defender/Operational': 'Антивирус Microsoft Defender: обнаружение угроз, сканирование в реальном времени, обновление антивирусных сигнатур.', 'Microsoft-Windows-Hyper-V-Compute-Operational': 'Платформа виртуализации Hyper-V: жизненный цикл виртуальных машин, контейнеров WSL2 и виртуальных адаптеров.', 'Microsoft-Windows-Firewall With Advanced Security/Firewall': 'Брандмауэр Windows: блокировки входящего/исходящего трафика, сетевые правила фильтрации и аномалии сокетов.', 'Microsoft-Windows-Diagnostics-Performance/Operational': 'Диагностика производительности: анализ времени загрузки Windows, выключения и зависания системных
```

```
компонентов.', 'Microsoft-Windows-Bits-Client/Operational': 'Фоновая интеллектуальная служба передачи (BITS): загрузка файлов и обновлений по сети с контролем канала.', 'Microsoft-Windows-DNS-Client/Operational': 'DNS-клиент Windows: резолвинг доменных имен, таймауты DNS серверов и сетевые кэширования.', 'Microsoft-Windows-CodeIntegrity/Operational': 'Контроль целостности кода: проверка цифровых подписей драйверов и исполняемых системных файлов.', 'Microsoft-Windows-PowerShell/Operational': 'Исполнение скриптов PowerShell: запуск командлетов, сценариев автоматизации и модулей администрирования.', 'Microsoft-Windows-Sysmon/Operational': 'Microsoft System Monitor (Sysmon): глубокая телеметрия создания процессов (Event 1), сети (Event 3), файловых операций (11/23/26) и хэшей исполняемых файлов.'}
```

39

40 @dataclass

41 class ChannelMetadata:

42 """Метаданные о канале журнала Windows."""

43 channel_name: str

44 display_name: str

45 description: str = ''

46 is_enabled: bool = True

47 record_count: int = 0

48 category: str = 'Windows Event Log'

49

50 class WevtAPI:

51 """Ctypes-обертка для wevtapi.dll и реестра Windows Event Log."""

52

53 def __init__(self) -> None:

54 """Инициализация библиотеки wevtapi.dll."""

55 self._available = False

56 self.wevtapi: Optional[ctypes.WinDLL] = None

57 self._publisher_cache: Dict[str, Optional[wintypes.HANDLE]] = {}

58 if sys.platform == 'win32':

59 try:

60 self.wevtapi = ctypes.windll.LoadLibrary('wevtapi.dll')

```

61         self._setup_signatures()
62         self._available = True
63     except Exception as ex:
64         logger.debug(f'[WevtAPI] Не удалось загрузить
wevtapi.dll: {ex}')
65
66     def _setup_signatures(self) -> None:
67         """Настройка типов аргументов и возвращаемых значений
функций."""
68         if not self.wevtapi:
69             return
70         self.wevtapi.EvtOpenChannelEnum.argtypes = [wintypes.H
HANDLE, wintypes.DWORD]
71         self.wevtapi.EvtOpenChannelEnum.restype = wintypes.HAN
DLE
72         self.wevtapi.EvtNextChannelPath.argtypes = [wintypes.H
HANDLE, wintypes.DWORD, wintypes.LPWSTR, ctypes.POINTER(wintype
s.DWORD)]
73         self.wevtapi.EvtNextChannelPath.restype = wintypes.BOOL
L
74         self.wevtapi.EvtClose.argtypes = [wintypes.HANDLE]
75         self.wevtapi.EvtClose.restype = wintypes.BOOL
76         self.wevtapi.EvtOpenLog.argtypes = [wintypes.HANDLE, w
intypes.LPCWSTR, wintypes.DWORD]
77         self.wevtapi.EvtOpenLog.restype = wintypes.HANDLE
78         self.wevtapi.EvtGetLogInfo.argtypes = [wintypes.HANDLE
, wintypes.DWORD, wintypes.DWORD, ctypes.c_void_p, ctypes.POIN
TER(wintypes.DWORD)]
79         self.wevtapi.EvtGetLogInfo.restype = wintypes.BOOL
80         self.wevtapi.EvtQuery.argtypes = [wintypes.HANDLE, wint
ypes.LPCWSTR, wintypes.LPCWSTR, wintypes.DWORD]
81         self.wevtapi.EvtQuery.restype = wintypes.HANDLE
82         self.wevtapi.EvtNext.argtypes = [wintypes.HANDLE, wint
ypes.DWORD, ctypes.POINTER(wintypes.HANDLE), wintypes.DWORD, w
intypes.DWORD, ctypes.POINTER(wintypes.DWORD)]
83         self.wevtapi.EvtNext.restype = wintypes.BOOL
84         self.wevtapi.EvtRender.argtypes = [wintypes.HANDLE, wi
ntypes.HANDLE, wintypes.DWORD, wintypes.DWORD, wintypes.LPVOID

```

```

    , ctypes.POINTER(wintypes.DWORD), ctypes.POINTER(wintypes.DWORD)]
85         self.wevtapi.EvtRender.restype = wintypes.BOOL
86         self.wevtapi.EvtOpenPublisherMetadata.argtypes = [wintypes.HANDLE, wintypes.LPCWSTR, wintypes.LPCWSTR, wintypes.DWORD, wintypes.DWORD]
87         self.wevtapi.EvtOpenPublisherMetadata.restype = wintypes.HANDLE
88         self.wevtapi.EvtFormatMessage.argtypes = [wintypes.HANDLE, wintypes.HANDLE, wintypes.DWORD, wintypes.DWORD, ctypes.c_void_p, wintypes.DWORD, wintypes.DWORD, wintypes.LPWSTR, ctypes.POINTER(wintypes.DWORD)]
89         self.wevtapi.EvtFormatMessage.restype = wintypes.BOOL
90
91     def is_available(self) -> bool:
92         """Проверить доступность wevtapi.dll в текущей системе
93         ."""
94
95         return self._available
96
97     def get_channel_record_count(self, channel_name: str) -> int:
98         """Мгновенно получить точное количество записей в канале через EvtGetLogInfo."""
99         if not self._available or not self.wevtapi:
100             return 0
101         h_log = self.wevtapi.EvtOpenLog(None, channel_name, 1)
102         if not h_log:
103             return 0
104         try:
105             buf = (ctypes.c_byte * 128)()
106             used = wintypes.DWORD(0)
107             if self.wevtapi.EvtGetLogInfo(h_log, 5, 128, ctypes.byref(buf), ctypes.byref(used)):
108                 return struct.unpack('<Q', bytes(buf)[:8])[0]
109             except Exception:
110                 pass
111         finally:
112             self.wevtapi.EvtClose(h_log)

```



```
111 |         return 0
112 |
113 |     def get_channel_description(self, channel_name: str) -> str:
114 |         """Получить человекочитаемое описание назначения канала."""
115 |         if channel_name in CHANNEL_DESCRIPTIONS:
116 |             return CHANNEL_DESCRIPTIONS[channel_name]
117 |         if 'Defender' in channel_name:
118 |             return 'Журнал защитных механизмов, антивирусного сканирования и предотвращения вторжений Microsoft Defender.'
119 |         if 'Kernel-Power' in channel_name:
120 |             return 'Журнал управления электропитанием, состояний ACPI, спящего режима и аварийных отключений питания.'
121 |         if 'Kernel-PnP' in channel_name:
122 |             return 'Журнал конфигурации оборудования, обнаружения и инициализации устройств Plug and Play.'
123 |         if 'WindowsUpdate' in channel_name:
124 |             return 'Журнал службы обновлений Windows, загрузки пакетов KB и применения хотфиксов.'
125 |         if 'TaskScheduler' in channel_name:
126 |             return 'Журнал планировщика задач Windows, запуска автоматических фоновых процессов и триггеров.'
127 |         if 'Hyper-V' in channel_name:
128 |             return 'Журнал подсистемы виртуализации Hyper-V, WinSL2 контейнеров и гостевых служб.'
129 |         if 'PowerShell' in channel_name:
130 |             return 'Журнал среды PowerShell, выполнения скриптов автоматизации и команд администрирования.'
131 |         if 'Firewall' in channel_name:
132 |             return 'Журнал сетевого брандмауэра Windows, правил сетевого экрана и событий фильтрации трафика.'
133 |         if 'Storage' in channel_name or 'Disk' in channel_name:
134 |             return 'Журнал дисковой подсистемы, контроллеров накопителей, томов NTFS/ReFS и файловой системы.'
135 |         if 'Network' in channel_name or 'Wlan' in channel_name or 'NDIS' in channel_name:
```

```

136         return 'Журнал сетевых интерфейсов, Wi-Fi соединен
ий, сетевых адаптеров и стека TCP/IP.'
137         if 'Audio' in channel_name or 'Sound' in channel_name:
138             return 'Журнал аудиоподсистемы, службы Windows Aud
ио и конечных звуковых устройств.'
139         if 'Print' in channel_name or 'Spooler' in channel_nam
e:
140             return 'Журнал диспетчера очереди печати и драйвер
ов принтеров.'
141         if 'Bluetooth' in channel_name or 'BTN' in channel_nam
e:
142             return 'Журнал стека Bluetooth, сопряжения беспров
одных устройств и протоколов передачи данных.'
143         if 'USB' in channel_name:
144             return 'Журнал контроллеров USB, хабов и подключен
ных внешних периферийных устройств.'
145         return f"Специализированный системный журнал Windows д
ля компонента '{channel_name}'."
146
147     def enumerate_channels(self) -> List[ChannelMetadata]:
148         """Получить исчерпывающий список всех каналов Windows
Event Log с количеством записей."""
149         channels: List[ChannelMetadata] = []
150         seen: set[str] = set()
151         classic = [('System', 'System (Системный)'), ('Applica
tion', 'Application (Приложения)'), ('Security', 'Security (Бе
зопасность)'), ('Setup', 'Setup (Установка)'), ('Microsoft-Win
dows-WindowsUpdateClient/Operational', 'Windows Update Client'
), ('Microsoft-Windows-Kernel-PnP/Configuration', 'Kernel PnP
Configuration'), ('Microsoft-Windows-Kernel-Power/Operational'
, 'Kernel Power'), ('Microsoft-Windows-TaskScheduler/Operation
al', 'Task Scheduler'), ('Microsoft-Windows-Windows Defender/O
perational', 'Windows Defender'), ('Microsoft-Windows-Hyper-V-
Compute-Operational', 'Hyper-V Compute'), ('Microsoft-Windows-
PowerShell/Operational', 'Windows PowerShell')]
152         for cname, dname in classic:
153             rec_cnt = self.get_channel_record_count(cname)
154             channels.append(ChannelMetadata(channel_name=cname

```

```

    , display_name=dname, description=self.get_channel_description
    (cname), record_count=rec_cnt, is_enabled=True))
155         seen.add(cname.lower())
156         if self._available and self.wevtapi:
157             try:
158                 h_enum = self.wevtapi.EvtOpenChannelEnum(None,
0)
159                 if h_enum:
160                     buffer_size = 512
161                     buf = ctypes.create_unicode_buffer(buffer_
size)
162                     used = wintypes.DWORD()
163                     while True:
164                         success = self.wevtapi.EvtNextChannelP
ath(h_enum, buffer_size, buf, ctypes.byref(used))
165                         if not success:
166                             err = ctypes.GetLastError()
167                             if err == 122:
168                                 buffer_size = used.value + 10
169                                 buf = ctypes.create_unicode_bu
ffer(buffer_size)
170                                 continue
171                                 break
172                                 chan_str = buf.value.strip()
173                                 if chan_str and chan_str.lower() not i
n seen:
174                                     rec_cnt = self.get_channel_record_
count(chan_str)
175                                     channels.append(ChannelMetadata(ch
annel_name=chan_str, display_name=chan_str, description=self.g
et_channel_description(chan_str), record_count=rec_cnt, is_ena
bled=True))
176                                     seen.add(chan_str.lower())
177                                     self.wevtapi.EvtClose(h_enum)
178             except Exception as ex:
179                 logger.debug(f'[WevtAPI] Ошибка при перечислен
ии через EvtOpenChannelEnum: {ex}')
180         try:

```

```

181         reg_channels = self._enumerate_registry_channels()
182         for r_chan in reg_channels:
183             if r_chan.lower() not in seen:
184                 rec_cnt = self.get_channel_record_count(r_
chan)
185                 channels.append(ChannelMetadata(channel_na
me=r_chan, display_name=r_chan, description=self.get_channel_d
escription(r_chan), record_count=rec_cnt, is_enabled=True))
186                 seen.add(r_chan.lower())
187         except Exception as ex:
188             logger.debug(f'[WevtAPI] Ошибка чтения каналов из
реестра: {ex}')
189         return channels
190
191     def _enumerate_registry_channels(self) -> List[str]:
192         """Считать имена зарегистрированных каналов из реестра
Windows."""
193         res: List[str] = []
194         key_path = 'SOFTWARE\\Microsoft\\Windows\\CurrentVersi
on\\WINEVT\\Channels'
195         try:
196             with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE, key
_path, 0, winreg.KEY_READ | winreg.KEY_WOW64_64KEY) as key:
197                 num_subkeys, _, _ = winreg.QueryInfoKey(key)
198                 for i in range(num_subkeys):
199                     try:
200                         subkey_name = winreg.EnumKey(key, i)
201                         if subkey_name:
202                             res.append(subkey_name)
203                     except OSError:
204                         break
205         except Exception:
206             pass
207         return res
208
209     def read_events(self, channel: str='System', limit: int=10
0, level: str='', search: str='', event_id: int=0, hours: int=
24) -> List[Dict[str, Any]]:

```

```

210 |         """Быстрое чтение событий из указанного канала через E
vtQuery / EvtRender."""
211 |         if not self._available or not self.wevtapi:
212 |             return []
213 |         limit = max(1, min(limit, 2000))
214 |         xpath_parts: List[str] = []
215 |         if level:
216 |             lvl_lower = level.lower()
217 |             if 'crit' in lvl_lower:
218 |                 xpath_parts.append('Level=1')
219 |             elif 'err' in lvl_lower:
220 |                 xpath_parts.append('Level=2')
221 |             elif 'warn' in lvl_lower:
222 |                 xpath_parts.append('Level=3')
223 |             elif 'inf' in lvl_lower:
224 |                 xpath_parts.append('Level=4')
225 |             elif 'verb' in lvl_lower:
226 |                 xpath_parts.append('Level=5')
227 |         if event_id > 0:
228 |             xpath_parts.append(f'EventID={event_id}')
229 |         if xpath_parts:
230 |             clause = ' and '.join(xpath_parts)
231 |             query_xpath = f'*[System[{clause}]]'
232 |         else:
233 |             query_xpath = '*'
234 |         events: List[Dict[str, Any]] = []
235 |         try:
236 |             h_query = self.wevtapi.EvtQuery(None, channel, que
ry_xpath, EVT_QUERY_CHANNEL_PATH | EVT_QUERY_REVERSE_DIRECTION
)
237 |             if not h_query:
238 |                 return []
239 |             try:
240 |                 event_handles = (wintypes.HANDLE * limit)()
241 |                 returned = wintypes.DWORD(0)
242 |                 if self.wevtapi.EvtNext(h_query, limit, event_
handles, 2000, 0, ctypes.byref(returned)):
243 |                     for i in range(returned.value):

```

```

244         h_ev = event_handles[i]
245         if h_ev:
246             ev_dict = self._render_event_xml(h
_ev, channel)
247             if ev_dict:
248                 if search:
249                     s_lower = search.lower()
250                     msg = str(ev_dict.get('mes
sage', ''')).lower()
251                     prov = str(ev_dict.get('pr
ovider', ''')).lower()
252                     eid = str(ev_dict.get('eve
nt_id', '''))
253                     if s_lower not in msg and
s_lower not in prov and (s_lower not in eid):
254                         self.wevtapi.EvtClose(
h_ev)
255                         continue
256                         events.append(ev_dict)
257                         self.wevtapi.EvtClose(h_ev)
258             finally:
259                 self.wevtapi.EvtClose(h_query)
260         except Exception as ex:
261             logger.debug(f'[WevtAPI] Ошибка при чтении событий
({channel}): {ex}')
262         return events
263
264     def _get_publisher_metadata(self, provider_name: str) -> O
ptional[wintypes.HANDLE]:
265         """Получить кешированный дескриптор метаданных провайд
ера событий."""
266         if not self._available or not self.wevtapi or not prov
ider_name:
267             return None
268         if provider_name in self._publisher_cache:
269             return self._publisher_cache[provider_name]
270         try:
271             h_pub = self.wevtapi.EvtOpenPublisherMetadata(None

```

```

    , provider_name, None, 0, 0)
272         if h_pub:
273             self._publisher_cache[provider_name] = h_pub
274             return h_pub
275             self._publisher_cache[provider_name] = None
276     except Exception:
277         self._publisher_cache[provider_name] = None
278     return None
279
280     def format_event_message(self, h_event: wintypes.HANDLE, p
rovider_name: str, flag: int = 1) -> Optional[str]:
281         """Сформировать нативное локализованное системное сооб
щение события через EvtFormatMessage.
282
283         Args:
284             h_event: Дескриптор события Windows.
285             provider_name: Имя издателя/провайдера (например,
'Microsoft-Windows-Kernel-Power').
286             flag: Флаг форматирования (1 = EvtFormatMessageEve
nt, 2 = Level, 3 = Task).
287
288         Returns:
289             Optional[str]: Отформатированная локализованная ст
рока сообщения или None.
290         """
291         if not self._available or not self.wevtapi or not h_ev
ent or not provider_name:
292             return None
293         h_pub = self._get_publisher_metadata(provider_name)
294         if not h_pub:
295             return None
296         try:
297             used = wintypes.DWORD()
298             self.wevtapi.EvtFormatMessage(h_pub, h_event, 0, 0
, None, flag, 0, None, ctypes.byref(used))
299             err = ctypes.GetLastError()
300             if err != 122 or used.value == 0:
301                 return None

```

```

302         buf = ctypes.create_unicode_buffer(used.value + 2)
303         if self.wevtapi.EvtFormatMessage(h_pub, h_event, 0
, 0, None, flag, used.value, buf, ctypes.byref(used)):
304             val = buf.value
305             if val:
306                 return val.strip()
307     except Exception:
308         pass
309     return None
310
311     def close(self) -> None:
312         """Освободить все открытые дескрипторы издателей."""
313         if self._available and self.wevtapi:
314             for h in list(self._publisher_cache.values()):
315                 if h:
316                     try:
317                         self.wevtapi.EvtClose(h)
318                     except Exception:
319                         pass
320             self._publisher_cache.clear()
321
322     def __del__(self) -> None:
323         """Деструктор для гарантированного освобождения ресурс
324 ов."""
325         self.close()
326
327     def _render_event_xml(self, h_event: wintypes.HANDLE, chan
nel: str) -> Optional[Dict[str, Any]]:
328         """Отрендерить событие в XML строку и разобрать в слов
329 арь."""
330         if not self.wevtapi:
331             return None
332         used = wintypes.DWORD()
333         prop_count = wintypes.DWORD()
334         self.wevtapi.EvtRender(None, h_event, EVT_RENDER_EVENT
_XML, 0, None, ctypes.byref(used), ctypes.byref(prop_count))
335         err = ctypes.GetLastError()
336         if err != 122:

```



```

335         return None
336         buf = ctypes.create_unicode_buffer(used.value // 2 + 2
    )
337         if not self.wevtapi.EvtRender(None, h_event, EVT_RENDE
R_EVENT_XML, used.value, ctypes.byref(buf), ctypes.byref(used)
    , ctypes.byref(prop_count)):
338             return None
339             xml_str = buf.value
340             if not xml_str:
341                 return None
342             res = self._parse_event_xml(xml_str, channel)
343             if res:
344                 prov = res.get('provider', '')
345                 if prov:
346                     formatted = self.format_event_message(h_event,
prov, flag=1)
347                     if formatted:
348                         res['formatted_message'] = formatted
349                         res['message'] = formatted
350             return res
351
352     def _parse_event_xml(self, xml_str: str, channel: str) ->
Dict[str, Any]:
353         """Разобрать XML представление Windows события в норма
лизованную структуру."""
354         res: Dict[str, Any] = {'timestamp': '', 'level': 'Info
rmation', 'event_id': 0, 'provider': '', 'computer': '', 'proc
ess_id': 0, 'thread_id': 0, 'channel': channel, 'message': '',
    'raw_data': xml_str}
355         try:
356             clean_xml = ET.fromstring(xml_str)
357             system_elem = clean_xml.find('{http://schemas.micr
osoft.com/win/2004/08/events/event}System')
358             if system_elem is None:
359                 system_elem = clean_xml.find('System')
360             if system_elem is not None:
361                 for child in system_elem:
362                     tag = child.tag.split('}')[1]

```

```

363         if tag == 'TimeCreated':
364             raw_time = child.attrib.get('SystemTimeCreated', '')
365             if raw_time:
366                 res['timestamp'] = raw_time.replace('T', ' ').split('.')[0]
367             elif tag == 'EventID':
368                 try:
369                     res['event_id'] = int(child.text or '0')
370             except ValueError:
371                 res['event_id'] = 0
372             elif tag == 'Level':
373                 lvl_map = {'1': 'Critical', '2': 'Error', '3': 'Warning', '4': 'Information', '5': 'Verbose'}
374                 res['level'] = lvl_map.get(str(child.text or '').strip(), 'Information')
375             elif tag == 'Provider':
376                 res['provider'] = child.attrib.get('Name', '')
377             elif tag == 'Computer':
378                 res['computer'] = child.text or ''
379             elif tag == 'Execution':
380                 try:
381                     res['process_id'] = int(child.attrib.get('ProcessID', 0))
382                     res['thread_id'] = int(child.attrib.get('ThreadID', 0))
383                 except ValueError:
384                     pass
385             elif tag == 'Channel':
386                 if child.text:
387                     res['channel'] = child.text
388         data_dict: Dict[str, str] = {}
389         data_texts: List[str] = []
390         for d in clean_xml.iter():
391             tag = d.tag.split('}')[1]
392             if tag in ('Data', 'string', 'Value') and d.text:

```

```

xt:
393         t = d.text.strip()
394         name_attr = d.attrib.get('Name')
395         if name_attr:
396             data_dict[name_attr] = t
397             data_texts.append(f'{name_attr}: {t}')
398         elif t:
399             data_texts.append(t)
400         res['event_data'] = data_dict
401         if data_texts:
402             res['message'] = ' | '.join(data_texts[:12])
403         else:
404             res['message'] = f"Event {res['event_id']} from {res['provider']}"
405         except Exception:
406             res['message'] = f'Event recorded in {channel}'
407         return res
408
409     def query_process_events(self, limit: int=100, filter_process: Optional[str]=None, filter_user: Optional[str]=None) -> List[Dict[str, Any]]:
410         """
411         Запросить структурированные события создания процессов
412         из Sysmon (Event 1)
413         или Windows Security Log (Event 4688).
414
415         Args:
416             limit: Максимальное количество событий.
417             filter_process: Фильтр по имени/пути исполняемого файла.
418             filter_user: Фильтр по имени учетной записи пользователя.
419
420         Returns:
421             Список нормализованных словарей с информацией о запусках процессов.
422
423         """
424         results: List[Dict[str, Any]] = []

```

```

423         sysmon_events = self.read_events(channel='Microsoft-Wi
ndows-Sysmon/Operational', limit=limit, event_id=1)
424         for ev in sysmon_events:
425             ed = ev.get('event_data', {})
426             exe_path = ed.get('Image', '')
427             cmd_line = ed.get('CommandLine', '')
428             user = ed.get('User', '')
429             pid = int(ed.get('ProcessId', 0) or 0)
430             ppid = int(ed.get('ParentProcessId', 0) or 0)
431             parent_exe = ed.get('ParentImage', '')
432             parent_cmd = ed.get('ParentCommandLine', '')
433             hashes = ed.get('Hashes', '')
434             guid = ed.get('ProcessGuid', '')
435             parent_guid = ed.get('ParentProcessGuid', '')
436             if filter_process and filter_process.lower() not in
exe_path.lower() and (filter_process.lower() not in cmd_line
.lower()):
437                 continue
438             if filter_user and filter_user.lower() not in user
.lower():
439                 continue
440             results.append({'source': 'Sysmon (Event 1)', 'tim
estamp': ev.get('timestamp'), 'event_id': 1, 'process_id': pid
, 'process_name': exe_path.replace('\\', '/').split('/')[-1] if
exe_path else '', 'executable_path': exe_path, 'command_line
': cmd_line, 'user': user, 'parent_process_id': ppid, 'parent_
process_name': parent_exe.replace('\\', '/').split('/')[-1] if
parent_exe else '', 'parent_executable_path': parent_exe, 'pa
rent_command_line': parent_cmd, 'process_guid': guid, 'parent_
process_guid': parent_guid, 'hashes': hashes, 'raw_event': ev}
)
441         if len(results) < limit:
442             remaining = limit - len(results)
443             sec_events = self.read_events(channel='Security',
limit=remaining, event_id=4688)
444             for ev in sec_events:
445                 ed = ev.get('event_data', {})
446                 exe_path = ed.get('NewProcessName', '')

```

```

447         cmd_line = ed.get('CommandLine', '')
448         user = ed.get('SubjectUserName', '')
449         raw_pid = ed.get('NewProcessId', '0')
450         try:
451             pid = int(raw_pid, 16) if str(raw_pid).startsw
452             rtswith('0x') else int(raw_pid or 0)
453         except ValueError:
454             pid = 0
455             raw_ppid = ed.get('ProcessId', '0')
456             try:
457                 ppid = int(raw_ppid, 16) if str(raw_ppid).
458                 startswith('0x') else int(raw_ppid or 0)
459             except ValueError:
460                 ppid = 0
461                 parent_exe = ed.get('ParentProcessName', '')
462                 if filter_process and filter_process.lower() n
463                 ot in exe_path.lower() and (filter_process.lower() not in cmd_
464                 line.lower()):
465                     continue
466                 if filter_user and filter_user.lower() not in
467                 user.lower():
468                     continue
469                 results.append({'source': 'Security Audit (Eve
470                 nt 4688)', 'timestamp': ev.get('timestamp'), 'event_id': 4688,
471                 'process_id': pid, 'process_name': exe_path.replace('\\', '/')
472                 ).split('/')[-1] if exe_path else '', 'executable_path': exe_p
473                 ath, 'command_line': cmd_line, 'user': user, 'parent_process_i
474                 d': ppid, 'parent_process_name': parent_exe.replace('\\', '/')
475                 .split('/')[-1] if parent_exe else '', 'parent_executable_path
476                 ': parent_exe, 'parent_command_line': '', 'process_guid': '',
477                 'parent_process_guid': '', 'hashes': '', 'raw_event': ev})
478         return results
479 __all__ = ['WevtAPI', 'ChannelMetadata', 'CHANNEL_DESCRIPTIONS
480         ']

```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Init
5 # Description:
6 #   Модуль низкоуровневых биндингов Win32 FFI / ctypes.
7 #
8 # Usage Examples:
9 #
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: apps.windows.telemetry.win32_fffi
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # Updated: 2026-10-01 13:28:28
17 # =====
18
19 """Модуль низкоуровневых биндингов Win32 FFI / ctypes."""
20
21 from .kernel32 import Kernel32API
22 from .psapi import PsapiAPI
23 from .advapi32 import Advapi32API
24 from .ntdll import NtdllAPI
25 from .etw import EtwAPI
26 from .wevtapi import WevtAPI, ChannelMetadata, CHANNEL_DESCRIPTIONS
27 from .scm import ServiceControlManager, ServiceInfo
28 from .tasksched import TaskSchedulerAPI
29 from .nethelper import IPHelperAPI, NetworkSocketInfo
30 from .setupapi import SetupAPI, PnPDeviceInfo
31 from .error_decoder import WindowsErrorDecoder, format_system_
```

```
error, decode_bugcheck_code
32
33 __all__ = [
34     'Kernel32API',
35     'PsapiAPI',
36     'Advapi32API',
37     'NtdllAPI',
38     'EtwAPI',
39     'WevtAPI',
40     'ChannelMetadata',
41     'CHANNEL_DESCRIPTIONS',
42     'ServiceControlManager',
43     'ServiceInfo',
44     'TaskSchedulerAPI',
45     'IPHelperAPI',
46     'NetworkSocketInfo',
47     'SetupAPI',
48     'PnPDeviceInfo',
49     'WindowsErrorDecoder',
50     'format_system_error',
51     'decode_bugcheck_code',
52 ]
53
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Init
4 # =====
5 # Description:
6 #   Exports core system metrics models, sensor probers, and te
7 #  lemetry collectors.
8 #
9 # Usage Examples:
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: apps.windows.telemetry
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # Updated: 2026-10-06 00:20:00
17 # =====
18
19 """Exports core system metrics models, sensor probers, and tel
20   emetry collectors."""
21
22 from .models import (
23     AnomalyItem,
24     BatteryMetrics,
25     CpuMetrics,
26     DiskIoMetrics,
27     DiskPartitionMetrics,
28     DriverInfo,
29     ForensicsActivityReport,
30     GpuMetrics,
31     HardwareArchiveEntry,
```



```
31 |     HardwareAuditReport,
32 |     HardwareChangeItem,
33 |     HardwareDeviceAudit,
34 |     HardwareNode,
35 |     HardwareSensor,
36 |     KernelThrottlingReport,
37 |     MemoryMetrics,
38 |     NetworkInterfaceMetrics,
39 |     NetworkPortMetrics,
40 |     PeripheralsNetworkReport,
41 |     PhysicalDiskHealth,
42 |     ProcessLeakDiagnosticsReport,
43 |     ProcessLeakItem,
44 |     ProcessLifecycleEvent,
45 |     ProcessMetrics,
46 |     ProcessNetworkActivity,
47 |     ProcessProvenanceInfo,
48 |     ProcessProvenanceReport,
49 |     ProcessTokenInfo,
50 |     RamStickInfo,
51 |     StorageBatteryWearReport,
52 |     SystemDiagnosticReport,
53 |     SystemHealthAlerts,
54 |     SystemSnapshot,
55 |     W64CollectorStatus,
56 |     W64SystemEvent,
57 |     ETWTraceEvent,
58 | )
59 | from .sensors import get_hardware_sensors
60 | from .sqlite import (
61 |     AggregationLevel,
62 |     TelemetryBuffer,
63 |     TelemetryConnectionManager,
64 |     TelemetryMaintenance,
65 |     TelemetryReader,
66 |     TelemetrySqlAggregator,
67 |     TelemetryStorage,
68 |     TelemetryWriter,
```

```
69     sensors_aggregate,
70 )
71 from .collector import SystemCollector
72 from .service import TelemetryLoggerService
73 from .telemetry_config import TelemetryConfigManager, get_default_telemetry_config_path
74 from .json_logger import TelemetryJsonLogger
75 from .file_collector import FileCollector
76 from .sensor_collector import SensorCollector
77 from .device_flapping_sensor import DeviceFlappingSensor, DeviceTransitionEvent
78 from apps.windows.modules.storage_manager.core.windows_storage_sensor import (
79     StorageDiskHealthInfo,
80     WindowsStorageSensor,
81     collect_storage_snapshot,
82     save_snapshot,
83 )
84 from .storage_usage import WindowsStorageUsageCollector
85 from .w64_collector import (
86     AIW64Collector,
87     get_w64_collector,
88     start_w64_collector,
89     stop_w64_collector,
90 )
91 from .w64_etw_collector import AIW64ETWCollector
92 from apps.windows.telemetry_research.aggregator import TelemetryAggregator
93 from apps.windows.telemetry_research.grouped_telemetry import GroupedTelemetryBuilder
94 from apps.windows.telemetry_research.incident_detector import IncidentDetector
95 from apps.windows.telemetry_research.reboot_analyzer import WindowsRebootAnalyzer
96 from apps.windows.telemetry_research.hardware_auditor import HardwareAuditor
97 from apps.windows.telemetry_research.deep_diagnostics import DeepDiagnosticsEngine
```

```
98 | from apps.windows.telemetry_research.hardware_history_manager
    | import HardwareHistoryManager
99 |
100 |
101 |
102 | __all__ = [
103 |     "AIW64Collector",
104 |     "AIW64ETWCollector",
105 |     "get_w64_collector",
106 |     "start_w64_collector",
107 |     "stop_w64_collector",
108 |     "CpuMetrics",
109 |     "MemoryMetrics",
110 |     "RamStickInfo",
111 |     "GpuMetrics",
112 |     "DiskPartitionMetrics",
113 |     "PhysicalDiskHealth",
114 |     "DiskIoMetrics",
115 |     "NetworkInterfaceMetrics",
116 |     "NetworkPortMetrics",
117 |     "BatteryMetrics",
118 |     "SystemHealthAlerts",
119 |     "ProcessMetrics",
120 |     "ProcessProvenanceInfo",
121 |     "ProcessLifecycleEvent",
122 |     "ProcessProvenanceReport",
123 |     "ProcessTokenInfo",
124 |     "ProcessLeakItem",
125 |     "ProcessLeakDiagnosticsReport",
126 |     "ForensicsActivityReport",
127 |     "KernelThrottlingReport",
128 |     "StorageBatteryWearReport",
129 |     "PeripheralsNetworkReport",
130 |     "HardwareSensor",
131 |     "HardwareNode",
132 |     "SystemSnapshot",
133 |     "AnomalyItem",
134 |     "SystemDiagnosticReport",
```

```
135 | "DriverInfo",
136 | "HardwareDeviceAudit",
137 | "HardwareChangeItem",
138 | "HardwareAuditReport",
139 | "HardwareArchiveEntry",
140 | "get_hardware_sensors",
141 | "HardwareAuditor",
142 | "HardwareHistoryManager",
143 | "TelemetryStorage",
144 | "TelemetrySqlAggregator",
145 | "AggregationLevel",
146 | "sensors_aggregate",
147 | "SystemCollector",
148 | "TelemetryLoggerService",
149 | "TelemetryConfigManager",
150 | "TelemetryJsonLogger",
151 | "FileCollector",
152 | "SensorCollector",
153 | "TelemetryAggregator",
154 | "DeviceFlappingSensor",
155 | "DeviceTransitionEvent",
156 | "DeepDiagnosticsEngine",
157 | "IncidentDetector",
158 | "WindowsRebootAnalyzer",
159 | "GroupedTelemetryBuilder",
160 | "TelemetryCompactor",
161 | "AuditStartupChecker",
162 | "StorageDiskHealthInfo",
163 | "WindowsStorageSensor",
164 | "WindowsStorageUsageCollector",
165 | "collect_storage_snapshot",
166 | "save_snapshot",
167 | "W64CollectorStatus",
168 | "W64SystemEvent",
169 | "ETWTraceEvent",
170 | "init_telemetry_database",
171 | "get_default_telemetry_db_path",
172 | "get_default_telemetry_config_path",
```

```
173 | ]
174 |
175 |
176 | def __getattr__(name: str):
177 |     """Ленивый импорт диагностического движка для предотвращен
    |     ия циклических зависимостей."""
178 |     if name in ("DiagnosticEngine", "SystemDiagnosticEngine"):
179 |         from apps.windows.telemetry_research.diagnostic_engine
    |         import DiagnosticEngine, SystemDiagnosticEngine
180 |         if name == "DiagnosticEngine":
181 |             return DiagnosticEngine
182 |         return SystemDiagnosticEngine
183 |     raise AttributeError(f"module '{__name__}' has no attribut
    | e '{name}'")
```